Comprehensive Reference of Mathematical Functions in AI, ML, DL, and RL

A Systematic Collection of Formulas, Definitions, Usage Patterns, and Trade-offs

Compiled for

Nurdun Mijit

May 27, 2026

This reference covers cost functions, activation functions, regularization, normalization, evaluation metrics, softmax variants, probability distributions, distance metrics, optimization algorithms, attention mechanisms, pooling, convolutions, kernels, information theory, dimensionality reduction, initialization, learning rate schedules, sampling, gradient clipping, embeddings, positional encoding, RL functions, ensemble methods, decision trees, Bayesian methods, clustering, data augmentation, uncertainty quantification, calibration, multi-task learning, graph neural networks, and LLM-specific functions.

Contents

Preface **2**

1 Cost and Loss Functions **3**
1.1 Regression Losses . . . 3
1.1.1 Mean Squared Error (MSE / L2 Loss) . . . 3
1.1.2 Mean Absolute Error (MAE / L1 Loss) . . . 4
1.1.3 Huber Loss (Smooth L1) . . . 5
1.1.4 Log-Cosh Loss . . . 5
1.1.5 Quantile Loss (Pinball Loss) . . . 6
1.1.6 Poisson Loss . . . 7
1.1.7 Mean Squared Logarithmic Error (MSLE) . . . 7
1.1.8 Mean Absolute Percentage Error (MAPE) . . . 8
1.2 Classification Losses . . . 8
1.2.1 Binary Cross-Entropy (BCE / Log Loss) . . . 8
1.2.2 Categorical Cross-Entropy (Multi-Class Softmax Loss) . . . 9
1.2.3 Hinge Loss (SVM Loss) . . . 10
1.2.4 Focal Loss . . . 11
1.2.5 Dice Loss . . . 11
1.2.6 Tversky Loss . . . 12
1.2.7 ArcFace / CosFace / SphereFace (Angular Margin Losses) . . . 12
1.2.8 Label Smoothing Cross-Entropy . . . 13
1.3 Ranking Losses . . . 14
1.3.1 Triplet Loss . . . 14
1.3.2 Contrastive Loss (Siamese) . . . 15
1.3.3 Bayesian Personalized Ranking (BPR) . . . 15
1.4 Generative and Self-Supervised Losses . . . 16
1.4.1 VAE Evidence Lower Bound (ELBO) . . . 16
1.4.2 DDPM Loss (Denoising Diffusion Probabilistic Models) . . . 17
1.4.3 InfoNCE Loss (SimCLR / CPC) . . . 17
1.4.4 GAN Adversarial Losses . . . 18
1.4.5 CTC Loss (Connectionist Temporal Classification) . . . 19
1.5 Object Detection Losses . . . 19
1.5.1 IoU Loss Family . . . 19
1.5.2 Hungarian Matching Loss (DETR) . . . 21
1.6 Summary and Practical Guidance . . . 21

2 Activation Functions **23**
2.1 Sigmoid Family — Saturating Activations . . . 23
2.1.1 Logistic Sigmoid . . . 23
2.1.2 Hyperbolic Tangent (Tanh) . . . 24
2.1.3 Hard Sigmoid . . . 24
2.2 ReLU Family — Non-saturating Activations . . . 25
2.2.1 Rectified Linear Unit (ReLU) . . . 25
2.2.2 ReLU6 . . . 26
2.2.3 Leaky ReLU . . . 26
2.2.4 Parametric ReLU (PReLU) . . . 27
2.2.5 Exponential Linear Unit (ELU) . . . 27
2.2.6 Scaled Exponential Linear Unit (SELU) . . . 28

2.3 Smooth and Self-Gated Activations . . . 28
2.3.1 Softplus . . . 29
2.3.2 Gaussian Error Linear Unit (GELU) . . . 29
2.3.3 Swish / SiLU . . . 30
2.3.4 Mish . . . 30
2.3.5 Hard Swish (H-Swish) . . . 31
2.4 Specialized Activations . . . 31
2.4.1 Maxout . . . 32
2.4.2 Softmax . . . 32
2.4.3 Gumbel-Softmax . . . 33
2.4.4 Sinusoidal / SIREN Activation . . . 33
2.4.5 Bent Identity . . . 34
2.4.6 Gaussian Activation . . . 34
2.5 Summary Table . . . 35

3 Regularization Techniques 36
3.1 Parameter Norm Penalties . . . 36
3.1.1 L2 Regularization (Ridge / Weight Decay) . . . 36
3.1.2 L1 Regularization (Lasso) . . . 37
3.1.3 Elastic Net . . . 38
3.1.4 Group Lasso . . . 38
3.2 Dropout Methods . . . 39
3.2.1 Standard Dropout . . . 39
3.2.2 Spatial Dropout (Dropout2D) . . . 40
3.2.3 DropConnect . . . 40
3.2.4 Stochastic Depth (DropPath) . . . 41
3.2.5 Shake-Shake and ShakeDrop . . . 41
3.3 Data Augmentation as Regularization . . . 42
3.3.1 Mixup . . . 42
3.3.2 CutMix . . . 43
3.3.3 Cutout / Random Erasing . . . 43
3.3.4 Label Smoothing . . . 44
3.4 Implicit, Structural, and Advanced Regularization . . . 44
3.4.1 Early Stopping . . . 44
3.4.2 Batch Normalization as Regularizer . . . 45
3.4.3 Spectral Normalization . . . 45
3.4.4 Sharpness-Aware Minimization (SAM) . . . 46
3.4.5 Gradient Penalty (WGAN-GP) . . . 47
3.5 Knowledge Distillation . . . 47
3.6 Additional Regularization Techniques . . . 48
3.6.1 Virtual Adversarial Training (VAT) . . . 48
3.6.2 Entropy Regularization / Confidence Penalty . . . 49
3.6.3 Orthogonal Regularization . . . 49
3.6.4 Jacobian and Hessian Regularization . . . 50
3.7 Summary and Practical Guidance . . . 50

4 Normalization Methods 52
4.1 Batch Normalization (BN) . . . 52
4.1.1 Synchronized Batch Normalization . . . 53
4.2 Layer Normalization (LN) . . . 54
4.3 Instance Normalization (IN) . . . 55
4.4 Group Normalization (GN) . . . 56

4.5 LayerNorm Variants and Efficient Normalizations . . . 56
4.5.1 RMS Normalization (RMSNorm) . . . 57
4.5.2 DeepNorm . . . 57
4.6 Weight Normalization (WN) . . . 58
4.7 Adaptive Instance Normalization (AdaIN) . . . 59
4.8 Switchable Normalization (SN) . . . 60
4.9 Filter Response Normalization (FRN) . . . 60
4.10 Local Response Normalization (LRN) . . . 61
4.11 Advanced and Domain-Specific Normalizations . . . 61
4.11.1 PowerNorm . . . 61
4.11.2 EvoNorm . . . 62
4.11.3 Ghost Batch Normalization . . . 62
4.12 Graph Normalization (for GNNs) . . . 63
4.13 Comparison Table and Practical Guidance . . . 63

5 Evaluation Metrics 65
5.1 Regression Metrics . . . 65
5.1.1 Mean Squared Error (MSE) . . . 65
5.1.2 Root Mean Squared Error (RMSE) . . . 65
5.1.3 Mean Absolute Error (MAE) . . . 66
5.1.4 Coefficient of Determination (R^2) . . . 66
5.1.5 Adjusted R^2 . . . 66
5.1.6 Mean Absolute Percentage Error (MAPE) . . . 67
5.1.7 Symmetric Mean Absolute Percentage Error (SMAPE) . . . 67
5.1.8 Explained Variance Score . . . 67
5.1.9 Mean Bias Error (MBE) . . . 67
5.1.10 Concordance Correlation Coefficient (CCC) . . . 68
5.2 Binary Classification Metrics . . . 68
5.2.1 Accuracy . . . 68
5.2.2 Precision and Recall . . . 68
5.2.3 F-Score . . . 69
5.2.4 Matthews Correlation Coefficient (MCC) . . . 69
5.2.5 Balanced Accuracy . . . 70
5.2.6 Jaccard Index (Intersection over Union) . . . 70
5.2.7 Cohen's Kappa . . . 70
5.2.8 G-Mean . . . 70
5.2.9 Hamming Loss (Multi-Label) . . . 71
5.3 Threshold-Independent Metrics . . . 71
5.3.1 AUC-ROC . . . 71
5.3.2 AUC-PR (Precision-Recall AUC) . . . 71
5.3.3 Log-Loss (Cross-Entropy Loss) . . . 72
5.3.4 Brier Score . . . 72
5.4 Multi-Class Metrics . . . 72
5.4.1 Micro, Macro, and Weighted Averaging . . . 72
5.4.2 Top-K Accuracy . . . 72
5.4.3 Weighted Cohen's Kappa . . . 73
5.5 Ranking Metrics . . . 73
5.5.1 Precision@K, Recall@K, F1@K . . . 73
5.5.2 Mean Reciprocal Rank (MRR) . . . 73
5.5.3 Mean Average Precision (MAP) . . . 73
5.5.4 Discounted Cumulative Gain (DCG) and NDCG . . . 73
5.5.5 Correlation-Based Ranking Metrics . . . 74

5.5.6 Hit Rate . . . 74
5.6 Generative Model Metrics . . . 74
5.6.1 Inception Score (IS) . . . 74
5.6.2 Fréchet Inception Distance (FID) . . . 75
5.6.3 Kernel Inception Distance (KID) . . . 75
5.6.4 Peak Signal-to-Noise Ratio (PSNR) . . . 75
5.6.5 Structural Similarity Index (SSIM) . . . 75
5.6.6 Multi-Scale SSIM (MS-SSIM) . . . 76
5.7 NLP Generation Metrics . . . 76
5.7.1 BLEU (Bilingual Evaluation Understudy) . . . 76
5.7.2 ROUGE (Recall-Oriented Understudy for Gisting Evaluation) . . . 77
5.7.3 METEOR (Metric for Evaluation of Translation with Explicit ORdering) 77
5.7.4 CIDEr and SPICE . . . 77
5.7.5 BERTScore . . . 78
5.7.6 Perplexity (PPL) . . . 78
5.7.7 BARTScore and MoverScore . . . 78
5.8 Object Detection Metrics . . . 79
5.8.1 Mean Average Precision (mAP) . . . 79
5.8.2 AP by Object Size . . . 79
5.8.3 Average Recall (AR) . . . 79
5.9 Segmentation Metrics . . . 79
5.9.1 Pixel Accuracy . . . 79
5.9.2 Mean Intersection over Union (mIoU) . . . 80
5.9.3 Dice Coefficient (F1 for Segmentation) . . . 80
5.9.4 Panoptic Quality (PQ) . . . 80
5.10 Clustering Metrics . . . 80
5.10.1 Silhouette Score . . . 80
5.10.2 Davies-Bouldin Index . . . 80
5.10.3 Calinski-Harabasz Index . . . 81
5.10.4 Adjusted Rand Index (ARI) . . . 81
5.10.5 Normalized Mutual Information (NMI) . . . 81
5.10.6 Homogeneity, Completeness, and V-Measure . . . 81
5.11 Fairness and Calibration Metrics . . . 81
5.11.1 Demographic Parity . . . 82
5.11.2 Equalized Odds and Equal Opportunity . . . 82
5.11.3 Expected Calibration Error (ECE) . . . 82
5.12 Summary Comparison Table . . . 82

6 Softmax Variants and Temperature 84
6.1 Classical Softmax . . . 84
6.2 Temperature-Scaled Softmax . . . 85
6.2.1 Temperature in LLM Sampling . . . 86
6.2.2 Top-K and Top-P (Nucleus) Sampling . . . 86
6.2.3 Typical Sampling . . . 86
6.2.4 Mirostat . . . 87
6.2.5 Contrastive Decoding . . . 87
6.2.6 Beam Search . . . 87
6.2.7 Knowledge Distillation with Temperature . . . 87
6.2.8 Temperature Scaling for Calibration . . . 88
6.3 Log-Softmax . . . 88
6.4 Sparsemax (Martins & Astudillo, 2016) . . . 89
6.5 Entmax (α-entmax) . . . 89

6.6 Gumbel-Softmax (Concrete Distribution) . . . 90
6.6.1 Straight-Through Gumbel-Softmax . . . 91
6.7 Hierarchical Softmax . . . 91
6.8 Sampled Softmax and Noise Contrastive Estimation (NCE) . . . 92
6.9 Margin-Based Softmax (Face Recognition Family) . . . 92
6.9.1 ArcFace (Additive Angular Margin) . . . 93
6.9.2 CosFace / AM-Softmax (Additive Margin Softmax) . . . 93
6.9.3 SphereFace (Multiplicative Angular Margin) . . . 93
6.9.4 AdaFace, MagFace, CurricularFace, ElasticFace . . . 93
6.10 Comparison Table . . . 94

7 Probability Distributions in Machine Learning 96
7.1 Discrete Distributions . . . 96
7.1.1 Bernoulli Distribution . . . 96
7.1.2 Categorical / Multinoulli Distribution . . . 96
7.1.3 Binomial Distribution . . . 97
7.1.4 Multinomial Distribution . . . 97
7.1.5 Poisson Distribution . . . 97
7.1.6 Geometric Distribution . . . 98
7.1.7 Negative Binomial Distribution . . . 98
7.2 Continuous Distributions on $\mathbb{R}$ . . . 98
7.2.1 Gaussian (Normal) Distribution . . . 98
7.2.2 Laplace (Double Exponential) Distribution . . . 99
7.2.3 Cauchy Distribution . . . 100
7.2.4 Student's t-Distribution . . . 100
7.2.5 Logistic Distribution . . . 100
7.2.6 Gumbel Distribution . . . 101
7.3 Positive-Support Distributions ($\mathbb{R}^+$) . . . 101
7.3.1 Exponential Distribution . . . 101
7.3.2 Gamma Distribution . . . 101
7.3.3 Inverse Gamma Distribution . . . 102
7.3.4 Chi-Squared Distribution (χ^2) . . . 102
7.3.5 Weibull Distribution . . . 102
7.3.6 Log-Normal Distribution . . . 102
7.3.7 Pareto Distribution . . . 103
7.4 Bounded and Simplex Distributions . . . 103
7.4.1 Beta Distribution . . . 103
7.4.2 Dirichlet Distribution . . . 103
7.4.3 Kumaraswamy Distribution . . . 104
7.5 Spherical and Directional Distributions . . . 104
7.5.1 von Mises-Fisher (vMF) Distribution . . . 104
7.5.2 Kent, Bingham, and Watson Distributions . . . 105
7.6 Matrix Distributions . . . 105
7.6.1 Wishart Distribution . . . 105
7.6.2 LKJ Prior . . . 105
7.7 Mixture Distributions . . . 106
7.7.1 Gaussian Mixture Model (GMM) . . . 106
7.7.2 Dirichlet Process Mixture (Infinite Mixture) . . . 106
7.8 Reparameterization Tricks . . . 106
7.9 Exponential Family . . . 107
7.10 Summary Table . . . 108

8 Distance and Similarity Metrics **110**
8.1 Minkowski Family (ℓ_p Norms) . . . 110
8.1.1 Manhattan Distance ($p = 1$) . . . 110
8.1.2 Euclidean Distance ($p = 2$) . . . 110
8.1.3 Chebyshev Distance ($p \to \infty$) . . . 111
8.2 Inner Product / Angular Similarities . . . 111
8.2.1 Cosine Similarity . . . 111
8.2.2 Cosine Distance and Angular Distance . . . 111
8.2.3 Dot Product (for Normalized Vectors) . . . 112
8.2.4 Pearson Correlation Coefficient . . . 112
8.3 Distribution Distances . . . 112
8.3.1 Bhattacharyya Distance . . . 112
8.3.2 Hellinger Distance . . . 113
8.3.3 Total Variation Distance . . . 113
8.3.4 Wasserstein Distance (Earth Mover's Distance) . . . 113
8.3.5 Sinkhorn Distance (Entropy-Regularized OT) . . . 114
8.3.6 Maximum Mean Discrepancy (MMD) . . . 114
8.3.7 Energy Distance . . . 114
8.4 Information-Theoretic Divergences . . . 115
8.4.1 Kullback-Leibler (KL) Divergence . . . 115
8.4.2 Jensen-Shannon (JS) Divergence . . . 115
8.4.3 Cross-Entropy . . . 115
8.5 Binary / Set Similarities . . . 116
8.5.1 Hamming Distance . . . 116
8.5.2 Jaccard Similarity (IoU for Sets) . . . 116
8.5.3 Dice / Sørensen-Dice Coefficient . . . 116
8.5.4 Overlap Coefficient . . . 116
8.5.5 Tversky Index . . . 117
8.6 String / Sequence Distances . . . 117
8.6.1 Levenshtein Distance (Edit Distance) . . . 117
8.6.2 Damerau-Levenshtein Distance . . . 117
8.6.3 Jaro and Jaro-Winkler Similarity . . . 118
8.6.4 Smith-Waterman (Local Alignment) . . . 118
8.6.5 Needleman-Wunsch (Global Alignment) . . . 118
8.6.6 Longest Common Subsequence (LCS) . . . 119
8.7 Time Series Distances . . . 119
8.7.1 Dynamic Time Warping (DTW) . . . 119
8.7.2 Soft-DTW . . . 119
8.7.3 Fréchet Distance . . . 120
8.8 Mahalanobis Distance . . . 120
8.9 Image Structural Similarity (Extended) . . . 121
8.9.1 SSIM — Detailed Breakdown . . . 121
8.9.2 FSIM (Feature Similarity) . . . 121
8.9.3 HaarPSI . . . 121
8.10 Optimal Transport (Extended) . . . 122
8.10.1 Earth Mover's Distance (EMD) — Discrete Case . . . 122
8.10.2 Gromov-Wasserstein Distance . . . 122
8.11 Learned and Kernel-Based Distances . . . 122
8.11.1 Siamese Networks with Contrastive Loss . . . 122
8.11.2 Triplet Loss . . . 122
8.11.3 Kernel-Induced Distances . . . 123

8.12 Summary Comparison Table . . . 123

9 Optimization Algorithms 125
9.1 First-Order Non-Adaptive Methods . . . 125
9.1.1 Gradient Descent (GD) . . . 125
9.1.2 Stochastic Gradient Descent (SGD) . . . 126
9.1.3 SGD with Momentum (Polyak) . . . 126
9.1.4 SGD with Nesterov Accelerated Gradient (NAG) . . . 127
9.2 Adaptive First-Order Methods . . . 128
9.2.1 AdaGrad . . . 128
9.2.2 RMSProp . . . 129
9.2.3 Adam (Kingma & Ba, 2014) . . . 129
9.2.4 AdamW — Decoupled Weight Decay . . . 130
9.2.5 AMSGrad, NAdam, Adamax, RAdam, and AdaBelief . . . 131
9.3 Second-Order Methods . . . 132
9.3.1 Newton's Method . . . 132
9.3.2 BFGS and L-BFGS . . . 133
9.3.3 Kronecker-Factored Approximate Curvature (KFAC) . . . 134
9.3.4 Shampoo, AdaHessian, and Sophia . . . 134
9.4 Specialized Methods . . . 135
9.4.1 Projected Gradient Descent (PGD) . . . 135
9.4.2 Proximal Gradient Method . . . 135
9.4.3 FISTA (Nesterov Acceleration for Proximal Methods) . . . 136
9.4.4 Conjugate Gradient (CG) . . . 136
9.4.5 Stochastic Gradient Langevin Dynamics (SGLD) . . . 137
9.5 Distributed Optimization . . . 137
9.6 Zeroth-Order / Black-Box Optimization . . . 138
9.6.1 Evolution Strategies (ES) . . . 138
9.6.2 CMA-ES — Covariance Matrix Adaptation Evolution Strategy . . . 139
9.6.3 Bayesian Optimization Acquisition Functions . . . 139
9.7 Specialized Training Techniques . . . 140
9.7.1 Stochastic Weight Averaging (SWA) . . . 140
9.7.2 DP-SGD — Differentially Private SGD . . . 140
9.8 Comprehensive Optimizer Comparison . . . 141

10 Attention Mechanisms 142
10.1 Scaled Dot-Product Attention — Core Formulation . . . 142
10.1.1 Multi-Head Attention (MHA) . . . 142
10.2 Efficient / Linear Attention . . . 143
10.2.1 Linformer . . . 143
10.2.2 Performer — FAVOR+ . . . 144
10.2.3 Linear Transformer . . . 144
10.2.4 Synthesizer, Nyströmformer, and Other Efficient Variants . . . 145
10.3 Sparse Attention Patterns . . . 145
10.3.1 Sparse Transformer (Child et al., 2019) . . . 145
10.3.2 Longformer (Beltagy et al., 2020) . . . 146
10.3.3 BigBird (Zaheer et al., 2020) . . . 146
10.3.4 Reformer (Kitaev et al., 2020) . . . 147
10.3.5 Routing Transformer (Roy et al., 2020) . . . 147
10.4 Position-Biased Attention . . . 147
10.4.1 Relative Position Bias — T5 . . . 147
10.4.2 ALiBi — Attention with Linear Biases . . . 148

10.4.3 RoPE — Rotary Position Embedding . . . 148
10.5 FlashAttention . . . 149
10.6 Cross-Attention and Multi-Modal Attention . . . 150
10.7 Graph Attention Networks (GAT) . . . 150
10.8 Other Important Attention Variants . . . 151
10.9 State Space Models — Modern Sequence Models . . . 152
10.9.1 Mamba — Selective State Space Models . . . 152
10.10 Comprehensive Attention Comparison . . . 153

11 Pooling Functions **154**
11.1 Global Pooling . . . 154
11.1.1 Global Average Pooling (GAP) . . . 154
11.1.2 Global Max Pooling (GMP) . . . 155
11.1.3 Generalized Mean (GeM) Pooling . . . 155
11.2 Local Pooling (Sliding Window) . . . 156
11.2.1 Max Pooling . . . 156
11.2.2 Average Pooling . . . 157
11.2.3 L_p Pooling and Stochastic Pooling . . . 157
11.2.4 Fractional Max Pooling . . . 158
11.3 Hierarchical / Region-Based Pooling . . . 158
11.3.1 Spatial Pyramid Pooling (SPP) . . . 158
11.3.2 RoI Pooling . . . 159
11.3.3 RoI Align . . . 159
11.3.4 PrRoI Pooling and ASPP . . . 160
11.4 Attention-Based Pooling . . . 160
11.4.1 Attention Pooling . . . 160
11.4.2 Pooling by Multihead Attention (PMA) — Set Transformer . . . 161
11.5 Graph Pooling . . . 161
11.5.1 DiffPool — Differentiable Pooling . . . 161
11.5.2 SAGPool — Self-Attention Graph Pooling . . . 162
11.5.3 Top-K Pooling and Edge Pooling . . . 163
11.5.4 Wavelet Pooling and LIPooling . . . 163
11.6 Comprehensive Pooling Comparison . . . 164

12 Convolution Operations **165**
12.1 Standard Convolution . . . 165
12.1.1 2D Convolution . . . 165
12.1.2 1D Convolution . . . 166
12.1.3 3D Convolution . . . 166
12.1.4 Transposed Convolution (Deconvolution) . . . 167
12.2 Depthwise Separable Convolution . . . 167
12.3 Dilated (Atrous) Convolution . . . 168
12.4 Deformable Convolution . . . 169
12.4.1 Deformable Convolution v1 . . . 169
12.4.2 Deformable Convolution v2 . . . 170
12.5 Group Convolution . . . 170
12.6 ShuffleNet — Channel Shuffle . . . 171
12.7 Ghost Module (GhostNet) . . . 172
12.8 Conditionally Parameterized Convolution (CondConv) . . . 172
12.9 Involution . . . 173
12.10 Graph Convolution . . . 173
12.10.1 GCN — Graph Convolutional Network . . . 173

12.10.2 ChebNet, SGC, and GIN . . . 174
12.11 Spherical / 3D / Point-Cloud Convolutions . . . 175
12.11.1 Point-Cloud Convolutions . . . 175
12.11.2 Voxel-Based 3D Convolution . . . 176
12.12 FFT and Winograd Convolution . . . 176
12.13 Group Equivariant Convolution . . . 177
12.14 Comprehensive Convolution Comparison . . . 178

13 Kernel Functions **179**
13.1 Introduction to Kernels . . . 179
13.2 SVM Kernels . . . 179
13.2.1 Linear Kernel . . . 180
13.2.2 Polynomial Kernel . . . 180
13.2.3 RBF / Gaussian Kernel . . . 180
13.2.4 Laplacian Kernel . . . 181
13.2.5 Sigmoid Kernel . . . 181
13.2.6 Chi-Squared Kernel . . . 181
13.2.7 Histogram Intersection Kernel . . . 182
13.3 Gaussian Process Covariance Functions . . . 182
13.3.1 Squared Exponential (RBF) . . . 182
13.3.2 Matérn Kernel . . . 183
13.3.3 Rational Quadratic (RQ) . . . 183
13.3.4 Periodic Kernel . . . 184
13.3.5 Linear / Dot Product Kernel . . . 184
13.3.6 White Noise Kernel . . . 184
13.3.7 Neural Network Kernel (ArcSin) . . . 184
13.3.8 Spectral Mixture Kernel . . . 185
13.4 Neural Tangent Kernel (NTK) . . . 185
13.4.1 Definition . . . 185
13.4.2 Properties . . . 185
13.4.3 NTK Parameterization (μP) . . . 186
13.5 Sparse GP Approximations . . . 186
13.5.1 Inducing Points Methods . . . 186
13.5.2 Random Fourier Features (Rahimi & Recht, 2007) . . . 187
13.5.3 Nyström Approximation . . . 187
13.6 Graph Kernels . . . 187
13.7 Deep Kernel Learning . . . 188
13.8 Summary Comparison of Kernel Functions . . . 188

14 Information Theory Metrics **190**
14.1 Core Quantities . . . 190
14.1.1 Shannon Entropy (Discrete) . . . 190
14.1.2 Differential Entropy (Continuous) . . . 190
14.1.3 Joint Entropy . . . 191
14.1.4 Conditional Entropy . . . 191
14.2 Divergence Measures . . . 191
14.2.1 Kullback-Leibler (KL) Divergence . . . 191
14.2.2 Jensen-Shannon (JS) Divergence . . . 193
14.2.3 Cross-Entropy . . . 193
14.2.4 f-Divergence Family . . . 193
14.2.5 Rényi Entropy and Divergence . . . 194
14.3 Mutual Information . . . 194

14.3.1 Definition . . . 194
14.3.2 Conditional Mutual Information . . . 195
14.3.3 Pointwise Mutual Information (PMI) . . . 195
14.3.4 Variational Mutual Information Estimation . . . 196
14.3.5 MI Applications in Machine Learning . . . 196
14.4 Information Bottleneck (IB) . . . 197
14.5 Rate-Distortion Theory . . . 197
14.6 Entropy in Machine Learning . . . 198
14.6.1 Entropy Regularization in RL . . . 198
14.6.2 Information Gain and Active Learning . . . 198
14.6.3 Minimum Description Length (MDL) . . . 198
14.7 Entropy in Decision Trees . . . 199
14.8 Summary Table of Information-Theoretic Quantities . . . 199

15 Dimensionality Reduction **201**
15.1 Principal Component Analysis (PCA) . . . 201
15.1.1 Standard PCA . . . 201
15.1.2 Kernel PCA (kPCA) . . . 202
15.1.3 Robust PCA (rPCA) . . . 202
15.1.4 Probabilistic PCA (PPCA) . . . 202
15.1.5 Factor Analysis (FA) . . . 203
15.1.6 Independent Component Analysis (ICA) . . . 203
15.2 Manifold Learning . . . 204
15.2.1 Multidimensional Scaling (MDS) . . . 204
15.2.2 Isomap . . . 204
15.2.3 Locally Linear Embedding (LLE) . . . 204
15.2.4 Laplacian Eigenmaps . . . 205
15.2.5 Diffusion Maps . . . 206
15.2.6 Local Tangent Space Alignment (LTSA) . . . 206
15.3 t-Distributed Stochastic Neighbor Embedding (t-SNE) . . . 206
15.3.1 Algorithm . . . 206
15.3.2 Barnes-Hut t-SNE . . . 207
15.4 Uniform Manifold Approximation and Projection (UMAP) . . . 208
15.4.1 Algorithm . . . 208
15.5 Autoencoder-Based Dimensionality Reduction . . . 208
15.5.1 Standard Autoencoder . . . 208
15.5.2 Variational Autoencoder (VAE) . . . 209
15.5.3 Contractive Autoencoder (CAE) . . . 209
15.5.4 Denoising Autoencoder (DAE) . . . 209
15.6 Linear Discriminant Analysis (LDA) . . . 209
15.7 Non-Negative Matrix Factorization (NMF) . . . 210
15.8 Random Projections . . . 211
15.9 Tensor Decompositions . . . 211
15.10 Metric Learning . . . 212
15.11 Summary Comparison Table . . . 212

16 Initialization Methods **214**
16.1 Why Initialization Matters . . . 214
16.2 Classic Initialization Schemes . . . 214
16.2.1 Zero / Constant Initialization . . . 214
16.2.2 Random (Uniform / Normal) . . . 215
16.3 Famous Initialization Schemes . . . 215

16.3.1 Xavier / Glorot Initialization (2010) . . . 215
16.3.2 Kaiming / He Initialization (2015) . . . 216
16.3.3 LeCun Initialization (1998) . . . 217
16.3.4 Orthogonal Initialization . . . 217
16.4 Specialized Initialization Techniques . . . 218
16.4.1 LSUV (Layer-Sequential Unit-Variance) . . . 218
16.4.2 Normalization Layer Defaults . . . 218
16.4.3 LSTM Forget Gate Bias . . . 218
16.4.4 Transformer Initialization . . . 219
16.4.5 Fixup Initialization (2019) . . . 219
16.4.6 ReZero / SkipInit . . . 219
16.4.7 μP (Maximal Update Parameterization) . . . 220
16.5 Bias Initialization . . . 220
16.6 Embedding Initialization . . . 221
16.7 Summary Comparison Table . . . 221

17 Learning Rate Schedules **223**
17.1 Constant Learning Rate . . . 223
17.2 Step Decay . . . 223
17.3 Exponential Decay . . . 224
17.4 Time-Based Decay . . . 225
17.5 Polynomial Decay . . . 225
17.6 Inverse Square Root Decay . . . 226
17.7 Cosine Annealing . . . 227
17.8 Cosine Annealing with Warm Restarts (SGDR) . . . 227
17.9 Linear Warmup . . . 228
17.10 Warmup + Cosine Decay (LLM Standard) . . . 229
17.11 Cyclical Learning Rates . . . 230
17.11.1 Triangular Cyclical LR . . . 230
17.11.2 Triangular2 . . . 230
17.11.3 Exponential Range CLR . . . 230
17.12 1Cycle Policy (Super-Convergence) . . . 231
17.13 Reduce Learning Rate on Plateau . . . 232
17.14 Noam Schedule (Transformer) . . . 232
17.15 GPT and Chinchilla Schedules . . . 233
17.15.1 GPT Schedule . . . 233
17.15.2 Chinchilla Schedule . . . 234
17.16 Hypergradient Descent (Learning to Learn the LR) . . . 234
17.17 SWA Learning Rate Scheduling . . . 235
17.18 Summary Comparison Table . . . 236

18 Sampling Strategies **238**
18.1 Basic Sampling . . . 238
18.1.1 Uniform Random Sampling . . . 238
18.1.2 Stratified Sampling . . . 239
18.1.3 Reservoir Sampling . . . 239
18.2 Importance Sampling . . . 240
18.2.1 Standard Importance Sampling . . . 240
18.2.2 Weight-Normalized (Self-Normalized) Importance Sampling . . . 240
18.2.3 Prioritized Experience Replay (PER) . . . 241
18.2.4 Annealed Importance Sampling (AIS) . . . 241
18.3 Hard Example Mining . . . 242

18.3.1 Hard and Semi-Hard Negative Mining . . . 242
18.3.2 Batch-Hard Triplet Mining . . . 243
18.3.3 Online Hard Example Mining (OHEM) . . . 243
18.3.4 Focal Loss as Soft Mining . . . 244
18.4 MCMC Methods . . . 244
18.4.1 Metropolis–Hastings . . . 244
18.4.2 Gibbs Sampling . . . 245
18.4.3 Hamiltonian Monte Carlo (HMC) . . . 245
18.4.4 NUTS (No-U-Turn Sampler) . . . 246
18.4.5 Stochastic Gradient Langevin Dynamics (SGLD) . . . 246
18.5 Variational Samplers . . . 247
18.5.1 Reparameterization Trick . . . 247
18.5.2 REINFORCE / Score Function Estimator . . . 248
18.5.3 Straight-Through Estimator (STE) and Control Variates . . . 248
18.5.4 REBAR and RELAX . . . 249
18.6 Generative Model Sampling . . . 249
18.6.1 Ancestral / Autoregressive Sampling . . . 249
18.6.2 Langevin Dynamics . . . 250
18.6.3 Annealed Langevin Dynamics (NCSN) . . . 250
18.6.4 DDPM Sampling (Denoising Diffusion) . . . 251
18.6.5 DDIM (Deterministic Sampling) . . . 251
18.6.6 DPM-Solver (Diffusion ODE Solver) . . . 252
18.6.7 Classifier-Free Guidance (CFG) . . . 252
18.7 LLM Decoding Strategies . . . 253
18.7.1 Temperature Sampling . . . 253
18.7.2 Top-K and Top-P (Nucleus) Sampling . . . 254
18.7.3 Beam Search . . . 255
18.7.4 Diverse Beam, Contrastive, and Speculative Decoding . . . 255
18.7.5 Typical Sampling and Mirostat . . . 256
18.7.6 Classifier-Free Guidance for LLMs . . . 257
18.8 Curriculum and Active Learning . . . 257
18.8.1 Curriculum Learning . . . 257
18.8.2 Uncertainty Sampling, BALD, and Query by Committee . . . 258
18.9 Data Imbalance Handling . . . 259
18.9.1 SMOTE (Synthetic Minority Over-sampling Technique) . . . 259
18.9.2 SMOTE Variants and Under-Sampling Methods . . . 259
18.10Summary Comparison Table . . . 260

19 Gradient Clipping **262**
19.1 Why Gradient Clipping Is Necessary . . . 262
19.2 Norm-Based Global Clipping . . . 262
19.2.1 L2 Norm Clipping . . . 262
19.2.2 L∞ (Max Norm) Clipping . . . 263
19.2.3 L1 Norm Clipping . . . 264
19.3 Advanced Clipping Techniques . . . 264
19.3.1 Per-Sample Gradient Clipping (DP-SGD) . . . 264
19.3.2 Element-wise Value Clipping . . . 265
19.3.3 Adaptive Gradient Clipping (AGC) . . . 265
19.4 Related Stabilization Techniques . . . 266
19.4.1 Gradient Penalty (WGAN-GP) . . . 266
19.4.2 Automatic Mixed Precision (AMP) Loss Scaling . . . 267
19.5 Theoretical Motivation . . . 267

19.6 Practical Recommendations . . . 268
19.7 Summary Comparison Table . . . 268

20 Embedding Functions **270**
20.1 Foundational Embedding Methods . . . 270
20.1.1 One-Hot Encoding . . . 270
20.1.2 Word2Vec . . . 270
20.1.3 GloVe (Global Vectors) . . . 271
20.1.4 FastText . . . 272
20.2 Contextual Embeddings . . . 272
20.2.1 ELMo (Embeddings from Language Models) . . . 272
20.2.2 BERT and LLM Contextual Embeddings . . . 273
20.2.3 Sentence Embeddings . . . 274
20.3 Knowledge Graph Embeddings . . . 274
20.3.1 Translational Models: TransE, TransH, TransR . . . 275
20.3.2 Bilinear and Rotation Models: DistMult, ComplEx, RotatE . . . 275
20.4 Graph Node Embeddings . . . 276
20.4.1 Node2Vec and DeepWalk . . . 276
20.4.2 GraphSAGE . . . 277
20.5 Collaborative Filtering Embeddings . . . 277
20.5.1 Matrix Factorization (SVD, ALS) . . . 277
20.5.2 Factorization Machines (FM) . . . 278
20.6 Vector Quantization for Embeddings . . . 279
20.6.1 VQ-VAE Codebook Learning . . . 279
20.6.2 Product Quantization (PQ) and Residual Quantization (RQ) . . . 279
20.7 Locality-Sensitive Hashing (LSH) for Embeddings . . . 280
20.8 Cross-Modal Contrastive Embeddings . . . 280
20.8.1 CLIP and SigLIP . . . 281
20.8.2 ALIGN and ImageBind . . . 281
20.9 Mixture of Experts Embedding Gating . . . 282
20.10 Summary Comparison Table . . . 283

21 Positional Encoding **285**
21.1 Why Positional Encoding . . . 285
21.2 Absolute Sinusoidal Positional Encoding (Vaswani et al.) . . . 285
21.3 Learned Absolute Position Embeddings . . . 286
21.4 Relative Position Bias (Shaw et al., 2018) . . . 287
21.5 T5 Relative Position Bias . . . 287
21.6 Rotary Position Embedding (RoPE) . . . 288
21.7 RoPE Extensions for Longer Contexts . . . 289
21.7.1 NTK-Aware Scaling . . . 289
21.7.2 YaRN (Yet another RoPE extensioN) . . . 290
21.7.3 Linear Interpolation . . . 290
21.8 ALiBi (Attention with Linear Biases) . . . 290
21.9 xPos . . . 291
21.10 2D and 3D Positional Encodings . . . 291
21.10.1 Sinusoidal 2D . . . 291
21.10.2 Learned 2D Patch Embeddings (ViT) . . . 292
21.10.3 iRPE (Image Relative Position Encoding) . . . 292
21.11 Fourier Features (NeRF) . . . 293
21.12 Multi-resolution Hash Encoding (Instant NGP) . . . 293
21.13 NoPE — Positional Encoding-Free Architectures . . . 294

21.14 Summary Comparison Table . . . 294

22 Reinforcement Learning Functions **296**
22.1 Core Value Functions . . . 296
22.1.1 State-Value Function $V^\pi(s)$ . . . 296
22.1.2 Action-Value Function $Q^\pi(s,a)$ . . . 296
22.1.3 Advantage Function $A^\pi(s,a)$ . . . 297
22.2 Bellman Equations . . . 297
22.2.1 Bellman Expectation Equations . . . 297
22.2.2 Bellman Optimality Equations . . . 297
22.2.3 Bellman Operator as Contraction . . . 298
22.3 Value-Based Methods . . . 298
22.3.1 TD(0) — Temporal Difference Learning . . . 298
22.3.2 SARSA — On-Policy TD Control . . . 299
22.3.3 Q-Learning — Off-Policy TD Control . . . 299
22.3.4 Expected SARSA . . . 300
22.3.5 DQN — Deep Q-Network . . . 300
22.3.6 Double DQN . . . 300
22.3.7 Dueling DQN . . . 301
22.3.8 Distributional RL . . . 301
22.3.9 Rainbow . . . 302
22.3.10 n-Step TD and TD(λ) . . . 302
22.3.11 Retrace(λ) . . . 303
22.4 Policy Gradient Methods . . . 303
22.4.1 Policy Gradient Theorem . . . 304
22.4.2 REINFORCE . . . 304
22.4.3 REINFORCE with Baseline . . . 304
22.4.4 Actor-Critic Methods . . . 305
22.4.5 A2C / A3C . . . 305
22.4.6 GAE — Generalized Advantage Estimation . . . 306
22.5 Trust Region and Proximal Methods . . . 306
22.5.1 TRPO — Trust Region Policy Optimization . . . 306
22.5.2 Natural Policy Gradient . . . 307
22.5.3 PPO — Proximal Policy Optimization . . . 307
22.6 Continuous Action Methods . . . 308
22.6.1 DPG — Deterministic Policy Gradient . . . 309
22.6.2 DDPG — Deep Deterministic Policy Gradient . . . 309
22.6.3 TD3 — Twin Delayed DDPG . . . 309
22.7 Maximum Entropy Reinforcement Learning . . . 310
22.7.1 Soft Bellman and Soft Q-Function . . . 310
22.7.2 SAC — Soft Actor-Critic . . . 310
22.7.3 Automatic Temperature Tuning . . . 311
22.8 Model-Based Reinforcement Learning . . . 312
22.8.1 Dyna Architecture . . . 312
22.8.2 MCTS with UCT . . . 312
22.8.3 AlphaZero — PUCT . . . 313
22.8.4 Dreamer — RSSM . . . 313
22.9 Inverse and Imitation Reinforcement Learning . . . 314
22.9.1 Behavioral Cloning (BC) . . . 314
22.9.2 GAIL — Generative Adversarial Imitation Learning . . . 314
22.10 Exploration Methods . . . 315
22.10.1 Simple Exploration . . . 315

22.10.2 UCB1 . . . 315
22.10.3 Thompson Sampling . . . 316
22.10.4 Intrinsic Motivation: ICM and RND . . . 316
22.11 Offline Reinforcement Learning . . . 316
22.11.1 CQL — Conservative Q-Learning . . . 317
22.11.2 IQL — Implicit Q-Learning . . . 317
22.11.3 Decision Transformer . . . 318
22.12 Multi-Agent Reinforcement Learning . . . 318
22.12.1 Centralized and Decentralized Methods . . . 318
22.13 Summary Comparison of RL Algorithms . . . 319

23 Ensemble Methods **321**
23.1 Bagging (Bootstrap Aggregating) . . . 321
23.1.1 Bagging — Core Concept . . . 321
23.1.2 Random Forest . . . 322
23.1.3 Extra-Trees (Extremely Randomized Trees) . . . 323
23.1.4 Pasting . . . 323
23.2 Boosting . . . 323
23.2.1 AdaBoost . . . 324
23.2.2 Gradient Boosting . . . 325
23.2.3 XGBoost . . . 326
23.2.4 LightGBM . . . 326
23.2.5 CatBoost . . . 327
23.2.6 LogitBoost . . . 328
23.3 Stacking (Stacked Generalization) . . . 328
23.4 Bayesian Model Averaging (BMA) . . . 329
23.5 Deep Ensembles . . . 330
23.5.1 Naive Deep Ensemble . . . 330
23.5.2 MC Dropout . . . 331
23.5.3 BatchEnsemble . . . 331
23.5.4 Snapshot Ensemble . . . 332
23.5.5 FGE (Fast Geometric Ensembling) . . . 332
23.6 Mixture of Experts (MoE) . . . 333
23.6.1 Sparse Mixture of Experts . . . 333
23.6.2 Switch Transformer . . . 334
23.6.3 Soft MoE . . . 334
23.6.4 Other MoE Variants . . . 335
23.7 Negative Correlation Learning (NCL) . . . 335
23.8 Cascade Correlation . . . 336
23.9 Summary Comparison of Ensemble Methods . . . 337

24 Decision Tree Splitting Criteria **339**
24.1 Classification Splitting Criteria . . . 339
24.1.1 Gini Impurity . . . 339
24.1.2 Entropy and Information Gain . . . 340
24.1.3 Gain Ratio . . . 341
24.1.4 Misclassification Error . . . 342
24.1.5 Twoing Rule . . . 342
24.1.6 Chi-Square Splitting . . . 343
24.1.7 Hellinger Distance . . . 344
24.2 Regression Splitting Criteria . . . 344
24.2.1 Variance / MSE Reduction . . . 345

24.2.2 MAE / LAD Reduction . . . 345
24.2.3 Poisson Splitting . . . 346
24.2.4 Friedman's MSE . . . 347
24.3 Pruning Criteria . . . 347
24.3.1 Cost-Complexity Pruning . . . 347
24.3.2 Reduced Error Pruning . . . 348
24.3.3 Minimum Description Length Pruning . . . 349
24.3.4 Pessimistic Pruning . . . 349
24.4 Ensemble-Specific Split Finding . . . 350
24.4.1 XGBoost Gain . . . 350
24.4.2 LightGBM Leaf-Wise Growth . . . 351
24.4.3 Histogram-Based Split Finding . . . 351
24.5 Oblique / Multivariate Splits . . . 352
24.5.1 Axis-Parallel vs. Oblique Splits . . . 352
24.5.2 OC1 and Householder CART . . . 352
24.6 Multi-Way vs. Binary Splits . . . 353
24.7 Summary Comparison of Splitting Criteria . . . 354

25 Bayesian Methods 356
25.1 Core Bayesian Formulas . . . 356
25.1.1 Bayes' Theorem . . . 356
25.1.2 MAP Estimation . . . 357
25.1.3 Marginal Likelihood and Bayes Factors . . . 357
25.1.4 Posterior Predictive Distribution . . . 358
25.1.5 Conjugate Priors . . . 358
25.2 Variational Inference . . . 359
25.2.1 Evidence Lower Bound (ELBO) . . . 359
25.2.2 Mean-Field VI and CAVI . . . 360
25.2.3 Black-Box, Stochastic, and Reparameterization VI . . . 360
25.3 EM Algorithm . . . 361
25.3.1 Standard EM . . . 361
25.3.2 EM for Specific Models . . . 362
25.3.3 Variants: Variational EM and MCEM . . . 363
25.4 Bayesian Neural Networks . . . 363
25.4.1 Bayes by Backprop . . . 363
25.4.2 MC Dropout . . . 364
25.4.3 Flipout . . . 365
25.4.4 Laplace Approximation . . . 365
25.4.5 SWAG . . . 366
25.5 Gaussian Processes . . . 366
25.5.1 GP Prior and Posterior . . . 367
25.5.2 GP Classification . . . 368
25.5.3 Sparse Gaussian Processes . . . 368
25.6 Bayesian Optimization . . . 368
25.6.1 Expected Improvement (EI) . . . 369
25.6.2 Probability of Improvement (PI) . . . 369
25.6.3 Upper Confidence Bound (GP-UCB) . . . 370
25.6.4 Thompson Sampling and Information-Theoretic Methods . . . 370
25.7 State Space Models . . . 371
25.7.1 Kalman Filter . . . 371
25.7.2 Particle Filter . . . 372
25.8 Summary Comparison of Bayesian Methods . . . 372

26 Clustering Algorithms Math **374**
26.1 Centroid-Based Clustering . . . 374
26.1.1 K-means . . . 374
26.1.2 K-means++ . . . 375
26.1.3 K-medoids (PAM) . . . 375
26.1.4 Fuzzy C-Means . . . 376
26.2 Hierarchical Clustering . . . 376
26.2.1 Agglomerative Linkage Criteria . . . 376
26.2.2 BIRCH . . . 377
26.3 Density-Based Clustering . . . 378
26.3.1 DBSCAN . . . 378
26.3.2 OPTICS . . . 378
26.3.3 HDBSCAN . . . 379
26.3.4 Mean-Shift . . . 379
26.4 Spectral Clustering . . . 380
26.4.1 Graph Laplacian and Normalized Cut . . . 380
26.5 GMM and LDA . . . 381
26.5.1 Gaussian Mixture Model (GMM) EM . . . 381
26.5.2 Dirichlet Process GMM . . . 382
26.5.3 Latent Dirichlet Allocation (LDA) . . . 382
26.6 Deep Clustering . . . 383
26.6.1 Deep Embedded Clustering (DEC) . . . 383
26.7 Summary Comparison of Clustering Algorithms . . . 384

27 Data Augmentation **385**
27.1 Image Geometric Augmentation . . . 385
27.1.1 Rotation, Crop, Flip, Affine, Perspective . . . 385
27.1.2 Elastic Deformation and Thin Plate Spline . . . 386
27.1.3 Random Erasing / Cutout . . . 386
27.2 Image Color Augmentation . . . 387
27.2.1 Color Jitter and Intensity Transforms . . . 387
27.2.2 PCA Color Augmentation . . . 387
27.3 Policy-Based Augmentation . . . 388
27.3.1 AutoAugment . . . 388
27.3.2 RandAugment . . . 389
27.3.3 TrivialAugment . . . 389
27.3.4 AugMix . . . 390
27.4 Mixing-Based Augmentation . . . 390
27.4.1 Mixup . . . 390
27.4.2 CutMix . . . 391
27.4.3 Manifold Mixup, FMix, and Other Mixing Variants . . . 392
27.5 Text Augmentation . . . 392
27.5.1 Lexical and Syntactic Augmentation . . . 393
27.5.2 Model-Based Text Augmentation . . . 393
27.6 Audio Augmentation . . . 394
27.6.1 Time and Frequency Domain Augmentation . . . 394
27.6.2 SpecAugment . . . 394
27.7 Generative Augmentation . . . 395
27.7.1 GAN-Based and VAE-Based Augmentation . . . 395
27.8 Summary Comparison of Augmentation Methods . . . 396

28 Uncertainty Quantification **398**
28.1 Types of Uncertainty . . . 398
28.1.1 Aleatoric vs. Epistemic Uncertainty . . . 398
28.1.2 Total Predictive Variance Decomposition . . . 398
28.2 Ensemble Methods for Uncertainty . . . 399
28.2.1 Deep Ensembles . . . 399
28.2.2 MC Dropout . . . 400
28.2.3 SWAG and Laplace for Uncertainty . . . 401
28.3 Bayesian Methods . . . 401
28.4 Single Network Methods . . . 401
28.4.1 Heteroscedastic Regression . . . 401
28.4.2 Deep Evidential Regression . . . 402
28.5 Conformal Prediction . . . 403
28.5.1 Split Conformal Prediction . . . 403
28.5.2 Conformalized Quantile Regression (CQR) . . . 404
28.6 Calibration Metrics . . . 404
28.7 Out-of-Distribution (OOD) Detection . . . 405
28.7.1 Maximum Softmax Probability (MSP) . . . 405
28.7.2 Energy-Based OOD Detection . . . 405
28.7.3 Mahalanobis Distance in Feature Space . . . 406
28.7.4 ODIN . . . 406
28.8 Summary Comparison of Uncertainty Methods . . . 407

29 Calibration Methods **409**
29.1 Why Calibration Matters . . . 409
29.2 Calibration Metrics . . . 409
29.2.1 Expected Calibration Error (ECE) . . . 410
29.2.2 Maximum Calibration Error (MCE) . . . 410
29.2.3 Brier Score . . . 411
29.2.4 Reliability Diagrams and Confidence Histograms . . . 411
29.2.5 Adaptive and Class-wise ECE . . . 411
29.3 Calibration Methods . . . 412
29.3.1 Platt Scaling . . . 412
29.3.2 Temperature Scaling . . . 413
29.3.3 Matrix Scaling and Vector Scaling . . . 413
29.3.4 Isotonic Regression . . . 414
29.3.5 Histogram Binning . . . 415
29.3.6 Bayesian Binning into Quantiles (BBQ) . . . 415
29.3.7 Spline Calibration . . . 416
29.4 Post-Hoc vs. During-Training Calibration . . . 416
29.5 Calibration in LLMs and NLG . . . 417
29.6 Multi-Class vs. Binary Calibration . . . 417
29.7 Summary Comparison Table . . . 418

30 Multi-Task and Meta-Learning **420**
30.1 Multi-Task Learning . . . 420
30.1.1 Hard Parameter Sharing . . . 420
30.1.2 Soft Parameter Sharing . . . 421
30.1.3 Uncertainty Weighting (Kendall et al., 2018) . . . 421
30.1.4 GradNorm (Chen et al., 2018) . . . 422
30.1.5 PCGrad — Projection Conflict Gradient . . . 422
30.1.6 MGDA and CAGrad — Pareto Optimality . . . 423

30.1.7 DWA — Dynamic Weight Average . . . 424
30.2 Meta-Learning — Learning to Learn . . . 424
30.2.1 MAML — Model-Agnostic Meta-Learning . . . 424
30.2.2 FOMAML and Reptile . . . 425
30.2.3 Prototypical Networks . . . 426
30.2.4 Matching Networks . . . 426
30.2.5 Relation Networks . . . 427
30.3 Gradient-Based Hyperparameter Optimization . . . 427
30.4 Transfer Learning Strategies . . . 428
30.5 Summary Comparison Table . . . 428

31 Graph Neural Network Operations 430
31.1 Message Passing Framework . . . 430
31.2 Spectral Methods — Graph Convolution via Fourier Transform . . . 430
31.2.1 GCN — Graph Convolutional Network . . . 430
31.2.2 ChebNet — Chebyshev Spectral CNN . . . 431
31.2.3 SGC — Simplifying Graph Convolution . . . 432
31.2.4 ARMA Filters . . . 432
31.3 Spatial Methods — Message Passing on Neighbors . . . 433
31.3.1 GraphSAGE . . . 433
31.3.2 GAT — Graph Attention Networks . . . 433
31.3.3 GATv2 — Dynamic Attention . . . 435
31.3.4 GIN — Graph Isomorphism Network . . . 435
31.4 Graph Pooling — From Nodes to Graph-Level Representations . . . 436
31.5 Graph Normalization . . . 436
31.5.1 GraphNorm . . . 436
31.5.2 PairNorm . . . 436
31.5.3 Other Graph Normalizations . . . 437
31.6 Positional Encoding for Graphs . . . 437
31.7 Scalable GNNs . . . 438
31.8 Heterogeneous GNNs — Relational Graphs . . . 438
31.9 Summary Comparison Table . . . 439

32 LLM-Specific Functions 441
32.1 Tokenization Math . . . 441
32.2 Attention Variants in LLMs . . . 442
32.2.1 Multi-Query Attention (MQA) . . . 442
32.2.2 Grouped Query Attention (GQA) . . . 442
32.2.3 FlashAttention . . . 443
32.2.4 PagedAttention (vLLM) . . . 443
32.2.5 Sliding Window Attention . . . 443
32.2.6 KV Caching . . . 444
32.3 Activation Functions in LLMs . . . 444
32.4 Normalization in LLMs . . . 445
32.5 Loss Functions for LLM Training . . . 446
32.5.1 Next-Token Prediction (Causal LM) . . . 446
32.5.2 Masked Language Modeling (MLM — BERT) . . . 446
32.5.3 Span Corruption (T5) . . . 446
32.5.4 Prefix Language Model . . . 447
32.5.5 RLHF and DPO — Alignment Losses . . . 447
32.6 Efficient Training and Fine-Tuning . . . 448
32.6.1 LoRA — Low-Rank Adaptation . . . 448

32.6.2 QLoRA and Other Parameter-Efficient Methods 449
32.7 LLM Scaling Laws 449
32.8 LLM Inference Optimization 450
32.9 Length Extrapolation — Extending Context Beyond Training 451
32.10 Safety and Alignment 451
32.11 Summary Comparison Table — LLM Architectures 452

Preface

This book is a comprehensive reference guide for the mathematical functions, formulas, and techniques used across Artificial Intelligence (AI), Machine Learning (ML), Deep Learning (DL), and Reinforcement Learning (RL). It is organized to serve both as a learning resource and as a quick reference for practitioners.

Each entry follows a consistent structure:

- **Mathematical Definition** — The formal formula with notation explained.
- **Usage** — Where and how the function is applied in practice.
- **Pros** — Advantages and strengths.
- **Cons** — Limitations and weaknesses.
- **Variants** — Related forms and extensions.

The content spans activation functions, loss/cost functions, regularization techniques, normalization methods, evaluation metrics, optimization algorithms, and much more. Every effort has been made to be comprehensive while maintaining clarity and usefulness.

Chapter 1

Cost and Loss Functions

A loss function $\mathcal{L}(y, \hat{y})$ quantifies the discrepancy between predicted $\hat{y}$ and true y. Optimization algorithms minimize this quantity (or a regularized variant) to train a model. The choice of loss function encodes assumptions about the data distribution and determines what the model cares about most.

A **loss function** $\mathcal{L} : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_{\geq 0}$ maps a pair (true label, prediction) to a non-negative scalar penalty. Minimizing the empirical risk $\frac{1}{n}\sum_{i=1}^{n} \mathcal{L}(y_i, f(x_i; \boldsymbol{\theta}))$ over parameters $\boldsymbol{\theta}$ is the central optimization problem in supervised learning.

1.1 Regression Losses

Regression losses compare continuous scalar predictions $\hat{y}$ to ground-truth targets y. They differ primarily in how they penalize the residual $r = y - \hat{y}$.

1.1.1 Mean Squared Error (MSE / L2 Loss)

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

The squared L_2 norm of the residual vector, averaged over samples. Minimizing MSE is equivalent to maximum-likelihood estimation under the assumption of i.i.d. Gaussian noise with constant variance: $y = f(x) + \epsilon,\ \ \epsilon \sim \mathcal{N}(0, \sigma^2)$.

Default regression loss. Used in ordinary least-squares (OLS), linear regression, and as the reconstruction term in variational autoencoders (VAEs) and DDPM diffusion models. Also appears in multi-task settings where a continuous output head is stacked on a shared backbone. When the target distribution is roughly symmetric and unimodal, MSE is a safe first choice.

Pros:

- Differentiable everywhere and strongly convex — a unique global minimum exists for linear models.
- Closed-form solution exists for OLS: $\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$.
- Quadratic penalty heavily punishes large residuals, which is desirable when outliers are genuine signal.
- Gradient magnitude grows linearly with residual size, providing strong corrective signal for large errors.

Cons:

- Extreme sensitivity to outliers: a single far-away point can dominate the gradient and shift the solution dramatically (breakdown point = 0).
- Assumes homoscedasticity (constant noise variance). If variance depends on x (heteroscedasticity), MSE is suboptimal.
- Error measured in squared units of y, which can be hard to interpret.
- Gradient vanishes as residual $\to 0$, though this is usually benign.

1.1.2 Mean Absolute Error (MAE / L1 Loss)

$$\mathcal{L}_{\text{MAE}} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

The L_1 norm of the residual vector. Minimizing MAE is equivalent to MLE under a Laplace (double-exponential) noise model. The optimal point estimator is the **median** of the conditional distribution $p(y|x)$, rather than the mean.

Robust regression. Preferred when the data contains outliers or when the conditional distribution is heavy-tailed (e.g., financial returns). Also used in median regression, quantile regression (as a special case at $\tau = 0.5$), and as a component in composite losses like the smooth L1. Common in signal processing where sparse residuals are expected.

Pros:

- Robust to outliers: the penalty grows only linearly with residual magnitude. A single outlier cannot hijack the gradient.
- Sparse gradient: at the optimum the subgradient can be zero for many samples, unlike MSE.
- Interpretable in original units (same units as y).

Cons:

- Not differentiable at $r = 0$; optimization requires subgradient methods or a smooth surrogate.
- Constant gradient magnitude everywhere (except at zero) means the model receives the same corrective force for a residual of 0.1 as for 100. This can cause oscillation near the optimum, especially with large learning rates.
- May converge to solutions that ignore fine-grained variations in data that are not outliers.

1.1.3 Huber Loss (Smooth L1)

$$\mathcal{L}_{\text{Huber}}(r) = \begin{cases} \frac{1}{2}r^2 & \text{if } |r| \le \delta \\ \delta|r| - \frac{1}{2}\delta^2 & \text{if } |r| > \delta \end{cases}$$

where $r = y - \hat{y}$ and $\delta > 0$ is the transition threshold. The loss is quadratic for small residuals and linear for large ones, giving a C^1-smooth (continuously differentiable) hybrid of MSE and MAE.

Object detection (bounding-box regression). The Smooth L1 loss is the default box regression loss in Faster R-CNN and Mask R-CNN. It is also popular in robust regression problems where outliers exist but near-optimum smoothness is still desired. In PyTorch, `torch.nn.SmoothL1Loss` implements the case $\delta = 1$.

Pros:

- C^1-smooth everywhere — both the function and its gradient are continuous, enabling standard gradient-based optimization without subgradient hacks.
- Robust to outliers via the linear tail beyond δ.
- Behaves like MSE (smooth, small gradient) near zero, avoiding MAE's oscillation problem.

Cons:

- introduces an extra hyperparameter δ that must be tuned. Common choices are $\delta = 1.0$ or data-driven (e.g., the 90th percentile of residuals).
- The transition point creates a piecewise definition, which makes analytical manipulations slightly messier.

1.1.4 Log-Cosh Loss

$$\mathcal{L} = \sum_{i=1}^{n} \log\big(\cosh(\hat{y}_i - y_i)\big)$$

For small residuals, $\log\cosh(r) \approx \frac{1}{2}r^2$ (MSE-like). For large residuals, $\log\cosh(r) \approx |r| - \log 2$ (MAE-like). The transition is smooth and automatic — no δ threshold.

Smooth robust regression. A convenient drop-in replacement when both differentiability and outlier robustness are needed but tuning δ is undesirable. Used occasionally in finance and physics applications.

Pros:

- Twice differentiable everywhere (C^∞), enabling second-order optimization (Newton, L-BFGS) natively.

- Smoothly interpolates between MSE and MAE behavior without a hyperparameter.
- Robust to outliers by capping gradient magnitude at ± 1 for large residuals.

Cons:

- Computationally more expensive: evaluating cosh and log per sample.
- Numerical instability for very large $|r|$ unless implemented carefully (use $\log\cosh(x) = x + \log(1 + e^{-2x}) - \log 2$ for large x).
- Less commonly used in practice; limited framework support.

1.1.5 Quantile Loss (Pinball Loss)

For a target quantile $\tau \in (0, 1)$:

$$\mathcal{L}_\tau(y, \hat{y}) = \begin{cases} \tau\,(y - \hat{y}) & \text{if } y \geq \hat{y} \\ (1 - \tau)\,(\hat{y} - y) & \text{if } y < \hat{y} \end{cases}$$

The loss is asymmetric: over-prediction and under-prediction are penalized differently depending on τ. Minimizing $\mathcal{L}_\tau$ yields an estimate of the τ-th conditional quantile of $p(y|x)$.

Quantile regression and prediction intervals.

- $\tau = 0.5$ recovers median regression (MAE).
- Fitting models at $\tau = 0.05$ and $\tau = 0.95$ gives a 90% prediction interval $[\hat{y}_{0.05}, \hat{y}_{0.95}]$.
- Used in time-series forecasting (e.g., Amazon DeepAR), risk management (Value-at-Risk), and any application requiring uncertainty quantification beyond a point estimate.

Pros:

- Estimates arbitrary quantiles without distributional assumptions.
- Asymmetric penalty naturally encodes different costs for over- vs. under-prediction.
- Provides principled prediction intervals for uncertainty estimation.

Cons:

- Non-differentiable at zero; requires subgradient or smoothing.
- One model per quantile (unless using a multi-quantile architecture).
- Quantile crossing may occur (lower quantile exceeding upper quantile) without explicit constraints.

1.1.6 Poisson Loss

$$\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n}\left(\hat{y}_i - y_i \log \hat{y}_i\right)$$

Derived as the negative log-likelihood of a Poisson distribution with rate parameter $\hat{y}_i$: $p(y_i|\hat{y}_i) = \hat{y}_i^{y_i} e^{-\hat{y}_i}/y_i!$. The target $y_i \in \mathbb{N}_0$ (non-negative integers).

Count data regression. Used for predicting counts — website visits, insurance claims, RNA-seq read counts, demand forecasting for inventory. Commonly implemented with a log-link: $\hat{y} = \exp(\mathbf{w}^\top \mathbf{x} + b)$ to keep the rate positive. XGBoost and LightGBM support Poisson objective natively.

Pros:

- Statistically principled for count data — respects the discrete, non-negative nature of the target.
- Models the variance-mean relationship (Var = mean).
- Handles zero-inflation when extended to Zero-Inflated Poisson (ZIP).

Cons:

- Applicable only to non-negative integer targets.
- Requires $\hat{y}_i > 0$ (enforced via log-link or softplus).
- Overdispersion (Var > mean) violates the Poisson assumption; use Negative Binomial loss instead.

1.1.7 Mean Squared Logarithmic Error (MSLE)

$$\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n}\left(\log(1+y_i) - \log(1+\hat{y}_i)\right)^2$$

Penalizes *relative* rather than absolute errors by operating in log-space. The +1 offset ensures the argument is strictly positive even when $y_i = 0$.

Targets spanning multiple orders of magnitude. House-price prediction, population estimates, or any regression where a \$1,000 error on a \$10,000 house is far worse than a \$1,000 error on a \$1,000,000 house.

Pros:

- Scale-invariant: penalizes relative error, naturally handling wide dynamic ranges.
- Penalizes under-prediction more heavily than over-prediction (due to log curvature), which is desirable for inventory/safety-stock forecasting.

Cons:

- The +1 offset is a heuristic; the metric changes if units are scaled.
- Undefined (or ill-behaved) for negative predictions $\hat{y}_i < -1$.
- Not a proper scoring rule for any simple noise distribution.

1.1.8 Mean Absolute Percentage Error (MAPE)

$$\mathcal{L} = \frac{100\%}{n} \sum_{i=1}^{n} \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Percentage error, averaged across samples. The loss is normalized by the true value, making it scale-independent.

Business and economic forecasting. MAPE is ubiquitous in supply-chain forecasting because "15% error" is immediately interpretable to stakeholders, regardless of the unit (dollars, units, kilograms).

Pros:

- Scale-independent and interpretable as a percentage.
- Commonly reported in industry — management-friendly metric.

Cons:

- Undefined when any $y_i = 0$; unstable when $y_i \approx 0$.
- Asymmetric: over-forecasting has bounded error ($< 100\%$ if $\hat{y} < 2y$), but under-forecasting is unbounded.
- Not optimal for any symmetric noise distribution; not a proper scoring rule.
- Can produce infinite or misleading values with intermittent (sparse) demand data.

1.2 Classification Losses

Classification losses compare predicted class probabilities (or scores) to discrete true labels. They range from probabilistic (cross-entropy family) to margin-based (hinge).

1.2.1 Binary Cross-Entropy (BCE / Log Loss)

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^{n} \Big[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \Big]$$

where $y_i \in \{0, 1\}$ and $\hat{y}_i \in (0, 1)$ is the predicted probability. BCE is the negative log-likelihood of the Bernoulli distribution: $p(y|\hat{y}) = \hat{y}^y (1 - \hat{y})^{1-y}$.

Binary classification. Logistic regression, binary neural-network classifiers with sigmoid output, multi-label classification (as independent BCE per label). Implemented as `torch.nn.BCELoss` (expects sigmoid output) or `torch.nn.BCEWithLogitsLoss` (combines sigmoid + BCE for numerical stability).

Pros:

- A strictly proper scoring rule: the minimum is attained only at the true conditional probability $p(y = 1|x)$.
- Convex in the logits for a linear model, guaranteeing convergence to the global optimum.
- Produces well-calibrated probabilities (when optimized well and regularized appropriately).
- Gradients are well-behaved: $\partial\mathcal{L}/\partial z = \hat{y} - y$ for logit z, providing an intuitive error signal.

Cons:

- Unbounded: can approach $+\infty$ as $\hat{y} \to 0$ for a positive sample or $\hat{y} \to 1$ for a negative sample.
- Saturates for confident wrong predictions (gradient $\to 0$), which paradoxically makes it hard to correct very wrong predictions in later training stages.
- Sensitive to label noise: a flipped label can produce an extreme penalty.

1.2.2 Categorical Cross-Entropy (Multi-Class Softmax Loss)

For C classes with one-hot true labels $\mathbf{y}_i \in \{0, 1\}^C$ and predicted probabilities $\hat{\mathbf{y}}_i$:

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{c=1}^{C} y_{ic}\log(\hat{y}_{ic}), \qquad \hat{y}_{ic} = \frac{e^{z_{ic}}}{\sum_{k=1}^{C} e^{z_{ik}}}$$

This is the negative log-likelihood of the categorical (multinomial) distribution combined with the softmax function.

Multi-class classification. The default loss for CNNs (ResNet, EfficientNet on ImageNet), vision transformers, and virtually all modern classifiers with $C > 2$ mutually exclusive classes. Also serves as the base objective in language modeling (next-token prediction) and sequence tagging. Implemented as `torch.nn.CrossEntropyLoss` (expects raw logits, applies softmax internally).

Pros:

- Strictly proper scoring rule; maximum-likelihood under the categorical model.
- Convex in logits for the linear case.

- Computationally efficient with the log-sum-exp trick for numerical stability.
- Produces probability estimates that can be interpreted as confidence.

Cons:

- No built-in margin — does not explicitly encourage intra-class compactness or inter-class separation. Angular margin variants (ArcFace, CosFace) address this.
- Susceptible to noisy labels (same unbounded nature as BCE).
- When C is large (e.g., language modeling with vocabulary size 50 000+), computing the full softmax denominator is expensive. Remedies include hierarchical softmax, sampled softmax, and noise-contrastive estimation.

1.2.3 Hinge Loss (SVM Loss)

Binary ($y \in \{-1, +1\}$):

$$\mathcal{L} = \max\big(0,\ 1 - y \cdot f(x)\big)$$

Multi-class (Crammer–Singer):

$$\mathcal{L} = \max\Big(0,\ \max_{j \neq y}\big(1 + f_j(x) - f_y(x)\big)\Big)$$

The loss is zero when the correct score exceeds all incorrect scores by at least a margin of 1.

Support Vector Machines (SVMs). The canonical loss for max-margin classifiers. Hinge loss combined with L_2 regularization yields the SVM objective. Also used in some structured prediction settings (max-margin Markov networks). Less common in deep learning due to non-differentiability, but has been successfully used (e.g., deep SVMs with squared hinge).

Pros:

- Leads to sparse solutions: only support vectors (samples within or violating the margin) contribute to the gradient.
- Maximum-margin property improves generalization for linearly separable data.
- Convex objective for linear models.

Cons:

- Not differentiable at the margin boundary (kink at $1 - yf(x) = 0$).
- Does not produce calibrated class probabilities (though Platt scaling can be applied post-hoc).
- Gradient is zero for correctly classified samples with sufficient margin, which can slow convergence when initialization is good.

1.2.4 Focal Loss

$$\mathcal{L} = -\alpha_t (1 - p_t)^\gamma \log(p_t)$$

where $p_t = \begin{cases} \hat{y} & \text{if } y = 1 \\ 1 - \hat{y} & \text{otherwise} \end{cases}$ and α_t is a class-balancing weight. The modulating factor $(1 - p_t)^\gamma$ down-weights easy examples (where p_t is high).

Dense object detection and extreme class imbalance. Introduced in RetinaNet for one-stage object detection where the vast majority of anchor boxes are background. Also used in imbalanced classification, medical imaging, and any scenario where easy negatives dominate. Common settings: $\gamma = 2$, $\alpha = 0.25$.

Pros:

- Down-weights well-classified examples dynamically: the model focuses learning on hard, misclassified cases.
- α balances positive/negative class frequency; γ controls how aggressively easy examples are suppressed.
- Simple modification to standard cross-entropy; drop-in replacement.
- Significantly improves one-stage detector performance (closes gap with two-stage detectors).

Cons:

- Two hyperparameters (α, γ) to tune, though defaults work well in many settings.
- If γ is too large, even moderately easy examples are ignored, potentially preventing fine-grained calibration.
- The modulating factor reduces the total gradient magnitude, so learning rates may need adjustment.

1.2.5 Dice Loss

$$\mathcal{L} = 1 - \frac{2 \sum_i y_i \hat{y}_i + \epsilon}{\sum_i y_i + \sum_i \hat{y}_i + \epsilon}$$

where $y_i \in \{0, 1\}$ is the ground-truth mask and $\hat{y}_i \in [0, 1]$ is the predicted probability. ϵ is a small smoothing constant (typically 10^{-7} to 10^{-5}). The Dice coefficient $\frac{2|A \cap B|}{|A| + |B|}$ is the F1-score for binary masks.

Medical image segmentation. Dice loss is the standard loss for segmenting organs, tumors, and lesions in CT/MRI where the foreground region is tiny relative to the background (e.g., a tumor occupying $< 1\%$ of voxels). Also used in satellite imagery segmentation and any task with extreme foreground-background imbalance.

Pros:

- Handles extreme class imbalance natively — the loss depends on overlap, not on raw pixel counts.
- Directly optimizes a metric similar to the evaluation criterion (Dice score / IoU).
- Smooth and differentiable; compatible with standard optimizers.

Cons:

- Gradient can be unstable for very small predicted regions (denominator $\to 0$).
- May converge to a degenerate local minimum that predicts all background or all foreground; often stabilized by combining with cross-entropy (Combo Loss).
- Not calibrated for probability estimation; post-hoc calibration may be needed.

1.2.6 Tversky Loss

$$\mathcal{L} = 1 - \frac{\sum_i y_i \hat{y}_i + \epsilon}{\sum_i y_i \hat{y}_i + \alpha \sum_i y_i (1 - \hat{y}_i) + \beta \sum_i (1 - y_i) \hat{y}_i + \epsilon}$$

where α weights false negatives and β weights false positives. The Dice loss is recovered when $\alpha = \beta = 0.5$.

Segmentation with tunable precision-recall trade-off. When false negatives are more costly than false positives (e.g., missing a tumor), set $\alpha > \beta$. Conversely, $\beta > \alpha$ penalizes false positives more. Used in medical imaging where clinical requirements dictate the acceptable trade-off.

Pros:

- Explicit, interpretable control over the false-positive/false-negative trade-off via α and β.
- Generalizes Dice (F1-equivalent) and F-beta scores.
- Same robustness to class imbalance as Dice.

Cons:

- Two additional hyperparameters beyond the base Dice loss.
- Tuning requires domain knowledge about acceptable error rates.

1.2.7 ArcFace / CosFace / SphereFace (Angular Margin Losses)

ArcFace:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^{n} \log \frac{e^{s \cos(\theta_{y_i} + m)}}{e^{s \cos(\theta_{y_i} + m)} + \sum_{j \neq y_i} e^{s \cos \theta_j}}$$

where θ_j is the angle between the normalized feature embedding $\mathbf{x}_i$ and the class weight vector $\mathbf{W}_j$, s is a scaling factor (feature scale), and m is an additive angular margin. **CosFace** uses an additive cosine margin m instead: $\cos(\theta_{y_i}) - m$. **SphereFace** uses a multiplicative angular margin.

Face recognition and fine-grained classification. ArcFace is the de-facto standard for face verification (insightface, ArcFace-based systems achieve $> 99.8\%$ on LFW). Also used for speaker verification, person re-identification, fine-grained species classification, and any task where maximizing inter-class separation and intra-class compactness is paramount.

Pros:

- Enforces intra-class compactness and inter-class separability in angular space, producing highly discriminative embeddings.
- Interpretable geometry: the decision boundary is an angular margin.
- State-of-the-art on face recognition benchmarks; plug-and-play replacement for softmax in embedding models.

Cons:

- Hyperparameters (s, m) require tuning: typical values are $s = 64$, $m = 0.5$ for ArcFace.
- Converges more slowly than plain softmax due to the harder angular constraint.
- Requires normalized embeddings ($\|\mathbf{x}\| = 1$) and normalized class weights, which constrains the representation capacity somewhat.

1.2.8 Label Smoothing Cross-Entropy

$$\mathcal{L} = -\sum_{c=1}^{C} \tilde{y}_c \log \hat{y}_c, \qquad \tilde{y}_c = (1-\epsilon)\, y_c + \frac{\epsilon}{C}$$

The target distribution is smoothed: instead of a one-hot vector $(0, \ldots, 1, \ldots, 0)$, a small probability mass ϵ/C is assigned to every incorrect class.

Large-scale classification. Standard in ImageNet training ($\epsilon = 0.1$), BERT pretraining, and Vision Transformer (ViT) training. Also used in knowledge distillation as a softer target than one-hot labels.

Pros:

- Prevents the model from becoming overconfident (logit values are penalized from growing unboundedly).
- Improves calibration: the predicted probabilities better reflect true accuracy.
- Acts as a regularizer, often improving generalization and robustness to adversarial

examples.

- Encourages tighter clustering in the penultimate feature layer.

Cons:

- Slightly reduces training accuracy (by design).
- ϵ is an extra hyperparameter.
- May hurt knowledge distillation: the teacher's dark knowledge is partially erased by smoothing, reducing the benefit for the student.

1.3 Ranking Losses

Ranking losses learn relative ordering rather than absolute values. They are the backbone of metric learning, recommender systems, and information retrieval.

1.3.1 Triplet Loss

$$\mathcal{L} = \max\Big(0,\ d(\mathbf{a}, \mathbf{p}) - d(\mathbf{a}, \mathbf{n}) + m\Big)$$

where $\mathbf{a}$ is an anchor sample, $\mathbf{p}$ a positive (same class) sample, $\mathbf{n}$ a negative (different class) sample, $d(\cdot, \cdot)$ is a distance metric (usually Euclidean), and $m > 0$ is the margin.

Metric learning / embedding learning. Face verification (FaceNet), person re-identification, image retrieval (content-based), and similarity-based one-shot learning. The goal is to learn an embedding space where $d(\mathbf{a}, \mathbf{p}) + m < d(\mathbf{a}, \mathbf{n})$.

Pros:

- Learns embeddings with semantically meaningful relative distances.
- The margin m directly controls the separation between positive and negative pairs.
- Works with any differentiable distance function.

Cons:

- Naively, $O(n^3)$ possible triplets for n samples. Crucial to use mining strategies: **semi-hard** (most common — hardest negative that is still closer than the positive), **hard** (violating the margin), and **random**.
- Convergence is highly sensitive to the triplet mining strategy. Poor mining leads to slow or no learning.
- Requires careful batch construction (PK sampling: P classes, K samples each).

1.3.2 Contrastive Loss (Siamese)

$$\mathcal{L} = y \cdot d^2 \; + \; (1-y) \cdot \big(\max(0,\, m - d)\big)^2$$

where $d = \|f(x_1) - f(x_2)\|_2$ is the Euclidean distance between embeddings of a pair, $y \in \{0, 1\}$ indicates whether the pair is similar, and m is the margin for dissimilar pairs.

Siamese networks for signature verification, face comparison, and one-shot learning (e.g., Omniglot). Trained on pairs of samples with binary similarity labels.

Pros:

- Simple formulation; easy to implement.
- Works well with pairwise supervision.
- Dissimilar pairs are not forced arbitrarily far away — only beyond the margin m, which prevents embedding-space collapse.

Cons:

- Pair sampling strategy strongly affects results; random pairing leads to many easy negatives.
- Margin m is a sensitive hyperparameter.
- Does not model relative triple-wise relationships, which can be more informative.

1.3.3 Bayesian Personalized Ranking (BPR)

$$\mathcal{L} = -\sum_{(u,i,j)\in\mathcal{D}} \log \sigma\big(\hat{r}_{ui} - \hat{r}_{uj}\big)$$

where $\mathcal{D} = \{(u, i, j) \mid \text{user } u \text{ interacted with item } i \text{ but not } j\}$, $\hat{r}_{ui}$ is the predicted preference score, and σ is the sigmoid function.

Recommender systems with implicit feedback. Matrix factorization for collaborative filtering (rendle2012bpr). Widely used when only positive-only feedback is available (clicks, purchases, views) — unobserved interactions are treated as (weak) negatives.

Pros:

- Optimizes ranking (relative order) directly from implicit feedback, rather than trying to predict a regression target.
- Pairwise approach: naturally suited for top-K recommendation tasks.
- Convex in model parameters for linear models.

Cons:

- Uniform negative sampling is weak — most randomly sampled negatives are trivially easy, contributing little gradient.
- Does not distinguish between unobserved and truly negative interactions.
- Requires careful negative sampling (e.g., popularity-based) for good performance.

1.4 Generative and Self-Supervised Losses

1.4.1 VAE Evidence Lower Bound (ELBO)

$$\text{ELBO} = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\big[\log p_\theta(\mathbf{x}|\mathbf{z})\big] \;-\; \text{KL}\big[q_\phi(\mathbf{z}|\mathbf{x}) \,\|\, p(\mathbf{z})\big]$$

The **reconstruction term** (first) measures how well the decoder reconstructs $\mathbf{x}$ from latent $\mathbf{z}$. The **KL divergence term** (second) regularizes the approximate posterior toward the prior $p(\mathbf{z})$, typically $\mathcal{N}(\mathbf{0}, \mathbf{I})$. Maximizing the ELBO is equivalent to minimizing $\text{KL}[q_\phi(z|x) \,\|\, p_\theta(z|x)]$.

Variational Autoencoders. Standard training objective for VAEs in generative modeling, representation learning, anomaly detection, and latent variable modeling. The reparameterization trick enables gradient-based optimization through the stochastic sampling step.

Pros:

- Principled probabilistic framework: learns both an inference network (encoder) and a generative model (decoder).
- Amortized inference: inference is fast at test time (single forward pass) unlike MCMC.
- The KL term provides a built-in regularizer that prevents the latent space from overfitting.

Cons:

- **Posterior collapse:** the decoder learns to ignore $\mathbf{z}$, driving the KL term to zero and reducing the VAE to a standard autoencoder. Mitigations include KL annealing, free bits, and stronger decoders.
- Gaussian likelihood produces blurry reconstructions for images. Alternatives: use a learned likelihood (PixelCNN decoder) or a GAN discriminator on reconstructions (VAE-GAN).
- The isotropic Gaussian prior may be too simple for complex data.

1.4.2 DDPM Loss (Denoising Diffusion Probabilistic Models)

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t,\mathbf{x}_0,\boldsymbol{\epsilon}}\Big[\big\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\big(\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon},\ t\big)\big\|^2\Big]$$

where $\mathbf{x}_0$ is a clean data sample, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is Gaussian noise, $\bar{\alpha}_t$ is the cumulative noise schedule product, and $\boldsymbol{\epsilon}_\theta$ is a neural network (typically a U-Net) trained to predict the noise added at timestep $t \sim \text{Uniform}(1, T)$.

Diffusion models for generation. State-of-the-art in image synthesis (Stable Diffusion, DALL-E 2, Imagen), audio generation (DiffWave), video generation, and molecular conformer generation. The loss is a simple weighted MSE; the complexity is in the architecture and the iterative sampling process.

Pros:

- Simple MSE-based loss — no adversarial training; stable optimization.
- Does not require explicit likelihood evaluation during training (only denoising score matching).
- State-of-the-art generation quality and mode coverage.
- Can be conditioned on text, class labels, images, etc.

Cons:

- Slow sampling: requires T sequential denoising steps (typically 50–1000). Accelerated sampling (DDIM, DPM-Solver) mitigates this.
- No explicit likelihood tractable during training (though variational bounds exist).
- Training requires careful noise-schedule design.

1.4.3 InfoNCE Loss (SimCLR / CPC)

$$\mathcal{L} = -\log \frac{\exp\big(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau\big)}{\sum_{k=1}^{2N} \mathbf{1}_{[k \neq i]} \exp\big(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau\big)}$$

where $(\mathbf{z}_i, \mathbf{z}_j)$ is a positive pair (two augmentations of the same image), the denominator sums over all other samples in the batch (negatives), $\text{sim}(\cdot,\cdot)$ is cosine similarity, and τ is a temperature parameter.

Contrastive self-supervised learning. SimCLR, MoCo, CLIP, BYOL (variant), and CPC use InfoNCE (or its close relatives) to pretrain encoders on unlabeled data. The learned representations transfer well to downstream tasks via linear probing or fine-tuning.

Pros:

- Learns useful visual/semantic representations without any labels.

- Maximizes mutual information between different views of the same sample.
- Theoretically grounded as a lower bound on mutual information.

Cons:

- Requires a large batch size (SimCLR uses 4096+) or a memory bank (MoCo) to provide enough negatives.
- Sensitive to the choice of data augmentations — different domains require different augmentation strategies.
- Temperature τ needs tuning (typically 0.07–0.5).

1.4.4 GAN Adversarial Losses

Minimax GAN (original):

$$\min_G \max_D \; \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_{\mathbf{z}}}[\log(1 - D(G(\mathbf{z})))]$$

Non-saturating GAN (stronger gradients for G):

$$\mathcal{L}_G = -\mathbb{E}_{\mathbf{z}}[\log D(G(\mathbf{z}))]$$

Wasserstein GAN (WGAN):

$$\mathcal{L} = \max_{\|D\|_L \leq 1} \; \mathbb{E}_{\mathbf{x}\sim p}[D(\mathbf{x})] - \mathbb{E}_{\tilde{\mathbf{x}}\sim p_g}[D(\tilde{\mathbf{x}})]$$

with gradient penalty $\lambda \mathbb{E}_{\hat{\mathbf{x}}}\left[(\|\nabla_{\hat{\mathbf{x}}} D(\hat{\mathbf{x}})\|_2 - 1)^2\right]$ for the 1-Lipschitz constraint.
Least-Squares GAN (LSGAN):

$$\min_D \tfrac{1}{2}\mathbb{E}[(D(\mathbf{x}) - 1)^2] + \tfrac{1}{2}\mathbb{E}[D(G(\mathbf{z}))^2], \qquad \min_G \tfrac{1}{2}\mathbb{E}[(D(G(\mathbf{z})) - 1)^2]$$

Generative Adversarial Networks. GANs are used for unconditional and conditional image generation (StyleGAN, BigGAN), image-to-image translation (pix2pix, CycleGAN), super-resolution, and data augmentation. The minimax loss is rarely used in practice; the non-saturating and WGAN-GP variants are standard. LSGAN is common when one wants to avoid sigmoid saturation in the discriminator.

Pros:

- Produces sharp, realistic samples; avoids the blurriness of MSE-based generators.
- WGAN-GP provides a meaningful loss curve that correlates with sample quality.
- Unsupervised — learns p_{data} implicitly.

Cons:

- **Training instability:** mode collapse, discriminator-generator imbalance, and non-convergence are classic challenges.

- Two networks to tune and balance.
- WGAN-GP adds a significant computational cost for the gradient penalty.
- Hard to evaluate: no single metric (FID, IS, KID all have caveats).

1.4.5 CTC Loss (Connectionist Temporal Classification)

$$\mathcal{L}_{\text{CTC}} = -\log \sum_{\boldsymbol{\pi} \in \mathcal{B}^{-1}(\mathbf{y})} p(\boldsymbol{\pi}|\mathbf{x})$$

where $\mathbf{x}$ is the input sequence, $\mathbf{y}$ is the target sequence, $\boldsymbol{\pi}$ is a frame-level alignment path (including a blank symbol), $\mathcal{B}$ is the many-to-one mapping that collapses consecutive repeated labels and removes blanks, and $p(\boldsymbol{\pi}|\mathbf{x}) = \prod_t p(\pi_t|\mathbf{x})$ under the conditional independence assumption. The sum over all valid paths is computed efficiently via the forward-backward algorithm (dynamic programming).

Sequence-to-sequence tasks without explicit alignment. Speech recognition (DeepSpeech), handwriting recognition (IAM, RIMES), and optical character recognition. CTC is paired with recurrent or transformer encoders and a linear+softmax output layer over the extended vocabulary (labels + blank).

Pros:

- Eliminates the need for frame-level alignment labels — learns alignment automatically.
- Handles variable-length input and output sequences natively.
- Efficient forward-backward computation using dynamic programming.

Cons:

- Conditional independence assumption: π_t depends only on $\mathbf{x}$, not on π_{t-1}. Violated for language modeling, so CTC is often augmented with an external language model.
- Tends to produce "spiky"/peaky output distributions where one frame carries most of the probability mass.
- The blank symbol can dominate, causing slow convergence.

1.5 Object Detection Losses

1.5.1 IoU Loss Family

IoU loss:

$$\mathcal{L}_{\text{IoU}} = 1 - \frac{|B \cap B^{gt}|}{|B \cup B^{gt}|}$$

where B is the predicted bounding box and B^{gt} is the ground truth.

GIoU (Generalized IoU):

$$\mathcal{L}_{\text{GIoU}} = 1 - \text{IoU} + \frac{|C \setminus (B \cup B^{gt})|}{|C|}$$

where C is the smallest enclosing convex hull of B and B^{gt}.
DIoU (Distance IoU):

$$\mathcal{L}_{\text{DIoU}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2}$$

where ρ is Euclidean distance between box centers and c is the diagonal length of the smallest enclosing box.
CIoU (Complete IoU):

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2}{c^2} + \alpha v$$

where $v = \frac{4}{\pi^2}(\arctan \frac{w^{gt}}{h^{gt}} - \arctan \frac{w}{h})^2$ captures aspect-ratio consistency and $\alpha = \frac{v}{(1-\text{IoU})+v}$ is a balancing weight.

Bounding-box regression in object detectors. IoU-based losses are used in YOLO (v3 onward), DETR, Sparse R-CNN, and most modern detectors as a complement to or replacement for Smooth L1. They are scale-invariant and correlate better with the evaluation metric (mAP) than L_n losses.

Pros:

- Scale-invariant: IoU is dimensionless; the loss works equally for small and large boxes.
- GIoU provides a meaningful gradient even when boxes do not overlap (IoU = 0 for non-overlapping boxes gives zero gradient).
- DIoU and CIoU incorporate center-distance and aspect-ratio penalties, converging faster and more accurately.
- Better correlation with mAP evaluation metric.

Cons:

- IoU is non-differentiable at certain configurations; GIoU/DIoU/CIoU smooth this.
- CIoU's aspect-ratio term can be numerically unstable during early training when box sizes are chaotic.
- Computational overhead (intersection, union, enclosing hull) is higher than Smooth L1, though negligible in practice.

1.5.2 Hungarian Matching Loss (DETR)

The DETR loss combines a bipartite matching step with a per-prediction loss:

$$\mathcal{L} = \sum_{i=1}^{N} \Big[-\log \hat{p}_{\hat{\sigma}(i)}(c_i) + \mathbf{1}_{c_i \neq \varnothing} \, \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\hat{\sigma}(i)}) \Big]$$

where $\hat{\sigma}$ is the optimal permutation (one-to-one assignment) between predicted and ground-truth objects, found via the Hungarian algorithm minimizing the matching cost. The box loss is a weighted combination of $\mathcal{L}_{\text{IoU}}$ and L_1 loss.

End-to-end object detection without hand-crafted components. DETR (DEtection TRansformer) eliminates the need for anchor boxes, non-maximum suppression (NMS), and region proposal networks. The Hungarian matching ensures a one-to-one assignment that acts as a learned NMS.

Pros:

- Fully differentiable pipeline — no NMS, no anchors, no hand-crafted post-processing.
- Direct set prediction: the model outputs a set of detections and the loss matches them to ground truth optimally.
- Simpler architecture conceptually; easier to extend to panoptic segmentation.

Cons:

- The Hungarian algorithm is $O(N^3)$ in the number of predictions, though N is typically small (100).
- Slow convergence: DETR requires hundreds of training epochs, compared to tens for anchor-based detectors.
- Training instability from the matching step: a small change in scores can change the bipartite matching drastically.

1.6 Summary and Practical Guidance

Choosing a loss function — practical heuristics:

- **Regression with no outliers** → start with MSE.
- **Regression with outliers** → Huber or Log-Cosh.
- **Count data** → Poisson (or Negative Binomial for overdispersion).
- **Binary classification** → BCE with logits.
- **Multi-class classification** → Categorical Cross-Entropy. For fine-grained, add ArcFace margin.
- **Class imbalance** → Focal Loss or weighted CE.

- **Segmentation** → combo of Dice + CE (or Tversky for precision-recall tuning).
- **Object detection** → Focal Loss (classification) + CIoU/GIoU (regression).
- **Face/embedding learning** → ArcFace + Triplet loss.
- **Generative images** → DDPM (MSE) or non-saturating WGAN-GP.
- **Recommender systems** → BPR or sampled softmax.

Chapter 2

Activation Functions

Activation functions introduce non-linearity into neural networks, enabling them to approximate arbitrary functions. Without non-linearity, stacking layers simply composes affine transformations, which collapses to a single linear map. The choice of activation function profoundly affects gradient flow, training dynamics, and the representational capacity of the network.

An **activation function** $f : \mathbb{R} \to \mathbb{R}$ (or $f : \mathbb{R}^n \to \mathbb{R}^n$) is applied element-wise to the pre-activation output of a linear layer: $\mathbf{h}^{(l)} = f(\mathbf{W}^{(l)}\mathbf{x}^{(l-1)} + \mathbf{b}^{(l)})$. For a network to be a universal approximator, f must be non-linear and non-polynomial (bounded or unbounded).

2.1 Sigmoid Family — Saturating Activations

2.1.1 Logistic Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}, \qquad \sigma : \mathbb{R} \to (0,1)$$

The output is strictly bounded between 0 and 1. The derivative is $\sigma'(x) = \sigma(x)(1-\sigma(x))$, attaining a maximum of 0.25 at $x = 0$ and decaying exponentially as $|x| \to \infty$.

Binary classification output and gating mechanisms.

- Final layer of binary classifiers (before BCE loss).
- Gating in LSTMs: the forget gate $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$ and input/output gates use sigmoid to produce values in $(0, 1)$.
- Gated Recurrent Units (GRU) similarly use sigmoid for update and reset gates.
- Historically used as hidden-layer activation in 1980s–1990s networks, now largely replaced by ReLU variants.

Pros:

- Smooth and differentiable everywhere (C^∞).
- Output is bounded and interpretable as probability (0 to 1).
- Monotonic — preserves ordering of activations.
- Saturation effectively "turns off" a neuron for extreme inputs.

Cons:

- **Vanishing gradients:** $\max \sigma'(x) = 0.25$, and for $|x| > 5$, $\sigma'(x) \approx 0$. In deep networks, gradients shrink exponentially, preventing learning in early layers.

- Not zero-centered: outputs are always positive ($\sigma(x) > 0 \; \forall x$). This introduces a bias in gradient updates for subsequent layers: all gradients w.r.t. weights of a sigmoid-activated layer have the same sign as the backpropagated gradient, causing zigzag optimization or slow convergence.
- $\exp(-x)$ is computationally slower than max or piecewise operations.

2.1.2 Hyperbolic Tangent (Tanh)

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1, \qquad \tanh : \mathbb{R} \to (-1, 1)$$

The derivative is $\tanh'(x) = 1 - \tanh^2(x)$, reaching a maximum of 1 at $x = 0$.

Hidden layer activation pre-ReLU era; RNN/LSTM cell states.

- Standard hidden activation in LSTMs: the output gate modulates $\tanh(c_t)$ to produce the hidden state h_t.
- GRU candidate activation.
- Still used in some attention mechanisms and as a bounded nonlinearity in reinforcement learning policy outputs.

Pros:

- Zero-centered: outputs span $(-1, 1)$, reducing the zigzag optimization issue of sigmoid.
- Stronger gradients near zero compared to sigmoid (max derivative = 1 vs. 0.25).
- Bounded — prevents activations from exploding.

Cons:

- Still suffers from saturation: $\tanh'(x) \to 0$ for $|x| > 3$, causing vanishing gradients in deep networks.
- Exponential computation cost (though hardware implementations have improved).
- The "sweet spot" of non-saturation is narrower than ReLU's.

2.1.3 Hard Sigmoid

$$\text{HardSigmoid}(x) = \max\big(0,\ \min(1,\ 0.2x + 0.5)\big)$$

A piecewise-linear approximation of the logistic sigmoid. Saturates at 0 and 1 at $x = -2.5$ and $x = 2.5$ respectively.

Mobile/efficient architectures. Used in LSTM variants on resource-constrained devices. Computationally cheaper than the exponential sigmoid. Implemented as `torch.nn.Hardsigmoid`.

Pros: Piecewise linear — fast and cheap. No exponentials. **Cons:** Coarse approximation; gradient is zero outside $[-2.5, 2.5]$. Less accurate.

2.2 ReLU Family — Non-saturating Activations

2.2.1 Rectified Linear Unit (ReLU)

$$f(x) = \max(0, x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

Range: $[0, \infty)$. The derivative is 1 for $x > 0$ and 0 for $x < 0$. At $x = 0$, the derivative is undefined; in practice, frameworks assign 0 (or sometimes 0.5 for the subgradient).

The default activation for deep networks (2012–2020). The first standard activation that enabled training very deep architectures. Used in ResNet, VGG, AlexNet, CNNs, MLPs, and most feed-forward architectures. `torch.nn.ReLU()`.

Pros:

- **Non-saturating:** gradient is 1 for all positive inputs, avoiding the vanishing gradient problem entirely in the active region.
- **Sparse activation:** 50% of neurons are zero, creating sparse representations that can be computationally and statistically efficient.
- Computationally trivial: $\max(0, x)$ is a simple threshold operation.
- Empirically enables training of very deep networks without pre-training.

Cons:

- **Dying ReLU:** if a neuron receives consistently negative inputs, its gradient is zero and weights never update. Once dead, it stays dead (unless the data distribution shifts). This permanently reduces model capacity.
- Not zero-centered: outputs are always non-negative, causing bias in subsequent layer gradients (mitigated by Batch Normalization).
- Unbounded: output can grow arbitrarily large, potentially causing numerical instability.
- Non-differentiable at 0, though this is rarely a practical issue.

2.2.2 ReLU6

$$f(x) = \min(\max(0, x), 6)$$

Rectified output is capped at 6. This bounds the dynamic range, which is beneficial when activations are quantized to fixed-point arithmetic.

MobileNets and quantized models. ReLU6 limits activation range to $[0, 6]$, which maps cleanly to 8-bit integer quantization. Used in MobileNetV1, MobileNetV2, and other mobile-first architectures. `torch.nn.ReLU6()`.

Pros: Bounded output → better numerical behavior under quantization. Encourages compact representations. **Cons:** Clipping at 6 loses information for features that would benefit from larger activations. Slightly less expressive than unbounded ReLU.

2.2.3 Leaky ReLU

$$f(x) = \max(\alpha x,\, x) = \begin{cases} x & \text{if } x \geq 0 \\ \alpha x & \text{if } x < 0 \end{cases}$$

where α is a small constant (default 0.01). For $x < 0$, the gradient is α instead of 0, giving a small nonzero slope.

Fixing the dying ReLU. A drop-in replacement for ReLU when dead neurons are problematic. Used in GAN discriminators, some autoencoder architectures, and whenever ReLU yields too many dead units. `torch.nn.LeakyReLU()`.

Pros:

- Eliminates the dying ReLU problem — all neurons receive gradient signal (though small for negative inputs).
- Maintains non-saturating behavior for positive inputs.
- Simple, no extra parameters (for fixed α).

Cons:

- α is fixed and not learned; different tasks may benefit from different slopes.
- Performance gain over ReLU is often marginal in practice.
- Still not differentiable at 0 (kink).

2.2.4 Parametric ReLU (PReLU)

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases}$$

where α is a *learned* parameter, updated via gradient descent along with the network weights. α can be shared across channels or learned per-channel.

Adaptive negative slope. PReLU slightly improves over Leaky ReLU on ImageNet classification (He et al., 2015). The learned α adapts to the data distribution, potentially giving better gradient flow.

Pros:

- Adaptively learns the best negative slope per channel.
- Negligible parameter overhead (one per channel).
- Can outperform Leaky ReLU when α is well-chosen by the optimizer.

Cons:

- Extra parameter can overfit on small datasets.
- Learning α adds (minor) optimization complexity.
- The improvement over Leaky ReLU is often small or within noise.

2.2.5 Exponential Linear Unit (ELU)

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if } x \leq 0 \end{cases}$$

where $\alpha > 0$ is typically 1.0. For $x \to -\infty$, $f(x) \to -\alpha$. The function is continuous and differentiable everywhere including $x = 0$ (provided the derivative is defined piecewise with limits).

Robust alternative to ReLU. ELU pushes the mean activation toward zero (negative values for $x < 0$ shift the mean down), which can speed up learning by reducing bias shift. Used in some computer vision networks and as a comparison baseline.

Pros:

- Smooth (C^1) and differentiable everywhere, unlike ReLU and its leaky variants.
- Negative outputs for $x < 0$ help center activations near zero, reducing internal covariate shift.
- Robust to noise: the saturating negative side acts as a noise filter for strongly negative pre-activations.

Cons:

- $\exp(x)$ computation is more expensive than $\max(0, x)$.
- α is a hyperparameter.
- Can still saturate for very negative x (gradient $\to 0$), though this is a feature for robustness.

2.2.6 Scaled Exponential Linear Unit (SELU)

$$f(x) = \lambda \begin{cases} x & \text{if } x > 0 \\ \alpha e^x - \alpha & \text{if } x \leq 0 \end{cases}$$

with $\alpha \approx 1.6733$ and $\lambda \approx 1.0507$. These constants are derived from fixed-point analysis to ensure that activations converge to zero mean and unit variance under specific conditions.

Self-normalizing networks (SNNs). SELU is the core of SNNs — feed-forward networks that automatically maintain normalized activations without Batch Normalization. Used when BN is undesirable or impossible (e.g., some online learning or small-batch settings).

Pros:

- **Self-normalizing:** under the right conditions, activations converge to $\mathcal{N}(0, 1)$ automatically.
- Enables training deep networks without batch normalization.
- Strong theoretical foundation (Banach fixed-point theorem on activation distributions).

Cons:

- Requires specific weight initialization (LeCun normal) and alpha-dropout (a variant of dropout that preserves mean/variance).
- The self-normalizing property breaks if the architecture violates assumptions (skip connections, non-sequential structure).
- Less effective when BN is already present.
- Not widely adopted outside SNN literature.

2.3 Smooth and Self-Gated Activations

2.3.1 Softplus

$$f(x) = \ln(1 + e^x)$$

A smooth (differentiable) approximation of ReLU. The derivative is the sigmoid function: $f'(x) = \sigma(x) = 1/(1 + e^{-x})$. For large x, $f(x) \approx x$; for $x \ll 0$, $f(x) \approx 0$.

Smooth ReLU surrogate. Used when strictly positive output is required with differentiability — e.g., modeling variance parameters in VAEs or ensuring positivity of certain latent variables. Also used in some reinforcement learning settings.

Pros:

- Smooth and everywhere differentiable (C^∞).
- Always strictly positive — useful for parameters that must be > 0.

Cons:

- Saturates slowly for large negative x: even at $x = -5$, $\ln(1 + e^{-5}) \approx 0.0067$, not exactly zero.
- Non-zero at $x = 0$ ($\ln 2 \approx 0.693$), which may require offset correction.
- exp and log are computationally heavier than ReLU.

2.3.2 Gaussian Error Linear Unit (GELU)

$$\text{GELU}(x) = x \cdot \Phi(x) = x \cdot \frac{1}{2}\left[1 + \text{erf}\left(x/\sqrt{2}\right)\right]$$

where $\Phi(x)$ is the CDF of the standard normal distribution. A fast approximation widely used in practice:

$$\text{GELU}(x) \approx 0.5x\left(1 + \tanh\left[\sqrt{2/\pi}\,(x + 0.044715x^3)\right]\right)$$

The function stochastically multiplies input by either 0 or 1 with probability $\Phi(x)$, similar to dropout but based on the magnitude of x itself.

The transformer activation. GELU is used in BERT, GPT (GPT-1/2), Vision Transformer (ViT), and most transformer-based language models. It is a strong default for NLP and has proven highly effective for attention-based architectures.

Pros:

- Smooth and non-monotonic near zero, combining properties of ReLU (linear for large x), dropout (stochastic regularization), and zoneout.
- Probabilistic interpretation: input is gated by the probability that it's positive under a standard normal.

- Consistently outperforms ReLU and ELU in transformer architectures.
- The tanh approximation is fast enough for production.

Cons:

- More computationally expensive than ReLU (tanh or erf evaluation).
- The exact form with erf is not optimized on all hardware.
- Non-monotonic behavior can occasionally cause issues in certain architectures.

2.3.3 Swish / SiLU

$$f(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

where σ is the sigmoid function. Swish is a smooth, self-gated (sigmoid-gated) activation. It is *non-monotonic*: for small negative x, the function dips slightly below zero before approaching zero from below as $x \to -\infty$.

EfficientNet and modern vision models. Swish was discovered via neural architecture search and is used in EfficientNet, EfficientDet, and some CNN variants. PyTorch implements it as `torch.nn.SiLU()` (Sigmoid Linear Unit).

Pros:

- Smooth, self-gated, and consistently outperforms ReLU on ImageNet benchmarks.
- Non-monotonic bump may provide a useful inductive bias.
- Simple to implement: just $x \times \sigma(x)$.

Cons:

- More computation than ReLU (sigmoid requires exp).
- Marginal gain over GELU in many settings.
- Non-monotonic behavior may be undesirable for some tasks (e.g., value functions in RL).

2.3.4 Mish

$$f(x) = x \cdot \tanh\big(\text{softplus}(x)\big) = x \cdot \tanh\big(\ln(1 + e^{x})\big)$$

Mish is another smooth, self-gated, non-monotonic activation. It is unbounded above, bounded below (approaches 0 as $x \to -\infty$), and has a small negative lobe near $x \approx -1$.

Experimental replacement for ReLU/Swish. Mish was proposed by Misra (2019) and shown to slightly outperform Swish and ReLU on some image classification bench-

marks (CIFAR, ImageNet) and object detection. It is self-regularizing due to its smoothness.

Pros:

- Smooth and non-monotonic — strong empirical performance in vision tasks.
- Self-regularizing: the smooth landscape may guide optimization toward flatter minima.
- Often slightly better than Swish/ReLU on classification accuracy.

Cons:

- Heavier compute: $\tanh(\ln(1+e^x))$ involves exp, ln, and tanh.
- Not a standard activation in major frameworks (no `torch.nn.Mish`); requires custom implementation.
- Marginal and inconsistent gains over Swish/GELU across broader benchmarks.

2.3.5 Hard Swish (H-Swish)

$$f(x) = x \cdot \frac{\text{ReLU6}(x+3)}{6} = \begin{cases} 0 & \text{if } x \leq -3 \\ x & \text{if } x \geq 3 \\ x \cdot \frac{x+3}{6} & \text{otherwise} \end{cases}$$

A piecewise-linear approximation of Swish, replacing the sigmoid with a hard sigmoid variant. Optimized for mobile/edge inference.

MobileNetV3. Hard Swish was introduced alongside MobileNetV3 to provide Swish-like accuracy at a fraction of the compute cost. It eliminates the exp calls in Swish while preserving the general shape. `torch.nn.Hardswish()`.

Pros:

- Computationally cheap on mobile/edge — only ReLU6 and arithmetic.
- Matches or nearly matches Swish accuracy on mobile-scale models.
- Fully quantizable.

Cons:

- Slightly less accurate than full Swish on large-scale models.
- The piecewise-linear approximation introduces non-smooth joints.

2.4 Specialized Activations

2.4.1 Maxout

$$f(\mathbf{x}) = \max_{k=1,\ldots,K} \left(\mathbf{w}_k^\top \mathbf{x} + b_k\right)$$

where K is the number of linear "pieces." Each output neuron has K sets of weights. The max operation selects the maximum among K affine transformations.

Universal approximation within convex sets. Maxout networks (Goodfellow et al., 2013) can approximate any convex function. Historically paired with dropout for state-of-the-art results on MNIST, CIFAR-10, and SVHN. Rare in modern architectures due to parameter inefficiency.

Pros:

- Learns the activation shape rather than using a fixed function.
- Can approximate any continuous function given enough units (with at least two linear pieces).
- Works well with dropout.

Cons:

- Multiplies parameters by K — memory and compute costs.
- Requires K to be chosen per layer.
- Outperformed by simpler architectures with BN + ReLU.

2.4.2 Softmax

$$f_i(\mathbf{z}) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}, \qquad \sum_{i=1}^{C} f_i(\mathbf{z}) = 1$$

Maps a vector of C unnormalized logits to a valid probability distribution. The exponential ensures positivity; the denominator ensures normalization. The Jacobian is $J_{ij} = f_i(\delta_{ij} - f_j)$, where δ_{ij} is the Kronecker delta.

Multi-class classification output layer. Softmax is the universal choice for the final layer in C-class classifiers, typically followed by cross-entropy loss. Also used in attention mechanisms: $\text{Attention}(Q, K, V) = \text{softmax}(QK^\top/\sqrt{d_k})V$, where it converts similarity scores into attention weights.

Pros:

- Produces a valid probability simplex (non-negative, sum to 1).
- Differentiable and smooth.
- Amplifies differences: exponentiation makes the largest logit dominate.

Cons:

- Can produce overly confident (peaked) distributions, especially with large logits. Label smoothing and temperature scaling mitigate this.
- Computationally expensive for large C (e.g., 50k-vocabulary language models) — softmax over the full vocabulary at each timestep is impractical.
- Saturation: if one logit is much larger than others, the softmax gradient for incorrect classes approaches 0.

2.4.3 Gumbel-Softmax

The Gumbel-Softmax provides a continuous relaxation of categorical sampling:

$$y_i = \frac{\exp\big((\log \pi_i + g_i)/\tau\big)}{\sum_{j=1}^{C} \exp\big((\log \pi_j + g_j)/\tau\big)}$$

where $g_i \sim \text{Gumbel}(0, 1)$ are i.i.d. samples drawn as $g_i = -\log(-\log(u_i))$ with $u_i \sim \text{Uniform}(0, 1)$. The temperature $\tau > 0$ controls discreteness: as $\tau \to 0$, samples approach one-hot; as $\tau \to \infty$, samples approach uniform.

Discrete latent variables and differentiable sampling. Used in VAEs with categorical latents (VQ-VAE), GANs for discrete sequence generation (SeqGAN), and reinforcement learning for differentiable action selection. The straight-through estimator is often combined with Gumbel-Softmax: discrete samples in the forward pass, continuous relaxation in the backward pass.

Pros:

- Enables gradient-based optimization through discrete sampling.
- Temperature τ allows annealing from smooth to discrete.
- Reparameterizable: the randomness is in g_i, not the parameters.

Cons:

- The gap between the relaxation and true discrete distribution causes bias in gradient estimates.
- Temperature annealing schedule needs tuning.
- Variance of gradients can be high.

2.4.4 Sinusoidal / SIREN Activation

$$f(x) = \sin(\omega_0 x)$$

where ω_0 is a frequency scaling factor (typically $\omega_0 = 30$). SIREN (Sinusoidal Representation Networks) uses periodic activations to model signals with fine spatial and temporal

derivatives.

Implicit neural representations. SIREN is used for representing images, 3D shapes (signed distance functions), audio, and video as continuous functions. Unlike ReLU networks, SIREN can represent signals with high-frequency detail. Also used in neural radiance fields (NeRF) variants.

Pros:

- Exact derivatives are also sinusoidal (closed-form), enabling representation of high-order derivatives.
- Models high-frequency detail that ReLU networks struggle with (spectral bias of ReLU is toward low frequencies).
- Ideal for implicit representations that need to be evaluated and differentiated anywhere.

Cons:

- Requires careful initialization: weights must be initialized with $w \sim \mathcal{U}(-\sqrt{6/f_{in}}/\omega_0, \sqrt{6/f_{in}}/\omega_0)$ to preserve signal distribution through layers.
- The periodic nature can cause aliasing and instability if ω_0 is too large.
- Not suitable for standard classification/regression — specialized to signal representation.

2.4.5 Bent Identity

$$f(x) = \frac{\sqrt{x^2+1}-1}{2} + x$$

A smooth, differentiable function that behaves like ReLU for $|x| \gg 0$ but has non-zero derivative everywhere. Saturates slowly for large negative x.

Pros: Smooth and differentiable everywhere, non-zero gradient always. **Cons:** Rarely used; outclassed by simpler options. Adds $\sqrt{x^2+1}$ computation.

2.4.6 Gaussian Activation

$$f(x) = e^{-x^2}$$

A radial basis function (RBF) activation. Produces a bell-shaped response centered at zero. Used in RBF networks.

Radial basis function networks. RBF networks use Gaussian activations in their hidden layer to create locally-tuned receptive fields. Less common in deep learning but used in some kernel-based and few-shot learning methods.

Pros: Localized activation — each neuron responds to a specific region of input space. Smooth and differentiable. **Cons:** Vanishing gradient for large $|x|$. Requires careful center/width initialization. Not deep-learning-friendly.

2.5 Summary Table

Table 2.1: Summary of Activation Functions

Activation	Range	Monotonic	Smooth	Use Case
Sigmoid	$(0, 1)$	Yes	Yes	Binary output, gates
Tanh	$(-1, 1)$	Yes	Yes	LSTM/RNN hidden
ReLU	$[0, \infty)$	Yes	No	Default CNN/MLP
Leaky ReLU	$(-\infty, \infty)$	Yes	No	Fix dying ReLU
PReLU	$(-\infty, \infty)$	Yes	No	Learnable slope
ELU	$(-\alpha, \infty)$	Yes	C^1	Robust + zero-mean
SELU	$(-\lambda\alpha, \infty)$	Yes	C^1	Self-normalizing nets
GELU	$(-0.17, \infty)$	No	Yes	Transformers
Swish/SiLU	$(-0.28, \infty)$	No	Yes	EfficientNet
Mish	$(-0.31, \infty)$	No	Yes	Vision experiments
Hard Swish	$[0, \infty)$	Yes	No	Mobile/edge
Softplus	$(0, \infty)$	Yes	Yes	Positive output
Maxout	$(-\infty, \infty)$	No	No	Historical/universal
Softmax	$(0, 1)^C$	No	Yes	Multi-class output
SIREN	$[-1, 1]$	No	Yes	Implicit representations

Practical recommendations:

- **For CNNs/MLPs:** Start with ReLU. Switch to GELU or SiLU if computation budget allows.
- **For Transformers:** GELU is the standard; SiLU/Swish is a reasonable alternative.
- **For Mobile/Edge:** ReLU6 for early layers, Hard Swish for non-linearities.
- **For output layers:** Softmax (multi-class), Sigmoid (binary or multi-label), Linear (regression).
- **Avoid Sigmoid/Tanh in hidden layers** of deep networks unless specifically needed (e.g., LSTM gates).
- **If dying ReLU is a problem:** try Leaky ReLU or ELU before SELU.

Chapter 3

Regularization Techniques

Regularization encompasses any modification to a learning algorithm that reduces its generalization error at the expense of increased training error. In deep learning, regularization combats overfitting by constraining model complexity, injecting noise, or encouraging simpler representations.

Regularization is any mechanism that restricts the hypothesis space $\mathcal{H}$ or penalizes complexity, trading bias for variance. Formally, we minimize $\mathcal{L}_{\text{total}}(\boldsymbol{\theta}) = \mathcal{L}_{\text{data}}(\boldsymbol{\theta}) + \lambda\,\Omega(\boldsymbol{\theta})$ where Ω is a complexity penalty and $\lambda > 0$ controls the regularization strength.

3.1 Parameter Norm Penalties

These directly constrain the magnitude of model weights $\mathbf{w}$ via an additive penalty in the objective.

3.1.1 L2 Regularization (Ridge / Weight Decay)

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \frac{\lambda}{2}\|\mathbf{w}\|_2^2 = \mathcal{L}_{\text{data}} + \frac{\lambda}{2}\sum_i w_i^2$$

The gradient update becomes $\mathbf{w} \leftarrow \mathbf{w} - \eta(\nabla_{\mathbf{w}}\mathcal{L}_{\text{data}} + \lambda\mathbf{w})$, which shrinks all weights toward zero by a factor $(1 - \eta\lambda)$ each step.

Default weight regularizer. L2 is the standard regularizer in neural networks, SVMs, and logistic regression. In PyTorch, it is implemented as `torch.optim.SGD(params, weight_decay=λ)`. Note that for Adam and other adaptive optimizers, decoupled weight decay (AdamW) is necessary because the standard L2 regularization interacts incorrectly with per-parameter adaptive learning rates.

Pros:

- Differentiable everywhere — smooth optimization with no subgradient hacks.
- Shrinks all weights, preventing any single weight from exploding and dominating the output.
- Handles correlated features gracefully: L2 distributes weight mass across correlated predictors rather than arbitrarily selecting one.
- Equivalent to a Gaussian prior $\mathbf{w} \sim \mathcal{N}(0, 1/\lambda)$ in the Bayesian interpretation.

Cons:

- No feature selection — all weights remain non-zero, leading to dense models.
- Uniformly shrinks all weights regardless of their importance.

- At very high λ, underfitting occurs.

L2 vs. Decoupled Weight Decay in Adam. In SGD, L2 regularization and weight decay are mathematically equivalent. In Adam, they are *not*. Adam's per-parameter learning rates cause L2 regularization to shrink weights with small gradients less than those with large gradients. AdamW (`torch.optim.AdamW`) implements decoupled weight decay: $\mathbf{w} \leftarrow \mathbf{w} - \eta\lambda\mathbf{w}$ separately from the adaptive gradient step, which consistently yields better generalization.

3.1.2 L1 Regularization (Lasso)

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda\|\mathbf{w}\|_1 = \mathcal{L}_{\text{data}} + \lambda \sum_i |w_i|$$

The gradient is $\text{sign}(w_i)$ for $w_i \neq 0$, with the subgradient $[-1, 1]$ at $w_i = 0$. This pushes weights exactly to zero, inducing sparsity.

Feature selection and interpretable models. L1 is the go-to regularizer when model interpretability matters (e.g., genomics, econometrics) or when the input dimensionality is high but only a few features are predictive. It produces sparse weight vectors where many entries are exactly zero. `torch.optim.SGD(params, lr=lr)` with manual L1 penalty addition.

Pros:

- **Induces sparsity:** exact zeros mean built-in feature selection. The model becomes smaller, faster, and more interpretable.
- Equivalent to a Laplace prior $p(w_i) = \frac{\lambda}{2}e^{-\lambda|w_i|}$, which is sparsity-inducing.
- Works well when the true model is sparse (most features are irrelevant).

Cons:

- Non-differentiable at $w_i = 0$; requires subgradient methods or proximal gradient descent (ISTA/FISTA).
- Can arbitrarily select one feature from a group of highly correlated predictors, discarding others that might be useful.
- Optimization is harder than L2: the non-smooth objective can cause oscillation.
- In deep learning, L1 is less common than L2 because exact sparsity is rarely needed and subgradient noise can be harmful.

3.1.3 Elastic Net

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda_1 \|\mathbf{w}\|_1 + \frac{\lambda_2}{2} \|\mathbf{w}\|_2^2$$

Combines L1 sparsity with L2 smoothness. The L2 term stabilizes the L1 selection when features are correlated — instead of arbitrarily picking one, the elastic net tends to select entire groups.

When both sparsity and stability are needed. Elastic Net is popular in bioinformatics (microarray data), text classification with TF-IDF features, and any setting with more features than samples ($p \gg n$) where correlated feature groups exist. Scikit-learn's `ElasticNet` and `SGDClassifier` with `penalty='elasticnet'`.

Pros:

- Combines L1's sparsity with L2's grouping effect.
- Handles correlated features more gracefully than pure L1.
- Two hyperparameters allow fine control over the sparsity-smoothness trade-off.

Cons:

- Two hyperparameters (λ_1, λ_2) instead of one.
- More complex optimization than either L1 or L2 alone.

3.1.4 Group Lasso

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda \sum_{g=1}^{G} \|\mathbf{w}_g\|_2$$

where $\mathbf{w}_g$ is the weight vector for group g. The L_2-norm of each group is penalized, which tends to zero out entire groups at once. This is the $L_{2,1}$ mixed norm.

Structured sparsity. Use Group Lasso when features are naturally partitioned — e.g., one-hot encoded categorical variables (all dummy variables for one category form a group), multichannel EEG signals, or genomic pathways. Also used for pruning entire neurons, channels, or layers in network compression.

Pros:

- Structured sparsity: entire feature groups are selected or discarded together.
- Interpretable: group-level feature importance.
- Can be extended to overlapping groups via the Latent Group Lasso.

Cons:

- Requires predefining feature groups, which is not always natural.

- The L_2 norm is not differentiable at zero vector; proximal optimization needed.

3.2 Dropout Methods

Dropout is a stochastic regularization technique that randomly removes units during training. At test time, all units are present, but their outputs are scaled down to match the expected value from training.

3.2.1 Standard Dropout

During training, each unit's output h is multiplied by an independent mask:

$$h' = h \odot \mathbf{m}, \quad m_i \sim \text{Bernoulli}(1-p)$$

where p is the dropout rate. At test time, the output is scaled: $h' = (1-p) \cdot h$. Equivalently, some implementations apply **inverted dropout**: scale by $1/(1-p)$ during training and leave test unchanged.

Fully connected layers. Standard dropout is highly effective in MLPs and the final classifier layers of CNNs. Typical settings: $p = 0.5$ for hidden layers, $p = 0.2$ for input layers. `torch.nn.Dropout(p)`.

Pros:

- Extremely simple to implement and effective — one of the most impactful regularization innovations.
- Interpreted as training an exponential ensemble of 2^n thinned sub-networks that share weights. At test time, this approximates model averaging.
- Prevents co-adaptation: neurons cannot rely on the presence of specific other neurons, forcing each to learn robust features.
- MC Dropout: applying dropout at *test* time and running multiple forward passes provides approximate Bayesian uncertainty estimates.
- No extra learnable parameters.

Cons:

- Increases training time: the effective ensemble means each weight update sees a different architecture, requiring more iterations.
- Less effective in convolutional layers (where adjacent pixels are highly correlated) — spatial dropout is preferred.
- Increases variance in gradient estimates.
- At test time, the $(1-p)$ scaling can interfere with Batch Normalization if not handled carefully.

3.2.2 Spatial Dropout (Dropout2D)

Instead of dropping individual activations, Spatial Dropout drops entire feature-map channels:

$$h'_{c,h,w} = h_{c,h,w} \cdot m_c, \quad m_c \sim \text{Bernoulli}(1-p)$$

The mask is identical across all spatial positions (h, w) within a channel.

Convolutional layers. Spatial Dropout is preferred over standard Dropout after convolutional layers because dropping individual pixels in a feature map is ineffective (neighbouring pixels carry similar information). Dropping an entire channel removes a learned feature detector entirely, forcing other detectors to compensate. `torch.nn.Dropout2d(p)`.

Pros:

- More effective in CNNs than standard Dropout.
- Drops entire feature maps, encouraging diverse feature detection.
- Respects the spatial structure of convolutional feature maps.

Cons:

- Still less effective than Batch Normalization + data augmentation for many CNN tasks.
- Can be too aggressive — dropping a critical feature map can harm performance.

3.2.3 DropConnect

Instead of dropping activations, DropConnect randomly zeroes individual *weights* (connections) in the weight matrix:

$$W' = W \odot M, \quad M_{ij} \sim \text{Bernoulli}(1-p)$$

Each training example sees a different sparse connectivity pattern.

Fine-grained weight regularization. DropConnect provides more granular regularization than Dropout. It is a generalization: Dropout drops activations (entire columns of the next weight matrix); DropConnect drops individual connections. Historically used on MNIST/CIFAR benchmarks.

Pros: More fine-grained than Dropout. Works as a general weight noise regularizer. **Cons:** More expensive to implement (mask per weight matrix). Less common in practice; standard Dropout is usually sufficient and simpler.

3.2.4 Stochastic Depth (DropPath)

For a residual network, randomly drop entire residual blocks during training:

$$H_l = \text{ReLU}\big(b_l \cdot \mathcal{F}_l(H_{l-1}) + H_{l-1}\big), \quad b_l \sim \text{Bernoulli}(p_l)$$

where $\mathcal{F}_l$ is the residual function of layer l. When $b_l = 0$, the block is skipped entirely: $H_l = \text{ReLU}(H_{l-1})$. The survival probability p_l can be constant or linearly decayed with depth.

Very deep residual networks and Vision Transformers. DropPath was introduced for training 1202-layer ResNets on CIFAR. It is now standard in ViT (Vision Transformer) and ConvNeXt training, where it drops entire transformer blocks or residual paths to prevent overfitting of deep architectures.

Pros:

- Reduces the effective depth during training, creating an implicit ensemble of networks at various depths.
- Enables training extremely deep networks (1000+ layers).
- Works synergistically with residual connections: shallow paths are always available for gradient flow.

Cons:

- The survival probability schedule needs tuning. Common schedules: $p_l = 1 - \frac{l}{L}(1 - p_L)$ (linear decay), where $p_L = 0.5$.
- Aggressive dropping can make training too easy (shallow networks have lower capacity) and hurt final performance.
- Implementation requires careful handling of identity connections.

3.2.5 Shake-Shake and ShakeDrop

Shake-Shake (for multi-branch ResNets): perturbs the forward and backward passes of the residual branches with different random coefficients. For a two-branch residual block:

$$H_l = H_{l-1} + \alpha F_1(H_{l-1}) + (1-\alpha)F_2(H_{l-1}) \quad \text{(forward)}$$
$$\frac{\partial \mathcal{L}}{\partial H_{l-1}} = \frac{\partial \mathcal{L}}{\partial H_l} + \beta \frac{\partial \mathcal{L}}{\partial H_l}\frac{\partial F_1}{\partial H_{l-1}} + (1-\beta)\frac{\partial \mathcal{L}}{\partial H_l}\frac{\partial F_2}{\partial H_{l-1}} \quad \text{(backward)}$$

where $\alpha, \beta \sim \text{Uniform}(-1, 1)$ or $\text{Uniform}(0, 1)$.
ShakeDrop: perturbs only the residual branch (not the identity): $H_l = H_{l-1} + (b_l + \alpha - \beta)\mathcal{F}_l(H_{l-1})$ where $b_l \sim \text{Bernoulli}(p_l)$.

Very deep ResNets with multi-branch architectures. Shake-Shake regularization achieved state-of-the-art on CIFAR with 26 2×96d ResNeXt. ShakeDrop extends the idea

to single-branch ResNets. Both are specialized regularization methods for deep residual architectures.

Pros: Strong regularization for deep ResNets. The perturbed forward and backward passes act as data-dependent noise injection. **Cons:** Complex to implement. Only applicable to residual architectures with specific branching patterns. Largely superseded by simpler augmentation methods (CutMix, Mixup) in modern training recipes.

3.3 Data Augmentation as Regularization

Data augmentation artificially expands the training set by applying label-preserving transformations. It is one of the most effective and widely used regularization strategies because it directly encodes prior knowledge about invariances.

3.3.1 Mixup

Given two training examples (x_i, y_i) and (x_j, y_j), Mixup creates a virtual example via convex combination:

$$\tilde{x} = \lambda x_i + (1-\lambda)x_j, \qquad \tilde{y} = \lambda y_i + (1-\lambda)y_j$$

where $\lambda \sim \text{Beta}(\alpha, \alpha)$ with $\alpha \in (0, \infty)$. Small α (e.g., 0.2) produces λ near 0 or 1 (mild mix); $\alpha = 1$ is uniform.

Image classification and NLP. Mixup is a standard component in modern training recipes (ImageNet, CIFAR, speech). It has been extended to NLP (word-level and sentence-level mixup), graph learning (Manifold Mixup), and reinforcement learning.

Pros:

- Encourages the model to behave linearly between training examples, producing smoother decision boundaries.
- Improves robustness to adversarial examples and noisy labels.
- Reduces overconfidence: the model learns that interpolated inputs correspond to interpolated labels, calibrating predictions.
- Simple to implement (a few lines of code).

Cons:

- The mixed labels may not be semantically meaningful: mixing a cat and a dog image does not produce a "half-cat, half-dog" label in any real sense, yet the model is forced to predict it. This can hurt when α is too large.
- Slower convergence: the model is learning from "unreal" data that doesn't cleanly point to one class.
- Requires labels to be one-hot or soft vectors (not compatible with all label formats).

3.3.2 CutMix

CutMix replaces a rectangular patch of image A with the corresponding patch from image B. The label is mixed proportionally to the area:

$$\tilde{x} = \mathbf{M} \odot x_A + (\mathbf{1} - \mathbf{M}) \odot x_B, \qquad \tilde{y} = \lambda y_A + (1 - \lambda) y_B$$

where $\mathbf{M} \in \{0,1\}^{H \times W}$ is a binary mask indicating which region comes from x_A, and λ is the area ratio $\frac{|\mathbf{M}|}{HW}$ sampled from $\text{Beta}(\alpha, \alpha)$.

Image classification with spatial mixing. CutMix is used alongside or instead of Mixup in modern ImageNet training. It provides a non-linear spatial mixing that encourages the model to attend to the full object rather than just the most discriminative region.

Pros:

- Non-linear spatial mix is more natural than the global pixel-wise interpolation of Mixup — objects remain recognizable.
- Improves localization and attention coverage: the model cannot rely on a single discriminative patch.
- The area-proportional label weighting is interpretable.

Cons:

- The randomly placed patch can occasionally cover the entire object, creating a misleading training signal.
- Introduces boundary artifacts at the cut edge.
- Two hyperparameters: α for the Beta distribution and the sampling strategy for the bounding box coordinates.

3.3.3 Cutout / Random Erasing

Cutout: Set a randomly placed square region of the input image to zero (or the dataset mean). **Random Erasing:** similar but also varies the aspect ratio and can fill with random noise.

$$\tilde{x}_{i,j} = \begin{cases} 0 & \text{if } (i,j) \text{ in mask region} \\ x_{i,j} & \text{otherwise} \end{cases}$$

CNNs for image classification. Cutout is simple and effective, especially for smaller datasets (CIFAR, SVHN). Random Erasing is used in person re-identification to simulate occlusion and improve robustness.

Pros:

- Extremely simple — just zero out a region.
- Improves robustness to occlusion and missing data.
- No label mixing, so labels remain clean.

Cons:

- Loses information: the occluded region is completely erased, which can remove crucial features.
- Less effective than CutMix/Mixup for large-scale datasets.

3.3.4 Label Smoothing

$$\tilde{y}_c = y_c(1-\epsilon) + \frac{\epsilon}{C}$$

Replaces the hard one-hot targets with soft targets, preventing the model from assigning full probability to any single class. (See also Chapter 1, Section 1.2.8.)

Large-scale classification. $\epsilon = 0.1$ is standard in ImageNet training. Used in BERT pretraining, ViT, and most modern classification pipelines. Label smoothing is a form of output regularization that complements input augmentation.

Pros:

- Prevents overconfident predictions; improves calibration.
- Acts as a regularizer that tightens feature clusters.
- Simple, no architectural changes.

Cons:

- Reduces training accuracy (by design).
- May hurt knowledge distillation (smoothed targets lose teacher's dark knowledge).
- ϵ requires tuning.

3.4 Implicit, Structural, and Advanced Regularization

3.4.1 Early Stopping

Training is halted when the validation loss stops improving for a pre-specified number of epochs (`patience`). The model with the best validation performance is restored. In the linear case, early stopping is equivalent to L2 regularization with an effective $\lambda \propto 1/(\eta t)$.

Universal regularizer. Early stopping is used in virtually every deep learning training loop. It requires no model modifications, no extra hyperparameters beyond patience, and is often the single most important regularization technique in practice.

Pros:

- Zero architectural or algorithmic overhead.
- Automatically determines the right amount of training.
- Equivalent to an L2 penalty for linear models.
- Works synergistically with other regularizers.

Cons:

- Requires a held-out validation set, reducing training data.
- The training loss is not fully minimized — some capacity is left unused.
- Requires storing the best model checkpoint.
- "Patience" is a hyperparameter; too short stops prematurely, too long wastes compute.

3.4.2 Batch Normalization as Regularizer

Batch Normalization (see Chapter 4) inherently acts as a regularizer. The mean and variance estimates computed on each mini-batch are noisy, which injects stochasticity into activations. This noise prevents overfitting and often reduces or eliminates the need for Dropout. BN's regularization effect is strongest with small batch sizes (more noise) and weakest with large batches.

3.4.3 Spectral Normalization

$$\bar{W} = \frac{W}{\sigma(W)}$$

where $\sigma(W)$ is the **spectral norm** (largest singular value) of the weight matrix. After normalization, $\|\bar{W}\|_2 = 1$, making the layer 1-Lipschitz with respect to the L_2 norm. The spectral norm is estimated efficiently via power iteration: $\mathbf{u} \leftarrow W^\top \mathbf{v}/\|W^\top \mathbf{v}\|_2$, $\mathbf{v} \leftarrow W\mathbf{u}/\|W\mathbf{u}\|_2$, yielding $\sigma(W) \approx \mathbf{u}^\top W \mathbf{v}$.

GAN discriminators. Spectral Normalization (Miyato et al., 2018) is the standard regularization for GAN discriminators, enforcing the 1-Lipschitz constraint required by Wasserstein GAN training. It can be applied to every layer of the discriminator with minimal computational overhead (one power iteration per forward pass).

Pros:

- Enforces a global Lipschitz constraint, stabilizing GAN training.
- Computationally cheap: power iteration adds negligible cost.
- Can be applied to any weight matrix.
- Improves generalization when applied to classifiers as well.

Cons:

- Only constrains the largest singular value; other singular values are unrestricted.
- The Lipschitz constant of the entire network may still exceed 1 if the non-linearity is not 1-Lipschitz (ReLU is).
- Can be too restrictive, limiting model expressiveness.

3.4.4 Sharpness-Aware Minimization (SAM)

SAM seeks parameters $\mathbf{w}$ that lie in neighborhoods of uniformly low loss, i.e., flat minima:

$$\min_{\mathbf{w}} \max_{\|\boldsymbol{\epsilon}\|_2 \leq \rho} \mathcal{L}(\mathbf{w} + \boldsymbol{\epsilon})$$

The inner maximization is approximated by a first-order Taylor expansion, yielding $\boldsymbol{\epsilon}^* = \rho \cdot \nabla\mathcal{L}(\mathbf{w})/\|\nabla\mathcal{L}(\mathbf{w})\|_2$. The gradient at the perturbed point is used for the update.

Improving generalization across tasks. SAM has been shown to improve generalization in image classification (ResNet, ViT), NLP fine-tuning, and various other domains. It consistently boosts performance across architectures by seeking flatter minima.

Pros:

- Improved generalization via flatter minima (well-connected to generalization theory).
- Architecture-agnostic: can be plugged into any training pipeline.
- Consistent improvements across benchmarks.

Cons:

- Doubles compute: each step requires two forward-backward passes (one for the perturbation, one for the update).
- The radius ρ is a sensitive hyperparameter. Typical $\rho \in [0.01, 0.2]$ for ImageNet.
- The first-order approximation and single perturbation step are crude.

3.4.5 Gradient Penalty (WGAN-GP)

$$\mathcal{L}_{\text{GP}} = \lambda \, \mathbb{E}_{\hat{\mathbf{x}} \sim \mathbb{P}_{\hat{\mathbf{x}}}} \Big[(\|\nabla_{\hat{\mathbf{x}}} D(\hat{\mathbf{x}})\|_2 - 1)^2 \Big]$$

where $\hat{\mathbf{x}} = \epsilon \mathbf{x} + (1 - \epsilon) G(\mathbf{z})$ with $\epsilon \sim \text{Uniform}(0, 1)$ is a random interpolation between real and generated samples. This softly enforces the 1-Lipschitz constraint required for the Wasserstein distance.

WGAN-GP discriminator regularization. Gradient penalty is the standard way to enforce the Lipschitz constraint in Wasserstein GANs, replacing the earlier weight-clipping approach. $\lambda = 10$ is a common default.

Pros:

- Provides a soft, differentiable Lipschitz constraint.
- More stable than weight clipping and produces better sample quality.
- The gradient norm penalty correlates with sample quality.

Cons:

- Computationally expensive: requires computing gradients of gradients (second derivatives) for each interpolated sample.
- The penalty is applied only at random interpolations, not the full input space.
- $\lambda = 10$ is an empirical default, not theoretically motivated for all settings.

3.5 Knowledge Distillation

A small **student** model is trained to mimic a larger pre-trained **teacher** model:

$$\mathcal{L} = (1 - \alpha) \, \mathcal{L}_{\text{CE}}(y, \sigma(z_s / T)) \; + \; \alpha \, T^2 \cdot \mathcal{L}_{\text{KL}}\big(\sigma(z_t / T), \, \sigma(z_s / T)\big)$$

where $T > 0$ is the temperature that softens the output distributions, α balances hard-label and soft-target supervision, and the T^2 factor compensates for gradient scaling due to temperature. High T (e.g., 3–20) reveals "dark knowledge" — relative class similarities encoded in the teacher's softmax.

Model compression and improved accuracy. Distillation is used to compress large models (BERT → DistilBERT, ResNet-152 → ResNet-18 student) while retaining much of the accuracy. It also improves small models beyond what training on hard labels alone achieves, because soft targets encode class relationships (e.g., a truck is more similar to a car than to a cat).

Pros:

- Transfers the "dark knowledge" of the teacher — relative class similarities, uncer-

tainty, and confidence calibration.

- Improves small models beyond standard training.
- Student can have a completely different architecture from the teacher.
- Works across modalities (e.g., vision teacher → audio student).

Cons:

- Requires a pre-trained teacher model, which may be expensive to obtain.
- Two hyperparameters (T, α) to tune. Common: $T \in [2, 20]$, $\alpha \in [0.7, 0.9]$.
- If the teacher is highly confident (low entropy), the soft targets are nearly one-hot and distillation provides little benefit — label smoothing on the teacher helps.

3.6 Additional Regularization Techniques

3.6.1 Virtual Adversarial Training (VAT)

VAT regularizes by enforcing local distributional smoothness: the model's output should be invariant to the most adversarial local perturbation:

$$\mathcal{L}_{\text{VAT}} = \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\Big[\text{KL}\big(p(y|\mathbf{x}) \,\big\|\, p(y|\mathbf{x}+\mathbf{r}_{\text{adv}})\big)\Big]$$

where $\mathbf{r}_{\text{adv}}$ is the perturbation that maximizes the KL divergence, computed via a power iteration approximation.

Semi-supervised learning. VAT is effective when unlabeled data is abundant. It encourages the decision boundary to lie in low-density regions of the input manifold without requiring label information for the perturbation.

Pros:

- Leverages unlabeled data effectively.
- Does not require ground-truth labels for the adversarial direction.
- Local smoothness is a broadly useful inductive bias.

Cons:

- Computationally heavy: requires an inner optimization loop per batch.
- The power iteration approximation can be inaccurate.
- Hyperparameters: perturbation radius ϵ and number of power iterations.

3.6.2 Entropy Regularization / Confidence Penalty

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \beta\, \mathcal{H}[p(y|\mathbf{x})]$$

where $\mathcal{H}[p] = -\sum_c p_c \log p_c$ is the Shannon entropy of the output distribution. Penalizing high entropy encourages confident predictions (the opposite of label smoothing, which encourages smoothness). A **confidence penalty** penalizes *low* entropy: $\mathcal{L} = -\beta\mathcal{H}[p]$ (minus entropy), discouraging overconfidence.

Preventing degenerate outputs. Entropy maximization is used in self-supervised learning and clustering to prevent collapse (all samples assigned to one class). Confidence penalty (negative entropy) discourages peaky, overconfident predictions.

Pros: Simple scalar penalty. Encourages or discourages output uncertainty as needed. **Cons:** The sign of β matters: positive = prefer uniform (high entropy), negative = prefer sharp (low entropy). Needs careful tuning.

3.6.3 Orthogonal Regularization

$$\Omega(W) = \left\|W^\top W - I\right\|_F^2$$

Penalizes deviation from orthonormality in the weight matrix. Orthogonal weights preserve gradient norm during backpropagation (no exploding or vanishing through the linear transform itself) and decorrelate features.

Stable training and feature decorrelation. Orthogonal regularization is used in GAN generators (BigGAN), RNNs (to prevent vanishing/exploding gradients), and neural ODEs. It is closely related to weight normalization and spectral normalization.

Pros:

- Preserves gradient norm during backpropagation (if activations are approximately norm-preserving, e.g., ReLU).
- Decorrelates features, reducing redundancy.
- Can improve GAN stability.

Cons:

- Only applicable to square or weight-tall matrices.
- The Frobenius norm penalty is not exactly a projection onto the Stiefel manifold; proper orthogonal optimization requires geodesic methods.
- Can over-regularize: strict orthogonality prevents learning statistical dependencies that are present in the data.

3.6.4 Jacobian and Hessian Regularization

Jacobian regularization:

$$\Omega = \mathbb{E}_{\mathbf{x}}\left[\left\|\nabla_{\mathbf{x}} f(\mathbf{x})\right\|_F^2\right]$$

Penalizes large input gradients, encouraging the function to be smooth and contractive. This makes the model robust to small input perturbations.
Hessian regularization penalizes curvature: $\Omega = \|\nabla_{\mathbf{x}}^2 f(\mathbf{x})\|$.

Robustness and stability. Jacobian regularization is used in contractive autoencoders and for adversarial robustness. It smooths the function landscape, reducing sensitivity to input noise.

Pros:

- Directly encourages local smoothness and adversarial robustness.
- Theoretically grounded connection to generalization (flat minima).

Cons:

- Computationally expensive: requires computing Hessian-vector products or double backpropagation.
- Approximation methods (finite differences, power iteration) introduce noise.
- Can over-smooth the function, hurting expressiveness.

3.7 Summary and Practical Guidance

Regularization checklist for deep learning:

1. **Always** use early stopping based on validation loss.
2. Use **AdamW** with $\lambda = 0.01$–0.1 for most tasks.
3. Apply **Dropout** ($p = 0.1$–0.5) in fully connected layers. Use Spatial Dropout for CNNs, DropPath for ViTs.
4. **Batch/Layer Normalization** provides implicit regularization.
5. **Data augmentation:** Mixup ($\alpha = 0.2$) and/or CutMix ($\alpha = 1.0$) are standard for image classification.
6. **Label smoothing** ($\epsilon = 0.1$) for large-scale classification.
7. For GANs: **Spectral Normalization** in the discriminator.
8. For model compression: **Knowledge Distillation**.
9. For extra generalization: **SAM** (if 2x compute is acceptable).

The most effective regularizers are data augmentation and early stopping. Parametric penalties (L1, L2) alone are rarely sufficient for deep networks.

Chapter 4

Normalization Methods

Normalization methods transform the activations of neural network layers to have consistent statistical properties (typically zero mean and unit variance). They stabilize training, enable higher learning rates, reduce sensitivity to initialization, and often provide a regularizing effect.

A **normalization layer** computes $\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$, followed by an affine transformation $y_i = \gamma \hat{x}_i + \beta$, where μ and σ^2 are statistics computed over a specified set of dimensions, and γ, β are learnable scale and shift parameters. The choice of which dimensions to normalize over distinguishes the different methods.

4.1 Batch Normalization (BN)

For a mini-batch of size m, over which statistics are computed:

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i \qquad \text{(batch mean, per-channel for CNNs)}$$

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_B)^2 \qquad \text{(batch variance)}$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \qquad \text{(normalize)}$$

$$y_i = \gamma \hat{x}_i + \beta \qquad \text{(scale and shift)}$$

During **training**: use batch statistics μ_B, σ_B^2. During **inference**: use running averages $\mu_{\text{running}}, \sigma^2_{\text{running}}$ accumulated via exponential moving average $\mu_{\text{running}} \leftarrow \rho\, \mu_{\text{running}} + (1 - \rho)\mu_B$ (momentum ρ, typically 0.9 or 0.99).

Convolutional neural networks and feed-forward networks. BN is the default normalization for ResNet, VGG, DenseNet, Inception, and most CNN architectures (pre-2020). It enabled the training of very deep networks by mitigating internal covariate shift. In PyTorch: `torch.nn.BatchNorm1d`, `BatchNorm2d`, `BatchNorm3d`.

Pros:

- Enables much higher learning rates (10–30× faster convergence) by reducing the scale sensitivity of gradients.
- Reduces sensitivity to weight initialization — the network is more robust to different initialization schemes.
- Acts as a regularizer: the noise in μ_B, σ_B^2 from small batches prevents overfitting. BN often reduces or eliminates the need for Dropout.

- Smooths the optimization landscape: BN was shown by Santurkar et al. (2018) to make the loss surface significantly smoother, not just reduce internal covariate shift.
- Allows saturating non-linearities (e.g., sigmoid in LSTMs) to be used more effectively by keeping pre-activations in their non-saturating regime.

Cons:

- **Small batch sizes:** μ_B and σ_B^2 become noisy estimates, degrading performance. BN typically fails below batch size 8. Batch size of 16+ is recommended.
- **Train/test discrepancy:** the use of batch statistics at training vs. running averages at test creates a mismatch that must be calibrated (the running average momentum must be carefully chosen).
- **Problematic in RNNs:** sharing statistics across time steps requires per-timestep statistics and separate running averages, making implementation complex. LayerNorm or RMSNorm is universally preferred for sequential models.
- **Dependency on batch composition:** the statistics depend on the specific samples in the batch, introducing non-determinism that complicates debugging and exact reproducibility.
- **Incompatible with some settings:** online learning (batch size 1), contrastive learning with large memory banks, and some differentially private training schemes cannot use BN effectively.

4.1.1 Synchronized Batch Normalization

When training across multiple GPUs, standard BN computes statistics only within each GPU's local batch. **SyncBN** synchronizes statistics across *all* GPUs via an all-reduce operation:

$$\mu_{\text{sync}} = \frac{1}{\sum_g m_g} \sum_{g=1}^{G} m_g \mu_g, \qquad \sigma^2_{\text{sync}} = \frac{1}{\sum_g m_g} \sum_{g=1}^{G} m_g \sigma_g^2$$

This effectively increases the batch size used for normalization to $m_{\text{total}} = \sum_g m_g$.

Small per-GPU batch sizes in distributed training. Object detection and segmentation models often require large input resolutions, forcing small per-GPU batch sizes. SyncBN restores BN's effectiveness by pooling statistics across GPUs. `torch.nn.SyncBatchNorm`.

Pros:

- Restores BN accuracy at small per-GPU batch sizes.
- Same semantics as single-GPU BN with effective batch size = total batch.

Cons:

- Communication overhead: all-reduce at every BN layer adds latency.

- Implementation complexity: must handle multi-GPU synchronization correctly.
- Still requires reasonably sized effective batch size.

4.2 Layer Normalization (LN)

Normalizes across the *feature* dimension (not the batch dimension) for each sample independently:

$$\mu_i = \frac{1}{H}\sum_{h=1}^{H} x_{i,h} \qquad \text{(mean over features)}$$

$$\sigma_i^2 = \frac{1}{H}\sum_{h=1}^{H}(x_{i,h} - \mu_i)^2 \qquad \text{(variance over features)}$$

$$\hat{x}_{i,h} = \frac{x_{i,h} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}$$

$$y_{i,h} = \gamma_h \hat{x}_{i,h} + \beta_h \qquad \text{(per-feature scale/shift)}$$

The statistics are computed per sample i, across all feature dimensions $h = 1, \ldots, H$. LN is independent of batch size and behaves identically at training and test time.

Transformers and sequential models. LN is the standard normalization for all transformer architectures (BERT, GPT, T5, ViT), where it is applied before or after the multi-head attention and feed-forward sublayers (Pre-LN vs. Post-LN). It is also used in RNNs and LSTMs as a drop-in replacement for BatchNorm where per-timestep normalization across features is needed. `torch.nn.LayerNorm(normalized_shape)`.

Pros:

- **Batch-size independent:** works with batch size 1 and does not suffer from small-batch noise. Train and test behavior are identical — no running averages needed.
- Naturally suited for **sequence models**: each time step and each sample is normalized independently, so sequence length and batch composition don't affect statistics.
- Works across variable-length sequences without special handling.
- Per-sample normalization makes the model naturally invariant to per-sample scaling of features (e.g., word embedding magnitudes).

Cons:

- **Less effective for CNNs:** normalizing across feature channels breaks the spatial structure of convolutional feature maps. Convolutional features are semantically different across channels, so normalizing them together is less meaningful than normalizing each channel separately (as BN does).
- For very high-dimensional features, computing mean/variance over all dimensions can be noisy.

- Unlike BN, LN does not inject noise during training, providing less implicit regularization.

4.3 Instance Normalization (IN)

Normalizes each sample *and* each channel independently across spatial dimensions. For a 2D feature map:

$$\mu_{n,c} = \frac{1}{HW}\sum_{h=1}^{H}\sum_{w=1}^{W} x_{n,c,h,w}$$

$$\sigma^2_{n,c} = \frac{1}{HW}\sum_{h=1}^{H}\sum_{w=1}^{W}(x_{n,c,h,w} - \mu_{n,c})^2$$

$$\hat{x}_{n,c,h,w} = \frac{x_{n,c,h,w} - \mu_{n,c}}{\sqrt{\sigma^2_{n,c} + \epsilon}}$$

$$y_{n,c,h,w} = \gamma_c \hat{x}_{n,c,h,w} + \beta_c$$

Each sample-channel pair gets its own mean and variance. IN can be seen as performing contrast normalization on each feature map.

Style transfer and image-to-image translation. IN is the key normalization in style transfer networks (AdaIN, Pix2Pix, CycleGAN) because it removes instance-specific contrast information (mean and variance), which encodes "style" (color, texture, lighting). Normalizing this away and then re-applying desired style statistics enables content-style disentanglement. `torch.nn.InstanceNorm2d`.

Pros:

- Removes instance-specific contrast (mean/variance), which is precisely what's needed for style transfer: content is preserved in normalized feature activations; style is encoded in channel-wise statistics.
- Independent of batch size; consistent train/test behavior.
- Simple and effective for image generation tasks.

Cons:

- **Hurts discriminative tasks:** for classification or detection, contrast information (absolute brightness, mean activation level) is often useful. IN discards this, reducing accuracy.
- Does not normalize across channels, so inter-channel relationships (which are important for feature encoding) are not regularized.
- Not generally applicable beyond image generation and style transfer.

4.4 Group Normalization (GN)

GN is a middle ground between LN and IN. The C feature channels are divided into G groups (typically $G = 32$ or $G = 16$). Statistics are computed per sample, per group, across spatial dimensions:

$$\mu_{n,g} = \frac{1}{(C/G)HW} \sum_{c \in \text{group}_g} \sum_{h,w} x_{n,c,h,w}$$

$$\sigma^2_{n,g} = \frac{1}{(C/G)HW} \sum_{c \in \text{group}_g} \sum_{h,w} (x_{n,c,h,w} - \mu_{n,g})^2$$

GN is independent of batch size and works with $m = 2$ or even $m = 1$. When $G = 1$, GN equals LN. When $G = C$, GN equals IN.

Small-batch training in computer vision. GN is the go-to normalization for object detection (Mask R-CNN, Faster R-CNN), video classification, and segmentation when GPU memory constraints force small batch sizes (e.g., batch size 2 with high-resolution inputs). It performs comparably to BN at batch size 8–16 and outperforms BN at batch size 2–4. `torch.nn.GroupNorm(num_groups, num_channels)`.

Pros:

- **Batch-size independent:** works excellently with batch sizes as small as 2, where BN completely fails.
- Identical train/test behavior — no running averages, no batch composition dependence.
- More expressive than LN (normalizes within groups, not across all channels) and more robust than IN (preserves some cross-channel structure).
- The number of groups G provides a tunable knob between LN and IN behavior.

Cons:

- The number of groups G is a hyperparameter. Common defaults ($G = 32$) work well for many settings but may not be optimal.
- Slightly worse than BN at large batch sizes (32+) on standard image classification benchmarks.
- For very few channels (e.g., 3-channel RGB input), there are too few channels to form meaningful groups.

4.5 LayerNorm Variants and Efficient Normalizations

4.5.1 RMS Normalization (RMSNorm)

$$y_i = \frac{x_i}{\text{RMS}(\mathbf{x})} \cdot \gamma_i, \qquad \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2}$$

RMSNorm normalizes only by the root-mean-square magnitude — there is *no mean subtraction*. The re-centering step (mean subtraction) of LN is omitted, which reduces computation. The learnable parameters are only γ (scale); there is typically no β (shift), though some implementations include it optionally.

Large Language Models. RMSNorm is used in LLaMA, LLaMA 2, Mistral, Gemma, and other modern LLMs. It provides similar training stability to LN while being noticeably faster — dropping the mean computation saves roughly 10–15% of the normalization wall-clock time in large models. This matters significantly when training on trillions of tokens.

Pros:

- **Faster than LayerNorm:** one fewer reduction operation per normalization (no mean subtraction).
- Same training stability as LayerNorm for transformers.
- Simpler: only RMS-based regularization.
- Empirically validated at massive scale (LLaMA 70B, Mistral 7B, etc.).

Cons:

- Slightly less expressive than full LN (no re-centering). The mean may drift over layers.
- May converge to a slightly different optimum than LN with the same initialization.
- Not standard in all frameworks yet (though implementations exist and are simple to write).

4.5.2 DeepNorm

DeepNorm modifies the residual connection to stabilize very deep transformers (up to 1000 layers):

$$\mathbf{x}_{l+1} = \text{LN}\big(\alpha \mathbf{x}_l + \text{Sublayer}(\mathbf{x}_l)\big)$$

where α is a constant weight on the residual connection (typically $\alpha > 1$ to amplify the skip path) and the Sublayer parameters are initialized with smaller variance. Post-LN transformer with DeepNorm uses specific initialization scaling for each attention and FFN weight.

Extremely deep transformers. DeepNorm was introduced to train 1000-layer transformers stably (DeepNet). It is used in research settings exploring the limits of transformer depth. The key insight is that amplifying the residual path and down-scaling the sublayer

initialization keeps the model closer to identity at initialization, ensuring stable gradient flow.

Pros:

- Enables training of transformers with 100–1000+ layers, far beyond what standard Post-LN or Pre-LN can handle.
- Stable at initialization and throughout training.
- Builds on standard LN + residual framework — no exotic components.

Cons:

- Requires specific initialization formulas derived per-architecture.
- The residual scaling α is architecture-specific.
- For most practical tasks, 12–96 layer transformers are sufficient; DeepNorm is specialized for research on depth scaling.

4.6 Weight Normalization (WN)

Weight Normalization reparameterizes each weight vector $\mathbf{w}$ into a magnitude g (scalar) and a direction $\mathbf{v}$ (unit-norm vector):

$$\mathbf{w} = g \cdot \frac{\mathbf{v}}{\|\mathbf{v}\|_2}$$

The gradient updates decouple the learning of magnitude and direction:

$$\nabla_g \mathcal{L} = \frac{\nabla_{\mathbf{w}} \mathcal{L} \cdot \mathbf{v}}{\|\mathbf{v}\|}, \qquad \nabla_{\mathbf{v}} \mathcal{L} = \frac{g}{\|\mathbf{v}\|}\left(\nabla_{\mathbf{w}} \mathcal{L} - \frac{\nabla_{\mathbf{w}} \mathcal{L} \cdot \mathbf{v}}{\|\mathbf{v}\|^2}\mathbf{v}\right)$$

Alternative normalization for recurrent and convolutional networks. WN was proposed as a lighter-weight alternative to BN that does not depend on batch statistics. It accelerates convergence of SGD and has been applied to RNNs, CNNs, and reinforcement learning (DQN). It is less commonly used today but remains useful when BN is impossible.

Pros:

- **Batch-independent:** like LN, no batch statistics needed.
- Decouples learning of direction and magnitude, which can speed up convergence.
- Fewer normalization statistics to compute per step than BN.
- Works with small datasets where batch statistics are unreliable.

Cons:

- **Less regularizing than BN:** no noise injection from batch statistics, so overfitting can be worse without additional regularization.

- The effective learning rate for the magnitude g can be tricky to tune — too fast and it explodes; too slow and it underfits.
- Empirical performance is generally below BN for CNNs.

4.7 Adaptive Instance Normalization (AdaIN)

AdaIN adaptively aligns the channel-wise mean and variance of a content input x to match those of a style input y:

$$\text{AdaIN}(x, y) = \sigma(y) \left(\frac{x - \mu(x)}{\sigma(x)} \right) + \mu(y)$$

where $\mu(\cdot)$ and $\sigma(\cdot)$ are channel-wise mean and standard deviation (computed across spatial dimensions, like Instance Normalization). The key difference from standard IN is that the scale and shift parameters are *not learnable* — they come from the style input.

Arbitrary style transfer. AdaIN is the core of real-time arbitrary style transfer networks (Huang & Belongie, 2017). The encoder extracts content features; AdaIN aligns their statistics to match extracted style features; the decoder reconstructs the styled image. This enables transferring the style of an arbitrary reference image to a content image in a single forward pass.

Pros:

- Real-time arbitrary style transfer — no per-style training or optimization.
- Simple and elegant: style is encoded entirely in channel-wise mean/variance statistics, which AdaIN directly aligns.
- Works with any content-style pair at test time, including styles unseen during training.

Cons:

- Style is captured only by first-order (mean) and second-order (variance) channel statistics — all higher-order correlations and spatial structure are lost. This prevents transferring complex textures and precise brush-stroke patterns.
- The alignment is local (channel-wise); global spatial structure of the style image (e.g., a face) cannot be transferred.
- Quality degrades for styles that require rich spatial interactions that cannot be captured by simple statistics.

4.8 Switchable Normalization (SN)

Switchable Normalization learns to combine BN, IN, and LN statistics via learnable importance weights:

$$\hat{h} = \frac{h - \sum_{k \in \{\text{bn, in, ln}\}} w_k \mu_k}{\sqrt{\sum_k w'_k \sigma_k^2 + \epsilon}}$$

where w_k and w'_k are softmax-normalized weights learned end-to-end. The network can learn to emphasize BN statistics when batch size is large, IN statistics for style-invariant operations, or LN statistics for per-sample normalization, at each layer independently.

Robust normalization across tasks and batch sizes. SN was proposed as a universal normalization that adapts to the task, architecture, and batch size automatically. It is particularly useful when the optimal normalization is not known a priori or varies across layers.

Pros:

- Learns the best normalization strategy per layer automatically.
- More robust to varying batch sizes than BN alone.
- Can outperform any single normalization in multi-task settings.

Cons:

- Three times more compute (computing BN, IN, and LN statistics per layer).
- Learning the softmax weights can be unstable during early training.
- Additional hyperparameters: initial weight values, whether to use shared weights, etc.

4.9 Filter Response Normalization (FRN)

FRN normalizes each sample-channel pair by the average squared activation across spatial dimensions, dividing out the norm without mean subtraction:

$$\hat{x}_{n,c,h,w} = \frac{x_{n,c,h,w}}{\sqrt{\nu^2 + \epsilon}}, \qquad \nu^2 = \frac{1}{HW} \sum_{h,w} x_{n,c,h,w}^2$$

FRN is combined with a **Thresholded Linear Unit (TLU)**: $z = \max(y, \tau)$, where τ is a learnable threshold, to prevent the normalized activations from collapsing to zero (which would happen with ReLU since FRN does not subtract the mean).

Small-batch image classification. FRN was designed as a batch- independent normalization that matches BN's performance at batch size 32 while working down to batch size 1. The TLU companion is crucial: without it, FRN + ReLU leads to degenerate behavior.

Pros:

- Batch-independent and works at batch size 1.
- Comparable to BN at large batch sizes.
- Simpler than GN (no group partitioning).

Cons:

- Requires the TLU activation — cannot be used with arbitrary activation functions.
- TLU adds a learned parameter per channel, which can overfit.
- Less widely adopted than GN for small-batch vision tasks.

4.10 Local Response Normalization (LRN)

LRN performs lateral inhibition across feature channels at the same spatial location:

$$b^i_{x,y} = \frac{a^i_{x,y}}{\left(k + \alpha \sum_{j=\max(0,\, i-n/2)}^{\min(N-1,\, i+n/2)} (a^j_{x,y})^2\right)^\beta}$$

where a^i is the activation of channel i at position (x, y), and n is the number of adjacent channels to pool over. Default values from AlexNet: $k = 2$, $\alpha = 10^{-4}$, $\beta = 0.75$, $n = 5$.

Historical: AlexNet. LRN was used in AlexNet to implement a form of local contrast normalization inspired by computational neuroscience (lateral inhibition). It provided a small accuracy boost (1–2%) before BN was introduced.

Pros: Biologically inspired. Simple, no learnable parameters. **Cons:** Largely obsolete — replaced by BN which provides far greater benefit. The marginal accuracy gain disappears when BN is used. Computes expensive channel-wise normalization per spatial position.

4.11 Advanced and Domain-Specific Normalizations

4.11.1 PowerNorm

PowerNorm addresses BN's small-batch problem by using running estimates of the *second moment* (mean of squares) rather than variance, which is more stable for small batches. It combines batch-independent running statistics with per-sample normalization.

Alternative to BN for small-batch vision tasks. PowerNorm was proposed as a batch-independent normalization that performs robustly with batch size as small as 2.

Pros: Better small-batch behavior than BN via second-moment tracking. No per-sample normalization noise. **Cons:** More complex than GN/FRN. Less community adoption.

4.11.2 EvoNorm

EvoNorm was discovered via neural architecture search. Two variants: **EvoNorm-S0** (batch-dependent) and **EvoNorm-B0** (batch- independent). EvoNorm combines normalization with an activation function into a single layer:

$$\text{EvoNorm-B0}(x) = \frac{x}{\text{std}(x) + \epsilon} \cdot \gamma + \beta \quad \text{(followed by a Swish-like gating)}$$

Image classification (NAS-discovered). EvoNorm matches or exceeds BN on ImageNet while being batch-independent for the B0 variant. It is primarily of interest in AutoML/NAS contexts.

Pros: Discovered via search, so optimized for the specific task. **Cons:** The search process is expensive; the resulting architecture is opaque and not widely studied.

4.11.3 Ghost Batch Normalization

Within a single mini-batch of size m, Ghost BN partitions the batch into k "ghost" sub-batches of size m/k and normalizes each independently:

$$y^{(j)} = \gamma \cdot \frac{x^{(j)} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} + \beta$$

where μ_j, σ_j^2 are computed only over the j-th virtual sub-batch. During inference, all sub-batch statistics are pooled.

Regularization through increased noise. Ghost BN is used to intentionally increase BN's noise (by using smaller virtual batches), acting as a regularizer. It is effective for training GANs and for improving generalization of large-batch training.

Pros:

- Strong regularization: smaller virtual batches increase noise, similar to training with smaller full batches but without losing hardware efficiency.
- Improves generalization in large-batch settings.

Cons:

- Hyperparameter k (number of ghost batches) requires tuning.
- Too-small virtual batches reintroduce BN's small-batch instability.
- Not standardized in frameworks.

4.12 Graph Normalization (for GNNs)

Graph neural networks have specific normalization needs due to the graph structure. The key approaches:
GraphNorm: Normalizes across all node features *within each graph* in the batch:

$$\mu_i = \frac{1}{N_i H} \sum_{n=1}^{N_i} \sum_{h=1}^{H} x_{i,n,h}$$

where N_i is the number of nodes in graph i. This is a per-graph global normalization (like a per-graph LN). The learnable shift β is further modulated by a function of node degree.
PairNorm: Designed to prevent **over-smoothing** — the phenomenon where stacking many GNN layers makes all node representations converge to the same value. PairNorm centers and rescales the node features of each graph such that the total pairwise squared distance $\sum_{i,j} \|\tilde{\mathbf{x}}_i - \tilde{\mathbf{x}}_j\|^2$ is preserved.
NodeNorm: Normalizes each node's feature vector individually: $\hat{x}_i = x_i / \|x_i\|_2$. Used as a simple regularizer in attention-based GNNs.

Graph neural networks. GraphNorm is used for graph-level prediction tasks (molecular property prediction, graph classification). PairNorm is critical for deep GNNs (> 4 layers) to prevent over-smoothing. NodeNorm is used in graph attention networks (GAT) and graph transformers.

Pros:

- GraphNorm: handles variable-sized graphs elegantly. Degree-aware shift improves performance.
- PairNorm: enables deep GNNs by preventing over-smoothing.
- NodeNorm: simple and effective for attention-based GNNs.

Cons:

- Domain-specific: these methods only apply to graph-structured data.
- GraphNorm's degree-aware mechanism adds complexity.
- The over-smoothing problem is only partially solved by PairNorm; residual connections and drop-edge are also important.

4.13 Comparison Table and Practical Guidance

Practical recommendations:

- **CNNs (classification, detection):** Start with BN. If batch size < 8, switch to GN with $G = 32$.
- **Transformers:** LN (Post-LN or Pre-LN) is standard. Consider RMSNorm for efficiency in large-scale LLMs.

Table 4.1: Comparison of Normalization Methods

Method	**Normalization Dims**	**Batch-Dep?**	**Train = Test?**	**Primary Use**
BatchNorm	Batch × Spatial	Yes	No	CNNs, MLPs
LayerNorm	Features	No	Yes	Transformers, RNNs
InstanceNorm	Spatial (per channel)	No	Yes	Style transfer
GroupNorm	Groups × Spatial	No	Yes	Small-batch vision
RMSNorm	Features (no mean)	No	Yes	LLMs
WeightNorm	Weight vector	No	Yes	RNNs, RL
AdaIN	Spatial (with style)	No	Yes	Style transfer
SyncBN	Batch (multi-GPU)	Yes	No	Small per-GPU batch
FRN	Spatial (per channel)	No	Yes	Small-batch vision
DeepNorm	Residual + LN	No	Yes	Very deep transformers
Switchable	Learned combination	Yes	No	Universal
GraphNorm	Graph × features	No	Yes	GNNs

- **Style transfer / image generation:** IN or AdaIN.
- **RNNs / LSTMs:** LN over BN.
- **Small-batch object detection / segmentation:** GN or SyncBN.
- **Very deep transformers (> 50 layers):** Pre-LN or DeepNorm.
- **Large language models:** RMSNorm for speed.
- **GNNs:** GraphNorm for graph-level tasks; PairNorm for deep GNNs.

The choice of normalization is *not* just an implementation detail — it significantly affects convergence speed, final accuracy, and batch-size flexibility. When in doubt, prefer LayerNorm for sequence models, BatchNorm for CNNs, and GroupNorm as a batch-size-independent fallback for vision.

Chapter 5

Evaluation Metrics

This chapter provides a comprehensive survey of evaluation metrics used across machine learning domains. Each metric is presented with its mathematical formulation, usage context, and critical trade-offs.

5.1 Regression Metrics

Regression metrics quantify the discrepancy between predicted continuous values $\hat{y}_i$ and ground-truth values y_i over n samples.

5.1.1 Mean Squared Error (MSE)

The **Mean Squared Error** is the arithmetic mean of squared residuals:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

It is the empirical risk under the squared-error loss $\ell(y, \hat{y}) = (y - \hat{y})^2$. MSE is differentiable everywhere and heavily penalizes large errors due to the square.

Usage: Default loss function for linear regression (ordinary least squares), neural network regression with linear output, and as a component of the bias-variance decomposition. Also serves as the Brier score when $y_i \in \{0, 1\}$ and $\hat{y}_i \in [0, 1]$.

Pros: Differentiable, convex, unique minimum, strong theoretical foundation (MLE under Gaussian noise). **Cons:** Unit is squared of target unit (not interpretable directly), sensitive to outliers (squared term amplifies them), and not scale-invariant.

5.1.2 Root Mean Squared Error (RMSE)

The **Root Mean Squared Error** is the square root of MSE:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

RMSE restores the original unit of measurement, improving interpretability. It is the sample standard deviation of residuals when residuals have zero mean.

Usage: Preferred over MSE for reporting because units match the target variable. Widely used in Kaggle competitions, climate modeling, and any regression task where interpretability matters.

RMSE and MSE share the same minimizer. The relationship RMSE $\geq$ MAE always holds, with equality only when all absolute residuals are equal.

5.1.3 Mean Absolute Error (MAE)

The **Mean Absolute Error** is the arithmetic mean of absolute residuals:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

MAE corresponds to the MLE under Laplace-distributed errors. It is not differentiable at zero residual but provides robust estimates less sensitive to outliers.

Pros: Same units as target, interpretable, robust to outliers (linear penalty vs. quadratic). **Cons:** Not differentiable at zero, no closed-form solution for linear regression (requires linear programming), and does not uniquely identify the optimal predictor for conditional medians in some cases.

5.1.4 Coefficient of Determination (R^2)

The **coefficient of determination** measures the proportion of variance in the dependent variable explained by the model:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

where $\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$ is the sample mean. Range is $(-\infty, 1]$. A value of 1 indicates perfect prediction; 0 indicates performance equal to always predicting the mean; negative values indicate performance worse than the mean predictor.

Usage: Standard goodness-of-fit measure for linear regression. Used to compare nested models. Must be interpreted with caution: R^2 always increases (or stays the same) when adding predictors, regardless of their relevance.

5.1.5 Adjusted R^2

The **adjusted** R^2 penalizes the inclusion of unnecessary predictors:

$$R^2_{\text{adj}} = 1 - \frac{(1 - R^2)(n-1)}{n - p - 1}$$

where p is the number of predictors (excluding the intercept). When p increases, the numerator decreases but the denominator $n - p - 1$ also decreases, creating a trade-off.

Pros: Penalizes model complexity, enables fair comparison between models with different numbers of features. **Cons:** Still not a formal model selection criterion (AIC/BIC are

preferred), heuristic penalty.

5.1.6 Mean Absolute Percentage Error (MAPE)

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^{n} \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

MAPE expresses error as a percentage of actual values. It is scale-independent and intuitive for business stakeholders.

Critical limitations: MAPE is undefined when any $y_i = 0$, and is asymmetric—under-predictions have a capped error (max 100%) while over-predictions have unbounded error. It also penalizes positive and negative errors unequally.
For forecasting tasks where y_i can be zero or near-zero, use SMAPE or scaled errors instead.

5.1.7 Symmetric Mean Absolute Percentage Error (SMAPE)

$$\text{SMAPE} = \frac{100\%}{n} \sum_{i=1}^{n} \frac{|y_i - \hat{y}_i|}{(|y_i| + |\hat{y}_i|)/2}$$

The denominator uses the average of actual and predicted values, bounding the error to $[0\%, 200\%]$. Despite its name, SMAPE is not fully symmetric in its treatment of over- and under-predictions.

Usage: Forecasting competitions (M3, M4), supply chain demand forecasting, and any time-series context where MAPE's zero-denominator issues arise.

5.1.8 Explained Variance Score

$$\text{EV} = 1 - \frac{\text{Var}(y - \hat{y})}{\text{Var}(y)}$$

where $\text{Var}(\cdot)$ denotes sample variance. When residuals have zero mean, EV equals R^2. When residuals have non-zero bias, EV $> R^2$ if the bias reduces residual variance.

5.1.9 Mean Bias Error (MBE)

$$\text{MBE} = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)$$

MBE captures systematic over- or under-prediction. A positive MBE indicates the model tends to over-predict; negative indicates under-prediction.

Usage: Assessing model calibration in physical simulations, climate models, and energy forecasting. Should be reported alongside RMSE or MAE since cancellation of positive and negative errors can hide poor performance.

5.1.10 Concordance Correlation Coefficient (CCC)

$$\rho_c = \frac{2\rho\sigma_{\hat{y}}\sigma_y}{\sigma_{\hat{y}}^2 + \sigma_y^2 + (\mu_{\hat{y}} - \mu_y)^2}$$

where ρ is the Pearson correlation between y and $\hat{y}$, σ denotes standard deviation, and μ denotes mean. CCC combines precision (Pearson correlation) with accuracy (bias). Range is $[-1, 1]$.

Usage: Measuring agreement in medical imaging, affective computing (emotion recognition), and any domain requiring both correlation and bias assessment.

5.2 Binary Classification Metrics

Let TP, TN, FP, FN denote true positives, true negatives, false positives, and false negatives respectively.

5.2.1 Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

The fraction of all predictions that are correct. Despite its simplicity, accuracy is misleading for imbalanced datasets.

Accuracy Paradox: On a dataset with 99% negatives, a model that always predicts negative achieves 99% accuracy. Always consider class distribution before using accuracy as a primary metric.

5.2.2 Precision and Recall

Precision (Positive Predictive Value):

$$P = \frac{TP}{TP + FP}$$

Recall (Sensitivity, True Positive Rate):

$$R = \frac{TP}{TP + FN}$$

Specificity (True Negative Rate):

$$\text{TNR} = \frac{TN}{TN + FP}$$

Precision is critical in search engines and recommendation systems (how many recommended items are relevant?). **Recall** is critical in medical diagnosis and fraud detection (how many actual cases did we catch?).

5.2.3 F-Score

The F_β score is the weighted harmonic mean of precision and recall:

$$F_\beta = (1 + \beta^2)\frac{P \cdot R}{\beta^2 P + R}$$

When $\beta = 1$, we obtain the standard F1 score:

$$F_1 = \frac{2PR}{P + R} = \frac{2TP}{2TP + FP + FN}$$

$\beta < 1$ weights precision more heavily; $\beta > 1$ weights recall more heavily. Common choices include $F_{0.5}$ (precision-focused) and F_2 (recall-focused).

The harmonic mean is used instead of the arithmetic mean because it penalizes extreme values more severely. A model with perfect recall but poor precision (or vice versa) will have a low F1 score.

5.2.4 Matthews Correlation Coefficient (MCC)

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

MCC ranges from -1 (perfectly wrong) to $+1$ (perfect), with 0 indicating performance no better than random. It is the Pearson correlation coefficient between actual and predicted binary labels.

Usage: The MCC is considered one of the best single-value metrics for binary classification because it accounts for all four confusion matrix entries proportionally. It is effective even with severely imbalanced datasets.

Pros: Balanced, symmetric, handles class imbalance well, single-value summary, range $[-1, 1]$ is intuitive. **Cons:** Undefined when any denominator factor is zero (e.g., all predictions are one class), less interpretable than F1 for non-technical audiences.

5.2.5 Balanced Accuracy

$$\text{Balanced Accuracy} = \frac{\text{TPR} + \text{TNR}}{2}$$

The arithmetic mean of sensitivity and specificity. Unlike standard accuracy, it gives equal weight to both classes regardless of size.

5.2.6 Jaccard Index (Intersection over Union)

$$\text{IoU} = \frac{TP}{TP + FP + FN}$$

Also known as the Tanimoto coefficient. It measures the overlap between predicted and actual positive sets relative to their union.

Usage: Dominant metric in object detection and semantic segmentation. The `mAP` score in COCO and PASCAL VOC is based on precision-recall curves evaluated at specific IoU thresholds.

5.2.7 Cohen's Kappa

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

where $p_o = \text{Accuracy}$ is the observed agreement and p_e is the probability of chance agreement:

$$p_e = \frac{(TP + FP)(TP + FN) + (TN + FN)(TN + FP)}{(TP + TN + FP + FN)^2}$$

Range is $[-1, 1]$, with > 0.8 generally considered substantial agreement.

Usage: Inter-rater reliability assessment, medical diagnosis evaluation, and any context where correcting for chance agreement is essential.

5.2.8 G-Mean

$$\text{G-Mean} = \sqrt{\text{TPR} \cdot \text{TNR}}$$

The geometric mean of sensitivity and specificity. A model must perform well on both classes to achieve a high G-Mean; poor performance on either class drives the metric toward zero.

5.2.9 Hamming Loss (Multi-Label)

The **Hamming Loss** is the fraction of incorrectly predicted labels over all labels:

$$\text{Hamming Loss} = \frac{1}{n \cdot L} \sum_{i=1}^{n} \sum_{\ell=1}^{L} \mathbf{1}[y_{i\ell} \neq \hat{y}_{i\ell}]$$

where L is the number of labels per instance. Lower is better; 0 indicates perfect multi-label prediction.

5.3 Threshold-Independent Metrics

5.3.1 AUC-ROC

The **Area Under the Receiver Operating Characteristic** curve plots TPR (y-axis) against FPR (x-axis) as the classification threshold varies. The AUC-ROC is the integral:

$$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR}^{-1}(t))\, dt$$

Equivalently, AUC equals the probability that a randomly chosen positive sample is ranked higher than a randomly chosen negative sample by the classifier:

$$\text{AUC} = 1 - \frac{1}{n^+ n^-} \sum_{i:y_i=+1} \sum_{j:y_j=-1} \left[\mathbf{1}_{f(x_i)<f(x_j)} + \frac{1}{2}\mathbf{1}_{f(x_i)=f(x_j)} \right]$$

where $f(x)$ is the classifier's decision score.

Usage: Binary classifier comparison independent of threshold choice. A model with AUC = 0.5 performs no better than random; AUC = 1.0 is perfect separation.

Pros: Threshold-invariant, scale-invariant, probabilistic interpretation, widely accepted. **Cons:** Can be overly optimistic for highly imbalanced datasets, does not reflect calibration quality, summarizes performance over all thresholds (many of which are irrelevant in practice).

5.3.2 AUC-PR (Precision-Recall AUC)

The **Average Precision (AP)** summarizes the precision-recall curve:

$$\text{AP} = \sum_{k=1}^{n} (R_k - R_{k-1}) P_k$$

where R_k and P_k are the recall and precision at the k-th threshold (sorted by decreasing confidence). This is a discrete approximation of $\int_0^1 P(R)\, dR$.

Usage: Preferred over AUC-ROC for imbalanced datasets because it focuses on the positive class (the minority class) and ignores true negatives, which dominate in imbalanced settings.

5.3.3 Log-Loss (Cross-Entropy Loss)

$$\text{Log-Loss} = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{p}_i) + (1-y_i)\log(1-\hat{p}_i)]$$

where $\hat{p}_i = P(y_i = 1 \mid x_i)$ is the predicted probability. Log-Loss is a strictly proper scoring rule, meaning the true probability distribution achieves the minimum expected loss.

5.3.4 Brier Score

$$\text{Brier Score} = \frac{1}{n}\sum_{i=1}^{n}(\hat{p}_i - y_i)^2$$

The Brier score is the MSE between predicted probabilities and binary outcomes. Like log-loss, it is a strictly proper scoring rule. Range is $[0, 1]$.

5.4 Multi-Class Metrics

5.4.1 Micro, Macro, and Weighted Averaging

For K classes with per-class metrics M_k:

Micro-average: Compute global TP, FP, FN then apply the metric formula.

$$\text{Macro-average:} \quad M_{\text{macro}} = \frac{1}{K}\sum_{k=1}^{K} M_k$$

$$\text{Weighted-average:} \quad M_{\text{weighted}} = \sum_{k=1}^{K} w_k M_k, \quad w_k = \frac{n_k}{n}$$

Micro: Favored when class imbalance is severe—it gives equal weight to each sample. **Macro:** Favored when each class is equally important regardless of frequency. **Weighted:** Compromise that accounts for class sizes.

5.4.2 Top-K Accuracy

Prediction is correct if the true class is among the top K highest predicted probabilities. Common in image classification (ImageNet uses top-1 and top-5 accuracy).

5.4.3 Weighted Cohen's Kappa

Linear weighting: $w_{ij} = 1 - \frac{|i-j|}{K-1}$. **Quadratic weighting:** $w_{ij} = 1 - \left(\frac{i-j}{K-1}\right)^2$. Quadratic-weighted kappa is common in ordinal classification tasks (e.g., diabetic retinopathy grading).

5.5 Ranking Metrics

5.5.1 Precision@K, Recall@K, F1@K

For a ranked list of items, consider only the top K recommendations:

$$\text{Precision@K} = \frac{|\{\text{relevant items in top } K\}|}{K}$$

$$\text{Recall@K} = \frac{|\{\text{relevant items in top } K\}|}{|\{\text{all relevant items}\}|}$$

5.5.2 Mean Reciprocal Rank (MRR)

$$\text{MRR} = \frac{1}{|Q|}\sum_{i=1}^{|Q|}\frac{1}{\text{rank}_i}$$

where rank_i is the position of the first relevant item for query i. MRR is appropriate when only the first relevant result matters (e.g., question answering).

5.5.3 Mean Average Precision (MAP)

MAP averages the Average Precision over multiple queries:

$$\text{MAP} = \frac{1}{|Q|}\sum_{q=1}^{|Q|}\text{AP}(q)$$

where $\text{AP}(q) = \frac{1}{|\text{Relevant}_q|}\sum_{k=1}^{n}\text{Precision@k} \cdot \mathbf{1}[\text{item}_k \text{ is relevant}]$.

5.5.4 Discounted Cumulative Gain (DCG) and NDCG

DCG@K accounts for graded relevance (not just binary):

$$\text{DCG}_K = \sum_{i=1}^{K}\frac{2^{\text{rel}_i} - 1}{\log_2(i+1)}$$

The exponential gain $2^{\text{rel}_i} - 1$ gives high weight to highly relevant documents. The log discount simulates decreasing user attention with rank.

NDCG@K normalizes DCG by the ideal DCG (perfect ranking):

$$\text{NDCG}_K = \frac{\text{DCG}_K}{\text{IDCG}_K}$$

where IDCG_K is computed by sorting documents by true relevance in descending order. Range is $[0, 1]$.

5.5.5 Correlation-Based Ranking Metrics

Kendall's τ: Measures rank correlation based on concordant and discordant pairs:

$$\tau = \frac{2(C - D)}{n(n-1)}$$

where C is the number of concordant pairs and D is the number of discordant pairs.
Spearman's ρ: Pearson correlation applied to rank values:

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

where d_i is the difference between the ranks of item i in the two rankings.

5.5.6 Hit Rate

$$\text{Hit Rate@K} = \frac{|\{\text{queries with at least 1 relevant item in top } K\}|}{|Q|}$$

Simple binary metric widely used in recommender systems evaluation.

5.6 Generative Model Metrics

5.6.1 Inception Score (IS)

$$\text{IS} = \exp\big(\mathbb{E}_{\mathbf{x}}[\text{KL}(p(y|\mathbf{x}) \parallel p(y))]\big)$$

where $p(y|\mathbf{x})$ is the conditional label distribution from an Inception classifier, and $p(y) = \int p(y|\mathbf{x}) p_g(\mathbf{x})\, d\mathbf{x}$ is the marginal. High IS requires both high confidence (low entropy per sample) and high diversity (high marginal entropy). IS evaluates only the generator, not fidelity to real data.

Pros: Correlates reasonably with human judgment of image quality. **Cons:** Insensitive to mode collapse (if all samples look like one class but high quality, IS is high), sensitive to the choice of classifier, does not compare against real data distribution.

5.6.2 Fréchet Inception Distance (FID)

$$\text{FID} = \|\boldsymbol{\mu}_r - \boldsymbol{\mu}_g\|_2^2 + \text{Tr}\big(\boldsymbol{\Sigma}_r + \boldsymbol{\Sigma}_g - 2(\boldsymbol{\Sigma}_r\boldsymbol{\Sigma}_g)^{1/2}\big)$$

where $(\boldsymbol{\mu}_r, \boldsymbol{\Sigma}_r)$ and $(\boldsymbol{\mu}_g, \boldsymbol{\Sigma}_g)$ are the mean and covariance of Inception features for real and generated data, assuming multivariate Gaussian distributions. Lower FID indicates higher similarity between real and generated distributions.

Usage: The de facto standard for evaluating GANs and diffusion models. FID captures both fidelity and diversity, and correlates well with human evaluation.

FID sensitivity: FID is biased for small sample sizes (use $\geq$ 50,000 samples), varies across feature extractors (torch-fidelity vs. clean-fid), and is affected by image preprocessing (resizing, normalization). Use `clean-fid` for reproducible evaluation.

5.6.3 Kernel Inception Distance (KID)

KID computes Maximum Mean Discrepancy (MMD) between Inception features of real and generated data using a polynomial kernel. Unlike FID, KID has an unbiased estimator with a finite sample bound (via the 3rd-degree MMD), making it more reliable for small sample sizes.

5.6.4 Peak Signal-to-Noise Ratio (PSNR)

$$\text{PSNR} = 10\log_{10}\left(\frac{\text{MAX}^2}{\text{MSE}}\right)$$

where MAX is the maximum possible pixel value (typically 255 for 8-bit images). Higher PSNR indicates better reconstruction quality. It is the negative log of MSE, scaled to decibels.

Usage: Image compression, super-resolution, image denoising, and autoencoder reconstruction quality. PSNR is computationally cheap and widely understood in signal processing.

Pros: Simple, fast to compute, physically interpretable (dB scale), widely used. **Cons:** Does not correlate well with perceived visual quality. Two images with identical PSNR can look very different due to the pixel-wise nature of MSE.

5.6.5 Structural Similarity Index (SSIM)

SSIM decomposes similarity into three components—luminance, contrast, and structure:

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}$$

where μ_x, μ_y are local means, σ_x^2, σ_y^2 are local variances, σ_{xy} is local covariance, and $c_1 = (k_1 L)^2, c_2 = (k_2 L)^2$ are stabilization constants with L being the dynamic range. Range is $[-1, 1]$ with 1 indicating perfect structural similarity.

Usage: Perceptual image quality assessment, loss function for deep learning (SSIM loss $= 1 - \text{SSIM}$), video quality metrics, and medical image reconstruction.

5.6.6 Multi-Scale SSIM (MS-SSIM)

MS-SSIM computes SSIM at multiple scales (resolution levels) via iterative downsampling and combines them:

$$\text{MS-SSIM}(x, y) = [l_M(x, y)]^{\gamma_M} \prod_{j=1}^{M} [c_j(x, y)]^{\beta_j} [s_j(x, y)]^{\gamma_j}$$

where l is luminance, c is contrast, s is structure at scale j, and M is the number of scales. The exponents β_j, γ_j control the relative importance of each scale.

5.7 NLP Generation Metrics

5.7.1 BLEU (Bilingual Evaluation Understudy)

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$$

where p_n is the modified n-gram precision (clipping each n-gram count to its maximum occurrence in any single reference), $w_n = 1/N$ for uniform weights, and the brevity penalty (BP) penalizes overly short candidates:

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{1-r/c} & \text{if } c \leq r \end{cases}$$

with c being candidate length and r being effective reference length.

Usage: Machine translation evaluation. BLEU-4 (4-gram) is standard. BLEU is a corpus-level metric and is unreliable for single-sentence evaluation.

Pros: Fast, language-independent, correlates with human judgment at corpus level. **Cons:** Does not account for meaning or synonymy, penalizes valid paraphrases, insensitive to word order beyond n-gram window, favours literal translations.

5.7.2 ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

ROUGE-N: n-gram recall between candidate and reference:

$$\text{ROUGE-N} = \frac{\sum_{S \in \text{Ref}} \sum_{\text{gram}_n \in S} \text{Count}_{\text{match}}(\text{gram}_n)}{\sum_{S \in \text{Ref}} \sum_{\text{gram}_n \in S} \text{Count}(\text{gram}_n)}$$

ROUGE-L: Based on the Longest Common Subsequence (LCS):

$$R_{LCS} = \frac{|LCS(X,Y)|}{|Y|}, \quad P_{LCS} = \frac{|LCS(X,Y)|}{|X|}, \quad F_{LCS} = \frac{(1+\beta^2) R_{LCS} P_{LCS}}{R_{LCS} + \beta^2 P_{LCS}}$$

where X is the candidate summary, Y is the reference, and β controls the precision-recall trade-off.
ROUGE-W: Weighted LCS that favors consecutive matches over scattered ones.
ROUGE-S: Skip-bigram co-occurrence statistics, capturing long-range dependencies.

Usage: Text summarization evaluation. ROUGE-L is the most popular variant for single-document summarization.

5.7.3 METEOR (Metric for Evaluation of Translation with Explicit ORdering)

METEOR computes a harmonic mean of unigram precision and recall, then applies a fragmentation penalty:

$$\text{METEOR} = F_{\text{mean}} \cdot (1 - \text{Penalty})$$

where the penalty is $\gamma \cdot \left(\frac{\text{chunks}}{\text{matches}}\right)^{\beta}$, with chunks being contiguous matched unigram sequences. METEOR also performs stemming, synonym matching, and paraphrase matching.

Usage: Machine translation evaluation with better sentence-level correlation than BLEU. The stemming and synonym matching provide robustness against morphological variation.

5.7.4 CIDEr and SPICE

CIDEr (Consensus-based Image Description Evaluation): Uses TF-IDF weighted n-gram matching to emphasize visually salient words:

$$\text{CIDEr}_n(c, S) = \frac{1}{M} \sum_{j=1}^{M} \frac{\boldsymbol{g}^n(c) \cdot \boldsymbol{g}^n(s_j)}{\|\boldsymbol{g}^n(c)\| \|\boldsymbol{g}^n(s_j)\|}$$

where $\boldsymbol{g}^n(c)$ is the TF-IDF vector of n-grams for candidate c. CIDEr is a weighted average across n-gram levels.
SPICE (Semantic Propositional Image Caption Evaluation): Parses captions into scene graphs and computes F1 over semantic propositions (objects, attributes, relations).

5.7.5 BERTScore

BERTScore computes token-level cosine similarity using BERT contextual embeddings, followed by greedy optimal matching:

$$R_{\text{BERT}} = \frac{1}{|y|} \sum_{y_i \in y} \max_{\hat{y}_j \in \hat{y}} \cos(\mathbf{e}(y_i), \mathbf{e}(\hat{y}_j))$$

$$P_{\text{BERT}} = \frac{1}{|\hat{y}|} \sum_{\hat{y}_j \in \hat{y}} \max_{y_i \in y} \cos(\mathbf{e}(\hat{y}_j), \mathbf{e}(y_i))$$

$$F_{\text{BERT}} = \frac{2 P_{\text{BERT}} R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}}$$

Usage: Text generation evaluation that captures semantic similarity beyond exact n-gram overlap. BERTScore correlates better with human judgment than BLEU or ROUGE.

5.7.6 Perplexity (PPL)

Perplexity is the exponential of the average negative log-likelihood per token:

$$\text{PPL} = \exp\left(-\frac{1}{N} \sum_{i=1}^{N} \log p(w_i \mid w_{<i}) \right) = \exp(\text{cross-entropy})$$

Lower perplexity indicates the model is less "surprised" by the data. PPL $= k$ means the model is as confused as if it were choosing uniformly among k equally likely options at each step.

Usage: Intrinsic evaluation of language models. Standard benchmark for comparing architectures (LSTM vs. Transformer vs. SSM). Also used for early stopping during LM training.

PPL is highly sensitive to tokenization—comparing models with different tokenizers is meaningless without normalization. PPL also does not correlate perfectly with downstream task performance (the "perplexity trap").

5.7.7 BARTScore and MoverScore

BARTScore: Uses the pre-trained BART model to compute the log-likelihood of generating the reference/candidate given the source/candidate. It frames evaluation as a text generation probability problem.
MoverScore: Uses Word Mover's Distance (Earth Mover's Distance) in the contextual embedding space, providing a semantically aware n-gram matching that accounts for many-to-one alignments.

5.8 Object Detection Metrics

5.8.1 Mean Average Precision (mAP)

In object detection, mAP is computed by:

1. For each class, sort detections by confidence score.
2. Match each detection to ground-truth boxes using an IoU threshold.
3. Compute precision-recall curve and interpolated AP (area under PR curve at 11 recall points or all points).
4. mAP is the mean AP across all classes.

mAP@0.5: IoU threshold of 0.5 (PASCAL VOC standard). **mAP@0.5:0.95:** Average mAP over IoU thresholds from 0.5 to 0.95 with step 0.05 (COCO standard, more stringent).

5.8.2 AP by Object Size

COCO evaluation categorizes objects by pixel area in the original image:

- $\mathbf{AP^{small}}$**:** Area $< 32^2$ pixels.
- $\mathbf{AP^{medium}}$**:** $32^2 \leq$ Area $< 96^2$ pixels.
- $\mathbf{AP^{large}}$**:** Area $\geq 96^2$ pixels.

5.8.3 Average Recall (AR)

AR measures the maximum recall given a fixed number of detections per image (e.g., AR@1, AR@10, AR@100). Higher AR indicates the detector can find objects, regardless of precision. AR@100 is the maximum achievable recall for a given detector.

5.9 Segmentation Metrics

5.9.1 Pixel Accuracy

$$\text{Pixel Accuracy} = \frac{\sum_{k=1}^{K} n_{kk}}{\sum_{k=1}^{K} \sum_{\ell=1}^{K} n_{k\ell}}$$

where $n_{k\ell}$ is the number of pixels of class k predicted as class ℓ. Suffers from class imbalance issues similar to standard accuracy.

5.9.2 Mean Intersection over Union (mIoU)

$$\text{mIoU} = \frac{1}{K}\sum_{k=1}^{K}\frac{n_{kk}}{\sum_{\ell=1}^{K} n_{k\ell} + \sum_{\ell=1}^{K} n_{\ell k} - n_{kk}}$$

mIoU is the most widely used segmentation metric. It computes IoU per class and averages, providing a balanced assessment.

5.9.3 Dice Coefficient (F1 for Segmentation)

$$\text{Dice}_k = \frac{2n_{kk}}{\sum_{\ell=1}^{K} n_{k\ell} + \sum_{\ell=1}^{K} n_{\ell k}}$$

This is identical to the F1 score applied to the pixel-level confusion matrix. The Dice loss ($1 - \text{Dice}$) is commonly used for training segmentation models, often combined with cross-entropy.

5.9.4 Panoptic Quality (PQ)

$$\text{PQ} = \frac{\sum_{(p,g)\in TP} \text{IoU}(p,g)}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|}$$

PQ unifies semantic and instance segmentation evaluation. It is factorized as $\text{PQ} = \text{SQ} \times \text{RQ}$, where SQ (Segmentation Quality) is the average IoU of matched segments and RQ (Recognition Quality) is the F1 score of segment detection.

5.10 Clustering Metrics

5.10.1 Silhouette Score

For each sample i:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where $a(i)$ is the mean intra-cluster distance (distance to other points in the same cluster) and $b(i)$ is the mean nearest-cluster distance (distance to points in the closest other cluster). Range is $[-1, 1]$; values near $+1$ indicate well-separated clusters, 0 indicates overlapping clusters, and -1 indicates misclassified points.

5.10.2 Davies-Bouldin Index

$$\text{DB} = \frac{1}{K}\sum_{i=1}^{K}\max_{j\neq i}\left(\frac{\sigma_i + \sigma_j}{d(c_i, c_j)}\right)$$

where σ_i is the average distance of points in cluster i to its centroid c_i, and $d(c_i, c_j)$ is the distance between centroids. Lower DB indicates better clustering (compact, well-separated clusters).

5.10.3 Calinski-Harabasz Index

$$\text{CH} = \frac{\text{Tr}(\mathbf{B}_K)}{\text{Tr}(\mathbf{W}_K)} \cdot \frac{n-K}{K-1}$$

where $\mathbf{B}_K$ is the between-cluster scatter matrix and $\mathbf{W}_K$ is the within-cluster scatter matrix. Higher CH indicates better-defined clusters. Equivalent to the F-statistic in one-way ANOVA.

5.10.4 Adjusted Rand Index (ARI)

The Rand Index measures agreement between two clusterings. ARI corrects for chance:

$$\text{ARI} = \frac{\text{RI} - \mathbb{E}[\text{RI}]}{\max(\text{RI}) - \mathbb{E}[\text{RI}]}$$

Range is $[-1, 1]$ with 0 indicating random agreement. ARI is symmetric and does not assume any specific cluster structure.

5.10.5 Normalized Mutual Information (NMI)

$$\text{NMI}(U, V) = \frac{2 \cdot I(U;V)}{H(U) + H(V)}$$

where $I(U;V)$ is the mutual information between cluster assignments U and V, and $H(\cdot)$ is entropy. NMI ranges from 0 (no mutual information) to 1 (perfect correlation). AMI (Adjusted MI) further corrects for chance.

5.10.6 Homogeneity, Completeness, and V-Measure

Homogeneity: Each cluster contains only members of a single class:

$$h = 1 - \frac{H(C \mid K)}{H(C)}$$

Completeness: All members of a class are assigned to the same cluster:

$$c = 1 - \frac{H(K \mid C)}{H(K)}$$

V-Measure: Harmonic mean of homogeneity and completeness:

$$V_\beta = \frac{(1+\beta^2) \cdot h \cdot c}{\beta^2 \cdot h + c}$$

5.11 Fairness and Calibration Metrics

5.11.1 Demographic Parity

A classifier satisfies demographic parity if the positive prediction rate is equal across protected groups:

$$P(\hat{Y} = 1 \mid A = a) = P(\hat{Y} = 1 \mid A = b)$$

where A denotes a protected attribute (e.g., gender, race). The **Disparate Impact** ratio $\frac{P(\hat{Y}=1|A=a)}{P(\hat{Y}=1|A=b)}$ should be between 0.8 and 1.25 under the "80% rule."

5.11.2 Equalized Odds and Equal Opportunity

Equalized Odds: TPR and FPR are equal across groups:

$$P(\hat{Y} = 1 \mid Y = y, A = a) = P(\hat{Y} = 1 \mid Y = y, A = b), \quad \forall y \in \{0, 1\}$$

Equal Opportunity: A relaxation requiring only equal TPR across groups (FPR may differ).

5.11.3 Expected Calibration Error (ECE)

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \big|\text{acc}(B_m) - \text{conf}(B_m)\big|$$

where predictions are partitioned into M equal-width bins B_m by confidence, $\text{acc}(B_m)$ is the average accuracy in bin m, and $\text{conf}(B_m)$ is the average predicted confidence. Lower ECE indicates better calibration. A perfectly calibrated model has $\text{acc}(B_m) = \text{conf}(B_m)$ for all bins.

Usage: Evaluating neural network confidence calibration. Modern deep networks are often poorly calibrated (overconfident on incorrect predictions). Temperature scaling and label smoothing are common fixes.

5.12 Summary Comparison Table

Key principle: No single metric captures all aspects of model performance. A robust evaluation strategy reports multiple complementary metrics (e.g., F1 + MCC + AUC-PR + ECE for classification; RMSE + MAE + R^2 for regression). The choice of metric should align with the problem's cost structure—the relative cost of false positives vs. false negatives, or the tolerance for large vs. small residuals.

Table 5.1: Comprehensive Comparison of Evaluation Metrics

Metric	**Range**	**Best**	**Type**	**Notes**
MSE	$[0, \infty)$	0	Regression	In units squared; outlier-sensitive
RMSE	$[0, \infty)$	0	Regression	Same unit as target; standard report
MAE	$[0, \infty)$	0	Regression	Robust to outliers; not differentiable at 0
R^2	$(-\infty, 1]$	1	Regression	Proportion of variance explained
MAPE	$[0, \infty)$	0	Regression	Undefined if $y_i = 0$; scale-independent
SMAPE	$[0, 200\%]$	0	Regression	Bounded; handles near-zero values
CCC	$[-1, 1]$	1	Regression	Combines correlation and bias
Accuracy	$[0, 1]$	1	Classif.	Misleading for imbalance
Precision	$[0, 1]$	1	Classif.	False positives are costly
Recall	$[0, 1]$	1	Classif.	False negatives are costly
F1	$[0, 1]$	1	Classif.	Harmonic mean of P and R
MCC	$[-1, 1]$	1	Classif.	Balanced; handles imbalance well
AUC-ROC	$[0, 1]$	1	Classif.	Threshold-independent; optimistic for imbalance
AUC-PR	$[0, 1]$	1	Classif.	Better for imbalanced data
Log-Loss	$[0, \infty)$	0	Classif.	Proper scoring rule; penalizes overconfidence
Brier	$[0, 1]$	0	Classif.	MSE of probabilities
Cohen's κ	$[-1, 1]$	1	Classif.	Corrects for chance agreement
NDCG@K	$[0, 1]$	1	Ranking	Graded relevance; log-discounted
MRR	$(0, 1]$	1	Ranking	Reciprocal of first relevant rank
FID	$[0, \infty)$	0	Generative	Lower = closer to real distribution
IS	$[1, \infty)$	Higher	Generative	Higher = better quality & diversity
PSNR	$[0, \infty)$ dB	Higher	Image	Pixel-level fidelity
SSIM	$[-1, 1]$	1	Image	Perceptual similarity
BLEU	$[0, 1]$	1	NLG	Corpus-level n-gram precision
ROUGE-L	$[0, 1]$	1	NLG	LCS-based recall; summarization
Perplexity	$[1, \infty)$	Lower	LM	Exponential of cross-entropy
mAP@0.5:0.95	$[0, 1]$	1	Detection	Averaged over IoU thresholds

Chapter 6

Softmax Variants and Temperature

The softmax function is a cornerstone of modern neural networks, serving as both a classification output layer and an attention mechanism. This chapter surveys classical softmax, its temperature-scaled extension, and a family of sparse, discrete, and margin-based variants.

6.1 Classical Softmax

The **softmax function** maps a vector of unnormalized logits $\mathbf{z} \in \mathbb{R}^K$ to a probability vector on the $(K-1)$-simplex Δ^{K-1}:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \quad i = 1, \ldots, K$$

Each output satisfies $y_i \in (0, 1)$ and $\sum_i y_i = 1$.

Properties of softmax:

1. **Positivity:** $y_i > 0$ for all i, meaning softmax can never produce exact zero probabilities.
2. **Normalization:** $\sum_i y_i = 1$; the output is a valid categorical distribution.
3. **Order preservation:** If $z_i > z_j$, then $y_i > y_j$. Softmax is strictly monotonic in its inputs.
4. **Translation invariance:** $\text{softmax}(\mathbf{z} + c) = \text{softmax}(\mathbf{z})$ for any scalar c. Adding a constant to all logits does not change the distribution.
5. **Gradient (softmax identity):**

   $$\frac{\partial y_i}{\partial z_j} = y_i(\delta_{ij} - y_j)$$

   where δ_{ij} is the Kronecker delta. This elegant form links the Jacobian to only the output probabilities themselves.
6. **Numerical stability:** To avoid overflow from large e^{z_i}, compute:

   $$\text{softmax}(z_i) = \frac{e^{z_i - \max(\mathbf{z})}}{\sum_j e^{z_j - \max(\mathbf{z})}}$$

   This subtracts the maximum logit and is mathematically equivalent due to translation invariance.

Usage:

- **Multi-class classification:** Output layer of a neural network, producing class probabilities. Typically paired with cross-entropy loss: $\mathcal{L} = -\sum_k y_k \log \hat{y}_k$.
- **Attention mechanisms:** Transformer self-attention computes $\text{softmax}(\mathbf{Q}\mathbf{K}^T/\sqrt{d_k})\mathbf{V}$ to obtain convex combinations of value vectors.
- **Mixture models:** Gating networks in Mixture of Experts (MoE) use softmax to produce routing probabilities.
- **Policy gradient RL:** Neural network policies output softmax probabilities over discrete action spaces.
- **Reinforcement learning with function approximation:** Softmax action selection for exploration.

Pros: Differentiable everywhere, smooth gradients for backpropagation, maps arbitrary reals to valid probabilities, translation invariant, numerically stable implementation exists, well-studied theoretical properties.
Cons: Cannot produce sparse outputs (all entries strictly positive regardless of relative magnitude), exponential can overflow or underflow without the max-subtraction trick, no inherent notion of margin between classes, and soft assignment distributes probability mass to all classes which can dilute attention in long sequences.

6.2 Temperature-Scaled Softmax

Introducing a temperature parameter $T > 0$:

$$p_i = \frac{e^{z_i/T}}{\sum_{j=1}^{K} e^{z_j/T}}$$

The temperature controls the entropy of the output distribution:

- $T \to 0^+$: The distribution approaches a one-hot vector at $\arg\max_i z_i$ (hard maximum).
- $T = 1$: Standard softmax.
- $T \to \infty$: The distribution approaches uniform, $p_i \to 1/K$ for all i.
- $T < 1$: Peaked/sharp distribution (lower entropy).
- $T > 1$: Flattened/smooth distribution (higher entropy).

Gradient with temperature:

$$\frac{\partial p_i}{\partial z_j} = \frac{1}{T} p_i(\delta_{ij} - p_j)$$

Lower temperatures produce larger gradients (sharper peak → larger derivatives), which can cause training instability. Higher temperatures produce smaller gradients, which can

slow convergence.

6.2.1 Temperature in LLM Sampling

LLM text generation pipeline:

1. Model produces logits $\mathbf{z} \in \mathbb{R}^V$ over vocabulary of size V.
2. Apply temperature: $\tilde{z}_i = z_i / T$.
3. Compute probabilities: $p_i = \text{softmax}(\tilde{z}_i)$.
4. Sample token from categorical distribution p.

Temperature effects:

- $T = 0$ (greedy decoding): Always select the most probable token. Deterministic, safe, but repetitive.
- $T \approx 0.2$–0.5: Conservative sampling. Stays on-topic, less creative. Typical for factual Q&A, code generation.
- $T = 1.0$: Default. Balances creativity and coherence.
- $T \approx 1.2$–2.0: Creative/diverse outputs. Higher risk of hallucination and incoherence. Useful for brainstorming, creative writing.

6.2.2 Top-K and Top-P (Nucleus) Sampling

Top-K sampling: Truncate the vocabulary to the K most probable tokens, renormalize, and sample:

$$p'_i = \begin{cases} p_i / \sum_{j \in \mathcal{V}_K} p_j & \text{if } i \in \mathcal{V}_K \\ 0 & \text{otherwise} \end{cases}$$

where $\mathcal{V}_K$ is the set of top-K tokens. This eliminates the long tail of low-probability tokens that collectively create "unlikely" text.

Top-P (nucleus) sampling: Dynamically select the smallest set $\mathcal{V}_p$ whose cumulative probability $\geq p$:

$$\mathcal{V}_p = \arg\min_{\mathcal{V}' \subseteq \mathcal{V}} |\mathcal{V}'| \quad \text{s.t.} \quad \sum_{i \in \mathcal{V}'} p_i \geq p$$

Top-P adapts the cutoff to the entropy of the distribution—it keeps more tokens when the model is uncertain (flat distribution) and fewer when confident (peaked distribution).

6.2.3 Typical Sampling

Rather than sampling from the most probable tokens, **typical sampling** selects tokens whose negative log-probability is close to the expected entropy of the distribution:

$$|\log p_i + H(p)| < \tau$$

where $H(p) = -\sum_i p_i \log p_i$ and τ is a threshold. This selects tokens that are "typical" of the distribution rather than simply most likely.

6.2.4 Mirostat

Mirostat dynamically adjusts temperature to maintain a target perplexity τ:

$$T_{t+1} = T_t - \eta \cdot (\text{surprise}_t - \tau)$$

where $\text{surprise}_t = -\log p(\text{chosen token}_t)$. If the model is consistently too surprised (higher perplexity than target), temperature decreases to make sampling more greedy; if too confident, temperature increases. Mirostat converges to a stable entropy without manual tuning.

6.2.5 Contrastive Decoding

Contrastive decoding maximizes the difference between an expert (large) model and an amateur (small) model:

$$p_i \propto \exp((1+\alpha)\log p_{\text{expert}}(x_i) - \alpha \log p_{\text{amateur}}(x_i))$$

This suppresses generations that both models agree are probable (generic, boring text) and emphasizes tokens where the expert disagrees with the amateur (knowledge-rich, specific text).

6.2.6 Beam Search

Beam search maintains B (beam width) most probable partial hypotheses at each step. The score for hypothesis $y_{1:t}$ of length t with length penalty α is:

$$\text{score}(y_{1:t}) = \frac{1}{t^\alpha} \sum_{\ell=1}^{t} \log p(y_\ell \mid y_{<\ell})$$

$\alpha = 0$: no penalty (favors long sequences). $\alpha = 1$: average log-probability. Typical range: $\alpha \in [0.6, 0.8]$. Beam search finds the approximate MAP sequence but can produce degenerate (overly short, repetitive) sequences without careful tuning.

6.2.7 Knowledge Distillation with Temperature

In knowledge distillation (Hinton et al., 2015), the teacher network's softened logits reveal "dark knowledge"—relative similarities between incorrect classes. The student is trained to match the teacher's softened distribution:

$$\mathcal{L}_{\text{KD}} = T^2 \cdot \text{KL}(\text{softmax}(\mathbf{z}_t/T) \parallel \text{softmax}(\mathbf{z}_s/T))$$

- $T > 1$ softens the teacher's output, amplifying the probabilities of near-miss classes.
- The factor T^2 scales the gradient to compensate for the $1/T^2$ reduction from temperature.

- At inference, the student uses $T = 1$ for standard predictions.
- Distillation loss is often combined with supervised cross-entropy: $\mathcal{L} = (1-\lambda)\mathcal{L}_{\text{CE}} + \lambda\mathcal{L}_{\text{KD}}$.

Gradient analysis for KD: The gradient of $\mathcal{L}_{\text{KD}}$ w.r.t. student logits $z_{s,i}$ is:

$$\frac{\partial \mathcal{L}_{\text{KD}}}{\partial z_{s,i}} = T \cdot \left(p_{t,i}^{(T)} - p_{s,i}^{(T)}\right)$$

where $p^{(T)} = \text{softmax}(\mathbf{z}/T)$. As $T \to \infty$, this approaches matching the logit differences: $\frac{\partial \mathcal{L}_{\text{KD}}}{\partial z_{s,i}} \approx \frac{1}{T}[(z_{t,i} - \bar{z}_t) - (z_{s,i} - \bar{z}_s)]$.

6.2.8 Temperature Scaling for Calibration

Temperature scaling (Guo et al., 2017) is a post-hoc calibration method. Given a trained classifier with logits $\mathbf{z}$, find a single temperature T that minimizes the negative log-likelihood on a validation set:

$$T^* = \arg\min_T -\sum_i \log \text{softmax}_{y_i}(z_i/T)$$

This is Platt scaling generalized to multi-class. Temperature scaling does not change the maximum prediction (argmax remains unchanged, so accuracy is unaffected), but it adjusts confidence scores to be better calibrated.

6.3 Log-Softmax

$$\log \text{softmax}(z_i) = z_i - \log \sum_{j=1}^{K} e^{z_j}$$

Log-softmax is numerically more stable than computing softmax followed by log. When combined with negative log-likelihood (NLL) loss, the gradient simplifies to $\hat{y} - y$ (the same as softmax with cross-entropy), but the computation avoids exponentiating then taking the log of near-zero values.

Usage: PyTorch's `F.log_softmax` followed by `F.nll_loss` is the preferred pattern for multi-class classification. It is computationally identical to `F.cross_entropy` but gives access to log-probabilities for intermediate computations.

6.4 Sparsemax (Martins & Astudillo, 2016)

Sparsemax is the Euclidean projection of logits onto the probability simplex:

$$\text{sparsemax}(\mathbf{z}) = \arg\min_{\mathbf{p}\in\Delta^{K-1}} \|\mathbf{p}-\mathbf{z}\|_2^2$$

Unlike softmax, sparsemax can produce **exact zero** probabilities, enabling sparse attention and interpretable classification.
Algorithm:

1. Sort logits descending: $z_{(1)} \geq z_{(2)} \geq \cdots \geq z_{(K)}$.
2. Find the largest index $\rho \in \{1,\ldots,K\}$ satisfying:

$$1 + \rho \cdot z_{(\rho)} > \sum_{j=1}^{\rho} z_{(j)}$$

3. Compute threshold: $\tau = \frac{\sum_{j=1}^{\rho} z_{(j)} - 1}{\rho}$.
4. Output: $p_i = \max(z_i - \tau,\ 0)$.

The algorithm is $O(K \log K)$ due to sorting. The corresponding loss is the sparsemax loss: $\mathcal{L}_{\text{sparsemax}} = \frac{1}{2}\sum_{j:p_j>0}(y_j^2 - z_j^2)$.

Pros: Produces exact zeros (sparse attention/probabilities), differentiable almost everywhere, Euclidean projection has nice geometric interpretation, loss is piecewise linear.
Cons: Requires sorting (bottleneck for large K), non-sparse classes receive mass identical to what softmax would assign (no additional concentration), gradient is zero for non-selected classes.

6.5 Entmax (α-entmax)

Entmax generalizes both softmax and sparsemax through the Tsallis α-entropy:

$$\text{entmax}(\mathbf{z}) = \arg\min_{\mathbf{p}\in\Delta^{K-1}} \mathbf{p}^T(-\mathbf{z}) + H_\alpha^T(\mathbf{p})$$

where the Tsallis α-entropy is:

$$H_\alpha^T(\mathbf{p}) = \begin{cases} \frac{1}{\alpha(\alpha-1)}\left(1 - \sum_j p_j^\alpha\right) & \alpha \neq 1 \\ -\sum_j p_j \log p_j & \alpha = 1 \end{cases}$$

Key special cases:

- $\alpha = 1$: Shannon entropy $\rightarrow$ standard **softmax**.
- $\alpha = 1.5$: 1.5-entmax, which produces sparse outputs with a sparse support comparable to soft attention.
- $\alpha = 2$: Gini entropy $\rightarrow$ **sparsemax**.

Larger α produces sparser outputs. Computation uses an iterative bisection root-finding algorithm on the threshold.

Usage: Sparse attention in transformers (improves interpretability without full sparsemax), multi-label text classification, and any scenario requiring tunable sparsity in probability assignments.

Pros: Tunable sparsity via single parameter α, subsumes softmax and sparsemax as special cases, differentiable, strong theoretical foundation in generalized entropy regularization.
Cons: More complex and slower than softmax (iterative algorithm), requires α tuning as a hyperparameter, less standard tooling support.

6.6 Gumbel-Softmax (Concrete Distribution)

The **Gumbel-Softmax** provides a continuous relaxation of categorical sampling, enabling gradient-based optimization of discrete variables.
Let $\pi_1, \ldots, \pi_K$ be unnormalized probabilities. Sample Gumbel noise $g_i \sim \text{Gumbel}(0, 1)$ via:

$$g_i = -\log(-\log(u_i)), \quad u_i \sim \text{Uniform}(0, 1)$$

Then the Gumbel-Softmax with temperature τ is:

$$y_i = \frac{\exp((\log \pi_i + g_i)/\tau)}{\sum_{j=1}^K \exp((\log \pi_j + g_j)/\tau)}$$

As $\tau \to 0$, y approaches a one-hot sample from Categorical(softmax(π)). As $\tau \to \infty$, y approaches uniform.
Gumbel-Max trick: $\arg\max_i(\log \pi_i + g_i) \sim$ Categorical(softmax(π)) exactly, without any temperature limit.

6.6.1 Straight-Through Gumbel-Softmax

In the **straight-through (ST)** estimator:

- **Forward pass:** Use $\arg\max$ of the Gumbel-Softmax distribution (true one-hot/discrete sample).
- **Backward pass:** Propagate gradients through the continuous Gumbel-Softmax relaxation.

This yields discrete behavior during inference and training forward passes, but meaningful gradients during backpropagation. The gradient is biased but often works well in practice.

Usage:

- **VAEs with discrete latents:** The Gumbel-Softmax enables gradient-based training of VAEs where the latent variables are categorical (e.g., discrete attributes in images).
- **Hard attention:** Selecting exactly one or a fixed number of input tokens rather than soft attention over all.
- **Discrete reinforcement learning:** Learning stochastic policies over discrete action spaces with end-to-end gradient.
- **Neural architecture search:** Selecting operations (convolution, pooling, etc.) as discrete choices.
- **Vector quantized models (VQ-VAE):** Training discrete codebooks via straight-through estimation.

Pros: Enables gradient-based learning of discrete latent variables, simple implementation, controllable relaxation tightness via τ annealing, straight-through variant gives truly discrete forward pass.
Cons: Biased gradient estimator (ST estimator does not follow the true gradient of the discrete function), high variance when τ is small, requires careful temperature annealing schedule, and Gumbel noise injection introduces stochasticity that can slow convergence.

6.7 Hierarchical Softmax

Hierarchical softmax organizes the vocabulary of size V into a binary tree (usually a Huffman tree based on word frequencies). The probability of word w given context $\mathbf{h}$ is:

$$p(w \mid \mathbf{h}) = \prod_{j=1}^{L(w)-1} \sigma\Big([\![n(w,j+1) = \mathrm{ch}(n(w,j))]\!] \cdot \mathbf{v}_{n(w,j)}^T \mathbf{h}\Big)$$

where:

- $L(w)$ is the path length from root to leaf w.
- $n(w,j)$ is the j-th node on the path from root to w (where $n(w,1) =$ root, $n(w,L(w)) = w$).

- $\mathrm{ch}(n)$ is the left child of node n.
- $[\![\cdot]\!]$ is $+1$ if the next step goes left, -1 if right.
- $\mathbf{v}_n$ are per-node learned vectors ($\log V$ vectors vs. V in standard softmax).
- $\sigma(x) = 1/(1+e^{-x})$ is the logistic sigmoid.

Usage: Word2Vec (Skip-gram and CBOW) for training word embeddings with very large vocabularies (millions of words). At each step, the model evaluates roughly $\log_2 V$ binary classifiers instead of computing a V-way softmax.

Pros: Reduces computational complexity from $O(V)$ to $O(\log V)$, memory-efficient for large vocabularies, naturally encodes semantic hierarchies.
Cons: Tree quality matters (frequency-balanced Huffman tree is optimal but potentially biased), each word has a different computational path length, still slower than sampled softmax in practice for very large V on GPUs, and modern subword tokenization has reduced its relevance.

6.8 Sampled Softmax and Noise Contrastive Estimation (NCE)

Sampled softmax approximates the full softmax by computing the denominator over only the true class and a small set of K randomly sampled negative classes:

$$p(\text{true} \mid \mathbf{h}) \approx \frac{e^{\mathbf{w}_{\text{true}}^T \mathbf{h}}}{\sum_{k \in \{\text{true}\} \cup \mathcal{S}} e^{\mathbf{w}_k^T \mathbf{h}}}$$

where $|\mathcal{S}| = K \ll V$. The gradient is an unbiased estimator of the true gradient (with appropriate importance sampling corrections).
Noise Contrastive Estimation (NCE): Reframes density estimation as binary classification between real data samples drawn from p_{data} and noise samples drawn from a known noise distribution p_{noise}:

$$p(D = 1 \mid w, \mathbf{h}) = \frac{p_\theta(w \mid \mathbf{h})}{p_\theta(w \mid \mathbf{h}) + k \cdot p_{\text{noise}}(w)}$$

where k is the ratio of noise to data samples. NCE learns the unnormalized model without requiring the partition function computation.

Usage: Training language models with large vocabularies, word embeddings, and neural machine translation before hierarchical softmax and subword tokenization became standard.

6.9 Margin-Based Softmax (Face Recognition Family)

Margin-based softmax variants are crucial for deep face recognition, where the goal is to maximize inter-class separation and minimize intra-class variance in the embedding space.

Let $\mathbf{x}_i$ be the i-th input, $\mathbf{W}_j$ be the j-th class weight vector (prototype), and s, m be scale and margin hyperparameters. All variants normalize both features and weights to lie on the unit hypersphere ($\|\mathbf{x}_i\| = 1$, $\|\mathbf{W}_j\| = 1$), and the logit for class j becomes $s \cdot \cos\theta_j$ where θ_j is the angle between $\mathbf{x}_i$ and $\mathbf{W}_j$.

6.9.1 ArcFace (Additive Angular Margin)

$$\mathcal{L}_{\text{ArcFace}} = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{e^{s\cdot\cos(\theta_{y_i}+m)}}{e^{s\cdot\cos(\theta_{y_i}+m)} + \sum_{j\neq y_i} e^{s\cdot\cos\theta_j}}$$

The margin m is added directly in angular space: the target logit becomes $s\cdot\cos(\theta_{y_i}+m)$, which is strictly smaller than the standard $s\cdot\cos\theta_{y_i}$, forcing the network to learn more discriminative features to compensate. Typical values: $s = 64$, $m = 0.5$.

6.9.2 CosFace / AM-Softmax (Additive Margin Softmax)

$$\mathcal{L}_{\text{CosFace}} = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{e^{s\cdot(\cos\theta_{y_i}-m)}}{e^{s\cdot(\cos\theta_{y_i}-m)} + \sum_{j\neq y_i} e^{s\cdot\cos\theta_j}}$$

Here the margin is subtracted from the cosine value directly, rather than applied inside the cosine. This is theoretically simpler and empirically competitive with ArcFace. Typical values: $s = 30$, $m = 0.35$.

6.9.3 SphereFace (Multiplicative Angular Margin)

$$\mathcal{L}_{\text{SphereFace}} = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{e^{s\cdot\cos(m\cdot\theta_{y_i})}}{e^{s\cdot\cos(m\cdot\theta_{y_i})} + \sum_{j\neq y_i} e^{s\cdot\cos\theta_j}}$$

The margin m multiplies the angle, which is non-uniform across angles (larger effect at small angles). This makes optimization harder. Monotonically decreasing for $\theta \in [0, \pi/m]$, so m is restricted. Largely superseded by ArcFace and CosFace.

6.9.4 AdaFace, MagFace, CurricularFace, ElasticFace

AdaFace: Adapts the margin based on image quality (estimated from the norm of feature embeddings). Low-quality images get smaller margins to avoid overfitting; high-quality images get larger margins.
MagFace: Learns feature magnitude (norm) as a proxy for sample quality, applying larger margins to easier (higher-magnitude) samples. Loss has a magnitude-aware angular margin and a magnitude regularization term.
CurricularFace: Applies curriculum learning to the margin—easy samples get hard margins, hard samples get easy margins. Uses an adaptive mechanism that weights positive and negative cosine similarities based on difficulty.
ElasticFace: Extends CosFace with an elastic margin that varies per sample, learned as part of the optimization. This allows the model to assign different margins to different samples based on their difficulty.

Unified margin framework: All margin-based losses can be expressed as:

$$\mathcal{L} = -\log \frac{e^{s \cdot f(\cos\theta_{y_i}, m)}}{e^{s \cdot f(\cos\theta_{y_i}, m)} + \sum_{j \neq y_i} e^{s \cdot \cos\theta_j}}$$

where $f(\cos\theta, m)$ is a margin function:

- ArcFace: $f(x, m) = \cos(\arccos(x) + m)$
- CosFace: $f(x, m) = x - m$
- SphereFace: $f(x, m) = \cos(m \cdot \arccos(x))$
- Softmax (no margin): $f(x, m) = x$

6.10 Comparison Table

Choosing a softmax variant:

- **Standard classification:** Classic softmax with cross-entropy is the safe default.
- **LLM text generation:** Temperature scaling + Top-P or Mirostat.
- **Knowledge distillation:** Temperature-scaled softmax with $T \in [2, 5]$.
- **Interpretable attention:** Sparsemax or 1.5-entmax for sparse attention weights.
- **Discrete latents:** Gumbel-Softmax with straight-through estimator.
- **Large vocabulary ($V > 10^5$):** Sampled softmax or hierarchical softmax.
- **Face/biometric recognition:** ArcFace or CosFace with $s = 64, m = 0.5$.
- **Calibrated probabilities:** Temperature scaling as a post-processing step.
- **Controllable sparsity:** Entmax with tunable α.

Table 6.1: Comparison of Softmax Variants

Variant	**Sparse?**	**Complexity**	**Margin?**	**Key Application**
Classic Softmax	No	$O(K)$	No	General classification, attention
Temp-Scaled	No	$O(K)$	No	LLM sampling, KD, calibration
Log-Softmax	No	$O(K)$	No	Numerically stable classification
Sparsemax	**Yes**	$O(K \log K)$	No	Sparse attention, interpretable NLP
α-Entmax	Tunable	$O(K \log K)$	No	Tunable sparse attention
Gumbel-Softmax	No (cont.)	$O(K)$	No	Discrete VAEs, RL, NAS
Straight-Through GS	**Yes** (disc.)	$O(K)$	No	Hard attention, VQ-VAE
Hierarchical Softmax	No	$O(\log K)$	No	Word2Vec, large-vocab LM
Sampled Softmax	No (approx.)	$O(K')$, $K' \ll K$	No	Large vocabulary LMs, NCE
ArcFace / CosFace	No	$O(K)$	**Yes**	Face recognition (FR)

Chapter 7

Probability Distributions in Machine Learning

Probability distributions are the mathematical foundation of machine learning—they model uncertainty, define generative processes, and serve as priors, likelihoods, and posteriors in Bayesian inference. This chapter surveys the major distributions used in ML, their formal definitions, and their practical roles.

7.1 Discrete Distributions

7.1.1 Bernoulli Distribution

The **Bernoulli distribution** models a single binary trial with success probability $\mu \in [0, 1]$:

$$\text{PMF:} \quad P(x \mid \mu) = \mu^x (1-\mu)^{1-x}, \quad x \in \{0, 1\}$$

$$\mathbb{E}[x] = \mu, \quad \text{Var}(x) = \mu(1-\mu)$$

ML Usage:

- **Binary classification:** The output of a sigmoid activation models $P(y = 1 \mid x) = \sigma(\mathbf{w}^T \mathbf{x} + b)$, which is a Bernoulli conditional distribution.
- **Binary cross-entropy loss:** $-\frac{1}{N} \sum [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$ is the negative log-likelihood of a Bernoulli model.
- **VAE decoders:** For binary images (e.g., binarized MNIST), the decoder outputs a Bernoulli parameter per pixel.
- **Reinforcement learning:** Bandit problems model each arm as a Bernoulli reward.
- **Conjugate prior:** The Beta distribution is conjugate to the Bernoulli likelihood.

7.1.2 Categorical / Multinoulli Distribution

The **Categorical distribution** models a single draw from K categories:

$$\text{PMF:} \quad P(x \mid \mathbf{p}) = \prod_{k=1}^{K} p_k^{\mathbf{1}[x=k]}, \quad \mathbf{p} \in \Delta^{K-1}, \quad x \in \{1, \ldots, K\}$$

$$\mathbb{E}[\mathbf{x}] = \mathbf{p}, \quad \text{Cov}(\mathbf{x}) = \text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^T$$

where $\mathbf{x}$ is the one-hot encoding of x.

ML Usage:

- **Multi-class classification:** Softmax output parameterizes a Categorical distribution over K classes.
- **Language modeling:** Each token prediction is a Categorical over the vocabulary (typically $V > 30{,}000$).
- **Policy gradient RL:** Discrete action policies sample from a Categorical distribution.
- **Mixture models:** The latent assignment variable $z \in \{1, \ldots, K\}$ in GMMs follows a Categorical with parameters π_k.
- **Conjugate prior:** The Dirichlet distribution is conjugate to the Categorical/Multinomial family.

7.1.3 Binomial Distribution

The **Binomial distribution** counts successes in n independent Bernoulli trials:

$$\text{PMF:} \quad P(k \mid n, p) = \binom{n}{k} p^k (1-p)^{n-k}, \quad k \in \{0, 1, \ldots, n\}$$

$$\mathbb{E}[k] = np, \quad \text{Var}(k) = np(1-p)$$

ML Usage: Ensemble voting (counting how many models predict class k), A/B testing (sample size calculations), modeling count data where the maximum count is bounded by n.

7.1.4 Multinomial Distribution

The **Multinomial distribution** counts the outcomes of n independent Categorical trials:

$$\text{PMF:} \quad P(\mathbf{x} \mid n, \mathbf{p}) = \frac{n!}{\prod_{k=1}^{K} x_k!} \prod_{k=1}^{K} p_k^{x_k}, \quad \sum_k x_k = n$$

$$\mathbb{E}[\mathbf{x}] = n\mathbf{p}, \quad \text{Cov}(\mathbf{x}) = n(\text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^T)$$

ML Usage: Bag-of-words document modeling (how many times each word appears), Naive Bayes for text classification (multinomial NB), count-based data modeling.

7.1.5 Poisson Distribution

The **Poisson distribution** models the number of events occurring in a fixed interval, given rate $\lambda > 0$:

$$\text{PMF:} \quad P(k \mid \lambda) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k \in \{0, 1, 2, \ldots\}$$

$$\mathbb{E}[k] = \lambda, \quad \text{Var}(k) = \lambda$$

The Poisson's equality of mean and variance (equidispersion) is both a defining property and a limitation.

ML Usage: Modeling count data (customer arrivals, clicks per day), Poisson regression (GLM with log link), modeling neuron spike counts in computational neuroscience, and as the loss function for count prediction tasks (Poisson NLL: $\hat{y} - y \log \hat{y}$).

7.1.6 Geometric Distribution

The **Geometric distribution** models the number of Bernoulli trials until the first success:

$$\text{PMF (trials form):} \quad P(k \mid p) = (1-p)^{k-1} p, \quad k \in \{1, 2, \dots\}$$

$$\mathbb{E}[k] = \frac{1}{p}, \quad \text{Var}(k) = \frac{1-p}{p^2}$$

7.1.7 Negative Binomial Distribution

The **Negative Binomial distribution** models the number of failures until r successes:

$$\text{PMF:} \quad P(k \mid r, p) = \binom{k+r-1}{k} (1-p)^k p^r, \quad k \in \{0, 1, \dots\}$$

$$\mathbb{E}[k] = \frac{r(1-p)}{p}, \quad \text{Var}(k) = \frac{r(1-p)}{p^2}$$

ML Usage: Negative binomial is preferred over Poisson for overdispersed count data ($\text{Var} > \mathbb{E}$) because it has a separate variance parameter. Used in scRNA-seq gene expression modeling, GLMs for overdispersed counts, and as a flexible count likelihood.

7.2 Continuous Distributions on $\mathbb{R}$

7.2.1 Gaussian (Normal) Distribution

The **univariate Gaussian** with mean μ and variance σ^2:

$$p(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad x \in \mathbb{R}$$

$$\mathbb{E}[x] = \mu, \quad \text{Var}(x) = \sigma^2, \quad \text{Mode} = \text{Median} = \mu$$

The **multivariate Gaussian** ($\mathbf{x} \in \mathbb{R}^d$):

$$p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{(2\pi)^d |\boldsymbol{\Sigma}|}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)$$

$$\mathbb{E}[\mathbf{x}] = \boldsymbol{\mu}, \quad \text{Cov}(\mathbf{x}) = \boldsymbol{\Sigma}$$

The precision matrix $\boldsymbol{\Lambda} = \boldsymbol{\Sigma}^{-1}$ is often used in conditioning formulas.

ML Usage (pervasive):

- **VAE:** Latent variable prior $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ and encoder output $q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}^2_\phi(\mathbf{x})))$.
- **Diffusion models:** Forward process adds Gaussian noise; reverse process denoises by predicting the added Gaussian noise.
- **Normalizing flows:** Base distribution is standard Gaussian; the flow transforms it to the data distribution.
- **Gaussian processes:** Prior over functions with a Gaussian joint distribution.
- **Bayesian neural networks:** Weight priors are typically Gaussian.
- **Kalman filter:** All transition and observation models are linear-Gaussian.
- **Regression:** Assuming Gaussian errors gives the MSE loss (MLE = OLS).
- **Conjugate prior:** Gaussian prior + Gaussian likelihood $\rightarrow$ Gaussian posterior.

Reparameterization trick for Gaussian: $z = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. This separates the stochasticity ($\boldsymbol{\epsilon}$) from the learnable parameters ($\boldsymbol{\mu}, \boldsymbol{\sigma}$), enabling backpropagation through the sampling operation.

7.2.2 Laplace (Double Exponential) Distribution

$$p(x \mid \mu, b) = \frac{1}{2b} \exp\left(-\frac{|x-\mu|}{b}\right), \quad x \in \mathbb{R},\ b > 0$$

$$\mathbb{E}[x] = \mu, \quad \text{Var}(x) = 2b^2$$

The Laplace distribution has heavier tails than the Gaussian (exponential decay vs. squared exponential), making it more robust to outliers.

ML Usage: L1 regularization (Laplace prior on weights induces sparsity), robust regression (MAE loss = MLE under Laplace noise), differential privacy (Laplace mechanism adds noise with scale $b = \Delta f / \epsilon$), and VAE with Laplace output distribution for images.

7.2.3 Cauchy Distribution

$$p(x \mid x_0, \gamma) = \frac{1}{\pi\gamma}\left[1 + \left(\frac{x - x_0}{\gamma}\right)^2\right]^{-1}, \quad x \in \mathbb{R},\ \gamma > 0$$

The Cauchy has **no defined mean or variance**—its tails are so heavy that the integrals diverge. The median and mode both equal x_0. It is a t-distribution with $\nu = 1$ degree of freedom.

ML Usage: Robust priors when extreme outliers are expected, prior for variance parameters in hierarchical models (half-Cauchy is recommended by Gelman), and simulation of heavy-tailed noise. Also arises as the ratio of two standard normal variables.

7.2.4 Student's t-Distribution

$$p(x \mid \nu, \mu, \sigma) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\Gamma\left(\frac{\nu}{2}\right)\sqrt{\nu\pi}\,\sigma}\left[1 + \frac{1}{\nu}\left(\frac{x - \mu}{\sigma}\right)^2\right]^{-\frac{\nu+1}{2}}$$

$$\mathbb{E}[x] = \mu \ \ (\nu > 1), \quad \mathrm{Var}(x) = \frac{\nu}{\nu - 2}\sigma^2 \ \ (\nu > 2)$$

As $\nu \to \infty$, it converges to $\mathcal{N}(\mu, \sigma^2)$. Small ν gives heavy tails. Degrees of freedom ν controls tail weight.

ML Usage:

- **Robust regression:** Replacing Gaussian likelihood with t-distribution to handle outliers.
- **t-SNE:** The low-dimensional similarity in t-SNE uses a Student-t distribution (with $\nu = 1$, i.e., Cauchy) to alleviate the crowding problem.
- **Bayesian modeling:** Robust alternative to Gaussian for likelihood and priors.
- **Conjugate prior:** Gaussian-Gamma prior $\to$ t-distribution marginal (when variance is integrated out).

7.2.5 Logistic Distribution

$$p(x \mid \mu, s) = \frac{e^{-(x-\mu)/s}}{s(1 + e^{-(x-\mu)/s})^2}, \quad x \in \mathbb{R},\ s > 0$$

$$\mathbb{E}[x] = \mu, \quad \mathrm{Var}(x) = \frac{s^2\pi^2}{3}$$

The CDF is the logistic sigmoid: $F(x) = \frac{1}{1+e^{-(x-\mu)/s}}$.

ML Usage: Logistic regression is named after this distribution: assuming a latent variable $y^* = \mathbf{w}^T\mathbf{x} + \epsilon$ with $\epsilon \sim \text{Logistic}(0, 1)$ gives the logistic model $P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x})$. Also used in Gumbel-Softmax as an alternative to Gumbel noise.

7.2.6 Gumbel Distribution

The **Gumbel** (type-I extreme value) distribution:

$$p(x \mid \mu, \beta) = \frac{1}{\beta} \exp\left(-\frac{x-\mu}{\beta} - e^{-(x-\mu)/\beta}\right), \quad x \in \mathbb{R},\ \beta > 0$$

$$\mathbb{E}[x] = \mu + \beta\gamma \ \ (\gamma \approx 0.5772 \text{ Euler-Mascheroni}), \quad \text{Var}(x) = \frac{\pi^2}{6}\beta^2$$

ML Usage: The Gumbel-Max trick for categorical sampling (see Chapter 6), modeling extreme events (floods, earthquakes via extreme value theory), and as the noise distribution in Gumbel-Softmax for discrete VAEs.

7.3 Positive-Support Distributions ($\mathbb{R}^+$)

7.3.1 Exponential Distribution

$$p(x \mid \lambda) = \lambda e^{-\lambda x}, \quad x \geq 0,\ \lambda > 0$$

$$\mathbb{E}[x] = \frac{1}{\lambda}, \quad \text{Var}(x) = \frac{1}{\lambda^2}$$

Memoryless property: $P(X > s + t \mid X > s) = P(X > t)$. The only continuous distribution with this property. Conjugate prior: Gamma.

7.3.2 Gamma Distribution

$$p(x \mid \alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}, \quad x > 0,\ \alpha, \beta > 0$$

$$\mathbb{E}[x] = \frac{\alpha}{\beta}, \quad \text{Var}(x) = \frac{\alpha}{\beta^2}$$

Shape α and rate β (or scale $\theta = 1/\beta$). Special cases: $\alpha = 1$ gives Exponential, $\alpha = k/2, \beta = 1/2$ gives Chi-squared with k df.

ML Usage:

- **Conjugate prior:** Gamma is conjugate to the precision $1/\sigma^2$ of a Gaussian, and to the rate λ of Poisson and Exponential.
- **Bayesian neural networks:** Gamma prior on inverse variance for Automatic Relevance Determination (ARD).
- **Latent Dirichlet Allocation:** Gamma random variables are used to sample Dirich-

let (via normalization of independent Gamma samples).

- **NMF (Non-negative Matrix Factorization):** Gamma priors promote sparsity.

7.3.3 Inverse Gamma Distribution

If $X \sim \text{Gamma}(\alpha, \beta)$, then $Y = 1/X \sim \text{Inv-Gamma}(\alpha, \beta)$:

$$p(y \mid \alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)} y^{-(\alpha+1)} e^{-\beta/y}, \quad y > 0$$

$$\mathbb{E}[y] = \frac{\beta}{\alpha - 1} \ (\alpha > 1), \quad \text{Var}(y) = \frac{\beta^2}{(\alpha-1)^2(\alpha-2)} \ (\alpha > 2)$$

Conjugate prior for the *variance* σ^2 of a Gaussian (as opposed to precision).

7.3.4 Chi-Squared Distribution (χ^2)

$$p(x \mid k) = \frac{1}{2^{k/2}\Gamma(k/2)} x^{k/2-1} e^{-x/2}, \quad x > 0, \ k \in \mathbb{N}$$

$$\mathbb{E}[x] = k, \quad \text{Var}(x) = 2k$$

Sum of k squared independent standard normals.

ML Usage: Hypothesis testing (likelihood ratio tests), feature selection (chi-squared test for independence), and as a special case of Gamma priors.

7.3.5 Weibull Distribution

$$p(x \mid \lambda, k) = \frac{k}{\lambda} \left(\frac{x}{\lambda}\right)^{k-1} e^{-(x/\lambda)^k}, \quad x \geq 0$$

$k < 1$: decreasing hazard (infant mortality). $k = 1$: Exponential. $k > 1$: increasing hazard (aging). $k = 2$: Rayleigh.

ML Usage: Survival analysis, reliability engineering, wind speed modeling (for renewable energy prediction), and modeling time-to-event with non-constant hazard rates.

7.3.6 Log-Normal Distribution

If $\log X \sim \mathcal{N}(\mu, \sigma^2)$, then $X \sim \text{LogNormal}(\mu, \sigma^2)$:

$$p(x \mid \mu, \sigma^2) = \frac{1}{x\sigma\sqrt{2\pi}} \exp\left(-\frac{(\log x - \mu)^2}{2\sigma^2}\right), \quad x > 0$$

$$\mathbb{E}[x] = e^{\mu + \sigma^2/2}, \quad \text{Var}(x) = (e^{\sigma^2} - 1)e^{2\mu+\sigma^2}$$

ML Usage: Modeling positive, right-skewed data (income, city sizes, citation counts), financial modeling (stock prices under Black-Scholes), and as the output distribution for positive regression targets.

7.3.7 Pareto Distribution

$$p(x \mid x_m, \alpha) = \frac{\alpha x_m^\alpha}{x^{\alpha+1}}, \quad x \geq x_m > 0,\ \alpha > 0$$

Heavy-tailed (power law decay). Mean exists only if $\alpha > 1$, variance only if $\alpha > 2$. The 80-20 rule: with $\alpha \approx 1.16$, the top 20% holds 80% of the mass.

ML Usage: Modeling extreme events, wealth distribution, file sizes on the internet, and heavy-tailed priors for robustness.

7.4 Bounded and Simplex Distributions

7.4.1 Beta Distribution

$$p(x \mid \alpha, \beta) = \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} x^{\alpha-1}(1-x)^{\beta-1}, \quad x \in (0,1),\ \alpha, \beta > 0$$

$$\mathbb{E}[x] = \frac{\alpha}{\alpha+\beta}, \quad \text{Var}(x) = \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}, \quad \text{Mode} = \frac{\alpha-1}{\alpha+\beta-2}\ (\alpha, \beta > 1)$$

ML Usage:

- **Conjugate prior:** Beta is conjugate to Bernoulli and Binomial. Update rule: Beta(a, b)+ observe k successes in n trials $\to$ Beta$(a + k, b + n - k)$.
- **Bayesian A/B testing:** Model conversion rates as Beta distributions; compute $P(p_A > p_B)$ analytically.
- **Thompson sampling:** Sample from Beta posterior, choose arm with highest sample.
- **VAEs:** Beta-VAE uses Beta distribution for bounded latent variables.
- **Mixture models:** Beta mixture for bounded data (e.g., proportions, ratings).

7.4.2 Dirichlet Distribution

The **Dirichlet distribution** is the multivariate generalization of Beta to the $(K-1)$-simplex:

$$p(\mathbf{x} \mid \boldsymbol{\alpha}) = \frac{\Gamma(\sum_{k=1}^K \alpha_k)}{\prod_{k=1}^K \Gamma(\alpha_k)} \prod_{k=1}^K x_k^{\alpha_k - 1}, \quad \mathbf{x} \in \Delta^{K-1},\ \alpha_k > 0$$

$$\mathbb{E}[x_k] = \frac{\alpha_k}{\alpha_0}, \quad \text{Var}(x_k) = \frac{\alpha_k(\alpha_0 - \alpha_k)}{\alpha_0^2(\alpha_0 + 1)}, \quad \text{where } \alpha_0 = \sum_{k=1}^{K} \alpha_k$$

The concentration parameter α_0 controls how peaked the distribution is: large α_0 concentrates mass near the mean; small α_0 pushes mass to simplex corners (sparse vectors).

ML Usage:

- **Latent Dirichlet Allocation (LDA):** Dirichlet prior over document-topic and topic-word distributions. The Dirichlet's conjugacy to the Multinomial enables collapsed Gibbs sampling.
- **Conjugate prior:** Dirichlet is conjugate to Categorical and Multinomial. Posterior: Dirichlet($\boldsymbol{\alpha} + \mathbf{c}$) where $\mathbf{c}$ are observed counts.
- **Mixture models:** Dirichlet prior over mixture weights $\boldsymbol{\pi}$.
- **Attention sparsity:** Dirichlet prior on attention weights to encourage focus on fewer tokens.
- **Neural network uncertainty:** Dirichlet output for evidential deep learning (predicting a distribution over class probabilities).
- **Bayesian topic models and HDP:** Hierarchical Dirichlet Process for nonparametric topic models.

7.4.3 Kumaraswamy Distribution

$$p(x \mid a, b) = abx^{a-1}(1 - x^a)^{b-1}, \quad x \in (0, 1),\ a, b > 0$$

Similar shape to Beta but with a closed-form CDF $F(x) = 1 - (1 - x^a)^b$ and quantile function. This makes the reparameterization trick straightforward: $x \sim \text{Kumaraswamy}(a, b)$ via $x = (1 - U^{1/b})^{1/a}$ with $U \sim \text{Uniform}(0, 1)$.

ML Usage: VAE with bounded latent variables (differentiable, reparameterizable alternative to Beta), deep latent Gaussian models with bounded support.

7.5 Spherical and Directional Distributions

7.5.1 von Mises-Fisher (vMF) Distribution

The **von Mises-Fisher distribution** is the analogue of the Gaussian on the unit hypersphere $\mathbb{S}^{d-1}$:

$$p(\mathbf{x} \mid \boldsymbol{\mu}, \kappa) = C_d(\kappa)\, e^{\kappa \boldsymbol{\mu}^T \mathbf{x}}, \quad \|\mathbf{x}\|_2 = 1,\ \|\boldsymbol{\mu}\|_2 = 1,\ \kappa \geq 0$$

where the normalization constant $C_d(\kappa)$ involves the modified Bessel function:

$$C_d(\kappa) = \frac{\kappa^{d/2-1}}{(2\pi)^{d/2} I_{d/2-1}(\kappa)}$$

κ is the concentration parameter: $\kappa = 0$ gives uniform on sphere; $\kappa \to \infty$ concentrates at $\boldsymbol{\mu}$.

ML Usage:

- **Hyperspherical embeddings:** Word embeddings normalized to unit sphere (cosine similarity). Face recognition embeddings are optimized on the hypersphere (CosFace, ArcFace).
- **Variational autoencoders:** vMF-VAE uses vMF as the latent distribution, suitable for directional data.
- **Text embeddings:** Sentence-BERT outputs are often L2-normalized, making the von Mises-Fisher a natural distributional model.
- **Directional statistics:** Wind direction, protein backbone angles, geological fracture orientations.

7.5.2 Kent, Bingham, and Watson Distributions

Kent (FB5): Elliptically symmetric on the sphere, with a mean direction $\boldsymbol{\mu}$, concentration κ, and ovalness β. Analogue of the bivariate normal conditioned to have unit norm.
Bingham: Antipodally symmetric (sign-agnostic: $p(\mathbf{x}) = p(-\mathbf{x})$). Useful for modeling axes, not directions (e.g., geological cleavage planes). Parameterized by an orthogonal matrix and concentration eigenvalues.
Watson: Bipolar or girdle distribution, a special case of Bingham with rotational symmetry. Models directions that cluster around an axis or girdle around an equator.

7.6 Matrix Distributions

7.6.1 Wishart Distribution

The **Wishart distribution** is the multivariate generalization of the Chi-squared distribution. For n i.i.d. vectors $\mathbf{x}_i \sim \mathcal{N}_d(\mathbf{0}, \boldsymbol{V})$, the scatter matrix $\mathbf{S} = \sum_{i=1}^{n} \mathbf{x}_i \mathbf{x}_i^T$ follows $\mathbf{S} \sim \mathcal{W}_d(\boldsymbol{V}, n)$ with PDF proportional to $|\mathbf{S}|^{(n-d-1)/2} e^{-\operatorname{Tr}(\boldsymbol{V}^{-1}\mathbf{S})/2}$.

ML Usage: Conjugate prior for the precision matrix $\boldsymbol{\Sigma}^{-1}$ of a multivariate Gaussian. Used in Bayesian covariance estimation and Gaussian graphical models.

7.6.2 LKJ Prior

The **LKJ** (Lewandowski-Kurowicka-Joe) prior is a distribution over correlation matrices $\mathbf{R}$ (not covariances):

$$p(\mathbf{R} \mid \eta) \propto |\mathbf{R}|^{\eta-1}$$

$\eta = 1$: uniform over correlation matrices. $\eta > 1$: concentrates toward identity. $0 < \eta < 1$: favors lower correlations.

ML Usage: Prior for the correlation matrix in hierarchical models, Stan's default prior for correlation matrices, and Bayesian factor analysis.

7.7 Mixture Distributions

7.7.1 Gaussian Mixture Model (GMM)

$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \quad \sum_{k=1}^{K} \pi_k = 1, \ \pi_k \geq 0$$

GMMs are universal density approximators: any continuous density can be approximated arbitrarily well with enough Gaussian components.

ML Usage: Clustering (soft assignments via posterior responsibility $\gamma_{ik} = P(z_i = k|\mathbf{x}_i)$), anomaly detection (low density regions), data generation, initialization for more complex models, and as a building block for mixture of experts.

7.7.2 Dirichlet Process Mixture (Infinite Mixture)

The **Dirichlet Process (DP)** provides a prior over discrete distributions with countably infinite support. The **stick-breaking construction**:

$$v_k \sim \text{Beta}(1, \alpha), \quad \pi_k = v_k \prod_{j=1}^{k-1}(1 - v_j), \quad \sum_{k=1}^{\infty} \pi_k = 1$$

The **Chinese Restaurant Process (CRP)** is the predictive distribution: $P(z_i = k \mid \mathbf{z}_{<i}) \propto n_k$ (existing) or α (new table).

ML Usage: Nonparametric clustering (number of clusters not fixed a priori), topic modeling (HDP), and Bayesian nonparametric density estimation.

7.8 Reparameterization Tricks

Gaussian reparameterization:

$$\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

This allows $\nabla_\phi \mathbf{z}$ to flow through $\boldsymbol{\mu}_\phi, \boldsymbol{\sigma}_\phi$ to the encoder. The KL divergence $\mathrm{KL}(q_\phi(\mathbf{z}|\mathbf{x})\|p(\mathbf{z}))$ with standard Gaussian prior is:

$$\mathrm{KL} = -\frac{1}{2}\sum_{j=1}^{d}\left(1 + \log\sigma_j^2 - \mu_j^2 - \sigma_j^2\right)$$

Gumbel-Softmax reparameterization (see Chapter 6):

$$y_i = \frac{\exp((\log\pi_i + g_i)/\tau)}{\sum_j \exp((\log\pi_j + g_j)/\tau)}, \quad g_i = -\log(-\log(u_i)),\ u_i \sim U(0,1)$$

Score function (REINFORCE) estimator:

$$\nabla_\phi \mathbb{E}_{q_\phi(\mathbf{z})}[f(\mathbf{z})] = \mathbb{E}_{q_\phi(\mathbf{z})}[f(\mathbf{z})\nabla_\phi \log q_\phi(\mathbf{z})]$$

Does not require f to be differentiable w.r.t. $\mathbf{z}$, but has high variance. Variance reduction via baseline subtraction: $\mathbb{E}[(f(\mathbf{z}) - b)\nabla_\phi \log q_\phi(\mathbf{z})]$.
Implicit reparameterization gradients (Figurnov et al., 2018): For distributions without explicit reparameterizations, use the gradient of the standardized CDF:

$$\nabla_\phi z = -\frac{\nabla_\phi F(z \mid \phi)}{p(z \mid \phi)}$$

where F is the CDF. Works for Gamma, Beta, Dirichlet (with appropriate standardizations).

7.9 Exponential Family

A distribution belongs to the **exponential family** if its density can be written as:

$$p(\mathbf{x} \mid \boldsymbol{\eta}) = h(\mathbf{x})\exp\left(\boldsymbol{\eta}^T\mathbf{T}(\mathbf{x}) - A(\boldsymbol{\eta})\right)$$

where:

- $\boldsymbol{\eta}$: natural parameters.
- $\mathbf{T}(\mathbf{x})$: sufficient statistics.
- $A(\boldsymbol{\eta})$: log-partition function (log-normalizer), ensuring $\int p = 1$.
- $h(\mathbf{x})$: base measure.

Key properties of exponential family:

- **Moment-generating:** $\mathbb{E}[\mathbf{T}(\mathbf{x})] = \nabla_{\boldsymbol{\eta}} A(\boldsymbol{\eta})$ and $\mathrm{Cov}(\mathbf{T}(\mathbf{x})) = \nabla^2_{\boldsymbol{\eta}} A(\boldsymbol{\eta})$.
- **Convexity:** $A(\boldsymbol{\eta})$ is strictly convex $\implies$ log-likelihood is concave, MLE is unique.
- **Conjugate priors:** The conjugate prior has the form $p(\boldsymbol{\eta}) \propto \exp(\boldsymbol{\nu}^T\boldsymbol{\eta} - \nu_0 A(\boldsymbol{\eta}))$.
- **Maximum entropy:** Given fixed expected sufficient statistics, the exponential family distribution maximizes entropy.

- **Bregman divergences:** $A(\boldsymbol{\eta})$ generates a Bregman divergence that equals the KL divergence between two exponential family members.
- **Pitman-Koopman-Darmois theorem:** Exponential family are the only distributions with sufficient statistics whose dimension remains bounded as sample size grows.

Exponential family membership: Almost all distributions in this chapter belong to the exponential family with fixed parameters: Gaussian, Bernoulli, Categorical, Binomial, Multinomial, Poisson, Exponential, Gamma, Beta, Dirichlet, Wishart, von Mises-Fisher, Geometric, Negative Binomial, Pareto (with fixed x_m), Laplace (with fixed μ), Weibull (with fixed k), Logistic, Log-Normal, Chi-squared, and Inverse Gamma. Notable exceptions include Student's t (scale mixture of Gaussians), Cauchy, and mixture distributions.

7.10 Summary Table

Choosing a distribution:

- **Binary outcome:** Bernoulli with Beta prior (or sigmoid/Bernoulli likelihood).
- **Multi-class outcome:** Categorical with Dirichlet prior (or softmax/Categorical likelihood).
- **Count data:** Poisson for equidispersed; Negative Binomial for overdispersed.
- **Real-valued, unbounded:** Gaussian (default), Student's t (robust), Laplace (sparsity).
- **Positive, right-skewed:** Log-Normal, Gamma, or Exponential.
- **Bounded [0,1]:** Beta or Kumaraswamy (if reparameterization needed).
- **Simplex (probabilities):** Dirichlet.
- **Directional/unit-sphere:** von Mises-Fisher.
- **Large-vocabulary LM:** Categorical (with sampled/hierarchical softmax).
- **Image pixels:** Bernoulli (binary) or Gaussian/Laplace (continuous, normalized).
- **Neural net weights (prior):** Gaussian (standard BNN), Laplace (sparse BNN), or horseshoe (sparsity).

Table 7.1: Comprehensive Summary of Probability Distributions

Distribution	**Support**	$\mathbb{E}$	Var	**Key ML Usage**	**Exp Fam?**
Bernoulli	$\{0,1\}$	μ	$\mu(1-\mu)$	Binary classification	Yes
Categorical	$\{1,\dots,K\}$	$\mathbf{p}$	$\text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^T$	Multi-class, language models	Yes
Binomial	$\{0,\dots,n\}$	np	$np(1-p)$	Ensemble voting, count models	Yes
Multinomial	$\sum x_k = n$	$n\mathbf{p}$	$n(\text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^T)$	Bag-of-words, Naive Bayes	Yes
Poisson	$\mathbb{N}_0$	λ	λ	Count data, GLM, spikes	Yes
Geometric	$\mathbb{N}$	$1/p$	$(1-p)/p^2$	Time-to-event (discrete)	Yes
Neg. Binomial	$\mathbb{N}_0$	$\frac{r(1-p)}{p}$	$\frac{r(1-p)}{p^2}$	Overdispersed counts	Yes
Gaussian	$\mathbb{R}$	μ	σ^2	VAEs, diffusions, priors, everywhere	Yes
Laplace	$\mathbb{R}$	μ	$2b^2$	L1 regularization, robust reg.	Yes*
Cauchy	$\mathbb{R}$	Undef.	Undef.	Robust priors, t-SNE	No
Student's t	$\mathbb{R}$	$\mu\,(\nu>1)$	$\frac{\nu}{\nu-2}\sigma^2$	t-SNE, robust regression	No
Logistic	$\mathbb{R}$	μ	$\frac{s^2\pi^2}{3}$	Logistic regression	Yes
Gumbel	$\mathbb{R}$	$\mu+\beta\gamma$	$\frac{\pi^2}{6}\beta^2$	Gumbel-Softmax, extreme values	Yes
Exponential	$\mathbb{R}^+$	$1/\lambda$	$1/\lambda^2$	Inter-arrival times, survival	Yes
Gamma	$\mathbb{R}^+$	α/β	α/β^2	Conjugate prior (precision)	Yes
Inv-Gamma	$\mathbb{R}^+$	$\frac{\beta}{\alpha-1}$	$\frac{\beta^2}{(\alpha-1)^2(\alpha-2)}$	Conjugate prior (variance)	Yes
Chi-squared	$\mathbb{R}^+$	k	$2k$	Hypothesis testing	Yes
Weibull	$\mathbb{R}^+$	$\lambda\Gamma(1+1/k)$	$\lambda^2[\Gamma(1+2/k)-\Gamma^2(1+1/k)]$	Survival, reliability	Yes*
Log-Normal	$\mathbb{R}^+$	$e^{\mu+\sigma^2/2}$	$(e^{\sigma^2}-1)e^{2\mu+\sigma^2}$	Pos. skewed data, finance	Yes
Pareto	$[x_m,\infty)$	$\frac{\alpha x_m}{\alpha-1}$	$\frac{\alpha x_m^2}{(\alpha-1)^2(\alpha-2)}$	Heavy-tailed modeling	Yes*
Beta	$(0,1)$	$\frac{\alpha}{\alpha+\beta}$	$\frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}$	Conjugate (Bernoulli), A/B test	Yes
Dirichlet	Δ^{K-1}	$\frac{\alpha_k}{\alpha_0}$	$\frac{\alpha_k(\alpha_0-\alpha_k)}{\alpha_0^2(\alpha_0+1)}$	LDA, conjugate (Categorical)	Yes
Kumaraswamy	$(0,1)$	$b\,\text{Beta}(1+1/a,b)$	–	Reparam. alternative to Beta	No
vMF	$\mathbb{S}^{d-1}$	$\boldsymbol{\mu}\frac{I_{d/2}(\kappa)}{I_{d/2-1}(\kappa)}$	–	Hyperspherical embedding	Yes
Wishart	$\mathbb{S}^d_{++}$	$n\mathbf{V}$	–	Conjugate (precision matrix)	Yes
LKJ	Corr. matrix	–	–	Prior on correlation matrices	No
GMM	$\mathbb{R}^d$	$\sum \pi_k\boldsymbol{\mu}_k$	$\sum \pi_k(\boldsymbol{\Sigma}_k+\boldsymbol{\mu}_k\boldsymbol{\mu}_k^T) - \mathbb{E}[\mathbf{x}]\mathbb{E}[\mathbf{x}]^T$	Clustering, density estimation	No

*Only when some parameters are fixed.

Chapter 8

Distance and Similarity Metrics

Distance and similarity measures quantify the relationship between data points, distributions, strings, time series, and sets. They are fundamental to nearest-neighbor methods, clustering, information retrieval, metric learning, and generative model evaluation.

8.1 Minkowski Family (ℓ_p Norms)

The **Minkowski distance** of order $p \geq 1$ is defined as:

$$d_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^{d} |x_i - y_i|^p \right)^{1/p}, \quad p \geq 1$$

For $p \geq 1$, d_p is a metric (satisfies non-negativity, identity of indiscernibles, symmetry, and triangle inequality). For $0 < p < 1$, the triangle inequality is violated, producing a non-metric distance.

8.1.1 Manhattan Distance ($p = 1$)

$$d_1(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{d} |x_i - y_i|$$

Also called City Block or Taxicab distance. It measures distance along axis-aligned directions (like navigating a grid of streets). The unit sphere is a diamond shape.

Usage: L1 regularization (Laplace prior induces sparse solutions), robust to outliers compared to Euclidean, and L1-norm nearest-neighbor search in compressed sensing and sparse coding.

8.1.2 Euclidean Distance ($p = 2$)

$$d_2(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2} = \|\mathbf{x} - \mathbf{y}\|_2$$

The most commonly used distance metric. Rotation-invariant (the only p-norm with this property). The squared Euclidean distance d_2^2 is often preferred in optimization because it avoids the square root and is everywhere differentiable.

Usage: K-means clustering, KNN classification (default), Gaussian kernel $K(\mathbf{x}, \mathbf{y}) = \exp(-\|\mathbf{x}-\mathbf{y}\|_2^2/(2\sigma^2))$, loss function for regression (MSE), and the foundation of Euclidean geometry used in almost all deep learning architectures.

8.1.3 Chebyshev Distance ($p \to \infty$)

$$d_\infty(\mathbf{x}, \mathbf{y}) = \max_{i=1,\ldots,d} |x_i - y_i| = \lim_{p\to\infty} \left(\sum |x_i - y_i|^p\right)^{1/p}$$

Only the single coordinate with the largest difference matters. The unit sphere is a hypercube.

Usage: Warehouse logistics (max travel time of gantry crane along any axis), chess (king's movement distance), and worst-case analysis in adversarial robustness.

8.2 Inner Product / Angular Similarities

8.2.1 Cosine Similarity

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^T\mathbf{y}}{\|\mathbf{x}\|_2\|\mathbf{y}\|_2} = \frac{\sum_{i=1}^d x_i y_i}{\sqrt{\sum x_i^2}\sqrt{\sum y_i^2}}$$

Range: $[-1, 1]$ for vectors with non-negative entries (or unrestricted). 1 = perfectly aligned, 0 = orthogonal, -1 = diametrically opposed. Invariant to scaling: $\cos(c\mathbf{x}, \mathbf{y}) = \cos(\mathbf{x}, \mathbf{y})$ for any $c > 0$.

Usage:

- **Information retrieval / TF-IDF:** Document similarity in high-dimensional sparse vector spaces, where magnitude (document length) should not dominate similarity.
- **Word embeddings (Word2Vec, GloVe):** Semantic similarity judged by cosine of embedding vectors.
- **Recommender systems:** User-item and item-item collaborative filtering.
- **Face verification:** Cosine similarity between normalized face embeddings (CosFace, SphereFace).
- **Cross-modal retrieval:** CLIP uses cosine similarity between image and text embeddings.

8.2.2 Cosine Distance and Angular Distance

$$\text{Cosine distance:} \quad d_{\cos}(\mathbf{x}, \mathbf{y}) = 1 - \cos(\mathbf{x}, \mathbf{y}) = 1 - \frac{\mathbf{x}^T\mathbf{y}}{\|\mathbf{x}\|\|\mathbf{y}\|}$$

Range: $[0, 2]$. Cosine distance is not a proper metric (violates triangle inequality), but it is a semi-metric.

$$\text{Angular distance:} \quad d_{\text{ang}}(\mathbf{x}, \mathbf{y}) = \frac{\arccos(\cos(\mathbf{x}, \mathbf{y}))}{\pi}$$

Range: $[0, 1]$. Angular distance *is* a proper metric on the unit sphere. The geodesic distance on the hypersphere is $\arccos(\mathbf{x}^T\mathbf{y})$.

8.2.3 Dot Product (for Normalized Vectors)

When vectors are L2-normalized ($\|\mathbf{x}\| = \|\mathbf{y}\| = 1$), the dot product equals cosine similarity:

$$\mathbf{x}^T\mathbf{y} = \cos(\mathbf{x}, \mathbf{y}) \in [-1, 1]$$

For unnormalized vectors, the dot product grows with vector magnitude, which can be desirable in attention mechanisms (more confident query-key matches get larger logits).

8.2.4 Pearson Correlation Coefficient

$$\rho(\mathbf{x}, \mathbf{y}) = \frac{\text{Cov}(\mathbf{x}, \mathbf{y})}{\sigma_\mathbf{x}\sigma_\mathbf{y}} = \frac{\sum_{i=1}^{d}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2}\sqrt{\sum(y_i - \bar{y})^2}}$$

where $\bar{x}, \bar{y}$ are the sample means of the vector entries. Pearson correlation measures linear dependence and is invariant to affine transformations ($a\mathbf{x} + b$). Range: $[-1, 1]$.

Usage: Feature selection (correlation with target), collaborative filtering (correlation-based user similarity), PCA (correlation matrix eigendecomposition instead of covariance), and as an evaluation metric for regression (correlation between predicted and actual values).

8.3 Distribution Distances

8.3.1 Bhattacharyya Distance

$$D_B(P, Q) = -\ln\left(\int \sqrt{p(\mathbf{x})q(\mathbf{x})}\, d\mathbf{x}\right)$$

The Bhattacharyya coefficient $BC(P, Q) = \int \sqrt{pq}\, d\mathbf{x}$ measures the overlap between two distributions. Range: $D_B \in [0, \infty)$. For multivariate Gaussians:

$$D_B = \frac{1}{8}(\boldsymbol{\mu}_P - \boldsymbol{\mu}_Q)^T\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_P - \boldsymbol{\mu}_Q) + \frac{1}{2}\ln\frac{|\boldsymbol{\Sigma}|}{\sqrt{|\boldsymbol{\Sigma}_P||\boldsymbol{\Sigma}_Q|}}$$

where $\boldsymbol{\Sigma} = (\boldsymbol{\Sigma}_P + \boldsymbol{\Sigma}_Q)/2$. Upper bounds the Bayes error rate.

Usage: Feature selection in pattern recognition (minimize Bhattacharyya distance between classes), signal processing, and as a separability measure in classification.

8.3.2 Hellinger Distance

$$H^2(P,Q) = \frac{1}{2}\int \left(\sqrt{p(\mathbf{x})} - \sqrt{q(\mathbf{x})}\right)^2 d\mathbf{x} = 1 - \int \sqrt{p(\mathbf{x})q(\mathbf{x})}\, d\mathbf{x}$$

Range: $[0,1]$. Related to Bhattacharyya: $H^2 = 1 - BC$. Hellinger distance is a proper metric and is invariant to change of measure (unlike KL divergence).

8.3.3 Total Variation Distance

$$d_{TV}(P,Q) = \frac{1}{2}\int |p(\mathbf{x}) - q(\mathbf{x})|\, d\mathbf{x} = \sup_{A\in\mathcal{F}} |P(A) - Q(A)|$$

Range: $[0,1]$. TV is the maximum difference in probability assigned to any event. For discrete distributions, $d_{TV}(P,Q) = \frac{1}{2}\sum |p_i - q_i|$. Related to Pinsker's inequality: $d_{TV}(P,Q) \leq \sqrt{\frac{1}{2}\,\mathrm{KL}(P\|Q)}$.

Usage: Theoretical analysis of MCMC convergence (TV distance to stationary distribution), differential privacy bounds, and proving consistency of density estimators.

8.3.4 Wasserstein Distance (Earth Mover's Distance)

The p-**Wasserstein distance** (Kantorovich-Rubinstein) is defined via optimal transport:

$$W_p(P,Q) = \left(\inf_{\gamma\in\Gamma(P,Q)} \mathbb{E}_{(\mathbf{x},\mathbf{y})\sim\gamma}[\|\mathbf{x}-\mathbf{y}\|^p]\right)^{1/p}$$

where $\Gamma(P,Q)$ is the set of all joint distributions with marginals P and Q. This is the minimum "cost" to transport mass from P to Q.
Kantorovich-Rubinstein duality (for $p=1$):

$$W_1(P,Q) = \sup_{\|f\|_L\leq 1} \mathbb{E}_{\mathbf{x}\sim P}[f(\mathbf{x})] - \mathbb{E}_{\mathbf{x}\sim Q}[f(\mathbf{x})]$$

where $\|f\|_L$ is the Lipschitz constant of f. This dual form is the theoretical basis of Wasserstein GANs (WGAN), where the discriminator estimates the dual supremum under a 1-Lipschitz constraint (enforced via weight clipping or gradient penalty).

2-Wasserstein for Gaussians (closed form):

$$W_2^2(\mathcal{N}(\boldsymbol{\mu}_P,\boldsymbol{\Sigma}_P),\mathcal{N}(\boldsymbol{\mu}_Q,\boldsymbol{\Sigma}_Q)) = \|\boldsymbol{\mu}_P - \boldsymbol{\mu}_Q\|_2^2 + \mathrm{Tr}\left(\boldsymbol{\Sigma}_P + \boldsymbol{\Sigma}_Q - 2(\boldsymbol{\Sigma}_P^{1/2}\boldsymbol{\Sigma}_Q\boldsymbol{\Sigma}_P^{1/2})^{1/2}\right)$$

This formula underlies the Fréchet Inception Distance (FID). The trace term measures the difference in covariance geometry.

8.3.5 Sinkhorn Distance (Entropy-Regularized OT)

The **Sinkhorn distance** adds entropy regularization to the optimal transport problem:

$$W_\epsilon(P,Q) = \min_{\boldsymbol{\gamma}} \sum_{i,j} \gamma_{ij} c_{ij} + \epsilon \sum_{i,j} \gamma_{ij} \log \gamma_{ij}$$

subject to $\boldsymbol{\gamma}\mathbf{1} = \mathbf{p}, \boldsymbol{\gamma}^T\mathbf{1} = \mathbf{q}$, where $\epsilon > 0$ controls regularization strength.
Sinkhorn-Knopp algorithm: The solution has the form $\boldsymbol{\gamma}^* = \text{diag}(\mathbf{u})\mathbf{K}\,\text{diag}(\mathbf{v})$ where $K_{ij} = e^{-c_{ij}/\epsilon}$ and $\mathbf{u}, \mathbf{v}$ are found by iterative row/column normalization:

$$\mathbf{u}^{(t+1)} = \frac{\mathbf{p}}{\mathbf{K}\mathbf{v}^{(t)}}, \quad \mathbf{v}^{(t+1)} = \frac{\mathbf{q}}{\mathbf{K}^T\mathbf{u}^{(t+1)}}$$

This converges linearly and is GPU-friendly, making OT practical for deep learning.

Usage: Differentiable OT loss for generative models, domain adaptation, shape matching, single-cell genomics (trajectory inference), and as a training loss for WGANs (Sinkhorn divergence).

8.3.6 Maximum Mean Discrepancy (MMD)

MMD is an integral probability metric defined via a reproducing kernel Hilbert space (RKHS) $\mathcal{H}$:

$$\text{MMD}^2(P,Q) = \|\boldsymbol{\mu}_P - \boldsymbol{\mu}_Q\|^2_{\mathcal{H}} = \mathbb{E}_{\mathbf{x},\mathbf{x}'\sim P}[k(\mathbf{x},\mathbf{x}')] - 2\mathbb{E}_{\mathbf{x}\sim P,\mathbf{y}\sim Q}[k(\mathbf{x},\mathbf{y})] + \mathbb{E}_{\mathbf{y},\mathbf{y}'\sim Q}[k(\mathbf{y},\mathbf{y}')]$$

where $k(\cdot,\cdot)$ is the RKHS kernel. MMD $= 0$ iff $P = Q$ for a characteristic kernel (e.g., Gaussian RBF, inverse multiquadric).
Empirical estimator: Given samples $\{\mathbf{x}_i\}_{i=1}^n \sim P$ and $\{\mathbf{y}_j\}_{j=1}^m \sim Q$:

$$\widehat{\text{MMD}}^2 = \frac{1}{n^2}\sum_{i,j} k(\mathbf{x}_i,\mathbf{x}_j) + \frac{1}{m^2}\sum_{i,j} k(\mathbf{y}_i,\mathbf{y}_j) - \frac{2}{nm}\sum_{i,j} k(\mathbf{x}_i,\mathbf{y}_j)$$

The unbiased estimator uses $i \neq j$ in the same-distribution terms.

Usage: Two-sample testing (is $P \neq Q$?), generative model evaluation (KID uses polynomial kernel MMD), domain adaptation (MMD-based alignment of source and target features), and fairness testing (are predictions equally distributed across groups?).

8.3.7 Energy Distance

Energy distance generalizes MMD with the Euclidean distance (negative-type kernel):

$$\mathcal{E}^2(P,Q) = 2\mathbb{E}[\|\mathbf{x}-\mathbf{y}\|] - \mathbb{E}[\|\mathbf{x}-\mathbf{x}'\|] - \mathbb{E}[\|\mathbf{y}-\mathbf{y}'\|]$$

Equivalent to MMD with the kernel $k(\mathbf{x},\mathbf{y}) = -\|\mathbf{x}-\mathbf{y}\|$ (conditionally negative definite). Energy distance characterizes equality of distributions ($\mathcal{E}^2 = 0 \iff P = Q$).

8.4 Information-Theoretic Divergences

8.4.1 Kullback-Leibler (KL) Divergence

$$D_{KL}(P \parallel Q) = \int p(\mathbf{x}) \log \frac{p(\mathbf{x})}{q(\mathbf{x})} \, d\mathbf{x} = \mathbb{E}_{\mathbf{x} \sim P}\left[\log \frac{p(\mathbf{x})}{q(\mathbf{x})}\right]$$

Properties:

- **Asymmetric:** $D_{KL}(P\|Q) \neq D_{KL}(Q\|P)$ generally. The forward KL $D_{KL}(P\|Q)$ is mean-seeking (covers all modes), the reverse KL $D_{KL}(Q\|P)$ is mode-seeking.
- **Non-negative:** $D_{KL} \geq 0$ by Gibbs' inequality. $D_{KL} = 0$ iff $P = Q$ almost everywhere.
- **Not a metric:** Violates symmetry and triangle inequality.
- **Invariant to reparameterization:** $D_{KL}(p(x)\|q(x)) = D_{KL}(p(y)\|q(y))$ for invertible transforms.

KL for multivariate Gaussians:

$$D_{KL}(\mathcal{N}_1 \parallel \mathcal{N}_2) = \frac{1}{2}\left[\mathrm{Tr}(\boldsymbol{\Sigma}_2^{-1}\boldsymbol{\Sigma}_1) + (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^T \boldsymbol{\Sigma}_2^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) - d + \ln \frac{|\boldsymbol{\Sigma}_2|}{|\boldsymbol{\Sigma}_1|}\right]$$

KL for Categorical distributions:

$$D_{KL}(\mathbf{p} \parallel \mathbf{q}) = \sum_{k=1}^{K} p_k \log \frac{p_k}{q_k}$$

The cross-entropy loss in classification is $\mathcal{L}_{CE} = H(\mathbf{y}) + D_{KL}(\mathbf{y}\|\,\hat{\mathbf{y}})$; since $H(\mathbf{y}) = 0$ for one-hot targets, minimizing cross-entropy = minimizing KL divergence.

8.4.2 Jensen-Shannon (JS) Divergence

$$\mathrm{JS}(P \parallel Q) = \frac{1}{2}D_{KL}(P \parallel M) + \frac{1}{2}D_{KL}(Q \parallel M), \quad M = \frac{P+Q}{2}$$

Properties: Symmetric: $\mathrm{JS}(P\|Q) = \mathrm{JS}(Q\|P)$. Bounded: $0 \leq \mathrm{JS} \leq \log 2$ (for natural log). The square root $\sqrt{\mathrm{JS}}$ is a metric. The original GAN formulation uses JS divergence as its objective (leading to vanishing gradients when discriminator is optimal—motivating WGAN).

8.4.3 Cross-Entropy

$$H(P, Q) = -\mathbb{E}_{\mathbf{x} \sim P}[\log q(\mathbf{x})] = H(P) + D_{KL}(P \parallel Q)$$

Cross-entropy decomposes into the true distribution's entropy plus the KL divergence. Minimizing cross-entropy (with fixed P) is equivalent to minimizing KL divergence.

8.5 Binary / Set Similarities

8.5.1 Hamming Distance

For binary vectors $\mathbf{x}, \mathbf{y} \in \{0,1\}^d$:

$$d_H(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{d} \mathbf{1}[x_i \neq y_i] = \|\mathbf{x} - \mathbf{y}\|_0$$

Counts the number of positions where two binary strings differ. Range: $[0, d]$. Normalized: d_H/d. A proper metric.

Usage: Error detection/correction in communication (minimum Hamming distance determines error correction capability), binary hash comparison for near-duplicate detection, DNA sequence comparison, and evaluation of multi-label classification.

8.5.2 Jaccard Similarity (IoU for Sets)

For two sets A, B:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Jaccard distance: $d_J(A, B) = 1 - J(A, B)$. Range $[0, 1]$. A proper metric. In binary vector form: $J = \frac{TP}{TP+FP+FN}$ (same as IoU).

Usage: Set overlap in recommendation (common items between two users), species similarity in ecology, duplicate document detection (MinHash estimates Jaccard), object detection (IoU), and clustering similarity.

8.5.3 Dice / Sørensen-Dice Coefficient

$$\text{Dice}(A, B) = \frac{2|A \cap B|}{|A| + |B|}$$

In binary vector form: $\text{Dice} = \frac{2TP}{2TP+FP+FN}$, which is identical to the F1 score. Range $[0, 1]$. Gives double weight to the intersection compared to union-based Jaccard.

Usage: Image segmentation (Dice loss is standard for medical image segmentation), string similarity (bigram Dice coefficient), and ecological community similarity.

8.5.4 Overlap Coefficient

$$\text{Overlap}(A, B) = \frac{|A \cap B|}{\min(|A|, |B|)}$$

Measures how much the smaller set is contained in the larger set. Range $[0, 1]$. Unlike Jaccard, if $A \subseteq B$, Overlap $= 1$ regardless of the relative sizes.

8.5.5 Tversky Index

A generalization of Jaccard and Dice with parameters $\alpha, \beta \geq 0$:

$$S(A, B) = \frac{|A \cap B|}{|A \cap B| + \alpha|A \setminus B| + \beta|B \setminus A|}$$

$\alpha = \beta = 1$: Jaccard. $\alpha = \beta = 0.5$: Dice. $\alpha = 1, \beta = 0$: Recall of A with respect to B. The Tversky loss $(1 - S)$ is used in medical image segmentation where false positives and false negatives carry different costs.

8.6 String / Sequence Distances

8.6.1 Levenshtein Distance (Edit Distance)

The **Levenshtein distance** between strings a and b of lengths m, n is the minimum number of single-character insertions, deletions, and substitutions to transform a into b. **Dynamic programming recurrence:**

$$d(i, j) = \begin{cases} 0 & i = 0,\ j = 0 \\ i & j = 0 \\ j & i = 0 \\ d(i-1, j-1) & a_i = b_j \\ 1 + \min(d(i-1, j),\ d(i, j-1),\ d(i-1, j-1)) & a_i \neq b_j \end{cases}$$

Complexity: $O(mn)$ time and space (can be optimized to $O(\min(m, n))$ space).

Usage: Spell checking, DNA sequence alignment, fuzzy string matching, plagiarism detection, and evaluating OCR (Optical Character Recognition) output by comparing recognized text to ground truth.

8.6.2 Damerau-Levenshtein Distance

Extends Levenshtein by adding **transposition** of two adjacent characters as a primitive operation. This is the minimum number of insertions, deletions, substitutions, and transpositions. Important for typo correction (80% of human misspellings involve transpositions).

8.6.3 Jaro and Jaro-Winkler Similarity

Jaro similarity of strings s_1, s_2:

$$\text{Jaro} = \frac{1}{3}\left(\frac{m}{|s_1|} + \frac{m}{|s_2|} + \frac{m-t}{m}\right)$$

where m is the number of matching characters (within a sliding window of $\lfloor \max(|s_1|, |s_2|)/2 \rfloor - 1$) and t is half the number of transpositions.
Jaro-Winkler boosts the score when strings share a common prefix:

$$\text{JW} = \text{Jaro} + \ell \cdot p \cdot (1 - \text{Jaro})$$

where $\ell \leq 4$ is the length of the common prefix and $p = 0.1$ is the scaling factor.

Usage: Record linkage (entity resolution in databases—matching names, addresses), census data deduplication, and fuzzy search. Jaro-Winkler is optimized for short strings like person names.

8.6.4 Smith-Waterman (Local Alignment)

The classic algorithm for local sequence alignment. For strings a, b:

$$H(i,j) = \max \begin{cases} 0 & \\ H(i-1, j-1) + s(a_i, b_j) & \text{(match/mismatch)} \\ H(i-1, j) + g & \text{(deletion/gap)} \\ H(i, j-1) + g & \text{(insertion/gap)} \end{cases}$$

where s is the substitution matrix (e.g., BLOSUM for proteins) and g is the gap penalty. The zero floor enables finding local (not forced-global) alignments.

Usage: Bioinformatics (BLAST uses heuristics based on Smith-Waterman), protein/DNA sequence alignment, and finding similar substrings within longer sequences.

8.6.5 Needleman-Wunsch (Global Alignment)

Global sequence alignment with dynamic programming:

$$F(i,j) = \max \begin{cases} F(i-1, j-1) + s(a_i, b_j) \\ F(i-1, j) + g \\ F(i, j-1) + g \end{cases}$$

Unlike Smith-Waterman, the zero floor is removed, forcing global alignment of the entire sequences.

8.6.6 Longest Common Subsequence (LCS)

$$\text{LCS}(i,j) = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ \text{LCS}(i-1, j-1) + 1 & a_i = b_j \\ \max(\text{LCS}(i-1,j),\ \text{LCS}(i,j-1)) & a_i \neq b_j \end{cases}$$

The length of the LCS is used as a similarity score. Basis of ROUGE-L for text summarization evaluation.

8.7 Time Series Distances

8.7.1 Dynamic Time Warping (DTW)

DTW aligns two time series $\mathbf{x} = (x_1, \ldots, x_n)$ and $\mathbf{y} = (y_1, \ldots, y_m)$ by warping the time axis non-linearly:

$$\text{DTW}(i,j) = d(x_i, y_j) + \min \begin{cases} \text{DTW}(i-1, j) \\ \text{DTW}(i, j-1) \\ \text{DTW}(i-1, j-1) \end{cases}$$

where $d(\cdot,\cdot)$ is a local distance (typically Euclidean). The resulting cost matrix is minimized with the constraint that the warping path is monotonic, continuous, and starts/stops at the endpoints.
Constraints: Sakoe-Chiba band (limits warping to a window of size w around the diagonal: $|i-j| \leq w$) and Itakura parallelogram improve efficiency and prevent pathological warpings.

Usage: Speech recognition (aligning utterances of different speeds), gesture recognition, ECG/EEG signal comparison, gait analysis, and time series clustering with DTW Barycenter Averaging (DBA).

Pros: Handles variable-length sequences, robust to temporal shifts and speed variations, widely used across domains.
Cons: $O(mn)$ complexity (quadratic), non-differentiable (but see Soft-DTW), sensitive to noise and outliers without pre-processing, and DTW is not a metric (can violate triangle inequality).

8.7.2 Soft-DTW

Soft-DTW replaces the min operator in DTW with a smooth $\min^{\gamma}$ (log-sum-exp):

$$\min^{\gamma}\{a_1, \ldots, a_n\} = \begin{cases} -\gamma \log \sum_i e^{-a_i/\gamma} & \gamma > 0 \\ \min_i a_i & \gamma = 0 \end{cases}$$

The soft-min operator is differentiable everywhere, making DTW a training loss for deep learning with time series data. The gradient flows through all alignment paths weighted by their soft-min probability.

Usage: End-to-end training of neural networks for time series classification, time series averaging (soft-DTW barycenters), and temporal alignment of multimodal sequences.

8.7.3 Fréchet Distance

The **Fréchet distance** measures the similarity of curves taking into account both the location and ordering of points. Informally: a person walks along one curve with a dog on a leash along the other, each moving forward; the Fréchet distance is the minimum leash length needed. Formally:

$$F(P, Q) = \inf_{\alpha,\beta} \max_{t\in[0,1]} \|P(\alpha(t)) - Q(\beta(t))\|$$

where α, β are continuous, non-decreasing reparameterizations. Commonly approximated via "discrete Fréchet distance" on sampled points.

8.8 Mahalanobis Distance

$$d_M(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \mathbf{y})}$$

Mahalanobis distance scales the coordinate axes by the inverse covariance, effectively measuring distance in units of standard deviation along each principal component. It accounts for feature correlations: a unit change in a highly correlated direction counts less than a unit change in an independent direction.

Special cases:

- $\boldsymbol{\Sigma} = \mathbf{I}$: Euclidean distance.
- $\boldsymbol{\Sigma} = \text{diag}(\sigma_1^2, \ldots, \sigma_d^2)$: Normalized Euclidean distance.

For a single point $\mathbf{x}$ and a distribution $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$:

$$d_M^2(\mathbf{x}, \boldsymbol{\mu}) = (\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \sim \chi_d^2$$

This is the basis of multivariate outlier detection (EllipticEnvelope).

Usage:

- **Outlier detection:** Points with Mahalanobis distance exceeding the χ^2 critical value are flagged as anomalous.

- **Metric learning:** Learn $\mathbf{M} = \mathbf{\Sigma}^{-1}$ (Mahalanobis matrix) from data to improve KNN. LMNN (Large Margin Nearest Neighbor) learns a Mahalanobis metric.
- **Fisher's Linear Discriminant:** Projects data to maximize the Mahalanobis distance between class means.
- **Clustering:** Gaussian Mixture Models use Mahalanobis distance in the exponent for each component.
- **Hotelling's T^2 test:** Multivariate generalization of Student's t-test based on Mahalanobis distance.

8.9 Image Structural Similarity (Extended)

8.9.1 SSIM — Detailed Breakdown

The **Structural Similarity Index** decomposes image similarity into three components:
Luminance:

$$l(x,y) = \frac{2\mu_x\mu_y + c_1}{\mu_x^2 + \mu_y^2 + c_1}, \quad c_1 = (k_1 L)^2$$

Contrast:

$$c(x,y) = \frac{2\sigma_x\sigma_y + c_2}{\sigma_x^2 + \sigma_y^2 + c_2}, \quad c_2 = (k_2 L)^2$$

Structure:

$$s(x,y) = \frac{\sigma_{xy} + c_3}{\sigma_x\sigma_y + c_3}, \quad c_3 = c_2/2$$

Combined: $\text{SSIM}(x,y) = [l(x,y)]^\alpha \cdot [c(x,y)]^\beta \cdot [s(x,y)]^\gamma$, typically with $\alpha = \beta = \gamma = 1$. Default parameters: $k_1 = 0.01$, $k_2 = 0.03$, $L = 255$ (8-bit images). SSIM is computed locally in 11×11 Gaussian windows and averaged across the image.

8.9.2 FSIM (Feature Similarity)

FSIM computes similarity based on low-level features: phase congruency (PC, for significance of local structure) and gradient magnitude (GM). The similarity at location i:

$$S_L(i) = S_{PC}(i)^\alpha \cdot S_{GM}(i)^\beta$$

weighted by PC as importance. FSIM correlates better with human perception than SSIM.

8.9.3 HaarPSI

HaarPSI uses the Haar wavelet decomposition to assess local similarities at different scales and orientations. It compares wavelet coefficients, weighting each region by its perceptual importance (high-weight for edges and textures, low for flat regions). HaarPSI shows state-of-the-art correlation with human opinion scores.

8.10 Optimal Transport (Extended)

8.10.1 Earth Mover's Distance (EMD) — Discrete Case

Given two discrete distributions $\mathbf{p} \in \Delta^n, \mathbf{q} \in \Delta^m$ and a ground cost matrix $\mathbf{C} \in \mathbb{R}^{n \times m}$:

$$\text{EMD}(\mathbf{p}, \mathbf{q}) = \min_{\boldsymbol{\gamma} \in \mathbb{R}^{n \times m}_{\geq 0}} \langle \boldsymbol{\gamma}, \mathbf{C} \rangle_F \quad \text{s.t.} \quad \boldsymbol{\gamma} \mathbf{1}_m = \mathbf{p},\ \boldsymbol{\gamma}^T \mathbf{1}_n = \mathbf{q}$$

This is a linear program with nm variables and $n + m$ constraints. For 1D distributions with ℓ_p cost, the optimal transport map is given by sorting (quantile function differences). For 2D images with equal total mass, EMD computes minimum pixel flow to transform one image into another.

8.10.2 Gromov-Wasserstein Distance

The **Gromov-Wasserstein (GW)** distance compares distributions supported on different metric spaces by aligning their internal distance structures:

$$GW_2^2(\mu, \nu) = \min_{\gamma \in \Gamma(\mu,\nu)} \int \int |d_X(x, x') - d_Y(y, y')|^2 \, d\gamma(x, y) \, d\gamma(x', y')$$

GW only requires pairwise distances within each space, not between spaces. It is invariant to isometries (rigid motions), making it ideal for shape matching, graph matching, and comparing metric measure spaces without a common coordinate system.

Usage: Shape correspondence (3D mesh alignment), single-cell multi-omics alignment (different measurement spaces for same cells), and graph matching.

8.11 Learned and Kernel-Based Distances

8.11.1 Siamese Networks with Contrastive Loss

A **Siamese network** learns a feature embedding $f_\theta(\mathbf{x})$ such that the Euclidean distance $d_\theta(\mathbf{x}_1, \mathbf{x}_2) = \|f_\theta(\mathbf{x}_1) - f_\theta(\mathbf{x}_2)\|_2$ reflects semantic similarity.
Contrastive loss:

$$\mathcal{L} = y \cdot d_\theta^2 + (1 - y) \cdot \max(0, m - d_\theta)^2$$

where $y = 1$ for similar pairs (minimize distance) and $y = 0$ for dissimilar pairs (push apart by at least margin m).

8.11.2 Triplet Loss

For an anchor $\mathbf{x}_a$, positive $\mathbf{x}_p$ (same class), and negative $\mathbf{x}_n$ (different class):

$$\mathcal{L}_{\text{triplet}} = \max(0, d(\mathbf{x}_a, \mathbf{x}_p) - d(\mathbf{x}_a, \mathbf{x}_n) + m)$$

The goal is to make the anchor-positive distance smaller than the anchor-negative distance by at least margin m. The choice of triplets (hard negative mining) is crucial for

convergence.

Usage: Face recognition (FaceNet uses triplet loss), person re-identification, image retrieval, and metric learning where a meaningful embedding distance is required.

8.11.3 Kernel-Induced Distances

Given a positive definite kernel $K(\cdot, \cdot)$ with associated feature map $\phi(\cdot)$ such that $K(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle_{\mathcal{H}}$, the kernel distance:

$$d_K(\mathbf{x}, \mathbf{y}) = \sqrt{K(\mathbf{x}, \mathbf{x}) - 2K(\mathbf{x}, \mathbf{y}) + K(\mathbf{y}, \mathbf{y})} = \|\phi(\mathbf{x}) - \phi(\mathbf{y})\|_{\mathcal{H}}$$

This is the Euclidean distance in the RKHS. For the Gaussian RBF kernel, $K(\mathbf{x}, \mathbf{y}) = \exp(-\gamma \|\mathbf{x} - \mathbf{y}\|^2)$, we get $K(\mathbf{x}, \mathbf{x}) = 1$, so $d_K = \sqrt{2 - 2K(\mathbf{x}, \mathbf{y})}$.

Usage: Kernel KNN, kernel PCA (distance preservation in reduced dimensions), and Support Vector Data Description (SVDD) for one-class classification.

8.12 Summary Comparison Table

Guidelines for choosing a distance metric:

- **General low-dim continuous data:** Euclidean (default).
- **High-dimensional sparse data (text):** Cosine similarity.
- **Correlated features:** Mahalanobis (or learn $\mathbf{M}$ via metric learning).
- **Image comparisons:** SSIM or HaarPSI (perceptual); PSNR or MSE (pixel).
- **Distribution comparison:** Wasserstein or MMD (sample-based); KL/JS (density-based).
- **Set overlap:** Jaccard for balanced sets; Dice for imbalance (like F1).
- **Strings / sequences:** Levenshtein (short), Jaro-Winkler (names), DTW (time-varying).
- **Nearest neighbor search with metric constraints:** Only proper metrics allow efficient structures (ball trees, cover trees).
- **Deep learning:** Embedding distance from a Siamese/triplet network replaces hand-crafted distances for semantic similarity.

Table 8.1: Comprehensive Comparison of Distance and Similarity Measures

Metric	**Type**	**Range**	**Metric?**	**Best For**
Euclidean (ℓ_2)	Vec-Vec	$[0, \infty)$	Yes	General purpose, K-means, KNN
Manhattan (ℓ_1)	Vec-Vec	$[0, \infty)$	Yes	Sparse/high-dim data, L1 reg.
Chebyshev (ℓ_∞)	Vec-Vec	$[0, \infty)$	Yes	Worst-case analysis
Cosine Sim.	Vec-Vec	$[-1, 1]$	No	Text (TF-IDF), embeddings, IR
Angular Dist.	Vec-Vec	$[0, 1]$	Yes	Hyperspherical data
Pearson ρ	Vec-Vec	$[-1, 1]$	No	Linear correlation, feature sel.
KL Divergence	Dist-Dist	$[0, \infty)$	No	Variational inference, model comp.
JS Divergence	Dist-Dist	$[0, \log 2]$	$\sqrt{}$ is metric	GAN training, symmetric comp.
TV Distance	Dist-Dist	$[0, 1]$	Yes	MCMC convergence, theory
Bhattacharyya	Dist-Dist	$[0, \infty)$	No	Class separability, overlap
Hellinger	Dist-Dist	$[0, 1]$	Yes	Distribution comparison
Wasserstein (W_1)	Dist-Dist	$[0, \infty)$	Yes	WGAN, domain adaptation
Sinkhorn Div.	Dist-Dist	$[0, \infty)$	No (div.)	Differentiable OT
MMD	Dist-Dist	$[0, \infty)$	Yes (kernel)	Two-sample test, fairness
Energy Dist.	Dist-Dist	$[0, \infty)$	Yes	Energy statistics
Mahalanobis	Vec-Vec	$[0, \infty)$	Yes	Correlated features, outlier det.
Hamming	Binary/Set	$[0, d]$	Yes	Error correction, multi-label
Jaccard (IoU)	Set	$[0, 1]$	Yes	Set overlap, detection, clustering
Dice (F1)	Set	$[0, 1]$	No	Medical seg., string sim.
Tversky	Set	$[0, 1]$	No	Weighted seg. (FN vs FP cost)
Levenshtein	String	$[0, \max(m, n)]$	Yes	Spell check, DNA, fuzzy match
Jaro-Winkler	String	$[0, 1]$	No	Name matching, record linkage
DTW	Time Series	$[0, \infty)$	No*	Speech, gesture, ECG alignment
Soft-DTW	Time Series	$[0, \infty)$	No*	Diff. time series loss
Fréchet	Curves	$[0, \infty)$	Yes	Shape/curve matching
SSIM	Image	$[-1, 1]$	No	Perceptual image quality
PSNR	Image	$[0, \infty)$ dB	No	Pixel-level reconstruction
GW (Gromov-W)	Spaces	$[0, \infty)$	Yes (metric)	Shape matching, graphs

*DTW and Soft-DTW are not metrics because they can violate the triangle inequality.

Chapter 9

Optimization Algorithms

Optimization lies at the heart of modern machine learning. This chapter surveys the landscape of optimization algorithms from basic gradient descent to specialized techniques for deep learning, distributed training, and black-box optimization. Each algorithm is presented with its mathematical definition, usage pattern, and a balanced assessment of strengths and limitations.

9.1 First-Order Non-Adaptive Methods

9.1.1 Gradient Descent (GD)

Gradient Descent

Given a differentiable loss function $\mathcal{L}(\mathbf{w})$ evaluated on the full training set, gradient descent updates the parameter vector $\mathbf{w} \in \mathbb{R}^n$ by moving in the direction of steepest descent:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t)$$

where $\eta > 0$ is the learning rate and $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t)$ is the gradient computed over all N training examples.

Applications

Classic convex optimization, linear regression, logistic regression, and small-scale neural network training. Forms the conceptual foundation for all gradient-based methods.

Trade-offs

Pros:

- Convergence guarantees: $O(1/T)$ rate for convex functions with appropriate diminishing η, linear convergence for strongly-convex functions.
- Simple to implement and analyze; no extra hyperparameters beyond learning rate.
- Deterministic updates give stable and reproducible optimization trajectories.

Cons:

- Computationally expensive: each step requires a full pass over the entire dataset (batch gradient).
- Fixed learning rate requires careful tuning; too large causes divergence, too small causes slow convergence.
- Performs poorly on ill-conditioned problems where the Hessian has a large condition number.
- Can become trapped in saddle points in high-dimensional non-convex landscapes.

9.1.2 Stochastic Gradient Descent (SGD)

SGD

SGD replaces the full-batch gradient with a noisy but unbiased estimate computed from a single example $(x^{(i)}, y^{(i)})$:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \, \nabla_{\mathbf{w}} \mathcal{L}\big(\mathbf{w}_t; \mathbf{x}^{(i)}, y^{(i)}\big)$$

Mini-batch SGD averages the gradient over a randomly sampled batch $\mathcal{B}$ of size B:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \, \frac{1}{B} \sum_{(\mathbf{x}^{(i)}, y^{(i)}) \in \mathcal{B}} \nabla_{\mathbf{w}} \mathcal{L}\big(\mathbf{w}_t; \mathbf{x}^{(i)}, y^{(i)}\big)$$

Applications

The default training algorithm for virtually all large-scale deep neural networks trained with stochastic gradient estimation. Batch sizes typically range from 32 to 1024.

Trade-offs

Pros:

- Per-step cost is $O(B)$ instead of $O(N)$, enabling training on datasets that do not fit in memory.
- Gradient noise acts as implicit regularization, improving generalization by escaping sharp minima.
- Frequent updates mean faster initial progress than batch GD.

Cons:

- Noisy gradient estimates cause oscillatory convergence; the iterates never settle exactly at the minimum.
- Requires careful learning-rate scheduling for convergence.
- Progress is slow in parameter directions with small gradient variance (the "ravine" problem).
- The noise introduces additional hyperparameters (batch size, learning rate schedule).

9.1.3 SGD with Momentum (Polyak)

Polyak Momentum

A velocity vector $\mathbf{v}$ accumulates an exponentially decaying moving average of past gradients:

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{v}_{t+1}$$

where $\beta \in [0, 1)$ is the momentum coefficient (typically 0.9).

EMA Interpretation

Expanding the velocity recurrence reveals that $\mathbf{v}_t$ is an exponentially weighted sum of all past gradients:

$$\mathbf{v}_t = \eta \sum_{\tau=1}^{t} \beta^{t-\tau} \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_\tau)$$

The effective "memory" of the velocity is roughly $1/(1-\beta)$ steps.

Applications

Ubiquitous in deep learning. Usually implemented as SGD + momentum + weight decay for image classification (e.g., ResNet training on ImageNet).

Trade-offs

Pros:

- Smooths out oscillations in directions of high curvature.
- Accelerates progress in consistent gradient directions.
- Helps navigate small local minima and shallow saddle points.

Cons:

- Overshooting when momentum is set too high ($\beta \to 1$).
- Introduces an additional hyperparameter: β (though 0.9 works well in practice).
- The accumulated momentum can cause the optimizer to "coast" past the minimum before turning around.

9.1.4 SGD with Nesterov Accelerated Gradient (NAG)

Nesterov Momentum (NAG)

NAG computes the gradient at a "look-ahead" position, not at the current parameter value:

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t - \beta \mathbf{v}_t)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{v}_{t+1}$$

An equivalent formulation amenable to automatic differentiation is:

$$\tilde{\mathbf{w}}_t = \mathbf{w}_t - \beta \mathbf{v}_t, \quad \mathbf{v}_{t+1} = \beta \mathbf{v}_t + \eta \nabla \mathcal{L}(\tilde{\mathbf{w}}_t), \quad \mathbf{w}_{t+1} = \tilde{\mathbf{w}}_t - \eta \nabla \mathcal{L}(\tilde{\mathbf{w}}_t) + \beta(\tilde{\mathbf{w}}_t - \tilde{\mathbf{w}}_{t-1})$$

Applications

Often preferred over plain momentum for tasks where aggressive acceleration is beneficial, such as training deeper convolutional networks or reinforcement learning with high-dimensional policy parameters.

Trade-offs

Pros:

- Provably faster convergence than standard momentum for convex functions: $O(1/t^2)$ vs. $O(1/t)$.
- The look-ahead gradient acts as a correction term that reduces overshooting and oscillation.

- Frequently yields better test performance than standard momentum.

Cons:

- Naïve implementation requires computing the gradient twice per step (once for the look-ahead, once for the update).
- The parameterization variant shown above is less intuitive and more error-prone to implement.
- The theoretical advantage over momentum is less clear for non-convex deep learning objectives.

9.2 Adaptive First-Order Methods

9.2.1 AdaGrad

AdaGrad (Duchi et al., 2011)

Let $\mathbf{g}_t = \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t)$ and let $G_t \in \mathbb{R}^{n \times n}$ be the diagonal matrix accumulating the sum of squared historical gradients:

$$G_t = \sum_{\tau=1}^{t} \mathbf{g}_\tau \odot \mathbf{g}_\tau \quad \text{(element-wise square and sum)}$$

The update rule is:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t$$

where ϵ (e.g., 10^{-8}) prevents division by zero and $\odot$ denotes element-wise multiplication. The division and square-root are applied element-wise.

Applications

Natural language processing with sparse features (word embeddings, bag-of-words models), recommendation systems, and any problem where infrequent features carry significant signal.

Trade-offs

Pros:

- Per-parameter adaptive learning rates: parameters with large gradient history receive smaller updates, while rare features with small history receive larger updates.
- No learning-rate schedule is needed in theory; η can be fixed.
- Especially effective for sparse data because the effective learning rate for rarely updated features does not decay rapidly.

Cons:

- G_t is monotonically non-decreasing, so the effective learning rate $\eta/\sqrt{G_t + \epsilon}$ decays toward zero for every parameter.
- In deep learning, training stalls prematurely because the accumulated squared gradient becomes too large.
- Does not incorporate any notion of momentum or direction consistency.

9.2.2 RMSProp

RMSProp (Tieleman & Hinton, 2012)

Instead of an ever-growing sum, RMSProp maintains an exponentially decaying moving average of the squared gradients:

$$\mathbb{E}[\mathbf{g}^2]_t = \beta\,\mathbb{E}[\mathbf{g}^2]_{t-1} + (1-\beta)\,\mathbf{g}_t^2$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{\sqrt{\mathbb{E}[\mathbf{g}^2]_t + \epsilon}}\,\mathbf{g}_t$$

where β (e.g., 0.9) controls the decay rate and ϵ is a small smoothing constant.

Applications

The optimizer of choice for recurrent neural networks (RNNs, LSTMs) before Adam was introduced. Still preferred for non-stationary objectives and some reinforcement learning tasks.

Trade-offs

Pros:

- The denominator uses a moving average, preventing the monotonic decay problem of AdaGrad; learning continues indefinitely.
- Handles non-stationary objectives gracefully because it "forgets" old gradient statistics.
- Normalizes gradient magnitudes per parameter, helping with vanishing/exploding gradients.

Cons:

- Does not incorporate momentum (first-moment estimate), so the direction signal is purely from the current gradient.
- The global learning rate η still needs to be tuned.
- Can be sensitive to the decay rate β on some problems.

9.2.3 Adam (Kingma & Ba, 2014)

Adam — Adaptive Moment Estimation

Adam maintains two exponential moving averages:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\,\mathbf{g}_t \qquad \text{(biased first moment)}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\,\mathbf{g}_t^2 \qquad \text{(biased second moment)}$$

Both estimates are initialized at zero and corrected for bias:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

The parameter update is:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta\,\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Default hyperparameters: $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

Bias Correction Rationale

Without correction, $m_0 = v_0 = 0$ biases the moving averages toward zero during early training. Dividing by $1 - \beta_1^t$ (which grows from $1 - \beta_1$ to 1) counteracts this initialization bias.

Applications

The most widely used adaptive optimizer in deep learning. Default choice for transformers, GANs (sometimes), NLP models, and many computer vision architectures. Works well "out of the box" across a wide range of problems.

Trade-offs

Pros:

- Combines the benefits of RMSProp (adaptive per-parameter rates) and momentum (smoothed direction).
- Bias correction makes early updates well-behaved.
- Robust to the choice of hyperparameters; defaults work for many problems.
- Invariant to diagonal rescaling of the gradients.

Cons:

- Convergence is not guaranteed: Adam can diverge on some convex problems (motivated AMSGrad).
- For certain vision tasks, SGD + momentum can generalize better than Adam because the noise provides implicit regularization that Adam's adaptivity suppresses.
- Couples weight decay with the adaptive learning rate, which diminishes the effectiveness of weight-decay regularization (fixed by AdamW).
- Introduces two additional hyperparameters (β_1, β_2) on top of the learning rate.

9.2.4 AdamW — Decoupled Weight Decay

AdamW (Loshchilov & Hutter, 2019)

AdamW decouples the weight penalty from the adaptive gradient scaling by applying weight decay directly to the parameters:

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \left(\frac{\alpha\, \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda\, \mathbf{w}_{t-1} \right)$$

where λ is the weight-decay coefficient (independent of η) and α is the step size for the gradient step. Equivalently, one can view AdamW as a gradient update followed by a multiplicative parameter shrinkage: $\mathbf{w} \leftarrow \mathbf{w} - \eta\alpha\, \hat{m}/(\sqrt{\hat{v}} + \epsilon)$ then $\mathbf{w} \leftarrow (1 - \eta\lambda)\mathbf{w}$.

Applications

Standard for transformer training (ViT, GPT, BERT fine-tuning) and widely adopted in modern PyTorch/TensorFlow/JAX codebases. Should be the default Adam variant for most tasks.

Trade-offs

Pros:

- Properly decouples weight-decay regularization from the adaptive learning-rate mechanism.
- Consistently yields equal or better generalization than Adam with L_2 regularization.
- λ can be tuned independently of the learning rate; the optimal η is closer to being transferable across weight-decay values.

Cons:

- Marginally more computation per step (one extra element-wise multiplication and subtraction).
- Adds another hyperparameter (λ) that must be tuned along with the learning rate.

9.2.5 AMSGrad, NAdam, Adamax, RAdam, and AdaBelief

AMSGrad (Reddi et al., 2018)

AMSGrad ensures that the effective learning rate is non-increasing by using the *maximum* of all past second-moment estimates:

$$\hat{v}_t = \max(\hat{v}_{t-1}, v_t), \qquad \mathbf{w}_{t+1} = \mathbf{w}_t - \eta \, \frac{m_t}{\sqrt{\hat{v}_t} + \epsilon}$$

This prevents the pathological cases where Adam fails to converge on simple one-dimensional convex problems.

NAdam (Dozat, 2016)

NAdam incorporates Nesterov momentum into Adam. Instead of using m_t directly, it substitutes a look-ahead version:

$$m_t^{\text{NAdam}} = (1 - \beta_1^t)\, \mathbf{g}_t + \beta_1^{t+1}\, m_t$$

and then updates with $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \, m_t^{\text{NAdam}} / (\sqrt{\hat{v}_t} + \epsilon)$.

Adamax (Kingma & Ba, 2014)

Replaces the L_2-norm-based second moment with an L_∞ norm that uses the exponentially weighted infinity norm:

$$u_t = \max(\beta_2\, u_{t-1}, |\mathbf{g}_t|), \qquad \mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{u_t}\, m_t$$

More robust to large gradient outliers; no bias correction needed for u_t. Empirically useful for embedding-heavy models.

RAdam — Rectified Adam (Liu et al., 2020)

RAdam applies a rectification term that switches from SGD to Adam after the variance of the adaptive learning rate has stabilized:

$$\rho_t = \rho_\infty - \frac{2t\, \beta_2^t}{1 - \beta_2^t}, \quad \rho_\infty = \frac{2}{1 - \beta_2} - 1$$

When $\rho_t > 4$, the adaptive update is applied; otherwise the variance is deemed too high and a simple SGD step is used.

AdaBelief (Zhuang et al., 2020)

AdaBelief adapts the step size according to the *belief* in the observed gradient direction. The second moment is replaced by the variance of the gradient around the predicted value:

$$s_t = \beta_2 s_{t-1} + (1-\beta_2)(\mathbf{g}_t - m_t)^2 + \epsilon$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{s}_t} + \epsilon}$$

When $\mathbf{g}_t$ deviates significantly from the EMA prediction m_t, the denominator grows (smaller step) and vice versa.

Comparative Overview of Adam Variants

- **AMSGrad:** Guaranteed convergence, but often slower in practice.
- **NAdam:** Occasionally better convergence rate; marginal improvement over Adam on most benchmarks.
- **Adamax:** Useful when gradient norms have heavy tails; simpler update than Adam.
- **RAdam:** Stabilizes early training by not adapting prematurely; robust to learning-rate choice.
- **AdaBelief:** Achieves both fast convergence (like Adam) and good generalization (like SGD); strong on GANs and image classification.

9.3 Second-Order Methods

9.3.1 Newton's Method

Newton's Method

Newton's method uses the inverse Hessian matrix $\mathbf{H} = \nabla^2_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t)$ to scale the gradient, yielding a quadratic approximation of the objective:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{H}^{-1} \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t)$$

The update can be derived from the second-order Taylor expansion:

$$\mathcal{L}(\mathbf{w}_t + \Delta\mathbf{w}) \approx \mathcal{L}(\mathbf{w}_t) + \nabla\mathcal{L}(\mathbf{w}_t)^T \Delta\mathbf{w} + \tfrac{1}{2}\Delta\mathbf{w}^T \mathbf{H}\, \Delta\mathbf{w}$$

Minimizing this quadratic in $\Delta\mathbf{w}$ gives $\Delta\mathbf{w} = -\mathbf{H}^{-1}\nabla\mathcal{L}(\mathbf{w}_t)$.

Applications

Logistic regression, convex optimization in low dimensions, interior-point methods, and as a local optimizer for fine-tuning after SGD. The Hessian-free approach (Martens, 2010) approximates Newton steps for neural networks.

Trade-offs

Pros:

- Quadratic convergence rate near the optimum: the error approximately squares each iteration.
- Invariant to affine reparameterizations of the input space; the trajectory is the same regardless of scaling.
- The Hessian provides curvature information that enables much larger effective steps in flat directions.

Cons:

- Computing the full Hessian costs $O(n^2)$ memory and $O(n^3)$ for inversion, infeasible for modern networks with millions of parameters.
- Requires the Hessian to be positive-definite for guaranteed descent; in non-convex regions the step may increase the loss.
- No stochastic variant: full-batch gradient and Hessian are required.

9.3.2 BFGS and L-BFGS

BFGS (Broyden–Fletcher–Goldfarb–Shanno)

BFGS is a quasi-Newton method that iteratively builds an approximation $B_t \approx \mathbf{H}$ of the Hessian using only first-order gradient information. Given $\mathbf{s}_t = \mathbf{w}_{t+1} - \mathbf{w}_t$ and $\mathbf{y}_t = \nabla\mathcal{L}_{t+1} - \nabla\mathcal{L}_t$, the inverse Hessian approximation H_t is updated as:

$$H_{t+1} = \left(I - \frac{\mathbf{s}_t\mathbf{y}_t^T}{\mathbf{y}_t^T\mathbf{s}_t}\right)H_t\left(I - \frac{\mathbf{y}_t\mathbf{s}_t^T}{\mathbf{y}_t^T\mathbf{s}_t}\right) + \frac{\mathbf{s}_t\mathbf{s}_t^T}{\mathbf{y}_t^T\mathbf{s}_t}$$

The update is then $\mathbf{w}_{t+1} = \mathbf{w}_t - H_t\nabla\mathcal{L}_t$.

L-BFGS (Limited-memory BFGS)

L-BFGS avoids storing the full $n \times n$ matrix by keeping only the last m vector pairs $\{(\mathbf{s}_i, \mathbf{y}_i)\}_{i=t-m+1}^{t}$ and computing the matrix-vector product $H_t\mathbf{g}_t$ via the two-loop recursion algorithm. Memory cost: $O(mn)$, where m is typically 5–20.

Applications

Training of small-to-medium fully connected networks, logistic regression with many features, style-transfer optimization, and deterministic convex problems where batch size equals dataset size.

Trade-offs

Pros:

- Superlinear convergence without explicit Hessian computation.
- L-BFGS memory cost is only $O(mn)$, practical for problems with tens of thousands of parameters.
- The line-search step (common in BFGS implementations) guarantees monotonic loss decrease.

Cons:

- Stochastic (mini-batch) BFGS does not work well because the gradient differences

$\mathbf{y}_t$ become inconsistent between batches.
- Sequential by nature; the two-loop recursion does not parallelize easily.
- Not suitable for very large neural networks; SGD/Adam remain dominant for deep learning.

9.3.3 Kronecker-Factored Approximate Curvature (KFAC)

KFAC (Martens & Grosse, 2015)

For a neural-network layer with input $\mathbf{a}$ and pre-activation $\mathbf{s} = W\mathbf{a}$, the Fisher information matrix is approximated as a Kronecker product of two smaller matrices:

$$\mathbf{F} \approx \mathbb{E}[\mathbf{a}\mathbf{a}^T] \otimes \mathbb{E}[\nabla_{\mathbf{s}}\mathcal{L}\,(\nabla_{\mathbf{s}}\mathcal{L})^T]$$

where $\otimes$ is the Kronecker product. Inverting the Kronecker product reduces to inverting each factor separately: $(\mathbf{A} \otimes \mathbf{B})^{-1} = \mathbf{A}^{-1} \otimes \mathbf{B}^{-1}$. Each factor is inverted after adding a damping term $\lambda\mathbf{I}$.

Applications

Large-scale supervised learning where natural-gradient descent is desired, distributed training settings, and reinforcement learning (ACKTR).

Trade-offs

Pros: Efficient natural-gradient approximation; per-layer factorization reduces inversion cost from $O(n^3)$ to $O(d_{\text{in}}^3 + d_{\text{out}}^3)$. **Cons:** Complex implementation requiring layer-type-specific curvature blocks; empirical gains over SGD are mixed for many benchmarks.

9.3.4 Shampoo, AdaHessian, and Sophia

Shampoo (Gupta et al., 2018)

Generalizes AdaGrad to matrix-shaped parameters by maintaining a full-matrix preconditioner in each tensor dimension. For a weight matrix $W \in \mathbb{R}^{m\times n}$, Shampoo keeps two preconditioners $L_t \in \mathbb{R}^{m\times m}$ and $R_t \in \mathbb{R}^{n\times n}$:

$$L_t = \epsilon I_m + \sum_{\tau=1}^{t} \mathbf{g}_\tau \mathbf{g}_\tau^T, \quad R_t = \epsilon I_n + \sum_{\tau=1}^{t} \mathbf{g}_\tau^T \mathbf{g}_\tau$$

Update: $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta\, L_t^{-1/4}\, \mathbf{g}_t\, R_t^{-1/4}$.

AdaHessian (Yao et al., 2021)

Incorporates the diagonal of the Hessian, estimated via Hutchinson's method using random vectors $\mathbf{z} \sim \{\pm 1\}$:

$$\text{diag}(\mathbf{H}) \approx \frac{1}{N} \sum_{i=1}^{N} \mathbf{z}_i \odot \big(H(\mathbf{z}_i)\big)$$

where $H(\mathbf{z}) = \nabla(\mathbf{z}^T \nabla \mathcal{L})$ is a Hessian-vector product. The diagonal is smoothed by EMA and used in place of the squared gradient in Adam.

Sophia (Liu et al., 2024)

A lightweight second-order method that clips the Hessian-based preconditioned gradient:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \cdot \text{clip}\Big(\frac{m_t}{\max\{\hat{h}_t, \epsilon\}},\ \gamma\Big)$$

where $\hat{h}_t$ estimates the diagonal Hessian via Hutchinson's method and γ is a clipping threshold. Sophia achieves $2\times$ faster language-model pre-training than Adam.

9.4 Specialized Methods

9.4.1 Projected Gradient Descent (PGD)

Projected Gradient Descent

When parameters are constrained to a feasible set $\mathcal{C}$, PGD projects the unconstrained GD step back onto $\mathcal{C}$:

$$\mathbf{w}_{t+1} = \text{Proj}_{\mathcal{C}}\big(\mathbf{w}_t - \eta \nabla \mathcal{L}(\mathbf{w}_t)\big)$$

The projection $\text{Proj}_{\mathcal{C}}(\mathbf{z}) = \arg\min_{\mathbf{w} \in \mathcal{C}} \|\mathbf{w} - \mathbf{z}\|_2$. For common constraints (e.g., ℓ_∞ ball, $\{\mathbf{w} \mid \|\mathbf{w}\|_2 \le R\}$), the projection has a closed form.

Applications

Adversarial training (PGD attacks), Wasserstein GANs (weight clipping or gradient penalty on the Lipschitz constraint), constrained reinforcement learning, and convex optimization with simplex or box constraints.

Adversarial Training Context

PGD is the de facto standard for generating adversarial examples and for adversarial training. The multi-step variant $\mathbf{x}_{k+1} = \text{Proj}_{\mathcal{B}_\epsilon(\mathbf{x})}(\mathbf{x}_k + \alpha\ \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}))$ maximizes the loss within an ℓ_∞ ball of radius ϵ.

9.4.2 Proximal Gradient Method

Proximal Gradient Method

For a composite objective $\mathcal{L}(\mathbf{w}) = f(\mathbf{w}) + g(\mathbf{w})$ where f is smooth and convex and g is convex but possibly non-smooth (e.g., L_1 norm), the proximal gradient step is:

$$\mathbf{w}_{t+1} = \text{prox}_{\eta g}\big(\mathbf{w}_t - \eta \nabla f(\mathbf{w}_t)\big)$$

where the proximal operator is defined as:

$$\text{prox}_{\eta g}(\mathbf{v}) = \arg\min_{\mathbf{w}} \Big(g(\mathbf{w}) + \frac{1}{2\eta}\|\mathbf{w} - \mathbf{v}\|_2^2\Big)$$

For $g(\mathbf{w}) = \lambda\|\mathbf{w}\|_1$, the proximal operator is the soft-thresholding function:

$$[\text{prox}_{\eta\lambda\|\cdot\|_1}(\mathbf{v})]_i = \text{sign}(v_i) \cdot \max(|v_i| - \eta\lambda, 0)$$

Applications

LASSO regression, elastic-net, sparse coding, matrix completion with nuclear-norm regularization, and structured sparsity.

9.4.3 FISTA (Nesterov Acceleration for Proximal Methods)

FISTA

FISTA (Fast Iterative Shrinkage-Thresholding Algorithm) augments the proximal gradient method with Nesterov momentum:

$$\mathbf{y}_t = \mathbf{w}_t + \frac{k_t - 1}{k_{t+1}}(\mathbf{w}_t - \mathbf{w}_{t-1})$$
$$\mathbf{w}_{t+1} = \text{prox}_{\eta g}\big(\mathbf{y}_t - \eta\,\nabla f(\mathbf{y}_t)\big)$$

with $k_{t+1} = \frac{1}{2}(1 + \sqrt{1 + 4k_t^2})$, $k_0 = 1$. Achieves $O(1/t^2)$ convergence for convex composite problems, optimal among first-order methods.

Applications

Image deblurring, compressed sensing, nuclear-norm minimization, and total- variation denoising.

9.4.4 Conjugate Gradient (CG)

Conjugate Gradient for Quadratic Problems

CG solves the linear system $\mathbf{Ax} = \mathbf{b}$ for symmetric positive-definite $\mathbf{A}$ without storing the matrix explicitly. It generates a sequence of A-orthogonal (conjugate) search directions $\{\mathbf{p}_k\}$:

$$\alpha_k = \frac{\mathbf{r}_k^T\mathbf{r}_k}{\mathbf{p}_k^T\mathbf{A}\mathbf{p}_k}, \quad \mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k\mathbf{p}_k$$
$$\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k\mathbf{A}\mathbf{p}_k, \quad \beta_k = \frac{\mathbf{r}_{k+1}^T\mathbf{r}_{k+1}}{\mathbf{r}_k^T\mathbf{r}_k}$$
$$\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k\mathbf{p}_k$$

Only matrix-vector products $\mathbf{A}\mathbf{p}_k$ are required.

Applications

Hessian-free optimization (Martens, 2010) uses CG to solve $\mathbf{H}\Delta\mathbf{w} = -\nabla\mathcal{L}$ for the Newton step without building $\mathbf{H}$. Also used in Krylov subspace methods and large-scale linear systems.

9.4.5 Stochastic Gradient Langevin Dynamics (SGLD)

SGLD (Welling & Teh, 2011)

SGLD injects Gaussian noise into SGD to perform approximate Bayesian posterior sampling. The update rule is:

$$\Delta \mathbf{w} = \frac{\eta_t}{2} \nabla_{\mathbf{w}} \log p(\mathbf{w} \mid \mathcal{D}) + \boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \eta_t \, \mathbf{I})$$

where $\log p(\mathbf{w} \mid \mathcal{D}) = \log p(\mathcal{D} \mid \mathbf{w}) + \log p(\mathbf{w})$ is the log-posterior (up to a constant). The noise variance η_t must decay to zero at a rate satisfying $\sum \eta_t = \infty$, $\sum \eta_t^2 < \infty$.

Applications

Bayesian deep learning, uncertainty estimation in neural networks, posterior sampling for probabilistic models, and exploring multi-modal posteriors.

Trade-offs

Pros: Asymptotically samples from the true posterior; simple to implement on top of existing SGD code. **Cons:** Slow mixing for high-dimensional posteriors; the learning-rate schedule must satisfy Robbins–Monro conditions; can be unstable if noise and gradient scales are mismatched.

9.5 Distributed Optimization

Synchronous SGD (All-Reduce)

Each of K workers computes a gradient on its local mini-batch $\mathcal{B}_k$. The gradients are summed (or averaged) across all workers via an all-reduce collective communication primitive, then the central parameter server (or each worker, in a ring-all-reduce topology) applies the update:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \, \frac{1}{K} \sum_{k=1}^{K} \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t; \mathcal{B}_k)$$

The effective batch size equals $K \cdot |\mathcal{B}_k|$. Scaling the learning rate linearly with the batch size is common practice.

Asynchronous SGD

Workers update parameters *without* waiting for others. Staleness occurs: worker k computes a gradient at $\mathbf{w}_{t-\tau_k}$ but applies it to $\mathbf{w}_t$, where τ_k is the staleness. Convergence is maintained under bounded staleness, but the effective gradient is "stale." Used in the Downpour SGD architecture (Dean et al., 2012).

Elastic Averaging SGD (EASGD)

Workers maintain local copies $\mathbf{w}^{(k)}$ of the parameters and periodically synchronize with a central model $\tilde{\mathbf{w}}$ via an elastic force:

$$\mathbf{w}_{t+1}^{(k)} = \mathbf{w}_t^{(k)} - \eta\big(\nabla \mathcal{L}_k + \alpha(\mathbf{w}_t^{(k)} - \tilde{\mathbf{w}})\big)$$

The center is updated as $\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} + \beta \sum_k (\mathbf{w}^{(k)} - \tilde{\mathbf{w}})$. Encourages exploration of different basins.

Applications

Large-scale distributed training of deep networks across GPU clusters; synchronous for data-center training, asynchronous for geographically distributed or heterogeneous workers.

Hogwild! — Lock-Free Parallel SGD

For sparse problems (e.g., matrix factorization, word embeddings), Hogwild! allows threads to update shared parameters without locking. Collisions (writes to the same coordinate) cause negligible corruption when gradients are sparse because the probability of overlapping writes is low. This achieves near-linear speedup on multi-core machines.

9.6 Zeroth-Order / Black-Box Optimization

9.6.1 Evolution Strategies (ES)

Natural Evolution Strategies (NES)

The gradient of the expected fitness under a Gaussian search distribution is estimated via the log-likelihood trick:

$$\nabla_{\mathbf{w}} \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(0,\sigma^2 I)}[\mathcal{L}(\mathbf{w} + \boldsymbol{\epsilon})] = \frac{1}{\sigma} \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(0,\sigma^2 I)}[\mathcal{L}(\mathbf{w} + \boldsymbol{\epsilon})\, \boldsymbol{\epsilon}]$$

In practice, antithetic sampling reduces variance:

$$\nabla \mathcal{L} \approx \frac{1}{2N\sigma} \sum_{i=1}^{N} \left[\mathcal{L}(\mathbf{w} + \sigma \boldsymbol{\epsilon}_i) - \mathcal{L}(\mathbf{w} - \sigma \boldsymbol{\epsilon}_i)\right] \boldsymbol{\epsilon}_i$$

Applications

Reinforcement learning (OpenAI-ES for locomotion, Atari), hyperparameter optimization, neural architecture search, and problems where the objective is non-differentiable or a black-box simulator.

Trade-offs

Pros: Fully parallelizable (populations sample independently); no backpropagation required; handles non-differentiable objectives; works with any reward/feedback mechanism. **Cons:** High variance (scales as $O(d/\sigma^2)$ in parameter dimension d); sample-inefficient compared to gradient-based methods; performance depends critically on σ.

9.6.2 CMA-ES — Covariance Matrix Adaptation Evolution Strategy

CMA-ES

CMA-ES models the search distribution as a full multivariate Gaussian $\mathcal{N}(\mathbf{m}, \sigma^2\mathbf{C})$ where $\mathbf{m}$ is the mean, σ is the global step size, and $\mathbf{C} \in \mathbb{R}^{n\times n}$ is the covariance matrix. In each generation:

1. Sample λ offspring: $\mathbf{x}_k = \mathbf{m} + \sigma\,\mathbf{B}\mathbf{D}\mathbf{z}_k$, $\mathbf{z}_k \sim \mathcal{N}(0, I)$, with $\mathbf{C} = \mathbf{B}\mathbf{D}^2\mathbf{B}^T$.
2. Evaluate fitness and keep $\mu < \lambda$ best.
3. Update $\mathbf{m}$ as weighted average of selected offspring.
4. Update $\mathbf{C}$ using rank-μ and rank-one updates (evolution path).
5. Update σ adaptively using a cumulative step-size path.

Applications

Continuous black-box optimization in low-to-medium dimensions ($n < 1000$), robotics controller tuning, physical simulation calibration, and hyperparameter optimization.

Trade-offs

Pros: Invariant to rotation and scaling; self-adapts the full covariance; quasi-parameter-free.
Cons: $O(n^2)$ memory and per-generation cost; inefficient for $n \gg 1000$; requires large population sizes in high dimensions.

9.6.3 Bayesian Optimization Acquisition Functions

Expected Improvement (EI)

Given a surrogate model (typically a Gaussian process) with posterior mean $\mu(x)$ and variance $\sigma^2(x)$, and current best observation f^*, the expected improvement is:

$$\alpha_{\text{EI}}(x) = \mathbb{E}[\max(f(x) - f^*, 0)] = \big(\mu(x) - f^*\big)\,\Phi(Z) + \sigma(x)\,\phi(Z)$$

where $Z = (\mu(x) - f^*)/\sigma(x)$ and Φ, ϕ are the Gaussian CDF and PDF, respectively.

Upper Confidence Bound (UCB)

$$\alpha_{\text{UCB}}(x) = \mu(x) + \kappa\,\sigma(x)$$

The parameter κ trades off exploration ($\kappa \uparrow$) vs. exploitation ($\kappa \downarrow$). A common choice is the GP-UCB schedule with $\kappa_t = \sqrt{\nu \log t}$ that provably achieves no-regret.

Probability of Improvement (PI) and Thompson Sampling

$$\alpha_{\text{PI}}(x) = \mathbb{P}\big(f(x) \le f^* - \xi\big) = \Phi\left(\frac{f^* - \xi - \mu(x)}{\sigma(x)}\right)$$

Thompson Sampling draws a sample $\tilde{f} \sim \text{GP}(\mu, K)$ and selects $x_{\text{next}} = \arg\min_x \tilde{f}(x)$.

Applications

Hyperparameter tuning (learning rate, layer sizes, regularization coefficients), neural architecture search, A/B testing, and experimental design.

9.7 Specialized Training Techniques

9.7.1 Stochastic Weight Averaging (SWA)

SWA (Izmailov et al., 2018)

SWA averages the weights along the SGD/Adam trajectory after an initial standard training phase, typically using a cyclical or constant learning rate:

$$\mathbf{w}_{\text{SWA}} = \frac{1}{T} \sum_{\tau=T_{\text{start}}}^{T_{\text{end}}} \mathbf{w}_\tau$$

Unlike Polyak (EMA of weights), SWA uses an equal-weighted average of checkpoints and only starts averaging near convergence.

Applications

Improving generalization of CNNs on ImageNet, semantic segmentation, and other tasks where SGD plateaus at suboptimal points.

Trade-offs

Pros: No additional training cost (weights averaged from saved checkpoints); finds flatter, wider minima that generalize better; compatible with any optimizer.
Cons: Requires storing checkpoints or maintaining a running sum; needs a separate batch-norm re-estimation pass for models with BN layers.

9.7.2 DP-SGD — Differentially Private SGD

DP-SGD (Abadi et al., 2016)

DP-SGD guarantees (ϵ, δ)-differential privacy:

1. Compute per-example gradients $\mathbf{g}_i$.
2. Clip each gradient: $\bar{\mathbf{g}}_i = \mathbf{g}_i / \max(1, \|\mathbf{g}_i\|_2 / C)$.
3. Aggregate and add Gaussian noise: $\tilde{\mathbf{g}} = \frac{1}{B}\left(\sum_i \bar{\mathbf{g}}_i + \mathcal{N}(0, \sigma^2 C^2 \mathbf{I})\right)$.
4. Update parameters with $\tilde{\mathbf{g}}$.

The privacy budget (ϵ, δ) is tracked via the moments accountant.

Applications

Training models on sensitive data (medical records, user interactions) while providing formal privacy guarantees. Federated learning with differential privacy.

Trade-offs

Pros: Formal, provable privacy guarantee; can be added to any SGD-based training loop.
Cons: Substantial accuracy degradation for tight privacy budgets; computationally expensive (per-example gradient clipping requires removing batch-level vectorization); noise scale σ must be calibrated to the desired (ϵ, δ).

9.8 Comprehensive Optimizer Comparison

Table 9.1: Comparison of major optimization algorithms.

Algorithm	Type	Key Idea	Adaptive?	Momentum?	Second-O
GD	First-order	Full-batch gradient	No	No	No
SGD	First-order	Mini-batch stochastic gradient	No	No	No
Momentum	First-order	EMA of gradients	No	Yes	No
NAG	First-order	Look-ahead gradient	No	Yes	No
AdaGrad	First-order	Per-param sum of squared grads	Yes	No	No
RMSProp	First-order	EMA of squared gradients	Yes	No	No
Adam	First-order	EMA of gradient and squared grad	Yes	Yes	No
AdamW	First-order	Decoupled weight decay	Yes	Yes	No
Newton	Second-order	Inverse Hessian	Yes	No	Yes
L-BFGS	Quasi-Newton	Approx. inverse Hessian via vecs	Yes	No	Yes
KFAC	Second-order	Kronecker-factored Fisher	Yes	No	Yes
SGLD	Stochastic	Langevin noise for sampling	No	No	No
ES	Zeroth-order	Gaussian perturbations	No	No	No
CMA-ES	Zeroth-order	Full covariance adaptation	Yes	No	No
BayesOpt	Zeroth-order	GP surrogate + acquisition fn	N/A	N/A	N/A

Practical Recommendations

- **Default for most DL:** AdamW with $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.95$ or 0.999.
- **Vision benchmarks:** SGD + momentum ($\beta = 0.9$) + cosine LR schedule often gives best final accuracy.
- **Transformers / LLMs:** AdamW, Adam with warmup + cosine decay, or Sophia for pretraining speed.
- **Fine-tuning:** AdamW with small η and linear warmup.
- **Small data / convex:** L-BFGS, Newton-CG, or trust-region methods.
- **Black-box:** Bayesian optimization for low-dim (< 20); CMA-ES for medium-dim (< 1000); ES for high-dim with parallel evaluation.

Chapter 10

Attention Mechanisms

Attention mechanisms have revolutionized deep learning by enabling models to dynamically weigh the importance of different parts of the input. From the original additive and dot-product formulations to modern linear, sparse, and position-biased variants, this chapter systematically covers the full spectrum of attention architectures and their mathematical foundations.

10.1 Scaled Dot-Product Attention — Core Formulation

Scaled Dot-Product Attention (Vaswani et al.

Given queries $Q \in \mathbb{R}^{n \times d_k}$, keys $K \in \mathbb{R}^{m \times d_k}$, and values $V \in \mathbb{R}^{m \times d_v}$, scaled dot-product attention is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

where the softmax is applied row-wise. In self-attention, Q, K, V are all projections of the same input sequence $X \in \mathbb{R}^{n \times d}$: $Q = XW^Q$, $K = XW^K$, $V = XW^V$.

Why Scale by $\sqrt{d_k}$?

The dot products $q_i \cdot k_j$ have variance proportional to d_k when q, k entries are independent with zero mean and unit variance. Without scaling, $\|QK^T\|_\infty$ grows as $\sqrt{d_k}$, pushing the softmax into regions with extremely small gradients (the saturated tail of the exponential). Scaling by $1/\sqrt{d_k}$ keeps the softmax inputs at a variance of ≈ 1, avoiding the vanishing-gradient problem.

10.1.1 Multi-Head Attention (MHA)

Multi-Head Attention

Instead of a single attention function, MHA runs h attention "heads" in parallel and concatenates their outputs:

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$$
$$\text{head}_i = \text{Attention}(QW_i^Q,\, KW_i^K,\, VW_i^V)$$

where $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$. Typically $d_k = d_v = d_{\text{model}}/h$.

Attention Mask Variants

- **Bidirectional (encoder):** Full unmasked attention; each position attends to all positions.
- **Causal (decoder):** Upper-triangular mask M with $M_{ij} = -\infty$ for $j > i$; prevents attending to future tokens.

- **Cross-attention:** Q comes from the decoder (target sequence), K, V come from the encoder (source sequence). Used in encoder-decoder transformers, CLIP, and Stable Diffusion for text conditioning.

Multi-Head Attention Trade-offs

Pros:
- Parallel heads attend to different representation subspaces (syntactic vs. semantic, short-range vs. long-range).
- The concatenation-and-project step combines the information from all subspaces.
- Inductive bias: permutation equivariance (without positional encoding, attention is invariant to token order).

Cons:
- $O(n^2)$ time and memory complexity in sequence length; quadratic scaling limits context length.
- Each head can become redundant; pruning studies show many heads can be dropped without significant accuracy loss.
- High memory footprint during training due to the $n \times n$ attention matrix (unless FlashAttention is used).

10.2 Efficient / Linear Attention

10.2.1 Linformer

Linformer (Wang et al.

The Linformer circumvents the $O(n^2)$ bottleneck by projecting the $(n \times d_k)$ key and value matrices into a lower $(k \times d_k)$ dimension $(k \ll n)$ before computing attention:

$$\text{Attention}_{\text{Lin}}(Q, K, V) \approx \text{softmax}\left(\frac{Q\,(EK)^T}{\sqrt{d_k}}\right)(FV)$$

where $E, F \in \mathbb{R}^{k \times n}$ are learnable projection matrices that both compress along the sequence-length dimension. The complexity drops to $O(nk) \approx O(n)$ when k is fixed.

Applications

Long-document NLP, protein sequence modeling, and scenarios where $n \gg 1000$ would make standard attention infeasible.

Trade-offs

Pros: Provably low-rank approximation of the softmax attention matrix; $O(n)$ complexity.
Cons: Fixed k limits maximum context; projection matrices E, F must be retrained for different sequence lengths; loses per-token fine-grained attention.

10.2.2 Performer — FAVOR+

Performer (Choromanski et al.

The Performer applies a kernel trick to avoid computing the full $n \times n$ attention matrix. The softmax kernel is approximated by random feature maps $\phi(\cdot)$:

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \approx \widehat{D}^{-1}\phi(Q)\,\phi(K)^T$$

where $\phi(\mathbf{x}) = \frac{1}{\sqrt{m}}\left[\exp(\mathbf{w}_1^T\mathbf{x} - \|\mathbf{x}\|^2/2), \ldots, \exp(\mathbf{w}_m^T\mathbf{x} - \|\mathbf{x}\|^2/2)\right]$ and $\widehat{D} = \text{diag}\big(\phi(Q)(\phi(K)^T\mathbf{1}_n)\big)$.
The key insight: compute $\phi(Q)(\phi(K)^TV)$ in $O(n)$ by multiplying $\phi(K)^TV$ first (an $(m \times n) \times (n \times d)$ product yielding $O(nmd)$), then multiply $\phi(Q)$ by the result. This avoids the $(n \times n)$ intermediate.

FAVOR+ Orthogonal Random Features

FAVOR+ uses *orthogonal* random features (random orthogonal matrices instead of i.i.d. Gaussian) to reduce the variance of the kernel approximation, providing a tighter guarantee for the same number of features m.

Trade-offs

Pros: Unifies causal and bidirectional attention in one framework; provable approximation guarantees; $O(n)$ complexity.
Cons: Approximation degrades as the softmax temperature increases; random-feature variance can affect training stability; more FLOPs per token than standard attention for moderate n.

10.2.3 Linear Transformer

Linear Transformer (Katharopoulos et al.

Replacing the softmax with a general kernel $\kappa(\mathbf{q}, \mathbf{k}) = \phi(\mathbf{q})^T\phi(\mathbf{k})$ yields a recurrent formulation:

$$V_i' = \frac{\sum_{j=1}^{i}\phi(K_j)\,V_j^T}{\sum_{j=1}^{i}\phi(K_j)}$$

The numerator and denominator are maintained as running sums (a "matrix-valued memory" and a "scalar normalization"), enabling causal attention in $O(nd)$ time and $O(d^2)$ recurrent state. For $\phi(x) = \text{elu}(x) + 1$, the kernel approximates softmax.

Applications

Autoregressive sequence generation (causal attention), streaming inference, and long-range arena benchmarks.

Trade-offs

Pros: True $O(n)$ causal attention with constant memory per step; naturally handles infinite-length streams.
Cons: The recurrent state size $O(d^2)$ can be large for models with high embedding dimension; kernel approximation quality is inferior to exact softmax for some tasks.

10.2.4 Synthesizer, Nyströmformer, and Other Efficient Variants

Synthesizer (Tay et al.

The Synthesizer removes the QK^T dot-product interaction entirely, learning attention patterns directly:

- **Dense Synthesizer:** $A = \text{softmax}(F(X))$ where $F : \mathbb{R}^{n\times d} \to \mathbb{R}^{n\times n}$ is an MLP.
- **Random Synthesizer:** $A = \text{softmax}(R)$ where $R \in \mathbb{R}^{n\times n}$ is a fixed random matrix (learned regardless of the input).
- **Factorized variants:** Decompose the $n \times n$ pattern into two low-rank factors $A = A_1 A_2$.

The output is computed as $A\,(XW^V)$.

Nyströmformer (Xiong et al.

Selects a set of $m \ll n$ "landmark" points from the key sequence and uses the Nyström approximation of the attention matrix:

$$\tilde{S} = \text{softmax}\left(\frac{QK_m^T}{\sqrt{d_k}}\right) \text{softmax}\left(\frac{Q_m K_m^T}{\sqrt{d_k}}\right)^{\dagger} \text{softmax}\left(\frac{Q_m K^T}{\sqrt{d_k}}\right)$$

where $\dagger$ is the Moore–Penrose pseudoinverse. Complexity: $O(nm)$. MoMe (memory-augmented Nyström) further enhances this with landmark memory.

10.3 Sparse Attention Patterns

10.3.1 Sparse Transformer (Child et al., 2019)

Sparse Transformer

Reduces attention density by applying structured sparsity patterns tailored to the data modality:

- **Strided pattern:** Each token attends to every l-th token preceding it (local + strided history).
- **Fixed pattern:** For images, attend within the same row and column (row/column factorization).

Combined, these reduce complexity from $O(n^2)$ to $O(n\sqrt{n})$ for images and $O(n \log n)$ for text, while preserving the ability to propagate information across the full sequence.

Applications

Generating raw audio (WaveNet-scale), high-resolution images, and very long text documents.

10.3.2 Longformer (Beltagy et al., 2020)

Longformer Attention

Longformer deploys three complementary attention patterns:

1. **Sliding window:** Each token attends to w neighbors on each side (local attention, bandwidth w).
2. **Global attention:** A small set of "global" tokens (e.g., [CLS], [SEP]) attend to all tokens and all tokens attend to them.
3. **Dilated sliding window:** Window positions are spaced with increasing gaps (dilation), extending effective receptive field without increasing compute.

Overall complexity: $O(n \cdot w + g \cdot n)$, linear in sequence length.

Applications

Long-document question answering, summarization of lengthy texts, and document-level NLP tasks requiring context windows of 4k+ tokens.

Trade-offs

Pros: Drop-in replacement for standard MHA in pretrained transformers (can be initialized from RoBERTa/BERT weights); linear complexity.
Cons: The local window limits direct long-range interactions for non-global tokens (requires $O(n/w)$ hops through intermediate tokens); dilated patterns can miss important contiguous regions.

10.3.3 BigBird (Zaheer et al., 2020)

BigBird

BigBird's attention graph (viewed as an adjacency matrix over token pairs) is a combination of:

- **Random attention:** Each query attends to r randomly sampled keys (Erdős–Rényi graph).
- **Window (local) attention:** Tokens within a local band.
- **Global attention:** A small set of tokens that attend universally.

The composite graph is a "small-world" expander, ensuring that information propagates between any two nodes in $O(\log n)$ hops. Complexity: $O(n)$ when $w + r + g$ is constant.

Applications

Long-document NLP (PubMed abstracts, legal documents), genomics (DNA sequences with long-range dependencies), and protein structure prediction.

10.3.4 Reformer (Kitaev et al., 2020)

Reformer — LSH Attention

Reformer uses Locality-Sensitive Hashing (LSH) to assign queries and keys to buckets, then computes attention only within buckets:

1. Hash each query and key vector using a random-projection LSH (angular LSH for cosine similarity).
2. Sort tokens by hash bucket.
3. Chunk the sorted sequence into equal-sized blocks; each query attends only to keys in its own chunk and the preceding chunk.
4. Ensure the query can attend to itself by duplicating each query as a key in the same bucket.

Complexity: $O(n \log n)$.

Trade-offs

Pros: Memory-efficient via reversible residual layers (no intermediate activations stored); handles very long sequences.
Cons: LSH bucketing requires careful tuning of the number of hash rounds; large constant factors in practice; accuracy degradation compared to full attention.

10.3.5 Routing Transformer (Roy et al., 2020)

Routing Transformer

Uses online k-means clustering to route queries and keys into clusters, with attention computed only within each cluster:

$$\mu(\mathbf{q}) = \arg\min_{\mu \in \{\mu_1, \ldots, \mu_k\}} \|\mathbf{q} - \mu\|_2$$

The centroid vectors $\{\mu_i\}$ are updated via an exponentially-moving- average of the assigned queries. Each query attends only to keys in the same cluster, yielding $O(nk)$ complexity.

Applications

Language modeling with very long context, protein sequence modeling (PROTEIN-RT), and auto-regressive generation.

10.4 Position-Biased Attention

10.4.1 Relative Position Bias — T5

T5 Relative Position Bias (Raffel et al.

Instead of absolute positional embeddings, T5 adds a learned scalar bias to each attention logit based on the *relative* distance between positions i and j:

$$A_{ij} = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} + b_{f(i-j)}$$

where f maps relative distances into a set of 32 logarithmically-spaced buckets (finer resolution for nearby positions). Each bucket has a learnable bias parameter. The number of parameters is $O(\text{num_buckets})$, independent of sequence length.

T5 Bucketing Function

$$f(d) = \begin{cases} d, & d < \frac{n}{2} \\ \frac{n}{2} + \left\lfloor \frac{\log\left(\frac{d}{\lfloor n/2 \rfloor}\right)}{\log\left(\frac{2m}{n}\right)} \cdot \frac{n}{2} \right\rfloor, & \frac{n}{2} \le d < m \\ n-1, & d \ge m \end{cases}$$

where n is half the number of buckets and m is the maximum distance.

10.4.2 ALiBi — Attention with Linear Biases

ALiBi (Press et al.

ALiBi adds a *non-learned*, linearly-decreasing bias to the attention scores that penalizes attention to distant tokens:

$$A_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j - m_h \cdot (i - j) \quad (\text{for } i \ge j)$$

where m_h is a head-specific slope (a scalar). For H heads, slopes form a geometric sequence, e.g., for $H = 8$: $m \in \{\frac{1}{2}, \frac{1}{2^2}, \ldots, \frac{1}{2^8}\}$. No positional embeddings are learned; the bias is added at every layer.

Applications

Length extrapolation: models trained with ALiBi on 1024-token sequences can be directly applied (with no fine-tuning) to sequences of 2048+ tokens without loss of perplexity. Used in BLOOM.

Trade-offs

Pros: Zero additional parameters; remarkable length-extrapolation ability; simple to implement; works with any attention architecture.
Cons: Only encodes relative distance magnitude, not direction (though the left-to-right bias in causal attention handles direction); less expressive than learned relative-position embeddings for some tasks.

10.4.3 RoPE — Rotary Position Embedding

RoPE (Su et al.

RoPE encodes position information by rotating the query and key vectors in $d/2$-dimensional complex space. For a position m, the rotation matrix $R^d_{\Theta,m}$ applies a block-diagonal rotation with frequencies $\theta_i = 10000^{-2i/d}$, $i = 0, \ldots, d/2 - 1$:

$$f_{\{q,k\}}(\mathbf{x}_m, m) = R^d_{\Theta,m} W_{\{q,k\}} \mathbf{x}_m$$

$$R^d_{\Theta,m} = \begin{pmatrix} \cos m\theta_0 & -\sin m\theta_0 & & & \\ \sin m\theta_0 & \cos m\theta_0 & & & \\ & & \cos m\theta_1 & -\sin m\theta_1 & \\ & & \sin m\theta_1 & \cos m\theta_1 & \\ & & & & \ddots \end{pmatrix}$$

A key property: the attention score between positions m and n depends only on their relative distance:

$$(R^d_{\Theta,m}\,\mathbf{q})^T (R^d_{\Theta,n}\,\mathbf{k}) = \mathbf{q}^T R^d_{\Theta,n-m}\,\mathbf{k}$$

This follows from the rotation property $R_m^T R_n = R_{n-m}$.

RoPE Implementation Details

In practice, RoPE is implemented by splitting each vector into pairs of coordinates and applying 2D rotations:

$$\begin{pmatrix} q^{(2i)} \\ q^{(2i+1)} \end{pmatrix}_{\text{rot}} = \begin{pmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{pmatrix} \begin{pmatrix} q^{(2i)} \\ q^{(2i+1)} \end{pmatrix}$$

where $\theta_i = 10000^{-2i/d}$ as above. This is applied to both Q and K (not V) before the attention computation.

Applications

LLaMA, Llama-2, Mistral, Qwen, and most modern open-source LLMs. Also used in vision transformers and multi-modal models.

Trade-offs

Pros: Encodes relative position naturally; decays with distance (the dot product of rotated vectors oscillates and decays); enables length extrapolation better than absolute sinusoidal embeddings; no learned parameters.
Cons: Computation overhead (though minimal: an element-wise rotation); does not encode absolute position as strongly as learned embeddings for some tasks; the frequency band limits the maximum distinguishable sequence length.

10.5 FlashAttention

FlashAttention (Dao et al.

FlashAttention is an IO-aware exact-attention algorithm that avoids materializing the $N \times N$ attention matrix in GPU high-bandwidth memory (HBM). It computes attention in blocks (tiles), loading small chunks of Q, K, V into fast on-chip SRAM:

1. Partition Q, K, V into blocks small enough to fit SRAM.
2. For each Q block, iterate over K, V blocks; compute softmax in a numerically stable online manner using the "lazy softmax" trick (maintaining running max and sum).
3. Write only the final output O (not the $N \times N$ intermediate) back to HBM.

FlashAttention-2 further improves by parallelizing over the sequence-length dimension and reducing non-matmul operations.

Applications

Training and inference of large language models, vision transformers, and any transformer variant with long contexts. Integrated into PyTorch 2.0+ via `torch.nn.functional.scaled_dot_product_attention`.

Trade-offs

Pros: Exact attention (no approximation); 2–4× speedup and 10–20× memory reduction for long sequences; enables training with longer context windows than previously possible; tiling improves arithmetic intensity.
Cons: Requires careful kernel tuning for different GPU generations; the tiling strategy may not be optimal for all sequence-length and batch-size combinations; introduces CUDA-level implementation complexity.

10.6 Cross-Attention and Multi-Modal Attention

Cross-Attention

In encoder-decoder transformers, cross-attention lets the decoder (queries) attend to the encoder output (keys and values):

$$\text{CrossAttn}(X_{\text{dec}}, X_{\text{enc}}) = \text{Attention}(X_{\text{dec}}W^Q,\ X_{\text{enc}}W^K,\ X_{\text{enc}}W^V)$$

The decoder queries interact with all encoder positions, enabling the model to align source and target tokens.

Multi-Modal Applications

- **Machine translation:** Decoder Q from target language, encoder K, V from source language.
- **CLIP:** Image encoder and text encoder produce aligned embeddings; a contrastive loss on the cosine-similarity matrix aligns paired image–text samples.
- **Stable Diffusion:** Text embeddings (from CLIP text encoder) are injected as keys and values in cross-attention layers within the U-Net denoiser, conditioning image generation on the text prompt.
- **Perceiver:** A small set of learned "latent" queries cross-attend to a large input (e.g., images as pixel arrays), compressing information into a fixed-size bottleneck.

10.7 Graph Attention Networks (GAT)

GAT (Veličković et al.

For a graph with node features $\{\mathbf{h}_i\}_{i=1}^N$ and adjacency $\mathcal{N}(i)$ for each node, GAT computes attention coefficients:

$$\alpha_{ij} = \frac{\exp\big(\text{LeakyReLU}(\mathbf{a}^T[W\mathbf{h}_i \,\|\, W\mathbf{h}_j])\big)}{\sum_{k\in\mathcal{N}(i)} \exp\big(\text{LeakyReLU}(\mathbf{a}^T[W\mathbf{h}_i \,\|\, W\mathbf{h}_k])\big)}$$

where $\mathbf{a} \in \mathbb{R}^{2d'}$ is a learnable attention vector, $W \in \mathbb{R}^{d' \times d}$ is a weight matrix, and $\|$ denotes concatenation. The node update is:

$$\mathbf{h}_i' = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}\, W \mathbf{h}_j\right)$$

Multi-head GAT averages or concatenates multiple independent attention heads, analogous to transformer MHA.

GATv2 — Dynamic Attention

GATv2 (Brody et al., 2022) changes the order of operations to enable *dynamic* attention (the ranking of neighbors can vary per query):

$$\alpha_{ij} = \frac{\exp\!\big(\mathbf{a}^T \,\mathrm{LeakyReLU}(W_1 \mathbf{h}_i + W_2 \mathbf{h}_j)\big)}{\sum_{k \in \mathcal{N}(i)} \exp\!\big(\mathbf{a}^T \,\mathrm{LeakyReLU}(W_1 \mathbf{h}_i + W_2 \mathbf{h}_k)\big)}$$

This simple swap makes the attention function strictly more expressive.

Applications

Node classification, link prediction, molecular property prediction, traffic forecasting, and recommender systems over graph-structured data.

10.8 Other Important Attention Variants

Grouped Query Attention (GQA) and Multi-Query Attention (MQA)

To reduce the memory cost of the KV-cache during autoregressive LLM inference:

- **MQA (Shazeer, 2019):** All query heads share a single key–value head. KV-cache size divided by h.
- **GQA (Ainslie et al., 2023):** Interpolation between MHA and MQA: g groups of query heads, each group sharing one key–value head ($1 < g < h$). KV-cache size divided by h/g.

Used in Llama-2-70B, PaLM, and Falcon.

PagedAttention (vLLM)

Manages the KV-cache in fixed-size "pages" (blocks), akin to virtual memory paging in operating systems. Eliminates memory fragmentation and enables efficient memory sharing across requests in batched LLM inference. The core of the vLLM inference engine.

Talking-Heads

- **Talking-Heads (Shazeer et al., 2021):** Inserts an additional linear projection across heads before the output projection: $[\text{head}_1, \ldots, \text{head}_h] \cdot W^T$, letting heads "talk" to each other.
- **Gated Attention:** A learned scalar gate g modulates the attention output: output $= g \odot \text{Attention}(Q, K, V)$, where g is typically computed as $\sigma(W_g X)$.

- **Mixture-of-Heads:** Treats each head as an "expert" in a sparse mixture-of-experts framework, selecting a subset of heads per token per layer.

10.9 State Space Models — Modern Sequence Models

10.9.1 Mamba — Selective State Space Models

State Space Model (SSM) Foundation

A continuous-time linear state-space model maps an input signal $x(t)$ to an output $y(t)$ through a latent state $h(t)$:

$$h'(t) = \mathbf{A}\,h(t) + \mathbf{B}\,x(t)$$
$$y(t) = \mathbf{C}\,h(t) + \mathbf{D}\,x(t)$$

For discrete sequences, the zero-order hold (ZOH) discretization with step size Δ yields:

$$\overline{\mathbf{A}} = \exp(\Delta\mathbf{A})$$
$$\overline{\mathbf{B}} = (\Delta\mathbf{A})^{-1}(\exp(\Delta\mathbf{A}) - \mathbf{I}) \cdot \Delta\mathbf{B} \approx \Delta\mathbf{B}$$

The discrete SSM computes $h_t = \overline{\mathbf{A}}\,h_{t-1} + \overline{\mathbf{B}}\,x_t$ and $y_t = \mathbf{C}\,h_t$.

S4 — Structured State Space (Gu et al.

S4 initializes $\mathbf{A}$ using the HiPPO matrix, which is mathematically designed to memorize the history of $x(t)$ optimally in the sense of online function approximation. The HiPPO-LegS matrix compresses an infinitely long input sequence into a fixed-dimensional state with linearly decaying memory:

$$A_{nk} = -\begin{cases} \sqrt{(2n+1)(2k+1)}, & n > k \\ n+1, & n = k \\ 0, & n < k \end{cases}$$

Mamba — Selective SSM (Gu & Dao

Mamba introduces *selectivity*: the matrices $\mathbf{B}, \mathbf{C}, \Delta$ become functions of the input x_t:

$$\mathbf{B}_t = s_B(x_t), \quad \mathbf{C}_t = s_C(x_t), \quad \Delta_t = \text{softplus}(s_\Delta(x_t))$$

where s_B, s_C, s_Δ are linear projections. This enables the model to selectively propagate or forget information based on content. The selective scan algorithm computes the SSM recurrence in $O(n)$ via a hardware-aware parallel scan (prefix-sum) kernel.

Applications

Language modeling (Mamba matches transformers at scale), DNA sequence modeling (HyenaDNA), audio generation, and any long-range sequence task.

Mamba vs. Transformer Trade-offs

Pros: $O(n)$ complexity; no quadratic attention; constant per-step recurrent state; competitive with transformers on many benchmarks.
Cons: Relatively new; less mature training infrastructure; selective scan adds implementation complexity; the inductive bias of content-dependent state transitions is not yet fully understood.

10.10 Comprehensive Attention Comparison

Table 10.1: Comparison of attention mechanisms.

Mechanism	Category	Complexity	Exact?	Position Info
Scaled Dot-Product	Dense	$O(n^2)$	Yes	None (add PE)
MHA	Dense	$O(hn^2)$	Yes	None (add PE)
Linformer	Linear	$O(nk)$	No (approx.)	None
Performer	Linear	$O(nmd)$	No (approx.)	Via features
Linear Transformer	Linear (RNN)	$O(nd)$	No (approx.)	Inherent
Synthesizer	No-QK	$O(n^2)$	N/A	Input-dep. pattern
BigBird / Longformer	Sparse	$O(n)$	No (sparse)	Position bias
Reformer	LSH	$O(n \log n)$	No (sparse)	None
T5 Rel. Bias	Position	$O(n^2)$	Yes	Learned relative
ALiBi	Position	$O(n^2)$	Yes	Linear bias
RoPE	Position	$O(n^2)$	Yes	Rotational rel.
FlashAttention	IO-Aware	$O(n^2)$	Yes	None
GQA	Efficient	$O(n^2)$	Yes	None
Mamba (SSM)	Recurrent	$O(n)$	N/A	Inherent

Choosing an Attention Mechanism

- **Standard sequence modeling (n $<$ 2k):** Standard MHA with RoPE or ALiBi, using FlashAttention for efficiency.
- **Long documents (n $>$ 4k):** Longformer or BigBird for encoder-only; Linear Transformer for autoregressive.
- **LLM inference (KV-cache bottleneck):** Grouped Query Attention (GQA) with PagedAttention (vLLM).
- **Ultra-long / streaming sequences:** Mamba (selective SSM) or Linear Transformer.
- **Multi-modal (cross-domain):** Cross-attention with the appropriate masking.
- **Graph-structured data:** GAT / GATv2 multi-head attention.

Chapter 11

Pooling Functions

Pooling functions reduce the spatial or structural dimensionality of feature maps, graph representations, and set-structured data. They enforce translation invariance, control overfitting by summarizing activations, and enable architectures to handle variable-sized inputs. This chapter covers global, local, hierarchical, attention-based, and graph-theoretic pooling methods.

11.1 Global Pooling

11.1.1 Global Average Pooling (GAP)

Global Average Pooling

For a feature tensor $X \in \mathbb{R}^{C \times H \times W}$, global average pooling collapses each channel to a single scalar by averaging over all spatial positions:

$$y_c = \frac{1}{H \cdot W} \sum_{h=1}^{H} \sum_{w=1}^{W} X_{c,h,w}, \qquad c = 1, \ldots, C$$

The output is a vector $\mathbf{y} \in \mathbb{R}^C$.

Applications

- Replaces the fully-connected classifier layers in ResNet, Inception, and DenseNet architectures, turning a convolutional feature extractor into a fully-convolutional network that accepts arbitrary input sizes.
- Used in class activation mapping (CAM): the weights of the final linear layer combine with GAP features to produce discriminative localization maps.

Trade-offs

Pros:

- Zero parameters; no risk of overfitting from pooling.
- Translation invariant: shifting the input does not change the average.
- Enables the network to accept any spatial input size at inference time (fully convolutional).
- Encourages each channel's feature map to correspond to a semantic category (strong regularization effect).

Cons:

- Destroys all spatial information; impossible to know *where* a feature was detected, only that it was present.
- Dilutes strong activations: a small highly-active region may be averaged into near-zero contribution if the rest of the map is inactive.

11.1.2 Global Max Pooling (GMP)

Global Max Pooling

$$y_c = \max_{h=1,\dots,H} \max_{w=1,\dots,W} X_{c,h,w}, \qquad c = 1, \dots, C$$

The output is the maximum activation for each channel across all spatial locations.

Applications

Weakly-supervised object localization: the location of the maximum activation often corresponds to the object's most discriminative part. Also used in some GAN discriminators and metric-learning backbones.

Trade-offs

Pros:

- Emphasizes the most prominent feature; robust to background clutter as long as the target feature is present.
- Provides a natural form of attention: the output is dominated by the single strongest response.

Cons:

- Ignores spatial extent; a single-pixel response suffices to activate the channel, which may mislead training.
- Sensitive to noise: a single outlier activation can dominate the output.
- Zero parameters, but can underutilize feature map information compared to GAP.

11.1.3 Generalized Mean (GeM) Pooling

GeM Pooling (Radenović et al.

GeM interpolates between GAP and GMP via a power parameter p:

$$y_c = \left(\frac{1}{H \cdot W} \sum_{h=1}^{H} \sum_{w=1}^{W} |X_{c,h,w}|^p \right)^{1/p}$$

- $p = 1$: reduces to global average pooling (GAP).
- $p \to \infty$: approaches global max pooling (GMP).
- $p = 3$: a commonly used default.

The parameter p can be fixed, manually tuned, or *learned* via backpropagation (differentiable w.r.t. p).

Gradient of GeM with respect to p

When p is learned, the gradient requires the derivative of the p-norm. For positive inputs:

$$\frac{\partial y_c}{\partial p} = \frac{y_c}{p} \left(\frac{\sum_i X_i^p \log X_i}{\sum_i X_i^p} - \frac{\log\left(\frac{1}{N} \sum_i X_i^p\right)}{p} \right)$$

where the sum runs over $i = (h, w)$ and $N = H \cdot W$.

Applications

Image retrieval and metric learning (GeM is the standard pooling in landmark/historical-building retrieval pipelines); fine-grained recognition where a balance between average and max is beneficial.

Trade-offs

Pros: Unifies GAP and GMP in a single continuous family; learnable p adapts the pooling to the dataset and task.
Cons: Adds one hyperparameter (or learned parameter); gradient through power function can be numerically unstable for large p; modest improvement over GAP for standard classification.

11.2 Local Pooling (Sliding Window)

11.2.1 Max Pooling

Max Pooling — 2D

For an input feature map and a pooling region $\mathcal{R}_{i,j}$ (e.g., a $k \times k$ window with stride s):

$$Y_{i,j,k} = \max_{(p,q)\in\mathcal{R}_{i,j}} X_{k,p,q}$$

Typically $k = 2, s = 2$ (halves spatial dimensions). The operation is applied independently per channel.

Gradient Through Max Pooling — Switch Variable

During backpropagation, the gradient flows only through the position that achieved the maximum (the "argmax" or "switch" position), and is zero elsewhere:

$$\frac{\partial \mathcal{L}}{\partial X_{k,p,q}} = \begin{cases} \frac{\partial \mathcal{L}}{\partial Y_{k,i,j}}, & (p,q) = \arg\max_{(p,q)\in\mathcal{R}_{i,j}} X_{k,p,q} \\ 0, & \text{otherwise} \end{cases}$$

Applications

The canonical downsampling operator in CNNs (AlexNet, VGG). Also used in max-pooling-with-argmax for segmentation networks that need the pooling indices for unpooling layers in the decoder.

Trade-offs

Pros: Provides local translation invariance (small shifts do not change the maximum within the window); reduces spatial dimensions, controlling computational cost and receptive-field growth; introduces a strong locality bias.
Cons: Lossy: discards all non-maximum activations, which can contain useful fine-grained spatial information; the discrete argmax creates a sparser gradient flow (only $1/k^2$ positions receive gradients).

11.2.2 Average Pooling

Average Pooling

$$Y_{i,j,k} = \frac{1}{|\mathcal{R}_{i,j}|} \sum_{(p,q)\in\mathcal{R}_{i,j}} X_{k,p,q}$$

Uniform gradient flow: every element in the pooling window receives $\frac{1}{|\mathcal{R}_{i,j}|} \cdot \frac{\partial \mathcal{L}}{\partial Y_{k,i,j}}$.

Applications

Smoothing layers, transition layers between dense blocks (DenseNet), final feature summarization before fully-connected layers, and in some architectures as a replacement for max pooling to reduce information loss.

Trade-offs

Pros: Smooth gradient flow (all positions contribute); less information loss than max pooling; naturally anti-aliases the feature map when combined with appropriate stride.
Cons: Dilutes strong activations (similar to GAP); can blur edges and fine structures; empirical performance often slightly below max pooling for classification tasks.

11.2.3 L_p Pooling and Stochastic Pooling

L_p Pooling

$$Y_{i,j,k} = \left(\sum_{(p,q)\in\mathcal{R}_{i,j}} |X_{k,p,q}|^p \right)^{1/p}$$

$p = 1$ gives average pooling; $p \to \infty$ gives max pooling. This is the local-window analogue of GeM pooling.

Stochastic Pooling (Zeiler & Fergus

Sample the output from a multinomial distribution over activations within each pooling region:

$$p_{k,p,q} = \frac{X_{k,p,q}}{\sum_{(r,s)\in\mathcal{R}_{i,j}} X_{k,r,s}}, \qquad Y_{k,i,j} = X_{k,p^*,q^*}, \quad (p^*, q^*) \sim \text{Multinomial}(p)$$

At test time, probabilistic averaging is used: $Y_{k,i,j} = \sum p_{k,p,q} X_{k,p,q}$.

Stochastic Pooling Trade-offs

Pros: Acts as a regularizer (like dropout, but in the pooling layer); avoids the deterministic discarding of non-max activations.
Cons: Slower than deterministic pooling due to random sampling; increases training variance; the test-time probabilistic averaging blurs the distinction vs. average pooling.

11.2.4 Fractional Max Pooling

Fractional Max Pooling (Graham

Instead of a fixed regular grid, pooling windows have *random* sizes and positions, producing fractional (non-integer) downsampling ratios. The pooling regions are generated by a randomized pseudorandom sequence such that the expected region size matches the target ratio. This introduces stochasticity in the spatial layout while maintaining differentiable filtering.

Applications

Used as a regularizing alternative to standard max pooling; sometimes combined with dropout for stronger regularization on small datasets.

Trade-offs

Pros: Introduces spatial stochasticity as regularization; can achieve non-integer downsampling ratios.
Cons: Non-deterministic training; tricky to implement efficiently; not widely adopted compared to strided convolutions + dropout.

11.3 Hierarchical / Region-Based Pooling

11.3.1 Spatial Pyramid Pooling (SPP)

SPP (He et al.

SPP pools feature maps at multiple scales simultaneously and concatenates the resulting vectors. For a feature map of size $C \times H \times W$, SPP with levels $\{L_1, L_2, \ldots, L_m\}$ splits the map into $L_i \times L_i$ bins (adaptive pooling), max- or average-pools each bin to a scalar, and concatenates all $C \cdot \sum L_i^2$ values.
Example with levels $\{1, 2, 4\}$: the output dimension is $C \times (1^2 + 2^2 + 4^2) = C \times 21$. Independent of the input spatial size, enabling variable-size inputs to a fully-connected classifier.

Applications

Object detection (SPP-Net), image classification with arbitrary input sizes, and any architecture that transitions from convolutional to fully-connected layers while supporting variable resolution.

Trade-offs

Pros: Handles arbitrary input resolutions; multi-scale representation captures both fine and coarse spatial information; simple to implement via adaptive pooling.
Cons: The concatenated representation can be large (e.g., $21C$ for 3 levels); the fixed grid ignores semantic object boundaries; loses spatial correspondence.

11.3.2 RoI Pooling

RoI Pooling (Girshick

Given a feature map and a region of interest (RoI) defined by a bounding box (x_1, y_1, x_2, y_2), RoI pooling divides the RoI into a fixed $H \times W$ grid (e.g., 7×7). Each grid cell is max-pooled:

$$Y_{c,i,j} = \max_{(x,y) \in \text{bin}(i,j)} X_{c,x,y}$$

where $\text{bin}(i, j)$ defines the (i, j)-th sub-rectangle of the RoI. The RoI coordinates are quantized to the feature-map grid, dividing the RoI into bins of sizes $\lfloor w/H \rfloor$ or $\lceil w/H \rceil$. This quantization introduces misalignment between the RoI and the extracted features.

Applications

Two-stage object detectors: Fast R-CNN, Faster R-CNN. The classifier head receives fixed-size pooled features regardless of the proposal's original aspect ratio or size.

Trade-offs

Pros: Fixed-size output from arbitrary RoIs enables efficient batch processing of region proposals; differentiable (gradients propagate to the feature map).
Cons: Quantization errors: rounding coordinates to integer multiples of the feature-map stride causes misalignment between the RoI and image pixels (the "RoI misalignment" problem); two quantization steps (RoI-to-grid and bin-to-grid) compound the error.

11.3.3 RoI Align

RoI Align (He et al.

RoI Align eliminates quantization by using bilinear interpolation. For each bin in the $H \times W$ grid, it samples $k \times k$ regularly-spaced points (e.g., 2×2) and computes the value at each point via bilinear interpolation:

$$f(x, y) = \sum_{p,q} X_{c,p,q} \max(0, 1 - |x - p|) \max(0, 1 - |y - q|)$$

The bin output is the *average* (or max) of the interpolated values at the k^2 sample points.

Applications

Mask R-CNN for instance segmentation (pixel-level mask prediction requires precise spatial alignment), Keypoint R-CNN, and any detection task where fine localization matters.

Trade-offs

Pros: No quantization; preserves fine spatial alignment; critical for mask prediction (improves mask AP by several points over RoI pooling).
Cons: Slightly more computation than RoI pooling (bilinear interpolation at k^2 points per bin); still a grid-based approximation of the continuous RoI.

11.3.4 PrRoI Pooling and ASPP

Precise RoI Pooling — PrRoI (Jiang et al.

PrRoI pools by integrating the feature map *continuously* over each bin region, without discretization to sample points. For a bin $\mathcal{B} = [x_1, x_2] \times [y_1, y_2]$, the pooling value is:

$$Y_{c,i,j} = \frac{\int\int_{\mathcal{B}} X_c(x, y)\, dx\, dy}{\int\int_{\mathcal{B}} dx\, dy}$$

computed exactly via a piecewise bilinear integral. This is the most precise RoI pooling variant; avoids both coordinate quantization and sampling-point discretization.

ASPP — Atrous Spatial Pyramid Pooling (Chen et al.

ASPP applies multiple parallel dilated (atrous) convolutions with different dilation rates $\{6, 12, 18, 24\}$ to the same feature map, then concatenates the outputs. An image-level global-average-pooling branch is also added and bilinearly upsampled. This captures multi-scale context at the same spatial resolution without increasing parameters or losing resolution.

Applications

Semantic segmentation (DeepLab family), scene parsing, and any dense prediction task requiring multi-scale contextual reasoning.

11.4 Attention-Based Pooling

11.4.1 Attention Pooling

Attention Pooling

Instead of fixed pooling (max or average), attention pooling computes a weighted sum of elements based on learned relevance scores:

$$\mathbf{y} = \sum_{i=1}^{N} \alpha_i\, \mathbf{x}_i, \qquad \alpha_i = \frac{\exp(e_i)}{\sum_{j=1}^{N} \exp(e_j)}$$

where scores $e_i = \mathbf{w}^T \tanh(\mathbf{V}\mathbf{x}_i)$ or $e_i = \mathbf{x}_i^T \mathbf{W}\mathbf{q}$ with a learnable query vector $\mathbf{q}$. The softmax-normalized weights α_i sum to 1 and represent the relative importance of each element.

Applications

Multiple-instance learning (MIL), set-based classification (Deep Sets), point-cloud processing, and sequence aggregation in NLP (replacing simple averaging or last-hidden-state extraction).

Trade-offs

Pros: Learns to focus on task-relevant elements; handles variable-size input sets; produces interpretable attention weights.
Cons: Introduces learnable parameters; the softmax normalization can saturate, focusing all weight on one element; can be sensitive to initialization.

11.4.2 Pooling by Multihead Attention (PMA) — Set Transformer

PMA — Pooling by Multihead Attention (Lee et al.

PMA uses k learnable "seed" vectors $\mathbf{S} \in \mathbb{R}^{k \times d}$ that act as queries in a cross-attention block over the set elements $\mathbf{X} \in \mathbb{R}^{n \times d}$:

$$\mathrm{PMA}_k(\mathbf{X}) = \mathrm{MHA}(\mathbf{S}W^Q, \mathbf{X}W^K, \mathbf{X}W^V)$$

The output is a fixed-size matrix $\mathbb{R}^{k \times d}$ regardless of the set size n. PMA is permutation-equivariant (permuting the input set permutes the output rows accordingly, with k outputs corresponding to k "slots").

Applications

Set Transformer for set-input tasks (point-cloud classification, amodal instance segmentation, meta-learning with set-valued support sets).

Trade-offs

Pros: Permutation equivariant; handles arbitrary set sizes; the k slots can specialize to different set "clusters" or features; integrates naturally with transformer architectures.
Cons: Computationally more expensive than simple pooling ($O(nk)$ cross-attention); requires choosing k a priori; training can be slower than simpler pooling alternatives.

11.5 Graph Pooling

11.5.1 DiffPool — Differentiable Pooling

DiffPool (Ying et al.

DiffPool learns a soft cluster assignment matrix $S^{(l)} \in \mathbb{R}^{n_l \times n_{l+1}}$ at each layer that maps n_l nodes to n_{l+1} clusters:

$$\begin{aligned} S^{(l)} &= \mathrm{softmax}\big(\mathrm{GNN}^{(l)}_{\mathrm{pool}}(A^{(l)}, X^{(l)})\big) \\ X^{(l+1)} &= (S^{(l)})^T X^{(l)} \\ A^{(l+1)} &= (S^{(l)})^T A^{(l)} S^{(l)} \end{aligned}$$

The assignment is produced by a separate GNN run in parallel with the embedding GNN. The adjacency of the coarser graph is the projection of the fine adjacency through the assignment.

Auxiliary Losses for DiffPool

To prevent degenerate assignments (all nodes collapsing into one cluster), DiffPool adds:

- **Link prediction loss:** $\mathcal{L}_{\text{LP}} = \|A^{(l)} - S^{(l)}(S^{(l)})^T\|_F$ to encourage nearby nodes to be assigned together.
- **Entropy regularization:** penalize peaked assignment distributions to encourage softer clustering.

Applications

Graph classification (predicting a label for an entire graph, e.g., molecular property prediction, social-network categorization).

Trade-offs

Pros: End-to-end differentiable hierarchical clustering; produces a permutation-invariant graph-level representation.
Cons: Computationally expensive ($O(n_l^2 n_{l+1})$ for the adjacency projection); the auxiliary losses require careful balancing; training can be unstable; the cluster count n_{l+1} must be specified.

11.5.2 SAGPool — Self-Attention Graph Pooling

SAGPool (Lee et al.

SAGPool uses a graph attention layer to compute importance scores for each node, then selects the top-k nodes:

$$\begin{aligned} \mathbf{z} &= \text{GNN}(A, X) \quad \text{(e.g., a GCN layer)} \\ \text{idx} &= \text{top}_k(\mathbf{z}) \quad \text{(select } k \text{ nodes with highest scores)} \\ X' &= X_{\text{idx},:} \odot \text{sigmoid}(\mathbf{z}_{\text{idx}}) \\ A' &= A_{\text{idx, idx}} \end{aligned}$$

The gate $\text{sigmoid}(\mathbf{z}_{\text{idx}})$ allows the model to attenuate (not just select) the retained features.

Applications

Graph classification, interpretable node selection (the selected nodes reveal which subgraph the model deemed important).

Trade-offs

Pros: Interpretable (selected nodes can be visualized); simple to implement; computationally cheaper than DiffPool ($O(n)$ for scoring).
Cons: Discontinuous selection (top-k) can cause training instability; information from dropped nodes is permanently lost; the pooling ratio k/n is a sensitive hyperparameter; not permutation-invariant for the dropped set.

11.5.3 Top-K Pooling and Edge Pooling

Top-K Pooling (Gao & Ji

A generalization of SAGPool:

$$\mathbf{y} = \frac{\mathbf{X}_{\text{idx},:} \odot \text{sigmoid}(\mathbf{z}_{\text{idx}})}{\|\mathbf{X}_{\text{idx},:}\|}$$

where the normalization by $\|\cdot\|$ ensures bounded activations. The index set idx is obtained by projecting node features to scalars and selecting the top $k = \lceil pn \rceil$ entries ($p \in (0, 1]$ is the pooling ratio).

Edge Pooling (Diehl et al.

Instead of scoring nodes, Edge Pooling scores *edges*, and contracts the highest-scoring edges (merging their endpoints via a learned feature combination). This avoids the information loss of discarding nodes entirely, but requires careful handling of the contracted graph topology.

11.5.4 Wavelet Pooling and LIPooling

Wavelet Pooling (Williams & Li

Uses the discrete wavelet transform (DWT) to decompose feature maps into approximation and detail coefficients. Pooling discards the detail coefficients and keeps the approximation, which provides anti-aliased downsampling without introducing checkerboard artifacts. The inverse DWT reconstructs the upsampled signal.

LIPooling — Local Importance Pooling (Gao et al.

LIPooling learns a per-position importance map via a small convolutional subnetwork and performs weighted downsampling:

$$Y_{c,i,j} = \frac{\sum_{(p,q)\in\mathcal{R}_{i,j}} W_{p,q}\, X_{c,p,q}}{\sum_{(p,q)\in\mathcal{R}_{i,j}} W_{p,q}}$$

where W is a learned smooth importance map (positive by construction, e.g., via exp activation). This adaptively selects which spatial positions are most relevant within each pooling window.

Applications

Wavelet pooling: anti-aliased CNNs for image-to-image translation (pix2pix, CycleGAN) to suppress checkerboard artifacts. LIPooling: adaptive downsampling in architectures where uniform pooling discards task-relevant spatial structure.

Table 11.1: Comparison of pooling methods.

Method	Type	Key Mechanism	Learnable?	Invariant?
GAP	Global	Channel-wise mean	No	Translation
GMP	Global	Channel-wise max	No	Translation
GeM	Global	Generalized mean (p)	Opt.	Translation
Max Pool	Local	Max in window	No	Local shift
Avg Pool	Local	Mean in window	No	Local shift
Stochastic	Local	Sampling over activations	No	Probabilistic
Fractional	Local	Randomized grid	No	Jitter
SPP	Multi-scale	Binning at multiple scales	No	Arbitrary size
RoI Pooling	Region	Quantized binning	No	RoI
RoI Align	Region	Bilinear interpolation	No	RoI
PrRoI	Region	Continuous integral	No	RoI
ASPP	Multi-scale	Dilated conv pyramid	Yes	Translation
Attn Pool	Set	Softmax-weighted sum	Yes	Permutation
PMA	Set	Multi-head cross-attn	Yes	Permutation
DiffPool	Graph	Soft cluster assignment	Yes	Permutation
SAGPool	Graph	Top-k self-attention	Yes	Subset equiv.
Wavelet	Local	DWT approximation coeffs	No	Translation
LIPooling	Local	Learned importance map	Yes	Local shift

11.6 Comprehensive Pooling Comparison

Choosing a Pooling Method

- **Image classification:** GAP after the final conv layer (standard in modern CNNs); Max Pool for intermediate downsampling.
- **Object detection:** RoI Align (or PrRoI) for region-level feature extraction; ASPP for multi-scale context.
- **Instance segmentation:** RoI Align is essential for accurate mask prediction.
- **Image retrieval / metric learning:** GeM pooling with learnable p.
- **Set inputs:** Attention pooling or PMA (Set Transformer).
- **Graph classification:** SAGPool for simplicity and interpretability; DiffPool for hierarchical structure.
- **Anti-aliased downsampling:** Wavelet pooling or LIPooling when checkerboard artifacts are a concern.

Chapter 12

Convolution Operations

Convolution is the foundational operation of deep learning for spatial and structured data. This chapter systematically surveys the wide variety of convolutional operators: standard 2D/3D, depthwise-separable, dilated, deformable, group, channel-shuffled, ghost, conditioned, graph, and equivariant variants, along with their mathematical definitions, usage contexts, and trade-offs.

12.1 Standard Convolution

12.1.1 2D Convolution

2D Discrete Cross-Correlation

In deep learning, "convolution" is technically cross-correlation (the kernel is not flipped). For an input tensor $X \in \mathbb{R}^{C_{\text{in}} \times H \times W}$ and a kernel $W \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k_h \times k_w}$:

$$Y_{c_{\text{out}},\, i,\, j} = \sum_{c_{\text{in}}=0}^{C_{\text{in}}-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} X_{c_{\text{in}},\, i \cdot s+u-p,\, j \cdot s+v-p} \cdot W_{c_{\text{out}},\, c_{\text{in}},\, u,\, v}$$

where s is the stride, p is the padding amount, and the output coordinates (i, j) iterate over the valid range of the convolution.

Output Size Formula

For input spatial size $i \times i$, kernel $k \times k$, padding p, stride s, and dilation d:

$$o = \left\lfloor \frac{i + 2p - k - (k-1)(d-1)}{s} \right\rfloor + 1$$

For "same" padding (output size $= \lceil i/s \rceil$), the pad is $p = \lfloor (s(i-1) - i + k)/2 \rfloor$ when $d = 1$.

Applications

The universal spatial feature extractor: image classification (ResNet, VGG, EfficientNet), object detection (Faster R-CNN, YOLO), semantic segmentation (U-Net, DeepLab), and visual representation learning.

2D Convolution Properties

Key concepts:

- **Parameter sharing:** The same kernel is applied at all spatial positions, dramatically reducing parameter count vs. a fully-connected layer.
- **Translation equivariance:** Shifting the input shifts the output identically. Pooling introduces translation *invariance* (coarser representation).
- **Locality:** Each output neuron depends only on a local receptive field of size $k \times k$.
- **Receptive field:** Stacking layers with kernels of size k and stride s yields a receptive field that grows combinatorially with depth.

Trade-offs:
- Parameters: $C_{\text{in}} \cdot C_{\text{out}} \cdot k_h \cdot k_w$
- FLOPs per position: $2 \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot k_h \cdot k_w$ (multiply-add).
- Inductive bias is strong but may be suboptimal for data without spatial locality (text is better handled by transformers, though 1D CNNs can be competitive).

12.1.2 1D Convolution

1D Convolution

For sequential data $X \in \mathbb{R}^{C_{\text{in}} \times L}$ with length L:

$$Y_{c_{\text{out}}, i} = \sum_{c_{\text{in}}=0}^{C_{\text{in}}-1} \sum_{u=0}^{k-1} X_{c_{\text{in}}, i \cdot s+u} \cdot W_{c_{\text{out}}, c_{\text{in}}, u}$$

1D CNNs slide a kernel along the temporal or sequential axis.

Applications

Text classification (character-level and word-level CNNs), time-series forecasting, audio analysis (raw waveform processing with large receptive-field 1D convolutions), and sequence-to-sequence preprocessing.

1D vs. RNN/Transformer

Pros: Parallelizable across the time dimension (unlike RNNs); efficient; smaller receptive field per layer but can be expanded via dilation (e.g., WaveNet).
Cons: Limited receptive field without depth or dilation; no inherent handling of variable-length sequences (requires padding or global pooling).

12.1.3 3D Convolution

3D Convolution

For volumetric data $X \in \mathbb{R}^{C_{\text{in}} \times D \times H \times W}$:

$$Y_{c_{\text{out}}, i, j, \ell} = \sum_{c_{\text{in}}=0}^{C_{\text{in}}-1} \sum_{u=0}^{k_d-1} \sum_{v=0}^{k_h-1} \sum_{w=0}^{k_w-1} X_{c_{\text{in}}, i s_d+u, j s_h+v, \ell s_w+w} \cdot W_{c_{\text{out}}, c_{\text{in}}, u, v, w}$$

Parameters: $C_{\text{in}} \cdot C_{\text{out}} \cdot k_d \cdot k_h \cdot k_w$.

Applications

Video action recognition (3D ResNet, I3D: inflated 2D filters to 3D), volumetric medical imaging (CT, MRI segmentation with 3D U-Net), and 4D weather/climate modeling.

Trade-offs

Pros: Native handling of 3D structure; captures spatio-temporal patterns jointly.
Cons: $O(k^3)$ parameter and compute scaling; memory-intensive; often replaced by (2+1)D factorized convolutions or 2D convolutions with temporal pooling or attention.

12.1.4 Transposed Convolution (Deconvolution)

Transposed Convolution

Transposed convolution (fractionally-strided convolution) upsamples a feature map by swapping the forward and backward passes of a standard convolution. Given input $X \in \mathbb{R}^{C_{\text{in}} \times H \times W}$:

$$Y_{c_{\text{out}},\, i,\, j} = \sum_{c_{\text{in}}=0}^{C_{\text{in}}-1} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} X_{c_{\text{in}},\, \lfloor (i+u)/s \rfloor,\, \lfloor (j+v)/s \rfloor} \cdot W_{c_{\text{out}},\, c_{\text{in}},\, u,\, v} \cdot \mathbf{1}\big((i+u) \bmod s = 0\big) \cdot \mathbf{1}\big((j+v) \bmod s = 0\big)$$

Equivalently, the input is "spread out" with zeros between entries (insert $s-1$ zeros), then a standard convolution is applied.

Output Size for Transposed Convolution

$$o = s \cdot (i-1) + k - 2p$$

for stride s, kernel size k, and padding p.

Applications

Decoder networks in autoencoders, U-Net upsampling path, GAN generators (DCGAN), semantic segmentation decoders, and learned upsampling in super-resolution.

Trade-offs

Pros: Learnable upsampling (vs. fixed interpolation); can produce higher-quality upsampled features when trained end-to-end.
Cons: Prone to "checkerboard artifacts" (overlapping kernel applications produce uneven coverage on the output grid); often replaced by resize + standard convolution (which avoids checkerboarding).

12.2 Depthwise Separable Convolution

Depthwise Separable Convolution

This factorizes a standard convolution into two steps:
1. Depthwise convolution: Each input channel is convolved with its own spatial kernel, with no cross-channel mixing:

$$\tilde{Y}_{c,\, i,\, j} = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} X_{c,\, i+u,\, j+v} \cdot W^{\text{depth}}_{c,\, u,\, v}$$

Output channels = input channels ($C_{\text{out}} = C_{\text{in}}$).

2. Pointwise (1×1) convolution: A 1×1 kernel mixes the depthwise outputs across channels:

$$Y_{c_{\text{out}},i,j} = \sum_{c_{\text{in}}=0}^{C_{\text{in}}-1} \tilde{Y}_{c_{\text{in}},i,j} \cdot W^{\text{point}}_{c_{\text{out}},c_{\text{in}}}$$

Parameter count: $k^2 \cdot C_{\text{in}} + C_{\text{in}} \cdot C_{\text{out}}$ vs. standard $k^2 \cdot C_{\text{in}} \cdot C_{\text{out}}$. For $k = 3$ and large $C_{\text{in}} \approx C_{\text{out}}$, this is approximately a $k^2 = 9\times$ reduction.

Applications

MobileNets (MobileNetV1–V3), Xception (extreme Inception), EfficientNet, and edge/mobile deployment of CNNs. The standard convolution block for resource-constrained inference.

Trade-offs

Pros:

- Massive parameter and FLOP reduction ($\sim$ 8–9$\times$).
- Retains most of the representational power of a full convolution; the pointwise 1×1 still allows cross-channel interaction.
- Empirically, the loss in accuracy is small relative to the compute savings.

Cons:

- Slightly lower accuracy ceiling than full convolutions on very large-scale training.
- Lower arithmetic intensity (fewer FLOPs per byte of memory access) can make it memory-bound on some hardware.
- The depthwise step has no channel mixing, so it cannot represent filters that require cross-channel interactions within the spatial kernel.

12.3 Dilated (Atrous) Convolution

Dilated / Atrous Convolution

A dilated convolution inserts "holes" (zeros) between kernel elements. With dilation rate r, the effective kernel spacing is increased:

$$Y_{c_{\text{out}},i,j} = \sum_{c_{\text{in}}} \sum_{u,v} X_{c_{\text{in}},\, i+r\cdot u,\, j+r\cdot v} \cdot W_{c_{\text{out}},c_{\text{in}},u,v}$$

The *effective* kernel size (the receptive field span) is:

$$k_{\text{eff}} = k + (k-1)(r-1)$$

Dilation exponentially expands the receptive field without increasing parameters or losing spatial resolution (no downsampling).

Receptive Field with Dilation

Consider stacking L layers with kernel sizes k_ℓ and dilation rates r_ℓ. The receptive field at layer L is:

$$\mathrm{RF}_L = \mathrm{RF}_{L-1} + (k_L - 1) \cdot r_L \cdot \prod_{\ell=1}^{L-1} s_\ell$$

The WaveNet architecture stacks layers with exponentially increasing dilation rates $r = 1, 2, 4, \ldots, 512$ to achieve an exponentially growing receptive field.

Applications

Semantic segmentation (DeepLab, dilated ResNet backbones with `output_stride=8`), audio generation (WaveNet), time-series forecasting, and any task where preserving spatial resolution is critical while needing a large field of view.

Trade-offs

Pros:

- Exponentially larger receptive field without pooling or increased parameters.
- Preserves spatial resolution; crucial for dense prediction tasks (segmentation, depth estimation).

Cons:

- **Gridding effect:** the systematic skipping of intermediate pixels means not all locations are used equally, creating a "grid" pattern in the receptive field. Mitigated by using dilation rates without a common factor (e.g., $r = 1, 2, 5$ instead of $1, 2, 4$).
- Combining large dilation with stride can cause some pixels to never be sampled (dead zones).
- Less effective for capturing local fine structure; often combined with standard convolutions.

12.4 Deformable Convolution

12.4.1 Deformable Convolution v1

Deformable Convolution v1 (Dai et al.

Standard convolution samples a fixed regular grid $\mathcal{R}$ (e.g., $\{(-1,-1), (-1,0), \ldots, (1,1)\}$ for a 3×3 kernel). Deformable convolution augments each sampling position with a learned offset Δp_n:

$$Y(p_0) = \sum_{p_n \in \mathcal{R}} W(p_n) \cdot X(p_0 + p_n + \Delta p_n)$$

The offsets $\Delta p_n \in \mathbb{R}^2$ are produced by a separate convolutional layer applied to the same input feature map. Since offsets are fractional, bilinear interpolation is used:

$$X(p) = \sum_q G(q, p) \cdot X(q), \qquad G(q, p) = \max(0, 1 - |q_x - p_x|) \cdot \max(0, 1 - |q_y - p_y|)$$

Applications

Object detection (models with non-rigid object deformations), semantic segmentation (adapting to object boundaries), and fine-grained recognition where geometric transformations are significant.

Trade-offs

Pros: Adaptive receptive field that deforms to object shape; models geometric transformations more naturally than augmentations; improves detection/segmentation accuracy over standard convolutions.
Cons: Additional parameters for offset prediction; training can be unstable (the offset network must learn meaningful deformations); bilinear interpolation adds compute overhead.

12.4.2 Deformable Convolution v2

Deformable Conv v2 (Zhu et al.

DCNv2 adds a learnable modulation scalar $\Delta m_n \in [0, 1]$ that weights the contribution of each sampling position:

$$Y(p_0) = \sum_{p_n \in \mathcal{R}} W(p_n) \cdot X(p_0 + p_n + \Delta p_n) \cdot \Delta m_n$$

The modulation allows the network to reduce the influence of irrelevant sampling locations (e.g., background regions within the receptive field). Both Δp_n and Δm_n are produced by a separate convolutional branch.

Applications

State-of-the-art object detection (used in many winning COCO detection entries), instance segmentation, and any dense prediction task where learned spatial attention complements deformable sampling.

Trade-offs

Pros: Modulation provides learned spatial attention on top of spatial deformation; strictly more expressive than DCNv1.
Cons: Even more parameters than DCNv1 (modulation branch); increased risk of overfitting on small datasets; implementation complexity is high.

12.5 Group Convolution

Group Convolution

Group convolution partitions the input and output channels into g groups and convolves each group independently (no cross-group connections):

$$Y_{c_{\text{out}}}^{(m)} = X^{(m)} * W_{c_{\text{out}}}^{(m)}, \qquad m = 1, \ldots, g$$

where each group has C_{in}/g input channels and C_{out}/g output channels. The total parameters:

$$\frac{k^2 \cdot C_{\text{in}} \cdot C_{\text{out}}}{g}$$

Setting $g = C_{\text{in}}$ recovers depthwise convolution.

Applications

AlexNet (group conv split across two GPUs), ResNeXt ("cardinality" as a new dimension beyond depth and width), MobileNet, and ShuffleNet (combined with channel shuffle).

Trade-offs

Pros: $g\times$ fewer parameters and FLOPs; promotes learning of diverse filter patterns (each group specializes); enables model parallelism across devices.
Cons: No information flow between groups within a single convolution layer (requires 1×1 conv or channel shuffle to mix); the group count g is an additional hyperparameter; accuracy can degrade if g is too large relative to the channel count.

12.6 ShuffleNet — Channel Shuffle

Channel Shuffle (Zhang et al.

Group convolution prevents cross-group information flow. ShuffleNet restores this by rearranging the output channels after each group convolution. For g groups with output $Y \in \mathbb{R}^{C\times H\times W}$:

1. Reshape: $(g, C/g, H, W)$.
2. Transpose (swap group and channel dims): $(C/g, g, H, W)$.
3. Flatten back to (C, H, W).

The core ShuffleNet block is: 1×1 group conv $\to$ channel shuffle $\to 3 \times 3$ depthwise conv $\to 1 \times 1$ group conv, with a residual connection.

Applications

Mobile and embedded vision (ShuffleNetV1/V2), efficient image classification on edge devices, and lightweight feature extraction for detection/segmentation backbones.

Trade-offs

Pros: Enables cross-group information flow while retaining the efficiency of group convolution; ShuffleNetV2 additionally optimizes for memory access cost (MAC) rather than FLOPs alone.
Cons: The channel shuffle operation adds memory-access overhead; marginally more complex to implement than plain group conv; diminishing returns when FLOPs are already very low.

12.7 Ghost Module (GhostNet)

Ghost Module (Han et al.

A standard convolution generates many redundant "ghost" feature maps that are similar to each other. GhostNet produces these cheaply:

$$Y' = X * F_{1\times 1} \quad \text{(intrinsic features, } \ll C_{\text{out}} \text{ channels)}$$
$$Y_{ij} = \Phi_{ij}(Y'_i), \quad \forall\, i = 1, \ldots, m,\ j = 1, \ldots, s \quad \text{(ghost features)}$$

$$Y = \text{Concat}([Y', Y_{11}, Y_{12}, \ldots, Y_{ms}])$$

where Φ_{ij} is a cheap linear operation (e.g., a small depthwise convolution) applied to the i-th intrinsic feature map. Speedup: $\approx s\times$ reduction in FLOPs vs. standard conv.

Applications

GhostNet for mobile image classification; the ghost module can be used as a drop-in replacement for any standard convolution block in resource-constrained settings.

Trade-offs

Pros: Significant FLOP reduction with negligible accuracy drop; the ghost module leverages feature-map redundancy explicitly; compatible with other efficient architectures.
Cons: The choice of cheap operations Φ_{ij} and the expansion ratio s are design-time decisions; the additional concatenation increases memory at the layer boundary; less benefit when feature redundancy is already addressed by other methods.

12.8 Conditionally Parameterized Convolution (CondConv)

CondConv (Yang et al.

Instead of a single static kernel, CondConv maintains a bank of K expert kernels $\{W_1, \ldots, W_K\}$ and dynamically combines them based on the input:

$$W = \sum_{k=1}^{K} \alpha_k(x) \cdot W_k$$

where the combination weights $\alpha(x) \in \mathbb{R}^K$ are computed by a routing function:

$$\alpha(x) = \text{sigmoid}\big(\text{FC}(\text{GAP}(x))\big)$$

The generated kernel W is then applied as a standard convolution. Each input sample uses a slightly different kernel, tailored to its content.

Applications

Image classification and detection where increasing model capacity without proportionally increasing FLOPs is desired; the expert kernels can specialize to different visual concepts (textures, shapes, scales).

Trade-offs

Pros: Increases model capacity with minimal FLOP increase (only the routing function and the weighted sum); the dynamic kernel adapts to the input content.
Cons: The kernel bank requires $K\times$ more parameters (storage, not compute); the routing function is a bottleneck if poorly designed; marginal gains on tasks with limited intra-class variation.

12.9 Involution

Involution (Li et al.

Involution inverts the design principles of standard convolution: kernels are *channel-agnostic* (shared across channels) but *spatial-specific* (different at each spatial position). For each pixel (i, j), a kernel $\mathcal{H}_{i,j}$ is generated from the input feature vector:

$$\mathcal{H}_{i,j} = \phi(X_{i,j}) = W_1\,\sigma(W_0\,X_{i,j})$$

where $W_0 \in \mathbb{R}^{\frac{C}{r}\times C}$ and $W_1 \in \mathbb{R}^{(K\times K\times G)\times\frac{C}{r}}$ are learnable matrices (r is a reduction ratio, G is the number of groups). Then:

$$Y_{i,j,k} = \sum_{(u,v)\in\Delta_K} \mathcal{H}_{i,j,u,v,\lfloor kG/C\rfloor}\; X_{i+u,\,j+v,\,k}$$

Applications

Dense prediction tasks (segmentation, detection) where spatial-specific features are more important than channel-specific features; can be used as an alternative or complement to self-attention and convolution.

Trade-offs

Pros: Spatial-specific kernels capture long-range spatial interactions better than convolution; fewer channels per kernel;
Cons: Generating per-pixel kernels is computationally expensive; the kernel size $K \times K$ must be kept moderate; less mature than standard convolution or attention.

12.10 Graph Convolution

12.10.1 GCN — Graph Convolutional Network

GCN (Kipf & Welling

For a graph with adjacency matrix $A \in \mathbb{R}^{N\times N}$ and node features $H^{(0)} \in \mathbb{R}^{N\times d}$, the GCN layer performs localized spectral filtering via a first-order approximation of Chebyshev polynomials:

$$H^{(l+1)} = \sigma\Big(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\Big)$$

where $\tilde{A} = A + I_N$ (add self-loops) and $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$ (degree matrix). The renormalized adjacency $\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$ normalizes the aggregation to avoid exploding activations.

Message-Passing Interpretation

Each GCN layer can be decomposed into three steps:

1. **Message computation:** $\mathbf{m}_j = W^{(l)}\mathbf{h}_j^{(l)}$.
2. **Aggregation:** $\tilde{\mathbf{h}}_i^{(l+1)} = \sum_{j \in \mathcal{N}(i)\cup\{i\}} \frac{1}{\sqrt{\tilde{d}_i \tilde{d}_j}} \mathbf{m}_j$.
3. **Update:** $\mathbf{h}_i^{(l+1)} = \sigma(\tilde{\mathbf{h}}_i^{(l+1)})$.

Applications

Node classification (Cora, CiteSeer), link prediction, graph classification (molecular property prediction, social networks), and recommendation systems (collaborative filtering on user-item graphs).

Trade-offs

Pros: Simple, efficient (sparse matrix multiplication); strong empirical performance on transductive node classification.
Cons: Transductive only (cannot generalize to unseen nodes without retraining); over-smoothing (deep GCNs collapse node features); limited expressiveness (no edge features in the basic formulation); first-order approximation limits spectral resolution.

12.10.2 ChebNet, SGC, and GIN

ChebNet (Defferrard et al.

ChebNet uses a K-th order Chebyshev polynomial expansion to approximate the spectral graph filter:

$$H^{(l+1)} = \sigma\left(\sum_{k=0}^{K-1} T_k(\tilde{L}) H^{(l)} W_k^{(l)}\right)$$

where $\tilde{L} = \frac{2}{\lambda_{\max}} L - I_N$ is the scaled Laplacian and T_k are Chebyshev polynomials defined recursively: $T_0(x) = 1$, $T_1(x) = x$, $T_k(x) = 2x\,T_{k-1}(x) - T_{k-2}(x)$. GCN is the special case $K = 1$ with additional approximations.

SGC — Simple Graph Convolution (Wu et al.

SGC removes all intermediate nonlinearities, collapsing the GCN into a single linear model applied to pre-processed features:

$$H^{(K)} = \text{softmax}\Big((\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2})^K X\, W\Big)$$

The K-hop normalized adjacency powers are precomputed once, and only W is learned. Despite its simplicity, SGC matches GCN on many benchmarks.

GIN — Graph Isomorphism Network (Xu et al.

GIN is designed to be as expressive as the Weisfeiler-Lehman (WL) graph isomorphism test:

$$\mathbf{h}_v^{(k)} = \text{MLP}^{(k)}\left((1+\epsilon^{(k)})\,\mathbf{h}_v^{(k-1)} + \sum_{u\in\mathcal{N}(v)} \mathbf{h}_u^{(k-1)}\right)$$

where $\epsilon^{(k)}$ is a learnable or fixed scalar. The "readout" (global pooling) concatenates features from all layers before the final classifier to capture substructures at multiple scales.

Applications

GIN is the standard baseline for graph classification benchmarks (TUDataset, OGB); ChebNet for tasks requiring multi-hop spectral filters; SGC when computational simplicity is paramount.

12.11 Spherical / 3D / Point-Cloud Convolutions

12.11.1 Point-Cloud Convolutions

EdgeConv — DGCNN (Wang et al.

For each point $\mathbf{x}_i$ in a point cloud, EdgeConv constructs a k-NN graph and computes edge features:

$$\mathbf{e}_{ij} = h_\Theta(\mathbf{x}_i,\ \mathbf{x}_j - \mathbf{x}_i)$$

where h_Θ is an MLP with shared weights across all edges. The node update aggregates edge features:

$$\mathbf{x}_i' = \square_{j\in\mathcal{N}(i)}\mathbf{e}_{ij}$$

where $\square$ is a symmetric aggregation function (max, sum, or mean). EdgeConv dynamically recomputes the graph in feature space at each layer, enabling non-local interactions.

PointConv (Wu et al.

PointConv: Approximates the continuous convolution integral $\int f(\mathbf{y})\,W(\mathbf{x}-\mathbf{y})\,d\mathbf{y}$ as a Monte Carlo estimate over the point set using a density-corrected weight function learned by an MLP.
KPConv: Defines a set of K learnable kernel points in 3D space with a radius influence function. The convolution at each point is the sum of kernel-point responses weighted by a correlation function that measures the proximity of neighboring points to each kernel point.

12.11.2 Voxel-Based 3D Convolution

Sparse 3D Convolution

For point-cloud detection, points are voxelized into a regular 3D grid (most cells are empty). Sparse convolution applies standard 3D convolution only at non-empty voxel locations and skips computation at empty cells:

$$Y_{c_{\text{out}},i,j,k} = \sum_{c_{\text{in}}} \sum_{u,v,w} X_{c_{\text{in}},i+u,j+v,k+w} \cdot W_{c_{\text{out}},c_{\text{in}},u,v,w} \quad \text{if } X \text{ is non-empty at the output location}$$

The sparse convolution engine maintains a hash table mapping spatial coordinates to active sites, dramatically reducing memory and compute.

Applications

Autonomous driving (VoxelNet, PointPillars, SECOND for LiDAR point clouds), 3D object detection, indoor scene understanding, and 3D medical image segmentation.

12.12 FFT and Winograd Convolution

Convolution via FFT — Fast Fourier Transform

The convolution theorem states that convolution in the spatial domain is element-wise multiplication in the frequency domain:

$$X * W = \mathcal{F}^{-1}\big(\mathcal{F}(X) \odot \mathcal{F}(W)\big)$$

For large kernels ($k \gtrsim 5$), the $O(n \log n)$ FFT approach can be faster than the $O(nk^2)$ direct method. The kernel W must be zero-padded to the size of X before the FFT.

Winograd Minimal Filtering Algorithm (Lavin & Gray

Winograd convolution transforms the input and kernel into a space where the convolution reduces to element-wise multiplication, then transforms back. For a 3×3 kernel with stride 1 and an output tile of 2×2 ($\mathrm{F}(2 \times 2,\, 3 \times 3)$), the theoretical speedup is $2.25\times$ (9 multiplies instead of 36):

$$Y = A^T \big[(G\, W\, G^T) \odot (B^T\, X\, B)\big] A$$

where A, B, G are constant transformation matrices derived from the Chinese remainder theorem for polynomials.

Applications

Inference acceleration on GPUs (cuDNN uses Winograd for small kernels), especially for 3×3 stride-1 convolutions. FFT-based conv for large-kernel signal processing and some scientific computing workloads.

Trade-offs

Pros: Significant speedup for certain kernel/tile size combinations; reduced arithmetic complexity.
Cons: Numerical precision issues for large transforms; the transformation overhead means it only pays off for specific sizes; not suitable for small batch sizes or depthwise convolutions; the transform matrices must be precomputed.

12.13 Group Equivariant Convolution

G-Equivariant Convolution (Cohen & Welling

A group-equivariant convolutional layer is designed such that transforming the input by any element g of a symmetry group G (rotations, reflections) produces a correspondingly transformed output:

$$[\pi'_g \circ \Phi](f) = [\Phi \circ \pi_g](f), \qquad \forall g \in G$$

where Φ is the layer, π_g is the group action on the input, and π'_g is the group action on the output.
For the discrete rotation group C_4 $(0°, 90°, 180°, 270°)$, the kernel is constrained: applying any rotation to the kernel produces another valid kernel within the same filter bank. The group convolution over G is defined as:

$$[f *_G \psi](g) = \sum_{h \in G} f(h)\, \psi(g^{-1}h)$$

Applications

Medical image analysis (where orientation is arbitrary), dealing with arbitrary rotations in satellite imagery/remote sensing, crystal structure prediction, and any domain where the data has known symmetries that should be exploited rather than learned from data augmentation.

Trade-offs

Pros: Guaranteed equivariance to group transformations (not just approximate, as with data augmentation); improved sample efficiency since symmetries are built into the architecture; weight sharing across group elements reduces parameters.
Cons: Significantly more complex implementation; restricted to discretized symmetry groups (continuous groups require sampling or steerable filters); the group dimension increases data dimensionality (output channels $\times$ group size); not beneficial when the data does not exhibit the assumed symmetries.

Table 12.1: Comparison of convolution types.

Type	Key Idea	Params	Adaptive?	Invaria
Standard 2D	Full spatial-channel cross-correlation	$k^2C_iC_o$	No	Transl. e
1D Conv	Kernel slides along 1 axis	kC_iC_o	No	Transl. e
3D Conv	Kernel slides in 3D volume	$k^3C_iC_o$	No	Transl. e
Transposed	Learnable upsampling via backward pass	$k^2C_iC_o$	No	Transl. e
Depthw. Sep.	Depthwise + pointwise factorization	$k^2C_i + C_iC_o$	No	Transl. e
Dilated	Expands receptive field via holes	$k^2C_iC_o$	No	Transl. e
Deformable	Learned offset for each kernel point	$k^2C_iC_o$+ offset params	Yes	Adapt
Group Conv	Split channels into groups	$(k^2C_iC_o)/g$	No	Transl. e
ShuffleNet	Group conv + channel permutation	$(k^2C_iC_o)/g$	No	Transl. e
Ghost	Cheap operations generate extra maps	Reduced	No	Transl. e
CondConv	Input-dependent weighted expert combo	$Kk^2C_iC_o$	Yes	Transl. e
Involution	Spatial-specific, channel-agnostic	Generated per-pixel	Yes	Spatial-sp
GCN	Graph neighbor averaging	C_iC_o	No	Perm. eq
GAT	Attention over graph neighbors	C_iC_o+ attn params	Yes	Perm. eq
EdgeConv	MLP over point-pair features	C_iC_o	Yes	Perm. eq
G-Equivariant	Constrained group-symmetric kernels	$k^2C_iC_o/\|G\|$	No	Group eq

12.14 Comprehensive Convolution Comparison

Choosing a Convolution Operation

- **Standard image processing:** 2D convolution with 3×3 kernels, batch normalization, and ReLU activation.
- **Mobile / edge deployment:** Depthwise separable (MobileNet) or Ghost module, possibly with channel shuffle.
- **Semantic segmentation:** Dilated convolutions to maintain resolution with large receptive field (ASPP), or deformable convolutions for precise object boundaries.
- **Object detection:** Deformable Conv v2 for improved detection of non-rigid objects; standard 2D conv for the backbone.
- **Video understanding:** (2+1)D factorized conv (spatial 2D + temporal 1D) is often more efficient than full 3D conv.
- **Graph-structured data:** GCN for basic tasks; GAT or GIN for more expressive graph learning; DiffPool/SAGPool for graph classification.
- **Point clouds:** EdgeConv (DGCNN) or KPConv for direct point-cloud processing; Sparse 3D Conv for voxelized LiDAR.
- **Symmetry-rich data:** Group-equivariant convolutions when the data exhibits known symmetries (rotation, reflection).
- **Large kernels / signal processing:** FFT-based or Winograd convolution for acceleration.
- **Super-resolution / upsampling:** Transposed convolution, or resize + standard conv to avoid checkerboard artifacts.

Chapter 13

Kernel Functions

A kernel function $K(x, y)$ computes a similarity measure between two data points by implicitly mapping them into a (possibly infinite-dimensional) feature space $\mathcal{H}$ via a feature map $\phi(\cdot)$, and then taking the inner product in that space. The **kernel trick** avoids explicitly computing $\phi(x)$, enabling efficient non-linear learning.

Kernel (Mercer Kernel): A function $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is a valid kernel if there exists a Hilbert space $\mathcal{H}$ and a map $\phi : \mathcal{X} \to \mathcal{H}$ such that

$$K(x, y) = \langle \phi(x), \phi(y) \rangle_{\mathcal{H}} \qquad \forall x, y \in \mathcal{X}$$

Equivalently, by **Mercer's Theorem**, K is a valid kernel iff for any finite set $\{x_1, \ldots, x_n\}$ the Gram matrix $\mathbf{K}_{ij} = K(x_i, x_j)$ is positive semi-definite.

13.1 Introduction to Kernels

Kernels lie at the heart of many machine learning algorithms. By replacing dot products with kernel evaluations, linear methods (SVMs, PCA, ridge regression) become non-linear while retaining convexity guarantees. The key insight is that many algorithms only access data through inner products; substituting $K(x_i, x_j)$ for $\langle x_i, x_j \rangle$ yields a non-linear version without altering the algorithm's structure.

Core applications: Support Vector Machines (SVM), Kernel Ridge Regression, Kernel PCA, Gaussian Processes, Kernelized Perceptron, Kernel Density Estimation, independence tests (HSIC), two-sample tests (MMD), and kernel-based clustering (kernel K-means, spectral clustering).

Kernel design principles:

1. **Closure properties:** If K_1, K_2 are valid kernels, then $K_1 + K_2$, cK_1 $(c > 0)$, $K_1 \cdot K_2$, and $f(x)K_1(x, y)f(y)$ are also valid.
2. **Kernel on structured data:** Define K directly on strings, graphs, trees, or sets without explicit vector representation.
3. **Representer Theorem:** The minimizer of a regularized empirical risk in an RKHS $\mathcal{H}$ takes the form $f^*(\cdot) = \sum_{i=1}^{n} \alpha_i K(\cdot, x_i)$.

13.2 SVM Kernels

Support Vector Machines rely on kernels to find maximum-margin separating hyperplanes in feature space. The choice of kernel determines the family of decision boundaries the model can express.

13.2.1 Linear Kernel

$$K_{\text{linear}}(x, y) = x^\top y + c$$

where c is an optional constant (bias in feature space). With $c = 0$ this is simply the standard dot product; with $c \neq 0$ it corresponds to augmenting data with a constant feature.

Usage: Linear SVMs for text classification (high-dimensional sparse data already linearly separable), baseline model before trying non-linear kernels, interpretable feature importance via weight vector.

Pros: Extremely fast training (linear in n), interpretable coefficients, few hyperparameters, scales to millions of samples/features. Numerically stable and well-conditioned. **Cons:** Cannot model non-linear decision boundaries. Inadequate for image, speech, or other domains where features interact non-linearly.

13.2.2 Polynomial Kernel

$$K_{\text{poly}}(x, y) = (\gamma\, x^\top y + c)^d$$

Parameters: degree $d \in \mathbb{N}$, scale $\gamma > 0$, constant $c \geq 0$. The feature map spans all monomials of degree up to d in $\binom{\dim(\mathcal{X})+d}{d}$ dimensions.

Usage: Non-linear SVMs when polynomial interactions among features are expected. Natural language processing with n-gram features, image classification with moderate non-linearity, tasks where degree of interaction is known a priori.

Pros: Explicit control over non-linearity via degree d, interpretable feature interactions (monomials), extends linear kernel ($d = 1$). Well-studied and available in all SVM libraries. **Cons:** For $d > 3$, numerical instability and severe overfitting; gradients explode with high-degree terms. Kernel evaluations become expensive. Poor extrapolation beyond training data range.

13.2.3 RBF / Gaussian Kernel

$$K_{\text{RBF}}(x, y) = \exp\bigl(-\gamma \|x - y\|^2\bigr) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right)$$

where $\gamma = 1/(2\sigma^2)$. The RBF kernel maps to an **infinite-dimensional** feature space (Taylor expansion of the exponential contains all polynomial terms). σ controls the length-scale of local influence.

Usage: Default SVM kernel for non-linear problems. Universal choice when no prior knowledge about data geometry exists. Also the foundational stationary kernel for Gaus-

sian Process regression, kernel ridge regression, and spectral clustering via Gaussian affinity matrices.

Pros: Universal approximator (dense in $C(\mathcal{X})$), handles arbitrary non-linear boundaries, local behavior avoids global influence of outliers. Only one hyperparameter (γ). Empirically the most robust and versatile kernel across domains. Smooth (infinitely differentiable). **Cons:** γ is critical—too small (large σ): underfitting, behaves like linear kernel; too large (small σ): overfitting, each point becomes its own support vector. Poor extrapolation (kernel values decay exponentially with distance). Kernel matrix can become ill-conditioned. $O(n^2)$ storage.

13.2.4 Laplacian Kernel

$$K_{\text{Laplacian}}(x, y) = \exp\bigl(-\gamma\|x - y\|_1\bigr)$$

The L_1 analogue of the RBF kernel. Less smooth (only continuous, not differentiable at $x = y$), based on the Laplace distribution rather than Gaussian.

Usage: When the L_1 metric is more appropriate than L_2 (e.g., sparse data, robust to outliers), or when a less smooth kernel is desired (GP with Ornstein-Uhlenbeck prior).

Pros: More robust to outliers in individual coordinates (linear penalty vs. quadratic). The induced RKHS contains less smooth functions, potentially better for non-smooth target functions. **Cons:** Not differentiable at $x = y$, which can cause issues in gradient-based kernel learning. Less widely supported in libraries.

13.2.5 Sigmoid Kernel

$$K_{\text{sigmoid}}(x, y) = \tanh(\gamma\, x^\top y + c)$$

Despite its historical use in SVMs (mimicking two-layer neural networks), this kernel is **NOT** positive semi-definite for all parameter choices, violating Mercer's condition. Only for certain γ, c values (and orthogonal data) does it yield a valid kernel.

The NN analogy: A two-layer neural network with sigmoid activation $f(x) = \sum_i v_i\, \sigma(w_i^\top x + b_i)$ is related to the sigmoid kernel SVM, but the kernel's conditional PSD property means SVM training may not converge to a global optimum.

13.2.6 Chi-Squared Kernel

$$K_{\chi^2}(x, y) = \exp\left(-\gamma \sum_{i=1}^{d} \frac{(x_i - y_i)^2}{x_i + y_i + \varepsilon}\right)$$

Adding a small $\varepsilon > 0$ prevents division by zero. Widely used with histogram features (e.g., bag-of-visual-words) because the χ^2 distance is a natural metric for comparing probability mass functions.

Usage: Computer vision with histogram-based image representations (SIFT, HOG), texture classification, and any domain where features are non-negative histograms.

13.2.7 Histogram Intersection Kernel

$$K_{\text{HI}}(x, y) = \sum_{i=1}^{d} \min(x_i, y_i)$$

For normalized histograms ($\sum x_i = \sum y_i = 1$), this equals $1 - \frac{1}{2}\|x - y\|_1$. It measures the overlap area between two histograms and is provably a Mercer kernel (Odone et al., 2005).

Usage: Image retrieval, color histogram matching, object recognition with normalized feature vectors. Popular in early computer vision literature.

Pros: Positive definite, fast to compute (only comparisons and additions), intuitively interpretable as histogram overlap. Robust to small perturbations in bin counts. **Cons:** Only meaningful for non-negative features. Does not capture correlations between different histogram bins.

13.3 Gaussian Process Covariance Functions

In Gaussian Process regression, the kernel (covariance function) encodes prior assumptions about the function's smoothness, periodicity, trend, and noise characteristics. A GP is fully specified by its mean function $m(x)$ and covariance function $k(x, x')$:

$$f(x) \sim \mathcal{GP}\big(m(x), k(x, x')\big)$$

13.3.1 Squared Exponential (RBF)

$$k_{\text{SE}}(x, x') = \sigma_f^2 \exp\left(-\frac{\|x - x'\|^2}{2\ell^2}\right)$$

Parameters: output variance σ_f^2 (vertical scale), lengthscale ℓ (horizontal scale—how far one must move in input space for the function value to change significantly). This kernel is **infinitely differentiable**, yielding very smooth sample functions.

Pros: Universally used stationary kernel, only two interpretable hyperparameters, smooth samples, well-conditioned kernel matrix for moderate lengthscales. **Cons:** Excessive smoothness can be unrealistic for physical processes. Rapid decay of covariance with distance can cause numerical issues for small ℓ. Not suitable for functions with sharp transitions.

13.3.2 Matérn Kernel

$$k_\nu(x, x') = \sigma_f^2 \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu}\, r}{\ell} \right)^\nu K_\nu \left(\frac{\sqrt{2\nu}\, r}{\ell} \right)$$

where $r = \|x - x'\|$, $\nu > 0$ is the smoothness parameter, K_ν is the modified Bessel function of the second kind, and $\Gamma(\cdot)$ is the gamma function. Sample functions are $\lfloor \nu - 1 \rfloor$ times differentiable.

Special cases of the Matérn family:

- $\nu = 1/2$ (Exponential):
$$k_{1/2}(x, x') = \sigma_f^2 \exp\left(-\frac{r}{\ell}\right)$$
Corresponds to the Ornstein-Uhlenbeck process. Very rough (continuous but not differentiable). Equivalent to the Laplacian kernel.

- $\nu = 3/2$ (Once differentiable):
$$k_{3/2}(x, x') = \sigma_f^2 \left(1 + \frac{\sqrt{3}\, r}{\ell}\right) \exp\left(-\frac{\sqrt{3}\, r}{\ell}\right)$$

- $\nu = 5/2$ (Twice differentiable):
$$k_{5/2}(x, x') = \sigma_f^2 \left(1 + \frac{\sqrt{5}\, r}{\ell} + \frac{5r^2}{3\ell^2}\right) \exp\left(-\frac{\sqrt{5}\, r}{\ell}\right)$$

- $\nu \to \infty$: approaches the Squared Exponential (RBF) kernel.

Usage: Default GP kernel in many packages (GPy, GPflow, GPyTorch). $\nu = 5/2$ is the standard choice, balancing smoothness with realism for modeling physical processes. The half-integer ν values have closed-form expressions without Bessel functions.

Pros: Explicit control over smoothness via ν. Better calibrated uncertainty than RBF for many real-world problems. Half-integer cases are computationally cheap. **Cons:** Extra hyperparameter ν (though usually fixed). The general form requires Bessel function evaluation, which is slower than exponential-only kernels.

13.3.3 Rational Quadratic (RQ)

$$k_{\text{RQ}}(x, x') = \sigma_f^2 \left(1 + \frac{r^2}{2\alpha\ell^2}\right)^{-\alpha}$$

where $\alpha > 0$ is the scale mixture parameter. The RQ kernel is an **infinite mixture of RBF kernels** with lengthscales drawn from a Gamma distribution $\ell^{-2} \sim \text{Gamma}(\alpha, \alpha/\ell^2)$.

Scale mixture interpretation:

$$k_{\text{RQ}}(x, x') = \int_0^\infty k_{\text{RBF}(\tau)}(x, x')\, p(\tau \mid \alpha, \ell)\, d\tau$$

where $\tau = 1/\ell^2$. As $\alpha \to \infty$, the RQ kernel approaches the RBF kernel with lengthscale ℓ.

Usage: Modeling functions that vary across multiple lengthscales simultaneously. When the data exhibits both short-range and long-range correlations, the RQ kernel captures this without manual lengthscale selection.

13.3.4 Periodic Kernel

$$k_{\text{Per}}(x, x') = \sigma_f^2 \exp\left(-\frac{2\sin^2\left(\pi\|x - x'\|/p\right)}{\ell^2}\right)$$

where p is the period. This kernel maps the input onto a circle via $u(x) = [\sin(2\pi x/p), \cos(2\pi x/p)]$ and applies the RBF kernel on the circle.

Usage: Modeling seasonal/cyclic patterns (temperature, sales, energy demand), quasi-periodic time series, audio signals with pitch, or any domain with repeating structure.

13.3.5 Linear / Dot Product Kernel

$$k_{\text{Lin}}(x, x') = \sigma_f^2\, x^\top x' = \sigma_f^2 \sum_{i=1}^{d} x_i x_i'$$

This is the covariance function corresponding to Bayesian linear regression with a Gaussian prior $\mathbf{w} \sim \mathcal{N}(0, \sigma_f^2 \mathbf{I})$ on weights.

13.3.6 White Noise Kernel

$$k_{\text{WN}}(x, x') = \sigma_n^2\, \delta(x, x')$$

where δ is the Kronecker delta (1 if $x = x'$, else 0). This models i.i.d. observation noise and is typically **added** to any of the above kernels for likelihood computation:

$$k_{\text{total}}(x, x') = k_{\text{signal}}(x, x') + \sigma_n^2\, \delta(x, x')$$

13.3.7 Neural Network Kernel (ArcSin)

$$k_{\text{NN}}(x, x') = \sigma_f^2\, \frac{2}{\pi} \arcsin\left(\frac{2\tilde{x}^\top \Sigma\, \tilde{x}'}{\sqrt{(1 + 2\tilde{x}^\top \Sigma\, \tilde{x})(1 + 2\tilde{x}'^\top \Sigma\, \tilde{x}')}}\right)$$

where $\tilde{x} = [1, x^\top]^\top$ (augmented with bias) and Σ is a weight prior covariance. This kernel corresponds to an **infinite-width** single-hidden-layer neural network with erf or sign activation functions (Neal, 1996; Williams, 1997).

Connection to Deep Learning: The arcsin kernel is the GP limit of a Bayesian neural network as the hidden layer width $\to \infty$. It demonstrates that even infinite networks have limited expressiveness (no hierarchical feature learning) unless depth is also infinite.

13.3.8 Spectral Mixture Kernel

$$k_{\text{SM}}(x, x') = \sum_{q=1}^{Q} w_q \prod_{p=1}^{P} \exp\left(-2\pi^2 (x_p - x'_p)^2 v_q^{(p)}\right) \cos\left(2\pi (x_p - x'_p) \mu_q^{(p)}\right)$$

where each spectral component q has weight w_q, mean μ_q, and variance v_q. The spectral density is a mixture of Q Gaussians in the frequency domain (Wilson & Adams, 2013).

Usage: Very flexible multi-scale kernel that can discover patterns automatically. Used for time series with complex spectral structure, extrapolation tasks, and as a building block in deep kernel learning.

Pros: Highly expressive; can approximate any stationary kernel. Discovers periodicities, trends, and multiple lengthscales automatically from data. Interpretable through the learned spectral density. **Cons:** $O(QP)$ parameters to learn (though typically small Q). Non-convex optimization with many local minima. Overkill for simple problems.

13.4 Neural Tangent Kernel (NTK)

13.4.1 Definition

Neural Tangent Kernel: For a neural network $f(x; \mathbf{w})$ with parameters $\mathbf{w}$, the NTK is defined as

$$\Theta(x, x') = \mathbb{E}_{\mathbf{w} \sim \mathcal{N}(0,1)} \left[\left\langle \frac{\partial f(x; \mathbf{w})}{\partial \mathbf{w}}, \frac{\partial f(x'; \mathbf{w})}{\partial \mathbf{w}} \right\rangle \right]$$

In the infinite-width limit, the NTK becomes **deterministic** (doesn't depend on the random initial weights) and stays **constant during training** when using gradient descent with infinitesimal learning rate.

13.4.2 Properties

- **Infinite-width equivalence:** Training an infinitely wide NN with MSE loss and gradient flow is exactly equivalent to kernel ridge regression with the NTK (Jacot et al., 2018).
- **Closed form for ReLU:** For a ReLU network, the NTK has a closed-form expression in terms of the arccosine kernel.

- **Spectral bias:** The NTK's eigenvalue spectrum decays rapidly. NNs learn target functions aligned with top NTK eigenfunctions first (low-frequency patterns before high-frequency details).
- **Lazy training:** When the NTK stays nearly constant throughout training, the network behaves like a linear model in the tangent space—this is the "lazy regime."
- **Finite-width deviations:** At finite width, the NTK evolves during training ("feature learning"), which is responsible for the superior performance of finite networks over their linearized counterparts.

13.4.3 NTK Parameterization (μP)

Maximal Update Parameterization (μP): Under standard parameterization, increasing network width collapses the kernel to a trivial limit. μP (Yang & Hu, 2021) rescales initial weights and learning rates per-layer to ensure **maximal feature learning** at all widths:

- Hidden weights: init $\sim \mathcal{N}(0, 1/\text{fan_in})$, LR $\propto 1/\text{width}$
- Output weights: init $\sim \mathcal{N}(0, 1/\text{width}^2)$, LR $\propto 1/\text{width}$

This enables hyperparameter transfer from small to large models.

13.5 Sparse GP Approximations

Exact GP inference costs $O(n^3)$ due to the Cholesky decomposition of the $n \times n$ kernel matrix. Sparse approximations reduce this to $O(nm^2)$ by introducing $m \ll n$ **inducing points**.

13.5.1 Inducing Points Methods

Inducing Variables: Select m inducing inputs $\mathbf{Z} = \{z_1, \ldots, z_m\}$ with associated inducing variables $\mathbf{u} = f(\mathbf{Z})$. The GP prior over training and inducing points is:

$$\begin{bmatrix} \mathbf{f} \\ \mathbf{u} \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} \mathbf{K}_{nn} & \mathbf{K}_{nm} \\ \mathbf{K}_{nm}^\top & \mathbf{K}_{mm} \end{bmatrix}\right)$$

Approximate posterior $q(\mathbf{f}, \mathbf{u}) = p(\mathbf{f} \mid \mathbf{u})\, q(\mathbf{u})$ where $q(\mathbf{u}) = \mathcal{N}(\mathbf{m}, \mathbf{S})$.

Major sparse GP approximations:

- **SoR / DTC** (Subset of Regressors, Deterministic Training Conditional): $\mathbf{K}_{nn} \approx \mathbf{K}_{nm}\mathbf{K}_{mm}^{-1}\mathbf{K}_{nm}^\top$. Simple but underestimates variance; not a proper GP model.
- **FITC** (Fully Independent Training Conditional): Adds diagonal correction $\text{diag}(\mathbf{K}_{nn} - \mathbf{K}_{nm}\mathbf{K}_{mm}^{-1}\mathbf{K}_{nm}^\top)$. Fully independent conditional but heteroscedastic approximation.
- **VFE / SVGP** (Variational Free Energy): Variational lower bound with $\text{KL}(q(\mathbf{u}) \| p(\mathbf{u} \mid \mathbf{y}))$. Proper GP model, treats inducing points as variational parameters. Preferred modern approach (Titsias, 2009; Hensman et al., 2013).

13.5.2 Random Fourier Features (Rahimi & Recht, 2007)

Bochner's Theorem: A continuous shift-invariant kernel $k(x, y) = k(x - y)$ is positive definite **iff** $k(\delta)$ is the Fourier transform of a finite non-negative measure $p(\omega)$:

$$k(x - y) = \int_{\mathbb{R}^d} e^{i\omega^\top (x-y)}\, p(\omega)\, d\omega$$

For real-valued kernels, this simplifies to:

$$k(x - y) = \int_{\mathbb{R}^d} \cos\big(\omega^\top (x - y)\big)\, p(\omega)\, d\omega$$

Random Fourier Feature map: Sample D frequencies $\{\omega_d\}_{d=1}^D \sim p(\omega)$, D phases $\{b_d\}_{d=1}^D \sim \mathcal{U}(0, 2\pi)$. Then:

$$\phi_{\text{RFF}}(x) = \sqrt{\frac{2}{D}} \left[\cos(\omega_1^\top x + b_1),\ \ldots,\ \cos(\omega_D^\top x + b_D)\right]^\top$$

The kernel is approximated as an inner product:

$$K(x, y) \approx \phi_{\text{RFF}}(x)^\top \phi_{\text{RFF}}(y)$$

For the RBF kernel, $p(\omega) = \mathcal{N}(0, 2\gamma \mathbf{I})$.

Pros: Reduces kernel methods to linear models in D dimensions ($O(nD)$ training instead of $O(n^2)$). Enables stochastic gradient training of kernel machines. Easy to implement and parallelize. **Cons:** Approximation quality depends exponentially on input dimension for some kernels. Requires large D for high accuracy. Variance in random features can be high. Only for shift-invariant kernels.

13.5.3 Nyström Approximation

Sample m landmark points $\{z_j\}_{j=1}^m$ and form kernel sub-matrices $\mathbf{K}_{mm}$ (landmark–landmark) and $\mathbf{K}_{nm}$ (data–landmark). The Nyström low-rank approximation is:

$$\mathbf{K} \approx \mathbf{K}_{nm} \mathbf{K}_{mm}^{-1} \mathbf{K}_{nm}^\top$$

The rank-m eigenfunctions of the kernel operator are approximated using the eigenvectors of $\mathbf{K}_{mm}$.

13.6 Graph Kernels

Graph kernels define similarity between structured objects (molecules, social networks, parse trees) by comparing substructures.

Random Walk Kernel: $K(G, G') = \sum_{\ell=0}^{\infty} \lambda^\ell\, \mathbf{q}^\top (\mathbf{A}_\times)^\ell \mathbf{p}$, where $\mathbf{A}_\times$ is the product graph's adjacency matrix. Counts matching random walks. $O(n^6)$ naive.

Shortest-Path Kernel: $K(G, G') = \sum_{u,v \in G} \sum_{u',v' \in G'} k_{\text{base}}\big(d(u, v), d(u', v')\big)$, comparing shortest-path length distributions.

Weisfeiler-Lehman (WL) Kernel: Iteratively refines node labels by aggregating neighbors' labels with hashing. Compares label histograms at each iteration. Efficient ($O(hm)$ for h iterations, m edges). As expressive as 1-WL graph isomorphism test.

Graphlet Kernel: Counts occurrences of small induced subgraphs (graphlets) of size 3–5. Computationally expensive due to subgraph isomorphism enumeration.

Deep Graph Kernels: Learn kernel from graph embeddings (e.g., node2vec embeddings + RBF), or use GNN hidden representations.

13.7 Deep Kernel Learning

Deep Kernel Learning (DKL): Combine a neural network feature extractor $\phi_\theta(\cdot)$ with a base kernel k_{base}:

$$k_{\text{DKL}}(x, x') = k_{\text{base}}\big(\phi_\theta(x),\, \phi_\theta(x')\big)$$

The NN learns a representation tailored to the GP, and the GP provides calibrated uncertainty. Jointly train θ and GP hyperparameters by maximizing the GP marginal likelihood.

Usage: When input data is high-dimensional and unstructured (images, audio, text) and a GP with an RBF kernel on raw pixels fails. DKL learns a low-dimensional manifold where a simple stationary kernel suffices. Applications: Bayesian optimization in high dimensions, calibrated classification, few-shot regression.

Pros: Retains GP uncertainty calibration. Scales with NN advancements. Inductive biases of both architectures (CNN+GP, Transformer+GP) can be combined. **Cons:** Marginal likelihood surface may be highly non-convex. Overfitting of NN parameters with large networks. Loses some GP interpretability.

13.8 Summary Comparison of Kernel Functions

Kernel selection guidelines:

1. **Start with RBF:** It is the most robust and well-studied kernel. Tune γ via cross-validation.
2. **If data is structured:** Use a domain-specific kernel (χ^2 for histograms, graph kernel for molecules, string kernel for DNA).
3. **GP modeling:** Start with Matérn $\nu = 5/2$ — it captures realistic smoothness and almost always outperforms RBF in GP regression.
4. **Add domain knowledge:** Sum kernels (linear + periodic + RBF) to encode trend + seasonality + local variation.
5. **Scale to big data:** Use RFF or inducing points with stochastic variational inference.
6. **High-dimensional inputs:** Use Deep Kernel Learning to learn a representation, then apply a simple kernel.

Table 13.1: Comprehensive Comparison of Kernel Functions

Kernel	**Parameters**	**Feature Dim**	**Smooth**	**Key Property**
Linear	c	d	N/A	Fastest; interpretable
Polynomial	γ, c, d	$\binom{d+p}{p}$	∞	Explicit degree control
RBF / SE	γ or ℓ	∞	C^∞	Universal; most versatile
Laplacian	γ	∞	C^0	L_1 robust; sparse data
Sigmoid	γ, c	N/A	∞	NOT always PSD; NN-like
χ^2	γ	∞	C^∞	Histogram features
Hist. Intersection	—	d	—	Minimal computation
Matérn $\nu = 1/2$	σ_f^2, ℓ	∞	C^0	Exponential; rough
Matérn $\nu = 3/2$	σ_f^2, ℓ	∞	C^1	Once diff.; GP default
Matérn $\nu = 5/2$	σ_f^2, ℓ	∞	C^2	Twice diff.; standard
Rational Quadratic	σ_f^2, ℓ, α	∞	C^∞	Multi-scale via mixture
Periodic	σ_f^2, ℓ, p	∞	C^∞	Cyclic/seasonal patterns
ArcSin (NN)	σ_f^2, Σ	∞	—	Inf.-width NN limit
Spectral Mixture	w_q, μ_q, v_q	∞	C^∞	Learns spectral density
NTK	Network arch.	∞	Arch.-dep.	Infinite-width training
RFF (approx.)	D	$2D$	Approx.	Scales to large n
Nyström (approx.)	m	m	Approx.	Low-rank kernel matrix

Chapter 14

Information Theory Metrics

Information theory provides the mathematical language to quantify uncertainty, information content, and the relationships between random variables. Its concepts underpin loss functions, generative modeling, representation learning, active learning, and reinforcement learning.

14.1 Core Quantities

14.1.1 Shannon Entropy (Discrete)

Shannon Entropy of a discrete random variable X with distribution $p(x)$:

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x) \quad \text{(bits)}$$

or equivalently in natural units (nats):

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \ln p(x) \quad \text{(nats)}$$

Entropy measures the average uncertainty, surprise, or information content of X. It quantifies the minimum average number of bits needed to encode outcomes of X (source coding theorem).

Properties of Shannon entropy:

- **Non-negativity:** $H(X) \geq 0$, with equality iff X is deterministic (one outcome has probability 1).
- **Maximum:** $H(X) \leq \log |\mathcal{X}|$, with equality iff $p(x)$ is uniform over $|\mathcal{X}|$ outcomes.
- **Concavity:** $H(\lambda p + (1-\lambda)q) \geq \lambda H(p) + (1-\lambda)H(q)$.
- **Additivity:** For independent X, Y: $H(X, Y) = H(X) + H(Y)$.
- **Permutation invariance:** $H(X)$ depends only on the probability values, not the labels of outcomes.
- **Binary entropy:** For $X \sim \text{Bernoulli}(p)$: $H_b(p) = -p \log_2 p - (1-p) \log_2 (1-p)$.

14.1.2 Differential Entropy (Continuous)

Differential Entropy of a continuous random variable X with density $p(x)$:

$$h(X) = -\int p(x) \ln p(x)\, dx$$

Unlike discrete entropy, differential entropy **can be negative**. It is not a measure of absolute information but of relative concentration of probability mass.

Differential entropy for common distributions:

- **Gaussian:** $h\big(\mathcal{N}(\mu,\sigma^2)\big) = \frac{1}{2}\ln(2\pi e\sigma^2)$. The Gaussian maximizes differential entropy for fixed variance σ^2.
- **Uniform on $[a,b]$:** $h = \ln(b-a)$.
- **Exponential with rate λ:** $h = 1 - \ln\lambda$.
- **Laplace with scale b:** $h = 1 + \ln(2b)$.
- **Multivariate Gaussian:** $h\big(\mathcal{N}(\boldsymbol{\mu},\boldsymbol{\Sigma})\big) = \frac{1}{2}\ln\big((2\pi e)^d \det(\boldsymbol{\Sigma})\big)$.

Maximum entropy principle: Among all distributions with given constraints, the max-entropy distribution is the "least informative" — it encodes only the constraints and nothing else. With known mean and variance, the Gaussian is max-entropy. With known support $[a,b]$, the uniform is max-entropy. This principle justifies many distributional assumptions in ML.

14.1.3 Joint Entropy

$$H(X,Y) = -\sum_{x,y} p(x,y)\log p(x,y)$$

For continuous: $h(X,Y) = -\iint p(x,y)\ln p(x,y)\,dx\,dy$. The subadditivity property: $H(X,Y) \le H(X) + H(Y)$, with equality iff X and Y are independent.

14.1.4 Conditional Entropy

$$H(Y \mid X) = -\sum_{x,y} p(x,y)\log p(y \mid x) = H(X,Y) - H(X)$$

The average uncertainty remaining in Y after observing X. Always: $0 \le H(Y \mid X) \le H(Y)$. The right inequality means conditioning never increases entropy on average.

14.2 Divergence Measures

14.2.1 Kullback-Leibler (KL) Divergence

KL Divergence (Relative Entropy):

$$\mathrm{KL}(P \,\|\, Q) = \sum_x p(x)\log\frac{p(x)}{q(x)} \quad \text{(discrete)}$$

$$\mathrm{KL}(P \,\|\, Q) = \int p(x) \log \frac{p(x)}{q(x)}\, dx \quad \text{(continuous)}$$

Measures the inefficiency of assuming distribution Q when the true distribution is P. It is the expected extra number of bits/nats required to encode samples from P using a code optimized for Q.

Key properties of KL divergence:

- **Non-negativity (Gibbs' inequality):** $\mathrm{KL}(P \,\|\, Q) \geq 0$, with equality iff $P = Q$ almost everywhere.
- **Not symmetric:** $\mathrm{KL}(P \,\|\, Q) \neq \mathrm{KL}(Q \,\|\, P)$ in general.
- **Not a metric:** Violates triangle inequality.
- **Invariant under reparameterization:** If $y = g(x)$ is invertible, $\mathrm{KL}(p(x)\|q(x)) = \mathrm{KL}(p(y)\|q(y))$.
- **Convexity:** $\mathrm{KL}(P \,\|\, Q)$ is jointly convex in (P, Q).

KL divergence for common distributions:

- **Univariate Gaussian:**

$$\mathrm{KL}\big(\mathcal{N}(\mu_1, \sigma_1^2) \,\|\, \mathcal{N}(\mu_2, \sigma_2^2)\big) = \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$$

- **Multivariate Gaussian:**

$$\mathrm{KL}\big(\mathcal{N}(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1) \,\|\, \mathcal{N}(\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)\big) = \frac{1}{2}\Big[\mathrm{tr}(\boldsymbol{\Sigma}_2^{-1}\boldsymbol{\Sigma}_1) + (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^\top \boldsymbol{\Sigma}_2^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) - d + \log \frac{\det(\boldsymbol{\Sigma}_2)}{\det(\boldsymbol{\Sigma}_1)}\Big]$$

- **Categorical:** $\mathrm{KL}(p \,\|\, q) = \sum_i p_i \log \frac{p_i}{q_i}$
- **Bernoulli:** $\mathrm{KL}(p \,\|\, q) = p \log \frac{p}{q} + (1-p) \log \frac{1-p}{1-q}$

Forward KL ($\mathrm{KL}(P \,\|\, Q)$, inclusive, mean-seeking): Q must cover all of P's support (otherwise KL $\to \infty$). Tends to produce Q that averages over modes of P. Used in maximum-likelihood estimation (MLE $= \arg\min_Q \mathrm{KL}(P_{\text{data}} \,\|\, Q)$).
Reverse KL ($\mathrm{KL}(Q \,\|\, P)$, exclusive, mode-seeking): Q is zero where P is zero. Tends to find a single mode of P. Used in Variational Inference (VI $= \arg\min_Q \mathrm{KL}(Q \,\|\, P_{\text{posterior}})$).
Mode- vs. mean-seeking: Forward KL leads to mean-seeking behavior (covers all modes, high recall); reverse KL leads to mode-seeking behavior (captures one mode well, high precision). This explains why VI can underestimate posterior variance.

14.2.2 Jensen-Shannon (JS) Divergence

$$\mathrm{JS}(P \,\|\, Q) = \frac{1}{2}\,\mathrm{KL}\left(P \,\|\, \frac{P+Q}{2}\right) + \frac{1}{2}\,\mathrm{KL}\left(Q \,\|\, \frac{P+Q}{2}\right)$$

Let $M = \frac{1}{2}(P+Q)$. The JS divergence is the average KL divergence from each distribution to their mixture midpoint.

Properties:

- **Symmetric:** $\mathrm{JS}(P \,\|\, Q) = \mathrm{JS}(Q \,\|\, P)$.
- **Bounded:** $0 \le \mathrm{JS}(P \,\|\, Q) \le \log 2$ (or $\ln 2$ using natural log). For log-base-b, the bound is $\log_b 2$.
- **Metric:** $\sqrt{\mathrm{JS}(P \,\|\, Q)}$ is a proper metric on the space of distributions (Endres & Schindelin, 2003).
- **Used in GANs:** The original GAN objective (Goodfellow et al., 2014) minimizes $\mathrm{JS}(P_{\text{data}} \,\|\, P_{\text{gen}})$, motivating the generator's loss.

14.2.3 Cross-Entropy

$$H(P, Q) = -\sum_{x} p(x) \log q(x) = H(P) + \mathrm{KL}(P \,\|\, Q)$$

Cross-entropy decomposes into the true distribution's entropy plus the KL divergence. Minimizing cross-entropy w.r.t. Q is equivalent to minimizing $\mathrm{KL}(P \,\|\, Q)$ since $H(P)$ is fixed.

Usage: Standard classification loss function. For C-class classification with one-hot true label y and predicted probabilities $\hat{y}$:

$$\mathcal{L}_{\text{CE}} = -\sum_{c=1}^{C} y_c \log \hat{y}_c = -\log \hat{y}_{\text{true}}$$

Minimizing CE encourages the model to assign probability 1 to the correct class (equivalent to maximizing the likelihood of the data under a categorical distribution).

14.2.4 f-Divergence Family

f-Divergence: For a convex function $f : \mathbb{R}_+ \to \mathbb{R}$ with $f(1) = 0$:

$$D_f(P \,\|\, Q) = \int q(x)\, f\left(\frac{p(x)}{q(x)}\right) dx$$

All f-divergences satisfy non-negativity and the data processing inequality (monotonicity under Markov kernels).

Generator functions $f(t)$ for common divergences:

- $f(t) = t \log t \;\to\; \mathrm{KL}(P \,\|\, Q)$
- $f(t) = -\log t \;\to\; \mathrm{KL}(Q \,\|\, P)$ (reverse KL)
- $f(t) = (\sqrt{t} - 1)^2 \;\to\; H^2(P, Q)$ (squared Hellinger)
- $f(t) = \frac{1}{2}|t - 1| \;\to\; \mathrm{TV}(P, Q)$ (Total Variation)
- $f(t) = (t - 1)^2 \;\to\; \chi^2(P \,\|\, Q)$ (Pearson χ^2)
- $f(t) = \frac{(t-\alpha)^+}{1-\alpha}$ for $\alpha \in (0, 1) \to \alpha$-divergence

14.2.5 Rényi Entropy and Divergence

Rényi Entropy of order α ($\alpha > 0, \alpha \neq 1$):

$$H_\alpha(P) = \frac{1}{1-\alpha} \log \left(\sum_x p(x)^\alpha \right)$$

Rényi Divergence of order α:

$$D_\alpha(P \,\|\, Q) = \frac{1}{\alpha - 1} \log \left(\int p(x)^\alpha q(x)^{1-\alpha} \, dx \right)$$

Special cases of Rényi entropy:

- $\alpha \to 1$: $\lim_{\alpha \to 1} H_\alpha(P) = H(P)$ (Shannon)
- $\alpha = 0$: $H_0(P) = \log |\mathrm{supp}(P)|$ (Hartley / max-entropy)
- $\alpha = 2$: $H_2(P) = -\log \sum p(x)^2$ (collision entropy)
- $\alpha \to \infty$: $H_\infty(P) = -\log \max_x p(x)$ (min-entropy)

Usage: Rényi divergences generalize KL. Used in differentially private ML (α-Rényi DP), variational inference (Rényi VI gives tighter bounds), and robust statistics.

14.3 Mutual Information

14.3.1 Definition

Mutual Information between X and Y:

$$I(X;Y) = \mathrm{KL}\big(p(x,y) \,\|\, p(x)p(y)\big) = \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)}$$

Equivalently:

$$I(X;Y) = H(X) - H(X \mid Y) = H(Y) - H(Y \mid X) = H(X) + H(Y) - H(X,Y)$$

Properties:

- **Non-negativity:** $I(X;Y) \geq 0$, with equality iff $X \perp\!\!\!\perp Y$.
- **Symmetry:** $I(X;Y) = I(Y;X)$.
- **Self-information:** $I(X;X) = H(X)$.
- **Data processing inequality:** If $X \to Y \to Z$ forms a Markov chain, then $I(X;Z) \leq I(X;Y)$.
- **Bounds:** $I(X;Y) \leq \min(H(X), H(Y))$.
- **Gaussian MI:** For jointly Gaussian (X,Y) with correlation ρ: $I(X;Y) = -\frac{1}{2}\log(1-\rho^2)$.

14.3.2 Conditional Mutual Information

$$I(X;Y \mid Z) = H(X \mid Z) - H(X \mid Y,Z)$$

The mutual information between X and Y given Z. Used in feature selection (do we gain information about Y from X beyond what Z already tells us?) and in defining transfer entropy for causal analysis.

14.3.3 Pointwise Mutual Information (PMI)

$$\mathrm{PMI}(x,y) = \log \frac{p(x,y)}{p(x)p(y)}$$

Measures the information overlap between specific outcomes. PMI can be **negative** when $p(x,y) < p(x)p(y)$. **PPMI** (Positive PMI): $\max(0, \mathrm{PMI}(x,y))$.

Usage: Word embedding models (word2vec, GloVe implicitly optimize PMI). PPMI matrices are factorized by SVD to produce word embeddings. Also used in collocation extraction, association mining, and information retrieval.

14.3.4 Variational Mutual Information Estimation

Donsker-Varadhan representation of KL (dual formulation):

$$\mathrm{KL}(P \,\|\, Q) = \sup_{T:\Omega\to\mathbb{R}} \mathbb{E}_{x\sim P}[T(x)] - \log \mathbb{E}_{x\sim Q}[e^{T(x)}]$$

Applied to MI: $I(X;Y) = \mathrm{KL}(p(x,y) \,\|\, p(x)p(y))$, giving the **MINE** estimator (Belghazi et al., 2018):

$$I(X;Y) \geq \sup_{T} \mathbb{E}_{p(x,y)}[T(x,y)] - \mathbb{E}_{p(x)p(y)}[e^{T(x,y)}]$$

where T is a neural network (statistics network).

Major variational MI bounds:

- **MINE (DV):** Lower bound using Donsker-Varadhan. High variance due to exponential.
- **InfoNCE (CPC, Oord et al., 2018):**

$$I(X;Y) \geq \mathbb{E}\left[\frac{1}{K}\sum_{i=1}^{K} \log \frac{f(x_i,y_i)}{\frac{1}{K}\sum_{j=1}^{K} f(x_i,y_j)}\right]$$

  where f is a learnable critic. Low variance, widely used in self-supervised representation learning (SimCLR, MoCo).
- **NWJ (Nguyen-Wainwright-Jordan):** Tighter than DV but still high variance.
- **JS bound:** Uses JS divergence variational form for more stable estimation.
- **CLUB (Contrastive Log-ratio Upper Bound, Cheng et al., 2020):** Provides an **upper** bound on MI (useful for MI minimization).

$$I(X;Y) \leq \mathbb{E}_{p(x,y)}[\log q_\theta(y \mid x)] - \mathbb{E}_{p(x)p(y)}[\log q_\theta(y \mid x)]$$

14.3.5 MI Applications in Machine Learning

Key applications of mutual information in ML:

- **InfoGAN** (Chen et al., 2016): Maximize $I(c; G(z,c))$ to encourage the generator to use latent code c in a semantically meaningful way, producing disentangled representations.
- **Deep InfoMax** (Hjelm et al., 2019): Maximize $I(\text{global repr}; \text{local patches})$ for unsupervised representation learning from images.
- **Feature selection:** Select features with maximal $I(f_i; Y)$ (JMI, MRMR, CMIM algorithms).
- **Fairness:** Enforce $I(Z;\hat{Y}) = 0$ where Z is a sensitive attribute.
- **Domain adaptation:** Maximize $I(X;Z)$ while minimizing $I(D;Z)$ for domain-invariant features.

- **RL exploration:** Use MI as intrinsic reward (VIME, EMI, RIDE) for curiosity-driven exploration.

14.4 Information Bottleneck (IB)

Information Bottleneck Principle (Tishby et al., 1999): Find a stochastic encoding T of X that is maximally compressive about X while retaining maximal information about Y:

$$\min_{p(t|x)} \; I(X;T) - \beta\, I(T;Y)$$

where $\beta \geq 0$ controls the compression–prediction trade-off. Equivalently: $\min I(X;T)$ s.t. $I(T;Y) \geq I_0$.

Optimal IB solution satisfies self-consistent equations:

$$p(t \mid x) = \frac{p(t)}{Z(x,\beta)} \exp\big(-\beta\, \mathrm{KL}\big(p(y \mid x) \,\|\, p(y \mid t)\big)\big)$$

$$p(y \mid t) = \sum_x p(y \mid x)\, p(x \mid t), \qquad p(t) = \sum_x p(t \mid x)\, p(x)$$

These are iterated until convergence (Blahut-Arimoto type algorithm).

Variational Information Bottleneck (VIB) (Alemi et al., 2017): For deep neural networks, the IB objective is approximated variationally:

$$\mathcal{L}_{\mathrm{VIB}} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(0,\mathbf{I})}\big[-\log q(y_i \mid f(x_i, \boldsymbol{\epsilon}))\big] + \beta\, \mathrm{KL}\big(p(\mathbf{z} \mid x_i) \,\|\, r(\mathbf{z})\big)$$

where $p(\mathbf{z} \mid x) = \mathcal{N}(\boldsymbol{\mu}(x), \mathrm{diag}(\boldsymbol{\sigma}^2(x)))$ is a stochastic encoder and $r(\mathbf{z}) = \mathcal{N}(0, \mathbf{I})$ is a fixed prior. This yields robustness to adversarial examples and better generalization.

14.5 Rate-Distortion Theory

Rate-Distortion: For a given source X and distortion measure $d(x, \hat{x})$, the rate-distortion function is:

$$R(D) = \min_{p(\hat{x}|x):\ \mathbb{E}[d(X,\hat{X})] \leq D} I(X;\hat{X})$$

$R(D)$ is the minimum information rate (bits per sample) needed to represent X with average distortion at most D.

Connection to representation learning: The rate-distortion trade-off $R(D)$ mirrors the compression–reconstruction trade-off in autoencoders and VAEs. The β-VAE objective $\mathbb{E}_{q(z|x)}[\log p(x \mid z)] - \beta\, \mathrm{KL}(q(z \mid x) \| p(z))$ is a Lagrangian relaxation of minimizing rate (KL term) subject to distortion (reconstruction) constraints.

14.6 Entropy in Machine Learning

14.6.1 Entropy Regularization in RL

MaxEnt / Soft Actor-Critic (SAC): Augment the RL objective with an entropy bonus to encourage exploration:

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t \big(r_t + \alpha H(\pi(\cdot \mid s_t))\big)\right]$$

where α is the temperature parameter controlling the exploration–exploitation balance. α can be automatically tuned by constraining the average entropy to a target value.

14.6.2 Information Gain and Active Learning

Expected Information Gain: In Bayesian experimental design, the information gain from observing y at input x is:

$$\mathrm{IG}(x) = H(\boldsymbol{\theta} \mid \mathcal{D}) - H(\boldsymbol{\theta} \mid \mathcal{D} \cup \{(x, y)\}) = I(\boldsymbol{\theta}; y \mid x, \mathcal{D})$$

where $\boldsymbol{\theta}$ are model parameters. The BALD (Bayesian Active Learning by Disagreement) criterion (Houlsby et al., 2011) approximates this for classification:

$$\mathrm{BALD}(x) = H(y \mid x, \mathcal{D}) - \mathbb{E}_{\boldsymbol{\theta} \sim p(\boldsymbol{\theta} \mid \mathcal{D})}\big[H(y \mid x, \boldsymbol{\theta})\big]$$

This selects points where the model is uncertain overall but individual models (MC samples) disagree (epistemic uncertainty).

14.6.3 Minimum Description Length (MDL)

MDL Principle: The best model M for data D is the one that minimizes the total description length:

$$L(D, M) = L(D \mid M) + L(M)$$

where $L(D \mid M) = -\log p(D \mid M)$ (negative log-likelihood) is the data description length and $L(M) = -\log p(M)$ is the model description length (prior). Minimizing $L(D, M)$ is equivalent to MAP estimation.

Connection to regularization: MDL provides an information-theoretic justification for complexity penalties (model selection, pruning, Occam's razor). The two-part code $L(D, M)$ is the exact objective minimized by Bayesian model selection when using the log-loss.

14.7 Entropy in Decision Trees

Information Gain for splitting dataset D on attribute a:

$$\text{IG}(D, a) = H(D) - \sum_{v \in \text{values}(a)} \frac{|D_v|}{|D|} H(D_v)$$

where $H(D) = -\sum_c p_c \log p_c$ is the entropy of class labels in D, and D_v is the subset where attribute $a = v$. Splitting criterion for ID3 and C4.5 algorithms.

Alternative splitting criteria:

- **Gain Ratio** (C4.5): $\text{GainRatio}(D, a) = \frac{\text{IG}(D,a)}{H_a(D)}$ where $H_a(D) = -\sum_v \frac{|D_v|}{|D|} \log \frac{|D_v|}{|D|}$ is the split info. Penalizes attributes with many values.
- **Gini Impurity** (CART): $\text{Gini}(D) = 1 - \sum_c p_c^2$. Faster to compute (no log) and typically yields similar trees.
- **Reduction in Variance** (regression trees): $\Delta\sigma^2 = \text{Var}(D) - \sum_v \frac{|D_v|}{|D|}\text{Var}(D_v)$.

14.8 Summary Table of Information-Theoretic Quantities

Practical guidance for practitioners:

1. **Loss functions:** Cross-entropy for classification, MSE for regression—both are MLE under natural distributional assumptions.
2. **Generative models:** Use forward KL (max-likelihood) for coverage, reverse KL (VI) for sharp samples. JS via GANs for high-dimensional distributions.
3. **Representation learning:** InfoNCE is the go-to lower bound for MI maximization. For MI *minimization*, CLUB provides a tractable upper bound.
4. **Exploration vs. exploitation:** Entropy regularization ($\alpha H(\pi)$) in SAC empirically works better than dithering strategies like ϵ-greedy.
5. **Model selection:** MDL provides a principled alternative to cross-validation when data is scarce—choose the model with shortest combined code length.

Table 14.1: Comprehensive Comparison of Information-Theoretic Quantities

Quantity	**Range**	**Sym?**	**Metric?**	**Key Relationship**
Shannon $H(X)$	$[0, \log \lvert\mathcal{X}\rvert]$	N/A	N/A	$H(X \mid Y) \leq H(X) \leq H(X,Y)$
Diff. $h(X)$	$(-\infty, \infty)$	N/A	N/A	Gaussian $= \frac{1}{2}\ln(2\pi e\sigma^2)$
Joint $H(X,Y)$	$[0, \log \lvert\mathcal{X}\rvert\lvert\mathcal{Y}\rvert]$	Yes	N/A	$= H(X) + H(Y\lvert X)$
Conditional $H(Y \mid X)$	$[0, H(Y)]$	No	N/A	$= H(X,Y) - H(X)$
$\mathrm{KL}(P \,\|\, Q)$	$[0, \infty)$	No	No	Cross-entropy $= H(P) +$ KL
$\mathrm{JS}(P \,\|\, Q)$	$[0, \log 2]$	Yes	$\sqrt{\mathrm{JS}}$ is	JS $= \frac{1}{2}\,\mathrm{KL}(P\|M) + \frac{1}{2}\,\mathrm{KL}(Q\|M)$
Cross-entropy $H(P,Q)$	$[H(P), \infty)$	No	No	Minimizing $=$ minimizing KL
Total Variation	$[0, 1]$	Yes	Yes	$\mathrm{TV} = \frac{1}{2}\lVert P - Q\rVert_1$
Hellinger H^2	$[0, 2]$	Yes	H is	$H^2 = 2(1 - \sum \sqrt{p_i q_i})$
Rényi D_α	$[0, \infty)$	No	No	$\lim_{\alpha\to 1} = \mathrm{KL}$
$I(X;Y)$	$[0, \min(H(X), H(Y))]$	Yes	No	$I(X;Y) = H(X) - H(X \mid Y)$
$I(X;Y \mid Z)$	$[0, \min(H(X \mid Z), H(Y \mid Z))]$	Yes	No	Chain rule: $I(X;Y,Z) = I(X;Z) + I(X;Y \mid Z)$
$\mathrm{PMI}(x,y)$	$(-\infty, \log \frac{1}{p(x)})$	Yes	No	$\mathbb{E}_{p(x,y)}[\mathrm{PMI}] = I(X;Y)$
InfoNCE	Lower bound	N/A	N/A	Log of $\frac{\text{positive}}{\text{sum of positives+negatives}}$
BALD	$[0, \log C]$	N/A	N/A	Epistemic uncertainty via disagreement
IG (tree)	$[0, H(D)]$	No	N/A	ID3/C4.5 splitting criterion
$I(X;T) - \beta I(T;Y)$	Objective	N/A	N/A	IB Lagrangian
Rate $R(D)$	Inverse of $D(R)$	N/A	N/A	Compression–distortion trade-off
MDL $L(D,M)$	> 0	N/A	N/A	$= -\log p(D \mid M) - \log p(M)$

Chapter 15

Dimensionality Reduction

Dimensionality reduction maps high-dimensional data $\mathbf{X} \in \mathbb{R}^{n \times d}$ to a lower-dimensional representation $\mathbf{Z} \in \mathbb{R}^{n \times k}$ ($k \ll d$) while preserving relevant structure. Applications include visualization, noise reduction, data compression, computational savings, and as preprocessing for downstream models.

15.1 Principal Component Analysis (PCA)

15.1.1 Standard PCA

PCA finds the orthogonal directions (principal components) that maximize the variance of the projected data.
Step 1 — Center: $\tilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}_n \boldsymbol{\mu}^\top$ where $\boldsymbol{\mu} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_i$.
Step 2 — Covariance: $\mathbf{S} = \frac{1}{n-1}\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$.
Step 3 — Eigendecomposition: $\mathbf{S} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\top$ where $\boldsymbol{\Lambda} = \text{diag}(\lambda_1, \ldots, \lambda_d)$ with $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0$, and $\mathbf{V} = [\mathbf{v}_1, \ldots, \mathbf{v}_d]$ is orthonormal.
Step 4 — Project: $\mathbf{Z} = \tilde{\mathbf{X}}\mathbf{V}_k$ where $\mathbf{V}_k$ contains the first k eigenvectors.

SVD formulation (more numerically stable):

$$\tilde{\mathbf{X}} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top$$

where $\mathbf{U} \in \mathbb{R}^{n \times n}$, $\boldsymbol{\Sigma} \in \mathbb{R}^{n \times d}$, $\mathbf{V} \in \mathbb{R}^{d \times d}$. The right singular vectors $\mathbf{V}$ are exactly the eigenvectors of $\mathbf{S}$, and the singular values satisfy $\sigma_i^2 = (n-1)\lambda_i$. Projection:

$$\mathbf{Z} = \tilde{\mathbf{X}}\mathbf{V}_k = \mathbf{U}_k\boldsymbol{\Sigma}_k$$

Variance explained by first k components:

$$\frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{d} \lambda_i}$$

This ratio guides the choice of k. Common heuristics: retain components until 90–95% variance explained, or use the "elbow" in the eigenvalue scree plot.

Usage: Visualization (project to 2D/3D), noise filtering (discard small-eigenvalue components), feature decorrelation (PCA whitening: $\mathbf{Z} = \tilde{\mathbf{X}}\mathbf{V}\boldsymbol{\Lambda}^{-1/2}$), data compression, and initialization of deep networks (PCA-based weight init).

Pros: Optimal linear DR in the sense of minimizing reconstruction MSE (Eckart-Young theorem). Efficient via SVD ($O(nd^2)$ for tall matrices). Interpretable through eigenvalues (variance) and eigenvectors (loadings). Unique solution. Closed form. **Cons:** Only

captures linear structure. Highly sensitive to feature scaling (always standardize before PCA). Assumes Gaussian data for probabilistic interpretation. Axes are constrained to be orthogonal, which may not reflect natural data manifolds. Poor for data lying on curved manifolds.

15.1.2 Kernel PCA (kPCA)

Apply PCA in a (potentially infinite-dimensional) feature space induced by a kernel K. Given centered kernel matrix $\tilde{\mathbf{K}}$:

$$\tilde{\mathbf{K}} = \mathbf{K} - \mathbf{1}_n\mathbf{K} - \mathbf{K}\mathbf{1}_n + \mathbf{1}_n\mathbf{K}\mathbf{1}_n$$

where $\mathbf{1}_n = \frac{1}{n}\mathbf{1}\mathbf{1}^\top$. Eigendecompose $\tilde{\mathbf{K}} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$. The i-th sample's k-th coordinate is:

$$z_{ik} = \sqrt{\lambda_k}\, u_{ik}$$

Projection of a new point x: $z_k(x) = \frac{1}{\sqrt{\lambda_k}} \sum_{i=1}^{n} u_{ik}\, K(x, x_i)$.

Pros: Captures non-linear manifolds (spiral, Swiss roll, concentric circles). Inherits flexibility from kernel choice. **Cons:** $O(n^2)$ memory for kernel matrix. $O(n^3)$ eigendecomposition (though approximations exist). Pre-image problem: cannot easily reconstruct original data from kernelized coordinates. Kernel and hyperparameter choice is critical.

15.1.3 Robust PCA (rPCA)

Robust PCA decomposes the data matrix $\mathbf{X}$ into a low-rank component $\mathbf{L}$ and a sparse outlier component $\mathbf{S}$:

$$\mathbf{X} = \mathbf{L} + \mathbf{S}$$

Solved via convex optimization (Principal Component Pursuit):

$$\min_{\mathbf{L},\mathbf{S}} \ \|\mathbf{L}\|_* + \lambda\|\mathbf{S}\|_1 \quad \text{s.t.} \quad \mathbf{X} = \mathbf{L} + \mathbf{S}$$

where $\|\cdot\|_*$ is the nuclear norm (sum of singular values, promoting low rank) and $\|\cdot\|_1$ promotes sparsity. Typical $\lambda = 1/\sqrt{\max(n, d)}$ (Candès et al., 2011).

Usage: Background subtraction in surveillance video (background = low-rank, foreground = sparse), removing shadows/specularities from face images, anomaly detection, and matrix completion with corrupt entries.

15.1.4 Probabilistic PCA (PPCA)

PPCA is a latent variable model:

$$p(\mathbf{z}) = \mathcal{N}(0, \mathbf{I}_k), \qquad p(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mathbf{W}\mathbf{z} + \boldsymbol{\mu}, \sigma^2\mathbf{I}_d)$$

where $\mathbf{W} \in \mathbb{R}^{d \times k}$ is the weight matrix (factor loadings). The marginal likelihood is:

$$p(\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \mathbf{W}\mathbf{W}^\top + \sigma^2 \mathbf{I})$$

Maximum likelihood solution: $\mathbf{W}_{\text{ML}} = \mathbf{V}_k(\boldsymbol{\Lambda}_k - \sigma^2\mathbf{I})^{1/2}\mathbf{R}$ where $\mathbf{R}$ is an arbitrary rotation matrix. As $\sigma^2 \to 0$, PPCA recovers standard PCA (Tipping & Bishop, 1999).

Pros: Probabilistic framework enables principled handling of missing data (EM algorithm), model selection via likelihood, mixture extensions (mixture of PPCA), and Bayesian treatment. Density estimation and generation. **Cons:** No closed form (requires EM or eigenvalue iteration). Rotational ambiguity in $\mathbf{W}$. Spherical noise assumption ($\sigma^2\mathbf{I}$) is restrictive for heterogeneous features.

15.1.5 Factor Analysis (FA)

Factor Analysis generalizes PPCA by allowing per-feature (diagonal) noise variances:

$$p(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mathbf{W}\mathbf{z} + \boldsymbol{\mu}, \boldsymbol{\Psi}), \qquad \boldsymbol{\Psi} = \text{diag}(\psi_1, \ldots, \psi_d)$$

Unlike PCA/PPCA, FA is **scale-invariant** — rescaling a feature changes ψ_i but the factors $\mathbf{z}$ remain consistent. Invariant to the choice of measurement units.

Usage: Psychometrics (intelligence factor models), finance (Fama-French factor models), and when features have different noise levels. FA discovers latent factors that explain correlations but not individual feature variance.

15.1.6 Independent Component Analysis (ICA)

ICA assumes data is generated by linearly mixing independent non-Gaussian sources:

$$\mathbf{x} = \mathbf{A}\mathbf{s}$$

Goal: recover the mixing matrix $\mathbf{A}$ and source signals $\mathbf{s}$ (up to permutation and scaling). The key insight is that non-Gaussianity of the sources enables identifiability (Darmois-Skitovich theorem). Criterion: maximize non-Gaussianity of $\mathbf{s} = \mathbf{W}\mathbf{x}$.
FastICA uses negentropy approximation:

$$J(y) \approx \sum_i k_i \big[\mathbb{E}[G_i(y)] - \mathbb{E}[G_i(\nu)]\big]^2$$

where $\nu \sim \mathcal{N}(0,1)$. Common contrast functions: $G_1(u) = \frac{1}{a}\log\cosh(au)$, $G_2(u) = -\exp(-u^2/2)$.

Usage: Blind source separation (cocktail party problem, EEG/MEG artifact removal), feature extraction for natural images (Gabor-like filters), financial time series decomposition, and fMRI signal analysis.

15.2 Manifold Learning

Manifold learning assumes data lies on (or near) a low-dimensional manifold embedded in $\mathbb{R}^d$. These non-linear methods recover the manifold coordinates.

15.2.1 Multidimensional Scaling (MDS)

Given pairwise dissimilarities δ_{ij}, **Metric MDS** finds points $\{\mathbf{z}_i\}$ minimizing the **stress**:

$$\text{Stress} = \sqrt{\frac{\sum_{i<j}\left(\delta_{ij} - \|\mathbf{z}_i - \mathbf{z}_j\|\right)^2}{\sum_{i<j}\delta_{ij}^2}}$$

Classical MDS (Torgerson-Gower): When δ_{ij} are Euclidean distances, form $\mathbf{B} = -\frac{1}{2}\mathbf{J}\mathbf{D}^{(2)}\mathbf{J}$ where $\mathbf{J} = \mathbf{I} - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$, $\mathbf{D}^{(2)}_{ij} = \delta^2_{ij}$. Then $\mathbf{Z} = \mathbf{V}_k\mathbf{\Lambda}_k^{1/2}$ from eigendecomposition of $\mathbf{B}$.

Usage: Visualization of dissimilarity data (perceptual color spaces, marketing brand maps), psychophysical scaling, and network layout.

15.2.2 Isomap

Isomap generalizes classical MDS by replacing Euclidean distances with **geodesic distances** along the data manifold:

1. Build k-NN (or ε-ball) neighborhood graph.
2. Compute all-pairs shortest path distances in the graph (Dijkstra or Floyd-Warshall) $\rightarrow$ geodesic distance matrix $\mathbf{D}_G$.
3. Apply Classical MDS to $\mathbf{D}_G$.

Usage: Swiss roll unfolding, face pose manifold, handwritten digit trajectories. Canonical example where PCA fails but Isomap succeeds.

Pros: Globally optimal for isometric embeddings. Provides a single global coordinate system. Intuitive extension of MDS. **Cons: Topological instability:** A single "short-circuit" edge (connecting different folds) catastrophically corrupts geodesic distances. Sensitive to k (too large $\rightarrow$ short circuits; too small $\rightarrow$ disconnected graph). $O(n^3)$ complexity. Does not provide explicit mapping for new points (Landmark Isomap addresses this).

15.2.3 Locally Linear Embedding (LLE)

LLE preserves local linear relationships:

1. For each point $\mathbf{x}_i$, find k nearest neighbors.

2. **Reconstruction weights:** Find $\mathbf{W}$ minimizing

$$\min_{\mathbf{W}} \sum_{i=1}^{n} \left\| \mathbf{x}_i - \sum_{j \in \mathcal{N}(i)} W_{ij} \mathbf{x}_j \right\|^2$$

subject to $\sum_j W_{ij} = 1$ for each i. Solve via local linear system (Gram matrix of neighbor differences).

3. **Embedding:** Find low-dimensional $\{\mathbf{z}_i\}$ preserving the same weights:

$$\min_{\mathbf{Z}} \sum_{i=1}^{n} \left\| \mathbf{z}_i - \sum_{j=1}^{n} W_{ij} \mathbf{z}_j \right\|^2$$

subject to $\mathbf{Z}^\top \mathbf{Z} = \mathbf{I}$ and zero mean. Solved as eigendecomposition of $(\mathbf{I} - \mathbf{W})^\top (\mathbf{I} - \mathbf{W})$.

Pros: Simple eigenproblem (no local minima). Good at unwrapping manifolds with complex topology. Only one hyperparameter k. **Cons:** Assumes manifold is locally linear and densely sampled. Weights are not guaranteed to be positive. Embedding can collapse if regularization is insufficient. $O(dnk^3)$ for weight computation.

15.2.4 Laplacian Eigenmaps

Laplacian Eigenmaps minimize the weighted sum of squared distances between neighbors:

$$\min_{\mathbf{Z}} \sum_{i,j} \|\mathbf{z}_i - \mathbf{z}_j\|^2 W_{ij} \quad \text{s.t.} \quad \mathbf{Z}^\top \mathbf{D} \mathbf{Z} = \mathbf{I}$$

where $\mathbf{W}$ is the adjacency matrix (heat kernel: $W_{ij} = \exp(-\|x_i - x_j\|^2/t)$ or binary), and $\mathbf{D} = \text{diag}(\sum_j W_{ij})$ is the degree matrix. Solution: generalized eigendecomposition $\mathbf{L}\mathbf{v} = \lambda \mathbf{D}\mathbf{v}$ where $\mathbf{L} = \mathbf{D} - \mathbf{W}$ is the graph Laplacian. Take eigenvectors corresponding to the smallest $k+1$ non-zero eigenvalues (discard $\mathbf{v}_0$ with $\lambda_0 = 0$).

Usage: Spectral clustering initialization, semi-supervised learning on graphs, and data visualization. The Laplacian eigenmaps framework is the foundation for spectral methods (spectral clustering, diffusion maps).

Pros: Elegant connection to spectral graph theory and heat equation. Preserves local neighborhood structure. Closed-form via eigendecomposition. **Cons:** Sensitive to graph construction (kernel bandwidth t). Embeddings can be distorted (eigenvectors minimize energy globally, not necessarily preserve global geometry). No natural extension to new points.

15.2.5 Diffusion Maps

Diffusion Maps define a random walk on the data:

$$P_{ij} = \frac{W_{ij}}{\sum_k W_{ik}}$$

where $\mathbf{P}$ is the Markov transition matrix. The t-step transition probabilities P^t reveal multi-scale geometric structure. The diffusion map embedding at time t is:

$$\mathbf{z}_i^{(t)} = \left[\lambda_2^t \psi_2(i),\ \lambda_3^t \psi_3(i),\ \ldots,\ \lambda_{k+1}^t \psi_{k+1}(i)\right]$$

where $\{\lambda_\ell, \psi_\ell\}$ are eigenvalues/eigenvectors of $\mathbf{P}$ (or equivalently, of the normalized Laplacian $\mathbf{D}^{-1/2}\mathbf{L}\mathbf{D}^{-1/2}$). The **diffusion distance** at time t:

$$D_t^2(i,j) = \|\mathbf{z}_i^{(t)} - \mathbf{z}_j^{(t)}\|^2$$

measures connectivity through all paths of length t.

Usage: Multi-scale data exploration (varying t reveals structure at different granularities), robust manifold learning (averages over all paths, less sensitive to noise than single shortest path), and spectral clustering (diffusion maps with $t = 1$ and manual thresholding).

Pros: Robust to noise and sampling density variations. Multi-scale analysis via diffusion time. Theoretical grounding in Markov processes and spectral geometry. Diffusion distance is more robust than geodesic distance. **Cons:** Choice of t is non-trivial. Computationally expensive (eigendecomposition of $n \times n$ sparse matrix). Parameter tuning (neighborhood size, kernel bandwidth, t) is complex.

15.2.6 Local Tangent Space Alignment (LTSA)

LTSA characterizes the local geometry via the tangent space at each point and aligns these local coordinate systems globally:

1. For each $\mathbf{x}_i$, find k neighbors and compute the local PCA (top d_{local} eigenvectors).
2. These define local tangent coordinates Θ_i.
3. Globally align all Θ_i to find global coordinates $\mathbf{Z}$ by solving a least-squares problem involving local affine transformations.

15.3 t-Distributed Stochastic Neighbor Embedding (t-SNE)

15.3.1 Algorithm

High-dimensional similarities (Gaussian kernel):

$$p_{j|i} = \frac{\exp\left(-\|x_i - x_j\|^2/2\sigma_i^2\right)}{\sum_{k\neq i}\exp\left(-\|x_i - x_k\|^2/2\sigma_i^2\right)}, \qquad p_{i|i} = 0$$

where σ_i is chosen so that the **perplexity** $\text{Perp}(P_i) = 2^{H(P_i)}$ equals a user-specified value (typically 5–50). Symmetrize:

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

Low-dimensional similarities (Student's t, $\nu = 1$):

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l}(1 + \|y_k - y_l\|^2)^{-1}}, \qquad q_{ii} = 0$$

Objective (minimize KL divergence):

$$C = \text{KL}(P \,\|\, Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

Gradient:

$$\frac{\partial C}{\partial y_i} = 4 \sum_j (p_{ij} - q_{ij})(y_i - y_j)(1 + \|y_i - y_j\|^2)^{-1}$$

Why the t-distribution? The heavy tails of the Student-t distribution alleviate the **crowding problem**: in low dimensions, the volume available to accommodate moderately distant points is too small. The t distribution gives distant points more "room" (higher q_{ij}) compared to a Gaussian, matching the high-dimensional Gaussian similarities.

Usage: The dominant method for visualizing high-dimensional data (2D or 3D scatter plots). Used extensively in single-cell genomics (scRNA-seq visualization), deep feature visualization, and exploratory data analysis across all sciences.

Pros: Excellent for revealing cluster structure at multiple scales. Robust to outliers (heavy-tailed low-dim distribution). Preserves local neighborhoods well. Widely implemented and understood. **Cons: No parametric mapping** for new test points (though parametric t-SNE exists). $O(n^2)$ complexity (Barnes-Hut reduces to $O(n \log n)$). Highly sensitive to perplexity—different values give different visualizations. Exaggerates cluster sizes and inter-cluster distances have no meaning. Not a metric-preserving embedding: global structure is lost. Non-convex optimization with random restarts.

15.3.2 Barnes-Hut t-SNE

Barnes-Hut approximation reduces the $O(n^2)$ gradient computation to $O(n \log n)$ by approximating long-range forces:

1. Build a quadtree (2D) or octree (3D) spatial index of the current embedding $\{\mathbf{y}_i\}$.
2. For each point, traverse the tree. If a cell is "far enough" (ratio of cell width to distance $< \theta$), approximate all points in that cell by their center of mass (multipole expansion).
3. θ (typically 0.5) controls accuracy vs. speed: smaller $\theta \to$ more accurate but slower.

15.4 Uniform Manifold Approximation and Projection (UMAP)

15.4.1 Algorithm

UMAP (McInnes et al., 2018) models data topology via fuzzy simplicial sets:
Step 1 — High-dimensional fuzzy set: Build weighted k-NN graph with edge weights:

$$v_{j|i} = \exp\big(-(d(x_i, x_j) - \rho_i)/\sigma_i\big)$$

where ρ_i is the distance to the nearest neighbor (local connectivity) and σ_i is found by binary search to normalize the total weight.
Symmetrize via fuzzy union: $w_h(e) = v_{j|i} + v_{i|j} - v_{j|i}\, v_{i|j}$.
Step 2 — Low-dimensional similarities:

$$w_l(e) = \big(1 + a\|y_i - y_j\|^{2b}\big)^{-1}$$

where a, b are fitted to approximate the function $\psi(d) = \exp(-d)$ for $d \leq \text{min_dist}$ (smooth near 0).
Step 3 — Optimize cross-entropy:

$$\mathcal{L}_{\text{UMAP}} = \sum_e \Big[w_h(e) \log \frac{w_h(e)}{w_l(e)} + (1 - w_h(e)) \log \frac{1 - w_h(e)}{1 - w_l(e)}\Big]$$

This is the fuzzy-set cross-entropy for each edge e.

Usage: Fast visualization of large datasets (millions of points). General-purpose nonlinear DR with better global structure preservation than t-SNE. Available as parametric UMAP for mapping new points.

Pros: Significantly faster than t-SNE ($O(n^{1.14})$ empirically vs. $O(n^2)$). Better preserves global structure (relative cluster positions). Fewer counter-intuitive hyperparameters. Parametric variant exists (learn a neural network encoder). Theoretical grounding in category theory and Riemannian geometry. **Cons:** More hyperparameters than t-SNE ($n_neighbors$, min_dist, metric). Results depend on initialization (spectral vs. random). Less established in literature than t-SNE. The Riemannian/ categorical theoretical justification is complex and not widely understood.

15.5 Autoencoder-Based Dimensionality Reduction

15.5.1 Standard Autoencoder

$$\mathcal{L}_{\text{AE}} = \frac{1}{n} \sum_{i=1}^{n} \|\mathbf{x}_i - g_\psi(f_\theta(\mathbf{x}_i))\|^2$$

The encoder $f_\theta : \mathbb{R}^d \to \mathbb{R}^k$ maps to a bottleneck $\mathbf{z} = f_\theta(\mathbf{x})$, and the decoder $g_\psi : \mathbb{R}^k \to \mathbb{R}^d$ reconstructs the input. The bottleneck dimension k enforces dimensionality reduction.

Pros: Non-linear DR via NN (universal approximator). Provides both an encoder (for new data) and decoder (for generation/reconstruction). Extensible (sparse, denoising, variational). **Cons:** Requires training; no unique/guaranteed solution. Hard to interpret the latent space. Hyperparameters: architecture, optimization, regularization. Undercomplete bottleneck may fail to learn useful structure.

15.5.2 Variational Autoencoder (VAE)

$$\mathcal{L}_{\text{VAE}} = -\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{z})] + \text{KL}\big(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z})\big)$$

where $q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}^2_\phi(\mathbf{x})))$ and $p(\mathbf{z}) = \mathcal{N}(0, \mathbf{I})$. The KL term regularizes the latent space to be smooth and continuous, enabling sampling and interpolation.

15.5.3 Contractive Autoencoder (CAE)

$$\mathcal{L}_{\text{CAE}} = \frac{1}{n}\sum_{i=1}^{n}[\|\mathbf{x}_i - g(f(\mathbf{x}_i))\|^2 + \lambda\|\mathbf{J}_f(\mathbf{x}_i)\|_F^2]$$

where $\mathbf{J}_f$ is the Jacobian of the encoder w.r.t. the input. Penalizing the Frobenius norm of the Jacobian encourages the learned representation to be **invariant to small input perturbations**.

15.5.4 Denoising Autoencoder (DAE)

$$\mathcal{L}_{\text{DAE}} = \frac{1}{n}\sum_{i=1}^{n}\|\mathbf{x}_i - g(f(\tilde{\mathbf{x}}_i))\|^2$$

where $\tilde{\mathbf{x}}_i$ is a corrupted version of $\mathbf{x}_i$ (e.g., additive Gaussian noise, dropout). Forces the encoder to learn robust features insensitive to the specific noise type applied.

15.6 Linear Discriminant Analysis (LDA)

LDA / Fisher's Linear Discriminant is a **supervised** dimensionality reduction method. It finds directions that maximize between-class scatter while minimizing within-class scatter:

Between-class scatter:

$$\mathbf{S}_B = \sum_{c=1}^{C} n_c(\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^\top$$

Within-class scatter:

$$\mathbf{S}_W = \sum_{c=1}^{C}\sum_{i:y_i=c}(\mathbf{x}_i - \boldsymbol{\mu}_c)(\mathbf{x}_i - \boldsymbol{\mu}_c)^\top$$

Optimization (Rayleigh quotient):

$$\mathbf{w}^* = \arg\max_{\mathbf{w}} \frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}}$$

Solution: eigenvectors of $\mathbf{S}_W^{-1}\mathbf{S}_B$. Projects to at most $C-1$ dimensions.

Usage: Supervised pre-processing for classification, face recognition (Fisherfaces), and when class labels are available and linear separation is adequate.

Pros: Uses label information for optimal class separation. Closed form. Fast. Interpretable discriminative directions. **Cons:** Assumes Gaussian class-conditionals with equal covariance. Singular $\mathbf{S}_W$ when $d > n$ (requires regularization or PCA pre-processing). Limited to $C-1$ dimensions. Only linear transformations.

15.7 Non-Negative Matrix Factorization (NMF)

NMF factorizes a non-negative data matrix $\mathbf{X} \in \mathbb{R}_+^{n \times d}$ into two non-negative matrices:

$$\mathbf{X} \approx \mathbf{W}\mathbf{H}, \qquad \mathbf{W} \in \mathbb{R}_+^{n \times k},\ \mathbf{H} \in \mathbb{R}_+^{k \times d}$$

Optimization (Frobenius norm version):

$$\min_{\mathbf{W},\mathbf{H} \geq 0} \|\mathbf{X} - \mathbf{W}\mathbf{H}\|_F^2$$

Multiplicative update rules (Lee & Seung, 2001):

$$\mathbf{H}_{kj} \leftarrow \mathbf{H}_{kj} \frac{(\mathbf{W}^\top \mathbf{X})_{kj}}{(\mathbf{W}^\top \mathbf{W}\mathbf{H})_{kj}}, \quad \mathbf{W}_{ik} \leftarrow \mathbf{W}_{ik} \frac{(\mathbf{X}\mathbf{H}^\top)_{ik}}{(\mathbf{W}\mathbf{H}\mathbf{H}^\top)_{ik}}$$

Alternative: KL divergence $\mathrm{KL}(\mathbf{X} \| \mathbf{W}\mathbf{H})$ with corresponding multiplicative updates.

Usage: Topic modeling (each topic = a row of $\mathbf{H}$ as a distribution over words), face decomposition (parts-based: eyes, nose, mouth as basis images), audio source separation, and collaborative filtering.

Pros: Parts-based, additive representation (no subtraction, more interpretable than PCA). Non-negativity constraints yield sparse, interpretable factors. Matrix $\mathbf{W}$ is the low-dimensional representation. **Cons:** Non-convex optimization (multiple local minima). Choice of k is critical and non-trivial. Convergence of multiplicative updates can be slow. No orthogonality or hierarchy among factors.

15.8 Random Projections

Johnson-Lindenstrauss Lemma: For any set of n points in $\mathbb{R}^d$ and $\varepsilon \in (0,1)$, there exists a linear map $f : \mathbb{R}^d \to \mathbb{R}^k$ with $k = O(\varepsilon^{-2} \log n)$ such that for all i, j:

$$(1-\varepsilon)\|\mathbf{x}_i - \mathbf{x}_j\|^2 \leq \|f(\mathbf{x}_i) - f(\mathbf{x}_j)\|^2 \leq (1+\varepsilon)\|\mathbf{x}_i - \mathbf{x}_j\|^2$$

The target dimension k depends only on n and ε, **not** on the original dimension d.

Random projection matrices $\mathbf{R} \in \mathbb{R}^{k \times d}$:

- **Gaussian:** $R_{ij} \sim \mathcal{N}(0, 1/k)$.
- **Sparse (Achlioptas, 2003):** $R_{ij} = \sqrt{3/k} \cdot \{-1, 0, 1\}$ with probabilities $\{1/6, 2/3, 1/6\}$.
- **Very sparse** (Li et al., 2006): $R_{ij} = \sqrt{s/k} \cdot \{-1, 0, 1\}$ with probabilities $\{1/2s, 1 - 1/s, 1/2s\}$, where $s = \sqrt{d}$ or $s = d/\log d$.

Projection: $\mathbf{Z} = \mathbf{X}\mathbf{R}^\top \in \mathbb{R}^{n \times k}$.

Pros: Extremely fast ($O(ndk)$), data-oblivious (no training), theoretical guarantees (JL lemma). Works as a pre-processing step for clustering, classification, etc. **Cons:** Randomness means different runs give different embeddings. Less compact than learned DR. Only preserves pairwise distances; may destroy other structures (clusters, manifolds). Needs moderately large k for practical utility.

15.9 Tensor Decompositions

CP Decomposition (CANDECOMP/PARAFAC): Decompose a tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}$ as sum of rank-1 outer products:

$$\mathcal{X} \approx \sum_{r=1}^{R} \lambda_r \, \mathbf{a}_r^{(1)} \circ \mathbf{a}_r^{(2)} \circ \cdots \circ \mathbf{a}_r^{(N)}$$

Each factor matrix $\mathbf{A}^{(n)}$ has R columns, one per component.
Tucker Decomposition: Core tensor $\mathcal{G}$ and factor matrices:

$$\mathcal{X} \approx \mathcal{G} \times_1 \mathbf{A}^{(1)} \times_2 \mathbf{A}^{(2)} \times_3 \cdots \times_N \mathbf{A}^{(N)}$$

Tensor Train (TT) Decomposition: Each element expressed as product of matrices: $\mathcal{X}_{i_1,\ldots,i_N} = \mathbf{G}_1[i_1]\,\mathbf{G}_2[i_2] \cdots \mathbf{G}_N[i_N]$.

Usage: Higher-order DR for multi-modal data (spatio-temporal, multi-subject fMRI). CP for fluorescence spectroscopy (unique under mild conditions unlike matrix factorization). Tucker for dimensionality reduction in each mode independently. Tensor Train for compressing large tensorized NN layers.

15.10 Metric Learning

Metric learning constructs a Mahalanobis distance for supervised DR:

$$d_{\mathbf{M}}(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{(\mathbf{x}_i - \mathbf{x}_j)^\top \mathbf{M}(\mathbf{x}_i - \mathbf{x}_j)}$$

with $\mathbf{M} = \mathbf{L}\mathbf{L}^\top$ (PSD). Projection: $\mathbf{z} = \mathbf{L}^\top \mathbf{x}$.
NCA (Neighbourhood Components Analysis): Maximizes the expected leave-one-out classification accuracy in the transformed space using a stochastic nearest-neighbor rule.
LMNN (Large Margin Nearest Neighbor): Pull target neighbors close while pushing differently-labeled examples beyond a margin. L= pull loss + push loss.

15.11 Summary Comparison Table

Method selection guide:

1. **Quick visualization of cluster structure:** UMAP (fast) or t-SNE (classic). Use Barnes-Hut for $n > 10^4$.
2. **Linear, interpretable, with variance:** PCA. Always standardize first.
3. **Non-linear manifold known to exist:** Isomap (if isometric), Diffusion Maps (if noisy), LLE (if locally linear).
4. **Supervised projection for classification:** LDA.
5. **Non-negative, interpretable factors:** NMF (text, images).
6. **Big data, need out-of-sample mapping:** Random projections or autoencoders.
7. **Probabilistic, handles missing data:** PPCA or FA.
8. **Source separation:** ICA.
9. **Multi-view / multi-modal:** Tensor decompositions.

Table 15.1: Comprehensive Comparison of Dimensionality Reduction Methods

Method	**Linear?**	**Super.?**	**Out-of-Sample?**	**Complexity**	**Key Featur**
PCA	Yes	No	Yes (proj.)	$O(nd^2)$	Optimal linear; va
kPCA	No	No	Yes (approx.)	$O(n^3)$	Kernel extension o
Robust PCA	Yes	No	Yes	Iterative	Handles sparse ou
PPCA	Yes	No	Yes (density)	EM $O(ndk)$	Probabilistic; missi
FA	Yes	No	Yes (density)	EM	Scale-invariant; per-fea
ICA	Yes	No	Yes (demixing)	$O(nd^2)$	Independent sou
MDS (classical)	Yes	No	No	$O(n^3)$	Preserves pairwis
Isomap	No	No	No	$O(n^3)$	Geodesic distan
LLE	No	No	No	$O(nk^3d)$	Local linear pat
Laplacian Eigenmaps	No	No	No	Sparse eig.	Spectral; Laplace-B
Diffusion Maps	No	No	No	Sparse eig.	Multi-scale; noise
LTSA	No	No	No	$O(nk^2d)$	Tangents aligned g
t-SNE	No	No	No (param. exists)	$O(n^2)$	Perplexity; visuali
Barnes-Hut t-SNE	No	No	No	$O(n \log n)$	Fast t-SNE for larg
UMAP	No	No	Yes (param.)	$O(n^{1.14})$	Fast; global + l
Autoencoder	No	No	Yes (encoder)	Train-dep.	NN-learned latent
VAE	No	No	Yes (encoder)	Train-dep.	Probabilistic; gene
CAE	No	No	Yes (encoder)	Train-dep.	Jacobian pena
DAE	No	No	Yes (encoder)	Train-dep.	Denoising crite
LDA	Yes	Yes	Yes (proj.)	$O(nd^2)$	Maximizes class sep
NMF	Yes	No	Yes (approx.)	Iterative	Non-negative; part
Random Proj.	Yes	No	Yes	$O(ndk)$	JL lemma; data-ob
Tensor Decomp.	Yes (mode)	No	Yes	Dep. on order	Multi-modal; stru
Metric Learning	Yes	Yes	Yes	$O(d^3)$	Mahalanobis; LM

Chapter 16

Initialization Methods

Weight initialization is a critical architectural choice that determines whether a deep neural network trains successfully. Poor initialization leads to vanishing gradients, exploding gradients, or symmetry that prevents learning. This chapter surveys initialization schemes and their mathematical justifications.

16.1 Why Initialization Matters

The initialization problem: Given a neural network with L layers, find initial values for weight matrices $\mathbf{W}^{(l)}$ and bias vectors $\mathbf{b}^{(l)}$ such that during forward propagation, activations maintain a stable variance across layers, and during backpropagation, gradients neither vanish nor explode.

Four critical effects of initialization:

1. **Convergence speed:** Bad init $\rightarrow$ many wasted epochs correcting the "wrong" starting point.
2. **Gradient flow:** Vanishing gradients make early layers untrainable; exploding gradients cause divergence.
3. **Local minimum quality:** The initialization determines which basin of attraction gradient descent falls into.
4. **Training success:** For very deep networks (100+ layers), training fails entirely without proper initialization.

16.2 Classic Initialization Schemes

16.2.1 Zero / Constant Initialization

All weights initialized to the same value:

$$\mathbf{W}^{(l)} = \mathbf{0} \quad \text{or} \quad \mathbf{W}^{(l)} = c\,\mathbf{1}\mathbf{1}^{\top}$$

Pros: Trivially simple. Fine for biases. **Cons: Fatal symmetry problem:** All neurons in the same layer compute identical functions and receive identical gradients. The network **never breaks symmetry** during gradient descent, learning only a single feature per layer regardless of width. **Never use zero init for weights.**

16.2.2 Random (Uniform / Normal)

$$\mathbf{W}_{ij}^{(l)} \sim \mathcal{U}(-a, a) \quad \text{or} \quad \mathbf{W}_{ij}^{(l)} \sim \mathcal{N}(0, \sigma^2)$$

With fixed scale parameters a or σ independent of layer dimensions.

Pros: Breaks symmetry (each neuron starts differently). Easy to implement. **Cons: No layer-size adjustment:** In deep networks, activations either grow exponentially (if σ too large) or shrink to zero (if σ too small) as they pass through layers. The variance compounds: $\text{Var}(y^{(L)}) \approx \text{Var}(x) \prod_{l=1}^{L} n_l \text{Var}(w^{(l)})$ for linear layers.

16.3 Famous Initialization Schemes

16.3.1 Xavier / Glorot Initialization (2010)

Goal: Keep the variance of activations **and** gradients constant across layers. Derived for linear activations (and works for tanh).
Forward pass variance analysis: For layer l with n_{in} inputs, assuming zero-mean inputs, independent weights, and linear activation:

$$\text{Var}(y^{(l)}) = n_{\text{in}} \, \text{Var}(w) \, \text{Var}(y^{(l-1)})$$

To maintain $\text{Var}(y^{(l)}) = \text{Var}(y^{(l-1)})$, we need $\text{Var}(w) = 1/n_{\text{in}}$.
Backward pass variance analysis:

$$\text{Var}\left(\frac{\partial \mathcal{L}}{\partial y^{(l-1)}}\right) = n_{\text{out}} \, \text{Var}(w) \, \text{Var}\left(\frac{\partial \mathcal{L}}{\partial y^{(l)}}\right)$$

To maintain gradient variance, we need $\text{Var}(w) = 1/n_{\text{out}}$.
Compromise (harmonic mean):

$$\text{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

Xavier initialization formulas:
Uniform:

$$\mathbf{W}_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

(Recall: $\text{Var}(\mathcal{U}(-a, a)) = a^2/3$. Setting $a^2/3 = 2/(n_{\text{in}} + n_{\text{out}}) \Rightarrow a = \sqrt{6/(n_{\text{in}} + n_{\text{out}})}$.)
Normal:

$$\mathbf{W}_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$$

Usage: Default initialization for tanh, sigmoid, softsign, and linear activations. Built into PyTorch as `xavier_uniform_` / `xavier_normal_` and TensorFlow/Keras as `glorot_uniform` / `glorot_normal`. **NOT suitable for ReLU** as the derivation assumes symmetric activations around zero.

Pros: Mathematically grounded variance analysis. Balances forward and backward signal propagation. Works well in practice for shallow to moderate-depth networks with symmetric activations. **Cons:** Derivation assumes linear activations—corrections needed for ReLU (see Kaiming init). The $2/(n_{\text{in}} + n_{\text{out}})$ compromise is a heuristic; actual optimal variance is activation-dependent.

16.3.2 Kaiming / He Initialization (2015)

Key insight: ReLU sets negative inputs to zero, halving the output variance: $\text{Var}(\text{ReLU}(y)) = \frac{1}{2}\text{Var}(y)$ for $y \sim \mathcal{N}(0, \sigma^2)$. To compensate, double the weight variance:

$$\text{Var}(w) = \frac{2}{n_{\text{in}}} \quad \text{(fan_in mode)}$$

or

$$\text{Var}(w) = \frac{2}{n_{\text{out}}} \quad \text{(fan_out mode)}$$

Kaiming initialization formulas:
Uniform (fan_in):

$$\mathbf{W}_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}}}, \sqrt{\frac{6}{n_{\text{in}}}}\right)$$

Normal (fan_in):

$$\mathbf{W}_{ij} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right)$$

Wait—the parameter to $\mathcal{N}(\mu, \sigma)$ is **standard deviation** σ, **not variance**. So:

$$\mathbf{W}_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right) \quad \text{(variance)}$$

or equivalently $\mathbf{W}_{ij} \sim \mathcal{N}(0, \sqrt{2/n_{\text{in}}})$ (std. dev.).
For Leaky ReLU with negative slope a:

$$\text{Var}(w) = \frac{2}{(1 + a^2)\, n_{\text{in}}}$$

Usage: Default initialization for ReLU, Leaky ReLU, PReLU, ELU, and all ReLU-family activations. Built into PyTorch as `kaiming_uniform_` / `kaiming_normal_` and TensorFlow/Keras as `he_uniform` / `he_normal`. This is the **standard choice for modern CNNs and most feedforward networks.**

Pros: Properly compensates for ReLU's variance-halving effect. Empirically validated across architectures (ResNet, VGG, Inception). Enables training of very deep ReLU networks. **Cons:** Assumes ReLU activation; different non-linearities need different correction factors. The $2/n_{\text{in}}$ derivation uses Gaussian input assumptions; actual distribution may differ.

16.3.3 LeCun Initialization (1998)

LeCun initialization targets unit output variance for each layer with symmetric activations:

$$\text{Var}(w) = \frac{1}{n_{\text{in}}}$$

Uniform: $w \sim \mathcal{U}(-\sqrt{3/n_{\text{in}}}, \sqrt{3/n_{\text{in}}})$. Normal: $w \sim \mathcal{N}(0, 1/n_{\text{in}})$.

Usage: Specifically designed for and works well with **SELU** (Scaled ELU) activation. With SELU and LeCun init, the network is self-normalizing—activations automatically converge to zero mean and unit variance.

16.3.4 Orthogonal Initialization

Initialize the weight matrix as a (semi-)orthogonal matrix:

$$\mathbf{W}^\top \mathbf{W} = \mathbf{I} \quad \text{or} \quad \mathbf{W}\mathbf{W}^\top = \mathbf{I}$$

Implemented by drawing a random Gaussian matrix $\mathbf{G}$, computing its SVD $\mathbf{G} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$, and setting $\mathbf{W} = \mathbf{U}\mathbf{V}^\top$ (or $\mathbf{U}$). This yields the nearest orthogonal matrix to $\mathbf{G}$.

Properties of orthogonal weights:

- **Norm preservation:** $\|\mathbf{W}\mathbf{x}\| = \|\mathbf{x}\|$ for any input $\mathbf{x}$. No exploding/vanishing in linear layers.
- **Gradient preservation:** The backpropagated gradient norm is also preserved (since $\mathbf{W}^\top$ is also orthogonal).
- **Weight diversity:** Orthogonality maximizes "angular diversity" among neuron receptive fields.
- **Non-square case:** For wide matrices ($d_{\text{in}} < d_{\text{out}}$), use output-orthogonal ($\mathbf{W}\mathbf{W}^\top = \mathbf{I}$). For tall ($d_{\text{in}} > d_{\text{out}}$), use input-orthogonal ($\mathbf{W}^\top\mathbf{W} = \mathbf{I}$).

Usage: Recurrent neural networks (hidden-to-hidden weight matrices in LSTM/GRU) to prevent vanishing/exploding gradients through time. Generative adversarial networks (GANs). Deep linear networks for theoretical analysis. Also beneficial for convolutional kernels reshaped to matrices.

Pros: Perfect norm preservation forward and backward. Ideal for very deep or recurrent architectures. No hyperparameters to tune. Mathematically principled. **Cons:** Only applicable to square or near-square weight matrices. SVD computation for large matrices is expensive. The orthogonality constraint is gradually lost during training (though orthogonal regularization can maintain it). Standard optimization may not preserve orthogonality.

16.4 Specialized Initialization Techniques

16.4.1 LSUV (Layer-Sequential Unit-Variance)

LSUV (Mishkin & Matas, 2016) is a data-dependent initialization:

1. Initialize all weights with orthogonal matrices.
2. For each layer, feed a mini-batch through the network.
3. Rescale weights (multiply by a scalar) so that the output of that layer has variance exactly 1.
4. Proceed to the next layer.

This guarantees that **for the given data batch**, every layer's output has unit variance, regardless of architecture.

Pros: Data-dependent calibration (adapts to actual input statistics). Works for any activation function. Empirically faster convergence than Kaiming init for some architectures.
Cons: Requires a data batch at initialization time. Slightly more complex to implement. Results depend somewhat on the specific batch used.

16.4.2 Normalization Layer Defaults

Batch Normalization: Scale γ initialized to 1, shift β initialized to 0. This starts BN as an identity operation (zero-mean, unit-variance then shifted), gradually learning the optimal affine transformation.
Layer Normalization: Similarly, $\gamma = 1$, $\beta = 0$.
Instance / Group Normalization: Same defaults.

16.4.3 LSTM Forget Gate Bias

LSTM forget gate bias initialization: Initialize $\mathbf{b}_f = \mathbf{1}$ (or some positive constant like 0.5–1.0). This biases the forget gate to "remember" initially:

$$f_t = \sigma\big(\mathbf{W}_f[h_{t-1}, x_t] + \mathbf{b}_f\big)$$

With $\mathbf{b}_f = \mathbf{1}$, $f_t \approx 0.73$ initially, meaning the cell state is preserved. This mitigates the problem of LSTM forgetting everything at the start of training, enabling better long-range dependency learning (Gers et al., 2000; Jozefowicz et al., 2015).

Usage: Standard practice in sequence modeling. PyTorch's LSTM default sets forget gate bias to 1. Should be used in any LSTM-based architecture (language models, seq2seq, speech recognition).

16.4.4 Transformer Initialization

Standard Transformer (Vaswani et al., 2017): Linear layers use Xavier/Glorot uniform initialization. Attention projection matrices treated as standard linear layers.
T5 (Raffel et al., 2020): Uses a factorized form for some feed-forward layers: $W = UV^\top$ with U, V initialized by $\mathcal{N}(0, 1/\sqrt{d_{\text{model}}})$.
GPT-series scaling: GPT-2/GPT-3 initialize weights from $\mathcal{N}(0, 0.02/\sqrt{n_{\text{layer}}})$, where n_{layer} is the number of transformer layers. This **scales initial weight variance inversely with depth**, preventing residual stream from growing exponentially. For GPT-3 (96 layers): $\sigma \approx 0.02/\sqrt{96} \approx 0.002$.

GPT-style residual scaling: Combined with pre-LayerNorm, the residual output is:

$$x_{l+1} = x_l + \text{LayerNorm}(\text{FFN}(\text{Attention}(x_l)))$$

With small initial weights, the contribution of each layer to the residual stream starts small, so the network roughly computes the identity function at initialization, which is optimal for deep residual networks.

16.4.5 Fixup Initialization (2019)

Fixup (Zhang et al., 2019) **trains ResNets without normalization layers** by carefully initializing residual branches:

- Initialize classification layer and the last layer of each residual branch to zero.
- Use Kaiming init for other layers.
- Add learnable scalar multipliers $c \propto 1/\sqrt{L}$ on residual branches.
- Add a scalar multiplier (init to 1) before each nonlinearity.

This prevents exploding activations while using standard SGD, replacing the stabilizing effect of BatchNorm.

16.4.6 ReZero / SkipInit

ReZero (Bachlechner et al., 2021): Parameterize each residual layer as

$$x_{l+1} = x_l + \alpha_l \cdot F_l(x_l)$$

with α_l initialized to $\mathbf{0}$ (scalar or per-channel). The network starts as the identity function, and each layer's contribution **gradually turns on** during training. This achieves:

- Signal propagation independent of depth.
- Training very deep Transformers (100+ layers) without warmup or normalization.
- Faster convergence than standard residual connections.

SkipInit (De & Smith, 2020) is an equivalent proposal.

Pros: Enables training arbitrary-depth networks without BatchNorm/ LayerNorm. Eliminates the need for learning rate warmup. Simple to implement (one scalar per layer). **Cons:** Zero-initialized layers take time to "activate," which may waste early training steps. Not all architectures benefit equally; success is architecture-dependent. Still less battle-tested than standard normalization-based training.

16.4.7 μP (Maximal Update Parameterization)

μP (Yang & Hu, 2021) prescribes per-layer scaling rules for both initialization variance and learning rate as functions of layer width:
Hidden weights (fan_in: n_{in}, fan_out: n_{out}):

$$\mathbf{W}_{ij}^{(l)} \sim \mathcal{N}\left(0, \frac{1}{n_{\text{in}}}\right), \quad \eta_{\mathbf{W}^{(l)}} \propto \frac{1}{n_{\text{out}}}$$

Readout weights (final layer):

$$\mathbf{W}_{ij}^{(\text{out})} \sim \mathcal{N}\left(0, \frac{1}{n_{\text{in}}^2}\right), \quad \eta_{\mathbf{W}^{(\text{out})}} \propto \frac{1}{n_{\text{in}}}$$

Under μP, the **optimal hyperparameters transfer** from a small proxy model to a large target model, enabling hyperparameter search at a fraction of the cost.

Usage: Large-scale training where hyperparameter search at full scale is prohibitively expensive. Used in Cerebras-GPT, some large language model training pipelines, and research on scaling laws.

16.5 Bias Initialization

Standard practice: **Initialize all biases to zero**:

$$\mathbf{b}^{(l)} = \mathbf{0}$$

Justification: with non-zero and random weights, the symmetry is already broken; zero biases merely shift the activation function's operating point.

Exceptions—when to use non-zero biases:

- **LSTM forget gate:** $\mathbf{b}_f = 1$ (see §16.4.3).
- **ReLU biases:** Some practitioners initialize biases to a small positive value (e.g., 0.1) to ensure all ReLUs are "alive" (non-zero output) initially, preventing dead ReLU paths.
- **Mean subtraction in BN:** If using BatchNorm, the bias term of the preceding linear layer is irrelevant (BN subtracts the mean), but some frameworks still recommend zero.
- **Output layer bias for imbalanced classification:** Set $\mathbf{b}$ so that initial predic-

tions match the prior class distribution (e.g., $\mathbf{b}_c = \log(p(c))$).

16.6 Embedding Initialization

Random initialization: Embedding matrices are typically initialized from $\mathcal{U}(-0.05, 0.05)$ or $\mathcal{N}(0, 0.01)$ — small values to avoid large initial dot product magnitudes. **Pretrained initialization:** Initialize with pre-trained word embeddings (Word2Vec, GloVe, FastText, BERT embeddings). For domain-specific tasks, fine-tune or freeze the embedding layer.

Usage: NLP models (word/ token embeddings), recommender systems (user/item embeddings), knowledge graph embeddings (TransE, RotatE), and categorical feature embeddings in tabular deep learning.

16.7 Summary Comparison Table

Table 16.1: Comprehensive Comparison of Initialization Methods

Method	**Variance** $\mathrm{Var}(w)$	**Activation**	**Year**	**Key Idea**
Zero	0	Any	—	All units learn same (DON'T use)
Small random	Fixed small	Linear	—	No layer-size compensation
LeCun	$1/n_{\text{in}}$	Tanh/SELU	1998	Forward variance = 1
Xavier/Glorot	$2/(n_{\text{in}} + n_{\text{out}})$	Tanh/Sigmoid	2010	Balances forward and backward
Kaiming/He	$2/n_{\text{in}}$	ReLU family	2015	Compensates ReLU variance halving
Orthogonal	$\mathbf{W}^\top\mathbf{W}=\mathbf{I}$	Any	—	Perfect norm preservation
LSUV	Data-dep.	Any	2016	Rescale to unit output variance
Fixup	Mixed	ReLU (w/o BN)	2019	Train ResNets without BN
ReZero / SkipInit	$\alpha_l = 0$ initially	Any	2020/21	Start as identity
μP	$1/n_{\text{in}}$ (hidden)	Any	2021	Hyperparameter transfer across widths
GPT scale	$\frac{0.02}{\sqrt{n_{\text{layer}}}}$	GeLU	2019–	Depth-scaled variance for Transformers
T5	$1/\sqrt{d_{\text{model}}}$	ReLU/GeLU	2020	Factorized projection init
Forget gate bias	$b_f = 1$	LSTM	2000	Biases towards remembering
Output bias	$\log(p(c))$	Classif.	—	Matches class priors

Initialization selection guide:

1. **ReLU networks (CNN, MLP, ResNet):** Kaiming/He uniform (standard choice).
2. **Tanh/Sigmoid networks:** Xavier/Glorot uniform (if you still use these activations).
3. **SELU networks:** LeCun normal (enables self-normalizing property).
4. **RNN/LSTM hidden weights:** Orthogonal initialization.
5. **Transformers:** Xavier/Glorot uniform with small scale, or depth-scaled $\mathcal{N}(0, 0.02/\sqrt{n_{\text{layer}}})$ for very deep models.
6. **Training without normalization:** Fixup or ReZero.
7. **Large-scale training with HP transfer:** μP.
8. **Biases:** Zero (except LSTM forget gate = 1, ReLU bias = small positive, and output bias for prior matching).
9. **If uncertain:** Kaiming/He uniform + zero biases. This works for 90%+ of cases.

Chapter 17

Learning Rate Schedules

The learning rate η is arguably the single most important hyperparameter in neural network training. While a fixed learning rate can work for simple problems, modern deep learning relies on carefully designed **learning rate schedules** that adapt the step size over the course of training. This chapter provides a comprehensive survey of learning rate schedules, each presented with its mathematical definition, usage patterns, and a balanced assessment of strengths and limitations.

17.1 Constant Learning Rate

Constant LR

The simplest possible schedule holds η fixed for the entire training run:

$$\eta_t = \eta_0 \quad \forall t$$

where η_0 is the initial learning rate and t indexes training steps (iterations or epochs).

Applications

Baseline for comparison with more sophisticated schedules. Occasionally used with optimizers like Adam that adapt per-parameter learning rates internally, or in on-policy RL algorithms (PPO) where a constant LR is standard. Also used in continual learning and online learning settings.

Trade-offs

Pros:

- No hyperparameters beyond η_0; trivially simple.
- Predictable behavior throughout training.
- Avoids introducing additional inductive biases from schedule shape.

Cons:

- **Impossible trade-off:** Large η_0 causes early instability; small η_0 causes slow convergence.
- Cannot escape narrow local minima that require learning rate reduction.
- Suboptimal final accuracy in most settings; typically underperforms any decay schedule by 1–3% on benchmarks.

17.2 Step Decay

Step Decay

The learning rate is reduced by a constant factor γ every T steps (or epochs):

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/T \rfloor}$$

where $\gamma \in (0, 1)$ (typical values: 0.1, 0.5), T is the step interval ("step size"), and $\lfloor \cdot \rfloor$ denotes the floor function.

Example values

For $\eta_0 = 0.1$, $\gamma = 0.5$, $T = 30$ epochs:

$$\text{Epoch 0–29: } \eta = 0.1, \quad \text{Epoch 30–59: } \eta = 0.05, \quad \text{Epoch 60–89: } \eta = 0.025, \ \ldots$$

Applications

Standard for training CNNs on ImageNet (ResNet, VGG, AlexNet). Common in academic benchmarks where hand-tuned schedules are specified per model. Often combined with SGD + momentum.

Trade-offs

Pros:

- Intuitive interpretation: "train at η_0, then refine at lower rates."
- Works reliably for many vision tasks with SGD.
- Only three hyperparameters (η_0, γ, T).

Cons:

- **Discontinuous jumps** cause loss spikes at each step boundary.
- γ and T require problem-specific tuning; no principled defaults.
- Spending too many epochs at a high LR wastes compute; dropping too early prevents convergence.
- Lacks smooth decay, so convergence near the optimum can be jittery.

17.3 Exponential Decay

Exponential Decay

The learning rate decays exponentially at every step:

$$\eta_t = \eta_0 \cdot \gamma^t$$

where $\gamma \in (0, 1)$ is the decay factor (settings closer to 1, e.g., 0.95–0.999, are typical for per-step decay).

Applications

Frequent in early DL frameworks. Used when a smooth, continuous decrease is desired without step boundaries. Effective for fine-tuning where the LR should gradually shrink to a tiny value.

Trade-offs

Pros:

- Smooth, continuous decay; no discontinuities.
- Only two hyperparameters (η_0, γ).
- Naturally approaches zero asymptotically.

Cons:

- The rate of decay is fastest early on, when exploration is most important.
- Choosing γ is difficult: 0.99 per step and 0.99 per epoch give wildly different behaviors.
- No plateau phase; the LR immediately starts decreasing.
- Optimizers with momentum/adaptive rates often compensate, making the exponential shape less meaningful.

17.4 Time-Based Decay

Time-Based (Inverse Time) Decay

The learning rate decreases according to the inverse of the training time:

$$\eta_t = \frac{\eta_0}{1 + kt}$$

where $k > 0$ is the decay rate (or "decay parameter"). This schedule derives from the theoretical learning rate for SGD convergence in convex optimization: $\eta_t \propto 1/t$.

Applications

Theoretically motivated for convex SGD. Used in some implementations of TensorFlow's built-in schedules. Occasionally seen in classic ML (SVM, perceptron) rather than modern deep learning.

Trade-offs

Pros:

- Theoretically justified for convex optimization; ensures $\sum_t \eta_t = \infty$ and $\sum_t \eta_t^2 < \infty$ (Robbins–Monro conditions) for asymptotic convergence.
- Smooth, parameter-free shape beyond η_0 and k.

Cons:

- Decays too quickly in early training for non-convex deep learning; exploration is prematurely curtailed.
- The $1/t$ asymptotics are too aggressive for modern over-parameterized models where long training is beneficial.
- Rarely competitive with cosine or step decay on benchmarks.

17.5 Polynomial Decay

Polynomial (Poly) Decay

The learning rate decays according to a polynomial function over a fixed total number of steps T:

$$\eta_t = \eta_0 \cdot \left(1 - \frac{t}{T}\right)^p$$

where $p > 0$ is the power (typical values: $p \in \{0.5, 1, 2\}$). At $t = T$, the LR reaches zero. An optional parameter $\eta_{\text{end}} > 0$ can prevent the LR from decaying below a floor:

$$\eta_t = (\eta_0 - \eta_{\text{end}}) \cdot \left(1 - \frac{t}{T}\right)^p + \eta_{\text{end}}$$

Applications

Used in semantic segmentation (DeepLab), object detection, and some NLP tasks. The linear case ($p = 1$) is equivalent to linear warmup in reverse.

Trade-offs

Pros:

- Flexible shape via power p: $p = 1$ gives linear decay, $p = 2$ gives quadratic, $p = 0.5$ gives concave.
- Guaranteed to reach near-zero LR by step T, which can be set to the known training duration.
- Compatible with learning rate warmup (adds a linear ramp at the start).

Cons:

- Requires knowing total training steps T in advance.
- Early decay is relatively gentle for $p > 1$, then accelerates near the end—this can cause premature convergence.
- Less commonly used than cosine decay in modern practice.

17.6 Inverse Square Root Decay

Inverse Sqrt Decay

The learning rate decays as the inverse square root of the step:

$$\eta_t = \frac{\eta_0}{\sqrt{t}} \quad \text{or} \quad \eta_t = \frac{\eta_0}{\sqrt{t/T}} = \eta_0 \cdot \sqrt{\frac{T}{t}}$$

Applications

Popularized in some GAN training recipes. Occasionally used in reinforcement learning for value function optimization. Appears in SGD theory for achieving asymptotic optimality (rates like $O(1/\sqrt{t})$ are minimax-optimal for convex non-smooth optimization).

Trade-offs

Pros:

- Decay rate is precisely minimax-optimal for convex Lipschitz functions.
- Decays more slowly than $1/t$, maintaining moderate step sizes longer into training.

Cons:

- Still decays unboundedly, with no plateau or annealing behavior.
- Not commonly used in deep learning because adaptive optimizers effectively implement a per-parameter inverse-sqrt-like decay via $\mathbf{v}_t$ accumulation.

17.7 Cosine Annealing

Cosine Annealing (Loshchilov & Hutter

The learning rate follows a cosine curve from an initial maximum $\eta_{\max}$ down to a minimum $\eta_{\min}$ over T steps (typically one full training run):

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\left(\frac{t}{T}\pi\right)\right)$$

When $t = 0$, $\eta_0 = \eta_{\max}$; when $t = T$, $\eta_T = \eta_{\min}$. The decay is smooth and the slope naturally approaches zero near the end, allowing fine-grained convergence.

Behavioral properties

- **Early phase** ($t \ll T$): $\eta_t \approx \eta_{\max}$, nearly flat—full exploration.
- **Middle phase** ($t \approx T/2$): steepest decrease, rapid reduction.
- **Late phase** ($t \to T$): slope $\to 0$, very gentle convergence to $\eta_{\min}$.

Applications

The default LR schedule for modern deep learning. Used almost universally for training vision Transformers, CNNs (ResNet, EfficientNet), large language models (combined with warmup), diffusion models, and GANs. Standard in PyTorch as `CosineAnnealingLR`.

Trade-offs

Pros:

- Smooth, principled decay with no discontinuous jumps.
- Balanced early exploration (flat plateau) and late convergence (zero slope).
- Only two hyperparameters ($\eta_{\max}$, $\eta_{\min}$) plus T.
- Empirically outperforms step and exponential decay on most benchmarks (often by 0.5–2% accuracy).
- In the limit $t \to T$, the gradient of η approaches zero, giving stable fine-tuning.

Cons:

- Requires knowing total training steps T in advance; extending training requires either restarting or using a different schedule.
- Monotonically decreasing—cannot escape new local minima that develop mid-training.
- Fixed shape offers no data-adaptive feedback (unlike Reduce on Plateau).

17.8 Cosine Annealing with Warm Restarts (SGDR)

SGDR — Stochastic Gradient Descent with Warm Restarts

Cosine annealing is applied in cycles. After each cycle of T_i steps, the learning rate is reset to $\eta_{\max}$:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\left(\frac{T_{\text{cur}}}{T_i}\pi\right)\right)$$

where T_{cur} is the number of steps since the **last restart** (T_{cur} resets to 0 after each cycle), and T_i is the length of the i-th cycle. In the original paper, T_i can optionally double after each restart: $T_{i+1} = 2T_i$.

Cycle doubling variant

If T_0 is the initial cycle length and cycles double each restart:

$$T_i = T_0 \cdot 2^i, \qquad i = 0, 1, 2, \ldots$$

This yields longer fine-tuning phases as training progresses.

Applications

Training CNNs on CIFAR and ImageNet where it can reduce training time by a factor of 2–4 compared to standard cosine. Useful when the total number of epochs is unclear—restarts allow continuing training naturally. Used in some ViT training recipes.

Trade-offs

Pros:

- Multiple restarts allow the model to escape local minima that developed during a previous cycle. Each restart enables a new phase of exploration.
- Cycle doubling produces longer and longer fine-tuning phases, naturally allocating more steps to final convergence.
- Strong empirical results: competitive or better accuracy with fewer total epochs.

Cons:

- Additional hyperparameter T_0 (and optionally T_{mult}) to tune.
- Periodic jumps in LR can cause temporary loss increases ("loss spikes" visible in training curves).
- Less interpretable learning curves with oscillatory validation metrics.

17.9 Linear Warmup

Linear Warmup

The learning rate increases linearly from a small value (often 0 or $\eta_{\max}/\text{warmup_factor}$) to $\eta_{\max}$ over the first t_{warmup} steps:

$$\eta_t = \eta_{\max} \cdot \frac{t}{t_{\text{warmup}}}, \qquad \text{for } t \leq t_{\text{warmup}}$$

After warmup, the LR follows the chosen decay schedule (cosine, polynomial, step, etc.).

Applications

Essential for training Transformers and large models. Used in every major LLM training recipe (GPT, BERT, LLaMA, PaLM, Chinchilla), ViT, and most large-scale training runs. Prevents divergence caused by the initial large gradient variance before the optimizer accumulates sufficient gradient statistics (especially with Adam).

Trade-offs

Pros:

- Prevents training collapse in the first few hundred steps, which is a well-known failure mode for large Transformers.
- Allows using a higher peak LR than would otherwise be stable from step 0.
- Trivially simple to implement; only one extra hyperparameter (t_{warmup}).
- Compatible with any decay schedule by concatenation.

Cons:

- Adds t_{warmup} steps of "wasted" sub-optimal training.
- Warmup steps count toward total budget, shortening the effective training duration under a fixed step/epoch limit.
- No theoretical justification for why a ramp is needed (active research area; explanations involve Adam's variance estimator bias, large initial gradient magnitude, and layer normalization interactions).

17.10 Warmup + Cosine Decay (LLM Standard)

Warmup + Cosine Decay

The *de facto* standard for training large language models combines linear warmup with cosine decay:

$$\eta_t = \begin{cases} \eta_{\max} \cdot \dfrac{t}{t_{\text{warmup}}}, & t < t_{\text{warmup}} \\ \eta_{\min} + \dfrac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\dfrac{t - t_{\text{warmup}}}{T - t_{\text{warmup}}}\pi\right)\right), & t \geq t_{\text{warmup}} \end{cases}$$

where T is the total training steps and $t_{\text{warmup}} \ll T$ (typically 1–5% of total steps).

Typical values for LLM training

- $\eta_{\max} \in [10^{-4}, 3 \times 10^{-4}]$ (AdamW)
- $t_{\text{warmup}} \in [1000, 4000]$ steps
- $\eta_{\min} = 0$ or $\eta_{\min} = 10^{-5}$ (10% of $\eta_{\max}$)
- T: total steps determined by dataset size and batch size
- Often combined with weight decay ($\lambda = 0.1$) and gradient clipping

Applications

Used in GPT-3, PaLM, Chinchilla, LLaMA, LLaMA 2/3, Mistral, Falcon, BLOOM, OPT, Gemma, and virtually every transformer-based LLM trained since 2020. Also standard for ViT and large diffusion models.

Trade-offs

Pros:

- Combines stability of warmup with strong convergence properties of cosine annealing.
- Empirically validated at the largest scales (hundreds of billions of parameters).

- Predictable training dynamics.

Cons:

- Requires knowing T in advance (inflexible for open-ended training).
- Adds t_{warmup} to hyperparameter budget.
- For extremely long training runs, the cosine can spend the majority of steps at a very low LR, potentially underfitting.

17.11 Cyclical Learning Rates

17.11.1 Triangular Cyclical LR

Triangular CLR (Smith

The learning rate oscillates linearly between a lower bound $\eta_{\min}$ and an upper bound $\eta_{\max}$ with a fixed step size (half-cycle length):

$$\eta_t = \begin{cases} \eta_{\min} + (\eta_{\max} - \eta_{\min}) \cdot \dfrac{2 \cdot (t \bmod 2S)}{2S}, & (t \bmod 2S) < S \\ \eta_{\max} - (\eta_{\max} - \eta_{\min}) \cdot \dfrac{2 \cdot (t \bmod 2S - S)}{2S}, & \text{otherwise} \end{cases}$$

where S is the step size (half the period) and the full cycle period is $2S$. The first half-cycle ramps up; the second ramps down.

17.11.2 Triangular2

Triangular2 CLR

Same as the triangular schedule, but $\eta_{\max}$ is halved after each full cycle:

$$\eta_{\max}^{(c+1)} = \frac{1}{2}\,\eta_{\max}^{(c)}$$

where c indexes the cycle number. The lower bound $\eta_{\min}$ remains fixed.

17.11.3 Exponential Range CLR

Exponential Range CLR

The boundaries of the triangular cycle decay exponentially:

$$\eta_{\max}^{(t)} = \eta_{\max}^{(0)} \cdot \gamma_{\text{upper}}^{t}, \qquad \eta_{\min}^{(t)} = \eta_{\min}^{(0)} \cdot \gamma_{\text{lower}}^{t}$$

where $\gamma_{\text{upper}}, \gamma_{\text{lower}} \in (0, 1)$.

Applications

Smith (2017) demonstrated that cyclical LRs can train CNNs on CIFAR-10/100 and ImageNet in fewer iterations than fixed schedules. Useful when the optimal LR is unknown: the cycle naturally explores a range of learning rates.

Trade-offs (Cyclical LRs)

Pros:

- Eliminates the need to tune a single fixed LR; the network benefits from both high and low learning rates in each cycle.
- Can train in substantially fewer iterations (Smith reports 2–8× reduction).
- High-LR phases help escape saddle points and poor local minima.
- Easy to implement; can replace any fixed/decaying schedule.

Cons:

- Introduces more hyperparameters: $\eta_{\min}$, $\eta_{\max}$, step size S (or period).
- Loss curves are oscillatory, making convergence hard to monitor.
- Less tested at the very largest scales; mixed results for Transformers.
- Not all tasks benefit from periodic high-LR exploration.

17.12 1Cycle Policy (Super-Convergence)

1Cycle Policy (Smith & Topin

The 1Cycle policy consists of exactly one full cycle: a linear warmup from $\eta_{\min}$ to $\eta_{\max}$ over the first $\phi \cdot T$ steps, followed by a linear (or cosine) decay from $\eta_{\max}$ down to $\eta_{\min}/d$ over the remaining $(1-\phi) \cdot T$ steps:

$$\eta_t = \begin{cases} \eta_{\min} + (\eta_{\max} - \eta_{\min}) \cdot \dfrac{t}{\phi T}, & t \le \phi T \\ \eta_{\min}/d + (\eta_{\max} - \eta_{\min}/d) \cdot \left(1 - \dfrac{t - \phi T}{(1-\phi)T}\right), & t > \phi T \end{cases}$$

where $\phi \in (0, 1)$ is the fraction of training spent in the warmup phase (typically $\phi \approx 0.3$), and $d > 1$ determines how far below $\eta_{\min}$ the final LR falls. **Momentum also cycles inversely:** high momentum $(1-\beta)$ when the LR is low, and vice versa.

Applications

"Super-convergence": training CNNs on CIFAR-10/100, ImageNet, and other benchmarks in a fraction of the usual epochs (e.g., ResNet-56 on CIFAR-10 in 90 epochs with 1Cycle vs. 200+ epochs conventionally). Used in fastai library as the default training policy.

Trade-offs

Pros:

- **Dramatically faster training:** often 2–10× fewer epochs than standard schedules.
- Combines the benefits of warmup (stability) and aggressive LR range exploration in one concise schedule.
- Momentum co-scheduling amplifies the effect (high LR + low momentum for exploration; low LR + high momentum for convergence).
- Well-supported in fastai and PyTorch (`OneCycleLR`).

Cons:

- More hyperparameters than cosine: $\eta_{\max}$, $\eta_{\min}$, ϕ, d, and optionally momentum bounds.
- The aggressive schedule can cause divergence if $\eta_{\max}$ is set too high.

- Less established at LLM scales; primarily designed for CNNs and moderate-scale vision models.

17.13 Reduce Learning Rate on Plateau

ReduceLROnPlateau

Rather than following a predefined time-based schedule, ReduceLROnPlateau monitors a validation metric (e.g., validation loss) and decreases the learning rate when improvement stalls:

$$\eta \leftarrow \eta \cdot \gamma \quad \text{if} \quad \text{val_loss has not improved for } p \text{ epochs}$$

where $\gamma \in (0, 1)$ is the decay factor (common: 0.1–0.5) and p is the "patience" (number of epochs to wait before reducing). A minimum LR floor $\eta_{\min}$ and a cooldown period between reductions can also be specified.

Applications

Training CNNs on custom datasets where no well-established schedule exists. Fine-tuning pretrained models on new tasks. Scenarios where validation loss dynamics are unknown a priori. Common as a safety net alongside a fixed schedule.

Trade-offs

Pros:

- **Adaptive:** responds to actual training dynamics rather than a blind time-based decay.
- Reduces the LR only when needed; if the model is still improving, the LR stays high.
- Requires no knowledge of total training steps T.
- Simple and intuitive; standard in PyTorch (`ReduceLROnPlateau`).

Cons:

- Introduces **label noise sensitivity:** the validation metric is used to make scheduling decisions, which can overfit to validation noise.
- Late-stage training: by the time the plateau is detected, many epochs of stagnation have already occurred.
- Adds hyperparameters: patience p, factor γ, threshold, cooldown.
- Step-wise reductions (not smooth) still cause minor loss spikes.
- Incompatible with epoch-free training where validation is not evaluated per-epoch.

17.14 Noam Schedule (Transformer)

Noam Schedule (Vaswani et al.

Introduced in the original Transformer paper, the Noam schedule combines linear warmup with inverse square root decay in a single formula:

$$\eta_t = d_{\text{model}}^{-0.5} \cdot \min\left(t^{-0.5},\ t \cdot \text{warmup_steps}^{-1.5}\right)$$

where d_{model} is the model dimension (e.g., 512 for the original Transformer) and warmup_steps is the number of warmup steps (typically 4000 in the original paper).

Piecewise interpretation

- **Warmup phase** ($t \leq$ warmup_steps): LR increases linearly: $\eta_t \propto$ step_num $\cdot$ warmup_steps$^{-1.5}$.
- **Decay phase** ($t >$ warmup_steps): LR decays as $\propto t^{-0.5}$, the inverse square root of the step number.

The scaling by $d_{\text{model}}^{-0.5}$ ensures that wider models use a smaller learning rate, compensating for the larger dot-product magnitudes in attention.

Applications

The original Transformer for machine translation (WMT). Early NLP Transformer models (pre-BERT). Still relevant for small-to-medium Transformer training where simplicity is preferred. Less common for LLMs since Warmup+Cosine became dominant.

Trade-offs

Pros:

- Single formula covering both warmup and decay.
- Model-dimension-aware scaling prevents large-model instability.
- No need to specify total training steps T.
- Devised specifically for the Transformer architecture.

Cons:

- The inverse-sqrt decay is more aggressive than cosine; the LR drops quickly and stays low for most of training.
- Limited flexibility: only warmup_steps is tunable beyond the architectural d_{model}.
- Empirically outperformed by warmup+cosine on most modern large-scale benchmarks.
- The $d_{\text{model}}^{-0.5}$ scaling is heuristic; μP provides a more principled scaling framework.

17.15 GPT and Chinchilla Schedules

17.15.1 GPT Schedule

GPT Schedule

Used in GPT-1/2/3 training (Radford et al., 2018–2020):

$$\eta_t = \eta_{\max} \cdot \min\left(t^{-0.5},\ t \cdot \text{warmup_steps}^{-1.5}\right)$$

This is essentially the Noam schedule without the $d_{\text{model}}^{-0.5}$ factor and with added flexibility for $\eta_{\max}$. The warmup is linear, and the decay follows $t^{-0.5}$ after warmup. Some GPT-3 variants used cosine decay to zero instead.

17.15.2 Chinchilla Schedule

Chinchilla Schedule (Hoffmann et al.

The Chinchilla paper used a linear warmup followed by cosine decay:

$$\eta_t = \begin{cases} \eta_{\max} \cdot \dfrac{t}{t_{\text{warmup}}}, & t \le t_{\text{warmup}} \\ \dfrac{\eta_{\max}}{2}\left(1 + \cos\left(\dfrac{t - t_{\text{warmup}}}{T - t_{\text{warmup}}}\pi\right)\right), & t > t_{\text{warmup}} \end{cases}$$

with $\eta_{\min} = 0$. This is a specific parameterization of warmup+cosine. The Chinchilla paper used $\eta_{\max} = 10^{-3}$ with AdamW and trained for T steps chosen by the Chinchilla scaling laws.

Applications

GPT schedule: GPT-1, GPT-2, and (some variants of) GPT-3. Chinchilla: the compute-optimal Chinchilla models, and influenced most subsequent open LLM training recipes.

Trade-offs

Pros:

- Both are validated at massive scale (hundreds of billions of tokens).
- GPT/Noam variant: no need to specify T, suitable for open-ended training.
- Chinchilla (warmup+cosine): best final perplexity among schedules compared in the scaling laws paper.

Cons:

- GPT schedule: inverse-sqrt decay can be too fast, under-utilizing compute.
- Chinchilla schedule: requires knowing T in advance, which is partly circular when T is determined by scaling laws.

17.16 Hypergradient Descent (Learning to Learn the LR)

Hypergradient Descent (Baydin et al.

The learning rate itself is optimized via gradient descent on the *hypergradient*—the gradient of the training loss with respect to the learning rate:

$$\eta_t = \eta_{t-1} - \beta \frac{\partial \mathcal{L}(\mathbf{w}_t)}{\partial \eta_{t-1}}$$

where $\mathbf{w}_t = \mathbf{w}_{t-1} - \eta_{t-1}\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}_{t-1})$. By the chain rule:

$$\frac{\partial \mathcal{L}(\mathbf{w}_t)}{\partial \eta_{t-1}} = \nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}_t)^\top \frac{\partial \mathbf{w}_t}{\partial \eta_{t-1}} = -\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}_t)^\top \nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}_{t-1})$$

which is simply the negative dot product of consecutive gradients. β is the **hyper-learning rate** (learning rate for the learning rate). A variant uses the online update:

$$\eta_t = \eta_{t-1} \cdot \exp\left(\beta \nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}_t)^\top \nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}_{t-1})\right)$$

Applications

Automatic LR tuning for SGD and Adam. Potentially useful for online and continual learning where manual schedule design is impractical. Research tool for understanding LR dynamics.

Trade-offs

Pros:

- Fully automated: no schedule design, no η_0 tuning beyond a rough initial value.
- Adapts online to changing loss landscape curvature.
- The update rule is computationally cheap (a dot product per step).
- Can be combined with existing optimizers (SGD-HD, Adam-HD).

Cons:

- Introduces a new hyperparameter β (the hyper-LR), replacing the schedule design problem with a hyper-LR tuning problem.
- Noisy gradient dot products can cause unstable LR updates, especially with small batch sizes.
- Less tested at scale; primarily a research concept.
- Can drive the LR to zero prematurely if gradients consistently align, causing underfitting.

17.17 SWA Learning Rate Scheduling

SWA LR Schedule

Stochastic Weight Averaging (SWA) uses a specific scheduling strategy: first train with a standard decaying schedule (cosine, step, etc.) until near convergence, then **switch to a constant or cyclical high learning rate** while collecting model checkpoints for averaging:

$$\mathbf{w}_{\text{SWA}} = \frac{1}{M} \sum_{m=1}^{M} \mathbf{w}_m$$

where $\mathbf{w}_m$ are checkpoints collected during the SWA phase. The SWA LR is typically set to a constant moderate-to-high value (e.g., 0.05 for CIFAR with SGD) or follows a cyclical schedule (cosine annealing with one restart). The key insight: a higher LR near the end of training explores the loss basin to find flatter, broader minima that average to better generalization.

Applications

Improving final test accuracy of CNNs on ImageNet and CIFAR. Semantic segmentation, object detection, and domain generalization. Available in PyTorch (`torch.optim.swa_utils`).

Trade-offs

Pros:

- Discovers flatter minima that generalize better; often provides a "free" 0.5–1% accuracy improvement.
- Compatible with any optimizer; simple to add after standard training.

- The averaging itself is cheap (no extra forward/backward passes).

Cons:

- Requires extra training epochs during the SWA collection phase.
- Models with batch normalization need a separate forward pass to recompute BN statistics for the averaged weights.
- Adds complexity: two-phase training with a schedule change point.

17.18 Summary Comparison Table

Table 17.1: Comprehensive Comparison of Learning Rate Schedules

Schedule	Hyperparams	Adaptive?	Needs T?	Shape	Best For
Constant	1	No	No	Flat	Baselines, Adam
Step Decay	3	No	No	Staircase	CNNs + SGD (classic)
Exponential	2	No	No	Smooth decay	Fine-tuning
Time-Based	2	No	No	$1/t$	Convex SGD theory
Polynomial	4	No	Yes	$\propto (1-t/T)^p$	Segmentation
Inverse Sqrt	1	No	No	$1/\sqrt{t}$	GANs, some RL
Cosine	3	No	Yes	Cosine curve	**Default modern DL**
SGDR	5+	No	Yes (cycle)	Cosine + resets	Fast training, escape minima
Linear Warmup	1+	No	No	Ramp	**Transformer stability**
Warmup+Cosine	4	No	Yes	Ramp + cosine	**LLM standard**
Cyclical LR	4	No	No	Triangular waves	Fast training, unknown LR
1Cycle	5	No	Yes (fraction)	1 triangle + decay	Super-convergence (CNNs)
ReduceOnPlateau	3+	**Yes**	No	Step drops (adaptive)	Unknown datasets
Noam	2	No	No	Warmup + $1/\sqrt{t}$	Original Transformer
GPT	2	No	No	Warmup + $1/\sqrt{t}$	GPT-1/2/3
Chinchilla	4	No	Yes	Warmup + cosine	Compute-optimal LLMs
Hypergradient	1+	**Yes**	No	Online gradient	Auto-tuning, research
SWA LR	3	No	Partial	Constant/cyclic (late)	Final accuracy boost

Selection Guide

Decision flowchart:

1. **Training an LLM / large Transformer?** → Warmup + Cosine Decay (de facto standard).
2. **Training a CNN on ImageNet?** → Cosine Annealing (preferred) or Step Decay (if following a known recipe).
3. **Want fastest possible CNN training?** → 1Cycle Policy (super-convergence).
4. **Don't know the total number of steps?** → ReduceLROnPlateau or Exponential Decay.
5. **Can't choose a single LR?** → Cyclical LR (cover a range).
6. **Dataset size unknown / online learning?** → Hypergradient Descent (experimental) or constant LR.
7. **Adding a "free" accuracy boost?** → Add SWA with a constant/cyclical LR

phase after standard training.

8. **If uncertain:** Warmup (5% of steps) + Cosine Annealing with $\eta_{\min} = 0$ works for 80%+ of cases.

Chapter 18

Sampling Strategies

Sampling lies at the heart of statistical machine learning—from drawing mini-batches for stochastic gradient descent to approximating intractable expectations in Bayesian inference. The choice of sampling strategy profoundly affects both the efficiency and quality of learning. This chapter surveys sampling techniques across basic statistics, Monte Carlo methods, generative modeling, and language model decoding, each presented with its mathematical definition, practical usage, and trade-offs.

18.1 Basic Sampling

18.1.1 Uniform Random Sampling

Uniform Sampling Without Replacement

Draw k items from a population of N such that each subset of size k has equal probability $1/\binom{N}{k}$:

$$P(\{i_1, \ldots, i_k\}) = \frac{1}{\binom{N}{k}}$$

Sequentially: at step j, each remaining item has probability $1/(N-j+1)$ of being selected. Implemented by a Fisher–Yates partial shuffle or `torch.randperm(N)[:k]`.

Uniform Sampling With Replacement

Each draw is independent and uniformly distributed over the N items. The probability of selecting any specific sequence $(i_1, \ldots, i_k)$ is $1/N^k$. The expected number of unique items in k draws is $N(1-(1-1/N)^k)$.

Applications

Mini-batch construction in SGD (without replacement per epoch, reshuffled), bootstrap resampling, Monte Carlo integration, and A/B test assignment. Without replacement is the default for training loops; with replacement is used for bootstrap and bagging.

Trade-offs

Pros: Simple, unbiased, requires no auxiliary data structures. **Cons:** No guarantee of representativeness; high variance for small k; can miss rare classes entirely in imbalanced datasets.

18.1.2 Stratified Sampling

Stratified Sampling

Divide the population into L mutually exclusive and exhaustive strata $\{S_1, \ldots, S_L\}$. Sample n_h items from stratum S_h, often proportional to stratum size (proportional allocation):

$$n_h = k \cdot \frac{|S_h|}{N}$$

yielding an estimator $\hat{\mu}_{\text{strat}} = \sum_h \frac{|S_h|}{N} \bar{y}_h$ with variance:

$$\text{Var}(\hat{\mu}_{\text{strat}}) = \sum_{h=1}^{L} \left(\frac{|S_h|}{N}\right)^2 \frac{\text{Var}(y \mid h)}{n_h} \cdot \left(1 - \frac{n_h - 1}{|S_h| - 1}\right)$$

where the finite-population correction factor applies for without-replacement sampling.

Applications

Survey sampling, ensuring class balance in mini-batches, Monte Carlo variance reduction (stratified Monte Carlo), and evaluating models on demographic subgroups. In ML: `StratifiedKFold` for cross-validation.

Trade-offs

Pros: Guarantees representation from every stratum; reduces estimator variance compared to simple random sampling when strata are internally homogeneous. **Cons:** Requires prior knowledge of stratum membership; uniform allocation can be suboptimal—Neyman allocation $n_h \propto |S_h|\sigma_h$ is optimal but needs variance estimates.

18.1.3 Reservoir Sampling

Reservoir Sampling (Algorithm R)

Maintain a reservoir of k items from a stream of unknown length N. For each incoming element i (1-indexed):

1. If $i \leq k$: place element in reservoir.
2. If $i > k$: with probability k/i, replace a uniformly chosen element in the reservoir.

When the stream ends, every element has probability k/N of being in the reservoir, and every subset of size k is equally likely.

Applications

Streaming data when total size is unknown a priori: online mini-batch selection from data lakes, uniform sample from Kafka/event streams, reservoir replay buffer in continual RL, and random subgraph selection from streaming graph edges.

Trade-offs

Pros: Single pass, $O(1)$ per element, memory bounded by k, equal probability guarantee regardless of stream length. **Cons:** Requires $O(k)$ random numbers per insertion (can be reduced with skip-based variants); order-dependent for early elements; weighted variants

(Chao's method) increase complexity.

18.2 Importance Sampling

18.2.1 Standard Importance Sampling

Importance Sampling

Given a target distribution $p(x)$ and a proposal distribution $q(x)$ with $\text{supp}(p) \subseteq \text{supp}(q)$, the expectation under p can be estimated using samples from q:

$$\mathbb{E}_p[f(x)] = \mathbb{E}_q\left[\frac{p(x)}{q(x)} f(x)\right] \approx \frac{1}{N}\sum_{i=1}^{N} \frac{p(x_i)}{q(x_i)} f(x_i), \quad x_i \sim q(x)$$

where $w_i = p(x_i)/q(x_i)$ are the importance weights. The estimator is unbiased but can have unbounded variance if q has lighter tails than p.

Applications

Off-policy evaluation in RL (estimate value under target policy using data from behavior policy), posterior inference with proposal distribution, and variance reduction in rare-event simulation.

Trade-offs

Pros: Unbiased; reuses samples from any proposal; no need to sample from p directly. **Cons:** Variance explodes if q is a poor match for p (diagnostic: effective sample size $N_{\text{eff}} = (\sum w_i)^2 / \sum w_i^2$). Large weights dominate, causing high variance.

18.2.2 Weight-Normalized (Self-Normalized) Importance Sampling

Weight-Normalized IS

Normalizing the weights by their sum yields the self-normalized estimator:

$$\mathbb{E}_p[f(x)] \approx \frac{\sum_{i=1}^{N} w_i f(x_i)}{\sum_{i=1}^{N} w_i}, \quad w_i = \frac{p(x_i)}{q(x_i)}$$

This is the ratio of two standard IS estimates ($\mathbb{E}[\sum w_i] = \mathbb{E}[\sum w_i f(x_i)] = 1$ and $\mathbb{E}_p[f]$ respectively).

Applications

When only unnormalized target densities $\tilde{p}(x)$ are known (e.g., Bayesian posteriors up to a normalizing constant). Particle filters (sequential Monte Carlo) use weight-normalized IS at each time step with resampling to avoid weight degeneracy.

Trade-offs

Pros: Requires only unnormalized densities; always produces a convex combination (bounded output). **Cons:** Biased (bias $\propto 1/N$); slightly higher MSE than standard IS when the normalizing constant is known; weight degeneracy still occurs when proposal is poor.

18.2.3 Prioritized Experience Replay (PER)

Prioritized Experience Replay

In DQN-based RL, transitions are sampled with priority proportional to TD error magnitude. The sampling probability for transition i is:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}, \quad p_i = |\delta_i| + \epsilon$$

where δ_i is the TD error, $\epsilon > 0$ prevents zero probability, and $\alpha \in [0, 1]$ controls the degree of prioritization ($\alpha = 0$ recovers uniform sampling). To correct the sampling bias, importance-sampling weights are applied in the loss:

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)}\right)^\beta$$

where β anneals from β_0 (often ≈ 0.4) to 1 over training. When $\beta = 1$, the updates are fully unbiased.

Applications

Standard component of deep Q-learning (DQN, Rainbow, DDPG, SAC). Particularly effective in sparse-reward settings where informative transitions are rare. The Rainbow agent combines PER with double DQN, dueling networks, noisy nets, distributional RL, and n-step returns.

Trade-offs

Pros: Accelerates learning 2–10x on Atari benchmarks by focusing on surprising transitions; reduces the number of interactions required. **Cons:** Introduces bias that must be corrected; sensitive to α, β hyperparameters; sum-tree data structure adds $O(\log N)$ per update (vs. $O(1)$ for uniform replay); can overfit to noisy high-error transitions early in training.

18.2.4 Annealed Importance Sampling (AIS)

Annealed Importance Sampling

AIS constructs a sequence of T intermediate distributions bridging a tractable proposal q to the target p:

$$p_t(x) \propto q(x)^{1-\beta_t} \, p(x)^{\beta_t}, \quad 0 = \beta_0 < \beta_1 < \cdots < \beta_T = 1$$

Starting from $x_0 \sim q = p_0$, each x_t is obtained by a transition $T_t(x_t \mid x_{t-1})$ that leaves p_t invariant (e.g., a Metropolis–Hastings or HMC step). The importance weight is:

$$w = \prod_{t=1}^{T} \frac{p_t(x_{t-1})}{p_{t-1}(x_{t-1})}$$

Multiple independent chains produce N weighted samples, which are then used in a self-normalized IS estimator.

Applications

Estimating the log marginal likelihood (model evidence) for Bayesian model selection, evaluating generative models, and providing ground-truth log-likelihood estimates for VAEs and normalizing flows when exact computation is intractable.

Trade-offs

Pros: Produces asymptotically exact log-likelihood estimates; more accurate than single-step IS for complex, multi-modal posteriors. **Cons:** Computationally expensive (T transitions per sample); requires careful annealing schedule design; chain variance can be large if T is too small.

18.3 Hard Example Mining

18.3.1 Hard and Semi-Hard Negative Mining

Triplet Mining Conditions

For a triplet (a, p, n) with anchor a, positive p, and negative n, and a margin $m > 0$, define the pairwise distances $d(a, p)$ and $d(a, n)$:
Hard negative: $d(a, n) < d(a, p)$, i.e., the negative is closer to the anchor than the positive (violates the margin easily).
Semi-hard negative: $d(a, p) < d(a, n) < d(a, p) + m$, i.e., the negative lies inside the margin band but is still farther than the positive. These are informative because they contribute to the loss without already violating the constraint trivially.
Easy negative: $d(a, n) > d(a, p) + m$, contributes zero loss.

Applications

Face recognition (FaceNet, ArcFace), person re-identification, metric learning, and image retrieval. Typically, semi-hard mining provides the best convergence; hard negatives can cause collapsed representations.

Trade-offs

Pros: Focuses training on informative examples; semi-hard mining yields stable gradients and faster convergence than random triplets. **Cons:** Requires computing pairwise distances over the batch or dataset; hard negatives may be noisy or mislabeled; offline mining requires periodic re-evaluation of the entire dataset.

18.3.2 Batch-Hard Triplet Mining

Batch-Hard Mining

Given a mini-batch of B examples with C classes and K samples per class ($B = C \cdot K$), for each anchor a:

$$n_{\text{hard}} = \arg \min_{n\,:\,y_n \neq y_a} d(a, n) \quad \text{(hardest negative in batch)}$$

$$p_{\text{hard}} = \arg \max_{p\,:\,y_p = y_a,\, p \neq a} d(a, p) \quad \text{(hardest positive in batch)}$$

The loss uses only these hardest pairs, discarding easy triplets.

Applications

Deep metric learning with PK sampling (sample P classes, K images each). Used in person re-identification (AlignedReID, MGN) and fine-grained visual recognition where intra-class variation is high.

Trade-offs

Pros: Maximizes gradient informativeness per batch; no offline mining needed; scales to large datasets. **Cons:** Sensitive to outliers/noisy labels (the hardest positive may be mislabeled); collapses if $K = 1$; batch must be carefully constructed with class-balanced PK sampling.

18.3.3 Online Hard Example Mining (OHEM)

OHEM

In two-stage object detectors (Fast/Faster R-CNN), OHEM selects the top-B RoIs (Regions of Interest) with the highest per-RoI loss from the current forward pass, and only backpropagates through those:

$$\mathcal{L}_{\text{OHEM}} = \frac{1}{B} \sum_{i \in \text{top-}B} \mathcal{L}_{\text{cls}}(r_i) + \mathcal{L}_{\text{reg}}(r_i)$$

The remaining RoIs contribute zero gradient. The selection is performed online using the current model state, naturally focusing on the most difficult regions.

Applications

Object detection (Faster R-CNN training), instance segmentation. Alternative to the standard random mini-batch sampling of RoIs that often overwhelms the loss with easy background proposals.

Trade-offs

Pros: Automatic focus on informative (hard) examples; improves detection AP by 1–2 points over random RoI sampling. **Cons:** Requires two forward passes (first for loss computation, second for selected RoIs); amplifies gradients of potentially noisy/mislabeled boxes; less compatible with modern single-stage detectors.

18.3.4 Focal Loss as Soft Mining

Focal Loss as Implicit Hard Mining

Focal loss (Chapter 1) down-weights easy examples rather than excluding them entirely:

$$\mathcal{L}_{\text{FL}}(p_t) = -\alpha_t(1-p_t)^{\gamma}\log p_t$$

where p_t is the model's estimated probability for the true class. When $\gamma > 0$, well-classified examples ($p_t \approx 1$) have negligible loss, while hard examples ($p_t \ll 1$) retain full weight. This "soft" hard mining avoids the need for explicit RoI selection and is the mechanism behind RetinaNet.

Applications

Dense object detection (RetinaNet), class-imbalanced classification, and any setting where easy negatives dominate the loss.

Trade-offs

Pros: End-to-end trainable; no extra forward pass; naturally handles extreme foreground-background imbalance; works with single-stage detectors. **Cons:** Hyperparameters α, γ require tuning; less interpretable than explicit RoI selection.

18.4 MCMC Methods

18.4.1 Metropolis–Hastings

Metropolis–Hastings Algorithm

To sample from a target distribution $p(x)$, MH proposes a new state x' from a proposal distribution $q(x' \mid x)$ and accepts it with probability:

$$\alpha(x \to x') = \min\left(1,\ \frac{p(x')\,q(x \mid x')}{p(x)\,q(x' \mid x)}\right)$$

If rejected, the chain stays at x. This acceptance ratio ensures detailed balance: $p(x)q(x'|x)\alpha(x \to x') = p(x')q(x|x')\alpha(x' \to x)$, guaranteeing that p is the stationary distribution of the Markov chain. For symmetric proposals ($q(x'|x) = q(x|x')$), this reduces to $\alpha = \min(1, p(x')/p(x))$, the original Metropolis criterion.

Applications

Bayesian posterior inference, phylogenetic tree reconstruction, protein structure prediction, and general-purpose sampling when only an unnormalized density $p(x) \propto e^{-U(x)}$ is available.

Trade-offs

Pros: Asymptotically exact; works with unnormalized densities; simple to implement. **Cons:** Requires tuning proposal step size (too small → slow mixing, too large → high rejection); correlated samples reduce effective sample size; struggles with multi-modal

distributions; burn-in period cannot be eliminated.

18.4.2 Gibbs Sampling

Gibbs Sampling

Sample each coordinate x_i from its full conditional distribution given all other coordinates:

$$x_i^{(t+1)} \sim p(x_i \mid x_1^{(t+1)}, \ldots, x_{i-1}^{(t+1)}, x_{i+1}^{(t)}, \ldots, x_d^{(t)})$$

Each draw is always accepted (acceptance probability = 1). The proposal is the exact conditional, making it a special case of MH where the acceptance ratio is identically 1.

Applications

Latent Dirichlet Allocation (LDA) topic models, Bayesian mixture models with conjugate priors, missing data imputation, and Ising models in statistical physics. Particularly effective when conditionals are from standard families (Gaussian, Gamma, Dirichlet).

Trade-offs

Pros: No rejection; no step-size tuning; closed-form updates for conjugate models. **Cons:** Requires tractable full conditionals; slow mixing when coordinates are strongly correlated (the sampler "crawls" along the support); the sequential nature limits GPU parallelization; random-scan or collapsed Gibbs can partially mitigate but not eliminate these issues.

18.4.3 Hamiltonian Monte Carlo (HMC)

HMC Dynamics

HMC augments the position variable q (parameters) with a momentum variable p, defining the Hamiltonian:

$$H(q,p) = U(q) + K(p) = -\log p(q) + \frac{1}{2} p^\top M^{-1} p$$

where $U(q) = -\log p(q)$ is the potential energy (negative log posterior) and $K(p)$ is the kinetic energy with mass matrix M (often $M = \mathbb{I}$). Hamilton's equations of motion:

$$\frac{dq}{dt} = \frac{\partial H}{\partial p} = M^{-1} p, \qquad \frac{dp}{dt} = -\frac{\partial H}{\partial q} = \nabla_q \log p(q)$$

The leapfrog integrator discretizes these equations with step size ϵ over L steps:

$$\begin{aligned} p(t+\epsilon/2) &= p(t) + \frac{\epsilon}{2} \nabla_q \log p(q(t)) \\ q(t+\epsilon) &= q(t) + \epsilon\, M^{-1} p(t+\epsilon/2) \\ p(t+\epsilon) &= p(t+\epsilon/2) + \frac{\epsilon}{2} \nabla_q \log p(q(t+\epsilon)) \end{aligned}$$

The final state (q^*, p^*) is accepted with Metropolis probability $\alpha = \min(1, \exp(H(q,p) - H(q^*,p^*)))$.

Applications

Gold-standard posterior inference in Bayesian statistics (Stan, PyMC). Training Bayesian neural networks with full posterior over weights. Sampling from high-dimensional, correlated distributions where random-walk Metropolis fails.

Trade-offs

Pros: Suppresses random-walk behavior via Hamiltonian dynamics; efficient exploration of high-dimensional target; gradient-informed proposals. **Cons:** Requires gradient of $\log p(q)$ (double backprop for NN-based models); sensitive to ϵ and L (too large $\to$ instability, too small $\to$ wasted computation); the leapfrog integrator adds $2L$ gradient evaluations per sample.

18.4.4 NUTS (No-U-Turn Sampler)

NUTS

NUTS eliminates the need to hand-tune the number of leapfrog steps L by adaptively determining when the trajectory has "turned around." It builds a binary tree of leapfrog steps forward and backward in time until a U-turn criterion is met:

$$\frac{(q^+ - q^-)^\mathsf{T} p^+}{\|q^+ - q^-\|\|p^+\|} < \text{threshold} \quad \text{or} \quad \frac{(q^+ - q^-)^\mathsf{T} p^-}{\|q^+ - q^-\|\|p^-\|} < \text{threshold}$$

where (q^+, p^+) and (q^-, p^-) are the extremes of the trajectory. The final state is sampled via multinomial sampling over the tree states (slice sampling criterion). The step size ϵ is adaptively tuned via dual averaging during warm-up.

Applications

Default sampler in Stan since version 2.8. Used for Bayesian hierarchical models, generalized linear mixed models, and any Bayesian model where manual tuning of HMC hyperparameters is undesirable.

Trade-offs

Pros: Fully adaptive—only requires specifying the target distribution; excellent effective sample size per gradient evaluation; no manual tuning of L. **Cons:** Tree-building can explore deep trajectories in challenging geometries; not trivially parallelizable; step-size adaptation still requires a warm-up phase.

18.4.5 Stochastic Gradient Langevin Dynamics (SGLD)

SGLD

SGLD approximates Bayesian posterior sampling via mini-batch stochastic gradients with injected Gaussian noise:

$$\Delta\theta_t = \frac{\epsilon_t}{2}\left(\nabla_\theta \log p(\theta) + \frac{N}{n}\sum_{i \in B_t} \nabla_\theta \log p(x_i \mid \theta)\right) + \eta_t, \quad \eta_t \sim \mathcal{N}(0, \epsilon_t \mathbb{I})$$

where N is the dataset size, $n = |B_t|$ is the mini-batch size, and ϵ_t is the step size. As $\epsilon_t \to 0$ (Robbins–Monro schedule), the dynamics converge to samples from $p(\theta \mid \mathcal{D})$.

Applications

Scalable posterior sampling for deep Bayesian neural networks, where full MCMC over millions of parameters is infeasible. Can be run with constant small ϵ as approximate posterior sampling.

Trade-offs

Pros: Scales to large datasets via mini-batching; asymptotically exact; conceptually simple (add noise to SGD). **Cons:** Requires decaying step size schedule; correlated samples; the noise scale must be tuned; constant-ϵ SGLD is biased; slower than VI for prediction-focused tasks.

18.5 Variational Samplers

18.5.1 Reparameterization Trick

Reparameterization Trick

Express a random variable $z \sim q_\phi(z)$ as a deterministic transformation of a base noise variable $\epsilon \sim p(\epsilon)$ independent of ϕ:

$$z = g(\epsilon, \phi), \quad \epsilon \sim p(\epsilon)$$

The gradient of an expectation then moves inside:

$$\nabla_\phi \mathbb{E}_{q_\phi(z)}[f(z)] = \nabla_\phi \mathbb{E}_{p(\epsilon)}[f(g(\epsilon, \phi))] = \mathbb{E}_{p(\epsilon)}\big[\nabla_z f(z) \cdot \nabla_\phi g(\epsilon, \phi)\big]$$

Common Reparameterizations

- **Gaussian:** $z = \mu + \sigma \odot \epsilon,\ \epsilon \sim \mathcal{N}(0, \mathbb{I})$
- **Gumbel-Softmax (relaxed Categorical):** $z_k = \frac{\exp((\log \pi_k + g_k)/\tau)}{\sum_j \exp((\log \pi_j + g_j)/\tau)}$, $g_k \sim$ Gumbel$(0, 1)$ (see Chapter **??**)
- **Inverse CDF:** For a continuous distribution with CDF F_ϕ, $z = F_\phi^{-1}(u)$ with $u \sim U(0, 1)$. Example: Exponential $z = -\frac{1}{\lambda}\log(1 - u)$.
- **Log-Normal:** $z = \exp(\mu + \sigma\epsilon),\ \epsilon \sim \mathcal{N}(0, 1)$

Applications

Variational Autoencoders (VAEs), where the reparameterization trick enables end-to-end training via backpropagation. Bayesian neural networks (Bayes by Backprop). Normalizing flows.

Trade-offs

Pros: Low-variance gradient estimates (lower than REINFORCE); computationally efficient—adds only a single noise draw per sample. **Cons:** Requires the distribution

to be reparameterizable (continuous); discrete distributions need continuous relaxations (Gumbel-Softmax); not applicable to arbitrary variational families.

18.5.2 REINFORCE / Score Function Estimator

REINFORCE Gradient Estimator

For distributions without a tractable reparameterization, the score function (log-derivative trick) gives:

$$\nabla_\phi \mathbb{E}_{q_\phi(z)}[f(z)] = \mathbb{E}_{q_\phi(z)}\big[f(z)\,\nabla_\phi \log q_\phi(z)\big]$$

using $\nabla_\phi q_\phi = q_\phi \nabla_\phi \log q_\phi$. The estimator is unbiased but typically has high variance.

Baseline Subtraction for Variance Reduction

To reduce variance, subtract a baseline b independent of z:

$$\nabla_\phi \mathbb{E}_{q_\phi(z)}[f(z)] = \mathbb{E}_{q_\phi(z)}\big[(f(z) - b)\,\nabla_\phi \log q_\phi(z)\big]$$

The optimal baseline minimizes Var and is given by:

$$b^* = \frac{\mathbb{E}_q[f(z)\|\nabla_\phi \log q\|^2]}{\mathbb{E}_q[\|\nabla_\phi \log q\|^2]}$$

In practice, a learned baseline (neural network regressing $f(z)$ from ϕ or context) is used.

Applications

Policy gradient RL (REINFORCE algorithm with baseline), discrete latent variable models (NVIL for VIMCO), and training stochastic computation graphs with non-differentiable nodes.

Trade-offs

Pros: Works for any distribution (discrete, continuous, reparameterizable or not); unbiased; model-agnostic. **Cons:** High variance, especially in high dimensions; requires careful baseline design; converges slower than reparameterization-based methods when both are applicable.

18.5.3 Straight-Through Estimator (STE) and Control Variates

Straight-Through Estimator (STE)

For discrete stochastic nodes, STE uses the discrete (hard) decision in the forward pass but passes the gradient of a continuous relaxation in the backward pass:

$$\text{Forward: } z = \arg\max_k(\pi_k), \qquad \text{Backward: } \frac{\partial \mathcal{L}}{\partial \pi_k} = \frac{\partial \mathcal{L}}{\partial z_k}$$

This is equivalent to assuming the Jacobian of the discrete operation is the identity. It is a biased but empirically effective estimator.

Applications

Binarized neural networks, VQ-VAE codebook quantization, Gumbel-Softmax with hard (one-hot) sampling, and training discrete communication channels in multi-agent RL.

Trade-offs

Pros: Simple implementation (identity backward hook); enables end-to-end training of discrete bottlenecks. **Cons:** Theoretically unjustified—bias is unquantified; gradient signal may not correspond to the true discrete optimization landscape; can lead to poor convergence if the relaxation is far from the discrete function.

18.5.4 REBAR and RELAX

REBAR and RELAX

REBAR (REINFORCE + Concrete + REBAR): Combines the REINFORCE estimator with a Gumbel-Softmax (Concrete) relaxation to form a lower-variance control variate for discrete latent variables. It uses a reparameterized relaxation as a baseline, leveraging both the unbiased score function and the low-variance reparameterization gradient.
RELAX: Generalizes REBAR by replacing the concrete relaxation with a learned, model-specific control variate $c_\psi(z)$:

$$\nabla_\phi \mathbb{E}_q[f] \approx \mathbb{E}_q[(f(z) - c_\psi(z))\nabla_\phi \log q] + \mathbb{E}_p[\nabla_\phi c_\psi(g(\epsilon, \phi))]$$

where c_ψ is trained to minimize the gradient variance jointly with the model parameters.

Applications

Training discrete latent variable models (e.g., discrete VAEs) where REINFORCE alone has prohibitively high variance and Gumbel-Softmax relaxations are biased. RELAX is particularly useful when no natural continuous relaxation exists for the discrete structure.

Trade-offs

Pros: REBAR provides unbiased, lower-variance gradients for discrete latents; RELAX removes the need for a hand-designed relaxation. **Cons:** Increased complexity—RELAX requires training a separate control variate network; can be outperformed by simpler methods (Gumbel-Softmax) in practice when bias is acceptable.

18.6 Generative Model Sampling

18.6.1 Ancestral / Autoregressive Sampling

Ancestral Sampling

For a directed graphical model $p(x_1, \ldots, x_d) = \prod_{i=1}^{d} p(x_i \mid x_{<i})$, sample sequentially:

$$x_1 \sim p(x_1),\ x_2 \sim p(x_2 \mid x_1),\ \ldots,\ x_d \sim p(x_d \mid x_1, \ldots, x_{d-1})$$

Each draw conditions on all previously sampled variables. For autoregressive models (PixelCNN, WaveNet, GPT), $p(x_i \mid x_{<i})$ is parameterized by a neural network with causal

masking.

Applications

Image generation (PixelCNN, VQ-VAE with PixelCNN prior), raw audio (WaveNet), language modeling (GPT autoregressive decoding), and probabilistic graphical models with directed edges.

Trade-offs

Pros: Produces exact samples from the modeled joint; no approximation or Markov chain needed; naturally handles conditional generation (initial tokens given). **Cons:** Sequential—d forward passes to generate d variables; error accumulation: errors in early dimensions propagate forward; slow for high-dimensional data (images, long sequences).

18.6.2 Langevin Dynamics

Langevin Dynamics

Given the (Stein) score function $\nabla_x \log p(x)$, Langevin dynamics iteratively refines an initial sample towards the high-density region of $p(x)$:

$$x_{t+1} = x_t + \frac{\epsilon}{2}\nabla_x \log p(x_t) + \sqrt{\epsilon}\,\mathbf{z}_t, \quad \mathbf{z}_t \sim \mathcal{N}(0, \mathbb{I})$$

where ϵ is the step size. As $\epsilon \to 0$ and $t \to \infty$, the iterates converge to samples from $p(x)$. The drift term $\frac{\epsilon}{2}\nabla_x \log p$ pushes towards modes, while the noise term $\sqrt{\epsilon}\,\mathbf{z}_t$ ensures exploration.

Applications

Score-based generative models (NCSN—Noise Conditional Score Network), where a neural network $s_\theta(x, \sigma)$ estimates the score at varying noise levels. Also used for refining samples from other generators and for sampling energy-based models.

Trade-offs

Pros: Only requires the score function (no normalizing constant); works in the data space directly; parallelizable across multiple chains. **Cons:** Requires many steps (T typically 100–1000); step-size sensitivity; mixes poorly between disconnected modes; requires accurate score estimation, which is hard in low-density regions.

18.6.3 Annealed Langevin Dynamics (NCSN)

Annealed Langevin (NCSN)

Uses a noise-conditional score network $s_\theta(x, \sigma_i)$ trained on $\sigma_1 > \sigma_2 > \cdots > \sigma_L$. At each noise level i:

$$x_{t+1} = x_t + \alpha_i\, s_\theta(x_t, \sigma_i) + \sqrt{2\alpha_i}\,\mathbf{z}_t$$

where $\alpha_i = \eta \cdot \sigma_i^2/\sigma_L^2$ controls the step size. Starting from random noise at σ_1, samples are progressively refined through all L levels, with each level's Langevin dynamics initialized

from the previous level's output.

Applications

The original NCSN (Song & Ermon, 2019) for unconditional image generation. The annealed approach is a precursor to diffusion models and remains important for score-based modeling when discretization flexibility is needed.

Trade-offs

Pros: Multi-scale approach overcomes the mode-mixing difficulty of single-scale Langevin; no forward diffusion process needed at generation time. **Cons:** Requires L score models (or a conditional model); total steps $= L \times T$ can be large; careful annealing schedule design is critical.

18.6.4 DDPM Sampling (Denoising Diffusion)

DDPM Reverse (Sampling) Process

Starting from $x_T \sim \mathcal{N}(0, \mathbb{I})$, the reverse diffusion iteratively denoises:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}_\theta(x_t, t)\right) + \sigma_t \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, \mathbb{I})$$

where $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, $\alpha_t = 1 - \beta_t$ with β_t the forward variance schedule, and $\boldsymbol{\epsilon}_\theta$ is the noise-prediction network. The variance σ_t may be fixed ($\sigma_t^2 = \beta_t$) or learned.

Applications

DDPM (Ho et al., 2020) for high-quality image generation, inpainting, super-resolution, and conditional generation (text-to-image via classifier guidance or classifier-free guidance). Foundation of Stable Diffusion, DALL-E 2, and Imagen.

Trade-offs

Pros: State-of-the-art image quality; stable training (regression target); flexible conditioning (classifier guidance, cross-attention). **Cons:** T typically 1000 steps—slow sampling; the Gaussian assumption for $p(x_{t-1} \mid x_t)$ is an approximation; quality degrades with aggressive step reduction.

18.6.5 DDIM (Deterministic Sampling)

DDIM Sampling

DDIM reinterprets the DDPM objective as a non-Markovian generative process, enabling deterministic and skip-step sampling:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \underbrace{\left(\frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}\right)}_{\text{predicted } x_0} + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2}\,\boldsymbol{\epsilon}_\theta(x_t, t) + \sigma_t \mathbf{z}$$

Setting $\sigma_t = 0$ yields a deterministic mapping from noise to data (forward ODE integration). With η-parametrization, $\sigma_t = \eta\sqrt{(1-\bar{\alpha}_{t-1})/(1-\bar{\alpha}_t)}\sqrt{1-\bar{\alpha}_t/\bar{\alpha}_{t-1}}$, where $\eta = 0$ gives DDIM and $\eta = 1$ recovers DDPM.

Applications

Accelerated diffusion sampling (10–50 steps vs. 1000 for DDPM). Semantic interpolation: deterministic mapping enables meaningful latent-space walks. Used in Stable Diffusion for fast inference and latent space manipulation.

Trade-offs

Pros: 10–50x faster sampling than DDPM; deterministic encoding (diffusion inversion) enables real-image editing; consistent samples (same noise → same output). **Cons:** Lower sample quality with very few steps; the deterministic ODE limit produces less diverse samples; requires retraining or fine-tuning to match DDPM quality at extreme step reductions.

18.6.6 DPM-Solver (Diffusion ODE Solver)

DPM-Solver

Treats the diffusion reverse process as an ODE:

$$\frac{dx_t}{dt} = f(t)x_t + \frac{g^2(t)}{2\sigma_t}\boldsymbol{\epsilon}_\theta(x_t, t)$$

with $f(t) = d\log\bar{\alpha}_t/dt$ and $g(t)$ from the forward diffusion SDE. DPM-Solver applies high-order ODE solvers specifically designed for the exponential integrator form of the diffusion ODE, achieving high quality with 10–20 steps.

Applications

State-of-the-art fast sampling for diffusion models. DPM-Solver++ achieves FID comparable to 1000-step DDPM with only 15–20 steps. Widely adopted in the Stable Diffusion ecosystem (diffusers library).

Trade-offs

Pros: Exponential convergence in number of steps; analytically derived from the diffusion SDE form; high quality at 10–20 NFE (number of function evaluations). **Cons:** More complex implementation than DDIM; requires carefully designed time-step discretization; the ODE formulation loses the stochastic benefits of the SDE sampler.

18.6.7 Classifier-Free Guidance (CFG)

Classifier-Free Guidance

CFG interpolates between conditional and unconditional score estimates at inference time, without a separate classifier:

$$\hat{\boldsymbol{\epsilon}}_\theta(x_t, c) = (1+w)\,\boldsymbol{\epsilon}_\theta(x_t, c) - w\,\boldsymbol{\epsilon}_\theta(x_t, \varnothing)$$

where c is the conditioning (text, class label), $\varnothing$ is the null condition (empty text, zero class), and $w \geq 0$ is the guidance scale. At $w = 0$, standard conditional generation; as $w \to \infty$, the sample is forced towards the conditional mode, improving adherence to c at the cost of diversity.

Implicit Classifier Interpretation

CFG implicitly uses a classifier gradient:

$$\nabla_x \log p_{\text{CFG}}(x \mid c) = \nabla_x \log p(x \mid c) + w \nabla_x \log \frac{p(x \mid c)}{p(x)}$$

The second term is proportional to $\nabla_x \log p(c \mid x)$, i.e., the gradient of an implicit classifier $p(c \mid x)$.

Applications

Modern text-to-image diffusion models (Stable Diffusion, DALL-E 2, Imagen). Responsible for the high prompt-adherence and image quality of these systems. Also used in audio diffusion (AudioLDM) and video generation.

Trade-offs

Pros: No separate classifier needed; joint training of conditional and unconditional model through random condition dropout; simple inference-time hyperparameter w. **Cons:** Reduces sample diversity as w increases (guidance–diversity trade-off); requires training with random null conditioning (10–20% dropout); large w can produce over-saturated, unrealistic images.

18.7 LLM Decoding Strategies

18.7.1 Temperature Sampling

Temperature-Scaled Softmax

Scale the logits by a temperature parameter $T > 0$ before sampling:

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

- $T \to 0$: distribution concentrates on the argmax (greedy decoding, maximum probability).
- $T = 1$: standard softmax (default sampling).
- $T > 1$: distribution flattens, increasing diversity but risk of sampling low-probability nonsense.
- $T \to \infty$: uniform distribution over the vocabulary.

Applications

Universal in autoregressive language model text generation (GPT, LLaMA, Claude). Typical values: $T = 0.7$–1.0 for creative writing, $T = 0.1$–0.3 for factual/structured outputs, $T = 0$ (greedy) for deterministic tasks like translation.

Trade-offs

Pros: Single knob controlling diversity–quality trade-off; computationally trivial (one division). **Cons:** Does not truncate the tail—even at $T = 1$, extremely unlikely tokens can be sampled, causing degeneration; requires tuning per task/domain; greedy mode can produce repetitive, degenerate text.

18.7.2 Top-K and Top-P (Nucleus) Sampling

Top-K Sampling

At each step, restrict sampling to the K tokens with the highest logits, then renormalize and sample from this subset. K is typically 10–50. If $K = 1$, this is greedy decoding.

Top-P (Nucleus) Sampling

At each step, select the smallest set of tokens whose cumulative probability mass exceeds a threshold $p \in (0, 1]$:

$$\mathcal{V}^{(p)} = \text{smallest set s.t.} \sum_{v \in \mathcal{V}^{(p)}} p(v) \geq p$$

Sample from the renormalized distribution over $\mathcal{V}^{(p)}$. This adapts the truncation threshold dynamically based on the shape of the distribution: flat distribution → many tokens kept, peaked distribution → few tokens kept.

Applications

Standard in most LLM APIs (OpenAI, Anthropic, Cohere). Top-p with $p = 0.9$–0.95 is the typical default for open-ended generation. Top-$K = 40$ is common in the Hugging Face `generate()` API.

Trade-offs

Pros: Top-p adapts to context (no fixed K); both methods prevent sampling from the unreliable tail; simple to implement. **Cons:** Top-K requires choosing a fixed K that may be too large or too small depending on context; Top-p can occasionally sample degenerate tokens if the tail is flat; no theoretical optimality guarantee.

18.7.3 Beam Search

Beam Search with Length Penalty

Maintain a beam of the B most probable partial sequences at each step. At termination, score sequences with a length penalty:

$$\text{score}(y_{1:T}) = \frac{1}{|y|^\alpha} \sum_{t=1}^{|y|} \log p(y_t \mid y_{<t})$$

where $\alpha \in [0, 1]$ controls length normalization: $\alpha = 0$ is no penalty (favors short sequences), $\alpha = 1$ is average log-probability per token. Common value: $\alpha = 0.6$–0.8. Beam size B is typically 4–8 for NMT and 1–4 for LLMs.

Applications

Neural machine translation (standard), image captioning, speech recognition (CTC/RNN-T decoding), and code generation where deterministic, high-probability outputs are preferred.

Trade-offs

Pros: Near-optimal MAP decoding for moderate B; produces higher-likelihood outputs than sampling for structured tasks (translation). **Cons:** $O(B \cdot V)$ per step; large B degrades quality in open-ended generation (beam search bias—longer beams produce shorter, more generic text); tends to produce repetitive outputs for LLMs; the length penalty introduces a hyperparameter.

18.7.4 Diverse Beam, Contrastive, and Speculative Decoding

Diverse Beam Search

Add a diversity penalty to beam search: when selecting the top B candidates for group g, subtract a penalty proportional to overlap with already selected sequences in other groups:

$$\text{score}(y, g) = \sum_t \log p(y_t \mid \dots) - \gamma \sum_{g' < g} \delta(y_t, y_t^{(g')})$$

This encourages beams from different groups to produce distinct output sequences, avoiding the beam collapse problem where all beams are identical.

Contrastive Decoding

Sample from a distribution that amplifies differences between an expert (large) LM and an amateur (small) LM:

$$p_{\text{CD}}(x_t \mid x_{<t}) \propto \frac{p_{\text{expert}}(x_t \mid x_{<t})^\alpha}{p_{\text{amateur}}(x_t \mid x_{<t})^\beta}$$

This suppresses the amateur's fluent-but-generic modes, encouraging the expert to produce more informative outputs.

Speculative Decoding

A small, fast draft model generates K tokens autoregressively; the large target model verifies them in parallel with a single forward pass:

$$\text{Accept } x_t \text{ if } r \leq \min\left(1, \frac{p_{\text{target}}(x_t)}{p_{\text{draft}}(x_t)}\right), \quad r \sim U(0,1)$$

Rejected tokens trigger resampling from the adjusted distribution $(p_{\text{target}} - p_{\text{draft}})_+$. This provides 2–3x wall-clock speedup with identical output distribution.

Applications

Diverse beam: image captioning (multiple distinct captions). Contrastive: improving factuality and reducing blandness in LLM outputs. Speculative: low-latency LLM serving (Google's inference, TGI/vLLM).

Trade-offs

Diverse beam: Pros—produces multiple diverse hypotheses; Cons—penalty coefficient γ requires tuning; may sacrifice per-beam quality. **Contrastive:** Pros—improved informativeness and factuality; Cons—requires a second model; sensitive to α, β. **Speculative:** Pros—no change to output distribution; 2–3x speedup; Cons—requires aligned draft model; speedup depends on draft-target agreement rate.

18.7.5 Typical Sampling and Mirostat

Typical Sampling (Locally Typical Sampling)

Rather than sampling from the highest-probability tokens (Top-K/Top-p), typical sampling selects tokens with negative log-probability close to the conditional entropy:

$$\mathcal{V}^{(t)} = \left\{v \in \mathcal{V} : \left|-\log p(v \mid x_{<t}) - H(p(\cdot \mid x_{<t}))\right| \leq \tau\right\}$$

where $H(p) = -\sum_v p(v) \log p(v)$ is the entropy. This targets "typical" tokens that the model expects, avoiding both high-probability generic tokens and low-probability random tokens.

Mirostat

A dynamic temperature algorithm that adapts T online to maintain a target perplexity τ. At each step t:

$$T_{t+1} = T_t + \eta \cdot (\text{observed_surprise}_t - \tau)$$

where the "surprise" is $-\log p(x_t)$. This keeps the generated text at a consistent information density, preventing both repetition and degeneration.

Applications

Typical sampling: alternative to Top-p for open-ended generation with fewer degenerate repetitions. Mirostat: long-form generation where sustained quality over hundreds of tokens matters (stories, essays).

Trade-offs

Typical sampling: Pros—theoretically motivated (information theory); produces more human-like text in some evaluations. Cons—requires computing entropy at each step; τ tuning is non-trivial. **Mirostat:** Pros—no fixed temperature; adapts to context; reduces degeneration in long sequences. Cons—η and initial T still require tuning; slower due to per-step adaptation.

18.7.6 Classifier-Free Guidance for LLMs

CFG for Language Models

Adapting the CFG principle from diffusion models to autoregressive LMs, a conditional and unconditional model are jointly trained (via random prompt dropout). At inference, the logits are adjusted:

$$\text{logits}_{\text{CFG}} = (1 + w)\,\text{logits}(x_t \mid \text{prompt}) - w\,\text{logits}(x_t \mid \varnothing)$$

where w is the guidance scale. This amplifies the effect of the prompt, improving instruction-following and reducing hallucination. Applicable to instruction-tuned and chat models.

Applications

Improving instruction adherence in chat LLMs, reducing hallucination, and strengthening the model's response to system prompts. Used in some open-source LLM training pipelines (e.g., Llama-derived models).

Trade-offs

Pros: No separate classifier or reward model needed; joint training with prompt dropout is simple; improves instruction following. **Cons:** Reduces diversity (over-focus on prompt); guidance scale w must be tuned per task; adds inference overhead (two forward passes or feature reuse); less studied than the diffusion equivalent.

18.8 Curriculum and Active Learning

18.8.1 Curriculum Learning

Curriculum Learning

Present training examples in a meaningful order—typically from "easy" to "hard"—rather than randomly. A scoring function $s(x)$ ranks examples by difficulty (e.g., shorter sequences first, cleaner images first, lower noise first). Training proceeds in stages:

1. **Stage 1:** Train only on examples with $s(x) \leq \tau_1$
2. **Stage 2:** Expand to $s(x) \leq \tau_2$ where $\tau_2 > \tau_1$
3. **Stage K:** Full dataset

The schedule can be hand-designed (curriculum) or automatically determined by the model's current performance (self-paced learning).

Applications

Neural machine translation (short sentences first), image classification (clean images before noisy ones), language model pre-training (simpler corpora first), and robotics (easy tasks before complex ones).

Trade-offs

Pros: Faster convergence and better generalization (acts as a continuation method); guides optimization to better local minima. **Cons:** Defining "easy" vs. "hard" is problem-dependent; wrong curriculum can harm performance; fixed curricula may not match the model's learning dynamics (self-paced learning addresses this).

18.8.2 Uncertainty Sampling, BALD, and Query by Committee

Uncertainty Sampling (Active Learning)

The model queries the unlabeled example for which it is most uncertain:

$$x^* = \arg\max_x \mathcal{U}(x)$$

where $\mathcal{U}(x)$ is an uncertainty measure: least-confidence ($1 - \max_y p(y|x)$), margin ($\max_y p - 2$nd-highest p), or entropy ($-\sum_y p(y|x) \log p(y|x)$). The queried example is labeled by an oracle and added to the training set.

BALD (Bayesian Active Learning by Disagreement)

BALD selects examples that maximize the mutual information between model parameters and predictions:

$$x^* = \arg\max_x I(y; \theta \mid x, \mathcal{D}) = \arg\max_x \Big(H(y \mid x, \mathcal{D}) - \mathbb{E}_{\theta|\mathcal{D}}[H(y \mid x, \theta)] \Big)$$

The first term is the total predictive entropy; the second is expected entropy under the posterior. BALD selects points where the model is uncertain overall (high total entropy) but where individual parameter samples disagree (low expected entropy). This avoids querying noisy or inherently ambiguous examples.

Query by Committee (QBC)

Train an ensemble (committee) of M models on the current labeled set. For each unlabeled example, the disagreement among committee predictions is measured (e.g., vote entropy, KL divergence to the consensus). The example with highest disagreement is queried:

$$x^* = \arg\max_x \frac{1}{M} \sum_{m=1}^{M} \mathrm{KL}\big(p_m(y \mid x) \,\big\|\, \bar{p}(y \mid x)\big)$$

where $\bar{p}(y|x) = \frac{1}{M} \sum_m p_m(y|x)$ is the committee consensus.

Applications

Label-efficient deep learning: medical image annotation, NLP annotation with expensive expert labelers, and any domain where unlabeled data is abundant but labeling is costly.

Trade-offs

Pros: Reduces labeling cost by 10–50x versus random sampling; BALD avoids querying irreducible-noise examples; QBC works with any model type. **Cons:** Uncertainty sampling can query outliers (redundant labeling); BALD requires approximate Bayesian inference; QBC requires training M models; all methods incur computational overhead for scoring the unlabeled pool.

18.9 Data Imbalance Handling

18.9.1 SMOTE (Synthetic Minority Over-sampling Technique)

SMOTE

For each minority-class example x_i, randomly select one of its k-nearest minority neighbors x_{kNN} and generate a synthetic sample along the line segment between them:

$$x_{\text{new}} = x_i + \lambda\,(x_{\text{kNN}} - x_i), \quad \lambda \sim U(0,1)$$

This creates new minority points that lie on the convex combination of existing minority examples, expanding the minority decision boundary without simply duplicating data.

Applications

Class-imbalanced classification (fraud detection, rare disease diagnosis, churn prediction). Typically used as a preprocessing step before training classifiers on tabular data. Often combined with random under-sampling of the majority class.

Trade-offs

Pros: Simple, model-agnostic; increases minority class density without exact duplication; works in arbitrary dimensions. **Cons:** Blind to the majority class—can create samples inside majority regions (noisy synthesis); amplifies existing noise in minority class; $O(|N_{\min}| \cdot k)$ pairwise distance computation; less effective for high-dimensional data (images, text) where interpolation in raw feature space is unnatural.

18.9.2 SMOTE Variants and Under-Sampling Methods

Borderline-SMOTE

Synthesize samples only from minority examples near the decision boundary ("borderline" points). A minority point is borderline if more than half of its k-nearest neighbors belong to the majority class. SMOTE is then applied only to these points, focusing oversampling on the classification frontier.

ADASYN (Adaptive Synthetic Sampling)

Generate more synthetic samples for minority examples that are harder to learn. Each minority example x_i is assigned a weight proportional to the fraction of majority-class points among its k neighbors:

$$w_i = \frac{\#\text{ majority neighbors of } x_i}{k}$$

The total number of synthetic samples per point is proportional to $w_i / \sum_j w_j$, adaptively focusing on the hardest regions.

Tomek Links and NearMiss

Tomek Links: A pair (x_i, x_j) from different classes is a Tomek link if no other example is closer to either than they are to each other. Removing the majority-class member of each Tomek link cleans the decision boundary (under-sampling). **NearMiss:** Select majority-class examples whose average distance to the k nearest minority examples is smallest (NearMiss-1), farthest (NearMiss-2), or to all minority examples (NearMiss-3). This provides controlled under-sampling to balance class distributions.

Applications

Borderline-SMOTE: when the decision boundary matters more than overall minority density. ADASYN: when the difficulty of minority examples varies widely. Tomek Links: cleaning noisy decision boundaries after SMOTE. NearMiss: controlled under-sampling for large-scale datasets where over-sampling is computationally prohibitive.

Trade-offs

Borderline-SMOTE: Pros—focuses on informative region; Cons—may miss intra-class structure away from the boundary. **ADASYN:** Pros—adaptive to local difficulty; Cons—sensitive to noise (hard examples may be outliers). **Tomek Links:** Pros—simple boundary cleaning; Cons—removes relatively few points; not sufficient alone for severe imbalance. **NearMiss:** Pros—computationally efficient; Cons—may discard informative majority examples; the "nearness" heuristics are not theoretically grounded.

18.10 Summary Comparison Table

Selection Guide

Practical recommendations:

1. **Basic data sampling:** Stratified sampling for imbalanced classification; reservoir sampling for streaming and online learning.
2. **Off-policy RL:** PER with $\alpha = 0.6, \beta_0 = 0.4$ annealing to 1 is the default for value-based methods.
3. **Metric learning:** Semi-hard negative mining with PK-batch sampling ($P = 16, K = 8$) is the most reliable recipe.
4. **Bayesian inference:** NUTS for models that fit in memory; SGLD for deep neural network posteriors with large datasets.
5. **VAE training:** Use reparameterization for continuous latents; Gumbel-Softmax

Table 18.1: Comprehensive Summary of Sampling Strategies

Method	Category	Key Formula/Mechanism	Best For
Uniform (w/o repl.)	Basic	Fisher–Yates, $\binom{N}{k}$ uniform	SGD mini-batches
Stratified	Basic	$n_h = k \cdot \lvert S_h \rvert / N$	Class-balanced splits
Reservoir (Alg. R)	Basic	$P(\text{replace}) = k/i$	Streaming data
Importance Sampling	IS	$\mathbb{E}_p[f] \approx \frac{1}{N} \sum \frac{p}{q} f$	Off-policy RL, posteriors
Weight-Norm. IS	IS	$\frac{\sum w_i f_i}{\sum w_i}$	Unknown normalizing const.
PER	IS	$P(i) \propto \lvert \delta_i \rvert^\alpha$, $w_i \propto (NP(i))^{-\beta}$	Deep Q-learning
AIS	IS	$\prod_t p_t / p_{t-1}$ T-step annealing	Model evidence estimation
Hard Neg. Mining	Hard Mining	$d(a,n) < d(a,p)$	Metric learning, face recog.
Semi-Hard Mining	Hard Mining	$d(a,p) < d(a,n) < d(a,p) + m$	Stable triplet training
Batch-Hard Mining	Hard Mining	Hardest pos. + neg. in batch	Re-ID, PK sampling
OHEM	Hard Mining	Top-B RoIs by loss	Object detection
Focal Loss	Soft Mining	$-\alpha_t (1-p_t)^\gamma \log p_t$	Dense detection (RetinaNet)
Metropolis–Hastings	MCMC	$\alpha = \min(1, \frac{p(x')q(x\lvert x')}{p(x)q(x'\lvert x)})$	General posterior sampling
Gibbs	MCMC	$x_i \sim p(x_i \mid x_{-i})$	Conjugate models, LDA
HMC	MCMC	Leapfrog: $H = -\log p + \frac{1}{2} p^T M^{-1} p$	Correlated high-dim posteriors
NUTS	MCMC	Auto tree depth via U-turn criterion	Stan default, hands-free HMC
SGLD	MCMC	$\Delta\theta = \frac{\epsilon}{2} \nabla \log p + \eta_t$	Large-scale BNN sampling
Reparameterization	Var. Samp.	$z = g(\epsilon, \phi)$, $\nabla_\phi \mathbb{E}_q = \mathbb{E}[\nabla_z f \cdot \nabla_\phi g]$	VAEs, continuous latents
REINFORCE	Var. Samp.	$\nabla_\phi \mathbb{E}_q[f] = \mathbb{E}_q[f \nabla_\phi \log q]$	Discrete latents, policy grad.
STE	Var. Samp.	Forward: argmax; Backward: identity	VQ-VAE, binarized nets
REBAR/RELAX	Var. Samp.	Control variate + relaxation	Discrete VAEs
Ancestral (AR)	Gen. Models	$x_i \sim p(x_i \mid x_{<i})$	GPT, PixelCNN, WaveNet
Langevin Dyn.	Gen. Models	$x_{t+1} = x_t + \frac{\epsilon}{2} \nabla \log p + \sqrt{\epsilon}\mathbf{z}$	Score-based (NCSN)
DDPM	Gen. Models	$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta) + \sigma_t \mathbf{z}$	High-quality image gen.
DDIM	Gen. Models	ODE solve, $\sigma_t = 0$ deterministic	Fast diff. sampling, inversion
DPM-Solver	Gen. Models	Exponential integrator ODE	10–20 NFE diff. sampling
CFG (Diffusion)	Gen. Models	$\hat{\epsilon} = (1+w)\epsilon(x,c) - w\epsilon(x,\varnothing)$	Text-to-image guidance
Temperature	LLM Decoding	$p_i \propto e^{z_i/T}$	LLM diversity control
Top-K / Top-p	LLM Decoding	Truncate to K / cum. prob. $\geq p$	LLM truncation default
Beam Search	LLM Decoding	$\log p(Y)/\lvert Y \rvert^\alpha$, beam size B	NMT, structured tasks
Speculative Decoding	LLM Decoding	Draft + verify: accept if $r \leq p_t/p_d$	Low-latency LLM serving
CFG (LLM)	LLM Decoding	Text $\propto (1+w)\log p(x\lvert c) - w \log p(x\lvert\varnothing)$	Instruction adherence
Curriculum Learn.	Curriculum	Easy $\rightarrow$ hard scheduling	NMT, warm-start training
BALD	Active Learn.	$x^* = \arg\max I(y; \theta \mid x, \mathcal{D})$	Label-efficient DL
SMOTE	Imbalance	$x_{\text{new}} = x_i + \lambda(x_{\text{kNN}} - x_i)$	Tabular class imbalance
ADASYN	Imbalance	Adaptive weight per minority point	Varying minority hardness
Tomek Links	Imbalance	Remove majority in mutual nearest pairs	Boundary cleaning
NearMiss	Imbalance	Select majority near minority	Controlled under-sampling

(straight-through) for discrete latents when bias is acceptable; REBAR when unbiased gradients are required.

6. **Image generation:** CFG with $w = 7$–9 on DDIM (20–50 steps) or DPM-Solver++ (15–20 steps) for quality/speed balance.
7. **LLM decoding:** Top-$p = 0.95$, $T = 0.7$–0.9 for creative text; beam search ($B = 4$–5) for translation and code.
8. **Label-efficient learning:** BALD (with MC Dropout or deep ensemble) for pool-based active learning; combine with curriculum when natural difficulty ordering exists.
9. **Class imbalance:** SMOTE + Tomek Links for tabular data; focal loss or class-weighted loss for deep learning on images/text.

Chapter 19

Gradient Clipping

Deep neural networks, especially recurrent architectures and large transformers, are prone to explosive gradient growth. A single large gradient can push parameters far outside the basin of attraction, causing divergence, NaN loss, and irrecoverable training failure. Gradient clipping is the simplest and most widely used remedy—it places a hard bound on gradient magnitude, trading a small amount of bias for dramatic stability gains. This chapter surveys the major clipping strategies and their theoretical underpinnings.

19.1 Why Gradient Clipping Is Necessary

The exploding gradient problem: Consider an L-layer network. During backpropagation, the gradient of the loss $\mathcal{L}$ with respect to parameters at layer l is the product of Jacobian matrices:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}} \prod_{k=l+1}^{L} \frac{\partial \mathbf{h}^{(k)}}{\partial \mathbf{h}^{(k-1)}} \frac{\partial \mathbf{h}^{(l)}}{\partial \mathbf{W}^{(l)}}$$

If the Jacobian spectral norms consistently exceed 1, the gradient norm grows exponentially with depth: $\|\nabla_{\mathbf{W}^{(l)}} \mathcal{L}\| \propto \rho^{L-l}$ for some $\rho > 1$. This produces update steps so large that the loss diverges to infinity.

Four root causes of exploding gradients:

1. **Deep recurrence (RNN/LSTM):** Repeated multiplication by the same weight matrix in BPTT creates a geometric series—if the largest eigenvalue of the recurrent weight matrix exceeds 1, gradients explode along the time axis.
2. **Large learning rate $\times$ large gradient:** Even if gradients are moderately large, a high learning rate amplifies the step, leading to divergence.
3. **Sharp loss surfaces:** Regions of high curvature produce gradients with large norms; Adam and other adaptive optimizers can partially mask but not eliminate this.
4. **Unstable architectures / poor initialization:** Without proper scaling (Kaiming, Xavier), the forward activations themselves explode, and backprop amplifies further.

19.2 Norm-Based Global Clipping

19.2.1 L2 Norm Clipping

Given the gradient vector $\mathbf{g}_t = \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t)$ concatenating all parameters, L2 norm clipping caps its Euclidean norm at a threshold c:

$$\tilde{\mathbf{g}}_t = \begin{cases} \mathbf{g}_t & \text{if } \|\mathbf{g}_t\|_2 \leq c \\ c \cdot \dfrac{\mathbf{g}_t}{\|\mathbf{g}_t\|_2} & \text{if } \|\mathbf{g}_t\|_2 > c \end{cases}$$

Equivalently: $\tilde{\mathbf{g}}_t = \mathbf{g}_t \cdot \min\big(1, c/\|\mathbf{g}_t\|_2\big)$. The update step becomes $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta\, \tilde{\mathbf{g}}_t$.

Geometric interpretation: L2 clipping projects the gradient onto the ℓ_2 ball of radius c. The direction is preserved; only the magnitude is truncated. For any two parameters w_i, w_j, the ratio $|\tilde{g}_i|/|\tilde{g}_j| = |g_i|/|g_j|$ is unchanged.

Usage: The default clipping strategy in most deep learning frameworks. Standard for RNNs, LSTMs, GRUs, and Transformers. PyTorch: `torch.nn.utils.clip_grad_norm_(params, max_norm)`. TensorFlow: `tf.clip_by_global_norm`. Typical thresholds: $c \in [0.5, 5.0]$ for RNNs; $c = 1.0$ is a common default.

Pros:

- Preserves the relative scaling among parameters (isotropic rescaling).
- Single hyperparameter c; easy to tune by monitoring gradient norms during the first few training steps.
- Computationally cheap: one dot product (squared norm), one square root, and element-wise rescaling.
- Compatible with any optimizer (SGD, Adam, AdamW, RMSProp).

Cons:

- The threshold c is a global constant—it does not adapt to different layers or parameter groups with different natural gradient scales.
- Does not protect against individual coordinate outlier spikes if the global norm remains below c.
- For very deep networks, the global norm can be dominated by a subset of layers, masking smaller but still harmful gradients elsewhere.

19.2.2 L∞ (Max Norm) Clipping

Caps each individual gradient coordinate at a threshold c:

$$[\tilde{\mathbf{g}}_t]_i = \text{clip}([\mathbf{g}_t]_i, -c, c) \quad \text{or equivalently} \quad [\tilde{\mathbf{g}}_t]_i = \max(-c, \min([\mathbf{g}_t]_i, c))$$

This is **not** the same as projecting the entire gradient vector onto an ℓ_∞ ball; it is element-wise clamping.

Usage: Less common than L2 clipping but useful when individual coordinates can spike independently (e.g., embedding lookups with rare tokens, or attention logits before softmax). TensorFlow: `tf.clip_by_value`. PyTorch: `torch.nn.utils.clip_grad_value_`.

Pros: Protects every coordinate individually. Simple element-wise operation. **Cons:** Destroys the relative scaling among parameters (the gradient vector direction changes), which can harm convergence. The threshold c is especially hard to calibrate since different layers have drastically different natural gradient magnitudes.

19.2.3 L1 Norm Clipping

Caps the ℓ_1 norm of the global gradient:

$$\tilde{\mathbf{g}}_t = \begin{cases} \mathbf{g}_t & \text{if } \|\mathbf{g}_t\|_1 \leq c \\ c \cdot \dfrac{\mathbf{g}_t}{\|\mathbf{g}_t\|_1} & \text{if } \|\mathbf{g}_t\|_1 > c \end{cases}$$

Like L2 clipping, this preserves the direction of $\mathbf{g}_t$ but uses the ℓ_1 norm as the measurement.

Pros: Less sensitive to a few large-magnitude outliers than L2 clipping (in L2, one extreme coordinate dominates the squared sum). **Cons:** Rarely used in practice; the ℓ_1 norm is not rotationally invariant. L2 clipping almost always preferred.

19.3 Advanced Clipping Techniques

19.3.1 Per-Sample Gradient Clipping (DP-SGD)

For differential privacy (DP-SGD), clipping is applied **per example** before averaging, not on the batch-aggregated gradient:

$$\bar{\mathbf{g}}_i = \mathbf{g}_i \cdot \min\left(1, \frac{C}{\|\mathbf{g}_i\|_2}\right)$$

where $\mathbf{g}_i = \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)})$ is the gradient of the loss on example i. Then noise is added:

$$\tilde{\mathbf{g}} = \frac{1}{B}\left(\sum_{i=1}^{B} \bar{\mathbf{g}}_i + \mathcal{N}(\mathbf{0}, \sigma^2 C^2 \mathbf{I})\right)$$

The clipping bound C limits the **sensitivity** of each individual's contribution, which is the key requirement for (ϵ, δ)-differential privacy.

Privacy guarantee (moments accountant): For $\sigma \geq 1$ and batch sampling rate $q = B/N$, each step satisfies (ϵ, δ)-DP where ϵ is tracked by the moments accountant. The total privacy budget after T steps is computed by composing the per-step budgets via advanced composition theorems or Rényi DP.

Usage: Training on sensitive data (medical, financial, user behaviour). Implemented in Opacus (PyTorch), TensorFlow Privacy. **Critical detail:** Per-sample clipping breaks the standard vectorized batch gradient computation—each sample's gradient must be computed and clipped independently, which is supported via gradient hooks and `per_sample_grad` backends.

Pros: Formal (ϵ, δ)-DP guarantee. Enables training on private data with provable bounds on information leakage. **Cons:** Large accuracy penalty for small ϵ. Significant computational overhead (disables vectorized backprop). The clipping norm C and noise multiplier σ require careful calibration; suboptimal choices waste the privacy budget.

19.3.2 Element-wise Value Clipping

Simple element-wise clamping after the gradient is computed:

$$[\tilde{\mathbf{g}}_t]_i = \max\big(-c,\ \min([\mathbf{g}_t]_i, c)\big)$$

Unlike L∞ norm clipping, this does not project onto an ℓ_∞ ball—each coordinate is independently clamped to $[-c, c]$. The resulting vector may have arbitrarily large norm if many coordinates are near the boundary.

Usage: Quick-and-dirty stabilization when NaN or Inf values appear during training. Common in prototype code and as a debugging tool. Also used for clipping attention logits or Q-values in reinforcement learning to avoid extreme predictions.

Pros: Trivial to implement. Guarantees no individual coordinate exceeds bounds. **Cons:** Biases the gradient direction. Does not guarantee a bounded global norm. Too aggressive clamping can prevent the model from escaping flat regions.

19.3.3 Adaptive Gradient Clipping (AGC)

AGC (Brock et al., 2021, NFNet) scales gradients per-layer based on the ratio of gradient norm to weight norm:

$$\mathbf{g}^{(l)} \leftarrow \begin{cases} \mathbf{g}^{(l)} & \text{if } \dfrac{\|\mathbf{g}^{(l)}\|_F}{\|\mathbf{W}^{(l)}\|_F} \le \lambda \\ \lambda \cdot \dfrac{\|\mathbf{W}^{(l)}\|_F}{\|\mathbf{g}^{(l)}\|_F} \cdot \mathbf{g}^{(l)} & \text{otherwise} \end{cases}$$

where $\|\cdot\|_F$ is the Frobenius norm, $\mathbf{g}^{(l)}$ is the gradient for weight matrix $\mathbf{W}^{(l)}$, and λ is a unitless clipping threshold (typically $\lambda = 0.01$).
Intuition: A single SGD step of size η changes the relative weight magnitude by approximately $\eta \cdot \|\mathbf{g}^{(l)}\|_F / \|\mathbf{W}^{(l)}\|_F$. AGC limits this relative change to $\eta\lambda$, preventing any layer from being "jerked" too far in one update while allowing layers with small gradients to update freely.

Relation to normalized optimizers: LARS (Layer-wise Adaptive Rate Scaling) uses a similar ratio $\|\mathbf{W}^{(l)}\|_F / \|\mathbf{g}^{(l)}\|_F$ to adapt per-layer learning rates. AGC is simpler—it only clips when the ratio exceeds λ, acting as a safety valve rather than a constant rescaling.

Usage: Training very deep networks **without BatchNorm** (NFNet family achieves state-of-the-art ImageNet accuracy without any normalization layers). AGC is also beneficial for large-batch training where gradient norms can fluctuate wildly across layers. PyTorch implementation available in `timm` (Ross Wightman's library). Typical $\lambda = 0.01$.

Pros: Per-layer adaptive threshold; no global clipping constant needed. Enables training without BatchNorm, which is desirable for small-batch or sequential models. Theoretically motivated by preventing large relative weight changes rather than large absolute gradients. **Cons:** Requires computing per-layer Frobenius norms at each step. The threshold λ is empirically tuned and its optimal value varies across architectures. AGC does not protect bias gradients (bias terms have no corresponding "weight norm").

19.4 Related Stabilization Techniques

19.4.1 Gradient Penalty (WGAN-GP)

The WGAN-GP (Gulrajani et al., 2017) enforces the 1-Lipschitz constraint on the discriminator D via a soft penalty on the gradient norm at interpolated points:

$$\mathcal{L}_{\text{GP}} = \lambda \, \mathbb{E}_{\hat{\mathbf{x}} \sim \mathbb{P}_{\hat{\mathbf{x}}}} \left[\left(\|\nabla_{\hat{\mathbf{x}}} D(\hat{\mathbf{x}})\|_2 - 1 \right)^2 \right]$$

where $\hat{\mathbf{x}} = \epsilon \mathbf{x}_{\text{real}} + (1 - \epsilon)\mathbf{x}_{\text{fake}}$ with $\epsilon \sim \mathcal{U}(0, 1)$.
Why this works: The optimal critic for the Wasserstein-1 distance has gradient norm exactly 1 almost everywhere. The penalty encourages D to have gradient norm ≈ 1 at all interpolated samples, enforcing the Lipschitz constraint **softly** instead of the hard weight clipping used in the original WGAN.

Connection to gradient clipping: WGAN-GP does not clip parameters. Instead, it adds an auxiliary loss that **encourages the gradient to have a bounded norm**. This is a "soft" form of gradient clipping applied through the loss function rather than post-hoc clipping. The gradient of $\mathcal{L}_{\text{GP}}$ with respect to D's parameters includes a Hessian-vector product term.

Usage: De facto standard for Wasserstein GAN training. Replaces weight clipping (which caused capacity underuse and optimization difficulty). $\lambda = 10$ is the standard coefficient. Also used in WGAN-div, and adapted for spectral normalization GANs.

Pros: Produces higher-quality GAN samples than weight clipping. Stabilizes training across diverse architectures. **Cons:** Requires a second backward pass for the gradient penalty term (double the discriminator computation). The Lipschitz constraint is only softly enforced; the critic's gradient norm may still deviate from 1. The choice of λ and sampling distribution for $\hat{\mathbf{x}}$ affects quality.

19.4.2 Automatic Mixed Precision (AMP) Loss Scaling

AMP (Micikevicius et al., 2018) trains with float16 for most operations but maintains a float32 master copy of weights. Because float16 has limited dynamic range (2^{-24} to 65504), small gradients underflow to zero. **Loss scaling** multiplies the loss by a scale factor S before backpropagation, then divides the resulting gradients by S before the weight update:

$$\mathbf{g}_{\text{fp32}} = \frac{1}{S} \cdot \nabla_{\mathbf{w}} \big(S \cdot \mathcal{L}(\mathbf{w}) \big)$$

This shifts small gradient values into the representable range of float16 without changing the effective update. S is typically chosen dynamically: start large (e.g., 2^{16}), and halve if overflow is detected in any gradient.

Relationship to gradient clipping: Loss scaling and gradient clipping are complementary. Loss scaling prevents **underflow** of small gradients; gradient clipping prevents **overflow** of large gradients. With AMP, clipping should be applied **after** unscaling: $\mathbf{g}_{\text{clipped}} = \text{clip}(\mathbf{g}_{\text{fp32}}/S)$.

Usage: Training large models (transformers, LLMs, diffusion models) where float16 halves memory and doubles throughput. Built into PyTorch (`torch.cuda.amp`), TensorFlow, and JAX. Gradient clipping is applied post-unscale. The combination of AMP + gradient clipping is standard for all large-scale training pipelines.

Pros: 2–3× training speedup with negligible accuracy loss. Scales to arbitrarily large networks. Dynamic loss scaling is automatic in most frameworks. **Cons:** Some operations are numerically unstable in float16 and must remain in float32 (e.g., softmax over large vocabularies, BatchNorm). Debugging NaN/Inf is more complex with mixed precision. Requires GPU hardware with Tensor Cores (Volta+) for full benefit.

19.5 Theoretical Motivation

Bias-variance tradeoff of clipping: Let $\mathbf{g}$ be the true gradient (unbiased under SGD). The clipped gradient $\tilde{\mathbf{g}} = \min(1, c/\|\mathbf{g}\|_2)\mathbf{g}$ introduces a **truncation bias**:

$$\mathbb{E}[\tilde{\mathbf{g}}] = \mathbf{g} - \mathbb{E}\big[\mathbf{g} \cdot \mathbf{1}_{\|\mathbf{g}\|_2 > c} \cdot (1 - c/\|\mathbf{g}\|_2)\big]$$

The clipped estimate is no longer unbiased, but its variance is dramatically reduced when large gradients are present. This is a classic bias-variance tradeoff: we accept biased gradients to prevent catastrophically high variance that would cause divergence.

Convergence with clipping (informal): Under smoothness assumptions on $\mathcal{L}$, SGD with L2 clipping converges to a stationary point provided the clipping threshold c satisfies:

$$c \geq \max_t \|\nabla \mathcal{L}(\mathbf{w}_t)\|_2$$

If c is too small (aggressive clipping), the algorithm may converge to a point where $\nabla \mathcal{L} \neq \mathbf{0}$—the **clipping bias** creates a spurious stationary point. In practice, c is set large enough

that clipping activates only for outlier steps and convergence is not meaningfully affected.

Key theoretical results:

- **Zhang et al. (2020):** Clipping provides an automatic "safety guarantee"—the probability of divergence decays exponentially with the clipping threshold. Even minimal clipping dramatically reduces the chance of a single large-gradient step destroying the model.
- **Chen et al. (2021):** For transformers, gradient clipping acts as an implicit learning-rate warmup. The clipped steps early in training have similar magnitude to the post-warmup steps, explaining why clipping can substitute for explicit warmup schedules.
- **Razin et al. (2023):** In deep linear networks, gradient clipping ensures that the trajectory stays within a bounded region, preventing the "catapult" effect where activations grow exponentially.

19.6 Practical Recommendations

When to use each clipping method:

1. **L2 Norm Clipping (default):** Always enable for RNNs, LSTMs, and Transformers. Start with $c = 1.0$ and monitor gradient norms for the first 100 steps. If the 90th percentile of $\|\mathbf{g}_t\|_2$ is below c, reduce c until clipping activates on ∼5–10% of steps.

2. **AGC:** When training without BatchNorm/LayerNorm, or when using very large batch sizes. Start with $\lambda = 0.01$.

3. **Gradient Penalty (WGAN-GP):** The standard for Wasserstein GANs. Use $\lambda = 10$.

4. **Element-wise clipping:** Only as a last resort when debugging NaN issues. Prefer L2 clipping.

5. **DP-SGD clipping:** Required for differential privacy. Tune C by monitoring the fraction of examples clipped (target: 50–80% clipped).

19.7 Summary Comparison Table

Quick-reference guide:

- **Standard DL training:** L2 norm clipping ($c = 1.0$).
- **RNN / LSTM / Transformer:** L2 norm clipping, always.
- **Normalization-free networks:** AGC ($\lambda = 0.01$).
- **GANs:** Gradient penalty ($\lambda = 10$).
- **Private training:** DP-SGD per-sample clipping.
- **Large-scale mixed precision:** AMP + L2 clipping combined.
- **If NaN/Inf appears:** Reduce c by factor 2–5; enable gradient norm logging; check for exploding activations upstream.

Table 19.1: Comprehensive Comparison of Gradient Clipping Methods

Method	**Scope**	**Preserves Direction?**	**Typical Setting**	**Key Characteristi**
L2 Norm Clip	Global	Yes	$c \in [0.5, 5.0]$	Direction-preserving, single hyperparamet
L∞ (Max) Clip	Per-element	No	c tuned per model	Protects every coo nate individually
L1 Norm Clip	Global	Yes	Rarely used	Less sensitive to la outliers
Element-wise clamp	Per-element	No	Debug only	Simple but directi biasing
AGC (NFNet)	Per-layer	Yes	$\lambda = 0.01$	Relative change bou no BatchNorm neede
DP-SGD clip	Per-sample	Yes	C via clipping fraction	Privacy guarantee; pensive
WGAN-GP	Loss-based	N/A	$\lambda = 10$	Soft Lipschitz enfo ment via penalty
AMP loss scaling	Loss-based	Yes	Dynamic S	Prevents underflow mixed precision

Chapter 20

Embedding Functions

Embeddings map discrete symbols—words, entities, graph nodes, images—to dense, low-dimensional vector spaces where semantic similarity is encoded as geometric proximity. This chapter surveys the landscape of embedding methods from classical one-hot encoding to modern cross-modal contrastive models, covering their mathematical formulations, usage patterns, and trade-offs.

20.1 Foundational Embedding Methods

20.1.1 One-Hot Encoding

For a vocabulary of size V, each token w is represented as a V-dimensional vector with a single 1 at index i and zeros elsewhere:

$$\mathbf{e}(w_i) = [0, \ldots, 0, 1, 0, \ldots, 0]^\top \in \{0,1\}^V, \quad \sum_{j=1}^{V} e_j = 1$$

An embedding layer generalizes this to a learned dense representation: $\mathbf{e}(w_i) = \mathbf{E}\,\mathbf{o}_i = E_i \in \mathbb{R}^d$, where $\mathbf{E} \in \mathbb{R}^{V \times d}$ is the embedding matrix and $\mathbf{o}_i$ is the one-hot vector.

Usage: The lookup-table foundation for all learned embeddings. One-hot input to an embedding layer is the standard approach for representing categorical features, word tokens, and entity IDs in deep learning. Never used directly as a representation (dimensionality V is too high and orthogonal coding provides no similarity information).

Pros: Unambiguous; every token has exactly one encoding. Computationally, the embedding lookup $\mathbf{E}\,\mathbf{o}_i$ is O(1). **Cons:** All pairs of tokens are equidistant ($\|\mathbf{o}_i - \mathbf{o}_j\|_2 = \sqrt{2}$ for all $i \neq j$), encoding **zero semantic information**. Memory cost $O(Vd)$ becomes prohibitive for large vocabularies (e.g., LLMs with $V > 10^5$).

20.1.2 Word2Vec

Skip-Gram with Negative Sampling (SGNS): The objective maximizes the log-probability of context words c appearing near a center word w:

$$\mathcal{L} = -\sum_{(w,c)\in\mathcal{D}} \left[\log \sigma(\mathbf{v}_c^\top \mathbf{v}_w) + \sum_{k=1}^{K} \mathbb{E}_{c_k \sim P_n}[\log \sigma(-\mathbf{v}_{c_k}^\top \mathbf{v}_w)]\right]$$

where $\mathbf{v}_w$ is the input (center) embedding, $\mathbf{v}_c$ is the output (context) embedding, $\sigma(x) = 1/(1+e^{-x})$, and the negative samples c_k are drawn from a noise distribution $P_n(w) \propto U(w)^{3/4}$ (subsampling of frequent words).

CBOW (Continuous Bag of Words): Predicts the center word from the average of context embeddings:

$$\mathbf{h} = \frac{1}{2m} \sum_{-m \le j \le m, j \neq 0} \mathbf{v}_{w+j}, \qquad \mathcal{L} = -\log \frac{\exp(\mathbf{u}_w^\top \mathbf{h})}{\sum_{w'} \exp(\mathbf{u}_{w'}^\top \mathbf{h})}$$

where m is the context window half-width and $\mathbf{u}_w$ are output embeddings. CBOW is faster than Skip-Gram but generally produces slightly lower-quality embeddings for rare words.

Usage: Large-scale unsupervised word representation learning from raw text corpora (Mikolov et al., 2013). Pre-trained embeddings serve as initialization for downstream NLP models. The **analogy** property is famous: $\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}$.

Pros: Computationally efficient (negative sampling avoids full softmax). Captures semantic and syntactic relationships in vector arithmetic. Scales to billion-word corpora. **Cons:** One fixed vector per word—no handling of polysemy ("bank" as river vs. financial). Requires large, high-quality corpora. Ignores word morphology and sub-word structure. Context window limits capture of long-range dependencies.

20.1.3 GloVe (Global Vectors)

GloVe (Pennington et al., 2014) factorizes the log co-occurrence matrix using weighted least squares:

$$\mathcal{L} = \sum_{i,j=1}^{V} f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

where X_{ij} is the co-occurrence count of word i and word j, $\mathbf{w}_i$ and $\tilde{\mathbf{w}}_j$ are word and context embeddings, $b_i, \tilde{b}_j$ are scalar biases, and $f(x) = \min\left(1, (x/x_{\max})^\alpha\right)$ is a weighting function that prevents dominance of very frequent word pairs (typical $\alpha = 3/4$, $x_{\max} = 100$).

Why the log co-occurrence? The ratio $P(k|i)/P(k|j)$ of co-occurrence probabilities captures meaning distinctions. GloVe encodes this by modeling $\mathbf{w}_i^\top \tilde{\mathbf{w}}_k \approx \log P(k|i) = \log X_{ik} - \log X_i$, so that differences between embedding dot products approximate log-probability ratios.

Usage: Pre-training word vectors on large corpora (Common Crawl, Wikipedia). The resulting embeddings combine the benefits of count-based methods (global statistics) and prediction-based methods (local context). Used as input features for NLP models before contextual embeddings became dominant.

Pros: Leverages global co-occurrence statistics efficiently. Faster training than Word2Vec for comparable quality on large corpora. The embeddings are often more interpretable. **Cons:** Requires constructing and storing the co-occurrence matrix ($\propto V^2$, though sparse). Still produces one fixed vector per word type (no polysemy). Sensitive to preprocessing choices (tokenization, window size, vocabulary cutoff).

20.1.4 FastText

FastText (Bojanowski et al., 2017) represents each word as the sum of its character n-gram embeddings. For a word w with n-gram set $\mathcal{G}_w$ (including the word itself as a special token):

$$\mathbf{v}_w = \frac{1}{|\mathcal{G}_w|} \sum_{g \in \mathcal{G}_w} \mathbf{z}_g$$

where $\mathbf{z}_g$ is the embedding of character n-gram g. Typical settings: $n \in \{3, 4, 5, 6\}$. The Skip-Gram objective is then applied to these composed word vectors, with negative sampling.

Out-of-vocabulary advantage: For an unseen word w_{new}, FastText can construct an approximate embedding from its character n-grams:

$$\mathbf{v}_{w_{\text{new}}} = \frac{1}{|\mathcal{G}_{w_{\text{new}}}|} \sum_{g \in \mathcal{G}_{w_{\text{new}}}} \mathbf{z}_g$$

This captures morphological patterns: "playing," "played," and "plays" share n-grams, so their embeddings will be similar.

Usage: Text classification in morphologically rich languages, handling rare and misspelled words, and out-of-vocabulary generalization. Standard for language identification, sentiment analysis on noisy text, and as a strong baseline for word-level tasks.

Pros: Handles out-of-vocabulary words natively via subword composition. Captures morphological regularities across related word forms. Small model size (only n-gram embeddings stored). **Cons:** Embeddings are context-independent (same issue as Word2Vec/GloVe). Character-level composition can group unrelated words that happen to share n-grams. Requires more memory than Word2Vec when the number of n-grams is large.

20.2 Contextual Embeddings

20.2.1 ELMo (Embeddings from Language Models)

ELMo (Peters et al., 2018) computes a task-specific weighted combination of hidden states from all layers of a bidirectional LSTM language model:

$$\text{ELMo}_k^{\text{task}} = \gamma^{\text{task}} \sum_{j=0}^{L} s_j^{\text{task}} \, \mathbf{h}_{k,j}^{\text{LM}}$$

where:

- $\mathbf{h}_{k,0}^{\text{LM}}$ is the token embedding layer (non-contextual).
- $\mathbf{h}_{k,j}^{\text{LM}}$ ($j \geq 1$) is the concatenated forward and backward LSTM hidden states at layer j for token k: $\mathbf{h}_{k,j}^{\text{LM}} = [\overrightarrow{\mathbf{h}}_{k,j}; \overleftarrow{\mathbf{h}}_{k,j}]$.
- s_j^{task} are learned per-layer softmax-normalized weights.
- γ^{task} is a learned scalar scaling factor.

Layer-wise information: Lower layers capture syntax (POS tagging, parse depth); higher layers capture semantics (word sense, coreference). The weighted sum allows each downstream task to select the appropriate level of abstraction, and γ is practically important for scaling the entire ELMo vector to match the task's activation distribution.

Usage: ELMo vectors are concatenated or added to the input representations of downstream NLP models (named entity recognition, question answering, textual entailment). At its release, it set state-of-the-art on six major NLP benchmarks.

Pros: Fully contextualized word representations. Different layers capture different linguistic phenomena; task can learn which layers to attend to. **Cons:** Requires a pre-trained bidirectional LM (two separate passes). Feature-based (not fine-tuned end-to-end for most downstream architectures). LSTM-based rather than transformer-based, limiting long-range context capture. Largely superseded by BERT-style approaches.

20.2.2 BERT and LLM Contextual Embeddings

BERT (Devlin et al., 2019) produces token-level contextual embeddings via a stack of transformer encoder layers trained with masked language modeling (MLM) and next-sentence prediction (NSP). For an input sequence $\mathbf{x} = [\texttt{[CLS]}, x_1, \ldots, x_n, \texttt{[SEP]}]$, the output at layer L is:

$$\mathbf{H}^{(L)} = \text{TransformerEncoder}^{(L)}(\text{TransformerEncoder}^{(L-1)}(\ldots(\mathbf{E}(\mathbf{x}))))$$

where $\mathbf{E}(\mathbf{x})$ includes token, positional, and segment embeddings. The contextual embedding for token i is $\mathbf{h}_i^{(L)} \in \mathbb{R}^d$.

Decoder-only LLMs (GPT, LLaMA, etc.) produce causal contextual embeddings via autoregressive transformer decoders:

$$\mathbf{h}_t = \text{TransformerDecoder}(\mathbf{h}_{<t}), \qquad \mathbf{h}_t \in \mathbb{R}^d$$

Each token's embedding depends only on preceding tokens due to the causal attention mask. The final hidden state $\mathbf{h}_T$ can be pooled or directly used for downstream tasks.

Usage: BERT for text classification, named entity recognition, question answering (fine-tuning the pre-trained model). GPT/LLaMA for text generation, few-shot in-context learning, and as the foundation for chat models. The token embeddings can be extracted from intermediate layers and used as features for lightweight classifiers.

Pros: Deeply contextualized representations capture syntax, semantics, world knowledge, and long-range dependencies. Transfer learning via fine-tuning achieves state-of-the-art across NLP. **Cons:** Computationally expensive ($O(n^2)$ attention over n tokens). Tokenizer-dependent: embeddings change with different tokenization schemes. Context window is limited (BERT: 512 tokens; though modern LLMs extend to 32K–1M). Static extraction (freezing the base model) yields lower performance than full fine-tuning.

20.2.3 Sentence Embeddings

Sentence-BERT (Reimers & Gurevych, 2019): Fine-tunes BERT with a siamese/triplet network structure to produce semantically meaningful sentence embeddings. Mean pooling over token outputs:

$$\mathbf{s} = \text{mean-pool}(\{\mathbf{h}_1^{(L)}, \ldots, \mathbf{h}_n^{(L)}\}) \quad \text{or} \quad \mathbf{s} = \mathbf{h}_{\texttt{[CLS]}}^{(L)}$$

Training uses cosine-similarity loss for pairs, or triplet loss with margin: $\max(0, \|\mathbf{s}_a - \mathbf{s}_p\| - \|\mathbf{s}_a - \mathbf{s}_n\| + \epsilon)$.

SimCSE (Gao et al., 2021): A contrastive framework using dropout as data augmentation. The same sentence is passed through the encoder twice with different dropout masks, producing two embeddings $(\mathbf{s}_i, \mathbf{s}_i^+)$ that are treated as a positive pair:

$$\mathcal{L} = -\sum_{i=1}^{N} \log \frac{\exp(\text{sim}(\mathbf{s}_i, \mathbf{s}_i^+)/\tau)}{\sum_{j=1}^{N} \exp(\text{sim}(\mathbf{s}_i, \mathbf{s}_j^+)/\tau)}$$

where τ is a temperature parameter and sim is cosine similarity. No labeled data required—purely unsupervised.

Usage: Semantic textual similarity (STS), information retrieval, clustering, duplicate detection, and as the embedding backbone for RAG (Retrieval-Augmented Generation) systems. Sentence-BERT embeddings can be compared using cosine similarity with standard vector databases (FAISS, Annoy, Pinecone).

Pros: Fixed-size representations for variable-length text. Efficient semantic comparison via dot product/cosine similarity. SimCSE achieves strong unsupervised performance without labeled pairs. **Cons:** Average pooling over token embeddings loses word-order information beyond what the transformer captured. Sentence embeddings are task-dependent; an embedding optimized for similarity may not work well for classification.

20.3 Knowledge Graph Embeddings

20.3.1 Translational Models: TransE, TransH, TransR

TransE (Bordes et al., 2013): Models a relation r as a translation vector from head entity h to tail entity t:

$$\mathbf{h} + \mathbf{r} \approx \mathbf{t}$$

Score function: $f(h, r, t) = -\|\mathbf{h} + \mathbf{r} - \mathbf{t}\|_2$. Training uses margin-based ranking loss:

$$\mathcal{L} = \sum_{(h,r,t)\in\mathcal{S}} \sum_{(h',r,t')\in\mathcal{S}'} \max\big(0, \gamma + f(h, r, t) - f(h', r, t')\big)$$

where $\mathcal{S}'$ is a set of corrupted triples (head or tail replaced with random entity) and $\gamma > 0$ is the margin.

TransH (Wang et al., 2014): Projects entities onto a relation-specific hyperplane with normal vector $\mathbf{w}_r$:

$$\mathbf{h}_\perp = \mathbf{h} - \mathbf{w}_r^\top \mathbf{h}\, \mathbf{w}_r, \quad \mathbf{t}_\perp = \mathbf{t} - \mathbf{w}_r^\top \mathbf{t}\, \mathbf{w}_r$$

Score: $f(h, r, t) = -\|\mathbf{h}_\perp + \mathbf{r} - \mathbf{t}_\perp\|_2$. This allows an entity to play different roles depending on the relation (e.g., "apple" as fruit vs. company).

TransR (Lin et al., 2015): Projects entities from entity space to relation-specific space via a learned projection matrix $\mathbf{M}_r \in \mathbb{R}^{k\times d}$:

$$\mathbf{h}_r = \mathbf{M}_r\mathbf{h}, \qquad \mathbf{t}_r = \mathbf{M}_r\mathbf{t}$$

Score: $f(h, r, t) = -\|\mathbf{h}_r + \mathbf{r} - \mathbf{t}_r\|_2$. TransR has more expressive power but many more parameters (one projection matrix per relation).

Usage: Knowledge graph completion (link prediction), recommender systems, relation extraction. The translational family is prized for its simplicity and interpretability. TransE is the go-to baseline; TransR is used when more expressive relation modeling is needed.

Pros (TransE): Extremely simple (only $O(d)$ parameters per entity and relation). Fast training. Intuitive geometric interpretation. **Cons (TransE):** Cannot model 1-to-N, N-to-1, or symmetric relations well (e.g., for "friend_of," $h + r \approx t$ and $t + r \approx h$ would imply $r \approx 0$). TransH/TransR partly fix this at increased parameter cost.

20.3.2 Bilinear and Rotation Models: DistMult, ComplEx, RotatE

DistMult (Yang et al., 2015): Models relations as diagonal matrices and scores triples via a bilinear form:

$$f(h, r, t) = \mathbf{h}^\top \operatorname{diag}(\mathbf{r})\, \mathbf{t} = \sum_{i=1}^{d} h_i \cdot r_i \cdot t_i$$

Equivalent to a three-way inner product. Extremely fast but can only model **symmetric** relations since $f(h, r, t) = f(t, r, h)$.

ComplEx (Trouillon et al., 2016): Extends DistMult to complex space. Embeddings $\mathbf{h}, \mathbf{r}, \mathbf{t} \in \mathbb{C}^d$:

$$f(h, r, t) = \text{Re}\big(\mathbf{h}^\top \text{diag}(\mathbf{r})\, \bar{\mathbf{t}}\big)$$

where $\text{Re}(\cdot)$ takes the real part and $\bar{\mathbf{t}}$ is the complex conjugate. The complex-valued embeddings enable modeling **asymmetric** relations while retaining efficient computation.

RotatE (Sun et al., 2019): Models relations as rotations in complex space. Each embedding dimension is a complex number on the unit circle:

$$\mathbf{h}, \mathbf{r}, \mathbf{t} \in \mathbb{C}^d, \quad |r_i| = 1 \; \forall i$$

Score: $f(h, r, t) = -\|\mathbf{h} \odot \mathbf{r} - \mathbf{t}\|_2$, where $\odot$ is element-wise complex multiplication. Since $|r_i| = 1$, multiplying by $\mathbf{r}$ is a pure rotation. RotatE can model **symmetry** ($r = e^{i\pi} = -1$), **inversion** ($r_1 r_2 = 1$), and **composition** ($r_1 r_2 = r_3$).

Usage: Knowledge graph completion benchmarks (WN18RR, FB15k-237, YAGO3-10). DistMult is the fastest baseline; ComplEx adds asymmetry; RotatE provides the most expressive geometric inductive biases (symmetry, antisymmetry, inversion, composition).

Pros (RotatE): High expressiveness with only $O(d)$ parameters per entity/relation. Can model all four fundamental relation patterns. **Cons (DistMult):** Cannot model asymmetric relations. **Cons (ComplEx):** Complex values double the effective dimensionality. **Cons (general):** All these models treat entities as single vectors; cannot capture multi-hop or graph structural features directly.

20.4 Graph Node Embeddings

20.4.1 Node2Vec and DeepWalk

Node2Vec (Grover & Leskovec, 2016) generates node sequences via biased random walks, then applies Word2Vec Skip-Gram to learn node embeddings. The walk transition probability from node v to x given previous node t is:

$$\alpha_{pq}(t, x) = \begin{cases} 1/p & \text{if } d_{tx} = 0 \text{ (return to } t\text{)} \\ 1 & \text{if } d_{tx} = 1 \text{ (1-hop neighbor)} \\ 1/q & \text{if } d_{tx} = 2 \text{ (2-hop neighbor)} \end{cases}$$

where d_{tx} is the shortest-path distance and p, q control **return** (p) and **in-out** (q) bias. DeepWalk (Perozzi et al., 2014) is equivalent to Node2Vec with $p = q = 1$ (uniform random walks).

Walk interpretation:

- Low $p \to$ walks stay local (DFS-like, structural equivalence).
- Low $q \to$ walks explore outward (BFS-like, homophily).
- The Skip-Gram objective on generated walks makes nodes that co-occur in walks

have similar embeddings.

Usage: Node classification, link prediction, community detection in social networks, citation graphs, and biological networks. Node2Vec is the classic baseline for graph representation learning before GNNs.

Pros: Captures both homophily (BFS) and structural equivalence (DFS) by tuning p, q. Scalable: random walks are embarrassingly parallel. No node features required—works on pure topology. **Cons:** Transductive: new nodes cannot be embedded without retraining. Ignores node/edge attributes. Random walk length and number of walks are additional hyperparameters. Cannot handle edge types (heterogeneous graphs) natively.

20.4.2 GraphSAGE

GraphSAGE (Hamilton et al., 2017) learns **inductive** node embeddings by aggregating features from a node's local neighborhood:

$$\mathbf{h}_v^{(l)} = \sigma\Big(\mathbf{W}^{(l)} \cdot \text{AGGREGATE}_l\big(\{\mathbf{h}_u^{(l-1)} : u \in \mathcal{N}(v)\}\big) \oplus \mathbf{B}^{(l)}\mathbf{h}_v^{(l-1)}\Big)$$

where $\oplus$ denotes concatenation and the aggregation function can be:

- **Mean:** $\frac{1}{|\mathcal{N}(v)|}\sum_{u\in\mathcal{N}(v)} \mathbf{h}_u^{(l-1)}$.
- **LSTM:** Apply an LSTM to a random permutation of neighbors.
- **Pooling:** $\max\big(\{\sigma(\mathbf{W}\mathbf{h}_u + \mathbf{b}) : u \in \mathcal{N}(v)\}\big)$.

The final embedding $\mathbf{z}_v = \mathbf{h}_v^{(L)}$ is used for node classification or link prediction.

Usage: Large-scale graphs with rich node features (text, images, categorical attributes). Inductive learning enables embedding unseen nodes at test time, which is critical for dynamic graphs (e.g., new users in a social network, new papers in a citation graph).

Pros: Inductive—embeds unseen nodes using their features and local neighborhood. Leverages node features (unlike Node2Vec). Sampled neighbors provide scalability to billion-node graphs. **Cons:** Requires node features (cannot embed featureless graphs). Depth L is limited by the neighborhood explosion problem. The aggregation function choice is architecture-dependent with no clear winner across domains.

20.5 Collaborative Filtering Embeddings

20.5.1 Matrix Factorization (SVD, ALS)

Matrix Factorization decomposes the user-item interaction matrix $\mathbf{R} \in \mathbb{R}^{m\times n}$ into user embeddings $\mathbf{U} \in \mathbb{R}^{m\times d}$ and item embeddings $\mathbf{V} \in \mathbb{R}^{n\times d}$:

$$\mathbf{R} \approx \mathbf{U}\mathbf{V}^\top, \qquad \hat{r}_{ui} = \mathbf{u}_u^\top \mathbf{v}_i$$

Training minimizes the regularized squared error over observed interactions $\mathcal{K}$:

$$\min_{\mathbf{U},\mathbf{V}} \sum_{(u,i)\in\mathcal{K}} \left(r_{ui} - \mathbf{u}_u^\top \mathbf{v}_i\right)^2 + \lambda\left(\|\mathbf{u}_u\|^2 + \|\mathbf{v}_i\|^2\right)$$

Optimized via alternating least squares (ALS) or stochastic gradient descent. ALS alternates between fixing $\mathbf{V}$ and solving a linear system for $\mathbf{U}$, and vice versa.

Usage: Recommender systems (Netflix Prize baseline), collaborative filtering for e-commerce, content recommendation. ALS is preferred when the interaction matrix is large and sparse, as each step solves many small independent least-squares problems.

Pros: Simple, interpretable (dot product between user/item vectors). ALS parallelizes naturally (independent per-user and per-item updates). **Cons:** Cannot model feature interactions or incorporate side information natively. The dot-product limitation restricts expressiveness (cannot capture non-linear user-item interactions). Cold-start problem: new users/items have no learned embedding.

20.5.2 Factorization Machines (FM)

Factorization Machines (Rendle, 2010) model all pairwise feature interactions using factorized parameters:

$$\hat{y}(\mathbf{x}) = w_0 + \sum_{i=1}^{n} w_i x_i + \sum_{i=1}^{n} \sum_{j=i+1}^{n} \langle \mathbf{v}_i, \mathbf{v}_j \rangle \, x_i x_j$$

where $\mathbf{v}_i \in \mathbb{R}^d$ is the factorized embedding for feature i and $\langle \cdot, \cdot \rangle$ is the dot product. The key insight: $\langle \mathbf{v}_i, \mathbf{v}_j \rangle$ can be estimated even for feature pairs (i, j) that never co-occur in training, because each feature has its own embedding.

Efficient computation: The pairwise term can be rewritten as:

$$\sum_{i=1}^{n} \sum_{j=i+1}^{n} \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j = \frac{1}{2} \sum_{f=1}^{d} \left(\left(\sum_{i=1}^{n} v_{if} x_i \right)^2 - \sum_{i=1}^{n} v_{if}^2 x_i^2 \right)$$

This reduces complexity from $O(kn^2)$ to $O(kn)$.

Usage: Click-through rate (CTR) prediction, recommendation with side features (user age, item category, context), and any tabular learning with high-cardinality categorical features. FMs generalize matrix factorization by allowing arbitrary feature vectors (not just user and item IDs).

Pros: Handles sparse feature interactions in linear time. Works with any real-valued feature vector. Strong baseline for recommendation and CTR tasks. **Cons:** Limited to second-order (pairwise) interactions only. Higher-order FMs exist but add significant

complexity. DeepFM, xDeepFM, and other neural variants often outperform vanilla FM by modeling higher-order interactions.

20.6 Vector Quantization for Embeddings

20.6.1 VQ-VAE Codebook Learning

VQ-VAE (van den Oord et al., 2017) learns a discrete latent representation by mapping continuous encoder outputs to the nearest codebook vector:

$$\mathbf{z}_q(\mathbf{x}) = \mathbf{e}_k, \quad \text{where} \quad k = \arg\min_j \|\mathbf{z}_e(\mathbf{x}) - \mathbf{e}_j\|_2$$

The codebook $\{\mathbf{e}_1, \ldots, \mathbf{e}_K\} \subset \mathbb{R}^d$ is learned jointly. The straight-through estimator copies gradients from $\mathbf{z}_q$ to $\mathbf{z}_e$:

$$\mathcal{L}_{\text{VQ}} = \|\,\text{sg}[\mathbf{z}_e] - \mathbf{e}\|_2^2 + \beta\|\mathbf{z}_e - \text{sg}[\mathbf{e}]\|_2^2$$

where $\text{sg}[\cdot]$ is the stop-gradient operator.

Usage: Discrete representation learning for images (VQ-VAE), audio (VQ-VAE-2 for raw audio), and video. VQ-VAE encodings serve as tokens for autoregressive transformers (VQ-GAN, DALL-E, Stable Diffusion's VAE). The discrete codebook is critical for bridging continuous and discrete modalities.

Pros: Discrete latent space enables autoregressive generation with transformers. Codebook acts as an information bottleneck that removes imperceptible details. **Cons:** Codebook collapse: many codes go unused ("dead" codes) because only a few codebook vectors attract encoder outputs. The straight-through gradient approximation is biased. Training requires careful tuning of β and codebook size K.

20.6.2 Product Quantization (PQ) and Residual Quantization (RQ)

Product Quantization (Jégou et al., 2011) splits a d-dimensional vector into M subvectors of dimension d/M, each quantized by its own codebook:

$$\mathbf{x} = [\mathbf{x}^{(1)}; \ldots; \mathbf{x}^{(M)}], \qquad \mathbf{x}^{(m)} \in \mathbb{R}^{d/M}$$

$$q(\mathbf{x}) = [q_1(\mathbf{x}^{(1)}); \ldots; q_M(\mathbf{x}^{(M)})], \qquad q_m(\mathbf{x}^{(m)}) = \arg\min_{\mathbf{c} \in \mathcal{C}_m} \|\mathbf{x}^{(m)} - \mathbf{c}\|_2$$

With M sub-codebooks of size K each, PQ represents K^M distinct centroids using only $M \times K \times (d/M)$ parameters.

Residual Quantization recursively quantizes the residual error:

$$\mathbf{r}_0 = \mathbf{x}$$
$$\mathbf{c}_i = \arg\min_{\mathbf{c} \in \mathcal{C}^{(i)}} \|\mathbf{r}_{i-1} - \mathbf{c}\|_2, \quad \mathbf{r}_i = \mathbf{r}_{i-1} - \mathbf{c}_i$$

After N stages, $\hat{\mathbf{x}} = \sum_{i=1}^{N} \mathbf{c}_i$. This is equivalent to a tree-structured quantization with exponentially many centroids.

Usage: Approximate nearest-neighbor search (FAISS), compression of dense embeddings for retrieval, model compression. PQ is used in FAISS IndexPQ for billion-scale vector search; RQ provides higher accuracy at the same bitrate. Both are essential for deploying large embedding-based systems.

Pros (PQ): Extremely memory-efficient: a d-dim vector compressed to $M \log_2 K$ bits. Search in compressed domain via precomputed distance tables (ADC). **Pros (RQ):** Higher recall than PQ at same bitrate; more flexible bit allocation. **Cons:** Quantization error limits recall. Choosing optimal M and K requires empirical tuning. PQ assumes sub-vector independence—correlated dimensions are not modeled jointly.

20.7 Locality-Sensitive Hashing (LSH) for Embeddings

LSH is a family of hash functions $\mathcal{H}$ such that:

$$\mathbb{P}_{\mathbf{h}\sim\mathcal{H}}[\mathbf{h}(\mathbf{x}) = \mathbf{h}(\mathbf{y})] = \text{sim}(\mathbf{x}, \mathbf{y})$$

where sim is a similarity function. The collision probability is monotonic in the similarity—similar items hash to the same bucket with high probability.
Random projection LSH for cosine similarity uses random hyperplanes: $\mathbf{h}(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x})$ where $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Then $\mathbb{P}[\mathbf{h}(\mathbf{x}) = \mathbf{h}(\mathbf{y})] = 1 - \frac{1}{\pi} \arccos(\text{sim}_{\cos}(\mathbf{x}, \mathbf{y}))$. Using b bits (concatenated hashes), the collision probability decays as a power of the cosine.

Usage: Approximate nearest-neighbor search at scale, duplicate detection, document clustering, and as a building block for attention mechanisms (Reformer LSH-attention, where attention is computed only within hash buckets to reduce $O(n^2)$ cost to $O(n \log n)$). Also used in learned LSH (data-dependent hash functions).

Pros: Sublinear query time (hash table lookup). Simple to implement and parallelize. Theoretical guarantees on collision probability. **Cons:** Requires many hash tables/tables for high recall. Random projections are data-independent—learned hashing can outperform. Accuracy-recall tradeoff controlled by the number of bits and hash tables. Not suitable as a general embedding; only for retrieval/similarity search.

20.8 Cross-Modal Contrastive Embeddings

20.8.1 CLIP and SigLIP

CLIP (Radford et al., 2021) trains an image encoder $f_{\boldsymbol{\theta}}$ and a text encoder $g_{\boldsymbol{\phi}}$ to maximize the cosine similarity of matching image-text pairs while minimizing it for non-matching pairs via a symmetric InfoNCE loss over a batch of N pairs:

$$\mathcal{L} = -\frac{1}{2N}\sum_{i=1}^{N}\Big[\log\frac{\exp(\tau\cdot\text{sim}(I_i,T_i))}{\sum_{j=1}^{N}\exp(\tau\cdot\text{sim}(I_i,T_j))} + \log\frac{\exp(\tau\cdot\text{sim}(T_i,I_i))}{\sum_{j=1}^{N}\exp(\tau\cdot\text{sim}(T_i,I_j))}\Big]$$

where $\text{sim}(\mathbf{u},\mathbf{v}) = \frac{\mathbf{u}^\top\mathbf{v}}{\|\mathbf{u}\|_2\|\mathbf{v}\|_2}$ and τ is a learnable temperature scaling parameter.

SigLIP (Zhai et al., 2023) replaces the softmax-based contrastive loss with a sigmoid-based pairwise loss:

$$\mathcal{L} = -\frac{1}{N^2}\sum_{i=1}^{N}\sum_{j=1}^{N}\log\frac{1}{1+\exp\big(z_{ij}(-t\cdot\mathbf{x}_i^\top\mathbf{y}_j+b)\big)}$$

where $z_{ij}=1$ if $i=j$ (positive pair) and $z_{ij}=-1$ otherwise. This operates on every pair independently, enabling substantially larger batch sizes and more stable training.

Usage: Zero-shot image classification, text-to-image retrieval, visual question answering. CLIP embeddings power DALL-E 2/3, Stable Diffusion (text conditioning), and Midjourney. SigLIP improves training efficiency and is adopted in recent multimodal LLMs (Gemini, PaLI).

Pros (CLIP): Remarkable zero-shot transfer across diverse vision tasks. Joint embedding space enables text-guided image manipulation. **Pros (SigLIP):** Higher throughput (no softmax normalization bottleneck). Better memory scaling. **Cons (CLIP):** Requires massive paired datasets (400M+ image-text pairs). Batch size is critical—too small batches make the contrastive signal weak. Struggles with fine-grained or compositional understanding. Compute-expensive to train.

20.8.2 ALIGN and ImageBind

ALIGN (Jia et al., 2021) uses a dual-encoder architecture with global contrastive loss similar to CLIP but trained on noisier, larger-scale alt-text data (∼1.8B pairs). The key insight: noisy supervision at massive scale compensates for label noise. Both image and text encoders are large (EfficientNet-L2 and BERT-Large).

ImageBind (Girdhar et al., 2023) learns a joint embedding space across **six modalities** (image, text, audio, depth, thermal, IMU) using image as the binding modality:

$$\mathcal{L} = \sum_{(M_1,M_2)\in\mathcal{P}}\mathcal{L}_{\text{contrastive}}(f_{M_1}(\mathbf{x}^{M_1}), f_{M_2}(\mathbf{x}^{M_2}))$$

where $\mathcal{P} = \{(\text{image}, M) : M \in \{\text{text, audio, depth, thermal, IMU}\}\}$. All non-image modalities align to the image embedding space, enabling emergent zero-shot cross-modal

retrieval between non-image modalities (e.g., audio → text) without explicit audio-text pairs.

Usage: Multimodal retrieval, zero-shot classification across modalities, and as a general-purpose embedding backbone for applications requiring cross-modal understanding. ImageBind's emergent alignment enables novel interfaces (e.g., generating images from sound).

Pros (ImageBind): Single shared space for six modalities; emergent cross-modal transfer. **Cons (ImageBind):** All modalities must align through the image modality—modality pairs without image as intermediary may suffer. Training requires paired data for each modality with images. **Pros (ALIGN):** Scale compensates for noise; strong zero-shot results. **Cons (ALIGN):** Requires even larger datasets than CLIP; noisy alt-text can introduce harmful biases.

20.9 Mixture of Experts Embedding Gating

In Mixture of Experts (MoE) architectures, the embedding layer can be gated to route tokens to specialized expert sub-embeddings:

$$\mathbf{y} = \sum_{i=1}^{n} G(\mathbf{x})_i \, E_i(\mathbf{x})$$

where $G : \mathbb{R}^d \to \Delta^{n-1}$ is the gating function (a softmax over expert scores) and $E_i : \mathbb{R}^d \to \mathbb{R}^{d_{\text{out}}}$ is the i-th expert sub-network. Commonly, the gating is sparsified (top-k routing):

$$G(\mathbf{x}) = \text{softmax}\big(\text{TopK}(\mathbf{W}_g\mathbf{x}, k)\big)$$

where TopK keeps only the k largest logits and sets the rest to $-\infty$. The output embedding is a sparse weighted combination of only k expert outputs.

Usage: Massive-scale language models (Mixtral 8×7B, GPT-4, GLaM). MoE embeddings decouple model capacity from computation by activating only a fraction of parameters per token. Also used in multi-task embedding systems where different expert spaces specialize to different domains (web, news, commerce, scientific text).

Pros: Exponential capacity increase with linear compute budget (only active experts are computed). Experts can naturally specialize to different domains or token types. **Cons:** Load balancing is critical—the gating network may collapse to always routing to the same few experts. Auxiliary load-balancing losses are required. Communication overhead in distributed training (experts on different devices). Hard to tune: routing strategy, number of experts, and top-k are interdependent hyperparameters.

20.10 Summary Comparison Table

Embedding selection guide:

1. **Text (static words):** FastText for morphologically rich languages; GloVe for efficient pre-training; Word2Vec as classic baseline.
2. **Text (contextual):** BERT/RoBERTa for classification and NER; Sentence-BERT for semantic similarity and retrieval; SimCSE for unsupervised sentence embeddings.
3. **Text (generation):** Decoder-only LLM embeddings (GPT, LLaMA) for generation and in-context learning.
4. **Knowledge Graphs:** TransE as baseline; ComplEx or RotatE for higher accuracy (asymmetric and compositional relations).
5. **Graphs:** Node2Vec for pure-topology transductive tasks; GraphSAGE for inductive tasks with node features.
6. **Recommendation:** Matrix factorization for pure collaborative filtering; Factorization Machines when side features are available.
7. **Image-text:** CLIP for zero-shot and retrieval; SigLIP for efficient large-scale training.
8. **Compression / retrieval:** PQ for nearest-neighbor search at scale; RQ for higher accuracy at fixed bitrate.
9. **Discrete representations:** VQ-VAE for bridging continuous and discrete modalities; useful as tokenizer for autoregressive generation.

Table 20.1: Comprehensive Comparison of Embedding Methods

Method	**Contextual?**	**Domain**	**Year**	**Key Idea**
One-Hot	No	Any	—	Sparse orthogonal coding
Word2Vec	No	Text	2013	Skip-Gram + negative sampling
GloVe	No	Text	2014	Weighted factorization of log co-occurrence
FastText	No	Text	2017	Character n-gram sub-word composition
ELMo	Yes	Text	2018	Weighted sum of bidirectional LSTM layers
BERT/LLM	Yes	Text	2019+	Deep transformer contextualization
Sent-BERT/SimCSE	Yes	Text	2019/21	Contrastive sentence embeddings
TransE	No	KG	2013	$\mathbf{h} + \mathbf{r} \approx \mathbf{t}$ translation
DistMult	No	KG	2015	Bilinear diagonal scoring
ComplEx	No	KG	2016	Complex-valued bilinear (handles asymmetry)
RotatE	No	KG	2019	Relation as rotation in complex space
Node2Vec	No	Graphs	2016	Biased random walks + Skip-Gram
GraphSAGE	Inductive	Graphs	2017	Neighborhood aggregation with features
Matrix Factorization	No	CF	2000s	$\mathbf{UV}^\top$ factorization of ratings
Factorization Machines	No	CF/Tabular	2010	Factorized pairwise feature interactions
VQ-VAE	No	Continuous	2017	Discrete codebook via nearest-neighbor lookup
Product Quant.	No	Any	2011	Split and quantize sub-vectors independently
Residual Quant.	No	Any	—	Recursively quantize residuals
LSH	No	Any	1990s	Random projections for similarity-preserving hashing
CLIP	Joint	Image+Text	2021	Contrastive dual-encoder on 400M pairs
SigLIP	Joint	Image+Text	2023	Sigmoid pairwise loss (replaces softmax)
ImageBind	Joint	6 modalities	2023	Image as binding modality for cross-modal space
MoE Gating	Conditional	Any	2017+	Sparse routing to expert sub-embeddings

Chapter 21

Positional Encoding

Transformers process tokens in parallel and are, by design, permutation-invariant: without additional structure, shuffling the input sequence yields the identical output (up to permutation). Positional encoding injects *order* into self-attention, enabling the model to distinguish "dog bites man" from "man bites dog." This chapter surveys the family of positional encoding (PE) schemes—absolute, relative, rotary, and learned—along with extensions to higher dimensions and PE-free architectures.

21.1 Why Positional Encoding

Permutation Invariance of Transformers

A function f is **permutation equivariant** if $f(PX) = Pf(X)$ for any permutation matrix P. Self-attention satisfies this:

$$\text{Attention}(PQ,\, PK,\, PV) = P \cdot \text{Attention}(Q, K, V)$$

For a set-based task (e.g., point clouds), this is an asset. For sequences, it is a fatal omission: without positional information, the model cannot learn n-gram patterns, word order, or any sequential dependency.

Desiderata for a Good Positional Encoding

1. **Uniqueness:** Each position should have a distinct encoding.
2. **Boundedness:** Values should not explode with sequence length.
3. **Distance awareness:** The encoding should allow the model to attend to relative positions easily.
4. **Extrapolation:** Should generalize to sequence lengths unseen during training.
5. **Deterministic:** Ideally, the encoding is a fixed function of position, not learned (though learned embeddings work in practice).

21.2 Absolute Sinusoidal Positional Encoding (Vaswani et al.)

Sinusoidal PE

The original Transformer uses a fixed sinusoidal pattern:

$$PE_{(pos,\,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \qquad PE_{(pos,\,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

where $pos \in \{0, \ldots, n-1\}$ is the token position and $i \in \{0, \ldots, d_{\text{model}}/2 - 1\}$ indexes the dimension. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$, resembling a set of sinusoids over different frequencies.

Rotation Matrix Interpretation

For a given frequency $\omega_k = 1/10000^{2k/d}$, the pair of dimensions $(2k, 2k+1)$ encodes position via a **rotation**:

$$\begin{pmatrix} PE_{(pos,\,2k)} \\ PE_{(pos,\,2k+1)} \end{pmatrix} = \begin{pmatrix} \cos(pos \cdot \omega_k) \\ \sin(pos \cdot \omega_k) \end{pmatrix}$$

which is the first column of the 2D rotation matrix $\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$ applied to a constant basis vector. Consequently, for any fixed offset Δ:

$$\begin{pmatrix} PE_{(pos+\Delta,\,2k)} \\ PE_{(pos+\Delta,\,2k+1)} \end{pmatrix} = \begin{pmatrix} \cos(\Delta \cdot \omega_k) & -\sin(\Delta \cdot \omega_k) \\ \sin(\Delta \cdot \omega_k) & \cos(\Delta \cdot \omega_k) \end{pmatrix} \begin{pmatrix} PE_{(pos,\,2k)} \\ PE_{(pos,\,2k+1)} \end{pmatrix}$$

That is, $PE(pos+\Delta) = \mathbf{M}_k(\Delta) \cdot PE(pos)$ for a learnable linear transformation $\mathbf{M}_k$. This means relative position Δ can be expressed as a *linear function* of the encoding, enabling attention to learn relative distance without parameterizing it explicitly.

Sinusoidal PE Trade-offs

Pros:

- Deterministic and closed-form; no learnable parameters.
- Easily extrapolates to longer sequences than seen in training.
- The linear relative-position property allows attention to deduce Δ without extra parameters.
- Bounded in $[-1, 1]$; numerically stable.

Cons:

- High-frequency components are only useful for very short-range interactions; low frequencies dominate for long sequences.
- Some studies find that learned PE slightly outperforms sinusoidal PE when dataset scale is sufficient.
- Only models absolute positions; does not directly inject relative distance into the self-attention score.

21.3 Learned Absolute Position Embeddings

Learned Position Embeddings (BERT

A learnable embedding matrix $W^{\text{pos}} \in \mathbb{R}^{L_{\max} \times d}$ is trained jointly with the rest of the model. The row W^{pos}_{pos} is added to the token embedding:

$$\mathbf{h}_{pos} = \mathbf{x}_{\text{token}} + W^{\text{pos}}_{pos}$$

Unlike sinusoidal PE, these vectors have no predefined mathematical structure.

Used in BERT, GPT-1/2, and many modern LLMs. Simpler to implement and yields slightly better downstream performance in large-data regimes.

Learned PE Trade-offs

Pros: Learnable; can adapt to the data distribution. Slightly better accuracy at scale compared to sinusoidal.
Cons: Fixed maximum length $L_{\max}$; sequences longer than training require heuristics (e.g., interpolation). Number of parameters grows with $L_{\max} \cdot d$ (though negligible relative to total model size).

21.4 Relative Position Bias (Shaw et al., 2018)

Shaw Relative Position Representations

Rather than adding absolute PE to the input, this approach modifies the attention score to include learnable relative position vectors:

$$e_{ij} = \frac{(\mathbf{x}_i W^Q)(\mathbf{x}_j W^K + \mathbf{a}_{ij}^K)^T}{\sqrt{d_k}}$$

where $\mathbf{a}_{ij}^K$ is a learned embedding for the relative distance $\text{clip}(j - i, -k, k)$. A similar term $\mathbf{a}_{ij}^V$ can be added to the value projection. The clipping ensures the number of learned parameters is bounded ($2k + 1$ distinct embeddings).

Used in Music Transformer (Shaw et al., 2018), Transformer-XL (extended to sinusoidal relative position), and various TTS / music generation models.

Shaw Relative PE Trade-offs

Pros: Explicitly encodes relative distance in the attention logits; better inductive bias for sequence tasks. Handles arbitrary sequence lengths (up to the clipping range k).
Cons: Adds $O(L \cdot d_k)$ memory for storing relative position embeddings per attention head. The clipping introduces a hyperparameter k; long-range dependencies beyond $\pm k$ are not distinguished.

21.5 T5 Relative Position Bias

T5 Bucketed Relative Position Bias

T5 simplifies relative encoding by learning a scalar bias per relative distance and attention head:

$$A_{ij} = \frac{Q_i K_j^T}{\sqrt{d_k}} + b_{\text{bucket}(|i-j|)}$$

where bucket$(\cdot)$ maps the relative distance to one of B buckets ($B = 32$ is typical). The bucketing is log-scaled for small distances and linearly scaled for large distances:

$$\text{bucket}(d) = \begin{cases} d & \text{if } d < 8 \\ 8 + \frac{d-8}{2} & \text{if } 8 \le d < 16 \\ \dots & \text{up to maximum bucket} \end{cases}$$

This allocates more resolution to nearby positions (where precision matters) and coarser resolution to far-away positions.

Standard in T5 and many encoder-decoder transformers. Simple to implement (B learnable scalars per head) and effective.

T5 Relative Bias Trade-offs

Pros: Minimal parameter overhead ($B \cdot h$ scalars). Log-scale bucketing provides high resolution for local context. Handles arbitrary sequence lengths seamlessly.
Cons: Only adds a bias (scalar), not a full vector, limiting representational capacity. The bucketing hyperparameters need to be chosen.

21.6 Rotary Position Embedding (RoPE)

RoPE

RoPE encodes absolute position through a rotation applied to the query and key vectors *before* the dot-product:

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \mathbf{R}^d_{\Theta,m}\, \mathbf{W}_{\{q,k\}} \mathbf{x}_m$$

where $\mathbf{R}^d_{\Theta,m}$ is a block-diagonal matrix of 2D rotations:

$$\mathbf{R}^d_{\Theta,m} = \text{diag}\Big(R(m\theta_1),\ R(m\theta_2),\ \dots,\ R(m\theta_{d/2})\Big), \quad R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

with frequencies $\theta_i = 10000^{-2(i-1)/d}$. The key property is that the attention score becomes a function of *relative* position:

$$\mathbf{q}_m^T \mathbf{k}_n = (\mathbf{R}_{\Theta,m} W^Q \mathbf{x}_m)^T (\mathbf{R}_{\Theta,n} W^K \mathbf{x}_n) = (W^Q \mathbf{x}_m)^T \mathbf{R}_{\Theta,n-m} (W^K \mathbf{x}_n)$$

since $\mathbf{R}^T_{\Theta,m} \mathbf{R}_{\Theta,n} = \mathbf{R}_{\Theta,n-m}$.

RoPE Efficient Computation

Instead of constructing the full rotation matrix, RoPE uses a Hadamard-product implementation for efficiency:

$$f_{\{q,k\}}(\mathbf{x}, m) = \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{d-2} \\ x_{d-1} \end{pmatrix} \odot \begin{pmatrix} \cos m\theta_1 \\ \cos m\theta_1 \\ \cos m\theta_2 \\ \cos m\theta_2 \\ \vdots \\ \cos m\theta_{d/2} \\ \cos m\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -x_1 \\ x_0 \\ -x_3 \\ x_2 \\ \vdots \\ -x_{d-1} \\ x_{d-2} \end{pmatrix} \odot \begin{pmatrix} \sin m\theta_1 \\ \sin m\theta_1 \\ \sin m\theta_2 \\ \sin m\theta_2 \\ \vdots \\ \sin m\theta_{d/2} \\ \sin m\theta_{d/2} \end{pmatrix}$$

RoPE is used in LLaMA, LLaMA 2, Mistral, Qwen, and most modern open-source LLMs. It has largely supplanted both sinusoidal and learned PEs in the open-source ecosystem due to its strong length-extrapolation properties.

RoPE Trade-offs

Pros:

- Encodes relative position *implicitly* in the attention dot-product through rotation; no extra bias terms or positional parameters.
- Decays attention with relative distance (proven theoretically for RoPE-based attention).
- Extrapolates to longer sequences better than absolute PE.
- Compatible with linear attention variants.

Cons:

- Slightly more FLOPs than absolute PE (rotation per token pair can be precomputed and cached, but the initial application adds cost).
- Base frequency (10000) is a hyperparameter; suboptimal for contexts beyond 4k tokens without adjustment.
- Long-range decay may be too aggressive for some tasks.

21.7 RoPE Extensions for Longer Contexts

21.7.1 NTK-Aware Scaling

NTK-Aware RoPE Scaling

Neural Tangent Kernel (NTK) theory suggests that extending context length requires adjusting the frequency base. The NTK-aware method scales the base:

$$\theta_i' = \theta_i \cdot \alpha^{-2i/(d-2)}$$

or equivalently, increases the RoPE base Θ while scaling the frequencies such that the "crowded" high-frequency dimensions are "spread out." A simpler heuristic is to increase the base from 10000 to a larger value (e.g., 500000 for CodeLLaMA)—this stretches the frequency spectrum, allowing each frequency to encode a wider range of positions without degeneracy.

21.7.2 YaRN (Yet another RoPE extensioN)

YaRN

YaRN combines NTK-aware scaling with a "ramp" function that applies different scaling factors to different frequency bands:

$$\gamma(m) = 1 - (1 - m/s) \cdot \mathbf{1}(m < s) \quad \text{(ramp function)}$$
$$\theta_i' = \gamma(i) \cdot \theta_i^{\text{NTK}} + (1 - \gamma(i)) \cdot \theta_i^{\text{linear}}$$

where s is the ramp length. Low frequencies (responsible for long-range dependencies) receive linear interpolation; high frequencies (short-range) retain NTK scaling. This yields minimal perplexity degradation when extending LLaMA 2 from 4k to 128k context.

21.7.3 Linear Interpolation

Position Interpolation (PI)

The simplest extension method: linearly interpolate positions when going beyond the training length. For a scale factor $s > 1$, positions are rescaled as $pos' = pos/s$. The RoPE rotation becomes:

$$\theta_i' = 10000^{-2i/d}, \quad \text{applied at position } m' = m/s$$

This is equivalent to down-scaling all frequencies by s. It preserves the shape of the frequency spectrum but reduces all frequencies uniformly, which degradates high-frequency resolution.

Extension Methods Comparison

Position Interpolation: Pros: Simple, no retraining needed.
Cons: Degrades high-frequency resolution; perplexity increases.
NTK-Aware: Pros: Better perplexity than PI, zero-shot extension.
Cons: Requires tuning the scale factor.
YaRN: Pros: Best perplexity-extension trade-off; fine-grained per-frequency control.
Cons: Slightly more complex implementation.

21.8 ALiBi (Attention with Linear Biases)

ALiBi

ALiBi adds **non-learned**, head-specific linear biases to the attention scores, penalizing attention to far-away tokens:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + \mathbf{m} \cdot \mathbf{M}\right) V$$

where $\mathbf{m} = [m_0, m_1, \ldots, m_{h-1}]$ are head-specific slopes (e.g., geometric progression: $m_h = 2^{-8h/H}$ or $1/2^{h/(H-1)+1}$), and $M_{ij} = -(i-j)$ for $i \geq j$ (0 otherwise) is the distance matrix. The bias decreases linearly as the distance grows.

Used in BLOOM (BigScience) and some encoder-only models. ALiBi is attractive because it requires zero additional parameters and naturally extrapolates to longer sequences with no training or fine-tuning.

ALiBi Trade-offs

Pros:

- Zero learnable parameters; purely arithmetic.
- Excellent length extrapolation: BLOOM trained on 2048 tokens infers on 4096+ with minimal degradation.
- Computationally free (a single subtraction per attention pair).

Cons:

- Linear bias is less expressive than sinusoidal or RoPE; some tasks benefit from non-monotonic position effects.
- Slightly underperforms RoPE at equivalent training budgets on many benchmarks.
- The head-specific slope schedule is heuristic.

21.9 xPos

xPos (Improved RoPE for Length Extrapolation)

xPos extends RoPE with a per-dimension exponential decay factor to the query and key before attention, improving long-range stability:

$$\tilde{\mathbf{q}}_m = \boldsymbol{\gamma}^{-m} \odot \mathbf{R}_{\Theta,m} W^Q \mathbf{x}_m, \quad \tilde{\mathbf{k}}_n = \boldsymbol{\gamma}^{n} \odot \mathbf{R}_{\Theta,n} W^K \mathbf{x}_n$$

where $\gamma_d = 1 - 2^{-5} \cdot 2^{-4d/d_{\text{model}}}$ decays with dimension index d, and $\boldsymbol{\gamma}$ is the decay vector applied element-wise. The attention logit becomes:

$$\tilde{\mathbf{q}}_m^T \tilde{\mathbf{k}}_n = \boldsymbol{\gamma}^{n-m} \odot \left[(W^Q \mathbf{x}_m)^T \mathbf{R}_{\Theta,n-m} (W^K \mathbf{x}_n) \right]$$

Used in some long-context language models; xPos achieves strong perplexity on sequences 2–4× longer than training length with minimal fine-tuning.

21.10 2D and 3D Positional Encodings

21.10.1 Sinusoidal 2D

2D Sinusoidal PE

For 2D inputs (images), position is a tuple (h, w). Sinusoidal PE can be extended by encoding row and column independently and concatenating:

$$PE_{(h,w)} = \begin{bmatrix} \sin(h/10000^{0/d_h}) \\ \cos(h/10000^{0/d_h}) \\ \vdots \\ \sin(w/10000^{0/d_w}) \\ \cos(w/10000^{0/d_w}) \\ \vdots \end{bmatrix}$$

where $d_h = d/4$ are dimensions for row and column respectively (each gets half the total dimensions). This ensures both spatial coordinates are encoded.

21.10.2 Learned 2D Patch Embeddings (ViT)

ViT Position Embeddings

Vision Transformer (ViT) learns a separate embedding for each 2D patch position:

$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}};\ \mathbf{x}_p^1\mathbf{E};\ \ldots;\ \mathbf{x}_p^N\mathbf{E}] + \mathbf{E}_{\text{pos}}$$

where $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1)\times d}$ is a learned matrix of N patch positions plus the class token. Each spatial grid position (h, w) maps to a unique row.

21.10.3 iRPE (Image Relative Position Encoding)

iRPE

For vision transformers, iRPE models relative position in 2D using learnable embeddings indexed by the Euclidean or quantized relative coordinate:

$$e_{ij} = \frac{\mathbf{q}_i\mathbf{k}_j^T + \mathbf{r}_{(\Delta h, \Delta w)}}{\sqrt{d_k}}$$

where $\Delta h = h_i - h_j$, $\Delta w = w_i - w_j$. The relative position index is quantized into a fixed vocabulary of 2D offsets. Popular in Swin Transformer and other hierarchical vision transformers.

2D PE Trade-offs

Sinusoidal 2D: Simple, deterministic, extrapolates. Limited for irregular grids.
Learned (ViT): Flexible, data-adaptive. Cannot extrapolate to higher-resolution images without interpolation.
iRPE: Explicit 2D relative modeling, scale-invariant. Higher memory cost for large feature maps.

21.11 Fourier Features (NeRF)

Fourier Feature Encoding

Used in NeRF and implicit neural representations to overcome the spectral bias of ReLU networks toward low frequencies:

$$\gamma(\mathbf{v}) = \left[\sin(2^0\pi\mathbf{v}), \cos(2^0\pi\mathbf{v}), \sin(2^1\pi\mathbf{v}), \cos(2^1\pi\mathbf{v}), \ldots, \sin(2^{L-1}\pi\mathbf{v}), \cos(2^{L-1}\pi\mathbf{v})\right]^T$$

where L is the number of frequency octaves. The input $\mathbf{v}$ is mapped from $\mathbb{R}^d$ to $\mathbb{R}^{2Ld}$ through a set of log-linearly spaced frequency bands. This is functionally a Gaussian RBF kernel approximation via Bochner's theorem.

Core component of NeRF (Mildenhall et al., 2020). Each 3D coordinate (x, y, z) and viewing direction (θ, ϕ) is passed through $\gamma(\cdot)$ before entering the MLP. Standard $L = 10$ for coordinates, $L = 4$ for directions.

Fourier Features Trade-offs

Pros: Enables MLPs to represent high-frequency spatial details (sharp edges, textures). Simple, deterministic, infinitely differentiable.
Cons: Multiplies input dimensionality by $2L$ ($2 \cdot 10 = 20$ for 3D coordinates). Fixed frequency bands may not be optimal for all scenes.

21.12 Multi-resolution Hash Encoding (Instant NGP)

Multi-resolution Hash Grid

Instant NGP (Müller et al., 2022) uses a multi-level hash table of learnable feature vectors at different spatial resolutions:

$$h_l(\mathbf{x}) = \left(\bigoplus_{i=1}^{d} x_i\pi_i\right) \bmod T$$

$$\mathbf{y} = \left[F_1[h_1(\mathbf{x})],\ F_2[h_2(\mathbf{x})],\ \ldots,\ F_L[h_L(\mathbf{x})]\right]$$

where π_i are large primes, $\oplus$ denotes XOR, T is the table size (typically 2^{14}–2^{22}), and F_l is the feature table at level l. Trilinear interpolation blends neighboring hash entries for continuous representations. Each level grid resolution $N_l = N_{\min} \cdot b^l$ grows geometrically from $N_{\min}$ to $N_{\max}$.

State-of-the-art NeRF acceleration (Instant NGP renders in seconds vs. hours) and general implicit neural representation tasks. The learned hash tables act as a learned, multi-resolution positional encoding.

Hash Encoding Trade-offs

Pros: Dramatically reduces MLP size (tiny MLP + hash tables achieves equal quality). Training converges in seconds. Multi-resolution handles fine and coarse detail simultane-

ously.
Cons: Hash collisions introduce noise; table size T is a critical hyperparameter. Less interpretable than Fourier features. Quality degrades under domain shift.

21.13 NoPE — Positional Encoding-Free Architectures

NoPE (No Positional Encoding)

Kazemnejad et al. (2023) demonstrated that decoder-only transformers *can* learn positional information implicitly through the causal attention mask alone —this is dubbed NoPE. In a causal transformer, token position is inherently encoded by *how many tokens precede it*, and the model learns to count through the attention mechanism.

Experiments show that NoPE works well for decoder-only (causal) LLMs at moderate lengths, with some degradation on long-context and position-sensitive tasks. Encoder-only models benefit more from explicit PE.

NoPE Trade-offs

Pros: Zero overhead; no positional parameters or computation. Elegant in its simplicity.
Cons: Limited length generalization. Performance degrades more sharply at unseen lengths than RoPE or ALiBi. Encoder (bidirectional) transformers underperform without explicit PE.

21.14 Summary Comparison Table

Practical Recommendations

- **For LLMs (decoder-only):** RoPE is the current standard (LLaMA, Mistral, Qwen). Use YaRN or NTK scaling for >4k context.
- **For encoder-decoder (T5-style):** T5 relative bias is simple and effective.
- **For ViT/image transformers:** Learned 2D PE or iRPE.
- **For implicit representations / NeRF:** Fourier features with $L = 10$ for coordinates.
- **For extreme extrapolation:** ALiBi if parameter budget is tight.
- **Simple baseline:** Sinusoidal PE is free and always works.

Table 21.1: Comparison of Positional Encoding Methods

Method	**Type**	**Learnable**	**Extrapolation**	**Key Property**
Sinusoidal (Vaswani)	Absolute	No	Good	Rotation matrix; linear relative-position property
Learned (BERT, GPT)	Absolute	Yes	Poor (needs interpolation)	Data-adaptive, no fixed math structure
Shaw et al. (2018)	Relative	Yes	Good (within clip range)	Adds relative vectors to keys/values
T5 Bias	Relative	Yes	Excellent	Bucketed scalar bias; very lightweight
RoPE	Relative (via rotation)	No	Very good	Rotation of Q, K; implicit relative in dot-product
ALiBi	Relative	No	Excellent	Linear bias, no parameters, zero cost
xPos	Relative	No	Very good	RoPE + exponential decay per dimension
Fourier Features	Absolute	No	Very good (continuous)	Sinusoid over log-spaced frequencies; NeRF
Hash Encoding	Absolute	Yes	Good (with sufficient table)	Multi-resolution hash grids; Instant NGP
NoPE	None	N/A	Moderate	Implicit via causal mask; decoder-only

Chapter 22

Reinforcement Learning Functions

Reinforcement learning (RL) studies how an agent should take actions in an environment to maximize cumulative reward. At the heart of every RL algorithm lie value functions, policy gradients, and auxiliary objectives that collectively define what is learned and how. This chapter surveys the mathematical functions that power modern RL—from classic Bellman backups to entropy-regularized objectives, trust-region constraints, and offline corrections.

22.1 Core Value Functions

A Markov Decision Process (MDP) is defined by $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $\mathcal{P}(s'|s,a)$ the transition dynamics, $\mathcal{R}(s,a)$ the reward function, and $\gamma \in [0,1)$ the discount factor. The return (cumulative discounted reward) is:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

22.1.1 State-Value Function $V^\pi(s)$

State-Value Function

The state-value function $V^\pi(s)$ is the expected return when starting in state s and following policy π thereafter:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \;\middle|\; s_t = s \right] = \mathbb{E}_\pi \big[G_t \mid s_t = s \big]$$

V^π answers: "How good is it to be in state s?" It is the long-term desirability of a state under a given policy.

22.1.2 Action-Value Function $Q^\pi(s,a)$

Action-Value Function

The action-value function $Q^\pi(s,a)$ is the expected return when taking action a in state s and then following policy π:

$$Q^\pi(s,a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \;\middle|\; s_t = s,\ a_t = a \right]$$

Q^π answers: "How good is it to take action a in state s?" The relationship to V^π is $V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s,a)$.

22.1.3 Advantage Function $A^\pi(s,a)$

Advantage Function

The advantage function measures the relative benefit of taking a specific action compared to the average action under the policy:

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$$

$A^\pi(s,a) > 0$ means action a is better than average; $A^\pi(s,a) < 0$ means it is worse. The advantage is zero in expectation: $\mathbb{E}_{a\sim\pi}[A^\pi(s,a)] = 0$.

Why Advantage?

Using the advantage instead of raw returns or Q-values in policy gradients reduces variance without biasing the estimator. The baseline $V^\pi(s)$ subtracts out variability that is not attributable to the action choice.

22.2 Bellman Equations

22.2.1 Bellman Expectation Equations

Bellman Expectation for V^π

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} \mathcal{P}(s'|s,a)\big[\mathcal{R}(s,a) + \gamma V^\pi(s')\big]$$

This recursive decomposition expresses the value of a state in terms of the immediate reward plus the discounted value of the next state, averaged over actions and transitions.

Bellman Expectation for Q^π

$$Q^\pi(s,a) = \mathcal{R}(s,a) + \gamma \sum_{s'} \mathcal{P}(s'|s,a) \sum_{a'} \pi(a'|s')\, Q^\pi(s',a') = \mathcal{R}(s,a) + \gamma \mathbb{E}_{s'\sim\mathcal{P}(\cdot|s,a)}\big[V^\pi(s')\big]$$

22.2.2 Bellman Optimality Equations

Bellman Optimality for V^* and Q^*

The optimal value functions satisfy:

$$V^*(s) = \max_a Q^*(s,a)$$
$$Q^*(s,a) = \mathcal{R}(s,a) + \gamma \sum_{s'} \mathcal{P}(s'|s,a) \max_{a'} Q^*(s',a')$$

An optimal policy is greedy w.r.t. Q^*: $\pi^*(s) = \arg\max_a Q^*(s,a)$. There always exists at least one deterministic optimal policy.

22.2.3 Bellman Operator as Contraction

Bellman Operator $\mathcal{T}^\pi$

For the policy evaluation setting, define the operator:

$$(\mathcal{T}^\pi V)(s) = \sum_a \pi(a|s) \sum_{s'} \mathcal{P}(s'|s,a)\big[\mathcal{R}(s,a) + \gamma V(s')\big]$$

$\mathcal{T}^\pi$ is a γ-contraction in the ℓ_∞-norm:

$$\|\mathcal{T}^\pi V_1 - \mathcal{T}^\pi V_2\|_\infty \le \gamma \|V_1 - V_2\|_\infty$$

By the Banach fixed-point theorem, iterating $V_{k+1} \leftarrow \mathcal{T}^\pi V_k$ converges to the unique fixed point V^π from any initial guess.

Optimality Operator

The Bellman optimality operator $\mathcal{T}^*$ defined by $(\mathcal{T}^* Q)(s,a) = \mathcal{R}(s,a) + \gamma \sum_{s'} \mathcal{P}(s'|s,a) \max_{a'} Q(s',a')$ is also a γ-contraction. Value iteration $Q_{k+1} \leftarrow \mathcal{T}^* Q_k$ converges to Q^*.

22.3 Value-Based Methods

Value-based methods learn the optimal value function (typically Q^*) and derive a policy by acting greedily.

22.3.1 TD(0) — Temporal Difference Learning

TD(0) Update

Temporal-difference learning bootstraps: it updates an estimate based on another estimate without waiting for the episode to end.

$$V(s_t) \leftarrow V(s_t) + \alpha\big[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)\big]$$

The **TD error** $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ measures the inconsistency between the current estimate and the target formed by the next state. The learning rate is $\alpha \in (0, 1]$.

TD(0)

Pros:

- Learns online and incrementally; no need to wait for episode termination.
- Lower variance than Monte Carlo (MC) because the target uses $V(s_{t+1})$ instead of the full empirical return.
- Converges to V^π under appropriate decreasing α (tabular case).
- Forms the foundation for SARSA, Q-Learning, DQN, and actor-critic.

Cons:

- Introduces bias from bootstrapping (the target itself is an imperfect estimate).

- Convergence can be slow; sensitive to the choice of α and γ.
- Off-policy divergence possible with function approximation (the "deadly triad" of function approximation, bootstrapping, and off-policy learning).

22.3.2 SARSA — On-Policy TD Control

SARSA

SARSA is an on-policy method that learns $Q(s, a)$ using the tuple $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\big[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)\big]$$

The action a_{t+1} is chosen by the behavior policy π (e.g., ϵ-greedy on the current Q). SARSA converges to the optimal Q-function for an ϵ-greedy policy that is gradually annealed (GLI—Greedy in the Limit with Infinite Exploration).

SARSA Applications

Used when safety during training matters. For example, in robot navigation where a cliff exists along the optimal path, SARSA learns a safer detour because it accounts for the occasional exploratory move near the edge, whereas Q-Learning would optimistically assume the agent always takes the best action.

22.3.3 Q-Learning — Off-Policy TD Control

Q-Learning (Watkins

Q-Learning is an off-policy algorithm that directly approximates Q^* regardless of the behavior policy:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\big[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\big]$$

The target uses the **max** over all actions at the next state, decoupling the learned policy from the exploration policy.

Q-Learning

Pros:

- Off-policy: can learn from any trajectory, including expert demonstrations or replay buffers.
- Directly estimates Q^*; the greedy policy derived from it is optimal.
- Simpler than SARSA in that it does not need the next action a_{t+1}.

Cons:

- **Maximization bias:** the max operator systematically overestimates Q-values (addressed by Double Q-Learning).
- Off-policy divergence with linear function approximation (the "deadly triad").
- Can be overly optimistic, causing danger in safety-critical exploration.

22.3.4 Expected SARSA

Expected SARSA

Expected SARSA replaces the sampled $Q(s_{t+1}, a_{t+1})$ in SARSA with its expectation under the policy:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\Big[r_{t+1} + \gamma \sum_{a'} \pi(a'|s_{t+1}) Q(s_{t+1}, a') - Q(s_t, a_t)\Big]$$

When π is the greedy policy, Expected SARSA reduces to Q-Learning. When π is stochastic (e.g., softmax), it removes the variance due to sampling a_{t+1}.

Expected SARSA vs Q-Learning

Expected SARSA with a uniform-random behavior policy and a greedy target policy eliminates the maximization bias while remaining off-policy. This makes it a computationally more expensive but more stable alternative to Q-Learning in tabular settings.

22.3.5 DQN — Deep Q-Network

DQN (Mnih et al.

DQN uses a neural network $Q(s, a; \boldsymbol{\theta})$ to approximate the optimal action-value function. Two key innovations stabilize training:
Experience Replay: transitions (s, a, r, s') are stored in a replay buffer $\mathcal{D}$ and sampled uniformly in mini-batches to break temporal correlations.
Target Network: a separate network $Q(s, a; \boldsymbol{\theta}^-)$ with frozen parameters provides stable targets. The target network is periodically updated: $\boldsymbol{\theta}^- \leftarrow \boldsymbol{\theta}$ every C steps.

$$\mathcal{L}_{\text{DQN}}(\boldsymbol{\theta}) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\Big[\big(r + \gamma \max_{a'} Q(s', a'; \boldsymbol{\theta}^-) - Q(s, a; \boldsymbol{\theta})\big)^2\Big]$$

The loss is minimized by SGD/Adam w.r.t. $\boldsymbol{\theta}$ only.

DQN Applications

Atari 2600 games (raw pixels as input, discrete action space). DQN was the first algorithm to achieve human-level performance across a wide suite of Atari games from raw visual input, sparking the modern deep RL revolution. Subsequent improvements (Double DQN, Dueling, Prioritized Replay, etc.) form the foundation of the Rainbow agent.

22.3.6 Double DQN

Double DQN (van Hasselt et al.

Double DQN decouples action selection from action evaluation to mitigate overestimation bias:

$$\mathcal{L}_{\text{DDQN}} = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\Big[\big(r + \gamma Q\big(s', \arg\max_{a'} Q(s', a'; \boldsymbol{\theta}); \boldsymbol{\theta}^-\big) - Q(s, a; \boldsymbol{\theta})\big)^2\Big]$$

The online network ($\boldsymbol{\theta}$) selects the action; the target network ($\boldsymbol{\theta}^-$) evaluates it. This prevents the max operator from consistently inflating Q-values by reusing the same estimator

for both selection and evaluation.

Overestimation Mechanism

Let $\hat{Q}_1, \ldots, \hat{Q}_k$ be unbiased but noisy estimates of the true Q^* for k actions. The max $\max_i \hat{Q}_i$ is an overestimate: $\mathbb{E}[\max_i \hat{Q}_i] \geq \max_i \mathbb{E}[\hat{Q}_i]$. Double DQN remedies this by using two independent estimators.

22.3.7 Dueling DQN

Dueling DQN (Wang et al.

Dueling DQN splits the Q-network into two streams: a state-value $V(s; \boldsymbol{\theta}, \boldsymbol{\beta})$ and action-advantages $A(s, a; \boldsymbol{\theta}, \boldsymbol{\alpha})$. They are combined as:

$$Q(s,a) = V(s) + \left(A(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a')\right)$$

The subtraction of the mean advantage forces $\mathbb{E}_a[A(s,a)] = 0$, making the decomposition identifiable. The shared convolutional encoder learns a common representation, and the two streams specialize.

Dueling Architecture

The dueling structure is most beneficial when many actions have similar values and only a few matter—the value stream can learn general state goodness quickly without having to estimate the value of every action. Primarily used in Atari domains; incorporated into Rainbow.

22.3.8 Distributional RL

Distributional RL — C51

Instead of predicting scalar expected returns, distributional RL models the entire distribution of returns $Z(s,a)$.

C51 (Bellemare et al., 2017): Discretizes the return distribution into $N = 51$ atoms over a fixed support $[v_{\min}, v_{\max}]$. The categorical projection (Cramér projection) maps the bootstrapped distribution $r + \gamma Z(s', a^*)$ back onto the fixed support.

QR-DQN (Dabney et al., 2018): Learns N quantiles $\{\theta_i(s,a)\}_{i=1}^N$ corresponding to $\tau_i = \frac{i-0.5}{N}$. The loss uses the **quantile Huber**:

$$\mathcal{L}_{\text{QR}} = \frac{1}{N^2}\sum_{i=1}^{N}\sum_{j=1}^{N} \rho_{\tau_i}^{\kappa}\left(r + \gamma\theta_j(s', a^*) - \theta_i(s,a)\right)$$

where $\rho_\tau^\kappa(u) = |\tau - \mathbb{I}_{u<0}| \cdot \mathcal{L}_{\text{Huber}}^\kappa(u)$.

IQN (Dabney et al., 2018b): Replaces fixed quantiles with an implicit network that can be queried at any $\tau \sim U(0,1)$, enabling risk-sensitive policies (e.g., optimizing CVaR).

Why Distributional RL Helps

Modeling the full distribution provides richer learning signals (auxiliary tasks), encodes uncertainty, and enables risk-sensitive decision-making. Distributional methods consistently improve sample efficiency and final performance over scalar-value DQN.

22.3.9 Rainbow

Rainbow (Hessel et al.

Rainbow combines six DQN improvements into a single integrated agent:

1. Double Q-Learning (decoupled selection/evaluation).
2. Prioritized Experience Replay (PER)—sampling proportionally to TD-error magnitude.
3. Dueling architecture ($V + A - \text{mean}(A)$).
4. Multi-step returns (n-step bootstrap targets).
5. Distributional RL (C51 categorical distribution).
6. Noisy Nets—parametric noise in the network weights for exploration.

All components are complementary and work together with minimal conflicts.

Rainbow

Achieved state-of-the-art results on the Atari 2600 benchmark. Serves as a testbed demonstrating that individually beneficial algorithmic improvements can be combined without catastrophic interference.

22.3.10 n-Step TD and TD(λ)

n-Step Return

The n-step return interpolates between 1-step TD ($n = 1$) and Monte Carlo ($n \to \infty$):

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n V(s_{t+n})$$

The corresponding n-step TD update is:

$$V(s_t) \leftarrow V(s_t) + \alpha \big[G_t^{(n)} - V(s_t) \big]$$

TD(λ) — Eligibility Traces

TD(λ) uses a geometrically weighted combination of all n-step returns:

$$G_t^{\lambda} = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

where $\lambda \in [0, 1]$ controls the bias-variance tradeoff. Equivalently, with eligibility trace e_t:

$$e_t(s) = \gamma \lambda \, e_{t-1}(s) + \mathbb{I}[s = s_t]$$
$$V(s) \leftarrow V(s) + \alpha \, \delta_t \, e_t(s)$$

where $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$. Every recently visited state is updated proportionally to its eligibility.

Eligibility Traces

Pros:

- Unifies TD and MC: $\lambda = 0$ gives 1-step TD; $\lambda = 1$ gives (essentially) MC returns.
- Intermediate λ often yields faster convergence than either extreme.
- Seamless integration with function approximation.

Cons:

- Maintaining traces adds memory and computation overhead.
- The trace decay λ is an extra hyperparameter.
- In non-episodic (continuing) tasks, setting λ near 1 can cause unbounded variance.

22.3.11 Retrace(λ)

Retrace(λ) — Munos et al.

Retrace is an off-policy return-based algorithm that corrects for distribution shift without the high variance of importance sampling ratios. The off-policy corrected return is:

$$\mathcal{R}Q(s,a) = Q(s,a) + \mathbb{E}_\mu\left[\sum_{t\geq 0}\gamma^t\Big(\prod_{s=1}^{t} c_s\Big)\delta_t\right]$$

where $c_s = \lambda \cdot \min\big(1, \frac{\pi(a_s|s_s)}{\mu(a_s|s_s)}\big)$ truncates the importance-sampling ratio to at most 1, and $\delta_t = r_t + \gamma Q(s_{t+1}, a_{t+1}^{\text{target}}) - Q(s_t, a_t)$. This operator is a contraction mapping, guaranteed to converge.

Retrace

Foundation of R2D2 (Recurrent Replay Distributed DQN) and the IMPALA actor-critic architecture (V-trace is a closely related variant). Used when off-policy data comes from many heterogeneous behavior policies.

22.4 Policy Gradient Methods

Policy gradient methods directly parameterize the policy $\pi_\theta(a|s)$ and optimize it by gradient ascent on the expected return.

22.4.1 Policy Gradient Theorem

Policy Gradient Theorem (Sutton et al.

For a policy π_θ parameterized by $\boldsymbol{\theta}$ and an objective $J(\boldsymbol{\theta}) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)]$ (or equivalently $J(\boldsymbol{\theta}) = V^{\pi_\theta}(s_0)$), the gradient is:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbb{E}_{\pi_\theta}\big[\nabla_{\boldsymbol{\theta}} \log \pi_\theta(a|s)\; Q^{\pi_\theta}(s,a)\big]$$

Crucially, the gradient does *not* involve the derivative of the state distribution—this makes stochastic gradient ascent feasible.

22.4.2 REINFORCE

REINFORCE (Williams

The simplest Monte Carlo policy gradient algorithm. It samples a full trajectory and uses the empirical return G_t as the Q-estimate:

$$\nabla_{\boldsymbol{\theta}} J \approx \frac{1}{T}\sum_{t=0}^{T-1} \nabla_{\boldsymbol{\theta}} \log \pi_\theta(a_t|s_t)\; G_t$$

The update rule is:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\; \nabla_{\boldsymbol{\theta}} \log \pi_\theta(a_t|s_t)\; G_t$$

REINFORCE

Pros:

- Unbiased gradient estimate (MC return has no bootstrap bias).
- Extremely simple—no need for a value function.
- Works with any parameterization (discrete or continuous).

Cons:

- **High variance:** the empirical return G_t has large variance, especially with long horizons and stochastic environments.
- Requires full episode rollouts; no online learning.
- Slow convergence; many episodes needed for reliable gradient estimates.

22.4.3 REINFORCE with Baseline

Baseline Subtraction

To reduce variance, subtract a baseline $b(s)$ that does not depend on the action:

$$\nabla_{\boldsymbol{\theta}} J = \mathbb{E}\big[\nabla_{\boldsymbol{\theta}} \log \pi_\theta(a|s)\; \big(G_t - b(s_t)\big)\big]$$

The most common baseline is the state-value function $V(s)$:

$$\nabla_{\boldsymbol{\theta}} J = \mathbb{E}\big[\nabla_{\boldsymbol{\theta}} \log \pi_\theta(a|s)\; \big(G_t - V(s_t)\big)\big]$$

This yields an unbiased gradient estimate with lower variance since $\mathbb{E}[\nabla \log \pi \cdot b(s)] = b(s) \cdot \nabla 1 = 0$.

Baseline Equals Advantage

When $b(s) = V^\pi(s)$, the multiplier becomes $G_t - V(s_t) \approx Q^\pi(s_t, a_t) - V^\pi(s_t) = A^\pi(s_t, a_t)$. Thus REINFORCE with a value baseline is a Monte Carlo estimate of the policy gradient theorem.

22.4.4 Actor-Critic Methods

Actor-Critic Architecture

Actor-critic methods maintain two components:

- **Actor** π_θ: the policy, updated via policy gradient.
- **Critic** V_ϕ or Q_ϕ: the value function, updated via TD learning.

The critic provides a low-variance estimate of the advantage, $A_t = \delta_t = r_{t+1} + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ (1-step actor-critic), and the actor update is:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha_\theta \, \nabla_{\boldsymbol{\theta}} \log \pi_\theta(a_t|s_t) \, \delta_t$$

Actor-Critic Applications

The dominant paradigm for deep RL with continuous actions. A2C, A3C, PPO, SAC, TD3, and DDPG are all actor-critic variants. The critic reduces gradient variance while the bootstrapping enables online, incremental updates.

22.4.5 A2C / A3C

Advantage Actor-Critic (A2C/A3C

A2C (Synchronous): Multiple agents run in parallel environments collecting rollouts; the gradients are averaged and applied synchronously.
A3C (Asynchronous): Each agent runs independently, computing gradients and updating a shared global model asynchronously without a synchronization barrier.
Both use the n-step advantage:

$$A_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V_\phi(s_{t+n}) - V_\phi(s_t)$$

and add an entropy bonus $\beta H(\pi(\cdot|s))$ to the policy loss to encourage exploration.

A2C vs A3C

Pros:

- Parallelism dramatically accelerates training (wall-clock time).
- A3C's asynchronous updates act as implicit regularization.

- Entropy bonus prevents premature convergence to deterministic policies.

Cons:

- A3C's asynchronous updates can be detrimental with large lag (stale gradients).
- A2C is simpler to implement and often performs equally well on GPU hardware.
- The entropy coefficient β requires tuning.

22.4.6 GAE — Generalized Advantage Estimation

GAE(λ) — Schulman et al.

GAE computes a multi-step advantage estimator as an exponentially weighted sum of TD errors:

$$A_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}$$

where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$.

- $\lambda = 0$: $A_t = \delta_t$ (1-step TD).
- $\lambda = 1$: $A_t = \sum_{l=0}^{\infty} \gamma^l \delta_{t+l} = G_t - V(s_t)$ (MC advantage).

GAE Bias-Variance Tradeoff

λ controls the bias-variance spectrum. Small λ yields low-variance but biased estimates; large λ yields low-bias but high-variance estimates. Typical choices: $\lambda \in [0.92, 0.98]$ works well for continuous control, $\lambda \in [0.95, 0.99]$ for MuJoCo tasks.

GAE

GAE is the default advantage estimator in PPO and many modern actor-critic implementations. It provides smoothly tunable bias-variance tradeoff with a single λ parameter and integrates naturally with value-function bootstrapping.

22.5 Trust Region and Proximal Methods

Trust region methods constrain the size of policy updates, preventing destructive large steps that could permanently degrade performance.

22.5.1 TRPO — Trust Region Policy Optimization

TRPO (Schulman et al.

TRPO solves a constrained optimization at each iteration:

$$\begin{aligned} \max_{\boldsymbol{\theta}} \quad & \mathbb{E}_{s\sim\rho^{\pi_{\mathrm{old}}},\, a\sim\pi_{\mathrm{old}}} \left[\frac{\pi_\theta(a|s)}{\pi_{\mathrm{old}}(a|s)} A^{\pi_{\mathrm{old}}}(s,a) \right] \\ \text{s.t.} \quad & \mathbb{E}_{s\sim\rho^{\pi_{\mathrm{old}}}} \left[D_{\mathrm{KL}}(\pi_{\mathrm{old}}(\cdot|s) \,\|\, \pi_\theta(\cdot|s)) \right] \le \delta \end{aligned}$$

The KL constraint (with δ typically ≈ 0.01) ensures the new policy stays within the trust region where the surrogate objective is a reliable approximation of the true objective.

TRPO Implementation

TRPO approximates the solution via a linear approximation of the objective and a quadratic approximation of the KL constraint, leading to the natural gradient: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta \mathbf{F}^{-1}\mathbf{g}$, where $\mathbf{F}$ is the Fisher information matrix and $\mathbf{g}$ is the policy gradient. Conjugate gradient solves $\mathbf{F}^{-1}\mathbf{g}$ without explicitly inverting the matrix. A backtracking line search enforces the KL constraint.

22.5.2 Natural Policy Gradient

Natural Policy Gradient (Kakade

The natural gradient scales the policy gradient by the inverse Fisher information matrix:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta\, \mathbf{F}(\boldsymbol{\theta})^{-1}\, \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

where $\mathbf{F}(\boldsymbol{\theta}) = \mathbb{E}_{\pi_\theta}\left[\nabla \log \pi_\theta(a|s)\, \nabla \log \pi_\theta(a|s)^\top\right]$.
This corresponds to steepest ascent in the Riemannian manifold of probability distributions (with the Fisher metric), rather than in Euclidean parameter space.

Natural Gradient

Pros:

- Invariant to reparameterization: equivalent policies yield equivalent updates regardless of parameterization.
- Typically requires far fewer iterations than vanilla policy gradient.
- Prevents the policy from changing too rapidly in terms of action probabilities.

Cons:

- Computing $\mathbf{F}^{-1}\mathbf{g}$ is $O(n^3)$ in the number of parameters (or $O(n^2)$ with conjugate gradient).
- Approximate methods are needed for large neural networks.
- The step-size η and KL target δ still require tuning.

22.5.3 PPO — Proximal Policy Optimization

PPO-Clip (Schulman et al.

PPO simplifies TRPO by using a clipped surrogate objective that penalizes large policy updates. Define the probability ratio:

$$r_t(\boldsymbol{\theta}) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$$

The clipped surrogate objective is:

$$\mathcal{L}^{\text{CLIP}}(\boldsymbol{\theta}) = \mathbb{E}_t\Big[\min\Big(r_t(\boldsymbol{\theta})A_t,\ \text{clip}\big(r_t(\boldsymbol{\theta}),\ 1-\epsilon,\ 1+\epsilon\big)A_t\Big)\Big]$$

where ϵ (typically 0.1 or 0.2) controls the clip range. The min operator ensures the objective is a pessimistic bound: it ignores improvements from moving r_t outside $[1-\epsilon, 1+\epsilon]$.

PPO with KL Penalty

An alternative PPO variant adds an adaptive KL penalty instead of clipping:

$$\mathcal{L}^{\text{KLPEN}}(\boldsymbol{\theta}) = \mathbb{E}_t\Big[r_t(\boldsymbol{\theta})A_t - \beta\, D_{\text{KL}}\big(\pi_{\text{old}}(\cdot|s_t) \parallel \pi_\theta(\cdot|s_t)\big)\Big]$$

The coefficient β is adjusted adaptively:

- If $D_{\text{KL}} > d_{\text{targ}} \times 1.5$: $\beta \leftarrow 2\beta$
- If $D_{\text{KL}} < d_{\text{targ}}/1.5$: $\beta \leftarrow \beta/2$

PPO Applications

The default deep RL algorithm for continuous control (MuJoCo, Robotics), game-playing (OpenAI Five, Hide and Seek), and RLHF (RL from Human Feedback) for training language models like ChatGPT and Claude. PPO strikes the best balance of simplicity, stability, and sample efficiency among policy-gradient methods.

PPO

Pros:

- First-order method (no conjugate gradient or line search needed).
- The clipped objective provides a simple, effective trust-region constraint.
- Compatible with RNNs, shared architectures, and multi-task objectives.
- Stable across a wide range of hyperparameters.

Cons:

- The clip ratio ϵ is an additional hyperparameter.
- Performance can degrade with code-level implementation details (e.g., advantage normalization, value-function clipping, entropy bonus).
- Sample efficiency is lower than off-policy methods like SAC/TD3.

22.6 Continuous Action Methods

22.6.1 DPG — Deterministic Policy Gradient

Deterministic Policy Gradient (Silver et al.

For a deterministic policy $a = \mu_\theta(s)$, the policy gradient is:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbb{E}_{s\sim\rho^\mu}\Big[\nabla_{\boldsymbol{\theta}}\mu_\theta(s)\ \nabla_a Q^\mu(s,a)\big|_{a=\mu_\theta(s)}\Big]$$

This is the limiting case of the stochastic policy gradient as the policy variance $\to 0$. The gradient chain-rules through the Q-function: we propagate gradients from the action-value to the action, then from the action to the policy parameters.

22.6.2 DDPG — Deep Deterministic Policy Gradient

DDPG (Lillicrap et al.

DDPG adapts DQN's training techniques to the continuous action domain:

- **Actor network** $\mu_\theta(s)$ produces actions.
- **Critic network** $Q_\phi(s,a)$ estimates action values.
- **Target networks** μ_{θ^-}, Q_{ϕ^-} with **soft updates**: $\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$ with $\tau \ll 1$ (e.g., 0.005).
- **Experience replay** from buffer $\mathcal{D}$.
- **Exploration noise:** Ornstein-Uhlenbeck process or Gaussian $\mathcal{N}(0,\sigma^2)$ added to actions.

The critic is updated by minimizing the TD error:

$$\mathcal{L}(\phi) = \mathbb{E}_{\mathcal{D}}\big[(r + \gamma Q_{\phi^-}(s', \mu_{\theta^-}(s')) - Q_\phi(s,a))^2\big]$$

The actor is updated via the deterministic policy gradient.

22.6.3 TD3 — Twin Delayed DDPG

TD3 (Fujimoto et al.

TD3 addresses DDPG's overestimation bias and sensitivity with three modifications:
1. Clipped Double Q-Learning: Two critic networks Q_{ϕ_1}, Q_{ϕ_2}; the target uses the minimum:

$$y = r + \gamma \min_{i=1,2} Q_{\phi_i^-}(s', a')$$

2. Delayed Policy Updates: The actor and target networks are updated less frequently than the critics (e.g., every other critic update).
3. Target Policy Smoothing: Add clipped noise to the target action to regularize:

$$a' = \mu_{\theta^-}(s') + \text{clip}(\epsilon,\ -c,\ c), \quad \epsilon \sim \mathcal{N}(0,\sigma^2)$$

This averages Q-values over a small region around the target action, smoothing the value estimate.

TD3

Pros:

- Clipped double Q effectively eliminates overestimation bias.
- Delayed updates prevent the policy from exploiting Q-function errors.
- Target smoothing regularizes the value landscape.
- Consistently outperforms DDPG on continuous control benchmarks.

Cons:

- Additional hyperparameters: two critics, target noise σ, noise clip c, policy delay d.
- Slower wall-clock training due to delayed updates and twin critics.
- No intrinsic exploration mechanism; relies on external noise.

22.7 Maximum Entropy Reinforcement Learning

Maximum entropy RL augments the standard return objective with an entropy bonus, encouraging stochastic policies that remain exploratory.

22.7.1 Soft Bellman and Soft Q-Function

Soft Q-Function

The soft (entropy-regularized) Q-function incorporates policy entropy:

$$Q_{\text{soft}}(s,a) = r + \gamma \mathbb{E}_{s'}\left[\alpha \log \sum_{a'} \exp\left(\tfrac{1}{\alpha} Q_{\text{soft}}(s',a')\right)\right]$$

where α is the temperature parameter controlling the tradeoff between reward maximization and entropy. As $\alpha \to 0$, soft Q reduces to standard Q. The soft value function is:

$$V_{\text{soft}}(s) = \alpha \log \sum_{a} \exp\left(\tfrac{1}{\alpha} Q_{\text{soft}}(s,a)\right) = \mathbb{E}_{a\sim\pi}[Q_{\text{soft}}(s,a) - \alpha \log \pi(a|s)]$$

22.7.2 SAC — Soft Actor-Critic

SAC (Haarnoja et al.

SAC optimizes the maximum entropy objective:

$$J(\pi) = \sum_{t=0}^{\infty} \mathbb{E}_{(s_t,a_t)\sim\rho_\pi}\left[r(s_t,a_t) + \alpha H(\pi(\cdot|s_t))\right]$$

where $H(\pi(\cdot|s)) = -\mathbb{E}_{a\sim\pi}[\log \pi(a|s)]$ is the policy entropy.

Soft Policy Evaluation: The soft Q-function is trained to minimize the soft Bellman residual:

$$\mathcal{L}_Q(\phi) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(Q_\phi(s,a) - (r + \gamma V_{\bar{\phi}}(s'))\right)^2\right]$$

where $V_{\bar{\phi}}(s') = \mathbb{E}_{a'\sim\pi_\theta}[Q_{\bar{\phi}}(s',a') - \alpha \log \pi_\theta(a'|s')]$ uses a target network $\bar{\phi}$.

Soft Policy Improvement: The policy is updated to minimize the KL divergence to the energy-based policy $\propto \exp(Q/\alpha)$:

$$\mathcal{L}_\pi(\theta) = \mathbb{E}_{s\sim\mathcal{D}}\Big[\mathbb{E}_{a\sim\pi_\theta}[\alpha \log \pi_\theta(a|s) - Q_\phi(s,a)]\Big]$$

The reparameterization trick $a = f_\theta(s,\epsilon)$ with $\epsilon \sim \mathcal{N}(0,1)$ enables gradient backpropagation.

22.7.3 Automatic Temperature Tuning

SAC with Learned α (Haarnoja et al.

Instead of manually setting α, SAC can learn it by minimizing:

$$\mathcal{L}(\alpha) = \mathbb{E}_{a\sim\pi_\theta}\left[-\alpha \log \pi_\theta(a|s) - \alpha\bar{\mathcal{H}}\right]$$

where $\bar{\mathcal{H}}$ is a target entropy (e.g., $-\dim(\mathcal{A})$). This formulation enforces an entropy constraint $H(\pi) \geq \bar{\mathcal{H}}$. When the policy's entropy exceeds the target, α decreases (relaxing the entropy bonus); when entropy is insufficient, α increases.

Why Automatic α Works

$\mathcal{L}(\alpha)$ is convex in α. At optimum, $\mathbb{E}_{a\sim\pi}[-\log \pi(a|s)] = \bar{\mathcal{H}}$. This adaptively balances exploration and exploitation without manual tuning as the policy's entropy naturally changes during training.

SAC Applications

State-of-the-art for continuous control tasks (MuJoCo, DM Control Suite). Combines the sample efficiency of off-policy methods (experience replay) with the stability of entropy regularization. Particularly effective in high-dimensional action spaces where exploration is challenging.

SAC

Pros:

- Off-policy, sample-efficient learning via replay buffer.
- Maximum entropy formulation encourages robust, multi-modal policies.
- Automatic temperature tuning removes the need to tune α.
- Learns stochastic policies—important for real-world robotics where deterministic policies are brittle.

Cons:

- More complex than PPO (twin critics, target networks, replay buffer, entropy).
- Stochastic policy may not be appropriate for some tasks (e.g., precise positioning).
- Sensitive to reward scaling; rewards may need normalization.
- Hyperparameters: replay buffer size, target update τ, target entropy $\bar{\mathcal{H}}$.

22.8 Model-Based Reinforcement Learning

22.8.1 Dyna Architecture

Dyna (Sutton

Dyna-Q integrates model-free RL (direct Q-learning from real experience) with model-based planning (simulated experience from a learned model):

1. **Direct RL:** Update $Q(s, a)$ using real experience.
2. **Model Learning:** Learn transition model $\hat{\mathcal{P}}(s'|s, a)$ and reward model $\hat{\mathcal{R}}(s, a)$ from real experience.
3. **Planning:** For n planning steps, sample a previously seen (s, a), simulate (s', r) from the model, and apply Q-learning updates.

The planning steps accelerate value propagation without additional environment interactions.

Dyna Tradeoffs

Large n (many planning steps per real step) accelerates learning when the model is accurate but can degrade performance with model errors. Dyna-Q demonstrates that model-free and model-based RL are complementary, not competing approaches.

22.8.2 MCTS with UCT

MCTS with UCT — Upper Confidence bounds for Trees

Monte Carlo Tree Search builds an asymmetric search tree forward from the current state. At each node, the UCT formula selects the child with:

$$a^* = \arg\max_a \left(\frac{w_a}{n_a} + c\sqrt{\frac{\ln N}{n_a}} \right)$$

where w_a is the total value from action a, n_a is the visit count, N is the parent visit count, and c is the exploration constant (typically $c = \sqrt{2}$ for UCB1).

The four MCTS phases: **Selection** (traverse using UCT), **Expansion** (add a new leaf node), **Simulation** (complete the episode with a default/rollout policy), **Backpropagation** (update w_a, n_a along the path).

MCTS

The go-to planning algorithm for board games (Go, Chess). Dominated computer Go before deep learning (MoGo, Crazy Stone). Used in AlphaGo's rollouts and in many discrete-action planning domains where enumerated search is feasible.

22.8.3 AlphaZero — PUCT

PUCT — Predictor + UCT (Silver et al.

AlphaZero replaces UCT's exploration term with a policy prior from a neural network $P(s,a)$ (the probability that action a is played from state s, as predicted by the network):

$$a^* = \arg\max_a \left(Q(s,a) + c_{\text{puct}}\, P(s,a)\, \frac{\sqrt{\sum_b N(s,b)}}{1+N(s,a)} \right)$$

where c_{puct} controls the influence of the policy prior relative to the empirical Q-values. The policy prior guides search toward promising moves without needing rollouts—the value network evaluates leaf nodes directly: $V(s) = f_\theta(s)$.
The neural network is trained with:

$$\mathcal{L} = (z-v)^2 - \boldsymbol{\pi}^\top \log \mathbf{p} + c\|\boldsymbol{\theta}\|^2$$

where z is the final game outcome, $\boldsymbol{\pi}$ is the MCTS-improved policy (visit counts), v is the predicted value, and $\mathbf{p}$ is the raw policy output.

AlphaZero

Pros:

- Achieved superhuman performance in Go, Chess, and Shogi with no domain knowledge beyond the rules.
- Self-play generates infinite training data; no human demonstrations needed.
- The policy prior dramatically reduces the effective branching factor.

Cons:

- Computationally intensive—requires thousands of TPU/GPU hours.
- Assumes a perfect simulator and deterministic two-player zero-sum dynamics.
- Not directly applicable to stochastic, partially observable, or continuous-action domains.

22.8.4 Dreamer — RSSM

Dreamer and RSSM (Hafner et al.

Dreamer learns a world model from pixels and plans in the learned latent space. The **Recurrent State Space Model (RSSM)** combines deterministic (recurrent) and stochastic (latent variable) components:

$$\begin{aligned} \text{Deterministic state:} \quad & h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1}) \\ \text{Stochastic state:} \quad & z_t \sim p_\phi(z_t \mid h_t) \\ \text{Decoders:} \quad & \hat{o}_t \sim p_\phi(o_t \mid h_t, z_t), \quad \hat{r}_t \sim p_\phi(r_t \mid h_t, z_t) \end{aligned}$$

Dreamer imagines latent rollouts and optimizes the policy via value gradients backpropagated through the learned dynamics. DreamerV2/V3 use categorical latent variables

(straight-through gradients), KL balancing, and symlog predictions for robust world-model learning.

Dreamer

Excels at visual control tasks (DM Control Suite, Atari) from pixels alone. DreamerV3 is the first algorithm to solve the Diamond environment in Minecraft (collecting a diamond) without human demonstrations, demonstrating world models' potential for long-horizon tasks.

22.9 Inverse and Imitation Reinforcement Learning

22.9.1 Behavioral Cloning (BC)

Behavioral Cloning

BC treats imitation as supervised learning over expert demonstrations $\mathcal{D}_E = \{(s_i, a_i)\}_{i=1}^N$:

$$\mathcal{L}_{\text{BC}}(\boldsymbol{\theta}) = -\mathbb{E}_{(s,a)\sim\mathcal{D}_E}\big[\log \pi_\theta(a|s)\big]$$

This is maximum-likelihood estimation of the expert's policy. BC is simple but suffers from **distributional shift**: errors accumulate when the agent visits states not seen in the expert data, leading to compounding errors (quadratic in horizon).

When BC Works

BC is effective when (1) the expert dataset covers the state distribution well, (2) the policy class has sufficient capacity, and (3) the task horizon is short. It is commonly used as a warm-start or auxiliary objective in more advanced imitation methods.

22.9.2 GAIL — Generative Adversarial Imitation Learning

GAIL (Ho & Ermon

GAIL frames imitation as a GAN-like minimax game between a policy π_θ and a discriminator D_ψ:

$$\min_\pi \max_D \; \mathbb{E}_{\pi_E}\big[\log D(s,a)\big] + \mathbb{E}_\pi\big[\log(1 - D(s,a))\big] - \lambda H(\pi)$$

The discriminator learns to distinguish expert from agent state-action pairs. The agent's reward is $r(s,a) = -\log(1 - D(s,a))$ (or equivalently $\log D(s,a)$), which is higher when the discriminator is fooled. The entropy bonus $\lambda H(\pi)$ prevents policy collapse.
GAIL can be optimized with any policy-gradient method (TRPO, PPO, etc.).

GAIL

Effective for learning complex motor skills from a handful of expert demonstrations (e.g., MuJoCo locomotion). More robust to distributional shift than BC because GAIL learns a reward that provides dense feedback even in states not covered by the expert.

22.10 Exploration Methods

22.10.1 Simple Exploration

ϵ-Greedy and Boltzmann

ϵ-Greedy: With probability $1 - \epsilon$, take the greedy action $a^* = \arg\max_a Q(s, a)$; with probability ϵ, take a uniformly random action. ϵ is typically annealed from 1.0 to 0.01 or 0.05 over training.
Boltzmann (Softmax) Exploration: Sample actions from a softmax over Q-values:

$$\pi(a|s) = \frac{\exp(Q(s, a)/\tau)}{\sum_{a'} \exp(Q(s, a')/\tau)}$$

where $\tau > 0$ is the temperature. As $\tau \to 0$, the policy becomes greedy; as $\tau \to \infty$, it becomes uniform.

Simple Exploration

Pros:

- Extremely simple with minimal overhead.
- Sufficient for many tasks with short horizons or dense rewards.
- Temperature provides tunable stochasticity without explicit probability of random action.

Cons:

- No directed exploration: random actions waste time in large state spaces.
- dithering: ϵ-greedy adds permanent noise even after convergence (unless annealed to zero).
- Does not scale to sparse-reward tasks where systematic exploration is needed.

22.10.2 UCB1

UCB1 (Auer et al.

For the multi-armed bandit, UCB1 selects:

$$a = \arg\max_a \left(\hat{Q}(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right)$$

where $c > 0$ controls exploration, t is the total number of pulls, and $N_t(a)$ is the number of times action a was chosen. The second term is the **exploration bonus** or **optimism bonus** that decreases as $N_t(a)$ grows.

22.10.3 Thompson Sampling

Thompson Sampling

Maintain a posterior distribution over Q-values, $p(Q|\mathcal{D})$. At each step, sample $\tilde{Q} \sim p(Q|\mathcal{D})$ and act greedily: $a = \arg\max_a \tilde{Q}(a)$. The uncertainty in the posterior drives exploration naturally: under-explored actions have wider distributions and are more likely to be sampled as optimal.

22.10.4 Intrinsic Motivation: ICM and RND

ICM — Intrinsic Curiosity Module (Pathak et al.

ICM generates intrinsic reward from prediction error in a learned feature space:
Forward model $f(s_t, a_t) \to \hat{\phi}(s_{t+1})$ predicts the next state's features given the current encoded state $\phi(s_t)$ and action.

$$r_{\text{intrinsic}} = \frac{\eta}{2}\|\hat{\phi}(s_{t+1}) - \phi(s_{t+1})\|_2^2$$

Large prediction error means the outcome was surprising, yielding high intrinsic reward. The feature encoder ϕ is trained by an **inverse model** $g(\phi(s_t), \phi(s_{t+1})) \to \hat{a}_t$ that predicts the action from consecutive states, ensuring ϕ captures action-relevant features only.

RND — Random Network Distillation (Burda et al.

RND uses the prediction error of a fixed random target network as intrinsic reward:

$$r_{\text{intrinsic}} = \|\hat{f}(s;\boldsymbol{\theta}) - f(s)\|_2^2$$

where f is a randomly initialized and **frozen** target network, and $\hat{f}$ is trained to predict $f(s)$. Prediction error is high for novel states (not well represented in the training distribution) and low for familiar states, making it a novelty detector.

ICM vs RND

ICM requires actions for its prediction, making it suitable for action-dependent tasks. RND has the advantage of being action-independent and simpler (no inverse model). Both have demonstrated state-of-the-art exploration on hard Atari games like Montezuma's Revenge. NGU (Never Give Up, Badia et al., 2020) combines episodic (within-episode) and lifelong novelty for further improvements.

22.11 Offline Reinforcement Learning

Offline RL (batch RL) learns policies entirely from a fixed dataset of pre-collected transitions, without any additional environment interaction.

22.11.1 CQL — Conservative Q-Learning

CQL (Kumar et al.

CQL modifies the standard Q-learning loss to penalize overestimated Q-values for out-of-distribution (OOD) actions:

$$\mathcal{L}_{\text{CQL}}(\boldsymbol{\theta}) = \alpha \left(\mathbb{E}_{s\sim\mathcal{D},\, a\sim\mu(\cdot|s)} \left[\log \sum_{a'} \exp Q(s, a'; \boldsymbol{\theta}) \right] - \mathbb{E}_{(s,a)\sim\mathcal{D}} [Q(s, a; \boldsymbol{\theta})] \right) + \mathcal{L}_{\text{Bellman}}(\boldsymbol{\theta})$$

The first term (log-sum-exp) pushes *down* the Q-values of actions sampled from a proposal distribution μ (typically the current policy). The second term pushes *up* the Q-values of actions seen in the dataset. The net effect is a conservative Q-function: $Q_\theta \leq Q^*$, preventing the policy from exploiting overestimated values for unseen (state, action) pairs.

CQL Regularization

The penalty α controls conservatism. $\alpha \to 0$ recovers standard Q-learning; large α yields overly pessimistic Q-values. Typical values: $\alpha \in [1, 10]$. CQL is compatible with DQN (discrete), SAC (continuous), and actor-critic architectures.

22.11.2 IQL — Implicit Q-Learning

IQL (Kostrikov et al.

IQL avoids querying OOD actions entirely by using **expectile regression** to estimate the τ-th expectile of the value distribution:

$$\mathcal{L}_V(\psi) = \mathbb{E}_{(s,a)\sim\mathcal{D}} \left[L_2^\tau \left(Q_{\bar{\phi}}(s, a) - V_\psi(s) \right) \right]$$

where $L_2^\tau(u) = |\tau - \mathbb{I}_{u<0}| \cdot u^2$ is the expectile loss. The Q-function is trained with SARSA-style in-sample targets:

$$\mathcal{L}_Q(\phi) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}} \left[(r + \gamma V_\psi(s') - Q_\phi(s, a))^2 \right]$$

The policy is extracted via advantage-weighted regression (AWR):

$$\mathcal{L}_\pi(\theta) = \mathbb{E}_{(s,a)\sim\mathcal{D}} \left[\exp(\beta A(s, a)) \cdot (-\log \pi_\theta(a|s)) \right]$$

where $A(s, a) = Q_\phi(s, a) - V_\psi(s)$.

IQL vs CQL

IQL Pros:

- No explicit OOD action penalty; avoids the bias of conservatism.
- Simpler to tune—the expectile τ ($\in (0.5, 1)$) replaces the CQL α.
- Naturally handles stochastic datasets where bellman backups on OOD pairs are misleading.

IQL Cons:

- The advantage-weighted extraction can be overly conservative if the dataset does not contain near-optimal actions.
- May underperform CQL when the offline dataset is narrow.

22.11.3 Decision Transformer

Decision Transformer (Chen et al.

Decision Transformer reframes offline RL as a sequence modeling problem using a causally masked transformer (GPT architecture):

$$\text{Input: } (\ldots, \hat{R}_{t-1}, s_{t-1}, a_{t-1}, \hat{R}_t, s_t)$$

The model is trained autoregressively to predict actions from the past sequence of returns-to-go $\hat{R}_t = \sum_{k=t}^{T} r_k$, states, and actions. At test time, the desired return (e.g., target cumulative reward) is specified as the initial $\hat{R}_0$, and the model generates actions that achieve it.

Decision Transformer

Demonstrates that generative sequence models can be effective for offline RL without explicit dynamic programming, Bellman backups, or value functions. Particularly suitable for datasets with suboptimal trajectories where conditioning on return-to-go enables stitching of good behavior segments.

22.12 Multi-Agent Reinforcement Learning

22.12.1 Centralized and Decentralized Methods

MADDPG (Lowe et al.

Multi-Agent DDPG uses **centralized training with decentralized execution** (CTDE). Each agent i has an actor $\mu_i(o_i)$ that uses only local observations o_i, but the critic $Q_i(\mathbf{o}, \mathbf{a})$ conditions on the joint observation-action space of all agents during training only. This gives each agent access to the full state during learning while maintaining decentralized execution.

Value Factorization: VDN and QMIX

For cooperative multi-agent settings, factorizing the joint action-value function Q_{tot} into per-agent utilities reduces the combinatorial complexity of the joint action space:
VDN (Sunehag et al., 2018):

$$Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a}) = \sum_{i=1}^{n} Q_i(\tau_i, a_i)$$

Additive decomposition. Assumes joint value is the sum of individual contributions.
QMIX (Rashid et al., 2018):

$$Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a}) = f_{\text{mix}}\big(Q_1(\tau_1, a_1), \ldots, Q_n(\tau_n, a_n);\ s\big)$$

where f_{mix} is a monotonic hypernetwork conditioned on the global state s, enforcing $\frac{\partial Q_{\text{tot}}}{\partial Q_i} \geq 0$. This allows the optimal joint action to be the argmax of each agent's individual Q-function (IGM—Individual-Global-Max principle).

COMA and MAPPO

COMA (Counterfactual Multi-Agent, Foerster et al., 2018): Uses a **counterfactual baseline** for each agent:

$$A_i(s, a_i) = Q(s, \mathbf{a}) - \sum_{a_i'} \pi_i(a_i'|o_i) Q(s, (a_i', \mathbf{a}_{-i}))$$

This marginalizes out agent i's action, providing a baseline that credits only the contribution of a_i beyond what would be expected.

MAPPO (Yu et al., 2022): Applies PPO in the multi-agent setting with CTDE. All agents share parameters with an agent ID conditioning. Training is centralized (value function sees global state), but execution is decentralized. MAPPO is simpler than value-factorization methods and performs competitively on cooperative benchmarks.

22.13 Summary Comparison of RL Algorithms

Choosing an RL Algorithm — Rule of Thumb

- **Discrete actions + high-dim inputs:** Rainbow, DQN variants.
- **Continuous actions + sample-efficient:** SAC, TD3.
- **Continuous actions + stable/parallel:** PPO.
- **Sparse rewards + exploration:** RND, ICM, NGU + PPO.
- **Model available / visual control:** Dreamer, MCTS/AlphaZero.
- **Only offline data:** CQL, IQL, Decision Transformer.
- **Multi-agent:** MAPPO (simple), QMIX (cooperative).

Table 22.1: Comprehensive comparison of RL algorithms with key formulas and properties.

Algorithm	**Type**	**Tradeoffs**	**Key Formula or Mechanism**	**Best Suited For**
TD(0)	Value-based	On-policy	$V(s_t) \leftarrow V(s_t) + \alpha\delta_t$	Prediction; policy evaluation
SARSA	Value-based	On-policy, safe	$Q(s,a) \leftarrow Q + \alpha(r + \gamma Q' - Q)$	Safety-critical exploration
Q-Learning	Value-based	Off-policy, optimistic	$Q(s,a) \leftarrow Q + \alpha(r + \gamma \max Q' - Q)$	Optimal control; replay buffers
DQN	Value-based	Off-policy, NN, discrete	$\mathcal{L} = \mathbb{E}[(r + \gamma \max Q_{\text{targ}} - Q)^2]$	Atari; high-dim discrete
Double DQN	Value-based	Reduces overestimation	Action from online, value from target	Stable Q-learning
Dueling DQN	Value-based	$V + A$ decomp.	$Q = V + A - \frac{1}{\lvert\mathcal{A}\rvert}\sum A$	Many actions, few important
C51 / QR-DQN	Value-based	Distributional	Categorical/quantile distribution of returns	Risk-aware; richer signals
Rainbow	Value-based	Integrated	Combines 6 DQN improvements	Atari benchmark (SOTA)
REINFORCE	Policy-based	MC, unbiased, high var	$\nabla J = \mathbb{E}[G_t \nabla \log \pi]$	Episodic; simple baselines
A2C / A3C	Actor-Critic	Parallel, online	$\nabla J = \mathbb{E}[A_t \nabla \log \pi]$	Scalable; game playing
TRPO	Policy-based	Trust region, 2nd-order	Conjugate gradient + line search	Stable but expensive
PPO	Actor-Critic	1st-order, clipped	$\mathcal{L}^{\text{CLIP}} = \mathbb{E}[\min(rA, \text{clip}(r, 1 \pm \epsilon)A)]$	Continuous control; RLHF
DDPG	Actor-Critic	Off-policy, deterministic	$\nabla J = \mathbb{E}[\nabla_\theta \mu \, \nabla_a Q\vert_{a=\mu}]$	Continuous actions; robotics
TD3	Actor-Critic	Off-policy, stable	Clipped double Q + delayed policy + smoothing	Continuous control (SOTA)
SAC	Actor-Critic	Off-policy, max-entropy	$\mathcal{L}_\pi = \mathbb{E}[\alpha \log \pi - Q]$; auto α	Continuous; exploration-heavy
Dreamer	Model-based	Latent imagination	RSSM + imagined rollouts + value gradients	Visual control; long horizon
AlphaZero	Model-based	MCTS + self-play	PUCT: $Q + cP\frac{\sqrt{\sum N}}{1+N}$	Board games; perfect sim
CQL	Offline	Conservative Q	$\mathcal{L}_{\text{CQL}} = \alpha(\log \sum \exp Q - \mathbb{E}[Q]) + \mathcal{L}_{\text{Bellman}}$	Offline control; D4RL
IQL	Offline	In-sample, expectile	$L_2^\tau(Q - V)$ + AWR extraction	Offline; stochastic data
Decision Transformer	Offline	Sequence model	Autoregressive transformer on $(\hat{R}, s, a)$	Offline; suboptimal trajectories
MADDPG	Multi-Agent	CTDE	Centralized critic, decentralized actor	Mixed coop./comp.
QMIX	Multi-Agent	Value factorization	Monotonic $f_{\text{mix}}(Q_1, \ldots, Q_n; s)$	Cooperative MARL

Chapter 23

Ensemble Methods

Ensemble methods combine multiple base learners to produce a predictor with lower bias, variance, or both. The central insight is that a committee of imperfect models can outperform any single model, provided the errors among members are sufficiently uncorrelated. This chapter covers bagging, boosting, stacking, Bayesian model averaging, deep ensembles, mixture of experts, negative correlation learning, and cascade correlation.

23.1 Bagging (Bootstrap Aggregating)

23.1.1 Bagging — Core Concept

Bagging (Breiman

Given a training set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, generate B bootstrap samples $\mathcal{D}_b$ (each: n draws with replacement from $\mathcal{D}$), train a base learner $\hat{f}_b$ on each, then aggregate:

$$\hat{f}_{\text{bag}}(x) = \begin{cases} \frac{1}{B}\sum_{b=1}^{B} \hat{f}_b(x) & \text{(regression)} \\ \text{majority vote: } \arg\max_k \sum_{b=1}^{B} \mathbf{1}[\hat{f}_b(x) = k] & \text{(classification)} \end{cases}$$

Each bootstrap sample contains approximately 63.2% of the original data (on average), leaving $\approx 36.8\%$ out-of-bag (OOB). The OOB samples serve as a free validation set, and the OOB error provides an unbiased estimate of the generalization error.

Applications

Bagging is most effective with **high-variance, low-bias** base learners (e.g., deep decision trees, unpruned CART). It reduces variance without increasing bias. Classic use cases include random forests for tabular data (classification, regression, survival analysis) and bagged nearest-neighbors. Also used as a building block in modern deep ensembles.

Trade-offs

Pros:

- Reduces variance: $\text{Var}[\hat{f}_{\text{bag}}] \approx \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ where ρ is pairwise correlation among base learners.
- OOB error: free, unbiased validation without a hold-out set.
- Embarrassingly parallel: each bootstrap model trained independently.
- Stable with respect to training-set perturbations.

Cons:

- Does *not* reduce bias; can slightly increase bias if base learners are unstable.
- Computational cost: B models $\times$ base-learner training cost.
- Identical base learners produce $\rho = 1$ and zero benefit.
- Less effective with stable learners (e.g., linear models, k-NN with large k).

Bias-Variance Decomposition for Bagging

For i.i.d. estimators $\hat{f}_b$ with equal variance σ^2 and pairwise correlation ρ:

$$\mathrm{MSE}[\hat{f}_{\mathrm{bag}}] = \underbrace{\mathrm{Bias}^2[\hat{f}_1]}_{\text{unchanged}} + \underbrace{\rho\sigma^2}_{\text{common}} + \underbrace{\frac{1-\rho}{B}\sigma^2}_{\text{reduced by averaging}}$$

Increasing $B \to \infty$ drives the last term to zero, but the correlated variance $\rho\sigma^2$ becomes the irreducible floor. Therefore, the effectiveness of bagging depends critically on **decorrelating** the base learners.

23.1.2 Random Forest

Random Forest (Breiman

Random Forest = Bagging + random feature subspace selection at each split. When growing each tree:

- At each node, randomly select m features from the total d features (without replacement).
- Find the best split *only among these m features.*
- Typical choices: $m = \lfloor\sqrt{d}\rfloor$ for classification, $m = \lfloor d/3\rfloor$ for regression.

The additional randomness reduces ρ (correlation among trees) substantially, yielding a lower-variance ensemble than plain bagging.

Applications

The de facto standard for tabular data. Strong out-of-the-box performance on structured datasets (Kaggle, UCI). Provides feature-importance scores (mean decrease in impurity, permutation importance) and proximity matrices for outlier detection and missing-value imputation. Implementations: scikit-learn `RandomForestClassifier/Regressor`, ranger (R).

Trade-offs

Pros:

- Often the best single off-the-shelf method for tabular data without tuning.
- Handles mixed data types, missing values (via surrogate splits).
- Inherent feature importance and OOB error.
- Resistant to overfitting as B increases.

Cons:

- Large number of trees needed ($B = 500$–2000), memory-intensive.
- Poor extrapolation beyond training-data range (tree prediction is piecewise constant).
- Underperforms gradient boosting on many benchmarks when both are well-tuned.
- Tends to favor features with many split points (biased impurity gains).

23.1.3 Extra-Trees (Extremely Randomized Trees)

Extra-Trees (Geurts et al.

Extra-Trees adds one more layer of randomness: at each split, instead of searching for the best threshold, it draws K random cut-points per candidate feature and picks the best among them (or the best random feature+candidate split). Combined with bagging or using the full training set, the additional randomness further decorrelates trees.

Applications

Faster than standard Random Forest (no exhaustive split search). Often competitive in terms of accuracy, especially on high-dimensional or noisy datasets where variance reduction dominates. Available in scikit-learn as `ExtraTreesClassifier/Regressor`.

Trade-offs

Pros: Even lower variance than RF; faster training (random thresholds). **Cons:** Slightly higher bias due to suboptimal splits; may require more trees to match RF accuracy on clean datasets.

23.1.4 Pasting

Pasting (Breiman

Like bagging but samples *without replacement*. Each base learner is trained on a random subset (typically 50–80%) of the training data. No bootstrap duplication; all samples within a subset are unique.

Applications

Useful when the training set is large enough that smaller subsets suffice or when bootstrap duplication would over-represent some samples. Can be combined with random subspaces (Random Patches method) for very large datasets.

Trade-offs

Pros: Smaller training sets per learner → faster training. **Cons:** Subset size is a hyper-parameter; no OOB estimate.

23.2 Boosting

Boosting converts a set of *weak learners* (models slightly better than random guessing) into a *strong learner* by training them sequentially, each focusing on the mistakes of its predecessor.

23.2.1 AdaBoost

AdaBoost — Adaptive Boosting (Freund & Schapire

Initialize sample weights $w_i^{(1)} = 1/n$. For $m = 1, \ldots, M$:

1. Train weak learner $G_m(x)$ on weighted samples.
2. Compute weighted error: $\text{err}_m = \frac{\sum_{i=1}^{n} w_i^{(m)} \mathbf{1}[y_i \neq G_m(x_i)]}{\sum_i w_i^{(m)}}$.
3. Compute classifier weight: $\alpha_m = \frac{1}{2} \ln\left(\frac{1-\text{err}_m}{\text{err}_m}\right)$.
4. Update sample weights: $w_i^{(m+1)} = w_i^{(m)} \exp(-\alpha_m y_i G_m(x_i))$, then renormalize by $Z_m = \sum_i w_i^{(m+1)}$.

Final classifier: $f(x) = \text{sign}\left(\sum_{m=1}^{M} \alpha_m G_m(x)\right)$. For multi-class, use SAMME (Stagewise Additive Modeling using a Multi-class Exponential loss) or SAMME.R (real-valued variant).

AdaBoost as Forward Stagewise Additive Modeling

AdaBoost is equivalent to minimizing the exponential loss $\sum_i \exp(-y_i f(x_i))$ via forward stagewise additive modeling:

$$(\alpha_m, G_m) = \arg\min_{\alpha, G} \sum_{i=1}^{n} w_i^{(m)} \exp(-\alpha y_i G(x_i))$$

with $w_i^{(m)} = \exp(-y_i f_{m-1}(x_i))$. The exponential loss upper-bounds the 0-1 loss and drives AdaBoost's aggressive focus on misclassified points.

Applications

Primarily binary classification with decision stumps or shallow trees as weak learners. Historically important in face detection (Viola-Jones cascade with AdaBoost + Haar features). Still used in rapid prototyping and when interpretability of weak-learner weights matters.

Trade-offs

Pros:

- Theoretical guarantees: training error drops exponentially fast.
- Simple to implement; no hyperparameter beyond M (and base-learner complexity).
- Automatic feature selection when stumps are used.

Cons:

- Sensitive to label noise: exponential loss heavily up-weights misclassified noisy examples → can overfit badly.
- Not directly applicable to regression (requires modification).
- Slower than modern gradient boosting (no second-order info).
- Weak learners must be re-weighted; not all algorithms support sample weights.

23.2.2 Gradient Boosting

Gradient Boosting (Friedman

At iteration m, fit a base learner $h_m(x)$ to the **negative gradient** (pseudo-residuals) of the loss w.r.t. the current ensemble:

$$r_{im} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f=f_{m-1}}$$

Then update:

$$f_m(x) = f_{m-1}(x) + \nu \cdot \rho_m \cdot h_m(x)$$

where $\rho_m = \arg\min_\rho \sum_i L(y_i, f_{m-1}(x_i) + \rho\, h_m(x_i))$ (line search) and $\nu \in (0, 1]$ is the learning rate (shrinkage). Commonly, ρ_m is set to 1 per leaf and each leaf value is fit to the response; this is called **TreeBoost**.

Pseudo-Residuals by Loss Function

Loss	$L(y, f)$	r_{im} (pseudo-residual)
L2 (MSE)	$\frac{1}{2}(y-f)^2$	$y_i - f_{m-1}(x_i)$
L1 (MAE)	$\lvert y-f\rvert$	$\text{sign}(y_i - f_{m-1}(x_i))$
Huber	$\begin{cases} \frac{1}{2}r^2 & \lvert r\rvert \le \delta \\ \delta\lvert r\rvert - \frac{1}{2}\delta^2 & \lvert r\rvert > \delta \end{cases}$	clipped residual
Logistic (binary)	$\log(1+e^{-2yf})$	$\frac{2y_i}{1+\exp(2y_i f_{m-1}(x_i))}$

Applications

The workhorse for tabular data in industry and competitions. LightGBM, XGBoost, and CatBoost are all gradient-boosting variants. Use L2 loss for regression, logistic/deviance for classification, and ranking losses (LambdaRank) for learning-to-rank. Shrinkage $\nu = 0.01$–0.1 with $M = 100$–1000 trees is standard.

Trade-offs

Pros:

- Any differentiable loss → flexible (custom losses possible).
- Typically best tabular-data accuracy (especially XGBoost/LightGBM/CatBoost).
- Built-in regularization: shrinkage, subsampling, tree constraints.
- Feature importance and partial dependence readily available.

Cons:

- Sequential training — inherently not parallel across trees.
- Can overfit without early stopping (monitor validation loss).
- More hyperparameters than Random Forest (tree depth, min child weight, subsample, colsample, ν, M, regularization).
- Sensitive to hyperparameters; grid/random search often needed.

23.2.3 XGBoost

XGBoost — eXtreme Gradient Boosting (Chen & Guestrin

XGBoost uses a second-order Taylor expansion of the loss, adding a regularization term $\Omega(f_t)$ to the objective at round t:

$$L^{(t)} = \sum_{i=1}^{n} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

where $g_i = \partial_{\hat{y}^{(t-1)}} L(y_i, \hat{y}_i^{(t-1)})$, $h_i = \partial^2_{\hat{y}^{(t-1)}} L(y_i, \hat{y}_i^{(t-1)})$, and $\Omega(f_t) = \gamma T + \frac{1}{2}\lambda \|w\|^2$ for a tree with T leaves and leaf weights w.

Split Gain: For a candidate split partitioning instances into left (L) and right (R) sets:

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

where $G_{L/R} = \sum_{i \in L/R} g_i$, $H_{L/R} = \sum_{i \in L/R} h_i$. A split is made only if Gain > 0 (prunes branches automatically).

Applications

The dominant method in Kaggle tabular competitions (2015–2019). Supports regression, binary/multi-class classification, ranking, and survival analysis. Features: approximate histogram-based split finding (faster than exact greedy), sparsity-aware split proposals, column block for parallelization, cache-aware access, out-of-core computation. GPU training via `tree_method='gpu_hist'`.

Trade-offs

Pros: Second-order gradients + regularization yields more accurate splits; highly optimized C++ backend; native handling of missing values (default direction per split); built-in cross-validation and early stopping. **Cons:** Many hyperparameters (η, max_depth, min_child_weight, γ, λ, α, subsample, colsample_bytree, colsample_bylevel); slower per-tree than LightGBM on large datasets; categorical features must be encoded (one-hot or label) unless using native support.

23.2.4 LightGBM

LightGBM — Light Gradient Boosting Machine (Ke et al.

Two key innovations for speed and scalability:

- **GOSS (Gradient-based One-Side Sampling):** Keep all instances with large absolute gradients (high error) and randomly sample from instances with small gradients, re-weighting the latter by $\frac{1-a}{b}$ to correct the sampling bias. Hyperparameters: top-rate a, other-rate b.
- **EFB (Exclusive Feature Bundling):** Bundle features that are mutually exclusive (rarely take non-zero values simultaneously) into a single feature, reducing dimensionality without much information loss. Solved via graph-coloring approximation.

Uses leaf-wise (best-first) tree growth with max_depth constraint, growing the leaf with the highest split gain.

Applications

Fastest GBDT for large-scale data ($n > 10^6$, high-dimensional). Native support for categorical features (no one-hot encoding needed). Strong on sparse data. Default choice for large tabular datasets where training speed matters.

Trade-offs

Pros: Much faster training than XGBoost on large data; native categorical support; lower memory footprint; leaf-wise growth can produce more accurate models with fewer trees (if max_depth is tuned). **Cons:** Leaf-wise growth can overfit on small datasets ($n < 10^4$); more sensitive to num_leaves and min_data_in_leaf settings; EFB may introduce small bias on non-sparse features.

23.2.5 CatBoost

CatBoost — Categorical Boosting (Prokhorenkova et al.

Key innovations:

- **Ordered Boosting:** Train on a random permutation of the data to avoid target leakage from the encoding step. For sample i, the model f_{i-1} is trained only on the first $i-1$ samples in the permutation, preventing the prediction shift problem of standard GBDT.
- **Ordered Target Encoding:** For categorical feature k with value σ, the encoding for sample i is computed only from samples preceding i in the permutation, using a smoothed prior:

$$\text{Enc}_{\sigma,i} = \frac{\sum_{j<i} \mathbf{1}[x_j^{(k)} = \sigma]\, y_j + a\, p}{\sum_{j<i} \mathbf{1}[x_j^{(k)} = \sigma] + a}$$

  where p is a prior (e.g., global target mean) and $a > 0$ smooths towards the prior.
- Symmetric trees (oblivious decision trees): same split condition used at every node of a given depth, enabling extremely fast inference.

Applications

Excels when the data contains many categorical features (dozens to hundreds) with high cardinality. Often achieves top accuracy with minimal hyperparameter tuning. Default choice when categorical features dominate and you want a single, reliable model with

strong out-of-the-box performance.

Trade-offs

Pros: Best accuracy with many categoricals; minimal tuning needed; symmetric trees → fast CPU inference; less prone to overfitting due to ordered approach. **Cons:** Slower training than LightGBM on purely numeric data; symmetric trees can be less expressive (offset by using more iterations); some overhead from permutation-based training.

23.2.6 LogitBoost

LogitBoost (Friedman

A boosting algorithm that minimizes the log-likelihood loss (rather than exponential or squared-error) using adaptive Newton steps. For binary classification:

1. Compute working response: $z_i = \frac{y_i^* - p(x_i)}{p(x_i)(1-p(x_i))}$ and weights: $w_i = p(x_i)(1-p(x_i))$ where $p(x) = \frac{1}{1+e^{-2f(x)}}$ and $y_i^* = (y_i+1)/2 \in \{0,1\}$.
2. Fit weak learner h_m to z_i with weights w_i.
3. Update: $f_m(x) = f_{m-1}(x) + \frac{1}{2}h_m(x)$.

Applications

Naturally handles probabilistic outputs (well-calibrated for logistic loss). Often used as a building block in ensemble meta-learners and as a direct competitor to logistic regression with non-linear basis functions.

Trade-offs

Pros: Directly optimizes log-likelihood (better calibrated probabilities); numerically stable with adaptive Newton steps. **Cons:** More computationally expensive per iteration than AdaBoost; can be unstable when $p(x_i)$ is near 0 or 1 (extreme weight values).

23.3 Stacking (Stacked Generalization)

Stacking (Wolpert

A two-level (or multi-level) ensemble architecture:

- **Level-0 (base learners):** Train K diverse models $f_1, f_2, \ldots, f_K$ on the full training set (or via cross-validation to avoid overfitting).
- **Level-1 (meta-learner):** Train a model g on the outputs of the base learners: the meta-features are $\boldsymbol{z}_i = (f_1(x_i), f_2(x_i), \ldots, f_K(x_i))$.

The final prediction is $g(\boldsymbol{z}(x))$. Using K-fold cross-validation to generate the meta-features (each fold's predictions from models trained on the remaining folds) prevents the meta-learner from simply memorizing which base learner memorized which training samples.

Super Learner (van der Laan et al.

The Super Learner uses cross-validated stacking and a loss-based meta-learner. Given a library of K candidate algorithms and a loss function L, choose the convex combination $\boldsymbol{\alpha}$ minimizing the cross-validated risk:

$$\hat{\boldsymbol{\alpha}} = \arg\min_{\boldsymbol{\alpha} \in \Delta^{K-1}} \sum_{i=1}^{n} L\left(y_i, \sum_{k=1}^{K} \alpha_k \hat{f}_k^{(-i)}(x_i)\right)$$

where $\hat{f}_k^{(-i)}$ is trained on all data except fold containing i. Under mild conditions, the Super Learner is asymptotically as good as the oracle (best convex combination), a consequence of the oracle inequality.

Applications

Widely used in Kaggle competitions to combine predictions from diverse model families (GBDT + neural nets + linear models + k-NN). The meta-learner is typically a simple model (linear/logistic regression) to avoid overfitting at the second level; non-linear meta-learners (e.g., shallow GBDT) can help when base predictions have complex interactions.

Trade-offs

Pros:

- Can combine arbitrarily different model families ($\rightarrow$ high diversity).
- Theoretically optimal (Super Learner) under convex combination.
- Cross-validation framework avoids data leakage.

Cons:

- $K\times$ training cost (or more with CV); expensive at scale.
- Risk of meta-learner overfitting if base-learner predictions are highly correlated or if the meta-learner is too complex.
- Requires careful cross-validation fold management; incorrect implementation leads to data leakage and over-optimistic estimates.

Stacking vs. Blending

Blending is a simpler variant: hold out a validation set, train base learners on the rest, and train the meta-learner on validation predictions. Faster but wastes data and risks higher variance. Stacking with k-fold CV uses all training data for meta-feature generation and is generally preferred.

23.4 Bayesian Model Averaging (BMA)

Bayesian Model Averaging

Given an observed dataset $\mathcal{D}$ and a set of candidate models $\{M_1, \ldots, M_m\}$, the predictive distribution is obtained by averaging over models weighted by their posterior probabilities:

$$p(y \mid x, \mathcal{D}) = \sum_{m=1}^{M} p(y \mid x, M_m, \mathcal{D})\, p(M_m \mid \mathcal{D})$$

where $p(M_m \mid \mathcal{D}) \propto p(\mathcal{D} \mid M_m)\, p(M_m)$ and the marginal likelihood (evidence) $p(\mathcal{D} \mid M_m)$ automatically implements an Occam's razor penalty against overly complex models.

Applications

Ideal when models represent genuinely different structural hypotheses about the data-generating process (e.g., different variable subsets in linear regression, different causal graphs). Used in econometrics, climate modeling, and probabilistic programming. Also inspired deep ensembles and MCMC-based Bayesian neural networks.

Trade-offs

Pros: Principled uncertainty quantification; automatic complexity penalty via marginal likelihood; full predictive distribution. **Cons:** Requires the true model to be in the candidate set (M-closed assumption); computing marginal likelihoods is often intractable (requires MCMC or variational approximations); model space grows combinatorially with features; prior sensitivity.

BMA vs. Model Selection vs. Ensemble Methods

- **BMA** averages over all models weighted by how likely they are to be true. Good for inference, but the M-closed assumption is rarely met in practice.
- **Stacking** weights models by predictive performance, not posterior probability. No assumption about truth of any model. Empirically, stacking often outperforms BMA for prediction.
- **Bagging/Boosting** combine weak learners of the same type, not structurally different models.

23.5 Deep Ensembles

23.5.1 Naive Deep Ensemble

Deep Ensemble (Lakshminarayanan et al.

Train M copies of the same neural network architecture independently with different random initializations and data-order shuffles. Aggregate by averaging the predicted probabilities (classification) or the predicted means and variances (regression, when each network outputs both μ and σ^2).

Applications

Surprisingly effective for uncertainty quantification, often outperforming more sophisticated Bayesian neural network approximations. Standard baseline in robustness, OOD detection, and active learning benchmarks. Training cost is $M\times$ the cost of one network, but this is embarrassingly parallel.

Trade-offs

Pros: Simple; no architectural changes; state-of-the-art uncertainty quality; well-calibrated when combined with temperature scaling. **Cons:** $M\times$ training and inference

cost; $M\times$ memory for deployment; no theoretical guarantee that the ensemble approximates the Bayesian posterior (it does not).

23.5.2 MC Dropout

MC Dropout (Gal & Ghahramani

Apply dropout at *test time* and perform T stochastic forward passes, then aggregate:

$$p(y \mid x) \approx \frac{1}{T} \sum_{t=1}^{T} p(y \mid x, \boldsymbol{\omega}_t)$$

where $\boldsymbol{\omega}_t$ is a random dropout mask. This can be interpreted as a variational approximation to a deep Gaussian process, with the dropout rate acting as the variational parameter. The predictive variance naturally captures epistemic uncertainty.

Applications

Uncertainty estimation without any architectural change; widely used in Bayesian deep learning baselines, medical image segmentation, and active learning where separate ensemble training is too expensive. Any dropout network becomes an ensemble for free at test time.

Trade-offs

Pros: No extra training cost; leverages existing dropout layers; well-studied connection to variational inference. **Cons:** Uncertainty quality depends heavily on dropout rate (often needs grid search per layer); under-estimates uncertainty compared to deep ensembles; T forward passes slow down inference; calibration is generally worse than deep ensembles.

23.5.3 BatchEnsemble

BatchEnsemble (Wen et al.

Factor the weight matrix of ensemble member i as:

$$W_i = W \circ (r_i s_i^\top)$$

where $r_i \in \mathbb{R}^{d_{\text{out}}}$, $s_i \in \mathbb{R}^{d_{\text{in}}}$ are low-rank per-member "fast weight" vectors and W is a shared "slow weight" matrix. All M ensemble members can be evaluated in a single vectorized forward pass, achieving ensemble benefits at $\approx 1\times$ the cost of a single network.

Applications

Memory-efficient deep ensembles for deployment-constrained settings. The factorization keeps the parameter overhead to $\mathcal{O}(M(d_{\text{in}} + d_{\text{out}}))$ per shared weight matrix, typically a $< 5\%$ parameter increase for $M = 4$.

Trade-offs

Pros: Near-ensemble performance at single-model cost; trivial vectorized implementation; well-suited for distributed training. **Cons:** Slightly lower accuracy than independent ensembles; hyperparameter(s) controlling rank-1 factor initialization; not all layer types trivially factorable (layernorm, attention).

23.5.4 Snapshot Ensemble

Snapshot Ensemble (Huang et al.

Train a *single* network with a cyclic (cosine annealing with restarts) learning rate schedule. At the end of each cycle (when the LR is at its minimum), take a **snapshot** of the model weights. After training, the ensemble consists of these snapshots, which are diverse because the sudden LR jumps between cycles push the model into different local minima.

Applications

Cost-free ensembles: obtain M diverse models from one training run. Useful when training budget is tight but ensemble benefits are desired for test-time accuracy or uncertainty. Implemented in the `torch.optim.lr_scheduler.CosineAnnealingWarmRestarts`.

Trade-offs

Pros: Single training run; no extra hyperparameters beyond cycle length and LR range; naturally diverse snapshots. **Cons:** Requires knowing total budget to set cycle lengths; snapshots from early cycles usually underfit; ensemble diversity is heuristic, not guaranteed; less effective than independently trained ensembles.

23.5.5 FGE (Fast Geometric Ensembling)

Fast Geometric Ensembling — FGE (Garipov et al.

Train a single network with a constant learning rate SGD. Once the loss has stabilized around a local minimum, continue training and periodically collect model checkpoints along the SGD trajectory. The resulting models live on a low-loss manifold (basin); averaging them yields a better predictor than any single point on that manifold. No cyclic schedule needed.

Applications

Similar use case to Snapshot but without requiring a cyclic LR. Can be applied to pre-trained models by continuing training at a low LR and collecting checkpoints. Often achieves better accuracy than Snapshot Ensemble with the same checkpoint budget.

Trade-offs

Pros: No LR schedule design; works with constant LR; easy to apply post-hoc to existing checkpoints. **Cons:** High checkpoint frequency needed (every 1–5 epochs); models may not be diverse enough if LR is too small; storage of M checkpoints.

23.6 Mixture of Experts (MoE)

Mixture of Experts — MoE (Jacobs et al.

A set of n **expert networks** $E_1, \ldots, E_n$ plus a **gating network** $G(x)$ that outputs a probability distribution over experts. The MoE output is a weighted combination:

$$y = \sum_{i=1}^{n} G(x)_i \cdot E_i(x)$$

where $G(x) = \text{softmax}(\text{TopK}(H(x), k))$ for sparse MoE and $G(x) = \text{softmax}(H(x))$ for dense MoE. Both the experts and the gate are trained jointly via backpropagation. The gate naturally learns to specialize experts to different input regions.

23.6.1 Sparse Mixture of Experts

Sparse MoE (Shazeer et al.

For each input x, compute gating logits $H(x) \in \mathbb{R}^n$ and select the top-k experts ($k \ll n$, typically $k = 1$ or 2):

$$G(x) = \text{softmax}\big(\text{TopK}(H(x), k)\big)$$

where TopK keeps the k largest logits and sets the rest to $-\infty$ (zero probability after softmax). Only the selected experts run their forward and backward passes, making MoE computationally efficient despite a very large total parameter count. With n experts and top-k selection, the effective FLOPs per token are $\approx k/n$ of a dense model with the same total parameters.

Load Balancing Loss for Sparse MoE

To prevent the gate from always routing to the same few experts, add an auxiliary loss encouraging uniform expert utilization:

$$\mathcal{L}_{\text{aux}} = w \cdot n \cdot \sum_{i=1}^{n} f_i \cdot P_i$$

where $f_i = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} \mathbf{1}[\text{expert } i \text{ selected for } x]$ (fraction of tokens dispatched to expert i) and $P_i = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} G(x)_i$ (average gating probability for expert i across batch $\mathcal{B}$). w is a small balancing coefficient (e.g., 0.01). Minimized when $f_i = P_i = 1/n$ (perfect uniform load).

Applications

Core scaling technique for large language models: Mixtral 8×7B, GPT-4 (reported), Gemini, DeepSeek-V2. Scales model capacity to trillions of parameters while keeping per-token inference cost near that of a dense model 1–2 orders of magnitude smaller. Also used in vision (V-MoE) and multi-modal models.

Trade-offs

Pros:

- Massive parameter scaling at sub-linear compute cost.
- Natural specialization: experts learn different skills/functions.
- Sparse activation can improve generalization (fewer co-adapted parameters active at once).

Cons:

- Hard Top-K selection is non-differentiable; requires straight-through estimator for end-to-end training.
- Load balancing is an additional tuning burden; poor balance causes under-utilized experts and wasted capacity.
- Communication overhead in distributed training (all-to-all dispatch and combine across devices).
- Load imbalance leads to straggler devices.
- Expert capacity limits can drop tokens, hurting accuracy.

23.6.2 Switch Transformer

Switch Transformer (Fedus et al.

A simplified sparse MoE using **top-1 gating** ($k = 1$): each token is routed to exactly one expert. This simplifies routing, reduces communication, and improves training stability. The load-balancing loss uses a simplified form: $\mathcal{L}_{\text{aux}} = w \cdot n \cdot \sum_{i=1}^{n} f_i \cdot P_i$ as above. Switch Transformer demonstrated that scaling the number of experts while keeping per-token FLOPs constant leads to improved downstream performance.

Applications

Scaling laws research; T5-based Switch Transformer achieved 7× faster pre-training than a dense T5-Large at matched quality. Preferred when simplicity and training speed outweigh the benefits of top-2 gating.

Trade-offs

Pros: Simpler routing (only 1 expert); reduced all-to-all communication; more stable training than top-$k > 1$ gating. **Cons:** Single expert per token may limit expressivity; expert capacity must be carefully set to avoid dropped tokens; still requires load-balancing tuning.

23.6.3 Soft MoE

Soft MoE (Puigcerver et al.

Instead of hard routing tokens to experts via TopK, Soft MoE assigns each token to *every* expert via softmax weights that are fully differentiable:

$$\tilde{E}_j = \sum_{i=1}^{n_{\text{tokens}}} \phi_{ij} \cdot \text{Token}_i$$

where $\phi_{ij} = \text{softmax}_i(W_\phi \cdot \text{Token})_j$ is a learnable dispatch weight. Each expert processes a convex combination of all tokens. Outputs are then combined back to token space via

another softmax combining weight. No auxiliary balancing loss is required in principle.

Applications

Emerging alternative to sparse MoE when fully differentiable training is desired (no straight-through estimator). Competitive performance with hard MoE in preliminary experiments on vision and language.

Trade-offs

Pros: Fully differentiable (no straight-through); no load-balancing auxiliary loss needed; naturally smooth expert specialization. **Cons:** Each expert processes all tokens (no compute savings from sparsity); dispatch and combine matrices scale as $n_{\text{tokens}} \times n_{\text{experts}}$ (heavy memory for long sequences); less proven at scale than sparse MoE.

23.6.4 Other MoE Variants

Notable MoE Architectures

- **GShard (Lepikhin et al., 2021):** First massive-scale sparse MoE for translation (600B parameters). Demonstrated scaling laws for expert count.
- **DeepSpeed-MoE (Rajbhandari et al., 2022):** System optimizations for MoE including pyramid-shaped MoE (more experts in later layers) and residual-MoE (dense residual + sparse MoE branch).
- **ST-MoE (Zoph et al., 2022):** Stability improvements: router z-loss for preventing large logits, multiplicative jitter noise on gating, and expert dropout for regularization. Key reference for training stable sparse MoEs at scale.

23.7 Negative Correlation Learning (NCL)

Negative Correlation Learning (Liu & Yao

A parallel ensemble training method that explicitly penalizes positive correlation among ensemble members during optimization. The loss for member i in an ensemble of M networks includes a correlation penalty:

$$\mathcal{L}_i = \mathcal{L}_{\text{task}} + \lambda \sum_{j \neq i} (f_i - f)(f_j - f)$$

where $f = \frac{1}{M} \sum_{k=1}^{M} f_k$ is the ensemble mean output. The penalty term is positive when members make similar errors in the same direction (positive correlation) and negative when errors are opposite (negative correlation). Training can be done simultaneously (all members updated in parallel) with a modified backpropagation.

NCL Ensemble Bias-Variance-Covariance Decomposition

For an ensemble with equal weights, the MSE decomposes as:

$$\text{MSE} = \overline{\text{bias}}^2 + \frac{1}{M}\overline{\text{var}} + \frac{M-1}{M}\overline{\text{covar}}$$

where $\overline{\text{bias}}$, $\overline{\text{var}}$, and $\overline{\text{covar}}$ are averages over the M members. NCL directly targets the covariance term, reducing it (or making it negative) to lower the overall ensemble error.

Applications

Training diverse neural network ensembles in one pass (no sequential or independent training needed). Works naturally with gradient-based optimization. Used in regression ensembles, time-series forecasting, and tasks where member diversity is critical.

Trade-offs

Pros: Explicit control over ensemble diversity; parallel training; strong theoretical foundation in bias-variance-covariance trade-off. **Cons:** Hyperparameter λ sensitive — too small gives little decorrelation, too large harms individual accuracy; gradient of the penalty term couples all members (synchronization overhead); may not scale well to very large M.

23.8 Cascade Correlation

Cascade Correlation (Fahlman & Lebiere

A constructive neural network architecture that grows its topology during training:

1. Start with a minimal network (inputs → outputs, no hidden units).
2. Train the output weights to convergence.
3. If residual error is too large, **add one hidden unit**:
   - Candidate unit receives connections from all inputs and existing hidden units (weights are trained to maximize the correlation between its output and the residual error).
   - Once trained, the candidate's input weights are frozen.
   - The candidate is added to the network with output connections that are trained (while previous weights remain frozen).
4. Repeat until error is sufficiently low.

The "cascade" architecture: each new hidden unit connects to all inputs and all previous hidden units, creating an increasingly deep and interconnected network.

Applications

Historically important for automatic architecture search and constructive learning. Useful when the required network depth/complexity is unknown a priori. Provides a principled way to incrementally grow a model's capacity. Modern applications include growing neural networks for continual learning and meta-learning.

Trade-offs

Pros: No need to choose network architecture upfront; each new unit specifically targets residual error; frozen weights prevent catastrophic forgetting during growth; fast training (only output weights and one candidate trained at a time). **Cons:** Tends to overfit on small datasets; cascade architecture creates deep, thin networks (each unit has one hidden neuron per layer) which may not be optimal; non-parallelizable (sequential unit addition); candidate training can be unstable (correlation maximization via gradient ascent).

23.9 Summary Comparison of Ensemble Methods

Table 23.1: Comparison of Ensemble Methods

Method	Combination	Diversity Source	Training	Key Characterist
Bagging	Average (or vote)	Bootstrap samples	Parallel	Reduces variance; ror free
Random Forest	Average / vote	Bootstrap + random features	Parallel	De facto tabular-da
Extra-Trees	Average / vote	Bootstrap + random splits	Parallel	Even more randon faster
Pasting	Average / vote	Without-replacement subsets	Parallel	Faster per-learner t ging
AdaBoost	Weighted vote	Sample re-weighting	Sequential	Exponential loss; to noise
Gradient Boosting	Additive (shrinkage)	Pseudo-residual fitting	Sequential	Any differentiable l
XGBoost	Additive (2nd-order)	2nd-order residual fitting	Sequential	Regularized; domi ML
LightGBM	Additive	GOSS + EFB	Sequential	Fastest GBDT; la data
CatBoost	Additive	Ordered boosting	Sequential	Best for many categ
LogitBoost	Additive	Log-likelihood Newton steps	Sequential	Well-calibrated prol
Stacking	Meta-learner	Diverse model families	2-level CV	Combines heter models
Super Learner	Convex combo (meta)	Diverse library + CV	2-level CV	Asymptotically optimal
BMA	Posterior average	Different model structures	MAP / MCMC	Principled uncertai closed
Deep Ensemble	Average (or vote)	Random seed + shuffling	Parallel	Simple; SOTA unce
MC Dropout	Average T passes	Random dropout masks	Single + T test	"Free" ensemble dropout
BatchEnsemble	Average	Rank-1 factorization	Parallel (vectorized)	$1\times$ cost of single m
Snapshot Ensemble	Average	LR cycle restarts	Single run	Cost-free; cyclic an
FGE	Average	SGD trajectory	Single run + collect	Cost-free; constant
MoE (Sparse)	Weighted sum (top-k)	Hard gating + load balancing	Joint (straight-through)	Massive capacity linear cost
Switch Transformer	Top-1 weighted sum	Top-1 gating	Joint	Simpler sparse M faster training
Soft MoE	Soft weighted sum	Fully soft assignment	Joint (differentiable)	No straight-throug token processing
NCL	Average	Correlation penalty	Parallel (coupled)	Explicit diversity c tion
Cascade Correlation	Feedforward cascade	Constructive growth	Sequential (unit-by-unit)	Grows architectur scratch

Practical Recommendations

- **Tabular data:** Start with LightGBM or XGBoost. If many categorical features dominate, CatBoost. For interpretability or quick baselines, Random Forest.
- **Uncertainty quantification:** Deep Ensembles (independent training) remain the strongest simple baseline. MC Dropout is a lightweight approximation.
- **Deployment-constrained:** BatchEnsemble gives near-ensemble performance at single-model cost. Snapshot Ensemble and FGE are training-time alternatives.

- **LLM / massive model scaling:** Sparse MoE (top-2 gating) or Switch Transformer. Use ST-MoE stability techniques (router z-loss, expert dropout).
- **Model combination in competitions:** Stacking with a linear meta-learner on diverse base models (GBDT + NNs + linear).
- **Small data with strong priors:** BMA with well-specified candidate models for principled inference.
- **One-pass diversity training:** NCL adds a lightweight decorrelation penalty; useful when independent training is impractical.
- **Unknown architecture:** Cascade Correlation (or modern NAS alternatives) for constructive model-building.

Chapter 24

Decision Tree Splitting Criteria

Decision trees recursively partition the input space into axis-aligned regions, assigning a prediction to each leaf. The choice of splitting criterion is the algorithmic core that determines which feature to split on and at what threshold. This chapter surveys classification and regression split criteria, pruning strategies, ensemble-specific gains, oblique splits, and multi-way versus binary partitioning.

24.1 Classification Splitting Criteria

24.1.1 Gini Impurity

Gini Impurity — CART

For a node t with C classes and empirical class proportions $p(c|t)$, the Gini impurity is:

$$I_G(t) = 1 - \sum_{c=1}^{C} p(c|t)^2$$

Range: $[0,\ 1 - \frac{1}{C}]$. Gini impurity measures the probability of misclassifying a randomly chosen sample if it were labeled according to the class distribution at node t. For a binary split into left child t_L and right child t_R with samples n_L and n_R respectively, the Gini gain is:

$$\Delta I_G = I_G(t) - \frac{n_L}{n} I_G(t_L) - \frac{n_R}{n} I_G(t_R)$$

where $n = n_L + n_R$. The split that maximizes ΔI_G is selected. Gini is the default criterion for CART (Classification and Regression Trees).

Applications

Gini impurity is the default splitting criterion in scikit-learn's `DecisionTreeClassifier` (`criterion='gini'`), XGBoost's `multi:softmax` classifier, and LightGBM's classification trees. It is computationally cheaper than entropy (no logarithms) and behaves very similarly in practice.

Trade-offs

Pros:

- Computationally efficient — avoids logarithms, only requires squaring class probabilities.
- Tends to isolate the majority class: Gini splits can produce purer nodes faster than entropy.
- Numerically stable even with many classes and small probabilities.
- Empirically, classification accuracy differences between Gini and entropy are negligible in most settings.

Cons:

- Not derived from an information-theoretic principle — lacks the interpretability of entropy-based measures.
- Slightly more biased toward creating balanced splits compared to entropy (which favors purity regardless of child size).
- For multi-class problems with many classes that are not well represented, Gini can be insensitive to rare class distributions.

Gini vs. Misclassification Error

Gini is a strictly concave function of $p(c|t)$, making it more sensitive to changes in class probabilities than misclassification error $1 - \max_c p(c|t)$. Consider a two-class node with split candidates:

- Node with $p = (0.5, 0.5)$: Gini $= 0.5$, Error $= 0.5$.
- Node with $p = (0.9, 0.1)$: Gini $= 0.18$, Error $= 0.1$.

The Gini decrease (0.32) far exceeds the error decrease (0.4), making it a "stronger" signal for split quality. Misclassification error often fails to discriminate between competing splits.

24.1.2 Entropy and Information Gain

Entropy — ID3 / C4.5

The entropy of node t with class proportions $p(c|t)$ is:

$$I_E(t) = -\sum_{c=1}^{C} p(c|t)\, \log_2 p(c|t)$$

Range: $[0,\ \log_2 C]$. Entropy is zero when all samples belong to the same class (perfect purity) and maximal when classes are uniformly distributed. The information gain for a candidate split on attribute a is:

$$\mathrm{IG}(t, a) = H(t) - \sum_{v \in \text{values}(a)} \frac{|t_v|}{|t|} H(t_v)$$

where $H(t) \equiv I_E(t)$ and t_v is the subset of t for which attribute a takes value v. The split that maximizes IG is selected. This is the core criterion in ID3 and, before normalization, C4.5.

Applications

Entropy is used in ID3 (Quinlan, 1986) and as the unnormalized component of Gain Ratio in C4.5. In scikit-learn, `criterion='entropy'` switches from Gini to information gain. Preferred in settings where interpretability or information-theoretic analysis of splits matters.

Trade-offs

Pros:

- Deep information-theoretic foundation: measurably quantifies the reduction in uncertainty about the class label.
- More aggressive pursuit of purity: entropy favors nodes dominated by a single class,

yielding shallower trees in some cases.
- Widely studied theoretical properties (concavity, additivity, chain rule of entropy).

Cons:
- Computes $\log_2$ at every candidate split — slightly slower than Gini on large datasets.
- **Favors features with many distinct values** (e.g., an ID column yields perfect information gain but no generalization). This is the well-known overfitting bias — mitigated by Gain Ratio.
- At extreme class probabilities near 0 or 1, numerical issues arise ($\log_2 0$ must be handled as limit 0).

24.1.3 Gain Ratio

Gain Ratio — C4.5

To counteract information gain's bias toward attributes with many values, C4.5 normalizes IG by the split's intrinsic entropy:

$$\mathrm{GR}(t,a) = \frac{\mathrm{IG}(t,a)}{\mathrm{SplitInfo}(t,a)}, \qquad \mathrm{SplitInfo}(t,a) = -\sum_{v \in \text{ values}(a)} \frac{|t_v|}{|t|} \log_2 \frac{|t_v|}{|t|}$$

SplitInfo measures how uniformly the samples are distributed across the outcomes of attribute a. An attribute with many values and uniform distribution of samples yields high SplitInfo, reducing the gain ratio. In C4.5, Gain Ratio is used *only* for attributes whose IG exceeds the average IG across all candidate attributes; otherwise the attribute may be skipped.

Applications

The definitive splitting criterion for the C4.5 algorithm and its successor C5.0. Implemented in Weka's J48 classifier, R's `C50` package, and available in the `chefboost` Python library. Preferred when the feature space includes categorical variables with widely varying cardinalities.

Example: Why Gain Ratio Helps

Consider a dataset where one feature is a unique row ID.
- IG $\approx \log_2 N$ (perfect classification of training data).
- SplitInfo $\approx \log_2 N$ (appears uniform across N values).
- GR ≈ 1, while a genuinely predictive feature might have GR $\gg 1$.

Without normalization, ID3 would choose the ID column. The denominator penalizes fragmentation, allowing Gain Ratio to select substantively predictive features.

Trade-offs

Pros:
- Effectively penalizes attributes with many values, mitigating overfitting from high-cardinality categorical features.
- Retains information-theoretic interpretability.
- The "average-IG threshold" heuristic avoids over-penalization of attributes that genuinely benefit from many values.

Cons:

- Can *over*-penalize: an attribute that is genuinely informative but has moderate cardinality may be skipped.
- The ratio is undefined when SplitInfo $= 0$ (all samples have the same attribute value); handled by fallback.
- More complex implementation than raw information gain.
- Not a standard option in many modern libraries (scikit-learn does not offer gain ratio natively).

24.1.4 Misclassification Error

Misclassification Error

The misclassification error (classification error) at node t is:

$$\text{Error}(t) = 1 - \max_{c \in \{1,\ldots,C\}} p(c|t)$$

This is the proportion of samples that would be misclassified if all samples at node t were predicted as the majority class. Range: $[0,\ 1 - \frac{1}{C}]$.

Trade-offs

Pros: Intuitive, directly measures classification accuracy. Fastest evaluation among all splitting criteria (only needs arg max).
Cons: Not sufficiently sensitive to split quality. Consider a two-class node where both children have identical majority class at 51%–49%. Misclassification error would score this split identically to a node where children are 99%–1%, because the majority class is the same. Gini and entropy both differentiate these. As a result, error-based splits often fail to find structure that Gini or entropy would detect, leading to worse splits and larger, more complex trees.

Why Not Use Misclassification Error for Splitting?

Although error is the natural loss for classification trees, it is a piecewise-linear, non-strictly-concave function of $p(c|t)$. Strict concavity (exhibited by Gini and entropy) is essential for a splitting criterion to prefer "purer" child nodes and penalize mixed ones. Error changes only when the majority class changes, making it insensitive to distributional refinements. It is, however, the standard pruning criterion (CART cost-complexity pruning uses misclassification rate).

24.1.5 Twoing Rule

Twoing Rule — CART

The Twoing criterion is designed for binary splits in multi-class settings where a single split can separate one superclass from the remainder. For a binary split s of node t into t_L, t_R, the twoing value is:

$$\Phi(s,t) = \frac{n_L\, n_R}{n^2} \Big[\sum_{c=1}^{C} \big| \, p(c|t_L) - p(c|t_R) \, \big| \Big]^2$$

The factor $n_L n_R/n^2$ encourages balanced splits. The inner term is the total variation distance between the class distributions in the two child nodes. Twoing searches for splits that separate classes into two groups as distinctly as possible.

Applications

Recommended by Breiman et al. (1984) when the number of classes C is large (e.g., $C > 5$). Twoing tends to produce more interpretable trees in multi-class problems because it groups classes into superclasses, yielding splits that have a clear "two-group" interpretation. Gini and entropy, by contrast, may split based on one class at a time, leading to many splits before all classes are isolated. Accessible in scikit-learn CART via custom extensions but not natively supported.

Twoing vs. Gini in Multi-Class Settings

For binary classification, twoing and Gini are virtually equivalent. For $C \gg 2$, Gini splits "one class vs. the rest" repeatedly, while twoing attempts to partition all classes into two balanced groups in one split. This is more efficient when the class structure is hierarchical or when balanced trees are desired.

24.1.6 Chi-Square Splitting

Chi-Square — CHAID

For a candidate split of a categorical predictor with r levels against a target with c classes, the chi-square statistic is:

$$\chi^2 = \sum_{i=1}^{r} \sum_{j=1}^{c} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

where O_{ij} is the observed count of class j in predictor level i, and $E_{ij} = (\text{row}_i \text{ total} \times \text{col}_j \text{ total})/n$ is the expected count under independence. The predictor with the highest χ^2 statistic (smallest p-value) is selected for splitting. Splitting stops when no predictor achieves a statistically significant association (Bonferroni-corrected p-value below a threshold α).

Applications

CHAID (Chi-squared Automatic Interaction Detection) is predominantly used in market research and survey analysis where most features are categorical. Commercial implementations exist in SPSS and SAS. Distinctive feature: CHAID can produce **multi-way splits** by automatically merging non-significant categories of a predictor, unlike CART (binary only) or C4.5 (one branch per category).

CHAID Merge-and-Split Process

For each predictor, CHAID:

1. Cross-tabulates predictor categories against the target class.
2. Iteratively merges the pair of categories with the least significant chi-square difference, stopping when all remaining categories differ significantly.
3. Computes the adjusted p-value for the merged predictor (Bonferroni correction for

the number of merges tested).
4. Splits on the predictor with the smallest adjusted p-value.

This built-in category merging makes CHAID uniquely suited for categorical data with many levels, avoiding the fragmentation of multi-way splits that plagues ID3/C4.5.

24.1.7 Hellinger Distance

Hellinger Distance Splitting

Let $p_L(c) = p(c|t_L)$ and $p_R(c) = p(c|t_R)$ be the class distributions in the left and right child nodes. The Hellinger distance between them is:

$$d_H(p_L, p_R) = \sqrt{2 \sum_{c=1}^{C} \left(\sqrt{p_L(c)} - \sqrt{p_R(c)} \right)^2}$$

Hellinger distance is a symmetric f-divergence measuring similarity between two probability distributions. For splitting, the split that maximizes $d_H(p_L, p_R)$ is selected.

Applications

Class-imbalanced datasets. Hellinger distance is insensitive to the class prior distribution — it measures the *distance between child distributions*, not weighted impurity reduction. This makes it robust to skewed class ratios where Gini and entropy can be swamped by the majority class. Also used in "Hellinger Distance Decision Trees" (HDDT), which are competitive with sampling/reweighting methods for imbalanced learning.

Trade-offs

Pros:

- **Class-imbalance robust**: no dependence on prior class probabilities; split quality depends only on distributional divergence between children.
- Symmetric and metric: d_H satisfies the triangle inequality, giving a geometric interpretation to splits.
- Bounded in $[0, 2]$, with 2 achieved when supports are disjoint.

Cons:

- Computes a square root per evaluation — modest overhead.
- Square-root transformation compresses differences at very small probabilities; may be less discriminative for tiny classes.
- Less widely adopted than Gini/entropy; limited library support.
- Does not account for child node sizes (all split positions equidistant from root are equally weighted).

24.2 Regression Splitting Criteria

24.2.1 Variance / MSE Reduction

MSE Reduction — CART Regression

For a regression node t with response values $\{y_i\}_{i \in t}$, let $\bar{y}_t = \frac{1}{|t|} \sum_{i \in t} y_i$ and define the variance (MSE) as:

$$\text{Var}_t = \frac{1}{|t|} \sum_{i \in t} (y_i - \bar{y}_t)^2$$

For a binary split into t_L, t_R, the split quality is measured by MSE reduction (variance reduction):

$$\Delta\text{Var} = \text{Var}_t - \frac{n_L}{n} \text{Var}_{t_L} - \frac{n_R}{n} \text{Var}_{t_R}$$

Equivalently, maximize:

$$\text{Var}(\bar{y}) - \frac{n_L \text{Var}_{t_L} + n_R \text{Var}_{t_R}}{n}$$

The split that maximizes ΔVar (i.e., minimizes weighted child variance) is selected. This is the standard criterion for CART regression trees.

Applications

The default (and only) splitting criterion in scikit-learn's `DecisionTreeRegressor`. Also the base split evaluation in Random Forest regressors and gradient-boosted trees (before regularization modifications). Variance reduction naturally follows from the principle of partitioning the input space to minimize within-leaf prediction error.

Connection to Least-Squares Estimation

MSE reduction is equivalent to choosing the split that, when followed by leaf-wise constant prediction (the leaf mean), minimizes the sum of squared residuals. Under a homoscedastic Gaussian noise assumption, this is the maximum-likelihood split criterion. The leaf prediction $\bar{y}_t$ is the MLE for a constant model.

24.2.2 MAE / LAD Reduction

MAE Reduction — Robust Regression Tree

Using mean absolute error (or least absolute deviation) as the impurity measure:

$$\text{MAE}_t = \frac{1}{|t|} \sum_{i \in t} |y_i - \tilde{y}_t|$$

where $\tilde{y}_t$ is the leaf **median** (the optimal constant predictor under L1 loss). The split minimizes:

$$\Delta\text{MAE} = \frac{n_L}{n} \text{MAE}_{t_L} + \frac{n_R}{n} \text{MAE}_{t_R}$$

MAE-based splitting produces trees that are robust to outliers because the median is less influenced by extreme values than the mean.

Applications

Robust regression and outlier-heavy datasets. When the target variable has heavy-tailed noise or contains gross measurement errors, MAE splitting outperforms MSE. Implemented in scikit-learn via `criterion='absolute_error'` (squared_error is default). Also useful for financial time series with fat-tailed returns and sensor data with occasional spikes.

Trade-offs

Pros:

- Robust to outliers: a single extreme value does not dramatically shift the split point or child impurity.
- Matches L1 loss objectives common in robust statistics.
- Produces median predictions at leaves, which are more interpretable for skewed distributions.

Cons:

- Slower than MSE: computing the median for every candidate split requires sorting (or order-statistic trees), adding $O(\log n)$ factor.
- Not differentiable at zero residual (subgradient issues, though irrelevant for tree splitting).
- Less statistically efficient when errors are truly Gaussian (median is 25% less efficient than mean asymptotically).

24.2.3 Poisson Splitting

Poisson Log-Likelihood Splitting

For count-valued targets $y_i \in \{0, 1, 2, \dots\}$, the prediction at a leaf is a Poisson rate $\hat{\lambda}_t = \frac{1}{|t|} \sum_{i \in t} y_i$. The split that maximizes the Poisson log-likelihood improvement is:

$$\Delta \ell_{\text{Poisson}} = \sum_{i \in t_L} (y_i \log \hat{\lambda}_L - \hat{\lambda}_L) + \sum_{i \in t_R} (y_i \log \hat{\lambda}_R - \hat{\lambda}_R) - \sum_{i \in t} (y_i \log \hat{\lambda}_t - \hat{\lambda}_t)$$

This treats the response as Poisson-distributed with constant rate within each leaf, appropriate for counting processes (events per time, rare occurrences).

Applications

Count data and rate estimation. XGBoost and LightGBM support Poisson regression objectives (`objective='count:poisson'` in XGBoost). Use cases: insurance claim frequency, website visit counts, RNA-seq gene expression, rare event counts in epidemiology. The tree structure captures nonlinear exposure-response relationships multiplicatively.

Poisson vs. Tweedie in Gradient Boosting

For count data with excess zeros (zero-inflated counts), the Tweedie distribution (index $1 < p < 2$) is often preferred. It generalizes Poisson ($p \to 1$) and compound Poisson-Gamma. XGBoost supports `objective='reg:tweedie'` for this purpose.

24.2.4 Friedman's MSE

Friedman's MSE — Gradient Boosting

Used in scikit-learn's `GradientBoostingRegressor` and histogram-based boosters, Friedman's MSE incorporates the improvement score h_i for each sample (the negative gradient, or pseudo-residual):

$$\text{FMSE}(t) = \frac{1}{|t|} \sum_{i \in t} (y_i - \bar{y}_t)^2$$

When splitting, the criterion maximizes the improvement:

$$\frac{n_L n_R}{n^2} \frac{\left(\frac{1}{n_L} \sum_{i \in t_L} h_i - \frac{1}{n_R} \sum_{i \in t_R} h_i\right)^2}{\frac{1}{n} \sum_{i \in t} (h_i - \bar{h})^2}$$

where h_i are the current-stage pseudo-residuals. This formulation weighs both the magnitude and significance of the child-node gradient difference.

Applications

The default criterion in scikit-learn's `HistGradientBoostingRegressor` and used internally in the Friedman (2001) gradient boosting algorithm. It optimizes the split for the *next boosting stage*, not for the raw target values — i.e., each tree is built to correct the residuals of the current ensemble.

24.3 Pruning Criteria

24.3.1 Cost-Complexity Pruning

Cost-Complexity Pruning — CART

Define the cost-complexity of a tree T with leaf set T_{leaf}:

$$R_\alpha(T) = R(T) + \alpha \, |T_{\text{leaf}}|$$

where $R(T)$ is the misclassification rate (or MSE for regression) summed over leaves, $|T_{\text{leaf}}|$ is the number of terminal nodes, and $\alpha \geq 0$ is the complexity parameter. For each α, we find the subtree $T(\alpha) \subseteq T_{\max}$ minimizing R_α.
The **weakest-link cutting** rule: for each internal node t, compute

$$g(t) = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

where $R(T_t)$ is the cost of the subtree rooted at t and $|T_t|$ is its leaf count. Prune the node with the smallest $g(t)$ — it contributes the least error reduction per leaf. Increase α to $g(t)$ and repeat. This produces a nested sequence of pruned subtrees, from which the cross-validation (or independent validation) optimum is selected.

Applications

The standard pruning method for CART. Implemented in scikit-learn as `DTC.cost_complexity_pruning_path(X, y)` and the `ccp_alpha` parameter in

`DecisionTreeClassifier/Regressor`. Produces a sequence of subtrees indexed by α; the optimal α is chosen via cross-validation using `ccp_alpha` grid search.

Interpretation of α

Each additional leaf "costs" α units of error. A leaf is worth keeping only if it reduces error by more than α. As $\alpha = 0$, we retain the full tree; as $\alpha \to \infty$, we retain only the root. The cost-complexity path provides the full regularization trajectory.

Trade-offs

Pros:
- Guarantees a sequence of nested optimal subtrees (no ad-hoc pruning decisions).
- Natural regularization path: one parameter α controls the bias-variance trade-off.
- Computationally efficient: weakest-link pruning takes $O(|T|)$ after the full tree is built.

Cons:
- Requires a held-out set or cross-validation to select α optimally, adding computational cost.
- The nestedness property means all subtrees share the same splits; a better tree may exist outside the cost-complexity path.
- The misclassification error used in $R(T)$ is insensitive as a *splitting* criterion, but works well as a pruning metric.

24.3.2 Reduced Error Pruning

Reduced Error Pruning

A post-pruning approach that uses a validation set separate from training:
1. Build the full (unpruned) tree on training data.
2. For each non-leaf node (bottom-up or top-down), tentatively replace the subtree rooted at that node with a leaf. The leaf prediction is the majority class (classification) or mean (regression) of the *training* samples reaching that node.
3. Keep the replacement if validation error does not increase.
4. Repeat until no further pruning improves validation performance.

Reduced error pruning is simple, direct, and uses validation data rather than statistical heuristics.

Applications

Common in academic implementations and early decision tree toolkits. Practically, reduced error pruning reduces tree size by 50–80% while maintaining or improving validation accuracy. Works best on moderate-sized datasets where a clean validation split is affordable.

Trade-offs

Pros: Simple, directly optimizes validation accuracy. No hyperparameter beyond the validation split ratio. Produces compact trees with good generalization.
Cons: Requires holding out validation data, reducing training samples. Bottom-up pruning can be $O(|T|^2)$ in worst case. Greedy decision: pruning decisions are final once made,

which can lead to suboptimal pruned trees.

24.3.3 Minimum Description Length Pruning

MDL Pruning

MDL formulates tree pruning as a coding problem. The total description length of data $\mathcal{D}$ given tree T is:

$$L(\mathcal{D}, T) = \underbrace{L(T)}_{\text{model cost}} + \underbrace{L(\mathcal{D} \mid T)}_{\text{data cost}}$$

- **Model cost** $L(T)$: bits to encode the tree structure (split features, thresholds, leaf values). A larger tree requires more bits.
- **Data cost** $L(\mathcal{D} \mid T)$: bits to encode prediction errors (classification: log-loss; regression: Gaussian log-likelihood residuum).

The tree that minimizes $L(\mathcal{D}, T)$ is selected. MDL naturally implements Occam's razor: a split is kept only if the reduction in data cost exceeds the cost of encoding the split itself.

Applications

MDL pruning is theoretically elegant and parameter-free (no α or validation set needed). Used less in practice than cost-complexity pruning due to implementation complexity. Useful when data are scarce and a principled, data-driven complexity penalty is required.

24.3.4 Pessimistic Pruning

Pessimistic Pruning — C4.5

C4.5 prunes using a "pessimistic" estimate of the error rate, derived from a binomial confidence bound:

$$e_{\text{pess}}(t) = e(t) + \frac{z_\gamma}{\sqrt{n(t)}}$$

where $e(t)$ is the training error rate at node t, $n(t)$ is the number of samples at t, and z_γ is the z-score for confidence level γ (C4.5 default: $\gamma = 0.25$, one-sided $z_{0.25} \approx 0.69$). For a subtree rooted at t with leaves $\{t_1, \ldots, t_k\}$:

$$e_{\text{pess}}(\text{subtree}) = \sum_{i=1}^{k} \frac{n(t_i)}{n(t)}\, e_{\text{pess}}(t_i)$$

Prune the subtree into a leaf if:

$$e_{\text{pess}}(t) \leq e_{\text{pess}}(\text{subtree})$$

The correction term penalizes nodes with few samples, where the error estimate has high variance.

Applications

The pruning method used in C4.5 (Quinlan, 1993). Does not require a separate validation set — all pruning decisions are based on training data with statistical correction. This is especially valuable when data are limited and cannot be split into train/validation.

Trade-offs

Pros: No validation set needed — uses all data for training. Statistically grounded via binomial confidence bounds. Automatically penalizes low-support nodes.
Cons: The confidence parameter z_γ effectively becomes a hyperparameter. The binomial approximation assumes independence, which does not strictly hold within a tree. Can be overly pessimistic (prunes too aggressively) when the default $z_{0.25}$ is used on small datasets.

24.4 Ensemble-Specific Split Finding

24.4.1 XGBoost Gain

XGBoost Regularized Gain

XGBoost formulates tree learning as regularized additive training. Let $G_L = \sum_{i \in I_L} g_i$ and $H_L = \sum_{i \in I_L} h_i$ be the sum of first-order gradients (pseudo-residuals) and second-order gradients (Hessians) for samples in the left child, and similarly G_R, H_R for the right child. The gain of a split is:

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}\right] - \gamma$$

where:

- λ is the L2 regularization coefficient on leaf weights (default $\lambda = 1$).
- γ is the minimum loss reduction required to make a split (default $\gamma = 0$; larger values penalize complexity, analogous to cost-complexity α in CART).

A split is accepted only if Gain > 0. The split yielding the maximum gain is selected.

Derivation Sketch

The objective at a leaf with weight w is:

$$\text{Obj}(w) = \Big(\sum g_i\Big) w + \frac{1}{2}\Big(\sum h_i + \lambda\Big) w^2$$

Differentiating and setting to zero gives the optimal leaf weight: $w^* = -\frac{G}{H+\lambda}$, with objective value $-\frac{1}{2}\frac{G^2}{H+\lambda}$. The Gain formula is the reduction in objective when splitting one leaf into two, minus the penalty γ for adding a split.

Applications

The core of XGBoost's tree learner. Supports both exact greedy search and approximate histogram-based search. The λ and γ parameters directly implement shrinkage and pruning within the splitting criterion, making XGBoost less prone to overfitting than unregularized gradient boosting even with deep trees.

Gradient vs. Hessian Terms

- **Squared error:** $g_i = \hat{y}_i - y_i$ (residual), $h_i = 1$ (constant Hessian). Gain reduces to a weighted variance-reduction-like formula.
- **Logistic loss:** $g_i = \hat{p}_i - y_i$ (probability residual), $h_i = \hat{p}_i(1 - \hat{p}_i)$ (variance).

- **General:** Any twice-differentiable convex loss yields valid g_i, h_i, making XGBoost a universal gradient booster.

24.4.2 LightGBM Leaf-Wise Growth

LightGBM Leaf-Wise Split Strategy

Traditional gradient boosting uses **level-wise** (depth-wise) tree growth: at each depth, split all leaves. LightGBM uses **leaf-wise** (best-first) growth:
Given a current tree with leaf set $\mathcal{L}$, compute the gain $G(\ell)$ for splitting each leaf ℓ. Split the leaf with $\max_{\ell \in \mathcal{L}} G(\ell)$. Repeat until reaching the maximum number of leaves or no split yields positive gain.
This produces asymmetric, deeper trees for the same leaf budget.

Applications

LightGBM's default growth strategy. Achieves lower training loss and faster convergence than level-wise growth, especially when the data has regions of varying difficulty (some branches need more splits than others). Parameter `num_leaves` controls complexity instead of max depth.

Trade-offs

Pros:
- Higher accuracy than level-wise for the same number of leaves, because splits are allocated where they help most.
- Faster convergence in boosting iterations.
- Naturally handles heterogeneous data (some regions more complex).

Cons:
- Can produce very deep, narrow trees that **overfit** small datasets if `num_leaves` is set too high.
- Not cache-friendly: leaf-wise trees are less balanced, causing irregular memory access patterns.
- Control via `max_depth` is still recommended alongside `num_leaves` to prevent pathological depth.

24.4.3 Histogram-Based Split Finding

Histogram-Based Split Search

For a continuous feature x_j, discretize its values into k bins (typically $k = 256$) with bin edges $\{b_0, b_1, \ldots, b_k\}$. The split finding procedure:
1. Construct a histogram of gradients G (and Hessians H) for each bin: accumulate gradient statistics of samples falling into each bin.
2. Scan the $k-1$ candidate split points (bin edges) and compute the gain at each using cumulative sums.
3. Select the bin edge with maximum gain.

Complexity: $O(k \times n)$ to build histograms per feature, vs. $O(n \log n)$ for exact greedy search. With $k \ll n$, histogram search is substantially faster.

Applications

Used in LightGBM (GOSS + EFB for further acceleration), XGBoost (`tree_method='hist'`), and scikit-learn's `HistGradientBoostingRegressor/Classifier`. Enables boosting on datasets with millions of samples and hundreds of features.

Exact vs. Approximate vs. Histogram Search

- **Exact greedy** (scikit-learn DecisionTree): sorts each feature at every node, evaluates every between-sample gap. $O(n \log n)$ per feature. Gold standard but slow.
- **Approximate** (XGBoost approx): proposes candidate split points via quantile sketching, then evaluates only those.
- **Histogram** (LightGBM, XGBoost hist): discretizes once per feature or per node, $O(k \times n)$ bin construction. The primary speed-memory trade-off knob is k.

24.5 Oblique / Multivariate Splits

24.5.1 Axis-Parallel vs. Oblique Splits

Split Types

- **Axis-parallel (univariate) split:** $x_j \leq \theta$, with $j \in \{1, \ldots, d\}$ and threshold $\theta \in \mathbb{R}$. The decision boundary is an axis-aligned hyperplane. All standard tree algorithms (CART, C4.5, ID3) use axis-parallel splits.
- **Oblique (multivariate) split:** $\mathbf{w}^\top \mathbf{x} \leq \theta$, where $\mathbf{w} \in \mathbb{R}^d$ is a weight vector (often normalized: $\|\mathbf{w}\| = 1$). The decision boundary is a hyperplane at an arbitrary orientation. Capable of capturing linear correlations between features.

Axis-parallel splits approximate diagonal boundaries with a staircase of many splits, while oblique splits can represent them directly with a single hyperplane.

Expressiveness Comparison

A binary oblique tree with k leaves can represent any partitioning of $\mathbb{R}^d$ by k convex polytopes where each polytope is the intersection of at most $\log_2 k$ halfspaces. An axis-parallel tree requires up to $O(d \times k)$ splits to approximate the same boundary when features are correlated.

24.5.2 OC1 and Householder CART

OC1 — Oblique Classifier 1

OC1 (Murthy et al., 1994) searches for linear splits of the form $\mathbf{w}^\top \mathbf{x} \leq \theta$ using two mechanisms:

1. **Deterministic hill-climbing:** Perturb each coefficient w_j by $\pm\epsilon$; if impurity decreases, accept the move. Repeat until no improvement.
2. **Randomized restart:** When hill-climbing stalls, perturb $\mathbf{w}$ by a random linear combination of the current best $\mathbf{w}$ and a randomly chosen axis-parallel direction, then restart climbing.

OC1 finds local optima for oblique splits, trading optimality for speed.

Householder CART

An alternative approach uses Householder reflections to search oblique splits:

1. At each node, construct the within-class and between-class scatter matrices.
2. Compute the linear discriminant direction (Fisher's LDA) as the first eigenvector of $\mathbf{S}_W^{-1}\mathbf{S}_B$.
3. Project data onto this direction and split with a 1D threshold.
4. Apply a Householder transformation to rotate the coordinate system so that subsequent splits remain axis-parallel in the new coordinate system.

This hybrid approach finds discriminative oblique directions while maintaining computational tractability.

Trade-offs: Oblique vs. Axis-Parallel

Pros (Oblique):

- More expressive decision boundaries — can capture linear feature interactions directly without staircase approximation.
- Shallower trees for the same accuracy (fewer splits needed).
- Naturally handles correlated features without rotation.

Cons (Oblique):

- **Optimization is harder:** finding the optimal $\mathbf{w}$ is non-convex and computationally expensive compared to scanning axis-aligned thresholds.
- Loss of interpretability: a weighted sum of features is harder to explain than "age $\leq$ 40".
- Fewer splits available — the search space is continuous d-dimensional vs. discrete 1D threshold scan.
- Limited library support (no scikit-learn native implementation).

24.6 Multi-Way vs. Binary Splits

Split Arity

- **Binary splits** (CART): At each internal node, data is split into exactly two child nodes using one feature and one threshold, $x_j \leq \theta$. All internal nodes have out-degree exactly 2.
- **Multi-way splits** (C4.5, ID3, CHAID): A single internal node can have more than two children. For categorical features, each category value gets its own branch. For continuous features, discretization into several intervals yields multi-way splits.
- **CHAID merging:** Categories are dynamically grouped into statistically homogeneous groups; the number of branches is determined by the merging process, not fixed a priori.

Binary splits are the default in modern implementations due to algorithmic simplicity and compatibility with ensemble methods.

Trade-offs: Binary vs. Multi-Way

Binary Split Pros:

- Simpler split search: find one optimal threshold per feature.
- Compatible with all ensemble splitting formulas (Gain, Friedman's MSE are binary by construction).

- No fragmentation: repeated binary splits on the same categorical feature can group values progressively without explosion of branches.
- Categorical features with high cardinality do not cause runtime blowup (CART uses binary splits even for categoricals).

Multi-Way Split Pros:

- More efficient for nominal categorical features: one split can separate all categories simultaneously.
- Shallower trees: multi-way splits reduce depth, which can improve interpretability.
- CHAID's merge-and-split is principled for survey-style categorical data.

Multi-Way Cons:

- Fragmentation: high-cardinality categoricals produce many small child nodes, rapidly depleting data and causing overfitting.
- Unbalanced splits make tree structures irregular, complicating balanced parallel implementations (XGBoost, LightGBM use binary only).
- Information gain bias toward many-valued attributes is exacerbated (Gain Ratio partially mitigates).

24.7 Summary Comparison of Splitting Criteria

Practical Guidelines for Choosing a Splitting Criterion

1. **Default:** Gini or entropy are effectively interchangeable for most classification tasks. Gini is slightly faster.
2. **High-cardinality categorical features:** Use Gain Ratio (C4.5) or ensure binary splits with categorical encoding.
3. **Multi-class with many classes ($C > 5$):** Twoing rule produces more interpretable splits and balanced trees.
4. **Class imbalance:** Hellinger distance or class-weighted Gini/entropy. Avoid unweighted information gain.
5. **Count data:** Poisson objective in XGBoost/LightGBM. For excess zeros, use Tweedie ($p \in (1, 2)$).
6. **Outliers in regression:** MAE reduction with median leaf prediction provides robustness without data cleaning.
7. **Large-scale boosting:** Histogram-based gain (LightGBM, XGBoost hist mode) for datasets with millions of samples. Set `max_bin` = 255–511.
8. **Pruning:** Cost-complexity pruning (CART) with cross-validated α is the most robust general-purpose approach. Use MDL when data are too scarce for a validation split.
9. **Oblique splits:** Consider only when axis-parallel splits produce excessively large trees and interpretability is not critical. For deep learning pipelines, neural networks are usually more effective at capturing feature interactions.

Table 24.1: Comparison of Decision Tree Splitting Criteria

Criterion	**Task**	**Algorithm**	**Split Arity**	**Key Characteristic**
Gini Impurity	Clf.	CART	Binary	Fast (no logs); isolates majority class
Entropy / IG	Clf.	ID3, C4.5	Multi-way	Information-theoretic; biased to many-valued attributes
Gain Ratio	Clf.	C4.5	Multi-way	Penalizes fragmentation; avoids IG bias
Miscl. Error	Clf.	—	Any	Insensitive; mainly used for *pruning*, not splitting
Twoing Rule	Clf.	CART	Binary	Groups classes into superclasses; good for $C \gg 2$
Chi-Square	Clf.	CHAID	Multi-way	Statistical test; merges non-significant categories
Hellinger Dist.	Clf.	HDDT	Binary	Class-imbalance robust; no dependence on class prior
MSE Reduction	Reg.	CART	Binary	Minimizes within-leaf variance; MLE under Gaussian noise
MAE Reduction	Reg.	Robust Trees	Binary	Robust to outliers; uses leaf median
Poisson LL	Reg.	XGBoost	Binary	Count data; multiplicative rate modeling
Friedman's MSE	Reg.	GBDT	Binary	Optimizes next boosting stage using pseudo-residuals
XGBoost Gain	Both	XGBoost	Binary	Regularized with λ, γ; uses gradient + Hessian
LightGBM Leaf	Both	LightGBM	Binary	Leaf-wise (best-first) growth; higher accuracy, risk of overfit
Histogram Gain	Both	LightGBM	Binary	Discretize to k bins; $O(k \times n)$, scalable
Oblique	Both	OC1, HCART	Binary	$\mathbf{w}^\top \mathbf{x} \leq \theta$; expressive but optimization-heavy

Chapter 25

Bayesian Methods

Bayesian methods provide a principled framework for reasoning under uncertainty by treating model parameters as random variables with prior distributions, and updating beliefs in light of observed data via Bayes' theorem. This chapter covers core Bayesian inference, approximate methods (variational inference, EM), and their modern instantiations in neural networks, Gaussian processes, and optimization.

25.1 Core Bayesian Formulas

25.1.1 Bayes' Theorem

Bayes' Theorem

Given observed data $\mathcal{D}$ and model parameters θ, the posterior distribution over parameters is:

$$p(\theta \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \theta)\, p(\theta)}{p(\mathcal{D})}$$

where $p(\mathcal{D} \mid \theta)$ is the likelihood, $p(\theta)$ is the prior, and $p(\mathcal{D}) = \int p(\mathcal{D} \mid \theta) p(\theta)\, d\theta$ is the marginal likelihood (evidence). The posterior expresses updated uncertainty about θ after observing $\mathcal{D}$.

Applications

Fundamental formula for all Bayesian inference. Used when prior knowledge about parameters is available and uncertainty quantification matters: parameter estimation in small-data regimes, hierarchical modeling, and probabilistic programming (Pyro, Stan, PyMC).

Trade-offs

Pros:

- Uncertainty quantification: full posterior, not just point estimate.
- Natural incorporation of prior knowledge.
- Principled model comparison via marginal likelihood.
- Avoids overfitting through automatic Occam's razor: the marginal likelihood trades off fit and complexity.

Cons:

- Computing $p(\mathcal{D})$ is intractable for most realistic models.
- Posterior inference requires approximation (MCMC, VI).
- Prior specification can be subjective and influential.
- Computationally demanding compared to point estimation.

25.1.2 MAP Estimation

Maximum a Posteriori (MAP)

The MAP estimate maximizes the posterior density:

$$\theta_{\text{MAP}} = \arg\max_{\theta} p(\theta \mid \mathcal{D}) = \arg\max_{\theta} \big[\log p(\mathcal{D} \mid \theta) + \log p(\theta)\big]$$

The marginal likelihood $p(\mathcal{D})$ is a constant w.r.t. θ and drops out. MAP is a point estimate that can be obtained by standard optimization with a regularizer derived from the log-prior.

Applications

When a point estimate suffices but prior information should be incorporated. L2 regularization $\leftrightarrow$ Gaussian prior; L1 regularization $\leftrightarrow$ Laplace prior. Used as initialization for Laplace approximation and as teacher for knowledge distillation.

Trade-offs

Pros: Simple to compute (optimization, not integration). Incorporates prior beliefs. Degrades gracefully to MLE as prior becomes uninformative. **Cons:** Single point, no uncertainty. Not invariant to reparameterization. Can select a narrow mode when the posterior is broad or multi-modal.

25.1.3 Marginal Likelihood and Bayes Factors

Marginal Likelihood and Bayes Factor

The marginal likelihood (model evidence) is:

$$p(\mathcal{D}) = \int p(\mathcal{D} \mid \theta)\, p(\theta)\, d\theta$$

The Bayes factor comparing two models M_1, M_2 is the ratio of their marginal likelihoods:

$$B_{12} = \frac{p(\mathcal{D} \mid M_1)}{p(\mathcal{D} \mid M_2)}$$

$B_{12} > 1$ favors M_1; Jeffreys' scale interprets magnitude: 1–3 anecdotal, 3–10 substantial, 10–100 strong, > 100 decisive.

Applications

Model selection and comparison without held-out validation sets. Bayesian hypothesis testing replacing p-values. Hyperparameter optimization for Gaussian processes (maximizing log marginal likelihood yields optimal kernel parameters).

Trade-offs

Pros: Automatically penalizes model complexity. No need for cross-validation split. Theoretically grounded decision rule. **Cons:** Marginal likelihood is highly sensitive to prior specification. Intractable integral for complex models. Lindley's paradox: with large

data, a simpler model is always preferred regardless of effect size.

25.1.4 Posterior Predictive Distribution

Posterior Predictive

Given the posterior, the predictive distribution for a new observation $\tilde{x}$ is:

$$p(\tilde{x} \mid \mathcal{D}) = \int p(\tilde{x} \mid \theta)\, p(\theta \mid \mathcal{D})\, d\theta$$

This averages the likelihood of $\tilde{x}$ over the entire posterior, naturally accounting for parameter uncertainty. For classification, $p(\tilde{y} = c \mid \mathcal{D}) = \int p(\tilde{y} = c \mid \theta)\, p(\theta \mid \mathcal{D})\, d\theta$.

Applications

Out-of-sample prediction with uncertainty. Anomaly detection (low posterior predictive density $\rightarrow$ outlier). Model checking via posterior predictive p-values. Bayesian decision theory: choose action maximizing expected utility under the posterior predictive.

Connection to Bias-Variance Decomposition

The posterior predictive variance decomposes as:

$$\mathrm{Var}[\tilde{x} \mid \mathcal{D}] = \underbrace{\mathbb{E}_{\theta \mid \mathcal{D}}[\mathrm{Var}[\tilde{x} \mid \theta]]}_{\text{Aleatoric}} + \underbrace{\mathrm{Var}_{\theta \mid \mathcal{D}}[\mathbb{E}[\tilde{x} \mid \theta]]}_{\text{Epistemic}}$$

See Chapter 28 for a detailed treatment.

25.1.5 Conjugate Priors

Conjugate Priors

A prior $p(\theta)$ is conjugate to likelihood $p(\mathcal{D} \mid \theta)$ if the posterior $p(\theta \mid \mathcal{D})$ belongs to the same parametric family as the prior. Key pairs:

$$\begin{aligned}
\mathrm{Beta}(\alpha, \beta) &\leftrightarrow \mathrm{Binomial}(n, \theta) \\
\mathrm{Dirichlet}(\boldsymbol{\alpha}) &\leftrightarrow \mathrm{Categorical/Multinomial} \\
\mathcal{N}(\mu_0, \sigma_0^2) &\leftrightarrow \mathcal{N}(\theta, \sigma^2) \ \ (\sigma^2 \text{ known}) \\
\mathrm{Gamma}(\alpha, \beta) &\leftrightarrow \mathrm{Poisson}(\lambda) \\
\text{Normal-Inverse-Gamma} &\leftrightarrow \mathcal{N}(\mu, \sigma^2) \ \ (\text{both unknown})
\end{aligned}$$

The posterior hyperparameters are simple updates of the prior hyperparameters using sufficient statistics of the data, yielding closed-form inference.

Applications

Analytically tractable Bayesian inference in exponential family models. Beta-Binomial for A/B testing, click-through rates. Dirichlet-Multinomial for topic models (LDA). Gaussian-Gaussian for Kalman filtering. NIG for Bayesian linear regression.

Trade-offs

Pros: Exact, fast posterior updates (no sampling or optimization). Interpretable hyperparameter updates. **Cons:** Restricted to exponential family models. The conjugate prior may not encode the analyst's true prior beliefs. Does not easily extend to deep or hierarchical models.

25.2 Variational Inference

Variational inference (VI) turns posterior inference into optimization by approximating the true posterior $p(\theta \mid \mathcal{D})$ with a simpler variational distribution $q_\phi(\theta)$ from a tractable family.

25.2.1 Evidence Lower Bound (ELBO)

ELBO

The log marginal likelihood decomposes as:

$$\log p(\mathcal{D}) = \underbrace{\mathbb{E}_{q_\phi(\theta)}[\log p(\mathcal{D} \mid \theta)] - D_{\mathrm{KL}}(q_\phi(\theta) \,\|\, p(\theta))}_{\mathcal{L}(\phi)} + D_{\mathrm{KL}}(q_\phi(\theta) \,\|\, p(\theta \mid \mathcal{D}))$$

Maximizing the ELBO $\mathcal{L}(\phi)$ simultaneously maximizes the expected log-likelihood (data fit) while minimizing KL divergence from the variational distribution to the prior (regularization). Equivalently, it minimizes $\mathrm{KL}(q_\phi \,\|\, p_{\text{posterior}})$.

Applications

Posterior approximation for any model with differentiable likelihood and prior. Scales to large datasets via stochastic VI (subsample data). Foundation of variational autoencoders (VAEs), Bayesian neural networks (Bayes by Backprop), and scalable Gaussian processes.

Trade-offs

Pros:

- Optimization rather than sampling: faster, scales to large data.
- Gradient-based: amenable to automatic differentiation and GPU.
- Deterministic convergence diagnostic (ELBO monotonic increase).
- Trades asymptotic exactness for computational speed.

Cons:

- The variational family may not contain the true posterior (approximation gap).
- Tends to underestimate posterior variance (mode-seeking behavior of reverse KL).
- ELBO optimization can get stuck in poor local optima.
- Model-specific derivations required for non-black-box approaches.

25.2.2 Mean-Field VI and CAVI

Mean-Field Factorization

The variational distribution factorizes across parameter groups:

$$q(\theta) = \prod_{i=1}^{m} q_i(\theta_i)$$

No parametric form is assumed for each q_i. The Coordinate Ascent Variational Inference (CAVI) algorithm iteratively updates each factor:

$$\log q_j^*(\theta_j) = \mathbb{E}_{i \neq j}[\log p(\theta, \mathcal{D})] + \text{const}$$

where the expectation is taken over all other factors $q_i(\theta_i)$ for $i \neq j$. This is a fixed-point iteration that converges to a local optimum of the ELBO.

Applications

Classic VI for conjugate exponential family models (LDA, Bayesian mixture models, hidden Markov models). When the complete conditionals $p(\theta_j \mid \theta_{-j}, \mathcal{D})$ belong to known exponential families, CAVI updates have closed form.

Trade-offs

Pros: Non-parametric within each factor; closed-form updates for conjugate models. **Cons:** Factorization assumes posterior independence, underestimating correlations. CAVI requires deriving model-specific expectations. Does not scale to large data without stochastic extensions.

25.2.3 Black-Box, Stochastic, and Reparameterization VI

Black-Box VI (BBVI)

BBVI estimates the ELBO gradient with Monte Carlo using the log-derivative (REINFORCE) trick:

$$\nabla_\phi \mathcal{L}(\phi) = \mathbb{E}_{q_\phi(\theta)}\left[(\log p(\mathcal{D}, \theta) - \log q_\phi(\theta))\, \nabla_\phi \log q_\phi(\theta)\right]$$

Stochastic VI (SVI)

SVI subsamples mini-batches of data to form a noisy but unbiased ELBO estimate, enabling VI on massive datasets:

$$\mathcal{L}(\phi) \approx \frac{N}{|B|} \sum_{i \in B} \mathbb{E}_{q_\phi(\theta)}[\log p(x_i \mid \theta)] - D_{\mathrm{KL}}(q_\phi(\theta) \,\|\, p(\theta))$$

Reparameterization Gradient (ADVI)

When $q_\phi(\theta)$ can be expressed as a deterministic transformation of a noise variable $\epsilon \sim s(\epsilon)$: $\theta = g_\phi(\epsilon)$, the ELBO gradient becomes:

$$\nabla_\phi \mathcal{L}(\phi) = \mathbb{E}_{s(\epsilon)}\left[\nabla_\phi \log p(\mathcal{D}, g_\phi(\epsilon)) - \nabla_\phi \log q_\phi(g_\phi(\epsilon))\right]$$

This typically yields lower-variance gradient estimates than REINFORCE. Automatic Differentiation Variational Inference (ADVI) automates this in Stan and PyMC.

Amortized VI (VAE Encoder)

Instead of optimizing per-datapoint variational parameters, an inference network (encoder) $f_\phi(x)$ maps each x to the parameters of $q_\phi(z \mid x)$. The ELBO becomes:

$$\mathcal{L}(\phi, \psi) = \mathbb{E}_{q_\phi(z|x)}[\log p_\psi(x \mid z)] - \mathrm{KL}(q_\phi(z \mid x) \,\|\, p(z))$$

This **amortization** shares statistical strength across data points and enables fast inference at test time.

Applications

BBVI: generic VI for any model. SVI: topic models at web scale, matrix factorization. Reparameterization VI: VAEs, Bayesian neural networks, normalizing flows. Amortized VI: VAEs, neural processes, sim-to-real transfer.

Trade-offs

BBVI: General but high-variance gradients. **SVI:** Scales to big data; noisy gradients may slow convergence. **Reparameterization:** Lower variance but requires continuous latent variables and differentiable g_ϕ. **Amortized:** Fast inference; inference network may underfit rare data points.

25.3 EM Algorithm

25.3.1 Standard EM

Expectation-Maximization

For a model with observed variables X and latent variables Z, EM iteratively maximizes the log-likelihood $\log p(X \mid \theta)$:
E-step: Compute the expected complete-data log-likelihood under the current posterior of the latents:

$$Q(\theta \mid \theta^{(t)}) = \mathbb{E}_{Z|X,\theta^{(t)}}[\log p(X, Z \mid \theta)]$$

M-step: Maximize Q w.r.t. θ:

$$\theta^{(t+1)} = \arg\max_\theta Q(\theta \mid \theta^{(t)})$$

The observed data log-likelihood is guaranteed to be non-decreasing: $\log p(X \mid \theta^{(t+1)}) \geq \log p(X \mid \theta^{(t)})$.

Applications

Gaussian Mixture Models (GMMs), Hidden Markov Models (Baum-Welch algorithm), Probabilistic PCA (PPCA), factor analysis, and any model with tractable complete-data likelihood but intractable marginal likelihood.

Trade-offs

Pros:

- Monotonic convergence guarantee (likelihood never decreases).
- Each M-step is typically a simple convex optimization.
- No learning rate or step size to tune.
- Widely applicable: any latent variable model with tractable posterior over latents.

Cons:

- Converges to a local maximum; initialization matters critically.
- The E-step may be intractable, requiring approximation.
- Can be slow: per-iteration improvement shrinks near convergence.
- Does not provide uncertainty in θ; only a point estimate.

25.3.2 EM for Specific Models

EM for GMM

For a GMM with K components, weights π_k, means μ_k, and covariances Σ_k:

$$\text{E-step:} \quad r_{ik} = p(z_i = k \mid x_i) = \frac{\pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_j \pi_j \mathcal{N}(x_i \mid \mu_j, \Sigma_j)}$$

$$\text{M-step:} \quad \pi_k = \frac{1}{N} \sum_i r_{ik}, \quad \mu_k = \frac{\sum_i r_{ik}\, x_i}{\sum_i r_{ik}}, \quad \Sigma_k = \frac{\sum_i r_{ik}(x_i - \mu_k)(x_i - \mu_k)^T}{\sum_i r_{ik}}$$

Baum-Welch for HMM

The E-step computes forward-backward probabilities. The forward variable $\alpha_t(i) = p(x_{1:t}, z_t = i)$ and backward variable $\beta_t(i) = p(x_{t+1:T} \mid z_t = i)$ are computed recursively. The posterior is $\gamma_t(i) = p(z_t = i \mid x_{1:T}) \propto \alpha_t(i)\beta_t(i)$. The M-step re-estimates transition and emission parameters.

EM for Probabilistic PCA (PPCA)

PPCA: $x = Wz + \mu + \epsilon$, $z \sim \mathcal{N}(0, I)$, $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$. E-step: $\mathbb{E}[z \mid x] = M^{-1}W^T(x - \mu)$, $\mathbb{E}[zz^T \mid x] = \sigma^2 M^{-1} + \mathbb{E}[z \mid x]\mathbb{E}[z \mid x]^T$ where $M = W^T W + \sigma^2 I$. M-step updates W and σ^2 with empirical covariances.

25.3.3 Variants: Variational EM and MCEM

Variational EM (VEM)

When the exact E-step posterior $p(Z \mid X, \theta)$ is intractable, VEM replaces it with a variational approximation $q(Z)$ and maximizes the ELBO:

$$\mathcal{L}(\theta, q) = \mathbb{E}_{q(Z)}[\log p(X, Z \mid \theta)] + \mathcal{H}[q(Z)]$$

Alternates between: **VE-step:** $q^{(t+1)} = \arg\max_q \mathcal{L}(\theta^{(t)}, q)$ (CAVI update); **VM-step:** $\theta^{(t+1)} = \arg\max_\theta \mathcal{L}(\theta, q^{(t+1)})$.

Monte Carlo EM (MCEM)

When the E-step expectation is intractable, MCEM approximates it via Monte Carlo samples from the posterior:

$$Q(\theta \mid \theta^{(t)}) \approx \frac{1}{S} \sum_{s=1}^{S} \log p(X, Z^{(s)} \mid \theta), \qquad Z^{(s)} \sim p(Z \mid X, \theta^{(t)})$$

The sample size S is typically increased over iterations as the estimates become more precise.

Applications

VEM: LDA topic models, Bayesian HMM, clustering with Dirichlet processes. MCEM: GLMMs (generalized linear mixed models), phylogenetic tree inference, complex latent variable models where the posterior is available only via sampling.

Trade-offs

VEM: Scales to large models; bias from variational approximation. **MCEM:** Asymptotically exact; Monte Carlo noise makes convergence diagnostics difficult; costly for large S. Both can be combined with stochastic subsampling for big data.

25.4 Bayesian Neural Networks

25.4.1 Bayes by Backprop

Bayes by Backprop (Blundell et al.

Weights are treated as random variables with variational posterior $q(w \mid \theta)$ (typically factorized Gaussian: $w \sim \mathcal{N}(\mu, \sigma^2)$). Using the reparameterization trick $w^{(i)} = \mu + \sigma \odot \epsilon^{(i)}$, $\epsilon^{(i)} \sim \mathcal{N}(0, I)$, the ELBO per mini-batch becomes:

$$\mathcal{L}(\theta) \approx \sum_{i=1}^{M} \Big[\log q(w^{(i)} \mid \theta) - \log p(w^{(i)}) - \log p(\mathcal{D} \mid w^{(i)})\Big]$$

where M is the number of Monte Carlo weight samples, $p(w)$ is the prior (often a scale mixture of Gaussians for sparsity), and the sign is flipped so minimizing this loss maximizes the ELBO.

Applications

Uncertainty-aware deep learning when calibrated confidence intervals are required. Applicable to classification, regression, and reinforcement learning. Implemented in PyTorch via `blitz`, `pyro`, or `tensorflow-probability`.

Trade-offs

Pros:
- Full posterior over weights; principled uncertainty estimates.
- Double the parameters (μ and σ per weight).
- Works with any network architecture.

Cons:
- $2\times$ parameter count; higher memory.
- Training is less stable than deterministic networks.
- The Gaussian mean-field assumption is a strong simplification.
- Tuning the prior scale and the KL weight (cold posterior effect) requires care.

25.4.2 MC Dropout

MC Dropout as Approximate BNN

Gal & Ghahramani (2016) showed that dropout applied at *test time* approximates a deep Gaussian process. Performing T stochastic forward passes with dropout enabled yields approximate posterior samples. The predictive mean and variance are:

$$\mathbb{E}[y^* \mid x^*] \approx \frac{1}{T}\sum_{t=1}^{T} f(x^*; \hat{W}_t), \qquad \mathrm{Var}[y^* \mid x^*] \approx \frac{1}{T}\sum_{t=1}^{T} f(x^*; \hat{W}_t)^2 - \Big(\frac{1}{T}\sum_{t=1}^{T} f(x^*; \hat{W}_t)\Big)^2 + \tau^{-1}$$

where $\hat{W}_t \sim q_\theta^*(W)$ via dropout masks and τ is the model precision. The variance captures epistemic uncertainty.

Applications

Lightweight uncertainty estimation for any pre-trained network with dropout layers. Widely used as a baseline for Bayesian deep learning benchmarks, active learning, and out-of-distribution detection.

Trade-offs

Pros:
- Trivially added to existing dropout-trained models.
- No extra parameters or architectural changes.
- T forward passes at test time trade off speed vs. accuracy.

Cons:
- The variational approximation is crude; uncertainty estimates are often poorly calibrated compared to deep ensembles.
- Dropout rate p must match between training and test.
- The prior is implicitly defined by the network and dropout rate; not user-controllable.
- Variance estimates saturate with T often beyond 50–100 passes.

25.4.3 Flipout

Flipout (Wen et al.

Flipout decorrelates the gradients of different weight samples within a mini-batch, reducing the variance of the stochastic gradient estimator in Bayes by Backprop. Given base parameters W_0 and perturbation ΔW, Flipout generates pseudo-independent perturbations for each example using random sign flips:

$$W_n = W_0 + (\Delta W \odot r_n s_n^T)$$

where $r_n \sim \text{Uniform}(\{\pm 1\})^{d_{\text{out}}}$ and $s_n \sim \text{Uniform}(\{\pm 1\})^{d_{\text{in}}}$ are random sign vectors. This decorrelates the gradients across batch elements without extra forward passes.

Applications

Scalable Bayesian neural networks where standard Bayes by Backprop suffers from high gradient variance. Flipout enables meaningful training of BNNs on ImageNet-scale problems with competitive accuracy.

Trade-offs

Pros: Dramatically reduces gradient variance; enables larger BNNs and batch sizes. Same parameter count as Bayes by Backprop. **Cons:** More complex implementation; still doubles parameters; mean-field assumption remains.

25.4.4 Laplace Approximation

Laplace Approximation

The posterior is locally approximated by a Gaussian centered at the MAP estimate θ_{MAP} with covariance equal to the inverse Hessian of the negative log posterior:

$$q(\theta) \approx \mathcal{N}\big(\theta_{\text{MAP}},\, \mathbf{H}^{-1}\big), \qquad \mathbf{H} = -\nabla_\theta^2 \log p(\theta \mid \mathcal{D})\big|_{\theta_{\text{MAP}}}$$

The Hessian is typically approximated by the Generalized Gauss-Newton (GGN) matrix or the Fisher information matrix. For neural networks, the marginal likelihood can be optimized for hyperparameter tuning.

Kronecker-Factored Laplace (KFLA)

K-FAC approximates the Hessian (or Fisher) as a Kronecker product per layer:

$$\mathbf{H} \approx \mathbf{A} \otimes \mathbf{B}$$

where $\mathbf{A}$ captures activation covariances and $\mathbf{B}$ captures gradient covariances. This reduces inversion cost from $O(d_{\text{out}}^3\, d_{\text{in}}^3)$ to $O(d_{\text{out}}^3 + d_{\text{in}}^3)$ and enables post-hoc Laplace on modern architectures (ResNet, ViT) without retraining.

Applications

Post-hoc uncertainty quantification for pre-trained networks: take a trained checkpoint, compute the MAP (it is already at a minimum), estimate the Hessian diagonal or K-FAC, and form a Gaussian posterior. Used for continual learning (EWC), model selection, and out-of-distribution detection.

Trade-offs

Pros:
- Post-hoc: applied after standard training, no retraining needed.
- Principled Bayesian interpretation.
- The Hessian diagonal is cheap; K-FAC is accurate and scalable.

Cons:
- Gaussian assumption may be poor for multi-modal posteriors.
- The MAP must be at a true mode (requires convergence to a local minimum).
- Hessian computation and storage is non-trivial for very large models.
- The curvature at the MAP may not represent the global posterior shape.

25.4.5 SWAG

Stochastic Weight Averaging Gaussian (SWAG)

SWAG approximates the posterior as a Gaussian by collecting SGD iterates near convergence:

$$p(\theta \mid \mathcal{D}) \approx \mathcal{N}\Big(\bar{\theta},\ \frac{1}{2}\big(\Sigma_{\text{diag}} + \Sigma_{\text{low-rank}}\big)\Big)$$

where $\bar{\theta}$ is the running mean of SGD iterates, $\Sigma_{\text{diag}} = \overline{\theta^2} - \bar{\theta}^2$ is the diagonal variance, and $\Sigma_{\text{low-rank}} = \frac{1}{K-1}\sum_{k=1}^{K}(\theta_k - \bar{\theta})(\theta_k - \bar{\theta})^T$ captures low-rank structure from K stored checkpoints. The $\frac{1}{2}$ factor accounts for the empirical observation that the posterior covers a wider region than SGD iterates.

Applications

Uncertainty calibration for image classification (ResNet, WideResNet) and out-of-distribution detection. Competitive with deep ensembles but at the cost of a single training run plus K stored checkpoints.

Trade-offs

Pros: Single training run (unlike ensembles). Simple to implement on top of any SGD training loop. Good empirical calibration. **Cons:** Requires storing K checkpoints (e.g., $K = 20$) for the low-rank component. Batch normalization statistics need re-estimation. The linear learning-rate schedule for collecting samples is a sensitive hyperparameter.

25.5 Gaussian Processes

25.5.1 GP Prior and Posterior

Gaussian Process

A Gaussian process is a collection of random variables, any finite subset of which is jointly Gaussian. It is fully specified by a mean function $m(x)$ and a covariance kernel $k(x, x')$:

$$f(x) \sim \mathcal{GP}(m(x),\ k(x, x'))$$

Given training inputs X and observed function values $\mathbf{f} = f(X)$, the joint distribution with test points X_* is:

$$\begin{pmatrix} \mathbf{f} \\ \mathbf{f}_* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} m(X) \\ m(X_*) \end{pmatrix},\ \begin{pmatrix} K_{ff} & K_{f*} \\ K_{*f} & K_{**} \end{pmatrix} \right)$$

GP Posterior — Noiseless

Conditioning on the training data, the posterior predictive at test points is:

$$\mathbf{f}_* \mid X_*, X, \mathbf{f} \sim \mathcal{N}\big(K_{*f} K_{ff}^{-1} \mathbf{f},\ K_{**} - K_{*f} K_{ff}^{-1} K_{f*}\big)$$

GP Posterior — Noisy Observations

With Gaussian noise σ_n^2, observations are $y = f(x) + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma_n^2)$. The posterior becomes:

$$\mathbf{f}_* \mid X_*, X, \mathbf{y} \sim \mathcal{N}\big(K_{*f}(K_{ff} + \sigma_n^2 I)^{-1} \mathbf{y},\ K_{**} - K_{*f}(K_{ff} + \sigma_n^2 I)^{-1} K_{f*}\big)$$

The noise term $\sigma_n^2 I$ also ensures numerical stability of the Cholesky decomposition.

Applications

Regression with uncertainty, Bayesian optimization, time-series forecasting, spatial statistics (kriging), and emulation of expensive simulators. GPy, GPflow, GPyTorch, and scikit-learn provide implementations.

Trade-offs

Pros:

- Non-parametric: model capacity grows with data.
- Closed-form predictive distribution with well-calibrated uncertainty.
- Interpretable kernel design encodes domain knowledge (smoothness, periodicity, additivity).
- Hyperparameters learned by maximizing marginal likelihood.

Cons:

- $O(N^3)$ training time and $O(N^2)$ memory from the Cholesky decomposition of K_{ff}.
- Infeasible beyond $N \approx 10^4$–10^5 data points without approximations.
- Kernel choice heavily influences predictions; misspecification leads to poor uncertainty.
- Standard kernels struggle in high-dimensional input spaces.

25.5.2 GP Classification

GP Classification via Laplace Approximation

For binary classification, a GP prior is placed over the latent function $f(x)$, and the likelihood is Bernoulli with a sigmoid link: $p(y = 1 \mid f) = \sigma(f)$. The posterior over f is non-Gaussian. The Laplace approximation:

1. Find the MAP latent values $\hat{\mathbf{f}} = \arg\max_{\mathbf{f}} \log p(\mathbf{y} \mid \mathbf{f}) + \log p(\mathbf{f})$ via Newton's method.
2. Approximate posterior as $\mathcal{N}(\hat{\mathbf{f}}, (K_{ff}^{-1} + W)^{-1})$ where $W = -\nabla^2 \log p(\mathbf{y} \mid \hat{\mathbf{f}})$.
3. Predictive: $p(y_* = 1 \mid X_*, X, \mathbf{y}) = \int \sigma(f_*) p(f_* \mid X_*, X, \mathbf{y})\, df_*$, approximated by probit or numerical integration.

Applications

Binary and multi-class classification with uncertainty. Expectation Propagation (EP) is an alternative that typically yields better calibrated probabilities than Laplace but is more complex.

25.5.3 Sparse Gaussian Processes

Sparse GP with Inducing Points

Introduce $M \ll N$ inducing points at locations Z with corresponding latent values $\mathbf{u} = f(Z)$. The inducing variables summarize the training data. The Variational Free Energy (VFE) / Titsias bound:

$$\mathcal{F}_V = -\frac{N}{2} \log 2\pi - \frac{1}{2} \log |Q_{ff} + \sigma_n^2 I| - \frac{1}{2} \mathbf{y}^T (Q_{ff} + \sigma_n^2 I)^{-1} \mathbf{y} - \frac{1}{2\sigma_n^2} \operatorname{Tr}(K_{ff} - Q_{ff})$$

where $Q_{ff} = K_{fu} K_{uu}^{-1} K_{uf}$ is the Nyström approximation of K_{ff}. The trace term $\operatorname{Tr}(K_{ff} - Q_{ff})$ penalizes the approximation gap, ensuring the objective is a true lower bound to the log marginal likelihood. Complexity: $O(NM^2)$.

Applications

Scaling GPs to datasets with millions of points. Used in Bayesian optimization with many evaluations, spatial statistics, and any GP application exceeding $N \approx 10^4$. Implemented in GPyTorch and GPflow.

Trade-offs

Pros: Reduces complexity from $O(N^3)$ to $O(NM^2)$ with $M \ll N$. The trace term ensures that Z and kernel hyperparameters can be optimized jointly without overfitting. **Cons:** Inducing point count M is a capacity hyperparameter. Large M approaches full GP cost; small M underfits. Optimization over Z is non-convex.

25.6 Bayesian Optimization

Bayesian optimization (BO) is a framework for global optimization of expensive, black-box, possibly noisy functions. It builds a probabilistic surrogate model (typically a GP) and uses an acquisition function to decide where to evaluate next.

25.6.1 Expected Improvement (EI)

Expected Improvement

Let $\mu(x)$ and $\sigma^2(x)$ be the GP posterior mean and variance at x, and f^* be the current best observed value. The expected improvement over f^* is:

$$\begin{aligned}\alpha_{\text{EI}}(x) &= \mathbb{E}\big[\max(f(x) - f^*, 0)\big] \\ &= (\mu(x) - f^* - \xi)\,\Phi(Z) + \sigma(x)\,\phi(Z)\end{aligned}$$

where $Z = (\mu(x) - f^* - \xi)/\sigma(x)$, Φ is the Gaussian CDF, ϕ is the Gaussian PDF, and $\xi \geq 0$ is an exploration parameter. Setting $\xi > 0$ encourages exploration by effectively lowering the threshold to improve upon.

Applications

The most widely used acquisition function. Hyperparameter tuning of machine learning models, experimental design, A/B testing, material discovery, and robotics.

Trade-offs

Pros:

- Balances exploration and exploitation naturally via the $\sigma\phi$ term.
- Non-negative and zero at evaluated points (no re-sampling without noise).
- The exploration parameter ξ gives explicit control.

Cons:

- Can be overly exploitative when the GP overestimates μ near evaluated points.
- The maximization of α_{EI} is itself a non-convex optimization problem.
- May perform poorly in high dimensions ($d > 20$) due to the curse of dimensionality.

25.6.2 Probability of Improvement (PI)

Probability of Improvement

PI selects the point with the highest probability of exceeding f^*:

$$\alpha_{\text{PI}}(x) = \Phi\left(\frac{\mu(x) - f^* - \xi}{\sigma(x)}\right)$$

where $\xi > 0$ is the exploration parameter. Without ξ, PI is purely exploitative and tends to sample very close to the current best point, getting stuck.

Applications

Simple BO baseline. Less common than EI in practice because it ignores the magnitude of improvement; a point has the same PI whether it is expected to improve by 0.01 or 100.

Trade-offs

Pros: Extremely simple; interpretable as the probability of improvement. **Cons:** Ignores the magnitude of improvement; requires careful ξ tuning; can cause over-exploitation without it.

25.6.3 Upper Confidence Bound (GP-UCB)

GP-UCB

The Upper Confidence Bound acquisition function is:

$$\alpha_{\text{UCB}}(x) = \mu(x) + \kappa\,\sigma(x)$$

For minimization, $\alpha_{\text{LCB}}(x) = \mu(x) - \kappa\,\sigma(x)$ is used. The GP-UCB algorithm sets $\kappa_t = \sqrt{\nu \log t}$ with appropriate ν for a Bayesian regret bound, ensuring provable convergence.

Applications

Theoretically grounded BO with regret guarantees. Popular in bandit problems and when d is moderate. The explicit κ parameter gives direct control over the exploration-exploitation trade-off.

Trade-offs

Pros: Simple, interpretable, and provably no-regret with appropriate κ schedule. **Cons:** Regret guarantees require κ that grows with t, making it overly exploratory for reasonable budgets. Fixed κ lacks justification. The linear combination of μ and σ is not scale-invariant.

25.6.4 Thompson Sampling and Information-Theoretic Methods

Thompson Sampling

At each iteration, draw a sample function $\tilde{f}(x)$ from the GP posterior, then select:

$$x_{\text{next}} = \arg\max_x \tilde{f}(x)$$

This naturally balances exploration (regions with high variance are likely to have large sampled values) and exploitation (regions near current best are likely to have high sampled values).

Entropy Search and Max-value Entropy Search (MES)

Entropy Search selects the point that maximally reduces uncertainty about the location of the global optimum:

$$\alpha_{\text{ES}}(x) = \mathcal{H}[p(x^* \mid \mathcal{D})] - \mathbb{E}_{y|x,\mathcal{D}}\left[\mathcal{H}[p(x^* \mid \mathcal{D} \cup (x,y))]\right]$$

MES simplifies by maximizing information about the *value* of the optimum f^*:

$$\alpha_{\text{MES}}(x) = \mathcal{H}[p(y \mid x,\mathcal{D})] - \mathbb{E}_{f^*|\mathcal{D}}\left[\mathcal{H}[p(y \mid x,\mathcal{D},f^*)]\right]$$

MES is cheaper to compute than ES and empirically competitive.

Applications

Thompson Sampling: simple, efficient, handles parallel evaluations by sampling multiple functions. ES/MES: principled global optimization with provable convergence; MES is preferred for lower computational cost. Both handle noisy evaluations well.

25.7 State Space Models

25.7.1 Kalman Filter

Kalman Filter

The KF estimates the hidden state $\mathbf{x}_t$ of a linear dynamical system from noisy observations $\mathbf{y}_t$:

$$\mathbf{x}_t = \mathbf{F}_t\mathbf{x}_{t-1} + \mathbf{B}_t\mathbf{u}_t + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(0, \mathbf{Q}_t)$$
$$\mathbf{y}_t = \mathbf{H}_t\mathbf{x}_t + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(0, \mathbf{R}_t)$$

Predict step:

$$\hat{\mathbf{x}}_{t|t-1} = \mathbf{F}_t\hat{\mathbf{x}}_{t-1|t-1} + \mathbf{B}_t\mathbf{u}_t$$
$$\mathbf{P}_{t|t-1} = \mathbf{F}_t\mathbf{P}_{t-1|t-1}\mathbf{F}_t^T + \mathbf{Q}_t$$

Update step:

$$\mathbf{K}_t = \mathbf{P}_{t|t-1}\mathbf{H}_t^T(\mathbf{H}_t\mathbf{P}_{t|t-1}\mathbf{H}_t^T + \mathbf{R}_t)^{-1}$$
$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t(\mathbf{y}_t - \mathbf{H}_t\hat{\mathbf{x}}_{t|t-1})$$
$$\mathbf{P}_{t|t} = (\mathbf{I} - \mathbf{K}_t\mathbf{H}_t)\mathbf{P}_{t|t-1}$$

The Kalman gain $\mathbf{K}_t$ balances prediction and measurement.

Applications

Tracking, navigation (GPS/IMU fusion), signal processing, econometrics (time-varying parameter models), and robotics. Extended KF (EKF) and Unscented KF (UKF) handle non-linear dynamics.

Trade-offs

Pros: Exact Bayesian filtering for linear-Gaussian systems; $O(d^3)$ per step; optimal in the MSE sense. **Cons:** Assumes linear dynamics and Gaussian noise. EKF/UKF are approximations for non-linear systems. Requires known model parameters $(\mathbf{F}, \mathbf{H}, \mathbf{Q}, \mathbf{R})$.

25.7.2 Particle Filter

Particle Filter (Sequential Monte Carlo)

Particle filters approximate the filtering distribution $p(\mathbf{x}_t \mid \mathbf{y}_{1:t})$ with a weighted set of N particles $\{\mathbf{x}_t^{(i)}, w_t^{(i)}\}_{i=1}^N$:

Sequential Importance Sampling (SIS):

1. Sample $\mathbf{x}_t^{(i)} \sim q(\mathbf{x}_t \mid \mathbf{x}_{t-1}^{(i)}, \mathbf{y}_t)$ (proposal distribution).
2. Update weights: $w_t^{(i)} \propto w_{t-1}^{(i)} \cdot \frac{p(\mathbf{y}_t|\mathbf{x}_t^{(i)})\, p(\mathbf{x}_t^{(i)}|\mathbf{x}_{t-1}^{(i)})}{q(\mathbf{x}_t^{(i)}|\mathbf{x}_{t-1}^{(i)}, \mathbf{y}_t)}$.

Sampling Importance Resampling (SIR) / Bootstrap Filter: Use the transition prior as the proposal: $q(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{y}_t) = p(\mathbf{x}_t \mid \mathbf{x}_{t-1})$. Weights simplify to $w_t^{(i)} \propto p(\mathbf{y}_t \mid \mathbf{x}_t^{(i)})$. Resample particles when the effective sample size $N_{\text{eff}} \approx 1/\sum_i (w_t^{(i)})^2$ drops below a threshold.

Applications

Non-linear, non-Gaussian state estimation: target tracking, robot localization (Monte Carlo localization), financial volatility modeling, and epidemiological modeling. The bootstrap filter is the most common variant.

Trade-offs

Pros:

- Handles arbitrary non-linear, non-Gaussian dynamics and observations.
- As $N \to \infty$, converges to the true filtering distribution.
- Simple to implement (bootstrap filter).

Cons:

- Sample degeneracy: a few particles dominate the weights.
- Sample impoverishment: after resampling, particle diversity decreases.
- $O(N)$ per time step; N must be large for high-dimensional state spaces (curse of dimensionality).
- Performance depends on the proposal distribution choice.

25.8 Summary Comparison of Bayesian Methods

Practical Recommendations

- **Small data / critical uncertainty:** MCMC (Stan, PyMC) gives asymptotically exact posteriors.
- **Practical BNN:** Deep Ensembles + MC Dropout are strong baselines. Use SWAG or KFLA-Laplace for single-training-run alternatives.
- **Large-scale regression with uncertainty:** Sparse GP (GPyTorch) with $M = 500$–2000 inducing points.
- **Bayesian optimization:** GP with Matérn 5/2 kernel + EI acquisition. Upgrade to MES for parallel or high-dimensional settings.
- **State estimation:** Kalman filter for linear-Gaussian; particle filter for non-linear/non-Gaussian with moderate state dimension.

Table 25.1: Comparison of Bayesian Inference Methods

Method	**Inference Type**	**Uncertainty**	**Scalability**	**Key Lim**
Exact (conjugate)	Closed-form posterior	Full	Small	Restricted tial family
MCMC (e.g., HMC)	Sampling from posterior	Asymptotically exact	Medium	Slow; req gence diag
Mean-Field VI	Optimization of ELBO	Underestimated variance	Large	Factorizat
BBVI / ADVI	Stochastic gradient VI	Approximate	Very large	High-varia quires repa
Bayes by Backprop	VI for BNN weights	Approximate	Large	2× param
MC Dropout	Dropout at test time	Approximate	Large	Crude app
Laplace (KFLA)	Post-hoc Gaussian	Local approximation	Large	Single-mo
SWAG	SGD iterates Gaussian	Approximate	Large	Stores che
Deep Ensembles	Independent MAPs	Disagreement	Medium (M× cost)	Not forma
GP (exact)	Analytical posterior	Well-calibrated	Small ($N < 10^4$)	$O(N^3)$
Sparse GP (VFE)	Variational bound	Approximate	Large ($N \gg 10^4$)	Inducing p

Chapter 26

Clustering Algorithms Math

Clustering partitions unlabeled data into groups such that points within the same cluster are similar and points across clusters are dissimilar. This chapter covers the mathematical foundations of centroid-based, hierarchical, density-based, spectral, model-based, and deep clustering methods.

26.1 Centroid-Based Clustering

26.1.1 K-means

K-means Objective

Given N data points $\{x_i\}_{i=1}^N \subset \mathbb{R}^d$ and K clusters, K-means minimizes the within-cluster sum of squared distances:

$$\min_{\{C_k\},\, \{\mu_k\}_{k=1}^K} \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

where C_k is the set of points assigned to cluster k and μ_k is the centroid of cluster k. The objective is equivalent to minimizing the variance within each cluster; the global optimum coincides with a Voronoi tessellation of the data space.

Lloyd's Algorithm

Alternating minimization:

1. **Assignment step:** assign each x_i to the nearest centroid: $C_k^{(t)} = \{x_i : \|x_i - \mu_k^{(t)}\|^2 \leq \|x_i - \mu_j^{(t)}\|^2 \; \forall j\}$
2. **Update step:** recompute centroids as the mean of assigned points: $\mu_k^{(t+1)} = \frac{1}{|C_k^{(t)}|} \sum_{i \in C_k^{(t)}} x_i$

Converges to a local minimum of the objective. The cost is $O(NKd)$ per iteration.

Applications

The most widely used clustering algorithm. Document clustering, customer segmentation, image quantization (color reduction), and initialization of other algorithms (GMM with identity covariances). Implemented in scikit-learn as `KMeans`.

Trade-offs

Pros:

- Simple and fast: linear in N, K, d per iteration.
- Guaranteed convergence.
- Easily parallelizable for large datasets (Mini-Batch K-means).

Cons:

- K must be specified in advance.
- Sensitive to initialization (mitigated by K-means++).
- Assumes spherical, equal-variance clusters → fails on elongated or irregular shapes.
- Hard assignments: a point belongs to exactly one cluster.
- Outliers can drastically shift centroids (use K-medoids instead).
- Local minima; multiple restarts required.

26.1.2 K-means++

K-means++ Initialization

K-means++ selects initial centroids sequentially to be far apart:

1. Choose first centroid μ_1 uniformly at random from the data.
2. For each subsequent centroid $k = 2, \ldots, K$: sample x_i with probability proportional to its squared distance from the nearest already-chosen centroid:

$$p(x_i) \propto \min_{j<k} \|x_i - \mu_j\|^2$$

This provides an $O(\log K)$ approximation guarantee to the optimal clustering cost in expectation.

Applications

The default initialization in scikit-learn's K-means (`init='k-means++'`). Significantly improves both convergence speed and final cluster quality over random initialization.

Trade-offs

Pros: Provable approximation bound; faster convergence; better local optima. **Cons:** Sequential selection is not trivially parallelizable. Marginal overhead of $O(NKd)$ for computing distances once.

26.1.3 K-medoids (PAM)

K-medoids (Partitioning Around Medoids)

K-medoids minimizes the sum of dissimilarities to *actual data points* (medoids) rather than geometric centroids:

$$\min_{\{C_k\},\, \{m_k \in \mathcal{D}\}} \sum_{k=1}^{K} \sum_{i \in C_k} d(x_i, m_k)$$

where $d(\cdot, \cdot)$ is any dissimilarity metric and medoids are constrained to be data points. The standard PAM algorithm alternates between BUILD (greedy initialization) and SWAP (iteratively swap medoid with non-medoid if it reduces total cost).

Applications

Clustering with arbitrary distance metrics (e.g., edit distance for strings). Robust clustering when outliers are present. The medoid is an interpretable representative point. Used in bioinformatics and text analysis.

Trade-offs

Pros: Works with any dissimilarity. Robust to outliers and noise. Medoids are actual data points (interpretable). **Cons:** $O(N^2)$ per SWAP iteration; does not scale to large datasets. More expensive than K-means. Still assumes K is known.

26.1.4 Fuzzy C-Means

Fuzzy C-Means (FCM)

FCM allows soft membership: each point has a degree of belonging $u_{ik} \in [0, 1]$ to each cluster, with $\sum_k u_{ik} = 1$:

$$\min_{\{c_k\},\, \{u_{ik}\}} \sum_{i=1}^{N} \sum_{k=1}^{K} u_{ik}^m \, \|x_i - c_k\|^2$$

The fuzziness parameter $m > 1$ controls membership softness ($m \to 1$ gives hard K-means; $m \to \infty$ gives uniform memberships). The update rules are:

$$u_{ik} = \left(\sum_{j=1}^{K} \left(\frac{\|x_i - c_k\|}{\|x_i - c_j\|} \right)^{\frac{2}{m-1}} \right)^{-1}, \qquad c_k = \frac{\sum_i u_{ik}^m x_i}{\sum_i u_{ik}^m}$$

Applications

Soft clustering where boundaries between clusters are ambiguous. Medical image segmentation, fuzzy rule-based systems, and pattern recognition.

Trade-offs

Pros: Soft assignments capture uncertainty in boundary regions. Naturally generalizes K-means. **Cons:** Sensitive to initialization. The fuzziness m is an extra hyperparameter (typical: $m = 2$). Computationally more expensive than K-means.

26.2 Hierarchical Clustering

26.2.1 Agglomerative Linkage Criteria

Agglomerative Clustering Linkages

Agglomerative clustering starts with each point as a singleton cluster and iteratively merges the two closest clusters. The **linkage criterion** defines "closest":

- **Single linkage:** $d(A, B) = \min_{a \in A, b \in B} \|a - b\|$ (chaining effect, can find elongated clusters).

- **Complete linkage:** $d(A,B) = \max_{a\in A, b\in B} \|a-b\|$ (tends to find compact, equal-diameter clusters).
- **Average linkage:** $d(A,B) = \frac{1}{|A||B|}\sum_{a\in A}\sum_{b\in B}\|a-b\|$ (compromise; robust to noise).
- **Ward's method:** merge the pair that minimizes the increase in total within-cluster variance:

$$\Delta(A,B) = \frac{|A||B|}{|A|+|B|}\|\mu_A - \mu_B\|^2$$

Equivalent to K-means objective pursued greedily.

Applications

Dendrogram visualization for exploratory data analysis. Taxonomy construction in biology (phylogenetic trees). Document hierarchy discovery. When a tree structure is more informative than flat clusters.

Trade-offs

Pros: No need to specify K upfront (can be determined by cutting the dendrogram). Produces a complete hierarchy. Deterministic (given linkage and metric). **Cons:** $O(N^3)$ naive, $O(N^2 \log N)$ for some linkages; does not scale to large datasets. Greedy merges are irreversible. Sensitive to noise and outliers (especially single linkage).

26.2.2 BIRCH

BIRCH — Balanced Iterative Reducing and Clustering

BIRCH builds a Clustering Feature (CF) tree to compress data before clustering. A Clustering Feature summarizes a subcluster C by a triple:

$$CF = (N,\ \mathbf{LS},\ SS), \qquad \mathbf{LS} = \sum_{i\in C} x_i, \quad SS = \sum_{i\in C}\|x_i\|^2$$

These are additive: merging two subclusters simply adds their CFs. From a CF, the centroid $\mu = \mathbf{LS}/N$ and radius/variance can be computed without storing individual points. The CF tree organizes CFs in a height-balanced tree with branching factor B and threshold T.

Applications

Large-scale clustering with limited memory. BIRCH can process data in a single pass, making it suitable for streaming and very large datasets. Implemented in scikit-learn as `Birch`.

Trade-offs

Pros: Sublinear memory ($O(N)$ compressed to O(tree size)). Single-pass capable. Incremental: new points can be inserted on-the-fly. **Cons:** Sensitive to data order (points aren't revisited). Works best for convex, isotropic clusters. The threshold T and branching factor B are tuning parameters.

26.3 Density-Based Clustering

26.3.1 DBSCAN

DBSCAN — Density-Based Spatial Clustering

Two parameters: ϵ (radius) and minPts (minimum points).

- **Core point:** x has at least minPts points (including itself) within its ϵ-neighborhood: $|N_\epsilon(x)| \geq \text{minPts}$.
- **Directly density-reachable:** y is directly reachable from a core point x if $y \in N_\epsilon(x)$.
- **Density-reachable:** there is a chain of core points $x = p_1, p_2, \ldots, p_n = y$ such that each p_{i+1} is directly reachable from p_i.
- **Density-connected:** x and y are both reachable from some core point o.

A cluster is a maximal set of density-connected points. Points not belonging to any cluster are labeled as noise.

Applications

Spatial data with arbitrary cluster shapes. Anomaly detection (points labeled as noise). Geospatial clustering (GPS trajectories), astronomy (star cluster detection), and network analysis.

Trade-offs

Pros:

- Does not require specifying K.
- Finds arbitrarily-shaped clusters.
- Robust to outliers (they are labeled as noise).
- Only two parameters (ϵ, minPts) with intuitive meanings.

Cons:

- Fails on clusters of varying density (global ϵ).
- Struggles in high dimensions due to the curse of dimensionality.
- ϵ selection is non-trivial; the k-distance plot heuristic uses the knee of the sorted k^{th}-neighbor distances.
- Border points are arbitrarily assigned to whichever core point processes them first.

26.3.2 OPTICS

OPTICS — Ordering Points To Identify the Clustering Structure

OPTICS extends DBSCAN by producing an augmented ordering of points that encodes density-based clustering structure at *all* ϵ scales. For each point, two values are computed:

- **Core distance:** the distance to the minPtsth nearest neighbor (or undefined if not a core point).
- **Reachability distance** from o to p: $\max(\text{core-dist}(o), \|o - p\|)$.

Points are processed in order of smallest reachability distance. The reachability plot visualizes cluster structure: valleys correspond to clusters, peaks to noise. Clusters can be extracted at any ϵ threshold.

Applications

Exploratory analysis where the density scale of clusters is unknown. The reachability plot provides a visual diagnostic of cluster hierarchy and density. Used in geospatial analysis and bioinformatics.

Trade-offs

Pros: Detects clusters over a continuum of densities. Removes the ϵ parameter from the core algorithm. The reachability plot is a rich visualization. **Cons:** More computationally expensive than DBSCAN (priority queue). Cluster extraction still requires a threshold (ξ steepness parameter). Not as widely supported in libraries.

26.3.3 HDBSCAN

HDBSCAN — Hierarchical DBSCAN

HDBSCAN builds a hierarchy of DBSCAN clusters by varying ϵ (or equivalently, minPts for a fixed min cluster size). Key concepts:

- **Mutual reachability distance:** $d_{\text{mreach}}(a, b) = \max\{\text{core}_k(a),\ \text{core}_k(b),\ \|a - b\|\}$ where $\text{core}_k(x)$ is the distance to the kth nearest neighbor.
- **Cluster stability:** the sum over points in a cluster of $(\lambda_{\text{birth}}^{-1} - \lambda_{\text{death}}^{-1})$ where $\lambda = 1/\epsilon$ is the inverse distance.

A condensed tree is built by collapsing small splits, and the most stable clusters are extracted.

Applications

State-of-the-art density-based clustering when clusters of varying density exist. The default choice over DBSCAN for exploratory analysis. Highly effective in practice with only the `min_cluster_size` parameter.

Trade-offs

Pros: Handles varying density. Minimal parameters (min cluster size only). Robust to noise. Hierarchical output with stable cluster extraction. **Cons:** $O(N^2)$ pairwise distance computation (alleviated by approximate algorithms). Memory-intensive for large datasets.

26.3.4 Mean-Shift

Mean-Shift Clustering

Mean-Shift is a mode-seeking algorithm based on kernel density estimation. Data points iteratively climb the density gradient toward modes:
For a point x, the mean shift vector at iteration t is:

$$\mathbf{m}(x^{(t)}) = \frac{\sum_{x_i \in N(x^{(t)})} K(x_i - x^{(t)})\, x_i}{\sum_{x_i \in N(x^{(t)})} K(x_i - x^{(t)})}$$

Update: $x^{(t+1)} \leftarrow \mathbf{m}(x^{(t)})$. Points converging to the same mode form a cluster. The kernel K (typically a flat kernel of bandwidth h or a Gaussian) defines the neighborhood.

Applications

Image segmentation (color clustering), object tracking (CamShift), and any application where the number of clusters is completely unknown. Mean-Shift automatically discovers the number of modes.

Trade-offs

Pros: No K required. Finds arbitrarily-shaped clusters. Robust kernel density estimation framework. **Cons:** $O(N^2)$ per iteration. The bandwidth h is a critical hyperparameter (dictates cluster granularity). Slow to converge. Struggles in high dimensions.

26.4 Spectral Clustering

26.4.1 Graph Laplacian and Normalized Cut

Graph Laplacian Matrices

Given a similarity graph with adjacency matrix W (e.g., $W_{ij} = \exp(-\gamma\|x_i - x_j\|^2)$ or k-nearest neighbors), let D be the diagonal degree matrix with $D_{ii} = \sum_j W_{ij}$:

- **Unnormalized Laplacian:** $L = D - W$
- **Symmetric normalized Laplacian:** $L_{\text{sym}} = I - D^{-1/2}WD^{-1/2}$
- **Random-walk normalized Laplacian:** $L_{\text{rw}} = I - D^{-1}W$

The Normalized Cut objective for a partition into sets A and B is:

$$\text{NCut}(A, B) = \frac{\text{cut}(A, B)}{\text{assoc}(A, V)} + \frac{\text{cut}(A, B)}{\text{assoc}(B, V)}$$

where $\text{cut}(A, B) = \sum_{i \in A,\, j \in B} W_{ij}$ and $\text{assoc}(A, V) = \sum_{i \in A,\, j \in V} W_{ij} = \sum_{i \in A} D_{ii}$. Minimizing NCut is NP-hard; the spectral relaxation solves the eigenproblem of L_{rw} or L_{sym}.

Spectral Clustering Algorithm

1. Construct similarity graph W from data.
2. Compute the first K eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_K$ corresponding to the K smallest eigenvalues of L_{sym} (or the generalized eigenproblem $L\mathbf{v} = \lambda D\mathbf{v}$ for L_{rw}).
3. Form matrix $U \in \mathbb{R}^{N \times K}$ with columns $\mathbf{v}_1, \ldots, \mathbf{v}_K$. Normalize rows to unit length.
4. Treat each row of U as an embedding of the corresponding data point; cluster with K-means.

Applications

Clustering non-convex, intertwined structures (concentric circles, spirals). Image segmentation (Shi-Malik normalized cuts). Community detection in graphs. Dimensionality reduction (Laplacian eigenmaps).

Trade-offs

Pros:

- Can separate clusters with complex, non-convex boundaries.
- Solid theoretical foundation in spectral graph theory.

- Only requires a similarity matrix; data coordinates are optional.
- Works with any metric (via the similarity kernel).

Cons:

- $O(N^3)$ eigen-decomposition; infeasible for large N. Approximations (Nyström, power iteration) reduce cost.
- The similarity graph construction (kernel + kNN) is critical and involves hyperparameters (γ, k).
- K must be specified.
- The K-means post-processing step introduces additional randomness.
- Eigengap heuristic for choosing K is not always reliable.

26.5 GMM and LDA

26.5.1 Gaussian Mixture Model (GMM) EM

GMM and Its EM Derivation

A GMM models the data density as a weighted sum of K Gaussians:

$$p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k), \qquad \sum_k \pi_k = 1$$

The latent variable $z_i \in \{1, \ldots, K\}$ indicates which component generated x_i. The complete-data log-likelihood is:

$$\log p(X, Z \mid \theta) = \sum_{i=1}^{N} \sum_{k=1}^{K} \mathbb{I}[z_i = k] \big(\log \pi_k + \log \mathcal{N}(x_i \mid \mu_k, \Sigma_k)\big)$$

E-step: Compute responsibilities (posterior probabilities):

$$r_{ik} = p(z_i = k \mid x_i) = \frac{\pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_i \mid \mu_j, \Sigma_j)}$$

M-step: Update parameters with weighted MLE:

$$\pi_k = \frac{N_k}{N}, \quad \mu_k = \frac{1}{N_k} \sum_i r_{ik}\, x_i, \quad \Sigma_k = \frac{1}{N_k} \sum_i r_{ik} (x_i - \mu_k)(x_i - \mu_k)^T$$

where $N_k = \sum_i r_{ik}$.

Applications

Soft clustering with per-component covariances (ellipsoidal clusters). Density estimation, anomaly detection, and as a building block for more complex models (HMM, switching state-space models).

Trade-offs

Pros: Soft assignments. Full covariance modeling captures correlation structure. Probabilistic framework: likelihood, BIC for model selection. **Cons:** $O(NKd^2 + Kd^3)$ per

iteration. EM converges to local maxima. Must specify K and covariance type (full, tied, diagonal, spherical). Sensitive to initialization and outliers.

26.5.2 Dirichlet Process GMM

DP-GMM — Infinite Mixture Model

A Dirichlet Process prior allows the number of mixture components to grow with data. The generative model is:

$$G \sim \text{DP}(\alpha, G_0) \quad \text{(base measure } G_0\text{, concentration } \alpha\text{)}$$
$$\theta_i \mid G \sim G, \qquad x_i \mid \theta_i \sim F(\theta_i)$$

The Chinese Restaurant Process (CRP) representation:

$$p(z_i = k \mid \mathbf{z}_{<i}) = \begin{cases} \frac{N_k}{i-1+\alpha} & \text{existing cluster } k \\ \frac{\alpha}{i-1+\alpha} & \text{new cluster} \end{cases}$$

This naturally infers the number of clusters; α controls how many new clusters are created.

Applications

When K is unknown and should be inferred from data. Bayesian non-parametric clustering, topic modeling with an unbounded number of topics, and density estimation with automatic model selection.

Trade-offs

Pros: No need to specify K; the posterior over K is inferred. **Cons:** MCMC or variational inference required. Concentration parameter α strongly affects the number of clusters. Interpretation of inferred clusters can be challenging.

26.5.3 Latent Dirichlet Allocation (LDA)

LDA for Topic Modeling

LDA is a generative probabilistic model for a corpus of D documents. Each document d has a topic distribution $\theta_d \sim \text{Dirichlet}(\boldsymbol{\alpha})$, and each topic k is a distribution over words $\beta_k \sim \text{Dirichlet}(\boldsymbol{\eta})$. For each word position n in document d:

$$z_{d,n} \sim \text{Categorical}(\theta_d), \qquad w_{d,n} \sim \text{Categorical}(\beta_{z_{d,n}})$$

Inference (via collapsed Gibbs sampling or variational EM) recovers θ (document-topic distributions) and β (topic-word distributions). LDA is a clustering of words into topics and documents into topic mixtures.

Applications

Topic discovery in text corpora, document clustering, recommender systems (collaborative filtering as LDA on user-item matrices), and bioinformatics (gene expression clustering).

Trade-offs

Pros: Interpretable topic-word distributions. Mixed membership (each document is a mixture of topics). Bayesian: handles sparsity, provides uncertainty. **Cons:** α and η require tuning (or inference). The bag-of-words assumption ignores word order. The number of topics K must be pre-specified (unless using HDP-LDA).

26.6 Deep Clustering

26.6.1 Deep Embedded Clustering (DEC)

DEC (Xie et al.

DEC jointly learns a feature embedding $z_i = f_\theta(x_i)$ and a clustering in the embedding space. It first pre-trains an autoencoder, then fine-tunes using a KL-divergence loss between soft assignments q_{ij} and an auxiliary target distribution p_{ij}:
Soft assignment (Student's t-distribution kernel):

$$q_{ij} = \frac{(1 + \|z_i - \mu_j\|^2/\alpha)^{-\frac{\alpha+1}{2}}}{\sum_{j'}(1 + \|z_i - \mu_{j'}\|^2/\alpha)^{-\frac{\alpha+1}{2}}}$$

where α is the degrees of freedom (typically $\alpha = 1$ for the Cauchy kernel).
Target distribution (sharpened, squared, and normalized):

$$p_{ij} = \frac{q_{ij}^2 \,/\, \sum_i q_{ij}}{\sum_{j'} \left(q_{ij'}^2 \,/\, \sum_i q_{ij'} \right)}$$

KL loss:

$$\mathcal{L} = \mathrm{KL}(P \,\|\, Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

This pulls the embedding toward the cluster centroids and sharpens the assignment distribution.

Applications

Clustering high-dimensional data (images, text) where raw features are not amenable to traditional clustering. DEC finds a low-dimensional manifold where clusters are separable. Used for unsupervised image clustering (MNIST, CIFAR, ImageNet).

Trade-offs

Pros:
- Learns features and clusters simultaneously.
- Handles high-dimensional data better than traditional methods.
- The target distribution encourages confident, high-quality clusters.

Cons:
- Sensitive to autoencoder pre-training quality.
- K must be specified.
- The target distribution p_{ij} can cause degenerate solutions (most points assigned to few clusters; mitigated by adding uniform regularization).

- Training can collapse: feature representation may distort to minimize KL loss at the expense of reconstruction fidelity.

26.7 Summary Comparison of Clustering Algorithms

Table 26.1: Comparison of Clustering Algorithms

Algorithm	Param	Shape	Assign.	Scalability	Best For
K-means	K	Spherical	Hard	$O(NKd)$	Large-scale, simple
K-medoids	K	Arbitrary	Hard	$O(N^2)$	Robust, non-Euclidean
Fuzzy C-Means	K, m	Spherical	Soft	$O(NKd)$	Ambiguous boundaries
DBSCAN	ϵ,minPts	Arbitrary	Hard	$O(N \log N)$	Arbitrary shapes, noise
HDBSCAN	min_clust	Arbitrary	Hard	$O(N^2)$	Varying density
Mean-Shift	bandwidth	Arbitrary	Hard	$O(N^2)$	Unknown K
Agglomerative	linkage	Depends	Hard	$O(N^3)$	Hierarchy needed
Spectral	K, γ	Non-convex	Hard	$O(N^3)$	Complex topology
GMM	K,cov	Ellipsoidal	Soft	$O(NKd^2)$	Density estimation
LDA	K, α, η	Mixed	Soft	$O(K^2D)$	Topic modeling
DEC	K (NN)	Learned	Soft	GPU $O(N)$	High-dim features

Clustering Selection Guidelines

- **Start with K-means** if you have many samples, expect spherical clusters, and know K. Use K-means++ initialization.
- **DBSCAN or HDBSCAN** if clusters have arbitrary shapes, density varies, and noise/exceptions exist. HDBSCAN is preferred.
- **Spectral clustering** for non-convex, manifold-like clusters with small-to-medium N.
- **GMM** when you need soft assignments and per-cluster covariance modeling.
- **Agglomerative** when a dendrogram/hierarchy adds value (though HDBSCAN also provides a hierarchy).
- **DEC / deep clustering** for high-dimensional data (images, text) where learned features improve separability.
- Validate K with the elbow method (K-means), silhouette score, gap statistic, or BIC/AIC (GMM).

Chapter 27

Data Augmentation

Data augmentation artificially expands a training dataset by applying label-preserving transformations. It is among the most effective regularization techniques, directly encoding prior knowledge about invariances and dramatically improving generalization, especially in data-scarce regimes.

27.1 Image Geometric Augmentation

27.1.1 Rotation, Crop, Flip, Affine, Perspective

Geometric Transformations

Random Rotation: applied via an affine matrix:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

with $\theta \sim \text{Uniform}(-\theta_{\max}, \theta_{\max})$.
Random Crop: selects a sub-region of target dimensions; a common variant in self-supervised learning resizes to larger dimensions then randomly crops (RandomResizedCrop).
Random Flip: horizontal flip (mirror across vertical axis) and less commonly vertical flip. Horizontal flip is near-universal for natural images.
Random Affine: generalizes rotation with additional translation t_x, t_y, scaling s_x, s_y, and shear ϕ_x, ϕ_y.
Random Perspective: applies a projective (homography) transformation parameterized by random displacement of the four corners.

Applications

Standard augmentations in virtually every image classification and object detection pipeline. `torchvision.transforms` provides `RandomRotation`, `RandomResizedCrop`, `RandomHorizontalFlip`, `RandomAffine`, and `RandomPerspective`.

Trade-offs

Pros: Encodes natural invariances (objects maintain identity under rotation, translation). Computationally cheap. Universally applicable. **Cons:** Excessive rotation can create unrealistic orientations. Vertical flip is unnatural for most objects. Stride effects from rotation can introduce aliasing artifacts.

27.1.2 Elastic Deformation and Thin Plate Spline

Elastic Deformation

Elastic deformation simulates non-rigid transformations by applying a smooth random displacement field to each pixel:

$$\tilde{x}_{i,j} = x\big(i + \Delta_i(i,j),\ j + \Delta_j(i,j)\big)$$

where Δ_i, Δ_j are generated by convolving a random field (typically $\sim \mathcal{N}(0, \sigma^2)$) with a Gaussian kernel of standard deviation σ_{smooth}. The deformation strength is controlled by α (scale of the displacement field).

Thin Plate Spline (TPS)

TPS generates a smooth non-rigid warping by defining a set of control points that are randomly displaced. The deformation $f(x,y) = a_1 + a_x x + a_y y + \sum_{i=1}^{n} w_i\, U(\|(x,y) - c_i\|)$ optimizes bending energy $\iint (f_{xx}^2 + 2f_{xy}^2 + f_{yy}^2)\, dx\, dy$ subject to matching the displaced control points. $U(r) = r^2 \log r$ in 2D.

Applications

Handwriting recognition (originally for MNIST digit distortion). Medical image augmentation (simulating anatomical variation). Data augmentation for segmentation where object boundaries should be deformed naturally.

Trade-offs

Pros: Produces highly realistic non-rigid deformations. TPS gives fine control via control points. **Cons:** Computationally more expensive than simple geometric ops. Hyperparameters (σ, α, control point density) are domain-dependent. Can create unrealistic deformations if parameters are too aggressive.

27.1.3 Random Erasing / Cutout

Random Erasing and Cutout

Random Erasing selects a rectangular region of random size and aspect ratio and fills it with either zero, random noise, or the dataset mean:

$$\tilde{x}_{i,j} = \begin{cases} v & \text{if } (i,j) \in R \\ x_{i,j} & \text{otherwise} \end{cases}$$

The region R has area ratio $S \sim \text{Uniform}(s_{\min}, s_{\max})$ and aspect ratio $r \sim \text{Uniform}(r_1, r_2)$. Cutout is a special case: a fixed-size square region filled with zero or mean. Both simulate occlusion, forcing the model to attend to the full object rather than a single discriminative part.

Applications

Image classification (CIFAR, ImageNet). Person re-identification (simulating real-world occlusion). Improves robustness when the object of interest may be partially occluded at test time.

Trade-offs

Pros: Simple; no label mixing. Improves occlusion robustness. Effective for small-to-medium datasets. **Cons:** Can erase the entire object if region is too large. The information in the erased region is completely lost (unlike CutMix which replaces it).

27.2 Image Color Augmentation

27.2.1 Color Jitter and Intensity Transforms

Color Jitter

Random perturbation of brightness, contrast, saturation, and hue:

- **Brightness:** $\tilde{x} = x + \delta_b$, $\delta_b \sim \text{Uniform}(-b, b)$.
- **Contrast:** $\tilde{x} = \mu + \alpha_c(x - \mu)$, $\alpha_c \sim \text{Uniform}(1 - c, 1 + c)$.
- **Saturation:** convert to HSV, scale S channel, convert back.
- **Hue:** rotate H channel by $\delta_h \sim \text{Uniform}(-h, h)$.

Solarize: Invert all pixel values above a threshold: $\tilde{x}_p = x_p$ if $x_p < T$ else $1 - x_p$.
Posterize: Reduce bit depth: $\tilde{x} = \lfloor x \cdot (2^b - 1) \rceil / (2^b - 1)$, where b is the reduced bit count.
AutoContrast: Remap pixel values so that the darkest becomes 0 and the brightest becomes $V_{\max}$.
Equalize: Apply histogram equalization so the intensity distribution is approximately uniform.

Applications

Core components of RandAugment, AutoAugment, and TrivialAugment. Used in all modern image classification training pipelines. Available in `torchvision.transforms` (`ColorJitter`, `Solarize`, etc.).

Trade-offs

Pros: Encodes invariance to lighting, camera settings, and color variations. Cheap to compute. Strongly beneficial when training data has limited illumination diversity. **Cons:** Hue rotation on colorful natural objects can create unrealistic colors. Posterize/solarize are aggressive and can destroy visual information.

27.2.2 PCA Color Augmentation

PCA Color Augmentation (AlexNet)

Introduced in AlexNet (Krizhevsky et al., 2012). Perform PCA on the 3×3 covariance matrix of RGB pixel values across the training set. For each image, add multiples of the principal components:

$$[p_1, p_2, p_3] \cdot [\alpha_1\lambda_1,\ \alpha_2\lambda_2,\ \alpha_3\lambda_3]^T$$

where p_i and λ_i are the ith eigenvector and eigenvalue, and $\alpha_i \sim \mathcal{N}(0, \sigma^2)$ (typically $\sigma = 0.1$). This captures the main directions of color variation in the dataset, producing realistic color shifts.

Applications

Historical importance in the original AlexNet training on ImageNet. Captures dataset-specific color variation more faithfully than generic color jitter. Still referenced in comparisons of augmentation strategies.

Trade-offs

Pros: Dataset-adaptive: color shifts respect the true covariance structure. Simple parameter (σ) controls magnitude. **Cons:** Requires computing PCA on the full training set (one-time cost). Assumes additive perturbations in RGB space are sufficient. Largely superseded by modern policy-based methods that discover better augmentations.

27.3 Policy-Based Augmentation

27.3.1 AutoAugment

AutoAugment (Cubuk et al.

AutoAugment treats augmentation policy search as a reinforcement learning problem. A controller (RNN) samples augmentation policies — each policy consists of 5 sub-policies, each sub-policy specifies 2 operations with their probabilities and magnitudes. The controller is trained to maximize the validation accuracy of a child model trained with the sampled policies. The final policy is the best-performing set of sub-policies discovered. Operations include shear, translate, rotate, autoContrast, invert, equalize, solarize, posterize, color, brightness, sharpness, etc.

Applications

Image classification (ImageNet, CIFAR, SVHN). AutoAugment discovered policies are publicly available and can be applied as a fixed set of transformations without re-running the search. Provides strong out-of-the-box augmentation.

Trade-offs

Pros:

- State-of-the-art performance when introduced.
- Learned policies transfer across datasets (CIFAR policy works on SVHN, ImageNet).
- Automates augmentation design.

Cons:

- Computationally expensive: policy search requires training thousands of child models.
- The search space is discrete and the RL training is unstable.
- Policies are fixed after discovery; they don't adapt during training.

27.3.2 RandAugment

RandAugment (Cubuk et al.

RandAugment simplifies augmentation policy search to just two hyperparameters: the number of augmentation operations N and a global magnitude M for all operations.

1. Sample N operations uniformly at random from a pre-defined set of 14 operations.
2. Apply each with the same magnitude M (scaled appropriately for each operation type).
3. No per-operation probability parameter; every selected operation is always applied.

The optimal (N, M) is found via a simple grid search over the validation set. This reduces the search space from 10^{32} (AutoAugment) to a 2D grid.

Applications

The default augmentation strategy for many modern image classification pipelines (EfficientNet, ViT training). Far simpler to tune than AutoAugment while matching or exceeding its performance.

Trade-offs

Pros:

- Only two hyperparameters (N, M); grid search is cheap.
- Matches or exceeds AutoAugment performance.
- No separate search phase; parameters are tuned alongside regular hyperparameter tuning.

Cons:

- The optimal (N, M) depends on model size and dataset size (larger models / more data $\rightarrow$ more aggressive augmentation).
- Random selection of operations adds stochasticity beyond the transformations themselves.

27.3.3 TrivialAugment

TrivialAugment (Müller & Hutter

Even simpler than RandAugment: a *single* augmentation operation is sampled uniformly and applied to each image. The magnitude M is fixed (no tuning needed: M is set to a strong default). This removes *all* augmentation hyperparameters while achieving results competitive with AutoAugment and RandAugment.

$$\tilde{x} = \mathrm{Op}_{\mathrm{sampled}}(x;\ M_{\mathrm{fixed}})$$

The insight: most of the benefit comes from the diversity and strength of individual augmentations, not from combining multiple.

Applications

Trivial-to-use augmentation that requires zero hyperparameter tuning. Ideal as a strong baseline for new datasets or when hyperparameter budget is limited.

Trade-offs

Pros: No hyperparameters. Performance competitive with tuned RandAugment. Extremely simple implementation. **Cons:** Single-operation per image may be less diverse than multi-op approaches for very large datasets. The fixed magnitude M is dataset-agnostic.

27.3.4 AugMix

AugMix (Hendrycks et al.

AugMix mixes multiple augmented versions of the same image to improve robustness and uncertainty calibration:

1. Sample k augmentation chains (each chain applies 1–3 random operations sequentially).
2. Mix the augmented images with the original via convex combination with random Dirichlet weights:

$$\tilde{x} = \sum_{i=1}^{k} w_i \, \text{AugChain}_i(x) + w_0 \, x, \qquad \mathbf{w} \sim \text{Dirichlet}(\alpha, \ldots, \alpha)$$

3. Add a Jensen-Shannon divergence consistency loss between the model's predictions on the original, the mixed, and two additional augmented versions.

Applications

Improving robustness to common corruptions (weather, noise, blur) and model calibration. Used when both accuracy and uncertainty quality are important (e.g., safety-critical applications).

Trade-offs

Pros: Significantly improves corruption robustness and calibration while maintaining clean accuracy. The JS consistency loss is a principled regularizer. **Cons:** More complex than RandAugment. Requires multiple forward passes per image (computational overhead at training).

27.4 Mixing-Based Augmentation

27.4.1 Mixup

Mixup (Zhang et al.

Mixup creates virtual training examples via convex combination of two random examples and their labels:

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j, \qquad \tilde{y} = \lambda y_i + (1 - \lambda) y_j$$

where $\lambda \sim \text{Beta}(\alpha, \alpha)$ with $\alpha > 0$. Small α (e.g., 0.2) concentrates λ near 0 or 1 (mild interpolation); $\alpha = 1$ gives uniform λ (all interpolations equally likely). Mixup encourages the model to behave linearly between training examples, resulting in smoother decision boundaries.

Applications

Standard in modern image classification training. Extended to NLP (word and sentence Mixup, embedding Mixup), speech recognition, and graph learning (Manifold Mixup). Works with any data modality where linear interpolation in input space is meaningful.

Trade-offs

Pros:

- Simple: a few lines of code.
- Improves generalization, adversarial robustness, and calibration.
- Reduces overconfidence and memorization of noisy labels.
- Applicable beyond images (tabular, text, speech).

Cons:

- Interpolated labels may not be semantically meaningful.
- Requires α tuning (common: $\alpha = 0.2$–1.0).
- Slower convergence: the model sees "unreal" data.
- Only works with one-hot or soft labels.

27.4.2 CutMix

CutMix (Yun et al.

CutMix replaces a rectangular patch of image A with the corresponding patch from image B. The label is mixed proportionally to the area of the patch:

$$\tilde{x} = \mathbf{M} \odot x_A + (\mathbf{1} - \mathbf{M}) \odot x_B, \qquad \tilde{y} = \lambda y_A + (1 - \lambda) y_B$$

where $\mathbf{M} \in \{0, 1\}^{H \times W}$ is a binary mask and $\lambda = |\mathbf{M}|/HW$ is the area fraction. The bounding box coordinates (r_x, r_y, r_w, r_h) are sampled as:

$$r_x \sim \text{Uniform}(0, W), \quad r_y \sim \text{Uniform}(0, H), \quad r_w = W\sqrt{1-\lambda}, \quad r_h = H\sqrt{1-\lambda}$$

with $\lambda \sim \text{Beta}(\alpha, \alpha)$.

Applications

Image classification, object detection, and weakly-supervised localization. Often used together with or as an alternative to Mixup. Encourages the model to attend to the full object rather than the most discriminative part.

Trade-offs

Pros:

- Spatial mixing is more natural than pixel-wise interpolation.
- Improves localization and CAM (Class Activation Map) coverage.
- The area-proportional label weighting is interpretable.

Cons:

- The randomly placed patch can cover the entire object, creating a misleading signal.
- Introduces boundary artifacts at the cut edge.
- Hyperparameters: α for Beta and the bounding box strategy.

27.4.3 Manifold Mixup, FMix, and Other Mixing Variants

Manifold Mixup (Verma et al.

Instead of mixing in the input (pixel) space, Manifold Mixup performs the convex combination in a hidden layer's representation:

$$\tilde{h} = \lambda \cdot h_\ell(x_i) + (1 - \lambda) \cdot h_\ell(x_j)$$

where $h_\ell(\cdot)$ is the output of layer ℓ, and the remainder of the network processes $\tilde{h}$. The layer ℓ is randomly sampled per batch. This learns flatter representations and produces more diverse augmented signals.

FMix (Harris et al.

FMix uses a continuous, smooth mask generated from a random low-frequency Fourier image instead of a hard rectangular patch:

$$\mathbf{M} = \text{threshold}\big(\text{IFT}(Z(\omega) \cdot \text{filter}(\omega)),\ \tau\big)$$

where $Z(\omega) \sim \mathcal{CN}(0, 1)$ is complex Gaussian noise and the filter function decays with frequency: $\text{filter}(\omega) = (1 + |\omega|)^{-(\delta/2)}$. This produces organic, blob-shaped masks without sharp boundaries, avoiding CutMix's boundary artifacts.

Puzzle Mix and Co-Mixup

Puzzle Mix (Kim et al., 2020): optimizes the mixing mask and transport plan jointly to maximize both saliency-based information and local smoothness. **Co-Mixup** (Kim et al., 2021): extends Mixup to mix multiple ($m > 2$) images simultaneously, creating richer combinations.

Applications

Manifold Mixup: regularization for any deep network; effective with normalization layers. FMix: alternative to CutMix with smoother masks; better for segmentation. Puzzle Mix / Co-Mixup: state-of-the-art mixing when computation budget allows.

Trade-offs

Manifold Mixup: Produces more diverse hidden representations; requires sampling the mix layer. **FMix:** Smoother masks than CutMix; additional Fourier computation. **Puzzle Mix:** SOTA performance but expensive optimization per batch.

27.5 Text Augmentation

27.5.1 Lexical and Syntactic Augmentation

Text Augmentation Methods

- **Back-translation:** translate text to an intermediate language and back: $\tilde{x} = T_{\text{target}\to\text{src}}(T_{\text{src}\to\text{target}}(x))$. Produces paraphrased text with preserved meaning.
- **Synonym replacement:** randomly select n non-stop words and replace each with a synonym from WordNet or a distributional thesaurus.
- **Random insertion:** insert a synonym of a randomly chosen word at a random position.
- **Random deletion:** delete each word with probability p.
- **Random swap:** randomly swap positions of two words.
- **EDA (Easy Data Augmentation):** combines all four (replacement, insertion, deletion, swap) with per-operation probabilities.

Applications

NLP classification (sentiment analysis, topic classification) when labeled data is scarce. EDA is a simple, effective baseline. Back-translation is the strongest but requires a translation model.

Trade-offs

Pros: Simple lexical augmentations improve robustness to phrasing variation. Back-translation produces fluent, diverse paraphrases. **Cons:** Synonym replacement can change meaning. Random deletion/swap can destroy grammaticality. Back-translation is slow and can introduce translation errors.

27.5.2 Model-Based Text Augmentation

MLM-Based Augmentation and Paraphrasing

MLM-based filling: randomly mask tokens and use a masked language model (BERT, RoBERTa) to fill them. For example, mask 15% of tokens and predict replacements from the MLM distribution:

$$\tilde{w}_i \sim p_{\text{MLM}}(\cdot \mid \tilde{x}_{\setminus i})$$

Paraphrasing with generative models: use a sequence-to-sequence model (T5, BART, GPT) fine-tuned for paraphrasing to generate diverse rephrasings while preserving semantics. Back-translation is a special case using translation models.

Applications

Text classification, NLI, and question answering with limited labeled data. Generates more diverse and fluent augmentations than lexical methods.

Trade-offs

Pros: Produces semantically coherent augmentations. MLM filling respects context better than synonym replacement. **Cons:** Requires a pre-trained language model. Slower than lexical methods. MLM filling can produce fluent but semantically-drifted text.

27.6 Audio Augmentation

27.6.1 Time and Frequency Domain Augmentation

Audio Augmentation Operations

- **Time stretch:** change the speed/duration without affecting pitch. Resample the signal at a different rate.
- **Pitch shift:** shift the pitch (fundamental frequency) up or down by n semitones while preserving duration.
- **Volume/gain:** multiply the waveform by $g \sim \text{Uniform}(g_{\min}, g_{\max})$ (usually in dB scale).
- **Noise injection:** add Gaussian noise, background noise, or environmental sounds at a controlled SNR.
- **Room Impulse Response (RIR):** convolve audio with an impulse response to simulate different acoustic environments (rooms, halls, outdoors).
- **Speed perturbation:** resample the audio at a slightly different rate (e.g., 0.9× or 1.1×), changing both duration and pitch.

Applications

Speech recognition, speaker verification, audio classification, music information retrieval. Speed perturbation and volume augmentation are standard in ASR (automatic speech recognition) training.

Trade-offs

Pros: Encodes natural acoustic variability. Significantly improves ASR robustness. RIR augmentation simulates diverse deployment conditions. **Cons:** Extreme pitch shift or time stretch produces unnatural sounds. RIR convolution requires collecting impulse response datasets. Real-time augmentation can be a pipeline bottleneck.

27.6.2 SpecAugment

SpecAugment (Park et al.

SpecAugment applies augmentation directly to the log-mel spectrogram rather than the raw waveform:

- **Time warping:** apply a random warp to the time axis using a sparse warp function parameterized by a warp parameter W.
- **Frequency masking:** mask f consecutive mel frequency channels: set columns $[f_0, f_0 + f)$ to zero, where $f \sim \text{Uniform}(0, F)$, $f_0 \sim \text{Uniform}(0, \nu - f)$.
- **Time masking:** mask t consecutive time steps: set rows $[t_0, t_0 + t)$ to zero, where $t \sim \text{Uniform}(0, T)$, $t_0 \sim \text{Uniform}(0, \tau - t)$. Multiple time and frequency masks are applied per spectrogram.

This removes chunks of the time-frequency representation, forcing the model to rely on non-masked regions and learn robust features. Typical settings: $F = 27$, $T = 100$ for LibriSpeech; adapt for domain.

Applications

De facto standard augmentation for end-to-end speech recognition (LAS, RNN-T, Transformer, Conformer). Also used for audio classification and speaker diarization. Zero additional data or augmentation models required.

Trade-offs

Pros:

- Extremely effective: dramatically improves ASR WER, especially on noisy/test data.
- Simple and fast; operates directly on spectrograms.
- Works synergistically with other augmentations (speed perturbation).
- Interpretable: time masking simulates time-domain distortions; frequency masking simulates channel dropouts.

Cons:

- Time warping is computationally expensive and sometimes omitted.
- Masking too many bands/time steps can destroy critical phonetic information.
- Hyperparameters F, T and number of masks require tuning per dataset/task.

27.7 Generative Augmentation

27.7.1 GAN-Based and VAE-Based Augmentation

DAGAN and Generative Augmentation

DAGAN (Data Augmentation GAN): a conditional GAN trained to generate samples from a few examples of a rare class. The generator takes a random noise vector z and a class condition to synthesize new examples for data-scarce classes.
VAE augmentation: train a VAE on the data and sample from the learned latent space: $z \sim \mathcal{N}(0, I)$, $\tilde{x} = \text{Decoder}(z)$. Interpolation in latent space can produce smooth semantic transformations.
Diffusion augmentation: use a pre-trained diffusion model to generate diverse, realistic samples. Can be conditioned on class labels for targeted augmentation. The stochastic generation process naturally produces variations.

Applications

Augmenting rare classes in imbalanced datasets. Generating synthetic training data for medical imaging (where data collection is expensive and privacy-constrained). Text-to-image diffusion models as data factories.

Trade-offs

Pros: Can generate unlimited, novel, realistic samples. Valuable for extremely data-scarce domains. Diffusion models produce near photorealistic quality. **Cons:** Requires training a generative model (expensive). Generated data may include artifacts or biases. Mode collapse (GANs) limits diversity. Synthetic data may not cover real-world tails.

Table 27.1: Comparison of Data Augmentation Methods

Method	Modality	Complexity	Tuning	Key Benefit
Geometric (rotate, flip, crop)	Image	Low	Low	Encodes basic invariances
Color jitter	Image	Low	Low	Lighting invariance
PCA Color Aug.	Image	Medium	Low	Dataset-adaptive color shifts
Cutout / Random Erasing	Image	Low	Low	Occlusion robustness
Elastic / TPS	Image	Medium	Medium	Non-rigid deformation
AutoAugment	Image	High (search)	Medium	Automated policy discovery
RandAugment	Image	Low	Low (N, M)	Simple, strong baseline
TrivialAugment	Image	Low	None	Zero-hyperparameter
AugMix	Image	Medium	Low	Corruption robustness
Mixup	Image, text	Low	Low (α)	Smoother decision boundaries
CutMix	Image	Low	Low (α)	Spatial mixing; localization
Manifold Mixup	Any	Low	Low	Mixing in feature space
FMix	Image	Medium	Low	Smooth frequency masks
Lexical (EDA)	Text	Low	Low	Simple, fast
Back-translation	Text	High (model)	None	Fluent paraphrases
MLM filling	Text	Medium	Low	Context-aware replacement
Speed / pitch / volume	Audio	Low	Low	Acoustic variability
SpecAugment	Audio (spectro.)	Low	Medium	Masks time-freq. regions
RIR convolution	Audio	Medium	Low	Environment simulation
GAN / VAE augmentation	Image, audio	High (training)	High	Unlimited synthetic data
Diffusion augmentation	Image	High (training)	High	Photorealistic diversity

27.8 Summary Comparison of Augmentation Methods

Augmentation Selection Guidelines

- **Image classification:** start with RandAugment ($N = 2, M = 9$) or TrivialAugment. Add CutMix and/or Mixup with $\alpha \approx 0.2$–0.8.
- **Object detection / segmentation:** geometric augmentations (flip, scale, crop) are essential. Strong color jitter. Mosaic augmentation (YOLO) combines 4 images.
- **NLP:** EDA for small data; back-translation for larger budgets. MLM filling for contextual augmentations.
- **Speech/audio:** SpecAugment + speed perturbation is the standard combination. Add RIR for robustness to environments.
- **Data-scarce / imbalanced:** generative augmentation (GAN, diffusion) if the domain allows training a high-quality generator.

- Less is often more: too many or too strong augmentations can hurt convergence. Tune magnitude via validation accuracy.

Chapter 28

Uncertainty Quantification

Uncertainty quantification (UQ) is essential for deploying machine learning models in safety-critical applications. This chapter covers the decomposition of predictive uncertainty into aleatoric and epistemic components, and surveys methods for estimating, calibrating, and leveraging uncertainty — from ensembles and Bayesian approximations to conformal prediction and OOD detection.

28.1 Types of Uncertainty

28.1.1 Aleatoric vs. Epistemic Uncertainty

Aleatoric Uncertainty

Aleatoric uncertainty (data uncertainty) is the irreducible noise inherent in the data-generating process. It cannot be reduced by collecting more training data.

- **Homoscedastic:** constant across all inputs — $y = f(x) + \epsilon,\ \epsilon \sim \mathcal{N}(0, \sigma^2)$ with fixed σ^2.
- **Heteroscedastic:** input-dependent noise — $y = f(x) + \epsilon(x),\ \epsilon(x) \sim \mathcal{N}(0, \sigma^2(x))$ where $\sigma^2(x)$ varies with x. Examples: depth estimation (far objects have higher uncertainty), sensor noise that depends on signal strength.

Epistemic Uncertainty

Epistemic uncertainty (model uncertainty) arises from limited knowledge about the optimal model parameters. It is *reducible* with more data: as $N \to \infty$, the posterior over parameters concentrates and epistemic uncertainty vanishes. Epistemic uncertainty is high in regions far from training data (OOD inputs) and for models with high capacity relative to data size.

Applications

Distinguishing aleatoric from epistemic uncertainty informs decision-making: high epistemic uncertainty in a region suggests collecting more data there (active learning); high aleatoric uncertainty suggests the task is inherently noisy and the model is already near-optimal.

28.1.2 Total Predictive Variance Decomposition

Law of Total Variance

The total variance of a Bayesian model's prediction y^* at input x^* decomposes as:

$$\operatorname{Var}(y^* \mid x^*) = \underbrace{\mathbb{E}_{\theta \mid \mathcal{D}}\big[\operatorname{Var}(y^* \mid x^*, \theta)\big]}_{\text{Aleatoric (data) uncertainty}} + \underbrace{\operatorname{Var}_{\theta \mid \mathcal{D}}\big[\mathbb{E}[y^* \mid x^*, \theta]\big]}_{\text{Epistemic (model) uncertainty}}$$

For a regression model outputting mean $\mu_\theta(x)$ and variance $\sigma^2_\theta(x)$ (heteroscedastic), and using Monte Carlo samples $\{\theta_t\}_{t=1}^T$ from the posterior:

$$\begin{aligned}\text{Aleatoric:} &\quad \frac{1}{T}\sum_{t=1}^{T}\sigma^2_{\theta_t}(x^*)\\ \text{Epistemic:} &\quad \frac{1}{T}\sum_{t=1}^{T}\mu_{\theta_t}(x^*)^2 - \left(\frac{1}{T}\sum_{t=1}^{T}\mu_{\theta_t}(x^*)\right)^2\\ \text{Total:} &\quad \text{Aleatoric} + \text{Epistemic}\end{aligned}$$

Classification Uncertainty Decomposition

For classification, the total predictive entropy decomposes using mutual information:

$$\mathcal{H}[p(y \mid x^*, \mathcal{D})] = \underbrace{\mathbb{E}_\theta[\mathcal{H}[p(y \mid x^*, \theta)]]}_{\text{Aleatoric}} + \underbrace{\mathcal{I}[y; \theta \mid x^*, \mathcal{D}]}_{\text{Epistemic}}$$

The mutual information (BALD — Bayesian Active Learning by Disagreement) quantifies epistemic uncertainty as the information gain about parameters from observing the label.

28.2 Ensemble Methods for Uncertainty

28.2.1 Deep Ensembles

Deep Ensembles (Lakshminarayanan et al.

Train M copies of the same architecture with different random initializations (and optionally different data shuffles) using a proper scoring rule (e.g., negative log-likelihood). The ensemble predictive distribution is an equally-weighted mixture:

$$p(y \mid x) = \frac{1}{M}\sum_{m=1}^{M} p(y \mid x, \theta_m)$$

For regression, the predictive mean and variance are:

$$\begin{aligned}\hat{\mu}(x) &= \frac{1}{M}\sum_{m=1}^{M}\mu_{\theta_m}(x)\\ \hat{\sigma}^2(x) &= \frac{1}{M}\sum_{m=1}^{M}\sigma^2_{\theta_m}(x) + \frac{1}{M}\sum_{m=1}^{M}\mu^2_{\theta_m}(x) - \hat{\mu}(x)^2\end{aligned}$$

where the first term is aleatoric and the second is epistemic (disagreement across ensemble members).

Applications

State-of-the-art uncertainty estimation in computer vision, medical imaging, and reinforcement learning. Deep ensembles consistently outperform single Bayesian neural networks for both predictive accuracy and uncertainty calibration.

Trade-offs

Pros:
- State-of-the-art uncertainty quality and calibration.
- Simple to implement (no Bayesian math required).
- Each member can be trained independently in parallel.
- Naturally captures multi-modality in function space.

Cons:
- $M\times$ training and memory cost.
- $M\times$ inference cost (though each member can run on a separate device).
- Lacks a formal Bayesian interpretation (ensembles approximate the posterior only indirectly through random initialization).
- The number of ensemble members M must be chosen; diminishing returns typically beyond $M = 5$–10.

28.2.2 MC Dropout

MC Dropout for Uncertainty

Perform T stochastic forward passes with dropout enabled at test time. Each pass uses a different dropout mask, effectively sampling from an approximate posterior over network weights:

$$p(y \mid x) \approx \frac{1}{T}\sum_{t=1}^{T} p(y \mid x, \hat{W}_t)$$

$$\mathrm{Var}[y \mid x] \approx \frac{1}{T}\sum_{t=1}^{T} f(x;\hat{W}_t)^2 - \left(\frac{1}{T}\sum_{t=1}^{T} f(x;\hat{W}_t)\right)^2 + \tau^{-1}$$

where $\hat{W}_t$ is a weight sample via dropout and τ is the model precision (observation noise). The first two terms capture epistemic uncertainty (variance across dropout masks); the last term captures homoscedastic aleatoric noise.

Applications

Lightweight uncertainty for any existing dropout-trained network. Widely used in active learning, Bayesian optimization with neural networks, and as a baseline in UQ benchmarking.

Trade-offs

Pros: No retraining; applies to existing models. T can be tuned for speed vs. accuracy. **Cons:** Calibration is often worse than deep ensembles. The dropout rate p regulates model uncertainty in an uncontrolled way. Uncertainty saturates for OOD inputs.

28.2.3 SWAG and Laplace for Uncertainty

SWAG Uncertainty

SWAG (see Chapter 25) approximates the posterior as $\mathcal{N}(\bar{\theta}, \frac{1}{2}(\Sigma_{\text{diag}} + \Sigma_{\text{low-rank}}))$. Predictive uncertainty is obtained by sampling T weight vectors from this Gaussian and computing the empirical mean and variance of predictions.

Laplace Approximation Uncertainty

Post-hoc Laplace (KFLA) fits a Gaussian posterior at the MAP estimate: $q(\theta) = \mathcal{N}(\theta_{\text{MAP}}, \mathbf{H}^{-1})$. For a linearized predictive (the network is linearized around the MAP), the predictive distribution is:

$$p(y^* \mid x^*, \mathcal{D}) \approx \mathcal{N}\big(f_{\theta_{\text{MAP}}}(x^*),\ J(x^*)^T \mathbf{H}^{-1} J(x^*) + \sigma^2_{\text{noise}}\big)$$

where $J(x^*) = \nabla_\theta f_\theta(x^*)\big|_{\theta_{\text{MAP}}}$ is the Jacobian. The term $J^T\mathbf{H}^{-1}J$ captures epistemic uncertainty propagated through the network.

Applications

Single-training-run alternatives to deep ensembles. SWAG is stronger empirically; Laplace provides a principled post-hoc UQ method.

Trade-offs

SWAG: Single training run; good calibration; stores checkpoints. **Laplace:** True post-hoc; the linearized predictive is a strong approximation; Hessian computation and storage can be burdensome.

28.3 Bayesian Methods

Cross-Reference

Bayes by Backprop, Flipout, SGLD, and other Bayesian neural network methods are covered in detail in Chapter 25, Sections 4–5. These methods provide full posterior approximations over weights, naturally yielding both aleatoric and epistemic uncertainty through the total variance decomposition. In practice, deep ensembles often outperform them in calibration metrics despite being non-Bayesian, highlighting a gap between theoretical elegance and empirical performance.

28.4 Single Network Methods

28.4.1 Heteroscedastic Regression

Heteroscedastic Neural Network

Instead of outputting only a mean prediction, the network outputs both $\mu(x)$ and $\sigma^2(x)$:

$$[\mu(x),\ \sigma^2(x)] = f_\theta(x), \qquad \sigma^2(x) = \text{softplus}(\text{logvar}(x))$$

The negative log-likelihood (NLL) loss under a Gaussian assumption is:

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left[\frac{(y_i - \mu(x_i))^2}{2\sigma^2(x_i)} + \frac{1}{2}\log\sigma^2(x_i)\right]$$

When the prediction error is large, the variance term grows (reducing the first term's contribution), and the $\log\sigma^2$ penalty prevents σ^2 from diverging to infinity. For inputs with high inherent noise, the model learns to output large $\sigma^2(x)$; for clean, well-represented inputs, $\sigma^2(x)$ is small.

Applications

Regression tasks with input-dependent noise: depth estimation, time-series forecasting with volatile regimes, sensor fusion, and any problem where prediction reliability varies with the input.

Trade-offs

Pros: Single forward pass; captures input-dependent aleatoric uncertainty. Simple to implement with any regression architecture. **Cons:** Does not capture epistemic uncertainty (model uncertainty about the mean). The Gaussian assumption may not hold. The variance head can be poorly calibrated; often requires a separate calibration step.

28.4.2 Deep Evidential Regression

Deep Evidential Regression (Amini et al.

Instead of outputting $\mu(x), \sigma^2(x)$ and minimizing NLL, deep evidential regression places an evidential prior (Normal-Inverse-Gamma, NIG) over the likelihood parameters:

$$p(y \mid m) = \mathcal{N}(y \mid \gamma,\ \sigma^2), \qquad p(\gamma, \sigma^2 \mid \Phi) = \mathrm{NIG}(\gamma, \nu, \alpha, \beta)$$

The network outputs four values for each input:

$$\Phi(x) = (\gamma(x),\ \nu(x),\ \alpha(x),\ \beta(x))$$

where γ is the predicted mean, ν is a virtual observation count (epistemic uncertainty proxy), α and β control the inverse-gamma over σ^2 (aleatoric uncertainty proxy). The loss maximizes the marginal likelihood (Student-t):

$$\mathcal{L}_{\mathrm{NLL}} = -\log \mathrm{St}\Big(y \mid \gamma,\ \frac{\beta(1+\nu)}{\nu\alpha},\ 2\alpha\Big)$$

plus a regularization term that penalizes evidence on errors: $\mathcal{L}_R = |y - \gamma| \cdot (2\nu + \alpha)$. The total evidence Φ is the sum of virtual observations.

Applications

Regression with both aleatoric and epistemic uncertainty from a single deterministic forward pass. Active learning, OOD detection, and safety-critical prediction where distinguishing uncertainty types matters.

Trade-offs

Pros: Single network; provides both aleatoric and epistemic uncertainty. Theoretically grounded in evidential reasoning. No sampling at inference time. **Cons:** Training can be unstable (the evidential regularizer is sensitive). The NIG prior is a convenience choice; may not reflect true prior knowledge. Can be overconfident on OOD data compared to ensemble methods.

28.5 Conformal Prediction

28.5.1 Split Conformal Prediction

Split Conformal Prediction

Conformal prediction produces prediction sets with finite-sample, distribution-free coverage guarantees:

1. Split the training data: proper training set $\mathcal{D}_{\text{train}}$ and calibration set $\mathcal{D}_{\text{cal}}$.
2. Fit model $\hat{f}$ on $\mathcal{D}_{\text{train}}$.
3. Compute nonconformity scores on the calibration set. For regression: $s_i = |y_i - \hat{f}(x_i)|$ (absolute residual). For classification: $s_i = 1 - \hat{p}(y_i \mid x_i)$ (one minus softmax of true class).
4. Determine the $(1-\alpha)$-quantile of calibration scores:

$$\hat{q} = \text{quantile}\big(\{s_i\}_{i \in \mathcal{D}_{\text{cal}}} \cup \{\infty\},\ \frac{\lceil (|\mathcal{D}_{\text{cal}}|+1)(1-\alpha) \rceil}{|\mathcal{D}_{\text{cal}}|}\big)$$

5. For a test point x_{test}, the prediction set is:

$$\hat{C}(x_{\text{test}}) = \{y : s(x_{\text{test}}, y) \le \hat{q}\}$$

For regression: $\hat{C}(x_{\text{test}}) = [\hat{f}(x_{\text{test}}) - \hat{q},\ \hat{f}(x_{\text{test}}) + \hat{q}]$ (constant-width interval).

Coverage guarantee: If (x_i, y_i) are exchangeable,

$$\mathbb{P}\big(Y_{\text{test}} \in \hat{C}(X_{\text{test}})\big) \ge 1 - \alpha$$

This holds exactly, not asymptotically, for any model $\hat{f}$.

Applications

Safety-critical applications requiring formal coverage guarantees: medical diagnosis, autonomous driving, financial risk assessment. Often used to wrap any black-box predictor with calibrated uncertainty intervals.

Trade-offs

Pros:

- Distribution-free and model-agnostic (works with any predictor).
- Finite-sample coverage guarantee (not asymptotic).
- Simple: a few lines of code on top of any trained model.
- The coverage probability is exact for exchangeable data.

Cons:

- Requires a held-out calibration set, reducing training data.

- Standard conformal intervals have constant width (for absolute residual scores), which ignores heteroscedasticity.
- Marginal coverage ($\mathbb{E}[P(...)]$) is weaker than conditional coverage ($P(... \mid X = x)$).
- Exchangeability assumption: violated for time-series, distribution shift, or when calibration and test distributions differ.

28.5.2 Conformalized Quantile Regression (CQR)

CQR (Romano et al.

CQR produces prediction intervals that adapt to heteroscedasticity by wrapping a quantile regression model. Train models $\hat{q}_{\alpha/2}(x)$ and $\hat{q}_{1-\alpha/2}(x)$ for the lower and upper quantiles. On the calibration set, compute nonconformity scores:

$$s_i = \max\big(\hat{q}_{\alpha/2}(x_i) - y_i,\ y_i - \hat{q}_{1-\alpha/2}(x_i)\big)$$

The score measures how far y_i falls outside the initial quantile interval (negative if inside). The conformalized prediction interval is:

$$\hat{C}(x_{\text{test}}) = \big[\hat{q}_{\alpha/2}(x_{\text{test}}) - \hat{Q}_{1-\alpha},\ \hat{q}_{1-\alpha/2}(x_{\text{test}}) + \hat{Q}_{1-\alpha}\big]$$

where $\hat{Q}_{1-\alpha}$ is the $(1-\alpha)(1+1/|I_{\text{cal}}|)$-quantile of calibration scores. The interval adapts to local variability while maintaining marginal coverage.

Applications

Heteroscedastic regression where prediction intervals should be wider in high-noise regions and narrower in low-noise regions. More powerful than standard split conformal while maintaining coverage guarantees.

Trade-offs

Pros: Adaptive interval width. Retains finite-sample coverage guarantee. **Cons:** Requires training a quantile regression model (instead of a single predictor). Slightly more complex than split conformal. The quantile model quality affects interval tightness (not coverage).

28.6 Calibration Metrics

Cross-Reference

Expected Calibration Error (ECE), reliability diagrams, and temperature scaling are covered in detail in Chapter **??**, Section 29.2.1. We briefly restate the core formula here for completeness.

ECE: Partition predictions into B bins by confidence. For bin b with N_b samples:

$$\text{ECE} = \sum_{b=1}^{B} \frac{N_b}{N} \big|\text{acc}(b) - \text{conf}(b)\big|$$

where $\text{acc}(b)$ is the accuracy within bin b and $\text{conf}(b)$ is the average confidence. Perfect calibration $\implies$ ECE $= 0$.

Reliability diagram: plots $\text{acc}(b)$ vs. $\text{conf}(b)$; the diagonal is perfect calibration. ECE is the weighted gap from the diagonal.
Brier Score (for binary classification): $\text{BS} = \frac{1}{N}\sum_i (p_i - y_i)^2$, the MSE of predicted probabilities. Decomposes into refinement and calibration components.
NLL (Negative Log-Likelihood): a proper scoring rule that jointly evaluates accuracy and calibration quality.

28.7 Out-of-Distribution (OOD) Detection

OOD detection identifies inputs that differ significantly from the training distribution, where model predictions are unreliable.

28.7.1 Maximum Softmax Probability (MSP)

MSP Baseline

The simplest OOD score:

$$\text{score}(x) = \max_c \; p(y = c \mid x) = \max_c \; \text{softmax}(f_\theta(x))_c$$

In-distribution (ID) samples tend to have higher maximum softmax values than OOD samples. A threshold distinguishes them. Despite its simplicity, MSP is a surprisingly strong baseline.

Applications

The default OOD detection baseline. Often outperformed by more sophisticated methods but provides a simple, calibration-free reference.

Trade-offs

Pros: No extra computation (reuses classification output). No hyperparameters. **Cons:** Overconfident predictions on OOD data (because softmax is forced to sum to 1). Degrades for OOD samples that happen to activate a known class strongly.

28.7.2 Energy-Based OOD Detection

Energy Score (Liu et al.

The energy-based score is derived from the log partition function of the softmax:

$$E(x; f) = -T \cdot \log \sum_{c=1}^{C} e^{f_c(x)/T}$$

where $f_c(x)$ are the logits, T is a temperature parameter (typically $T = 1$). Lower energy (more negative) indicates in-distribution; higher energy indicates OOD. This is equivalent to the negative log of the unnormalized density in an energy-based model interpretation.

Applications

State-of-the-art OOD detection for image classification. Outperforms MSP consistently across benchmarks. Fine-tuning with an energy regularization loss further improves separation.

Trade-offs

Pros: Simple (logsumexp of logits). Theoretically grounded in energy-based models. Does not require OOD data during training (though fine-tuning with an OOD-regularizer helps). **Cons:** Temperature T is usually set to 1; tuning can improve separation. The logsumexp can be dominated by the maximum logit, reducing to MSP in the limit $T \to 0$.

28.7.3 Mahalanobis Distance in Feature Space

Mahalanobis OOD Detection (Lee et al.

For each class c, fit a class-conditional Gaussian $\mathcal{N}(\mu_c, \Sigma)$ to the pre-logit features $\{h(x_i) : y_i = c\}$ of the training set:

$$\mu_c = \frac{1}{N_c} \sum_{i:y_i=c} h(x_i), \qquad \Sigma = \frac{1}{N} \sum_{c=1}^{C} \sum_{i:y_i=c} (h(x_i) - \mu_c)(h(x_i) - \mu_c)^T$$

The OOD score for a test point x is the minimum Mahalanobis distance across classes:

$$M(x) = \min_c \big(h(x) - \mu_c\big)^T \Sigma^{-1} \big(h(x) - \mu_c\big)$$

A small amount of input perturbation (adding $\epsilon \cdot \text{sign}(\nabla_x M(x))$) is applied to sharpen the separation between ID and OOD scores.

Applications

Feature-space OOD detection using a pre-trained classifier. The class- conditional Gaussians provide a lightweight generative model of normal features. The perturbation step (similar to ODIN) boosts performance.

Trade-offs

Pros: Uses rich feature representations. Class-conditional modeling captures multi-modal in-distribution data. **Cons:** Requires fitting Gaussians per class (and optionally per layer). The shared Σ is a strong simplification. Feature-space densities may not separate ID/OOD as well as output-space methods.

28.7.4 ODIN

ODIN (Liang et al.

ODIN (Out-of-DIstribution detector for Neural networks) combines temperature scaling and input perturbation:

1. **Temperature scaling:** divide logits by $T > 1$ to soften the softmax:

$$p(y = c \mid x; T) = \frac{e^{f_c(x)/T}}{\sum_j e^{f_j(x)/T}}$$

2. **Input perturbation:** add a small perturbation in the direction that increases the maximum softmax score:

$$\tilde{x} = x - \epsilon \cdot \operatorname{sign}\big(\nabla_x \log p(\hat{y} \mid x; T)\big)$$

3. Compute the OOD score as $\max_c p(y = c \mid \tilde{x}; T)$. ID samples have their scores increased by the perturbation; OOD samples do not, increasing the separation.

Applications

Improved OOD detection using only the output of a standard classifier. The perturbation and temperature are applied only at test time (no retraining). $T \approx 1000$ and $\epsilon \approx 0.0014$ are common defaults.

Trade-offs

Pros: No retraining needed. Significant improvement over MSP. **Cons:** Requires gradient computation for perturbation (extra cost). Hyperparameters (T, ϵ) require tuning on a validation OOD set. The perturbation can be adversarial if ϵ is too large.

28.8 Summary Comparison of Uncertainty Methods

Practical Recommendations

- **Best UQ quality:** Deep Ensembles ($M = 5$–10) with a proper scoring rule. For single-training-run alternatives, use SWAG or KFLA-Laplace.
- **Fast and simple:** MC Dropout with $T = 10$–30 passes if the network already uses dropout.
- **Formal coverage guarantees:** Conformal prediction (split conformal for constant-width; CQR for adaptive intervals).
- **Input-dependent noise:** Heteroscedastic regression for aleatoric uncertainty. Combine with an ensemble for epistemic.
- **OOD detection:** Energy-based score is a strong default. Mahalanobis or ODIN for improved separation at the cost of extra computation.
- **Always calibrate:** Apply temperature scaling (Platt scaling) on a calibration set. Report ECE and reliability diagrams alongside accuracy.
- Remember: aleatoric uncertainty *cannot* be reduced with more data; epistemic uncertainty *can*. Use this distinction to guide data collection (active learning targets high-epistemic regions).

Table 28.1: Comparison of Uncertainty Quantification Methods

Method	**Uncertainty**	**Samples**	**Training**	**Calibration**
Deep Ensembles	Both	M fwd.	$M\times$ cost	Excellent
MC Dropout	Both	T fwd.	Standard	Moderate
Bayes by Backprop	Both	T fwd.	$2\times$ params	Moderate
Laplace (KFLA)	Both	1 fwd. + Jacobian	Post-hoc	Moderate
SWAG	Both	T fwd.	Single run	Good
Heteroscedastic	Aleatoric only	1 fwd.	Standard	Needs post-cal.
Deep Evidential	Both	1 fwd.	Standard (special loss)	Moderate
Split Conformal	Set/Interval	1 fwd.	Standard + calib.	Guaranteed coverage
CQR	Adaptive sets	1 fwd.	Quantile + calib.	Guaranteed coverage
MSP (OOD)	Score	1 fwd.	None	N/A
Energy (OOD)	Score	1 fwd.	None	N/A
Mahalanobis (OOD)	Score	1 fwd.	Fit Gaussians	N/A
ODIN (OOD)	Score	1 fwd. + grad	None	N/A

Chapter 29

Calibration Methods

A probabilistic classifier is **well-calibrated** if its predicted confidence matches its actual accuracy. For example, among all predictions assigned probability p, exactly a fraction p should be correct. Modern deep networks tend to be overconfident, making calibration a critical post-processing step for any application where probability estimates matter.

29.1 Why Calibration Matters

Calibration Condition

A model is perfectly calibrated if for every confidence level $p \in [0, 1]$:

$$\mathbb{P}\big(\hat{Y} = Y \mid \hat{P} = p\big) = p$$

where $\hat{P}$ is the predicted probability for the predicted class $\hat{Y}$. Equivalently, the empirical accuracy in each confidence bin equals the bin center.

When Calibration Matters

Calibration is crucial in:

- **Medical diagnosis:** A 90% confidence must genuinely mean 90% accuracy for treatment decisions.
- **Autonomous driving:** Uncertainty estimates for object detection affect safety-critical braking decisions.
- **Finance:** Loan default probabilities fed into risk models.
- **LLM confidence:** Token-level probabilities should reflect actual correctness for chain-of-thought or agentic workflows.
- **Active learning / Bayesian optimization:** Acquisition functions depend on well-calibrated predictive distributions.

Why Neural Networks are Miscalibrated

Guo et al. (2017) showed modern deep networks (ResNet, DenseNet) are systematically overconfident. Reasons include:

- Weight decay acts as a regularizer but is applied to the norm, not to the sharpness of the softmax distribution.
- BatchNorm and modern data augmentation (mixup, cutout) interact with the softmax in complex ways.
- NLL loss is minimized when the correct-class logit diverges ($z_y \to \infty$), and softmax can approach one-hot even at intermediate decision boundaries.
- Overfitting on training set leads to overconfident predictions on shifted validation/test data.

29.2 Calibration Metrics

29.2.1 Expected Calibration Error (ECE)

ECE

Partition the $[0, 1]$ interval into M equal-width bins $B_1, \ldots, B_M$. For each bin, let $\text{acc}(B_m)$ be the empirical accuracy and $\text{conf}(B_m)$ be the average predicted confidence. Then:

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \big|\text{acc}(B_m) - \text{conf}(B_m)\big|$$

where $|B_m|$ is the number of samples that fall into bin m. Typical choices: $M = 10$ or $M = 15$. ECE is a weighted average of calibration gaps; it measures *expected* miscalibration.

ECE Usage

Primary calibration metric in most papers (Guo et al., Minderer et al.). Reported alongside accuracy and NLL for any probabilistic classifier. In practice, the binning choice affects the result, so authors often report ECE with $M = 10$ and $M = 15$.

ECE Trade-offs

Pros:

- Intuitive: literally the expected gap between confidence and accuracy.
- Scalar summary, easy to compare across models.
- Implemented in standard libraries (`torchmetrics.CalibrationError`).

Cons:

- Sensitive to the number of bins M — no universally optimal choice.
- Equal-width bins can be empty or overloaded when predictions cluster.
- Ignores the *direction* of miscalibration (over- vs. under-confident).
- Can be gamed by predicting uniformly across bins (trivially low ECE).

29.2.2 Maximum Calibration Error (MCE)

MCE

With the same binning as ECE, the maximum calibration error is:

$$\text{MCE} = \max_{m \in \{1,\ldots,M\}} \big|\text{acc}(B_m) - \text{conf}(B_m)\big|$$

MCE captures the *worst-case* deviation, which is especially relevant for high-stakes applications where any single miscalibration could be catastrophic.

ECE vs. MCE

ECE averages over bins and is robust to a single bad bin. **MCE** highlights the worst-case miscalibration. In safety-critical settings (medical, autonomous driving), MCE is often the more relevant metric.

29.2.3 Brier Score

Brier Score (Multi-Class)

For a dataset with N samples and C classes, the Brier score is:

$$\text{Brier} = \frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C}(p_{ic} - y_{ic})^2$$

where p_{ic} is the predicted probability of class c for sample i and y_{ic} is the one-hot true label. The Brier score is a *proper scoring rule* — its true expectation is minimized by reporting the true distribution. Lower is better; random guessing yields $\approx 1 - 1/C$.

Brier Score Decomposition

The Brier score can be decomposed into three terms:

$$\text{Brier} = \text{UNCERTAINTY} - \text{RESOLUTION} + \text{RELIABILITY}$$

where uncertainty is the inherent variance in the outcomes, resolution measures how much the predictions deviate from the base rate, and reliability is the calibration component. A well-calibrated forecaster can still have a high Brier score if it lacks resolution.

29.2.4 Reliability Diagrams and Confidence Histograms

Reliability Diagram

A reliability diagram plots $\text{acc}(B_m)$ (y-axis) against $\text{conf}(B_m)$ (x-axis) for each bin. Perfect calibration lies on the $y = x$ diagonal. Points above the diagonal indicate *under-confidence*; points below indicate *over-confidence* (far more common in modern networks). The bin sizes $|B_m|$ are often displayed as a histogram bar below the diagram.

Reading Reliability Diagrams

Modern deep networks typically exhibit a characteristic "S" shape: accurate for low-confidence predictions, then a dip below the diagonal at medium-high confidence, recovering somewhat at very high confidence. Temperature scaling (see Section 29.3) flattens this curve toward the diagonal.

29.2.5 Adaptive and Class-wise ECE

Adaptive ECE

Instead of fixed-width bins, adaptive ECE (AdaECE) uses bins with *equal number of samples* so that each bin has the same statistical precision. This avoids the problem of empty or sparse bins in fixed-width binning.

Class-wise ECE

Class-wise ECE computes ECE per class (one-vs-rest) and averages:

$$\mathrm{ECE}_{\mathrm{CW}} = \frac{1}{C}\sum_{c=1}^{C}\mathrm{ECE}_c$$

This is important when the model is miscalibrated differently across classes (e.g., rare classes may be systematically overconfident).

29.3 Calibration Methods

29.3.1 Platt Scaling

Platt Scaling (1999)

For binary classification, Platt scaling fits a logistic regression model on the *logits* (pre-softmax outputs) using a held-out calibration set:

$$P(y = 1 \mid f) = \frac{1}{1 + \exp(Af + B)}$$

where f is the raw logit for the positive class, A and B are scalar parameters optimized via maximum likelihood on the calibration set. Note that A is the slope ("temperature-like") and B is the intercept (shift).

Platt Scaling Usage

Originally introduced for SVM output calibration (SVMs output signed distances, not probabilities). Also used as a lightweight calibration step for binary classifiers. In multi-class settings, one can apply a one-vs-rest Platt scaling per class.

Platt Scaling

Pros:

- Simple: just two parameters (A, B) learned via logistic regression.
- Works well for binary problems with a moderate calibration set.
- Preserves accuracy — does not change the predicted *class*, only the confidence.

Cons:

- Assumes a logistic relationship between logit and true probability.
- Not a natural extension to multi-class (requires one-vs-rest or a different parameterization).
- May overfit on small calibration sets (two parameters can still overfit if the set is tiny).

29.3.2 Temperature Scaling

Temperature Scaling

Temperature scaling is a single-parameter extension of Platt scaling for multi-class problems. Given the logit vector $\mathbf{z}_i$, the calibrated probability is:

$$\hat{p}_i = \text{softmax}(\mathbf{z}_i / T)$$

where $T > 0$ is the **temperature** parameter, optimized on a held-out validation set to minimize NLL (or ECE). $T = 1$ recovers the original predictions. $T > 1$ softens (reduces confidence); $T < 1$ sharpens (increases confidence).

Temperature Scaling is the Standard Post-Hoc Method

Since Guo et al. (2017), temperature scaling has been the default post-hoc calibration method:

- Used in virtually every modern image classifier calibration paper.
- Applied to LLM token probabilities for improved sampling and sequence-level confidence.
- Integrated into the `torch.nn.Softmax(dim=-1)` with a learnable temperature parameter.
- Does not change the argmax — accuracy is unchanged.

Temperature Scaling

Pros:

- **Single parameter** (T only) — cannot overfit.
- Multi-class native; no one-vs-rest needed.
- Accuracy-preserving (monotonic transformation of logits).
- Extremely simple to implement: divide logits by T, apply softmax.
- Optimizing NLL on validation set is convex in T (for fixed logits).

Cons:

- A *single* scalar cannot correct for class-dependent or sample-dependent miscalibration.
- Does not fix the shape of miscalibration — only stretches the confidence distribution.
- Needs a sufficiently large calibration set for reliable T.
- If the model is poorly calibrated in a complicated way, T provides limited improvement.

29.3.3 Matrix Scaling and Vector Scaling

Vector and Matrix Scaling

Generalizations of temperature scaling that allow class-specific parameters:

- **Vector Scaling:** $\hat{p}_i = \text{softmax}(\mathbf{W}\mathbf{z}_i + \mathbf{b})$ where $\mathbf{W} = \text{diag}(w_1, \ldots, w_C)$ is a diagonal matrix (one scale per class) and $\mathbf{b}$ is a bias vector. C scales + C biases = $2C$ parameters.
- **Matrix Scaling:** $\hat{p}_i = \text{softmax}(\mathbf{W}\mathbf{z}_i + \mathbf{b})$ where $\mathbf{W} \in \mathbb{R}^{C \times C}$ is a full matrix. $C^2 + C$ parameters — can model class interactions.

Temperature vs. Vector vs. Matrix Scaling

Vector Scaling sits between temperature (1 param) and matrix ($C^2 + C$ params). It corrects class-dependent confidence but not between-class interactions. **Matrix Scaling** is the most expressive but risks overfitting and is rarely needed in practice. Guo et al. found that temperature scaling often matches or outperforms vector/matrix scaling on ImageNet benchmarks, likely because matrix parameters overfit the small calibration set.

29.3.4 Isotonic Regression

Isotonic Regression

A non-parametric, monotonic calibration method. Given sorted predictions $\hat{p}_{(1)} \leq \cdots \leq \hat{p}_{(n)}$ and corresponding binary outcomes $y_{(i)}$, isotonic regression finds calibrated probabilities $q_{(i)}$ that minimize:

$$\min_{q_1 \leq \cdots \leq q_n} \sum_{i=1}^{n} (y_{(i)} - q_{(i)})^2$$

subject to monotonicity ($q_{(1)} \leq \cdots \leq q_{(n)}$). Solved efficiently via the **Pool Adjacent Violators Algorithm (PAVA)**. The result is a step function mapping raw predictions to calibrated scores.

Isotonic Regression Usage

Popular in classical machine learning (scikit-learn's `CalibratedClassifierCV` with `method='isotonic'`). Works well when the calibration set is large enough to estimate many bin boundaries. Less common in deep learning due to the preference for parametric methods that are cheaper to optimize and generalize better with small calibration sets.

Isotonic Regression

Pros:

- **Non-parametric** — can model any monotonic miscalibration curve, no assumption about functional form.
- Asymptotically consistent: with enough data, converges to the true calibration map (subject to monotonicity).
- Guaranteed monotonicity enforced by PAVA.

Cons:

- **Overfits** on small calibration sets — many parameters (essentially one per unique prediction value).
- Assumes *monotonic* relationship between raw score and true probability (reasonable for most classifiers, but not guaranteed).
- Not a natural multi-class extension (requires one-vs-rest).
- Piecewise-constant output can be coarse.

29.3.5 Histogram Binning

Histogram Binning

A non-parametric method that partitions the $[0, 1]$ interval into M equal-width (or equal-frequency) bins. For each bin, the calibrated confidence is the **empirical accuracy** of samples in that bin:

$$\tilde{p}_{\text{cal}}(x) = \text{acc}(B_m) \quad \text{for } \hat{p}(x) \in B_m$$

All samples falling in the same bin get the same calibrated score.

Histogram Binning

Pros: Simple, intuitive, guarantees calibration *within* each bin (by construction). Easily extendable to equal-mass bins for better statistical support. **Cons:** Discontinuous — adjacent bins can have very different scores, causing ranking inconsistency. The number of bins M strongly affects performance (bias-variance trade-off). Does not exploit the ordering of bins.

29.3.6 Bayesian Binning into Quantiles (BBQ)

BBQ (Naeini et al.

BBQ is a Bayesian model-averaging approach over multiple binning models. Let M be a set of possible binning schemes (different numbers of bins and/or different bin boundaries). BBQ computes:

$$P(y = 1 \mid \hat{p}) = \sum_{M \in \mathcal{M}} P(M \mid \mathcal{D}) \cdot P(y = 1 \mid \hat{p}, M)$$

where $P(M \mid \mathcal{D})$ is the posterior weight of binning model M (computed via the Beta-binomial marginal likelihood), and $P(y = 1 \mid \hat{p}, M)$ is the bin-specific accuracy under model M. The final output is a smooth, weighted average over binning schemes.

BBQ Usage

Preferred when the calibration set is small for its regularizing effect (via Bayesian model averaging). Outperforms single-binning methods on small datasets. Implemented in `pycalib` and various calibration libraries.

BBQ

Pros:

- Bayesian averaging smooths out discontinuities of single-bin methods.
- Robust to small calibration sets.
- Provides uncertainty in the calibration map itself.

Cons:

- More computationally involved than simple binning.
- The set of candidate models $\mathcal{M}$ is a design choice.
- Less common in deep-learning pipelines where simplicity dominates.

29.3.7 Spline Calibration

Spline Calibration

Fit a smoothed, monotone spline (e.g., cubic spline with monotonicity constraints) to the reliability curve $\text{acc}(B_m) = f(\text{conf}(B_m))$. The spline allows a smooth, non-parametric, monotonic calibration function that avoids the discontinuities of binning. Parameters: knot locations and coefficients, optimized via likelihood or least-squares with a smoothness penalty.

Spline Calibration

Pros: Smooth and continuous; expressiveness controlled by number of knots; monotonicity can be enforced via constraints. **Cons:** More complex to implement; knot placement is a hyperparameter; less widely used than temperature scaling or isotonic regression.

29.4 Post-Hoc vs. During-Training Calibration

Calibration During Training vs. Post-Hoc

- **Post-hoc (temperature scaling, Platt, isotonic):** Apply after training on a held-out calibration set. Does not affect training dynamics. Simple and effective.
- **During-training (label smoothing, mixup, focal loss):** Modify the training objective or data to inherently produce better-calibrated models. No separate calibration set needed.
- **Combined:** Many modern pipelines use both — train with label smoothing or mixup, then apply temperature scaling on a validation set for final calibration.

Label Smoothing and Calibration

Label smoothing replaces one-hot targets $\mathbf{y}$ with a smoothed version:

$$\mathbf{y}^{\text{LS}} = (1-\alpha)\mathbf{y} + \frac{\alpha}{C}\mathbf{1}$$

where $\alpha \in [0,1]$ is the smoothing factor. This discourages the model from pushing logits toward infinity (a known cause of overconfidence). Müller et al. (2019) showed label smoothing improves calibration as a by-product, though it can hurt distillation if not handled carefully.

Focal Loss and Calibration

Focal loss $\mathcal{L}_{\text{FL}} = -(1-p_t)^\gamma \log p_t$ up-weights hard examples but also implicitly regularizes confidence on easy examples, which can improve calibration. Mukhoti et al. (2020) showed focal loss with $\gamma > 0$ yields better-calibrated models than standard CE.

Mixup and Augmentation Effects on Calibration

Data augmentation techniques like mixup ($x = \lambda x_i + (1-\lambda)x_j$, $y = \lambda y_i + (1-\lambda)y_j$) and CutMix inherently produce softer targets, encouraging the model to be less overconfident. Thulasidasan et al. (2019) showed mixup-trained models are significantly better calibrated

than standard training.

29.5 Calibration in LLMs and NLG

Temperature Scaling for LLM Token Probabilities

In autoregressive language models, the probability of a generated token is $\text{softmax}(\mathbf{z}_{\text{logits}}/T)$. Using $T > 1$ during inference (e.g., $T = 0.7$) flattens the distribution and increases diversity. For *calibration* purposes, one can optimize T on a held-out set to minimize token-level ECE.

Length Normalization Calibration

Beam search biases toward short sequences because $\log p(\text{sequence}) = \sum_t \log p(x_t \mid x_{<t})$ is a sum of negative terms, favoring shorter hypotheses. Length normalization corrects this:

$$\text{score} = \frac{1}{|y|^\alpha} \sum_{t=1}^{|y|} \log p(y_t \mid y_{<t}, x)$$

where $\alpha \in [0, 1]$ (typically $\alpha = 0.6$–0.7). This can be treated as a calibration step: α can be tuned on a validation set to maximize BLEU or minimize ECE of sequence-level confidence.

LLM Calibration Challenges

Unlike standard classifiers, LLMs produce distributions over an exponentially large output space (all possible sequences). Token-level calibration does not guarantee sequence-level calibration. Research directions include:

- Conformal prediction for LLM outputs.
- Semantic entropy: cluster generations by meaning, then measure entropy over semantic clusters, not raw token strings.
- Calibrating long-form generation is an open problem.

29.6 Multi-Class vs. Binary Calibration

Extending to Multi-Class

Binary calibration methods can be extended to multi-class settings via:

- **One-vs-rest:** Train C binary calibrators (one per class). Issue: calibrated probabilities may not sum to 1.
- **Native multi-class:** Temperature scaling, vector scaling, and matrix scaling are natively multi-class; softmax ensures summation to 1.
- **Dirichlet calibration:** Kull et al. (2019) proposed a Dirichlet-based calibration that learns class-specific and class-interaction parameters using log-probability ratios, ensuring a valid probability simplex.

Dirichlet Calibration

Dirichlet calibration models the calibrated probability as:

$$\hat{p}_k = \frac{\exp(\mathbf{w}_k^\top \mathbf{z} + b_k)}{\sum_{j=1}^{C} \exp(\mathbf{w}_j^\top \mathbf{z} + b_j)}$$

where $\mathbf{z} = [\log p_1, \ldots, \log p_C]^\top$ are the log-predicted probabilities. This is a logistic regression in log-probability space, generalizing both temperature scaling ($\mathbf{w}_k = \frac{1}{T}\mathbf{e}_k$) and matrix scaling. Normally, one imposes the restriction $\sum_k \mathbf{w}_k = \mathbf{0}$ and $\sum_k b_k = 0$ for identifiability, reducing to $C(C-1) + (C-1)$ free parameters.

29.7 Summary Comparison Table

Table 29.1: Comparison of Calibration Methods

Method	Type	Params	Multi-Class?	Key Characteristic
Temperature Scaling	Parametric	1	Yes (native)	Single scalar T; simplest and most popular
Platt Scaling	Parametric	2	Via one-vs-rest	Logistic regression on logits; binary origin
Vector Scaling	Parametric	$2C$	Yes (native)	Class-specific scales and biases
Matrix Scaling	Parametric	C^2+C	Yes (native)	Full linear transformation of logits
Isotonic Regression	Non-parametric	Data-dep.	Via one-vs-rest	Monotonic step function via PAVA
Histogram Binning	Non-parametric	M bins	Via one-vs-rest	Discrete, within-bin accuracy
BBQ	Bayesian	Model avg.	Via one-vs-rest	Bayesian averaging over binning schemes
Spline Calibration	Semi-parametric	Knots	Via one-vs-rest	Smooth monotonic spline
Dirichlet	Parametric	C^2-1	Yes (native)	Logistic regression in log-prob space

Practical Calibration Recommendations

1. **Start with temperature scaling:** It is simple, robust, and works for multi-class out-of-the-box. Use NLL on a validation set to optimize T.
2. **If calibration improvement is insufficient,** try vector scaling or Dirichlet calibration for class-specific adjustment.
3. **During training,** add label smoothing ($\alpha = 0.1$) and/or mixup to get inherently better-calibrated models.
4. **For small calibration sets,** BBQ is the most statistically principled choice.
5. **For LLMs,** tune the generation temperature on a dev set; consider semantic entropy for long-form calibration.
6. **Always report ECE (with multiple M values) alongside accuracy.** NLL

alone conflates calibration and discrimination.

7. **For safety-critical applications,** also report MCE to catch worst-case miscalibration.

Chapter 30

Multi-Task and Meta-Learning

Multi-task learning (MTL) trains a single model on multiple related tasks simultaneously, sharing representations to improve generalization on each task. Meta-learning (learning to learn) optimizes for fast adaptation to new tasks given only a few examples. Both paradigms are central to modern transfer learning and few-shot learning systems.

30.1 Multi-Task Learning

Multi-Task Learning Objective

Given T tasks, each with loss $\mathcal{L}_i$ and dataset $\mathcal{D}_i$, MTL minimizes a weighted combination:

$$\min_{\boldsymbol{\theta}_{\text{shared}}, \{\boldsymbol{\theta}_i\}_{i=1}^T} \sum_{i=1}^{T} w_i \mathcal{L}_i(\boldsymbol{\theta}_{\text{shared}}, \boldsymbol{\theta}_i; \mathcal{D}_i)$$

where $\boldsymbol{\theta}_{\text{shared}}$ are shared parameters (backbone) and $\boldsymbol{\theta}_i$ are task-specific parameters (heads). The challenge is choosing the weights w_i and balancing gradients.

30.1.1 Hard Parameter Sharing

Hard Parameter Sharing

Architecture: A shared backbone (encoder) processes the input, and each task has its own output head. This is the most common MTL paradigm.

- **Shared:** All layers except the final task-specific head. E.g., a shared ResNet-50 trunk with separate fully-connected heads for semantic segmentation, depth estimation, and surface normals.
- **Task-specific:** One head per task, receiving the shared representation as input.

Hard Sharing Applications

- **Computer vision:** Joint semantic segmentation, instance segmentation, and depth estimation from a single RGB image.
- **NLP:** Shared BERT/T5 encoder with task-specific heads for sentiment, NER, and question-answering (GLUE benchmark).
- **Recommender systems:** Shared user/item embeddings with per-task towers for click-through rate, conversion rate, etc.

Hard vs. Soft Parameter Sharing

Pros:

- Reduces overfitting risk via shared representations (acts as a strong regularizer).
- Computational efficiency: one forward pass through the shared backbone for all tasks.
- Simple architecture, easy to implement.

Cons:

- **Task interference:** Tasks with conflicting gradients (e.g., one wants low-frequency features, another high-frequency) degrade each other.
- "Negative transfer": learning on one task hurts another.
- All tasks forced to share the same representation space, which may be suboptimal for unrelated tasks.

30.1.2 Soft Parameter Sharing

Cross-Stitch Networks (Misra et al.

Each task has its own network, but layers are connected via **cross-stitch units** — learned linear combinations of activations:

$$\begin{bmatrix} \tilde{\mathbf{x}}_A^l \\ \tilde{\mathbf{x}}_B^l \end{bmatrix} = \begin{bmatrix} \alpha_{AA} & \alpha_{AB} \\ \alpha_{BA} & \alpha_{BB} \end{bmatrix} \begin{bmatrix} \mathbf{x}_A^l \\ \mathbf{x}_B^l \end{bmatrix}$$

where α_{ij} are learnable parameters that control how much of task i's representation is shared with task j at layer l.

Sluice Networks (Ruder et al.

Generalize cross-stitch to arbitrary numbers of tasks and layers, with learned gating that can skip sharing when tasks are unrelated. Sluice also learns which layers to share and which to keep task-specific, using sparsity-inducing regularization.

Soft Sharing

Pros: More flexible than hard sharing — can share some layers and not others; task-specific capacity not compromised. **Cons:** Parameter count scales linearly with number of tasks; more complex optimization; still requires balancing multiple losses.

30.1.3 Uncertainty Weighting (Kendall et al., 2018)

Homoscedastic Uncertainty Weighting

Treat each task's loss as a Gaussian log-likelihood with task-dependent (homoscedastic) noise σ_i. The multi-task loss is derived from maximum-likelihood:

$$\mathcal{L} = \sum_{i=1}^{T} \frac{1}{2\sigma_i^2} \mathcal{L}_i + \log \sigma_i$$

where σ_i is a learned parameter per task. The $\log \sigma_i$ term acts as a regularizer preventing σ_i from growing without bound. Tasks with higher inherent noise get lower weight automatically. For classification tasks, the temperature-scaled CE can be written similarly.

Derivation (Kendall et al.)

Maximize the joint likelihood: $\max \prod_i p(y_i \mid f^W(x), \sigma_i)$ with $p(y_i \mid f, \sigma_i) = \mathcal{N}(f, \sigma_i^2)$ for regression or a tempered softmax for classification. Taking the negative log yields the

above form. The $\log \sigma_i$ regularizer emerges from the Gaussian normalization constant.

Uncertainty Weighting

Popular in multi-task computer vision (e.g., predicting depth, segmentation, and normals from a single image). Simple to implement: σ_i are just learnable scalar parameters optimized jointly. In PyTorch, $\sigma =$ `nn.Parameter(torch.zeros(1))` and the loss is computed as $\mathcal{L}/(2\sigma^2) + \log \sigma$ for stability (optimize $s = \log \sigma^2$ to avoid division by zero).

Uncertainty Weighting

Pros: Theoretically grounded (maximum likelihood); only T learnable parameters; adapts weights automatically during training. **Cons:** Assumes independent homoscedastic Gaussian noise per task; can be unstable early in training; the learned σ_i values depend on task scale (loss magnitudes must be roughly comparable or normalized).

30.1.4 GradNorm (Chen et al., 2018)

GradNorm

Instead of learning loss weights w_i via uncertainty, GradNorm dynamically adjusts w_i to balance the *gradient magnitudes* of each task at the shared layers:

$$G_w^{(i)}(t) = \left\| \nabla_{\mathbf{W}} w_i(t) \mathcal{L}_i \right\|_2$$

where $\mathbf{W}$ is a chosen set of shared parameters (typically the last shared layer). GradNorm minimizes:

$$\mathcal{L}_{\text{grad}}(t) = \sum_{i=1}^{T} \left| G_w^{(i)}(t) - \bar{G}_w(t) \times [r_i(t)]^{\alpha} \right|$$

where $\bar{G}_w(t)$ is the average gradient norm, $r_i(t) = \tilde{\mathcal{L}}_i(t)/\tilde{\mathcal{L}}_i(0)$ is the *relative training speed* (inverse of convergence speed), and $\alpha > 0$ is a hyperparameter controlling the strength of balancing. The weights $w_i(t)$ are updated via gradient descent on $\mathcal{L}_{\text{grad}}$.

GradNorm vs. Uncertainty Weighting

GradNorm explicitly balances gradient magnitudes, preventing one task from dominating. **Uncertainty Weighting** balances losses but can still allow gradient norm disparities if the loss landscape is steep. GradNorm is more expensive (requires computing gradient norms for shared parameters) but provides more direct gradient-level balancing.

30.1.5 PCGrad — Projection Conflict Gradient

PCGrad (Yu et al.

When two task gradients $\mathbf{g}_i$ and $\mathbf{g}_j$ have a negative cosine similarity (conflicting), PCGrad projects each gradient onto the normal plane of the other:

$$\mathbf{g}_i^{\text{proj}} = \mathbf{g}_i - \frac{\mathbf{g}_i \cdot \mathbf{g}_j}{\|\mathbf{g}_j\|^2} \mathbf{g}_j$$

if $\mathbf{g}_i \cdot \mathbf{g}_j < 0$ (conflict). Otherwise, the gradient is unchanged. The final update direction is the sum of the (possibly projected) gradients. This removes the component of each gradient that conflicts with other tasks.

PCGrad

Applied in multi-task reinforcement learning and supervised MTL where tasks have qualitatively different objectives. Removes destructive gradient interference without changing the loss weights.

PCGrad

Pros: Simple, no new hyperparameters (beyond MR threshold); works at the gradient level, directly addressing conflict. **Cons:** Pairwise projection is $O(T^2)$ for T tasks; for many tasks, computing all pairwise projections can be expensive.

30.1.6 MGDA and CAGrad — Pareto Optimality

MGDA (Sener & Koltun

Find a convex combination of task gradients that is **Pareto optimal**:

$$\min_{\mathbf{w} \in \Delta_T} \left\| \sum_{i=1}^{T} w_i \mathbf{g}_i \right\|_2^2$$

where $\Delta_T = \{\mathbf{w} \in \mathbb{R}^T : w_i \geq 0, \ \sum_i w_i = 1\}$ is the probability simplex. This finds a direction along which no task's loss can be further decreased (Pareto stationary). Solved via the Frank-Wolfe algorithm or quadratic programming.

CAGrad (Liu et al.

CAGrad (Conflict-Averse Gradient) maximizes the worst-case improvement across tasks subject to a proximity constraint:

$$\max_{\mathbf{d}} \min_i \mathbf{g}_i^\top \mathbf{d} \quad \text{s.t.} \ \|\mathbf{d} - \mathbf{g}_0\| \leq c\|\mathbf{g}_0\|$$

where $\mathbf{g}_0$ is the average gradient and $c \in [0, 1)$ controls the tolerance to deviation from the average direction. When $c = 0$, CAGrad reduces to the average gradient; as $c \to 1$, it approaches the pure MGDA solution.

MGDA vs. CAGrad

MGDA: Guarantees Pareto optimality but can converge to a poor Pareto point if not initialized well. **CAGrad:** Provides a tunable knob (c) to balance between averaging and Pareto optimality. In practice, CAGrad often outperforms MGDA because averaging provides a useful anchor direction.

30.1.7 DWA — Dynamic Weight Average

DWA (Liu et al.

Simple heuristic: adjust loss weights based on the rate of loss change:

$$w_i(t) = \frac{N \exp\big(\omega_i(t-1)/T\big)}{\sum_{j=1}^{N} \exp\big(\omega_j(t-1)/T\big)}$$

where $\omega_i(t) = \mathcal{L}_i(t)/\mathcal{L}_i(t-1)$ is the ratio of current to previous loss (relative training speed). T is a temperature parameter controlling the sharpness of the weighting. Tasks with slower progress (higher ω_i) get higher weights.

DWA

Pros: Extremely simple, no gradient computation of hyperparameters. **Cons:** Heuristic; not theoretically grounded; sensitive to T and loss scale; relies on windowed loss ratios which can be noisy.

30.2 Meta-Learning — Learning to Learn

Few-Shot Learning Setup

N-way K-shot: Given N novel classes with K support examples each, classify query examples into one of the N classes. The model is meta-trained on many such episodes sampled from a set of *base classes*, then evaluated on *novel classes* unseen during meta-training.

30.2.1 MAML — Model-Agnostic Meta-Learning

MAML (Finn et al.

MAML learns an initialization $\boldsymbol{\theta}$ such that one (or a few) gradient steps on a **support set** $\mathcal{D}_{\mathcal{T}}^{\text{spt}}$ of task $\mathcal{T}$ produces good performance on the **query set** $\mathcal{D}_{\mathcal{T}}^{\text{qry}}$:
Inner loop (task adaptation):

$$\boldsymbol{\theta}_i' = \boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} \mathcal{L}_{\mathcal{T}_i}\big(f_{\boldsymbol{\theta}}; \mathcal{D}_{\mathcal{T}_i}^{\text{spt}}\big)$$

Outer loop (meta-update):

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \beta \nabla_{\boldsymbol{\theta}} \sum_{\mathcal{T}_i} \mathcal{L}_{\mathcal{T}_i}\big(f_{\boldsymbol{\theta}_i'}; \mathcal{D}_{\mathcal{T}_i}^{\text{qry}}\big)$$

The outer loop gradient flows *through* the inner loop update, requiring second-order derivatives (Hessian-vector products).

MAML Outer-Loop Gradient

By the chain rule, the gradient of query loss w.r.t. initial parameters:

$$\nabla_{\boldsymbol{\theta}} \mathcal{L}_{\mathcal{T}_i}(f_{\boldsymbol{\theta}_i'}) = \nabla_{\boldsymbol{\theta}_i'} \mathcal{L}_{\mathcal{T}_i}(f_{\boldsymbol{\theta}_i'}) \cdot \big(\mathbf{I} - \alpha \nabla_{\boldsymbol{\theta}}^2 \mathcal{L}(f_{\boldsymbol{\theta}}; \mathcal{D}^{\text{spt}})\big)$$

The Hessian term $\nabla^2_{\boldsymbol{\theta}}$ makes MAML computationally expensive.

MAML Applications

Few-shot image classification (Omniglot, miniImageNet, tieredImageNet), few-shot reinforcement learning (adapting policies to new environments with a few trajectories), and domain generalization.

MAML

Pros:
- Model-agnostic (works with any architecture trained by SGD).
- Learns a representation that is easy to fine-tune quickly.
- Outer loop is a standard meta-objective — no special loss.

Cons:
- Second-order derivatives are computationally expensive (Hessian through inner loop).
- Inner loop must be differentiable, limiting architecture choices.
- Sensitive to inner-loop learning rate α.
- For deeper networks, one gradient step may be insufficient.

30.2.2 FOMAML and Reptile

First-Order MAML (FOMAML)

FOMAML ignores the second-order term in the meta-gradient, treating $\boldsymbol{\theta}'_i$ as fixed w.r.t. $\boldsymbol{\theta}$ in the outer loop:

$$\nabla_{\boldsymbol{\theta}} \mathcal{L}(f_{\boldsymbol{\theta}'_i}) \approx \nabla_{\boldsymbol{\theta}'_i} \mathcal{L}(f_{\boldsymbol{\theta}'_i})$$

This is equivalent to taking one (or more) SGD steps on each task and then moving the initialization toward the average of those task-specific parameters. Much cheaper, often nearly as accurate.

Reptile (Nichol et al.

Even simpler: for each task, take $k > 1$ SGD steps to get $\boldsymbol{\theta}'_i$, then move the initialization toward the average of the adapted parameters:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \beta \frac{1}{n} \sum_{i=1}^{n} (\boldsymbol{\theta}'_i - \boldsymbol{\theta})$$

No second-order gradients, no keeping the inner-loop computation graph. Reptile is essentially a multi-step extension of FOMAML and can be interpreted as maximizing the inner product between gradients of different minibatches from the same task.

FOMAML / Reptile

Pros: Far cheaper than MAML; Reptile's simplicity enables scaling to larger models and more inner steps. **Cons:** Theoretically less precise than full MAML; performance gap is usually small on standard benchmarks.

30.2.3 Prototypical Networks

Prototypical Networks (Snell et al.

A metric-based meta-learner. For each class k, compute a **prototype** as the mean of embedded support examples:

$$\mathbf{c}_k = \frac{1}{|S_k|} \sum_{(\mathbf{x}_i, y_i) \in S_k} f_{\phi}(\mathbf{x}_i)$$

where f_{ϕ} is an embedding network. Classification of a query $\mathbf{x}$ uses a softmax over Euclidean distances to prototypes:

$$p(y = k \mid \mathbf{x}) = \frac{\exp\big(-d(f_{\phi}(\mathbf{x}),\, \mathbf{c}_k)\big)}{\sum_{k'} \exp\big(-d(f_{\phi}(\mathbf{x}),\, \mathbf{c}_{k'})\big)}$$

The squared Euclidean distance $d(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_2^2$ is equivalent to a linear classifier when prototypes are used as class weights (for Euclidean distance).

Bregman Divergence Prototypes

Prototypical Networks work with any **Bregman divergence**. The prototype is shown to be the minimizer of the sum of Bregman divergences to support points. Using squared Euclidean distance (ℓ_2), the mean is the optimal prototype. Using KL divergence, the geometric mean is optimal.

Prototypical Networks

Standard baseline for few-shot classification. Simple, no inner-loop optimization — just learn a good embedding space where classes cluster around their prototypes. Works best when the embedding dimension is moderate and classes are well-separated.

Prototypical Networks

Pros: Simple and fast (no inner loop); deterministic; effective when embedding space is well-structured. **Cons:** Assumes a single prototype per class (unimodal class distribution); struggles with multi-modal class distributions; requires a good embedding network.

30.2.4 Matching Networks

Matching Networks (Vinyals et al.

Another metric-based approach that uses **attention over the support set**:

$$\hat{y} = \sum_{i=1}^{k} a(\hat{\mathbf{x}}, \mathbf{x}_i)\, y_i$$

where the attention weight $a(\hat{\mathbf{x}}, \mathbf{x}_i) = \text{softmax}(\text{cosine}(f(\hat{\mathbf{x}}), g(\mathbf{x}_i)))$. Matching Networks combine an embedding function f for the query and g for the support examples, with the key innovation being **full-context embeddings**: a bidirectional LSTM over the entire support set so each support embedding is context-aware.

Matching Networks

Pros: Context-aware embeddings allow support examples to influence each other's representation. **Cons:** More complex than ProtoNets; the LSTM over the support set is slow and limits scalability; cosine attention is less expressive than learned relation modules.

30.2.5 Relation Networks

Relation Networks (Sung et al.

Learn a **relation module** $g_{\boldsymbol{\phi}}$ that scores the similarity between a query embedding and each class representation (computed by concatenating support embeddings per class). Instead of a fixed distance metric, the similarity is learned:

$$r_{i,j} = g_{\boldsymbol{\phi}}\big(\mathcal{C}(f_{\boldsymbol{\phi}}(\mathbf{x}_i), f_{\boldsymbol{\phi}}(\mathbf{x}_j))\big)$$

where $\mathcal{C}$ is concatenation and $g_{\boldsymbol{\phi}}$ is a neural network outputting a scalar relation score.

Relation Networks

Pros: Learnable metric — can capture nonlinear similarity patterns that fixed distances miss. **Cons:** Additional network $g_{\boldsymbol{\phi}}$ increases parameters and overfitting risk; requires more meta-training data.

30.3 Gradient-Based Hyperparameter Optimization

Hypergradient

For a model with parameters $\boldsymbol{\theta}$ and hyperparameters $\boldsymbol{\lambda}$, the bilevel optimization is:

$$\min_{\boldsymbol{\lambda}} \mathcal{L}_{\text{val}}\big(\boldsymbol{\theta}^*(\boldsymbol{\lambda}), \boldsymbol{\lambda}\big) \quad \text{s.t. } \boldsymbol{\theta}^*(\boldsymbol{\lambda}) = \arg\min_{\boldsymbol{\theta}} \mathcal{L}_{\text{train}}(\boldsymbol{\theta}, \boldsymbol{\lambda})$$

The hypergradient $\nabla_{\boldsymbol{\lambda}} \mathcal{L}_{\text{val}}$ can be computed via **implicit differentiation** (using the implicit function theorem on the optimality condition $\nabla_{\boldsymbol{\theta}} \mathcal{L}_{\text{train}} = 0$) or **iterative differentiation** (reverse-mode through the entire optimization trajectory — expensive but exact).

DARTS — Differentiable Architecture Search (Liu et al.

Relaxes the discrete architecture search to a continuous one by assigning a mixing weight $\alpha^{(i,j)}$ to each candidate operation $o \in \mathcal{O}$ between nodes i and j:

$$\bar{o}^{(i,j)}(x) = \sum_{o \in \mathcal{O}} \frac{\exp(\alpha_o^{(i,j)})}{\sum_{o' \in \mathcal{O}} \exp(\alpha_{o'}^{(i,j)})} \cdot o(x)$$

After bilevel optimization of $\boldsymbol{\alpha}$ (architecture) and $\mathbf{w}$ (weights), the final architecture is obtained by selecting the operation with the largest α for each edge. DARTS made architecture search orders of magnitude faster than RL or evolutionary approaches.

DARTS

Pros: Differentiable — can use gradient-based optimization; dramatically faster than RL/evolution-based NAS. **Cons:** The continuous relaxation introduces a bias toward parameter-free operations (skip-connections dominate in later stages); the final discretization step can cause a performance drop; GPU memory grows with number of candidate operations.

30.4 Transfer Learning Strategies

Fine-Tuning Spectrum

Strategies for adapting a pretrained model to a downstream task:

- **Full fine-tuning:** Update all parameters. Most expressive but most expensive, risk of catastrophic forgetting.
- **Linear probing:** Freeze the backbone, train only a linear classifier on top. Fast but limited if the backbone features are not well-aligned to the new task.
- **Partial fine-tuning:** Freeze early layers, fine-tune top layers (e.g., last 2–3 layers of a ResNet). Balances adaptation and efficiency.
- **LoRA (Low-Rank Adaptation):** Learn low-rank weight updates $\Delta\mathbf{W} = \mathbf{BA}$ while keeping $\mathbf{W}$ frozen. See Chapter 32 for details.
- **Adapter layers:** Insert small bottleneck modules between frozen transformer layers. See Chapter 32.
- **Prefix / P-Tuning:** Learn continuous prompt embeddings prepended to the input while keeping the model frozen.

Fine-Tuning vs. LoRA vs. Adapters

Full fine-tuning: Best performance potential, worst memory (stores full model per downstream task). **LoRA:** Lightweight (few MB per task), performance close to full fine-tuning, merged weights for inference. **Adapters:** Slightly slower inference (extra layers), but very parameter-efficient. **Prefix tuning:** Most parameter-efficient (only prefix tokens learned), but caps at moderate performance.

30.5 Summary Comparison Table

Practical Recommendations

- **Multi-task:** Start with uncertainty weighting for simplicity; use GradNorm or PCGrad if tasks are known to conflict. For many tasks ($T > 10$), PCGrad becomes expensive — use CAGrad or simple loss-balancing heuristics.
- **Meta-learning:** Prototypical Networks are the recommended baseline for few-shot classification (simple, strong). Use MAML when you need a general-purpose meta-learner that can adapt any architecture. For large models, consider Reptile or ANIL (split feature extractor from head).
- **Transfer learning:** Linear probing for quick baselines; full fine-tuning for best results when enough data; LoRA for LLMs and large vision transformers when efficiency matters.

Table 30.1: Comparison of Multi-Task Loss Balancing Methods

Method	**Category**	**Params**	**Key Idea**
Equal Weights	Baseline	0	Sum losses; simplest but suboptimal
Uncertainty Weighting	Loss-based	T	Learn task noise σ_i; MLE
GradNorm	Gradient-based	T	Balance gradient magnitudes
PCGrad	Gradient-based	0	Project conflicting gradients apart
MGDA	Gradient-based	0	Find Pareto-optimal gradient direction
CAGrad	Gradient-based	1 (c)	Maximize worst-case improvement
DWA	Loss-based	1 (T)	Weight by relative loss change

Table 30.2: Comparison of Meta-Learning Methods

Method	**Inner Loop?**	**Complexity**	**Learns**	**Key Characteristic**
MAML	Yes (GD)	High (2nd order)	Init. params	Learns to fine-tune fast
FOMAML	Yes (GD)	Medium	Init. params	MAML without 2nd-order
Reptile	Yes (multi-GD)	Low	Init. params	SGD-like multi-step average
ProtoNets	No	Low	Embedding	Distance to class prototype
Matching Nets	No	Medium	Embedding + LSTM	Attention over support set
Relation Nets	No	Medium	Embedding + relation	Learned similarity metric

Chapter 31

Graph Neural Network Operations

Graph Neural Networks (GNNs) operate on graph-structured data $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where nodes $\mathcal{V}$ have feature vectors and edges $\mathcal{E}$ define connectivity. GNNs learn node, edge, and graph-level representations through iterative message passing between neighbors, making them the default architecture for molecular modeling, social network analysis, recommendation, and knowledge graphs.

31.1 Message Passing Framework

General Message Passing

For a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with node features $\mathbf{h}_v^{(0)}$, the message passing at layer k is:

$$\mathbf{h}_v^{(k)} = \text{UPDATE}^{(k)}\Big(\mathbf{h}_v^{(k-1)},\ \ \text{AGGREGATE}^{(k)}\big(\{\mathbf{h}_u^{(k-1)} : u \in \mathcal{N}(v)\}\big)\Big)$$

where $\mathcal{N}(v)$ is the neighborhood of node v, AGGREGATE is a permutation-invariant function (sum, mean, max, or attention), and UPDATE typically involves a linear transformation followed by a nonlinearity σ. After K layers, node v's representation captures information from its K-hop neighborhood.

Key Design Choices in GNNs

Three axes differentiate GNN architectures:

1. **Aggregator:** Sum preserves multiset structure (strongest), mean normalizes for degree, max is limited but simple.
2. **Update:** Simple linear + activation, GRU, or MLP.
3. **Edge features:** Ignore, concatenate with node features, or use separate edge-network (message network).

31.2 Spectral Methods — Graph Convolution via Fourier Transform

31.2.1 GCN — Graph Convolutional Network

GCN (Kipf & Welling

The GCN uses a localized first-order approximation of spectral graph convolutions. The layer-wise propagation rule is:

$$\mathbf{H}^{(l+1)} = \sigma\Big(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\Big)$$

where:

- $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}_N$ — adjacency matrix with added self-loops (so each node includes its own features).

- $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$ — degree matrix of $\tilde{\mathbf{A}}$.
- $\mathbf{W}^{(l)} \in \mathbb{R}^{F_l \times F_{l+1}}$ — learnable weight matrix.
- σ — nonlinearity (ReLU).

The normalization $\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}$ implements symmetric normalization of the adjacency, preventing feature values from exploding with node degree.

GCN as First-Order Chebyshev

The GCN can be derived by taking the $K = 1$ truncation of the Chebyshev polynomial expansion of the spectral filter, setting $\lambda_{\max} \approx 2$ (the largest eigenvalue of the normalized Laplacian is ≤ 2), and tying the Chebyshev coefficients. This yields the renormalization trick $\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}$.

GCN Applications

Semi-supervised node classification (Cora, Citeseer, Pubmed), where only a small fraction of nodes are labeled. Also used for graph-level classification (via global pooling after multiple GCN layers) and as a building block in more complex GNNs.

GCN

Pros:

- Simple and scalable — just matrix multiplications.
- The renormalization trick stabilizes training.
- Strong performance on transductive node classification benchmarks.
- Efficient for sparse graphs via sparse-dense matrix multiplication.

Cons:

- **Transductive only** — requires the full graph Laplacian at training time; does not generalize to unseen nodes without retraining.
- Shallow (typically 2–3 layers); deeper GCNs suffer from over-smoothing and vanishing gradients.
- Mean aggregation over neighborhoods is degree-dependent, which can be suboptimal.
- Does not use edge features.

31.2.2 ChebNet — Chebyshev Spectral CNN

ChebNet (Defferrard et al.

ChebNet parameterizes the spectral filter as a Chebyshev polynomial of the graph Laplacian, avoiding the expensive eigendecomposition:

$$\mathbf{g}_{\boldsymbol{\theta}} \star \mathbf{x} \approx \sum_{k=0}^{K-1} \theta_k T_k(\tilde{\mathbf{L}})\mathbf{x}$$

where:

- $\tilde{\mathbf{L}} = \frac{2}{\lambda_{\max}}\mathbf{L} - \mathbf{I}_N$ — scaled Laplacian with eigenvalues in $[-1, 1]$.
- T_k are Chebyshev polynomials defined recursively: $T_0(\mathbf{L}) = \mathbf{I}$, $T_1(\mathbf{L}) = \mathbf{L}$, $T_k(\mathbf{L}) = 2\mathbf{L}T_{k-1}(\mathbf{L}) - T_{k-2}(\mathbf{L})$.
- $\theta_k \in \mathbb{R}$ are the learnable filter coefficients.

The filter is **exactly K-localized** in the graph — node v's output depends only on nodes within K hops.

ChebNet vs. GCN

ChebNet uses K filter parameters and learns a richer spectral filter (polynomial degree K). **GCN** is the $K = 1$ special case with tied parameters. ChebNet is more expressive but also more computationally involved due to recursive Chebyshev evaluation.

31.2.3 SGC — Simplifying Graph Convolution

SGC (Wu et al.

SGC removes nonlinearities between GCN layers, collapsing the model to a single linear classifier on preprocessed features:

$$\hat{\mathbf{Y}} = \text{softmax}\big(\tilde{\mathbf{S}}^K \mathbf{X}\mathbf{W}\big)$$

where $\tilde{\mathbf{S}} = \tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}$ and $\mathbf{W}$ is a single weight matrix. The preprocessing $\tilde{\mathbf{S}}^K\mathbf{X}$ is deterministic and can be computed once, making SGC orders of magnitude faster than GCN with comparable performance on many benchmarks.

SGC

Pros: Extremely fast (single logistic regression after preprocessing); shows that much of GCN's benefit comes from the Laplacian smoothing, not the nonlinearities. **Cons:** Loses expressive power of deep nonlinear GCNs on complex datasets; preprocessing must be rerun for new graphs.

31.2.4 ARMA Filters

ARMA Filters (Bianchi et al.

Auto-Regressive Moving Average filters generalize spectral convolutions by stacking ARMA_K layers:

$$\bar{\mathbf{X}}^{(t+1)} = \sigma\big(\tilde{\mathbf{L}}\bar{\mathbf{X}}^{(t)}\mathbf{W} + \mathbf{X}\mathbf{V}\big)$$

where $\tilde{\mathbf{L}}$ is the normalized Laplacian, and $\mathbf{W}, \mathbf{V}$ are learnable weights. Multiple ARMA layers are stacked, and the outputs from different stacks are averaged. ARMA filters pro-

vide a more expressive spectral response than simple polynomial filters, closer to rational filters in signal processing.

31.3 Spatial Methods — Message Passing on Neighbors

31.3.1 GraphSAGE

GraphSAGE (Hamilton et al.

GraphSAGE (SAmple and aggreGatE) is an inductive framework that samples a fixed-size neighborhood for each node and then aggregates:

$$\mathbf{h}_v^{(k)} = \sigma\Big(\mathbf{W}^{(k)} \cdot \text{CONCAT}\big(\mathbf{h}_v^{(k-1)},\ \text{AGG}\big(\{\mathbf{h}_u^{(k-1)} : u \in \mathcal{N}(v)\}\big)\big)\Big)$$

Three aggregator choices:

- **Mean:** $\text{AGG} = \frac{1}{|\mathcal{N}(v)|}\sum_{u\in\mathcal{N}(v)} \mathbf{h}_u^{(k-1)}$ — simple average.
- **LSTM:** Apply an LSTM to a random permutation of neighbors — more expressive but not permutation-invariant (the random ordering mitigates this in practice).
- **Pooling:** $\text{AGG} = \max(\{\sigma(\mathbf{W}_{\text{pool}}\mathbf{h}_u^{(k-1)} + \mathbf{b}) : u \in \mathcal{N}(v)\})$ — element-wise max after a projection.

GraphSAGE Strengths

Inductive capability: Can generate embeddings for unseen nodes without retraining — critical for evolving graphs (new users in social networks, new papers in citation graphs). The neighborhood sampling strategy decouples memory from graph size, enabling training on large graphs via minibatches.

GraphSAGE

Pros: Inductive; scalable via sampling; flexible aggregators. **Cons:** Random sampling introduces variance; LSTM aggregator is not strictly permutation-invariant; the neighborhood sampling depth is bounded (2–3 hops typical).

31.3.2 GAT — Graph Attention Networks

GAT (Veličković et al.

GAT computes attention coefficients α_{ij} for each neighbor, allowing the model to learn which neighbors are more important:

$$\alpha_{ij} = \frac{\exp\big(\text{LeakyReLU}\big(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_i \,\|\, \mathbf{W}\mathbf{h}_j]\big)\big)}{\sum_{k \in \mathcal{N}(i)\cup\{i\}} \exp\big(\text{LeakyReLU}\big(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_i \,\|\, \mathbf{W}\mathbf{h}_k]\big)\big)}$$

where $\|$ denotes concatenation, $\mathbf{W}$ is a shared linear transformation, and $\mathbf{a}$ is a learnable attention vector. The updated node embedding is:

$$\mathbf{h}'_i = \sigma\left(\sum_{j \in \mathcal{N}(i)\cup\{i\}} \alpha_{ij} \mathbf{W}\mathbf{h}_j\right)$$

Multi-Head GAT

To stabilize attention, GAT uses K independent attention heads and aggregates their outputs:

$$\mathbf{h}'_i = \Big\|_{k=1}^{K} \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^k \mathbf{W}^k \mathbf{h}_j\right)$$

where $\|$ is concatenation. For the final prediction layer, averaging (not concatenation) is used.

GAT Applications

Node classification with heterophilic graphs, molecular property prediction, knowledge graph completion, and anywhere that different neighbors have differing importance to the central node's task.

GAT

Pros:

- Learns edge importance implicitly — no need to know which edges matter a priori.
- Computationally efficient — attention is parallelizable across nodes and edges.
- Inductive: attention is computed per edge, so new nodes/edges can be handled.
- Multi-head attention improves robustness.

Cons:

- **Static attention (GATv2 fix):** In the original GAT, the attention ranking is monotonic within a neighborhood regardless of the query node — a limitation fixed by GATv2.
- Memory grows with number of edges ($O(|\mathcal{E}|)$ for the attention matrix), which can be prohibitive for dense graphs.
- Does not use edge features (though extensions like EGAT exist).

31.3.3 GATv2 — Dynamic Attention

GATv2 (Brody et al.

GATv2 fixes the static attention problem by applying the linear transformation after the LeakyReLU, not before:

$$\alpha_{ij} = \frac{\exp\big(\mathbf{a}^\top \text{LeakyReLU}\big(\mathbf{W}_1\mathbf{h}_i + \mathbf{W}_2\mathbf{h}_j\big)\big)}{\sum_{k\in\mathcal{N}(i)\cup\{i\}} \exp\big(\mathbf{a}^\top \text{LeakyReLU}\big(\mathbf{W}_1\mathbf{h}_i + \mathbf{W}_2\mathbf{h}_k\big)\big)}$$

The key change: the scoring function is parameterized *before* the LeakyReLU, making the attention truly **dynamic** — the ranking of neighbor importance can depend on the query node. GAT can only produce static attention; GATv2 produces dynamic attention.

Static vs. Dynamic Attention

A GNN attention mechanism computes $\alpha(\mathbf{h}_i, \mathbf{h}_j)$. It is **static** if there exists a "global" ranking of keys j independent of the query i. GAT's architecture implicitly enforces this; GATv2 removes the constraint. In practice, GATv2 consistently outperforms GAT, especially on graphs where neighbor relevance is query-dependent.

31.3.4 GIN — Graph Isomorphism Network

GIN (Xu et al.

GIN is designed to be as powerful as the Weisfeiler-Lehman (WL) graph isomorphism test. The node update is:

$$\mathbf{h}_v^{(k)} = \text{MLP}^{(k)}\left((1+\epsilon^{(k)})\,\mathbf{h}_v^{(k-1)} + \sum_{u\in\mathcal{N}(v)} \mathbf{h}_u^{(k-1)}\right)$$

where $\epsilon^{(k)}$ is a learnable or fixed parameter controlling the self-loop weight. The MLP ensures the function is **injective** over multisets — a necessary condition for WL-level expressiveness. When $\epsilon = 0$, this is simply **sum aggregation** + **MLP**.

Key Insight: Sum Aggregation is Maximally Expressive

Xu et al. proved that for a GNN to be as powerful as the WL test:

- The neighborhood aggregation must be **injective** over multisets of neighbor features.
- **Mean** and **max** aggregators fail injectivity for certain multisets (mean cannot distinguish same-average sets; max cannot distinguish different cardinalities).
- **Sum** + injective MLP (2-layer MLP with sufficient width) is theoretically maximally expressive.

GIN

Pros: Provably as powerful as WL test — represents an upper-bound on standard GNN expressiveness. Implementation is simple (sum + MLP). **Cons:** In practice, the theoretical advantage often does not translate to significantly better accuracy on real-world benchmarks (mean aggregation with GCN often works well enough). ϵ is a technical parameter; in practice often set to 0.

31.4 Graph Pooling — From Nodes to Graph-Level Representations

Global Pooling Methods

To produce a graph-level embedding from node embeddings, three standard approaches exist (see Chapter **??** for detailed treatment):

- **Global Mean Pool:** $\mathbf{h}_{\mathcal{G}} = \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} \mathbf{h}_v$.
- **Global Sum Pool:** $\mathbf{h}_{\mathcal{G}} = \sum_{v \in \mathcal{V}} \mathbf{h}_v$. Preferred when graph size matters (e.g., molecular weight).
- **Global Max Pool:** $\mathbf{h}_{\mathcal{G}} = \max_{v \in \mathcal{V}} \mathbf{h}_v$. Captures most prominent features.

Hierarchical Pooling

Beyond global pooling, hierarchical methods learn to coarsen the graph:

- **DiffPool:** Differentiable soft assignment of nodes to clusters at each layer.
- **SAGPool:** Self-attention scores determine which nodes to keep; top-k nodes retained.
- **Top-K Pooling:** Project nodes to a scalar score, select the top K scoring nodes, propagate on the induced subgraph.

See Chapter **??** for complete formulations.

31.5 Graph Normalization

31.5.1 GraphNorm

GraphNorm (Cai et al.

GraphNorm generalizes InstanceNorm to graphs, normalizing across nodes within each graph:

$$\text{GraphNorm}(\mathbf{h}_{i,g}) = \gamma \cdot \frac{\mathbf{h}_{i,g} - \alpha \cdot \mu_{\text{graph}} - (1-\alpha) \cdot \mu_{\text{node}}}{\sigma_g} + \beta$$

where:

- μ_{graph} is the mean over all nodes in graph g.
- μ_{node} is the mean over features of node i.
- $\alpha \in [0, 1]$ is a learnable parameter interpolating between node-level centering ($\alpha = 0$) and graph-level centering ($\alpha = 1$).
- σ_g is the standard deviation over all nodes in graph g.

GraphNorm outperforms BatchNorm and LayerNorm for graph-level prediction tasks by preserving graph-specific mean shifts that carry structural information.

31.5.2 PairNorm

PairNorm (Zhao & Akoglu

PairNorm addresses **over-smoothing** in deep GNNs by enforcing a target total pairwise distance between node embeddings:

$$\tilde{\mathbf{h}}_i = \mathbf{h}_i - \frac{1}{n}\sum_{j=1}^{n}\mathbf{h}_j$$

(center embeddings), then scale to match a target norm:

$$\dot{\mathbf{h}}_i = s \cdot \frac{\tilde{\mathbf{h}}_i}{\sqrt{\frac{1}{n}\sum_{j=1}^{n}\|\tilde{\mathbf{h}}_j\|_2^2}}$$

where $s > 0$ is a hyperparameter controlling the target per-node norm. PairNorm keeps nodes distinguishable even after many message-passing layers.

31.5.3 Other Graph Normalizations

NodeNorm and DiffGroupNorm

- **NodeNorm (Zhou et al., 2020):** Normalizes each node's features to unit variance (similar to InstanceNorm but applied per node). Prevents over-smoothing by preserving feature variance.
- **DiffGroupNorm:** Differentiable clustering-based normalization — groups nodes into soft clusters, then normalizes within each cluster. Combines the benefits of group-level statistics with graph structure awareness.

31.6 Positional Encoding for Graphs

Laplacian Positional Encoding

Compute the eigenvectors $\mathbf{U}$ of the graph Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{A}$ (or the normalized Laplacian):

$$\mathbf{L} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$$

Use the first k eigenvectors (corresponding to k smallest non-zero eigenvalues) as positional encodings:

$$\mathbf{PE}_i = [u_{i,1}, u_{i,2}, \ldots, u_{i,k}] \in \mathbb{R}^k$$

Sign ambiguity: Eigenvectors are determined only up to a sign flip ($\pm\mathbf{u}$); randomly flipping signs during training provides invariance.

Random Walk Positional Encoding (RWPE)

For each node, compute the k-step random walk landing probabilities:

$$\mathbf{RWPE}_i = [(\mathbf{A}\mathbf{D}^{-1})^1_{ii}, (\mathbf{A}\mathbf{D}^{-1})^2_{ii}, \ldots, (\mathbf{A}\mathbf{D}^{-1})^k_{ii}]$$

This encodes structural properties like the number of k-length closed walks (related to local subgraph counts). Unlike Laplacian PE, RWPE is sign-invariant and deterministic.

Distance Encoding (Li et al.

Augment edge features with the shortest-path distance between nodes:

$$\mathbf{e}_{ij}^{\text{dist}} = \text{Embedding}(\text{spd}(i, j))$$

or add a learnable bias based on distance to the attention score. Distance encoding provides global structural information that local message passing alone cannot capture.

31.7 Scalable GNNs

GraphSAINT (Zeng et al.

GraphSAINT samples **subgraphs** (not individual nodes) for minibatch training. A random walk sampler constructs a subgraph, and a full GCN is run on the subgraph. The loss is normalized by a sampling probability-based estimator to ensure unbiased gradient estimates. GraphSAINT achieves state-of-the-art accuracy on billion-scale graphs.

Cluster-GCN (Chiang et al.

Partition the graph into clusters using METIS or similar, then form minibatches from clusters. By restricting message passing within clusters (plus some between-cluster edges), Cluster-GCN achieves:

- Memory efficiency: only the subgraph adjacency needed in memory.
- Better utilization: clusters are denser than random node samples, reducing variance in gradient estimation.
- Applicable to very deep GNNs due to the reduced neighborhood explosion.

GraphSAGE Minibatch

GraphSAGE's native minibatch strategy: sample a target node, sample its k-hop neighbors recursively with a fixed fanout (e.g., $[25, 10]$ for a 2-layer GNN). This builds a computation graph for each minibatch independent of the full graph structure, enabling inductive inference on unseen nodes.

31.8 Heterogeneous GNNs — Relational Graphs

RGCN — Relational Graph Convolutional Network (Schlichtkrull et al.

For graphs with multiple edge types (relations) $\mathcal{R}$, RGCN applies separate weight matrices per relation type:

$$\mathbf{h}_i^{(l+1)} = \sigma\left(\sum_{r\in\mathcal{R}}\sum_{j\in\mathcal{N}_i^r}\frac{1}{c_{i,r}}\mathbf{W}_r^{(l)}\mathbf{h}_j^{(l)} + \mathbf{W}_0^{(l)}\mathbf{h}_i^{(l)}\right)$$

where $\mathcal{N}_i^r$ are neighbors of i under relation r, $c_{i,r} = |\mathcal{N}_i^r|$ is a normalization constant, and $\mathbf{W}_0$ is the self-loop weight. To prevent overparameterization with many relations, RGCN

uses **basis decomposition**:

$$\mathbf{W}_r^{(l)} = \sum_{b=1}^{B} a_{rb}^{(l)} \mathbf{V}_b^{(l)}$$

where $B \ll |\mathcal{R}|$ basis matrices are shared across relations, and a_{rb} are relation-specific scalar coefficients.

RGCN Applications

Knowledge graph completion (link prediction in Freebase, Wikidata), recommendation with heterogeneous user-item interactions (click, purchase, rate), and molecular graphs with typed bonds (single, double, aromatic).

RGCN

Pros: Handles multiple relation types natively; basis decomposition controls parameter count. **Cons:** Large memory footprint for many relations; scaling to graphs with thousands of relation types remains challenging; does not model relation paths or compositions explicitly.

31.9 Summary Comparison Table

Table 31.1: Comparison of Graph Neural Network Architectures

Model	Category	Aggregation	Inductive?	Edge Feat?	Key Idea
GCN	Spectral	Mean (renormalized)	No	No	1st-order Chebyshev with renormalization
ChebNet	Spectral	Poly (degree K)	No	No	Chebyshev polynomial of Laplacian
SGC	Spectral	Mean (K-hop)	No	No	Remove nonlinearities; linear classifier
ARMA	Spectral	Rational filter	No	No	Stacked ARMA layers for rich spectrum
GraphSAGE	Spatial	Mean/LSTM/Pool	Yes	No	Neighborhood sampling + aggregation
GAT	Spatial	Attention (static)	Yes	No	Learned attention coefficients
GATv2	Spatial	Attention (dynamic)	Yes	No	Fixed attention — dynamic per query
GIN	Spatial	Sum + MLP	Yes	No	WL-test equivalent via injective aggregation
RGCN	Heterogeneous	Per-relation mean	Yes	Yes	Separate weights per relation type

GNN Architecture Selection Guide

- **Node classification (citation networks):** GCN or GAT with 2 layers.
- **Graph classification (molecular):** GIN with sum pooling (provably most expressive for graph-level tasks).
- **Large-scale graphs:** GraphSAGE with minibatch sampling or GraphSAINT for billion-node graphs.
- **Deep GNNs (> 4 layers):** PairNorm or skip connections (ResGCN) to prevent over-smoothing.
- **Heterogeneous graphs:** RGCN with basis decomposition for multiple relations.
- **Position-aware GNNs:** Add Laplacian PE or RWPE to GCN/GAT for structure-aware learning.
- **Inductive new-node inference:** GraphSAGE or GAT (not GCN, which is transductive).

Chapter 32

LLM-Specific Functions

Large Language Models (LLMs) employ specialized mathematical functions across tokenization, attention, normalization, loss formulation, and inference. This chapter catalogs the key mathematical operations that distinguish LLMs from smaller-scale architectures, from subword tokenization algorithms to alignment techniques like RLHF and DPO.

32.1 Tokenization Math

BPE — Byte-Pair Encoding (Sennrich et al.

BPE iteratively builds a vocabulary by merging the most frequent pair of adjacent symbols. Starting from individual characters:

1. Count all adjacent symbol pairs in the corpus.
2. Merge the most frequent pair into a new symbol.
3. Repeat until vocabulary size reaches target V.

BPE guarantees that frequent words remain as single tokens while rare words decompose into subword units. The merge operations are recorded as a deterministic rule list for encoding new text.

WordPiece (Schuster & Nakajima

WordPiece merges based on **likelihood gain**, not raw frequency:

$$\text{score}(a, b) = \frac{p(ab)}{p(a)\,p(b)}$$

where $p(\cdot)$ is the unigram probability of a token. The pair maximizing this score is merged. This favors pairs where the joint probability is much higher than expected under independence — i.e., pairs that form linguistically meaningful units. BERT, DistilBERT, and ELECTRA use WordPiece tokenization.

Unigram Language Model (Kudo

Start with a large seed vocabulary, compute unigram probabilities with EM, then iteratively **prune** tokens whose removal causes the smallest increase in the corpus loss:

$$\Delta\mathcal{L} = \mathcal{L}(\mathcal{V}) - \mathcal{L}(\mathcal{V} \setminus \{x\})$$

At each step, remove the $p\%$ of tokens with the smallest $\Delta\mathcal{L}$ (the "least useful" tokens). The final segmentation of a word is the Viterbi path that maximizes the product of unigram token probabilities.

SentencePiece (Kudo & Richardson

SentencePiece treats the input as a raw byte stream with whitespace encoded as a special meta-character (e.g., _). This makes it language-agnostic (no pre-tokenization needed) and losslessly reversible (can reconstruct the exact input text). SentencePiece supports both BPE and Unigram models and is used by LLaMA, Mistral, T5, and ALBERT.

Tokenization Comparison: BPE vs. WordPiece vs. Unigram vs. SentencePiece

- **BPE:** Frequency-based, deterministic, simple. Used in GPT-2/3/4, RoBERTa.
- **WordPiece:** Likelihood-based, better linguistic splits. Used in BERT.
- **Unigram:** Probabilistic model with pruning. Used in T5, XLNet.
- **SentencePiece:** Raw text processing, lossless. Used in LLaMA, Mistral.

Most modern LLMs use SentencePiece with BPE (GPT-4, LLaMA-2/3) or a variant thereof (Tiktoken for GPT models).

32.2 Attention Variants in LLMs

32.2.1 Multi-Query Attention (MQA)

Multi-Query Attention (Shazeer

MQA reduces memory by sharing a single Key and Value head across all query heads:

$$\text{head}_i = \text{Attention}(Q_i, K_{\text{shared}}, V_{\text{shared}})$$

where $Q_i = XW_i^Q$ (different per head), but $K_{\text{shared}} = XW^K$ and $V_{\text{shared}} = XW^V$ (same for all heads). This dramatically reduces the KV-cache size during autoregressive inference by a factor of h (number of heads), at a small cost in model quality. Used in PaLM, Gemini, and Falcon.

32.2.2 Grouped Query Attention (GQA)

Grouped Query Attention (Ainslie et al.

GQA is a middle ground between MHA and MQA: the h query heads are divided into G groups, and each group shares one K,V pair:

$$\text{head}_{g,k} = \text{Attention}(Q_{g,k}, K_g, V_g)$$

where $g = 1, \ldots, G$ indexes the group and $k = 1, \ldots, h/G$ indexes heads within each group. When $G = 1$, GQA reduces to MQA; when $G = h$, it is standard MHA. Used in LLaMA 2 (70B), LLaMA 3, and Mistral (with $G = 4$ for Mistral-7B).

MHA vs. GQA vs. MQA

MHA: Best quality, largest KV-cache (factor h). **GQA:** Near-MHA quality with G-fold KV-cache reduction. **MQA:** Smallest KV-cache, noticeable quality degradation for smaller models. Nearly all modern open LLMs use GQA.

32.2.3 FlashAttention

FlashAttention (Dao et al.

FlashAttention is an **IO-aware** exact attention algorithm. It tiles the attention computation to minimize reads/writes to GPU HBM (high-bandwidth memory):

- **Tiling:** Split Q, K, V into blocks, compute softmax incrementally via online softmax rescaling.
- **Online softmax:** $\text{softmax}(x)_i = \frac{\exp(x_i - m)}{\sum_j \exp(x_j - m)}$ where $m = \max_j x_j$ is maintained and updated per block.
- **Recomputation:** In backward pass, recompute attention matrix from stored normalization statistics, avoiding $O(n^2)$ memory.

FlashAttention achieves exact attention with $O(n)$ memory and 2–4× wall-clock speedup. FlashAttention-2 further optimizes parallelism across the sequence length dimension.

32.2.4 PagedAttention (vLLM)

PagedAttention (Kwon et al.

PagedAttention manages the KV-cache in fixed-size **pages** (blocks), analogous to virtual memory paging in operating systems:

- Each request's KV-cache is stored in non-contiguous physical blocks, mapped to a contiguous logical sequence.
- Blocks can be shared across sequences (e.g., in beam search or parallel sampling), reducing memory waste.
- Eliminates **fragmentation** — the main source of memory waste (up to 80% in traditional implementations).

PagedAttention is the core innovation behind **vLLM**, enabling up to 24× higher throughput than HuggingFace Transformers.

32.2.5 Sliding Window Attention

Sliding Window Attention (Beltagy et al.

Each token attends only to the W previous tokens (plus itself):

$$\text{Attention}(Q, K, V) = \sum_{j=\max(1, i-W)}^{i} \alpha_{ij} V_j$$

where W is the window size (e.g., 4096 for Mistral-7B, 8192 for Mistral-Large-2). The $O(nW)$ complexity enables much longer sequences. For layers beyond the first, information from tokens outside the window propagates through the stacked layers (information can travel $L \times W$ steps through L layers).

32.2.6 KV Caching

KV-Cache for Autoregressive Generation

During autoregressive decoding, the Key and Value projections of past tokens are reused across timesteps. For each layer at time t:

$$\mathbf{K}_t = \text{Concat}(\mathbf{K}_{t-1}, \mathbf{x}_t W^K)$$

$$\mathbf{V}_t = \text{Concat}(\mathbf{V}_{t-1}, \mathbf{x}_t W^V)$$

The attention for the new token uses the cached $\mathbf{K}_t, \mathbf{V}_t$. This reduces per-step computation from $O(t^2)$ to $O(t)$, but the memory for storing the KV-cache grows as $O(n_{\text{layer}} \times t \times d_{\text{model}})$, which is the primary memory bottleneck in LLM serving.

32.3 Activation Functions in LLMs

GELU — Gaussian Error Linear Unit (Hendrycks & Gimpel

Used in BERT, GPT-2, GPT-3 (earlier versions):

$$\text{GELU}(x) = x \cdot \Phi(x) = x \cdot \frac{1}{2}\left[1 + \text{erf}(x/\sqrt{2})\right]$$

where $\Phi(x)$ is the Gaussian CDF. A faster approximation often used in practice:

$$\text{GELU}(x) \approx 0.5x \left(1 + \tanh\left[\sqrt{2/\pi}(x + 0.044715x^3)\right]\right)$$

GELU smooths the ReLU by making the cutoff probabilistic based on the standard normal CDF, giving non-zero gradients for negative inputs proportional to their magnitude.

SiLU / Swish (Elfwing et al.

Used in LLaMA, PaLM, Mistral, Gemma:

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

SiLU is a smooth, non-monotonic activation. For $x \to -\infty$, it approaches 0 (like ReLU). For $x \to +\infty$, it approaches x (identity). The small "bump" for negative values (minimum at $x \approx -1.28$) can be beneficial for optimization. SiLU with a learnable β is Swish.

ReLU-squared (So et al.

Used in Gemma (with ReLU on the gate in GeGLU):

$$\text{ReLU}^2(x) = \max(0, x)^2$$

Squared ReLU has a smoother derivative than ReLU near zero and provides non-zero second derivatives, which may improve optimization in large models. Used in the GEGLU activation (gated + ReLU^2).

LLM Activation Trends

- **Early GPT (1/2/3):** GELU.
- **LLaMA / Mistral / PaLM:** SiLU (SwiGLU in FFN).
- **Gemma:** GeGLU (gated linear unit with ReLU^2 on the gate).
- **General trend:** Moving from GELU toward SiLU and from standard FFN toward gated variants (SwiGLU, GeGLU).

32.4 Normalization in LLMs

Pre-Norm vs. Post-Norm in Transformers

The placement of LayerNorm (or RMSNorm) in a Transformer block:

- **Post-Norm (original Transformer):** $\mathbf{x} \leftarrow \text{LN}(\mathbf{x} + \text{Sublayer}(\mathbf{x}))$. Normalization is applied *after* the residual addition.
- **Pre-Norm (GPT-2+, most modern LLMs):** $\mathbf{x} \leftarrow \mathbf{x} + \text{Sublayer}(\text{LN}(\mathbf{x}))$. Normalization is applied *before* each sublayer.

Pre-Norm stabilizes training (especially in early stages) and is the standard for very deep Transformers. Post-Norm may achieve slightly better final performance but is harder to train.

RMSNorm — Root Mean Square Normalization (Zhang & Sennrich

Used in LLaMA, Mistral, and many modern LLMs:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \odot \mathbf{g}$$

where $\mathbf{g} \in \mathbb{R}^d$ is a learnable gain (scale) parameter and ϵ is a small constant for numerical stability. Unlike LayerNorm, RMSNorm **does not subtract the mean** — it only scales by the RMS of the input. This is simpler, faster, and empirically as effective.

Why RMSNorm?

Removing mean subtraction saves a reduction operation per normalization layer. For a d-dimensional vector, LayerNorm computes mean and variance (2 reductions), while RMSNorm computes only one reduction (RMS). For large models with billions of parameters and thousands of layers, this accumulates to measurable wall-clock savings. Empirically, the mean-centering step of LayerNorm contributes little to training stability or final accuracy in Transformer-based LLMs.

DeepNorm (Wang et al.

For very deep Transformers (> 100 layers), DeepNorm modifies the residual connection:

$$\mathbf{x} \leftarrow \mathbf{x} + \alpha \cdot \text{Sublayer}\big(\beta \cdot \text{LN}(\mathbf{x})\big)$$

where α, β are depth-dependent constants. DeepNorm enables stable training of 1000-layer Transformers, which standard Pre-LN cannot achieve. The scaling factors are derived analytically from the expected variance of the residual stream.

32.5 Loss Functions for LLM Training

32.5.1 Next-Token Prediction (Causal LM)

Causal Language Modeling Loss

Given a sequence $\mathbf{x} = (x_1, \ldots, x_T)$, the autoregressive loss is:

$$\mathcal{L}_{\text{CLM}} = -\sum_{t=1}^{T} \log p_{\boldsymbol{\theta}}(x_t \mid x_{<t})$$

where the probability is computed via softmax over the vocabulary given the hidden state. This is the standard pretraining objective for GPT, LLaMA, Mistral, and all modern autoregressive LLMs. The model learns to predict each token from its left context using a causal attention mask.

32.5.2 Masked Language Modeling (MLM — BERT)

MLM (Devlin et al.

15% of input tokens are randomly selected for masking:

- 80% replaced with `[MASK]`.
- 10% replaced with a random token.
- 10% left unchanged.

The loss is cross-entropy over the masked positions only:

$$\mathcal{L}_{\text{MLM}} = -\sum_{t \in \text{masked}} \log p_{\boldsymbol{\theta}}(x_t \mid \tilde{\mathbf{x}})$$

where $\tilde{\mathbf{x}}$ is the corrupted input. The 10% random and 10% unchanged tokens prevent the model from always predicting `[MASK]` and encourage it to use context even with non-mask tokens.

32.5.3 Span Corruption (T5)

Span Corruption (Raffel et al.

T5's pretraining replaces random spans of consecutive tokens with a unique sentinel token, then trains the decoder to generate the dropped span text (with sentinel separators):

Input: "Thank you $\langle X \rangle$ me to your party$\langle Y \rangle$ week."

Target: "$\langle X \rangle$for inviting$\langle Y \rangle$last$\langle Z \rangle$"

Only the spans are predicted (cross-entropy on target tokens). This is a **denoising** objective — the model learns to fill in missing text. Used in T5, Flan-T5, UL2, and many encoder-decoder LLMs.

32.5.4 Prefix Language Model

Prefix LM

A hybrid between bidirectional and causal attention: tokens in the **prefix** (input/prompt) attend bidirectionally among themselves, but **target tokens** attend causally (only to past tokens and the full prefix). This enables full bidirectional context for the prompt while training for autoregressive generation. Used in GLM (General Language Model), PaLM-2 (as UL2 mixture), and as one component of the T5.1.1 mixture-of-objectives.

32.5.5 RLHF and DPO — Alignment Losses

RLHF — Reward Modeling

The Bradley-Terry model of human preferences:

$$p(\text{prefer } y_1 \text{ over } y_2 \mid x) = \frac{\exp(r_{\boldsymbol{\phi}}(x, y_1))}{\exp(r_{\boldsymbol{\phi}}(x, y_1)) + \exp(r_{\boldsymbol{\phi}}(x, y_2))}$$

where $r_{\boldsymbol{\phi}}$ is a learnable reward model (typically the SFT model with a regression head). The reward training loss is:

$$\mathcal{L}_{\text{reward}} = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma\big(r_{\boldsymbol{\phi}}(x, y_w) - r_{\boldsymbol{\phi}}(x, y_l)\big)\right]$$

where y_w is the preferred (winning) response and y_l is the dispreferred (losing) response.

RLHF — PPO with KL Penalty

After training the reward model, RL fine-tuning uses PPO with a KL penalty to prevent policy drift:

$$r(x, y) = r_{\boldsymbol{\phi}}(x, y) - \beta \cdot \log \frac{\pi_{\text{RL}}(y \mid x)}{\pi_{\text{SFT}}(y \mid x)}$$

where β controls the strength of the KL divergence penalty, keeping π_{RL} close to the supervised fine-tuned (SFT) model π_{SFT}. This prevents reward hacking and maintains fluency. The PPO objective maximizes:

$$\mathbb{E}_{x,y\sim\pi_{\text{RL}}}[r(x, y)]$$

DPO — Direct Preference Optimization (Rafailov et al.

DPO eliminates the separate reward model and PPO loop by deriving an equivalent loss directly on the policy:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma\big(\beta \log \frac{\pi_{\boldsymbol{\theta}}(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_{\boldsymbol{\theta}}(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\big)\right]$$

where π_{ref} is the frozen reference model (usually the SFT model) and β is the temperature. The insight: under the Bradley-Terry model, the optimal policy can be expressed analytically in terms of the reward, yielding a loss that trains the policy directly from preference pairs without reinforcement learning.

RLHF-PPO vs. DPO

RLHF-PPO: Requires training and maintaining a separate reward model + running online RL (sample from the policy, compute reward, update). More complex pipeline but can leverage online exploration. **DPO:** Simpler — just a classification loss on preference pairs. No reward model, no RL loop. Empirically competitive with or better than PPO on many benchmarks. Dominant alignment method in open-source (Zephyr, Tulu, Llama-chat variants).

32.6 Efficient Training and Fine-Tuning

32.6.1 LoRA — Low-Rank Adaptation

LoRA (Hu et al.

Instead of fine-tuning the full weight matrix $\mathbf{W} \in \mathbb{R}^{d \times k}$, LoRA learns a low-rank update:

$$\mathbf{W}' = \mathbf{W} + \Delta\mathbf{W} = \mathbf{W} + \mathbf{BA}$$

where $\mathbf{B} \in \mathbb{R}^{d \times r}$, $\mathbf{A} \in \mathbb{R}^{r \times k}$, and the rank $r \ll \min(d, k)$ (typically $r = 8, 16, 32, 64$). Only $\mathbf{A}$ and $\mathbf{B}$ are trained; $\mathbf{W}$ is frozen. The forward pass is:

$$\mathbf{h} = \mathbf{Wx} + \frac{\alpha}{r}\mathbf{BAx}$$

where α is a scaling factor. The scaling α/r makes the update magnitude approximately independent of r.

LoRA Usage

The dominant fine-tuning method for open LLMs (LLaMA, Mistral, Gemma). Applied to attention projection matrices (W^Q, W^K, W^V, W^O) and sometimes FFN layers. A typical LLaMA-7B LoRA with $r = 16$ adds only ∼8M trainable parameters ($< 0.1\%$ of the 7B total), stored in a few MB. LoRA adapters can be swapped at runtime without reloading the base model.

LoRA

Pros:

- Hugely parameter-efficient — single-digit millions vs. billions.
- No inference latency overhead (adapter merged into $\mathbf{W}$).
- Multiple LoRA adapters can coexist and be swapped (multi-task).
- Empirically rivals full fine-tuning on most NLP tasks.

Cons:

- Rank r is a hyperparameter (must be chosen).
- Limited expressiveness — if the required weight change is not low-rank, LoRA will underfit.
- Scaling factor α interacts with learning rate in non-obvious ways.

32.6.2 QLoRA and Other Parameter-Efficient Methods

QLoRA (Dettmers et al.

QLoRA combines LoRA with model quantization: the base model's weights $\mathbf{W}$ are stored in 4-bit NormalFloat (NF4) format and kept frozen, while LoRA adapters $(\mathbf{B}, \mathbf{A})$ are trained in BF16. NF4 is a data-dependent quantization designed for normally-distributed weights. QLoRA enables fine-tuning a 65B model on a single 48GB GPU.

Prefix Tuning and P-Tuning

- **Prefix Tuning (Li & Liang, 2021):** Prepend learnable continuous vectors to the keys and values at every transformer layer. These "virtual tokens" steer the model toward the task without modifying any weights.
- **P-Tuning v2 (Liu et al., 2021):** Apply learnable prompts at every layer (not just the embedding layer), making it competitive with full fine-tuning on smaller models.
- **Adapter layers (Houlsby et al., 2019):** Insert small bottleneck MLPs ($\mathbf{W}_{\text{down}} \in \mathbb{R}^{d \times b}$, $\mathbf{W}_{\text{up}} \in \mathbb{R}^{b \times d}$ with $b \ll d$) after each attention and FFN sublayer. Only the adapter parameters are trained.

32.7 LLM Scaling Laws

Kaplan et al. (2020) Scaling Law

The test loss $\mathcal{L}$ as a function of model parameters N (non-embedding) and training tokens D:

$$\mathcal{L}(N, D) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D}$$

with fitted exponents $\alpha_N \approx 0.076$, $\alpha_D \approx 0.095$, and constants $N_c \approx 8.8 \times 10^{13}$, $D_c \approx 5.4 \times 10^{13}$. Key implication: model size should scale slightly faster than dataset size when increasing compute.

Chinchilla / Hoffmann et al. (2022) Scaling Law

A different parameterization yields the **compute-optimal** ratio:

$$\mathcal{L}(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}$$

with $\alpha \approx 0.34$, $\beta \approx 0.28$. The compute-optimal token-to-parameter ratio is:

$$D_{\text{opt}} \approx 20 \times N$$

For a given compute budget $C \approx 6ND$, the optimal model size $N_{\text{opt}} \propto C^{0.46}$ and the optimal dataset size $D_{\text{opt}} \propto C^{0.54}$. The Chinchilla paper showed that many large models (including GPT-3) were **undertrained** — they used too many parameters relative to their data.

Kaplan vs. Chinchilla

Kaplan: Predicts that model size should grow faster than data size. Led to large undertrained models (GPT-3 175B with 300B tokens). **Chinchilla:** Predicts near-equal scaling. The 70B Chinchilla model outperformed 280B Gopher by training on 4× more data. **Modern consensus:** Chinchilla-like recipes dominate — even LLaMA (trained on 1–2T tokens for 7–65B params) follows roughly Chinchilla-optimal ratios, though many recent models are *overtrained* (more tokens than Chinchilla predicts) for inference efficiency, since smaller overtrained models are cheaper to serve.

32.8 LLM Inference Optimization

Quantization — GPTQ and AWQ

Post-training quantization compresses LLM weights to lower precision:

- **GPTQ (Frantar et al., 2023):** One-shot weight quantization using approximate second-order information (Hessian-based). Quantizes weights column by column with optimal rounding, compensating for quantization error in subsequent columns.
- **AWQ (Lin et al., 2024):** Activation-aware quantization. Observes that a small fraction (∼1%) of weight channels have disproportionately large activations. Protects these "salient" channels by scaling them up before quantization and scaling activations down.
- **INT8 / INT4:** Weight-only INT8 preserves quality with 2× memory reduction; INT4 with GPTQ/AWQ achieves 4× reduction with minimal quality degradation.

Weight Pruning — SparseGPT and Wanda

- **SparseGPT (Frantar & Alistarh, 2023):** One-shot pruning to 50–60% sparsity using second-order information. Solves a layer-wise reconstruction problem with approximate Hessian.
- **Wanda (Sun et al., 2024):** Pruning by Weights & Activations. Magnitude pruning weighted by input activation norms: $\text{score}(W_{ij}) = |W_{ij}| \cdot \|\mathbf{X}_j\|_2$, where $\mathbf{X}_j$ is the input feature vector for neuron j. Simple, fast, no second-order information needed.
- **Magnitude Pruning:** Remove smallest-magnitude weights globally or per-layer. Baseline but suboptimal for LLMs.
- **Movement Pruning (Sanh et al., 2020):** Prune based on how weights *move* during fine-tuning (top-v scores by $|W_{ij} \cdot \nabla_{W_{ij}} \mathcal{L}|$).

Quantization vs. Pruning for LLMs

Quantization is more mature (GPTQ, AWQ, GGUF are production-ready) and preserves quality better at moderate compression (4×). **Pruning** achieves unstructured sparsity (which GPUs do not natively speed up without special hardware like sparse tensor cores for 2:4 structured sparsity). For practical deployment, quantization (INT4 via AWQ/GPTQ) is the dominant approach. Unstructured pruning is primarily research-facing; structured pruning (removing heads/layers) is more practical.

32.9 Length Extrapolation — Extending Context Beyond Training

ALiBi — Attention with Linear Biases (Press et al.

Add a static, non-learned linear bias to the attention scores:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + \mathbf{m} \cdot \mathbf{B}\right) V$$

where $B_{ij} = -|i - j|$ (penalize attention to distant tokens) and m is a head-specific slope (e.g., geometrically spaced: $m = 2^{-8\frac{h}{H}}$ for head h). ALiBi enables zero-shot length extrapolation — models trained at 1024 tokens can perform well at 2048+ without fine-tuning. Used in BLOOM.

RoPE Extensions for Length Extrapolation

Rotary Position Embedding (see Chapter 21) requires modification to generalize beyond the training length:

- **NTK-aware scaling (bloc97, 2023):** Scale the RoPE base frequency $\Theta = 10000 \to \Theta' = \alpha^{d/(d-2)} \cdot \Theta$, where α is the desired length extension factor. High-frequency dimensions are rotated less, preserving local resolution.
- **Linear Interpolation (Chen et al., 2023):** Scale positions linearly: pos $\to$ pos$/\alpha$. Simple but degrades at very long lengths.
- **NTK-aware + dynamic (YaRN, Peng et al., 2023):** Combines NTK-aware scaling with temperature tuning of the softmax attention. The ramp function adjusts RoPE frequencies dynamically based on sequence length.
- **ReRoPE (Su, 2023):** Rectified RoPE — applies RoPE normally for short-range tokens but caps rotation for long-range ones, allowing theoretically infinite extrapolation.
- **Self-Extend (Jin et al., 2024):** Uses a floor operation on positions to create grouped attention — nearby tokens use exact RoPE, far tokens use binned positions.

Length Extrapolation Recommendation

Most modern LLMs (LLaMA 3, Mistral, Gemma) use RoPE with increased base frequency (e.g., 500,000 or 1,000,000 instead of 10,000) for better intrinsic length extrapolation. For existing RoPE models, YaRN is the most widely adopted extension method (used in Code LLaMA, Mistral-7B community extensions, and the LLaMA-2-long variants).

32.10 Safety and Alignment

Constitutional AI (Bai et al.

Rather than relying on human-written preferences for harmlessness, Constitutional AI (CAI) uses a set of principles (a "constitution") to:

1. **Self-critique:** The model generates responses, then critiques and revises them according to constitutional principles.
2. **AI feedback:** Uses the revised outputs as preferred completions to train a prefer-

ence model (RLAIF — RL from AI Feedback) via the same Bradley-Terry / DPO setup.

The constitution includes rules like "choose the response that is least harmful," "avoid toxic, discriminatory content," etc. CAI was used to train Claude (Anthropic).

RLAIF — Reinforcement Learning from AI Feedback

RLAIF replaces human preference labels with AI-generated preferences. For each prompt, two responses are generated; a separate (typically larger/more capable) LLM is asked to compare them and provide a preference judgment. This preference pair is used to train the reward model, bypassing the need for expensive human annotations. RLAIF has been shown to approach human-level preference accuracy at scale.

Red-Teaming

Systematic adversarial testing of LLMs to discover harmful outputs:

- **Manual red-teaming:** Security researchers probe the model with adversarial prompts (jailbreaks, prompt injections).
- **Automated red-teaming:** Use another LLM to generate diverse adversarial test cases, scaling beyond human capacity.
- **Adversarial training:** Fine-tune the model on discovered failure cases to improve robustness (similar to adversarial training in vision).

Red-teaming is now a standard part of the LLM development pipeline at OpenAI, Anthropic, Google DeepMind, and Meta.

32.11 Summary Comparison Table — LLM Architectures

LLM Architecture Trends

1. **RMSNorm over LayerNorm:** Virtually all models since LLaMA use RMSNorm (faster, equally effective).
2. **SwiGLU over standard FFN:** Gated linear units with SiLU dominate (LLaMA, Mistral, PaLM). Gemma uses GeGLU.
3. **GQA over MHA:** Grouped Query Attention is the consensus for models $\geq$7B. MQA for edge devices (Gemma).
4. **RoPE over learned PE:** RoPE is universal in modern LLMs for length generalization.
5. **Chinchilla-optimal training:** Models are trained on more tokens than earlier generations ($D \approx 20N$ or even over-trained for inference efficiency).
6. **Alignment via DPO:** DPO is replacing RLHF-PPO in open-source for simplicity and stability.
7. **Longer context:** Sliding window, YaRN, and NTK-aware scaling enable 32k–1M+ token context windows.

Table 32.1: Comparison of Major LLM Architectures

Model	Attention	Norm	Activation	PE	Vocab	Key Innovation
GPT-2/3	MHA	Pre-LN	GELU	Learned	BPE(50k)	Scaling laws; zero-shot
GPT-4	MHA	Pre-LN	GELU?	Learned	Tiktoken	Multimodal; MoE (rumored)
LLaMA 1	MHA	Pre-RMSNorm	SiLU(SwiGLU)	RoPE	BPE(32k)	Chinchilla-optimal training
LLaMA 2	GQA(70B)	Pre-RMSNorm	SiLU(SwiGLU)	RoPE	BPE(32k)	GQA; 2T tokens; RLHF
LLaMA 3	GQA	Pre-RMSNorm	SiLU(SwiGLU)	RoPE	Tiktoken(128k)	15T tokens; 128k vocab
Mistral	GQA(SWA)	Pre-RMSNorm	SiLU(SwiGLU)	RoPE	BPE(32k)	Sliding window attention
Mixtral	GQA(SWA)	Pre-RMSNorm	SiLU(SwiGLU)	RoPE	BPE(32k)	Sparse MoE; 8 experts
Gemma	MQA	Pre-RMSNorm	GeGLU	RoPE	BPE(256k)	ReLU^2; large vocab
Gemini	MQA	Pre-RMSNorm	SiLU	RoPE	BPE(256k)	Multimodal; huge context
PaLM	MQA	Pre-LN	SiLU(SwiGLU)	RoPE	BPE(256k)	Pathways; 540B params
Falcon	MQA	Pre-LN	GELU	RoPE	BPE(65k)	FlashAttention; MQA
BLOOM	MHA	Pre-LN	GELU	ALiBi	BPE(250k)	Multilingual; ALiBi
Chinchilla	MHA	Pre-LN	GELU	RoPE	BPE(32k)	Compute-optimal scaling